

エンタープライズ 金融モデル構築

Rによる金融データ分析・数理モデル・並列計算

Eiichi Horigome

Table of contents

はじめに	9
金融モデル構築からエンタープライズへ	10
第 I 部 金融工学 均衡モデル	12
第 I-1 章 R によるデータ分析の基礎	13
I-1.1 R の基礎	13
I-1.2 tidyverse によるデータ整形	22
I-1.3 関数型プログラミングと並列計算	35
I-1.4 データベースと大規模データ処理	42
I-1.6 まとめ	49
第 I-2 章財務データの整理と分析	50
I-2.1 分析環境	50
I-2.2 財務データの読み込みと整備	50
I-2.3 標本構成と売上高分布	53
I-2.4 株主資本と ROE 分析	58
I-2.5 上級デュポン・モデル	64
I-2.6 財務指標の要約と中間データ	69
I-2.7 まとめ	72
第 I-3 章株式データと時系列分析	73
I-3.1 分析環境	73
I-3.2 株式データの読み込みと整備	73
I-3.3 月次リターンと時価総額	78
I-3.4 月次リターンの分析	87
I-3.5 年次データと財務データの結合	92
I-3.6 統計分析	97
I-3.7 分析用データの保存	103
I-3.8 まとめ	105

第 I-4 章ファクターモデル分析	106
I-4.1 分析環境	106
I-4.2 入力データ	106
I-4.3 ポートフォリオ形成と市場リターン	111
I-4.4 単変量ポートフォリオ・ソート	119
I-4.5 CAPM	124
I-4.6 2×3 ポートフォリオと SMB・HML	128
I-4.7 FF3 モデルと CAPM の比較	136
I-4.8 ファクター・データの保存	142
I-4.9 まとめ	143
第 I-5 章資産価格モデルとポートフォリオ最適化	144
I-5.1 分析対象とデータの接続	144
I-5.2 FF3 モデルによる期待リターンの推定	145
I-5.3 期待リターンと配当成長率	149
I-5.4 ファクターモデルによるリスク推定	152
I-5.5 平均分散最適化	156
I-5.6 効率的フロンティア	158
I-5.7 まとめ	160
第 I-6 章イベント・スタディ	161
I-6.1 イベントスタディの基本設計	161
I-6.2 入力データの読み込みと検証	164
I-6.3 決算サプライズとイベント分類	170
I-6.4 イベント日とイベント・パネル	175
I-6.5 マーケット・モデルの推定	181
I-6.6 異常リターンと累積異常リターン	186
I-6.7 イベント・ウィンドウ分析	195
I-6.8 決算サプライズと短期 CAR	202
I-6.9 分析結果の保存	205
I-6.10 まとめ	208
第 I-7 章機械学習によるファクター選定	210
I-7.1 機械学習によるファクター分析	210
I-7.2 分析データの作成	211
I-7.3 tidymodels による学習	214
I-7.4 ファクター選定	216
I-7.5 期待リターンの推定	219
I-7.6 ポートフォリオ候補の作成	220
I-7.7 第 8 章へのデータ保存	222

第 I-8 章制約付きポートフォリオ最適化とバックテスト	225
I-8.1 ポートフォリオ最適化の基本	225
I-8.2 投資ユニバースとリターンデータ	226
I-8.3 ポートフォリオ仕様と制約条件	227
I-8.4 基本的なポートフォリオ最適化	228
I-8.5 期待ショートフォールとリスク・バジェットティング	229
I-8.6 目標リターンと目標リスク	230
I-8.7 最適化手法の比較	231
I-8.8 Efficient Frontier	232
I-8.9 最適ウェイトの比較	236
I-8.10 アウト・オブ・サンプル評価	237
I-8.11 本章のまとめ	239
第 I-9 章市場リスクと市場 VaR	240
I-9.1 FRTB とは何か	240
I-9.2 市場 VaR とは何か	241
I-9.3 市場データとポートフォリオ損益	242
I-9.4 分散共分散法による市場 VaR	243
I-9.5 ヒストリカル法による市場 VaR	245
I-9.6 モンテカルロ法による市場 VaR	245
I-9.7 3つの市場 VaR の比較	247
I-9.8 期待ショートフォール	248
I-9.9 市場 VaR のリスク寄与度	250
I-9.10 市場 VaR バックテスト	251
I-9.11 FRTB と期待ショートフォール	252
第 I-10 章信用リスクと信用 VaR	254
I-10.1 信用リスクとポートフォリオ損失	254
I-10.2 デフォルト相関	255
I-10.3 One-Factor Gaussian モデル	255
I-10.4 条件付きデフォルト確率	256
I-10.5 信用ポートフォリオの作成	257
I-10.6 共通因子と条件付き PD	258
I-10.7 モンテカルロ法による相関デフォルト	259
I-10.8 信用損失分布	261
I-10.9 信用 VaR	263
I-10.10 Expected Loss と Unexpected Loss	264
I-10.11 信用期待ショートフォール	265
I-10.12 相関がテールリスクへ与える影響	266
I-10.13 Basel の ASRF モデルとの関係	270

I-10.14 市場 VaR と信用 VaR	270
I-10.15 第 11 章への接続—CLO のトランシェ	271
第 I-11 章 CLO と信用ポートフォリオのトランシェ評価	273
I-11.1 トランチとは何か	273
I-11.2 CLO の信用トランチ	274
I-11.3 MBS では何をトランチングするのか	275
I-11.4 MBS の Sequential Pay	276
I-11.5 PAC と Support Tranche	277
I-11.6 Z-Tranche—早期に元本償還されないトランチ	277
I-11.7 CLO と MBS に共通する考え方	278
I-11.8 Attachment Point と Detachment Point	279
I-11.9 トランチ損失	279
I-11.10 第 10 章の信用損失分布をトランチへ配分する	282
I-11.11 Expected Tranche Loss	283
I-11.12 Premium Leg と Loss Leg	283
I-11.13 CLO 評価の計算構造	284
 第 II 部 金融工学 無裁定モデル	 286
第 II-1 章債券評価と金利リスクの基礎	287
II-1.1 イールドカーブの基本	288
II-1.2 イールドカーブの形状と三つの金利	290
II-1.3 債券とキャッシュフロー	296
II-1.4 債券単価と市場利回り	299
II-1.5 利回りによる債券評価	301
II-1.6 債券の相対価値評価	304
II-1.7 債券の金利感応度	305
II-1.8 数値解法による複利利回りの計算	312
 第 II-2 章金利カーブ構築と債券先物の金利感応度	 328
II-2.1 イールドカーブおよび補間	328
II-2.2 金利カーブの基礎	339
II-2.3 補間と参照日	345
II-2.4 ブートストラップ	351
II-2.5 カーブ操作と検証	356
II-2.6 日本国債先物と CTD	365
II-2.7 イールドカーブ変動と最割安銘柄影響	371
II-2.8 まとめ	377
 第 II-3 章 OIS スワップとマルチカーブ評価	 379

II-3.1 IBOR 改革と OIS の基礎	379
II-3.2 TONA カーブと短期 OIS 評価	384
II-3.3 日次複利と Fixing	389
II-3.4 SOFR とマルチカーブ評価	392
II-3.5 ケーススタディ 1：EONIA カーブと OIS 評価	398
II-3.6 ケーススタディ 2：市場レート変動と評価損益	406
II-3.7 まとめ	415
II-3.8 参考資料	416
第 II-4 章株価・為替と幾何ブラウン運動モデル	417
II-4.1 ブラック－ショールズ－マートンとブラック 76	418
II-4.2 ブラック 76 による解析評価	420
II-4.3 ブラック－ショールズ－マートンとヨーロピアン・オプション	422
II-4.4 モンテカルロ評価	425
II-4.5 アメリカン put の早期行使価値	431
II-4.6 準モンテカルロとロングスタッフ－シュワルツ法	433
II-4.7 まとめ	437
第 II-5 章信用リスクとクレジットデリバティブ	439
II-5.1 信用リスクとクレジットデリバティブの基礎	440
II-5.2 フラットカーブによるクレジットデリバティブ評価	441
II-5.3 Premium Leg と Protection Leg	450
II-5.4 Hazard Rate 推定と感応度	453
II-5.5 標準 CDS と ISDA カーブ	456
II-5.6 ISDA Engine と評価比較	462
II-5.7 まとめ	469
第 II-6 章金利モデルのキャリブレーション	471
II-6.1 市場価格からモデルパラメータへ	472
II-6.2 Black 型と Normal 型のボラティリティ	473
II-6.3 SABR によるスマイルキャリブレーション	474
II-6.4 Hull－White のパラメータ推定	479
II-6.5 BGM モデルと Brigo－Mercurio への接続	483
II-6.6 キャリブレーションサマリー	485
第 II-7 章ショートレートモデルとフォワード金利モデル	487
II-7.1 短期金利とフォワード金利	487
II-7.2 短期金利：Vasicek から Hull－White へ	488
II-7.3 HJM：瞬間フォワードモデル	492
II-7.4 BGM/LMM：期間別フォワード	493
II-7.5 後決め金利と FMM モデル	496

II-7.6 各種モデルサマリー	499
第 II-8 章金利モデルによる早期償還評価	500
II-8.1 発行体条件と債券評価	500
II-8.2 満期債と発行体コール	502
II-8.3 発行体の行使判断とバックワード・インダクション	503
II-8.4 Hull-White モデル	503
II-8.5 金利変動とコールオプション価値	511
II-8.6 数値計算検証	512
II-8.7 Bermudan Swaption への拡張	513
II-8.8 MBS のプリペイメント	514
第 II-9 章金利為替系デリバティブ	522
II-9.1 金利為替系デリバティブの商品構造と市場条件	523
II-9.2 FX プロセスと Coupon ペイオフ	529
II-9.3 モンテカルロによる金利為替系デリバティブ評価	533
II-9.4 Coupon の期間構造と分布	541
II-9.5 FX 感応度とモンテカルロ誤差	545
II-9.6 モデルの限界と Part II 総括	550
II-9.7 まとめ	552
エンタープライゼーション	555
第 III-1 章金融計算技術の発展	556
III-1.1 金融における情報工学の歩み	556
III-1.2 リーマンショックを経たテクノロジーの潮流	558
III-1.3 金融計算におけるハードウェアの限界	559
III-1.4 並列コンピューティング	560
III-1.5 R による並列計算	564
III-1.6 並列化の効果と限界	565
III-1.7 自動微分	566
第 III-2 章分散コンピューティング	567
III-2.1 計算処理の分割	568
III-2.2 計算資源への分配	568
III-2.3 計算処理の制御	570
III-2.4 分散コンピューティング	571
III-2.5 計算資源の統合	573
第 III-3 章金融計算アーキテクチャ	575
III-3.1 金融取引とマーケットデータ	576

III-3.2 プライシング・エンジン	577
III-3.3 リスク・シミュレーション・シナリオ	577
III-3.4 計算エンジン	578
III-3.5 リスクアグリゲーション・レポート	580
III-3.6 エンタープライズ金融計算	581
第 III-4 章自動微分計算	583
III-4.1 金融計算と微分	583
III-4.2 有限差分法	584
III-4.3 自動微分の原理と実装	585
III-4.4 深層学習ライブラリによる市場リスク感応度	590
第 III-5 章自動微分応用：オプション	596
III-5.1 オプション評価とリスク感応度	596
III-5.2 ブラック・ショールズ式による解析解	597
III-5.3 モンテカルロ法における自動微分のリスク感応度	598
III-5.4 微分不可なオプションペイオフのケーススタディ	603
第 III-6 章自動微分応用：金利リスク	612
III-6.1 金利スワップ評価とリスク感応度	613
III-6.2 金利リスクにおける自動微分	615
III-6.3 イールドカーブと自動微分	621
III-6.4 バケット DV01 と GPS	624
第 III-7 章自動微分応用：多因子モデル	631
III-7.1 金利為替系デリバティブと多因子市場モデル	631
III-7.2 多因子モンテカルロ法	634
III-7.3 多因子リスク感応度と自動微分	640
III-7.4 非線形ペイオフと自動微分	648
III-7.5 エンタープライズ・リスク計算	649

はじめに

“All models are wrong, but some are useful.”

「すべてのモデルは正しくない。しかし、その中には有用なものもある。」

— *George E. P. Box*

金融モデル構築からエンタープライズへ

2008 年のリーマン・ショックを契機として、バーゼル規制をはじめとする金融規制の枠組みは再整備され、金融機関には、より精緻な価格評価やリスク計測が求められるようになった。その過程で、金融工学の理論や数値計算手法は実務の中に広く定着し、現在では一定の成熟段階に達している。

一方、それらを支えるソフトウェア技術と計算環境は、この 20 年で劇的に進歩した。クラウドコンピューティング、オープンソースソフトウェア、並列処理および分散処理技術の発展により、かつて金融機関や研究機関のみが利用できたスーパーコンピューターやグリッドコンピューティング環境は、クラウドサービスや PC クラスタを利用することで、個人や小規模組織でも比較的 low コストに構築できるようになった。また、モンテカルロシミュレーションやリスク計測など、大量の計算資源を必要とする金融工学計算も、現実的な時間で実行できる時代となっている。

こうした時代の変化を背景として、

金融工学を学ぶ時代から、自ら金融工学システムを構築する時代へ。

という新しい潮流が生まれている。

なお、本書は三部構成となっている。

第 I 部では、均衡モデルを中心とした実証ファイナンスを扱い、財務データや市場データを用いた分析手法を学ぶ。第 II 部では、無裁定価格理論を中心として、債券、金利期間構造、デリバティブ、信用リスクなどの金融工学モデルを実装する。さらに第 III 部では、第 II 部で実装した無裁定価格モデルを基盤として、並列処理および分散処理環境へ展開し、大規模な金融工学計算を実装する方法を扱う。

また、その実装基盤として R および tidyverse を採用する。R は統計解析言語として広く利用されているが、本書では、優れたデータハンドリング機能、ベクトル演算、関数型プログラミングの特徴を備えた実装基盤として位置付けている。これらの特徴を活かすことで、金融デー

タの取得・加工から価格評価、リスク計測、さらには並列・分散コンピューティング環境を利用した大規模金融工学計算までを、一貫した設計思想のもとで実装することができる。

本書の目的は、金融理論を理解することだけではなく、それらをソフトウェアとして実装し、現代の金融工学システムを構築するための基礎技術を体系的に習得することにある。

第Ⅰ部 金融工学 均衡モデル

第 I-1 章 R によるデータ分析の基礎

本章では、R によるデータ分析に必要な基本操作を、Base R から tidyverse へ段階的に学ぶ。前半では、オブジェクト、ベクトル、行列、リスト、データフレーム、関数、条件判定、記述統計および確率分布を扱う。後半では、dplyr、stringr、tidyr、purrr、ggplot2 を用いて、データの読み込み、整形、集計、反復処理、可視化を一つの流れとして組み立てる。さらに、nest() によるリスト列、future と furrr による並列計算、dbplyr と Arrow によるデータベース・大規模データ処理へ進む。

Base R の項目構成は、Posit が公開している [Base R Cheatsheet](#) を参考にしている。本章のコードでは、反復処理に明示的なループを使わず、ベクトル化または purrr::map() 系関数を用いる。また、パイプ演算子はすべて |> に統一する。

I-1.1 R の基礎

R では、計算結果やデータをオブジェクトとして保存する。代入には <- を用いる。

```
risk_free_rate <- 0.01
future_cash_flow <- 100

present_value <- future_cash_flow / (1 + risk_free_rate)

present_value
#> [1] 99.0099
```

オブジェクト名には、内容が分かる英単語を用いる。本書では複数の単語をアンダースコアで結ぶ snake_case を基本とする。

```
2 + 3
#> [1] 5

10 - 4
#> [1] 6

6 * 7
#> [1] 42
```



```
100 / 4
#> [1] 25

2^5
#> [1] 32

sqrt(16)
#> [1] 4

log(exp(1))
#> [1] 1

abs(-3)
#> [1] 3

round(pi, digits = 3)
#> [1] 3.142

signif(pi, digits = 4)
#> [1] 3.142
```

```
numeric_value <- 10.5
integer_value <- 10L
character_value <- "Finance"
logical_value <- TRUE

class(numeric_value)
#> [1] "numeric"

class(integer_value)
#> [1] "integer"

class(character_value)
#> [1] "character"

class(logical_value)
#> [1] "logical"

typeof(numeric_value)
#> [1] "double"

str(numeric_value)
#>  num 10.5
```

欠損値は NA で表す。欠損値の有無は `is.na()` で確認する。

```
returns_with_na <- c(0.02, NA, -0.01, 0.03)

is.na(returns_with_na)
#> [1] FALSE  TRUE FALSE FALSE

sum(is.na(returns_with_na))
#> [1] 1
```

```
mean(returns_with_na, na.rm = TRUE)
#> [1] 0.01333333
```

```
interest_rates <- c(0.01, 0.015, 0.02)
year_sequence <- 1:5
fine_grid <- seq(from = 0, to = 1, by = 0.25)
repeated_values <- rep(100, times = 5)

interest_rates
#> [1] 0.010 0.015 0.020
year_sequence
#> [1] 1 2 3 4 5
fine_grid
#> [1] 0.00 0.25 0.50 0.75 1.00
repeated_values
#> [1] 100 100 100 100 100
```

R の添字は 1 から始まる。

```
interest_rates[1]
#> [1] 0.01
interest_rates[2:3]
#> [1] 0.015 0.020
interest_rates[-1]
#> [1] 0.015 0.020

interest_rates[interest_rates >= 0.015]
#> [1] 0.015 0.020
```

```
portfolio_weights <- c(stock = 0.6, bond = 0.3, cash = 0.1)

portfolio_weights
#> stock bond cash
#> 0.6 0.3 0.1
portfolio_weights["bond"]
#> bond
#> 0.3
sum(portfolio_weights)
#> [1] 1
```

R では、多くの計算をベクトル全体に一度に適用できる。

```
discount_rates <- seq(0.01, 0.05, by = 0.01)
cash_flow <- 100
```

```
present_values <- cash_flow / (1 + discount_rates)

tibble(discount_rates, present_values)
#> # A tibble: 5 x 2
#>   discount_rates present_values
#>       <dbl>         <dbl>
#> 1         0.01         99.010
#> 2         0.02         98.039
#> 3         0.03         97.087
#> 4         0.04         96.154
#> 5         0.05         95.238
```

```
return_matrix <- matrix(
  c(0.01, 0.02, -0.01, 0.03, 0.00, 0.01),
  nrow = 3,
  ncol = 2,
  byrow = TRUE
)

colnames(return_matrix) <- c("stock_a", "stock_b")
rownames(return_matrix) <- c("month_1", "month_2", "month_3")

return_matrix
#>      stock_a stock_b
#> month_1    0.01    0.02
#> month_2   -0.01    0.03
#> month_3    0.00    0.01
return_matrix[, "stock_a"]
#> month_1 month_2 month_3
#>    0.01  -0.01    0.00
colMeans(return_matrix)
#> stock_a stock_b
#>    0.00    0.02
```

リストには、型や長さの異なるオブジェクトをまとめて保存できる。

```
analysis_result <- list(
  model_name = "simple return model",
  coefficients = c(alpha = 0.001, beta = 1.05),
  observations = 60L,
  converged = TRUE
)
```

```
analysis_result$model_name
#> [1] "simple return model"
analysis_result$coefficients
#> alpha  beta
#> 0.001 1.050
analysis_result[["observations"]]
#> [1] 60
```

```
firm_data_base <- data.frame(
  firm_id = 1:3,
  firm_name = c("Firm A", "Firm B", "Firm C"),
  roe = c(0.08, 0.12, 0.15)
)

firm_data_base
#>   firm_id firm_name  roe
#> 1       1   Firm A 0.08
#> 2       2   Firm B 0.12
#> 3       3   Firm C 0.15
str(firm_data_base)
#> 'data.frame':   3 obs. of  3 variables:
#>  $ firm_id   : int  1 2 3
#>  $ firm_name: chr   "Firm A" "Firm B" "Firm C"
#>  $ roe       : num  0.08 0.12 0.15
```

tidyverse では、データフレームを拡張した `tibble` を用いる。

```
firm_data <- tibble(
  firm_id = 1:3,
  firm_name = c("Firm A", "Firm B", "Firm C"),
  roe = c(0.08, 0.12, 0.15)
)

firm_data
#> # A tibble: 3 x 3
#>   firm_id firm_name  roe
#>   <int> <chr>      <dbl>
#> 1     1 Firm A      0.08
#> 2     2 Firm B      0.12
#> 3     3 Firm C      0.15
```

少量のデータを行単位で記述する場合は `tribble()` が便利である。

```
project_data <- tribble(
  ~project, ~initial_cost, ~cash_flow,
  "A",      100,      120,
  "B",      150,      175,
  "C",      200,      215
)
```

```
project_data
#> # A tibble: 3 x 3
#>   project initial_cost cash_flow
#>   <chr>      <dbl>      <dbl>
#> 1 A          100        120
#> 2 B          150        175
#> 3 C          200        215
```

```
npv <- 5.2
```

```
investment_decision <- if (npv >= 0) {
  "投資を実行する"
} else {
  "投資を見送る"
}
```

```
investment_decision
#> [1] "投資を実行する"
```

ベクトルに対する複数の条件分岐には `case_when()` を用いる。

```
credit_scores <- tibble(score = c(85, 72, 58, 91))
```

```
credit_scores |>
  mutate(
    rating = case_when(
      score >= 80 ~ "A",
      score >= 65 ~ "B",
      TRUE ~ "C"
    )
  )
```

```
#> # A tibble: 4 x 2
#>   score rating
#>   <dbl> <chr>
#> 1    85 A
#> 2    72 B
#> 3    58 C
```

```
#> 4      91 A
```

```
calculate_present_value <- function(cash_flow, discount_rate, years = 1) {  
  cash_flow / (1 + discount_rate)^years  
}  
  
calculate_present_value(cash_flow = 100, discount_rate = 0.02, years = 3)  
#> [1] 94.23223
```

関数はベクトルをそのまま受け取れるように設計すると再利用しやすい。

```
calculate_present_value(  
  cash_flow = c(100, 100, 100),  
  discount_rate = 0.02,  
  years = 1:3  
)  
#> [1] 98.03922 96.11688 94.23223
```

```
sample_returns <- c(0.01, 0.03, -0.02, 0.00, 0.04, 0.02)  
  
mean(sample_returns)  
#> [1] 0.01333333  
median(sample_returns)  
#> [1] 0.015  
min(sample_returns)  
#> [1] -0.02  
max(sample_returns)  
#> [1] 0.04  
range(sample_returns)  
#> [1] -0.02 0.04  
var(sample_returns)  
#> [1] 0.0004666667  
sd(sample_returns)  
#> [1] 0.02160247  
quantile(sample_returns)  
#>      0%      25%      50%      75%     100%  
#> -0.0200 0.0025 0.0150 0.0275 0.0400
```

```
stock_a <- c(0.01, 0.03, -0.02, 0.02, 0.04)  
stock_b <- c(0.00, 0.02, -0.01, 0.01, 0.03)  
  
cov(stock_a, stock_b)  
#> [1] 0.00035  
cor(stock_a, stock_b)
```

```
#> [1] 0.9615239
```

```
firm_returns <- c(0.08, 0.12, -0.03, 0.05)
```

```
rank(firm_returns)
```

```
#> [1] 3 4 1 2
```

```
rank(-firm_returns)
```

```
#> [1] 2 1 4 3
```

```
order(firm_returns, decreasing = TRUE)
```

```
#> [1] 2 1 4 3
```

R の確率分布関数には共通した命名規則がある。

接頭辞	内容
r	乱数を生成する
d	確率密度または確率質量を求める
p	累積確率を求める
q	累積確率に対応する分位点を求める

```
set_random_seed_LFL(1234)
```

```
uniform_random <- runif(n = 10, min = 0, max = 1)
```

```
uniform_random
```

```
#> [1] 0.113703411 0.622299405 0.609274733 0.623379442 0.860915384 0.640310605
```

```
#> [7] 0.009495756 0.232550506 0.666083758 0.514251141
```

```
dunif(0.5, min = 0, max = 1)
```

```
#> [1] 1
```

```
punif(0.5, min = 0, max = 1)
```

```
#> [1] 0.5
```

```
qunif(0.95, min = 0, max = 1)
```

```
#> [1] 0.95
```

```
set_random_seed_LFL(1234)
```

```
normal_random <- rnorm(n = 10, mean = 0, sd = 1)
```

```
normal_random
```

```
#> [1] -1.2070657 0.2774292 1.0844412 -2.3456977 0.4291247 0.5060559
```

```
#> [7] -0.5747400 -0.5466319 -0.5644520 -0.8900378
```

```
dnorm(0, mean = 0, sd = 1)
```

```
#> [1] 0.3989423
```



```
pnorm(1.96, mean = 0, sd = 1)
#> [1] 0.9750021
qnorm(0.975, mean = 0, sd = 1)
#> [1] 1.959964
```

`dnorm()` は正規分布の密度、`pnorm()` は指定した値以下となる確率、`qnorm()` は指定した累積確率に対応する分位点を返す。

```
probability_below <- pnorm(-0.02, mean = 0.01, sd = 0.03)
value_at_risk_95 <- qnorm(0.05, mean = 0.01, sd = 0.03)

probability_below
#> [1] 0.1586553
value_at_risk_95
#> [1] -0.03934561
```

```
set_random_seed_LFL(1234)

binomial_random <- rbinom(n = 10, size = 20, prob = 0.6)

binomial_random
#> [1] 15 11 11 11 10 11 17 14 11 12
dbinom(12, size = 20, prob = 0.6)
#> [1] 0.1797058
pbinom(12, size = 20, prob = 0.6)
#> [1] 0.5841071
qbinom(0.95, size = 20, prob = 0.6)
#> [1] 16
```

```
set_random_seed_LFL(1234)

poisson_random <- rpois(n = 10, lambda = 3)

poisson_random
#> [1] 1 3 3 3 5 3 0 2 4 3
dpois(2, lambda = 3)
#> [1] 0.2240418
ppois(2, lambda = 3)
#> [1] 0.4231901
qpois(0.95, lambda = 3)
#> [1] 6
```

```
distribution_functions <- tribble(
  ~distribution, ~random_function, ~density_function, ~cumulative_function,
  ↪ ~quantile_function,
  "一様分布",    "runif()",      "dunif()",      "punif()",
  ↪ "qunif()",
  "正規分布",    "rnorm()",      "dnorm()",      "pnorm()",
  ↪ "qnorm()",
  "二項分布",    "rbinom()",     "dbinom()",     "pbinom()",
  ↪ "qbinom()",
  "ポアソン分布", "rpois()",      "dpois()",      "ppois()",
  ↪ "qpois()"
)
```

```
distribution_functions
#> # A tibble: 4 x 5
#>   distribution random_function density_function cumulative_function
#>   <chr>         <chr>         <chr>         <chr>
#> 1 一様分布      runif()         dunif()         punif()
#> 2 正規分布      rnorm()         dnorm()         pnorm()
#> 3 二項分布      rbinom()        dbinom()        pbinom()
#> 4 ポアソン分布 rpois()         dpois()         ppois()
#> # i 1 more variable: quantile_function <chr>
```

I-1.2 tidyverse によるデータ整形

本書で使用するデータは、プロジェクトルートの `data` ディレクトリに置き、外部入力に近いデータを `raw`、分析によって作成したデータを `derived` に分ける。プロジェクトルートの探索やファイル検索は `R/utility_LFL.R` へ集約し、各章では分析対象となる相対パスだけを指定する。

```
tbl_daily_stock_return <-
  read_parquet_file_LFL(
    file.path("raw", "market", "daily_stock_returns.parquet")
  )

vec_required_columns <- c("date", "firm1", "firm2")
vec_missing_columns <- setdiff(vec_required_columns, names(tbl_daily_stock_return))

if (length(vec_missing_columns) > 0) {
  stop("必要な列が不足しています: ", paste(vec_missing_columns, collapse = ", "))
}
```

```
show_table_LFL(tbl_daily_stock_return)
#> # A tibble: 21 x 3
#>   date          firm1      firm2
#>   * <date>         <dbl>    <dbl>
#> 1 2020-04-01 -0.039482  0.076963
#> 2 2020-04-02  0.0059787 -0.0072488
#> 3 2020-04-03  0.055787  -0.017304
#> 4 2020-04-06  0.041935  0.0021690
#> 5 2020-04-07 -0.020193  0.075545
#> 6 2020-04-08 -0.0022947 -0.096212
#> 7 2020-04-09  0.081707  0.078843
#> 8 2020-04-10  0.034561  0.021536
#> 9 2020-04-13  0.032601  0.13520
#> 10 2020-04-14 -0.037663  0.037979
#> # i 11 more rows
```

```
show_data_structure_LFL(tbl_daily_stock_return)
#> Rows: 21
#> Columns: 3
#> $ date <date> 2020-04-01, 2020-04-02, 2020-04-03, 2020-04-06, 2020-04-07, 202~
#> $ firm1 <dbl> -0.039482142, 0.005978689, 0.055786605, 0.041934669, -0.02019279~
#> $ firm2 <dbl> 0.0769625974, -0.0072488484, -0.0173040800, 0.0021689573, 0.0755~
summary(tbl_daily_stock_return)
#>      date          firm1          firm2
#> Min.   :2020-04-01   Min.   : -0.072071   Min.   : -0.0962117
#> 1st Qu.:2020-04-08   1st Qu.: -0.027029   1st Qu.: -0.0196147
#> Median :2020-04-15   Median :  0.005979   Median : -0.0003056
#> Mean    :2020-04-15   Mean    :  0.014161   Mean    :  0.0080253
#> 3rd Qu.:2020-04-22   3rd Qu.:  0.044812   3rd Qu.:  0.0379793
#> Max.    :2020-04-30   Max.    :  0.171297   Max.    :  0.1352008
nrow(tbl_daily_stock_return)
#> [1] 21
ncol(tbl_daily_stock_return)
#> [1] 3
names(tbl_daily_stock_return)
#> [1] "date" "firm1" "firm2"
```

```
tbl_daily_stock_return |>
  select(date, firm1)
#> # A tibble: 21 x 2
#>   date          firm1
#>   <date>         <dbl>
#> 1 2020-04-01 -0.039482
#> 2 2020-04-02  0.0059787
```

```
#> 3 2020-04-03 0.055787
#> 4 2020-04-06 0.041935
#> 5 2020-04-07 -0.020193
#> 6 2020-04-08 -0.0022947
#> 7 2020-04-09 0.081707
#> 8 2020-04-10 0.034561
#> 9 2020-04-13 0.032601
#> 10 2020-04-14 -0.037663
#> # i 11 more rows
```

```
tbl_daily_stock_return |>
  filter(firm1 > 0)
#> # A tibble: 12 x 3
#>   date          firm1      firm2
#>   <date>         <dbl>     <dbl>
#> 1 2020-04-02 0.0059787 -0.0072488
#> 2 2020-04-03 0.055787  -0.017304
#> 3 2020-04-06 0.041935   0.0021690
#> 4 2020-04-09 0.081707   0.078843
#> 5 2020-04-10 0.034561   0.021536
#> 6 2020-04-13 0.032601   0.13520
#> 7 2020-04-15 0.0098798  0.011427
#> 8 2020-04-17 0.17130    0.039463
#> 9 2020-04-20 0.046325   0.037274
#> 10 2020-04-22 0.051543  -0.053818
#> 11 2020-04-27 0.0045375 -0.019615
#> 12 2020-04-30 0.044812  -0.013410
```

複数の条件を同時に指定できる。

```
tbl_daily_stock_return |>
  filter(firm1 > 0, firm2 > 0)
#> # A tibble: 7 x 3
#>   date          firm1      firm2
#>   <date>         <dbl>     <dbl>
#> 1 2020-04-06 0.041935   0.0021690
#> 2 2020-04-09 0.081707   0.078843
#> 3 2020-04-10 0.034561   0.021536
#> 4 2020-04-13 0.032601   0.13520
#> 5 2020-04-15 0.0098798  0.011427
#> 6 2020-04-17 0.17130    0.039463
#> 7 2020-04-20 0.046325   0.037274
```

```
return_data <- tbl_daily_stock_return |>
```

```
  mutate(
    spread = firm1 - firm2,
    firm1_gross_return = 1 + firm1,
    firm2_gross_return = 1 + firm2,
    market_direction = case_when(
      firm1 > 0 & firm2 > 0 ~ "both_positive",
      firm1 < 0 & firm2 < 0 ~ "both_negative",
      TRUE ~ "mixed"
    )
  )
```

```
return_data
```

```
#> # A tibble: 21 x 7
```

```
#>   date          firm1      firm2      spread firm1_gross_return
#>   <date>         <dbl>      <dbl>      <dbl>          <dbl>
#> 1 2020-04-01 -0.039482  0.076963 -0.11644          0.96052
#> 2 2020-04-02  0.0059787 -0.0072488  0.013228          1.0060
#> 3 2020-04-03  0.055787  -0.017304  0.073091          1.0558
#> 4 2020-04-06  0.041935  0.0021690  0.039766          1.0419
#> 5 2020-04-07 -0.020193  0.075545 -0.095738          0.97981
#> 6 2020-04-08 -0.0022947 -0.096212  0.093917          0.99771
#> 7 2020-04-09  0.081707  0.078843  0.0028641          1.0817
#> 8 2020-04-10  0.034561  0.021536  0.013025          1.0346
#> 9 2020-04-13  0.032601  0.13520  -0.10260          1.0326
#> 10 2020-04-14 -0.037663  0.037979 -0.075642          0.96234
```

```
#> # i 11 more rows
```

```
#> # i 2 more variables: firm2_gross_return <dbl>, market_direction <chr>
```

```
return_data |>
```

```
  arrange(firm1)
```

```
#> # A tibble: 21 x 7
```

```
#>   date          firm1      firm2      spread firm1_gross_return
#>   <date>         <dbl>      <dbl>      <dbl>          <dbl>
#> 1 2020-04-28 -0.072071  -0.046873  -0.025197          0.92793
#> 2 2020-04-24 -0.041378  -0.0041105  -0.037268          0.95862
#> 3 2020-04-01 -0.039482  0.076963  -0.11644          0.96052
#> 4 2020-04-14 -0.037663  0.037979  -0.075642          0.96234
#> 5 2020-04-23 -0.028875  -0.035484  0.0066092          0.97112
#> 6 2020-04-16 -0.027029  -0.053488  0.026460          0.97297
#> 7 2020-04-07 -0.020193  0.075545  -0.095738          0.97981
#> 8 2020-04-21 -0.014593  -0.00030564 -0.014288          0.98541
#> 9 2020-04-08 -0.0022947 -0.096212  0.093917          0.99771
#> 10 2020-04-27  0.0045375 -0.019615  0.024152          1.0045
```

```
#> # i 11 more rows
```

```
#> # i 2 more variables: firm2_gross_return <dbl>, market_direction <chr>

return_data |>
  arrange(desc(firm1))
#> # A tibble: 21 x 7
#>   date          firm1      firm2      spread firm1_gross_return
#>   <date>         <dbl>      <dbl>      <dbl>         <dbl>
#> 1 2020-04-17 0.17130    0.039463 0.13183         1.1713
#> 2 2020-04-09 0.081707    0.078843 0.0028641        1.0817
#> 3 2020-04-03 0.055787   -0.017304 0.073091        1.0558
#> 4 2020-04-22 0.051543   -0.053818 0.10536         1.0515
#> 5 2020-04-20 0.046325    0.037274 0.0090516        1.0463
#> 6 2020-04-30 0.044812   -0.013410 0.058222        1.0448
#> 7 2020-04-06 0.041935    0.0021690 0.039766        1.0419
#> 8 2020-04-10 0.034561    0.021536 0.013025        1.0346
#> 9 2020-04-13 0.032601    0.13520   -0.10260        1.0326
#> 10 2020-04-15 0.0098798 0.011427  -0.0015473       1.0099
#> # i 11 more rows
#> # i 2 more variables: firm2_gross_return <dbl>, market_direction <chr>
```

```
return_data |>
  summarise(
    observations = n(),
    mean_firm1 = mean(firm1),
    sd_firm1 = sd(firm1),
    mean_firm2 = mean(firm2),
    sd_firm2 = sd(firm2),
    correlation = cor(firm1, firm2)
  )
#> # A tibble: 1 x 6
#>   observations mean_firm1 sd_firm1 mean_firm2 sd_firm2 correlation
#>   <int>         <dbl>    <dbl>      <dbl>    <dbl>         <dbl>
#> 1      21      0.014161 0.053840 0.0080253 0.054390      0.23349
```

```
return_data |>
  group_by(market_direction) |>
  summarise(
    observations = n(),
    mean_firm1 = mean(firm1),
    mean_firm2 = mean(firm2),
    .groups = "drop"
  ) |>
  arrange(desc(observations))
#> # A tibble: 3 x 4
```

```
#>   market_direction observations mean_firm1 mean_firm2
#>   <chr>                <int>      <dbl>      <dbl>
#> 1 mixed                  8  0.0081649  0.0098864
#> 2 both_positive          7  0.059758   0.046559
#> 3 both_negative          6 -0.031040 -0.039412
```

```
return_data |>
  count(market_direction, sort = TRUE)
#> # A tibble: 3 x 2
#>   market_direction     n
#>   <chr>             <int>
#> 1 mixed              8
#> 2 both_positive      7
#> 3 both_negative      6
```

```
firm_master <- tribble(
  ~firm, ~firm_name, ~industry,
  "firm1", "Firm A", "Machinery",
  "firm2", "Firm B", "Chemicals"
)
```

```
return_summary <- return_data |>
  summarise(
    firm1 = mean(firm1),
    firm2 = mean(firm2)
  ) |>
  pivot_longer(
    cols = everything(),
    names_to = "firm",
    values_to = "mean_return"
  )
```

```
return_summary |>
  left_join(firm_master, by = "firm")
#> # A tibble: 2 x 4
#>   firm mean_return firm_name industry
#>   <chr>      <dbl> <chr>      <chr>
#> 1 firm1  0.014161 Firm A      Machinery
#> 2 firm2  0.0080253 Firm B      Chemicals
```

文字列処理では、関数名が `str_` で始まる `stringr` を使用する。

```
company_names <- tibble(
  raw_name = c(
```

```

    " Alpha Holdings Co., Ltd. ",
    "Beta Financial Group",
    "Gamma Industries"
  )
)

company_names <- company_names |>
  mutate(
    clean_name = str_squish(raw_name),
    lower_name = str_to_lower(clean_name),
    name_length = str_length(clean_name),
    contains_group = str_detect(clean_name, regex("group", ignore_case = TRUE)),
    first_word = str_extract(clean_name, "^[A-Za-z]+")
  )

company_names
#> # A tibble: 3 x 6
#>   raw_name          clean_name lower_name name_length contains_group first_word
#>   <chr>          <chr>      <chr>      <int> <lgl>          <chr>
#> 1 " Alpha Holdings~ Alpha Hol~ alpha hol~      24 FALSE        Alpha
#> 2 "Beta Financial G~ Beta Fina~ beta fina~      20 TRUE         Beta
#> 3 "Gamma Industries" Gamma Ind~ gamma ind~      16 FALSE        Gamma

```

```

company_names |>
  mutate(
    short_name = str_remove(clean_name, regex(" co\\.\\. , ltd\\.\\. $", ignore_case =
      TRUE)),
    display_name = str_c(first_word, " / ", name_length, " characters")
  ) |>
  select(clean_name, short_name, display_name)
#> # A tibble: 3 x 3
#>   clean_name          short_name          display_name
#>   <chr>          <chr>          <chr>
#> 1 Alpha Holdings Co., Ltd. Alpha Holdings Alpha / 24 characters
#> 2 Beta Financial Group Beta Financial Group Beta / 20 characters
#> 3 Gamma Industries Gamma Industries Gamma / 16 characters

```

元のデータでは、`firm1` と `firm2` が別々の列に格納されている。`pivot_longer()` を使うと、企業名とリターンをそれぞれ一つの列へまとめられる。

```

return_long <- tbl_daily_stock_return |>
  pivot_longer(
    cols = c(firm1, firm2),
    names_to = "firm",

```



```
  values_to = "return"  
)
```

```
return_long
```

```
#> # A tibble: 42 x 3  
#>   date      firm      return  
#>   <date>   <chr>    <dbl>  
#> 1 2020-04-01 firm1 -0.039482  
#> 2 2020-04-01 firm2  0.076963  
#> 3 2020-04-02 firm1  0.0059787  
#> 4 2020-04-02 firm2 -0.0072488  
#> 5 2020-04-03 firm1  0.055787  
#> 6 2020-04-03 firm2 -0.017304  
#> 7 2020-04-06 firm1  0.041935  
#> 8 2020-04-06 firm2  0.0021690  
#> 9 2020-04-07 firm1 -0.020193  
#> 10 2020-04-07 firm2  0.075545  
#> # i 32 more rows
```

```
return_long |>  
  pivot_wider(  
    names_from = firm,  
    values_from = return  
  )  
#> # A tibble: 21 x 3  
#>   date      firm1      firm2  
#>   <date>    <dbl>    <dbl>  
#> 1 2020-04-01 -0.039482  0.076963  
#> 2 2020-04-02  0.0059787 -0.0072488  
#> 3 2020-04-03  0.055787  -0.017304  
#> 4 2020-04-06  0.041935  0.0021690  
#> 5 2020-04-07 -0.020193  0.075545  
#> 6 2020-04-08 -0.0022947 -0.096212  
#> 7 2020-04-09  0.081707  0.078843  
#> 8 2020-04-10  0.034561  0.021536  
#> 9 2020-04-13  0.032601  0.13520  
#> 10 2020-04-14 -0.037663  0.037979  
#> # i 11 more rows
```

```
security_codes <- tibble(  
  security_id = c("JP-1001", "JP-1002", "US-2001")  
)
```

```
security_codes |>
```

```
separate_wider_delim(
  security_id,
  delim = "-",
  names = c("country", "code")
)
#> # A tibble: 3 x 2
#>   country code
#>   <chr>   <chr>
#> 1 JP     1001
#> 2 JP     1002
#> 3 US     2001
```

```
missing_data <- tibble(
  firm = c("A", "B", "C"),
  return = c(0.02, NA, -0.01)
)

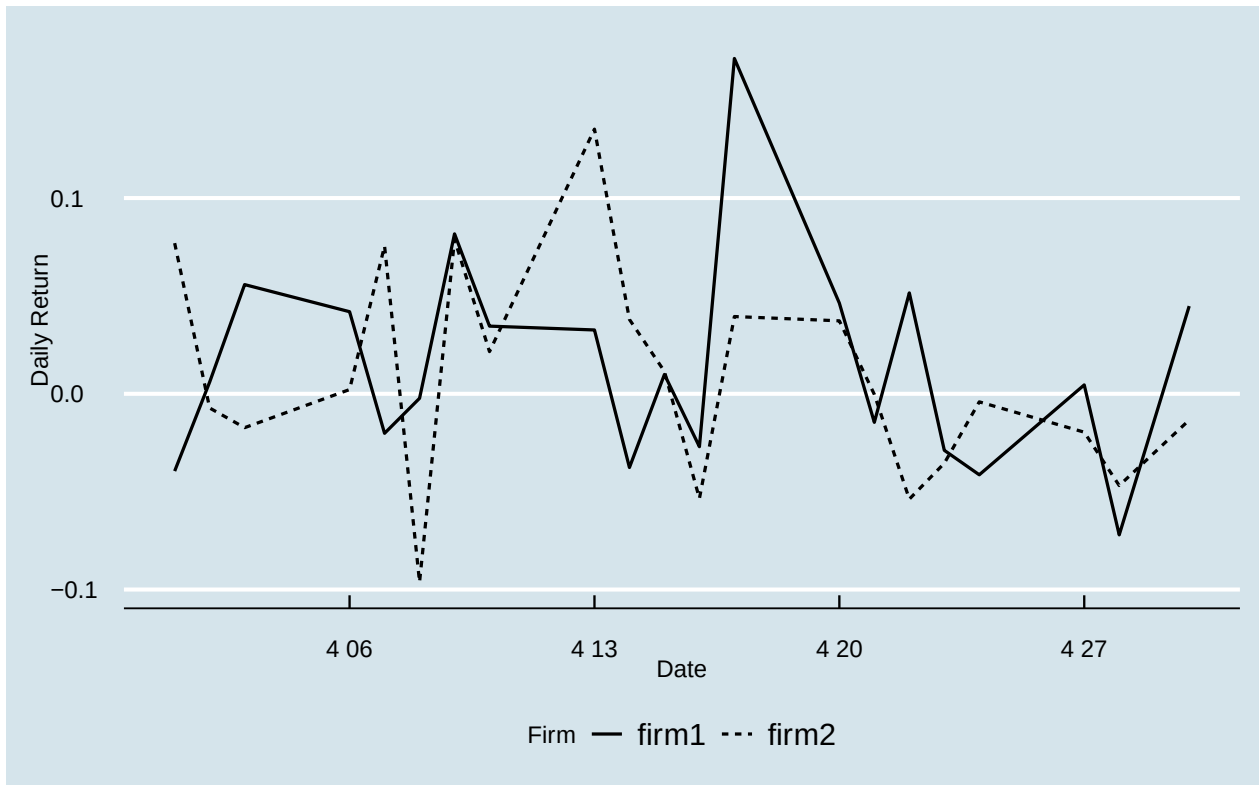
missing_data |>
  replace_na(list(return = 0))
#> # A tibble: 3 x 2
#>   firm return
#>   <chr> <dbl>
#> 1 A     0.02
#> 2 B     0
#> 3 C    -0.01

missing_data |>
  drop_na()
#> # A tibble: 2 x 2
#>   firm return
#>   <chr> <dbl>
#> 1 A     0.02
#> 2 C    -0.01
```

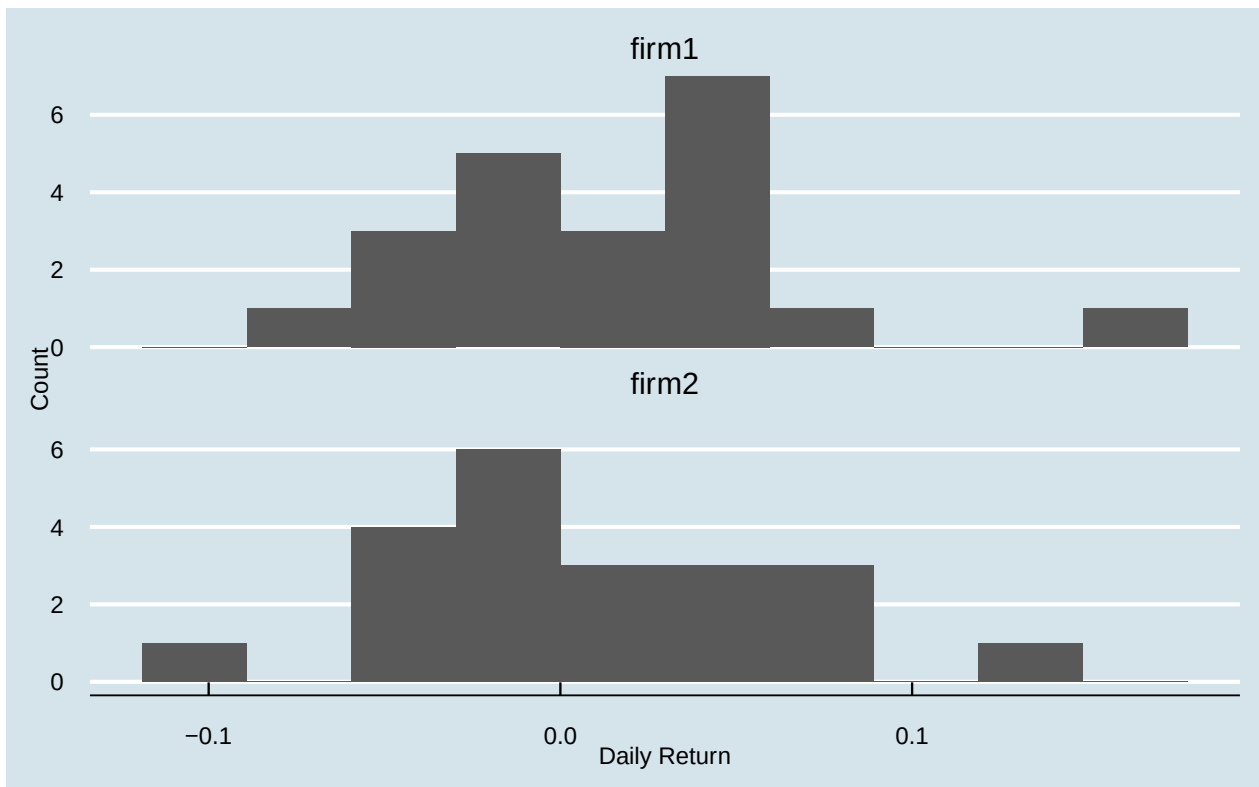
ggplot2 では、データを `ggplot()` へ渡し、`aes()` で変数と視覚要素の対応を指定し、`geom_*()` で図形を追加する。

```
(return_long |>
  ggplot(aes(x = date, y = return, linetype = firm)) +
  geom_line(linewidth = 0.7) +
  labs(
    x = "Date",
    y = "Daily Return",
    linetype = "Firm"
```

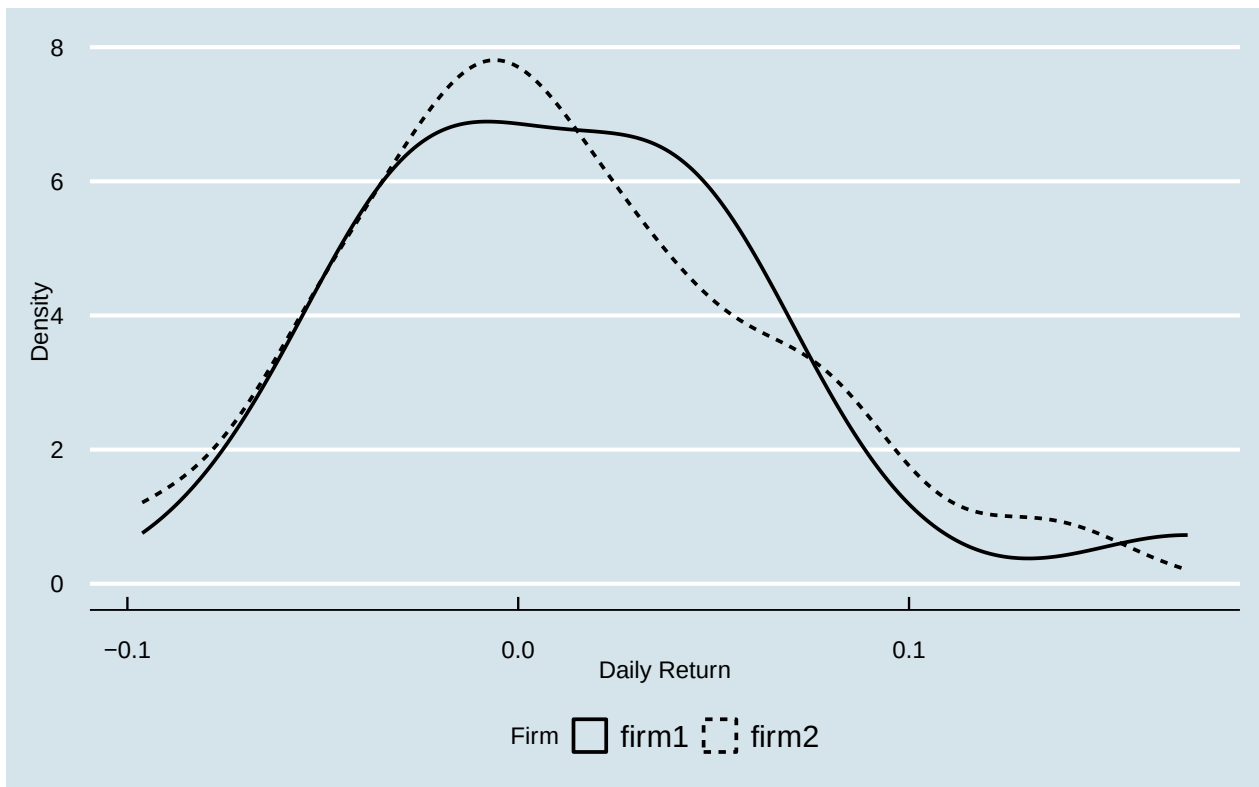
```
)) |> add_lagrange_theme_LFL()
```



```
(return_long |>
  ggplot(aes(x = return)) +
  geom_histogram(bins = 10, boundary = 0) +
  facet_wrap(vars(firm), ncol = 1) +
  labs(
    x = "Daily Return",
    y = "Count"
  )
)) |>
add_lagrange_theme_LFL()
```

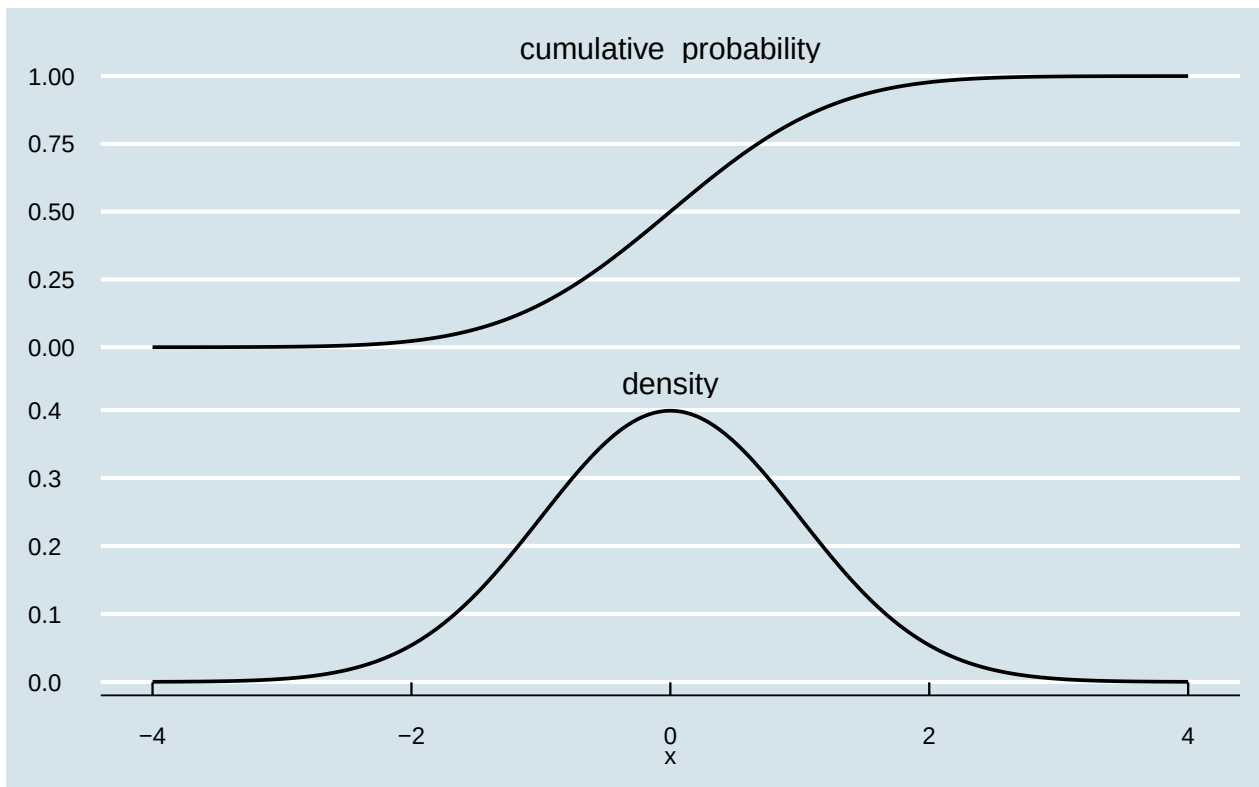


```
(return_long |>
  ggplot(aes(x = return, linetype = firm)) +
  geom_density(linewidth = 0.8) +
  labs(
    x = "Daily Return",
    y = "Density",
    linetype = "Firm"
  ) |>
  add_lagrange_theme_LFL())
```



```
normal_curve <- tibble(
  x = seq(-4, 4, length.out = 401),
  density = dnorm(x),
  cumulative_probability = pnorm(x)
)

(normal_curve |>
  pivot_longer(
    cols = c(density, cumulative_probability),
    names_to = "function_type",
    values_to = "value"
  ) |>
  ggplot(aes(x = x, y = value)) +
  geom_line(linewidth = 0.8) +
  facet_wrap(vars(function_type), scales = "free_y", ncol = 1) +
  labs(
    x = "x",
    y = NULL
  ) |>
  add_lagrange_theme_LFL())
```



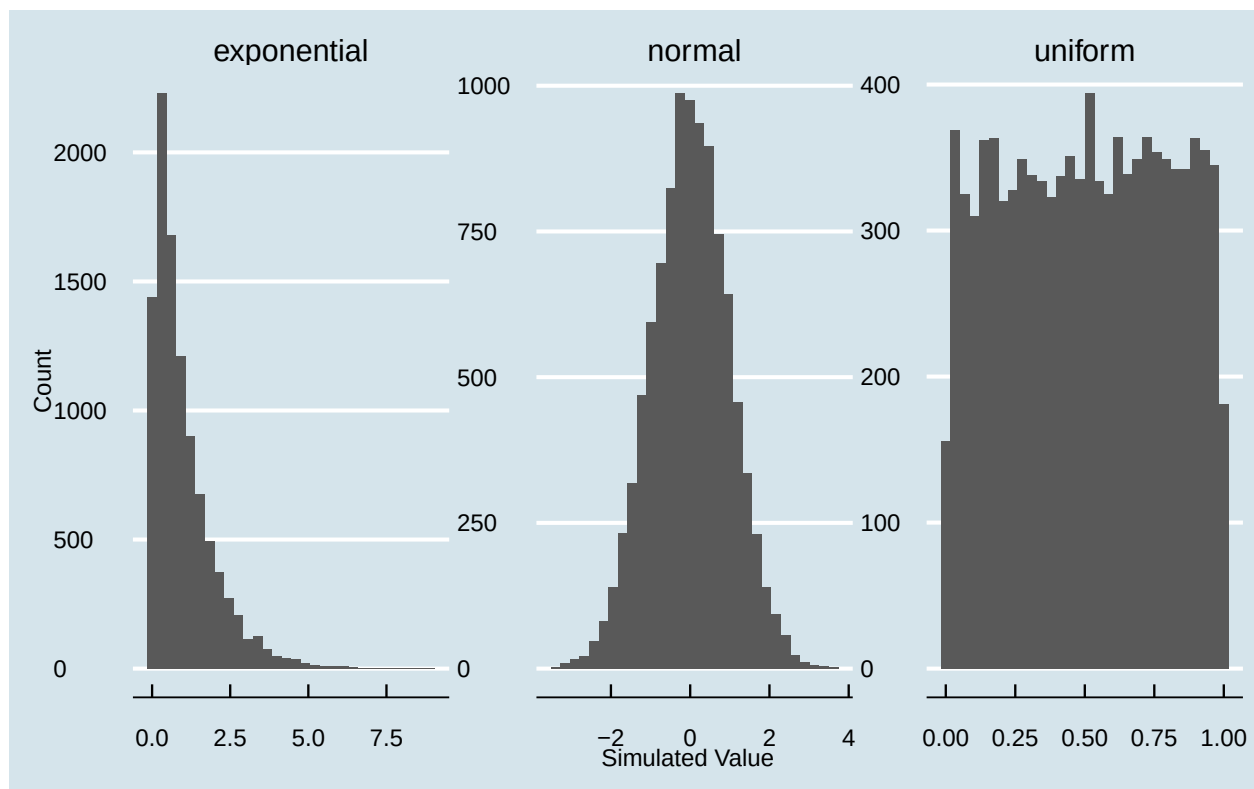
```

simulated_distributions <- tibble(
  distribution = c("normal", "uniform", "exponential"),
  values = list(rnorm(10000), runif(10000), rexp(10000))
) |>
  tidyr::unnest(values)

p <- simulated_distributions |>
  ggplot(aes(x = values)) +
  geom_histogram(bins = 30) +
  facet_wrap(vars(distribution), scales = "free") +
  labs(
    x = "Simulated Value",
    y = "Count"
  )

add_lagrange_theme_LFL(p)

```



I-1.3 関数型プログラミングと並列計算

`map()` は結果をリストとして返す。数値ベクトルが必要な場合は `map_dbl()` を用いる。

```
discount_rates <- c(0.01, 0.02, 0.03, 0.04)

discount_rates |>
  map(\(rate) calculate_present_value(100, rate))
#> [[1]]
#> [1] 99.0099
#>
#> [[2]]
#> [1] 98.03922
#>
#> [[3]]
#> [1] 97.08738
#>
#> [[4]]
#> [1] 96.15385
```

```
present_value_table <- tibble(
  discount_rate = discount_rates,
  present_value = discount_rates |>
    map_dbl(\(rate) calculate_present_value(100, rate))
)
```

```
present_value_table
#> # A tibble: 4 x 2
#>   discount_rate present_value
#>   <dbl>         <dbl>
#> 1      0.01      99.010
#> 2      0.02      98.039
#> 3      0.03      97.087
#> 4      0.04      96.154
```

```
cash_flows <- c(100, 120, 150)
years <- c(1, 2, 3)

map2_dbl(
  cash_flows,
  years,
  \(cash_flow, year) calculate_present_value(cash_flow, 0.02, year)
)
#> [1] 98.03922 115.34025 141.34835
```

```
cash_flow_scenarios <- tribble(
  ~cash_flow, ~discount_rate, ~years,
    100,      0.01,      1,
    120,      0.02,      2,
    150,      0.03,      3
)

cash_flow_scenarios |>
  mutate(
    present_value = pmap_dbl(
      list(cash_flow, discount_rate, years),
      \(cash_flow, discount_rate, years) {
        calculate_present_value(cash_flow, discount_rate, years)
      }
    )
  )
#> # A tibble: 3 x 4
#>   cash_flow discount_rate years present_value
#>   <dbl>         <dbl> <dbl>         <dbl>
#> 1    100      0.01      1      99.010
#> 2    120      0.02      2     115.34
#> 3    150      0.03      3     137.27
```



```

return_columns <- c("firm1", "firm2")

return_columns |>
  map_dfr(
    \(column_name) {
      values <- tbl_daily_stock_return[[column_name]]

      tibble(
        firm = column_name,
        observations = length(values),
        mean = mean(values),
        standard_deviation = sd(values),
        minimum = min(values),
        maximum = max(values)
      )
    }
  )
#> # A tibble: 2 x 6
#>   firm observations      mean standard_deviation  minimum maximum
#>   <chr>         <int>    <dbl>          <dbl>      <dbl>   <dbl>
#> 1 firm1             21 0.014161          0.053840 -0.072071 0.17130
#> 2 firm2             21 0.0080253        0.054390 -0.096212 0.13520

```

```

set_random_seed_LFL(1234)

distribution_specs <- tribble(
  ~distribution, ~generator,
  "uniform",     \(n) runif(n, min = -1, max = 1),
  "normal",      \(n) rnorm(n, mean = 0, sd = 1),
  "poisson",     \(n) rpois(n, lambda = 3)
)

simulated_distributions <- distribution_specs |>
  mutate(values = map(generator, \(generator) generator(1000))) |>
  select(-generator) |>
  unnest_longer(values)

simulated_distributions
#> # A tibble: 3,000 x 2
#>   distribution values
#>   <chr>         <dbl>
#> 1 uniform     -0.77259
#> 2 uniform      0.24460
#> 3 uniform      0.21855
#> 4 uniform      0.24676

```

```
#> 5 uniform      0.72183
#> 6 uniform      0.28062
#> 7 uniform     -0.98101
#> 8 uniform     -0.53490
#> 9 uniform      0.33217
#> 10 uniform     0.028502
#> # i 2,990 more rows
```

`walk()` は、戻り値を保存するのではなく、表示などの副作用もしくはファイル出力などに実行するときに使う。

```
c("firm1", "firm2") |>
  walk(\(firm) print(str_c("集計対象: ", firm)))
#> [1] "集計対象: firm1"
#> [1] "集計対象: firm2"
```

`tidyr::nest()` は、グループごとのデータを一つのリスト列へ格納する。`group_by()` が主に集計単位を指定するのに対し、`nest()` は銘柄別、業種別、ポートフォリオ別などのデータそのものを保持する。これにより、各グループへ同じ回帰分析、リスク計算、バックテストを適用しやすくなる。

```
tbl_return_long <- tbl_daily_stock_return |>
  pivot_longer(
    cols = c(firm1, firm2),
    names_to = "firm",
    values_to = "return"
  )

tbl_nested <- tbl_return_long |>
  nest(tbl_data = c(date, return), .by = firm)

tbl_nested
#> # A tibble: 2 x 2
#>   firm  tbl_data
#>   <chr> <list>
#> 1 firm1 <tibble [21 x 2]>
#> 2 firm2 <tibble [21 x 2]>
```

リスト列 `tbl_data` の各要素は `tibble` である。最初の企業のデータは次のように確認できる。

```
tbl_nested$tbl_data[[1]]
#> # A tibble: 21 x 2
#>   date      return
```

```
#>   <date>         <dbl>
#> 1 2020-04-01 -0.039482
#> 2 2020-04-02  0.0059787
#> 3 2020-04-03  0.055787
#> 4 2020-04-06  0.041935
#> 5 2020-04-07 -0.020193
#> 6 2020-04-08 -0.0022947
#> 7 2020-04-09  0.081707
#> 8 2020-04-10  0.034561
#> 9 2020-04-13  0.032601
#> 10 2020-04-14 -0.037663
#> # i 11 more rows
```

`nest()` と `map()` を組み合わせると、グループ別の処理を一つのパイプラインとして記述できる。

```
tbl_nested_summary <- tbl_nested |>
  mutate(
    tbl_summary = map(
      tbl_data,
      \(tbl_data) tibble(
        num_mean_return = mean(tbl_data$return),
        num_volatility = sd(tbl_data$return)
      )
    )
  ) |>
  select(firm, tbl_summary) |>
  unnest(tbl_summary)

tbl_nested_summary
#> # A tibble: 2 x 3
#>   firm num_mean_return num_volatility
#>   <chr>         <dbl>         <dbl>
#> 1 firm1      0.014161      0.053840
#> 2 firm2      0.0080253     0.054390
```

`purrr::map()` は要素を順番に処理する。互いに独立した計算が多数ある場合、`future` と `furrr` を使うと、`map()` に近い記法のまま複数の R セッションへ処理を分配できる。Windows では `plan(multisession)` を使用する。

ここでは、モンテカルロ法による円周率の推定を 1000 回繰り返し、逐次処理と並列処理の実行時間を比較する。各試行では、正方形内に発生させた点のうち単位円内に入った割合を 4 倍する。

```
compute_pi_path <- function(int_path_id, int_points = 100000L) {
  set_random_seed_LFL(100000L + int_path_id)

  vec_x <- runif(int_points, min = -1, max = 1)
  vec_y <- runif(int_points, min = -1, max = 1)

  tibble(
    int_path_id = int_path_id,
    num_pi = 4 * mean(vec_x^2 + vec_y^2 <= 1)
  )
}
```

1000 本の試行を、利用可能な CPU 数を上限として最大 8 個のチャンクへ分割する。

```
int_paths <- 1000L
int_workers <- min(8L, future::availableCores())

tbl_chunks <- tibble(int_path_id = seq_len(int_paths)) |>
  mutate(
    int_chunk_id = rep(seq_len(int_workers), length.out = int_paths)
  ) |>
  nest(tbl_path_ids = int_path_id, .by = int_chunk_id)

tbl_chunks
#> # A tibble: 8 x 2
#>   int_chunk_id tbl_path_ids
#>       <int> <list>
#> 1           1 <tibble [125 x 1]>
#> 2           2 <tibble [125 x 1]>
#> 3           3 <tibble [125 x 1]>
#> 4           4 <tibble [125 x 1]>
#> 5           5 <tibble [125 x 1]>
#> 6           6 <tibble [125 x 1]>
#> 7           7 <tibble [125 x 1]>
#> 8           8 <tibble [125 x 1]>
```

```
compute_pi_chunk <- function(tbl_path_ids, int_points = 100000L) {
  tbl_path_ids$int_path_id |>
    map_dfr(compute_pi_path, int_points = int_points)
}
```

まず `map()` による逐次処理を実行する。

```
future::plan(future::sequential)

num_time_map <- system.time({
  tbl_result_map <- tbl_chunks |>
    mutate(
      tbl_result = map(tbl_path_ids, compute_pi_chunk)
    ) |>
    select(int_chunk_id, tbl_result) |>
    unnest(tbl_result)
})[["elapsed"]]
```

次に `future_map()` による並列処理を実行する。

```
future::plan(future::multisession, workers = int_workers)

num_time_future_map <- system.time({
  tbl_result_future <- tbl_chunks |>
    mutate(
      tbl_result = future_map(
        tbl_path_ids,
        compute_pi_chunk,
        .options = furrr::furrr_options(seed = TRUE)
      )
    ) |>
    select(int_chunk_id, tbl_result) |>
    unnest(tbl_result)
})[["elapsed"]]

future::plan(future::sequential)
```

推定値と実行時間を比較する。

```
tbl_pi_benchmark <- tribble(
  ~chr_method, ~num_pi, ~num_elapsed_seconds,
  "map", mean(tbl_result_map$num_pi), num_time_map,
  str_c("future_map (", int_workers, " workers)", mean(tbl_result_future$num_pi),
  ↪ num_time_future_map
) |>
mutate(
  num_absolute_error = abs(num_pi - pi),
  num_speedup = first(num_elapsed_seconds) / num_elapsed_seconds
)

tbl_pi_benchmark
#> # A tibble: 2 x 5
```

```
#>   chr_method      num_pi num_elapsed_seconds num_absolute_error num_speedup
#>   <chr>          <dbl>          <dbl>          <dbl>          <dbl>
#> 1 map            3.1414            2.37            0.00016693            1
#> 2 future_map (8 worke~ 3.1415            2.21            0.000094774            1.0724
```

```
p <- tbl_pi_benchmark |>
  ggplot(aes(x = reorder(chr_method, num_elapsed_seconds), y =
    ↪ num_elapsed_seconds)) +
  geom_col() +
  coord_flip() +
  labs(x = NULL, y = "Elapsed time (seconds)")
  add_lagrange_theme_LFL(p)
```

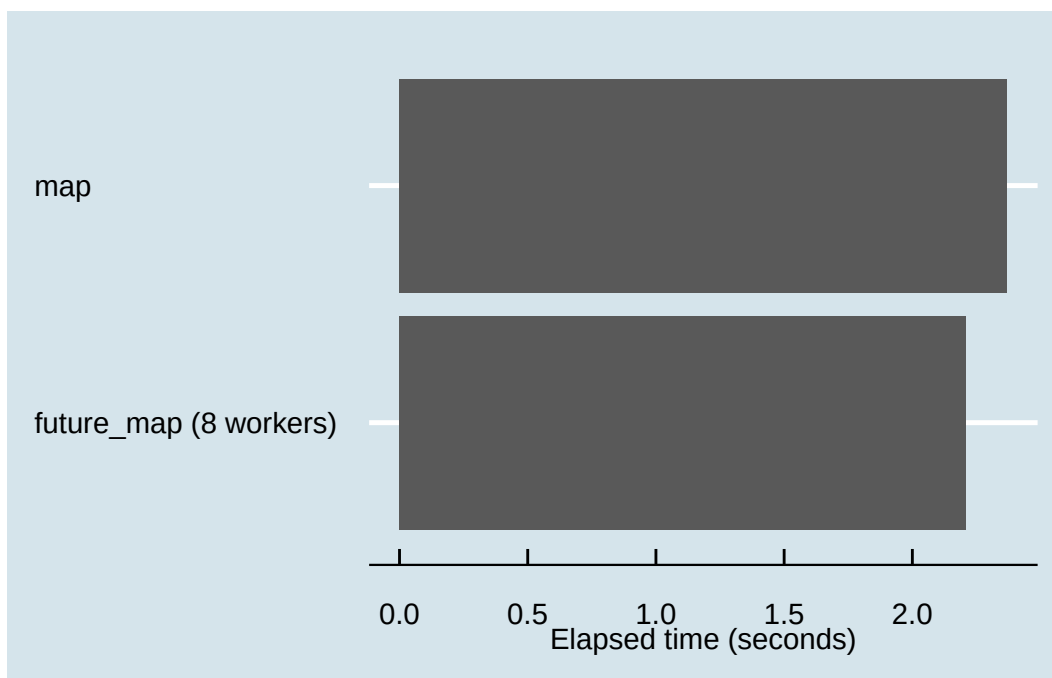


Figure1: map と future_map の実行時間比較

並列計算には、ワーカーの起動、データ転送、結果回収のオーバーヘッドがある。そのため、処理が軽い場合は `map()` の方が速いこともある。計算量が大きいモンテカルロ・シミュレーション、銘柄別モデル推定、シナリオ分析では、`future_map()` の効果が現れやすい。

I-1.4 データベースと大規模データ処理

金融データが大きくなると、すべての行と列を R のメモリへ読み込んでから処理する方法には限界がある。本節では、データベースを `dplyr` の文法で操作する `dbplyr` と、Parquet ファイルや Dataset を効率よく扱う Arrow を確認する。

`dbplyr` は、`dplyr` の処理を SQL へ変換する。データベース上で `filter()`、`select()`、

`mutate()`、`summarise()` などを実行し、必要な結果だけを `collect()` で R へ取り込める。

ここでは、再現可能な例として一時的な SQLite データベースを使用する。

`dbplyr` では、処理は直ちに R 上で実行されず、SQL クエリとして組み立てられる。
`show_query()` を使うと、生成された SQL を確認できる。

重要なのは、`collect()` を最後まで遅らせることである。先にデータベース側で行と列を絞り込めば、R へ転送するデータ量を小さくできる。

Apache Arrow は、列指向データを効率よく扱うための仕組みである。R では、Parquet ファイルの読書き、複数ファイルから成る Dataset の検索、Python や C++ など他言語との効率的なデータ交換に利用できる。

```
set_random_seed_LFL(20260726)

tbl_arrow_price <- crossing(
  date = seq.Date(as.Date("2025-01-01"), by = "day", length.out = 365),
  chr_symbol = str_c("STOCK_", str_pad(1:20, width = 2, pad = "0"))
) |>
  arrange(chr_symbol, date) |>
  mutate(
    num_return = rnorm(n(), mean = 0.0002, sd = 0.015),
    num_price = 100 * exp(cumsum(num_return)),
    .by = chr_symbol
  )

tbl_arrow_price
#> # A tibble: 7,300 x 4
#>   date      chr_symbol num_return num_price
#>   <date>    <chr>         <dbl>    <dbl>
#> 1 2025-01-01 STOCK_01    -0.014847    98.526
#> 2 2025-01-02 STOCK_01     0.00017318   98.543
#> 3 2025-01-03 STOCK_01     0.0027754    98.817
#> 4 2025-01-04 STOCK_01     0.010886    99.899
#> 5 2025-01-05 STOCK_01     0.016885   101.60
#> 6 2025-01-06 STOCK_01     0.017499   103.39
#> 7 2025-01-07 STOCK_01     0.046333   108.30
#> 8 2025-01-08 STOCK_01    -0.017287   106.44
#> 9 2025-01-09 STOCK_01     0.0033907   106.80
#> 10 2025-01-10 STOCK_01    -0.0078836   105.96
#> # i 7,290 more rows
```

```
path_parquet <- base::file.path(
  base::tempdir(),
```

```

  "financial_prices.parquet"
)

arrow::write_parquet(
  tbl_arrow_price,
  sink = path_parquet
)

base::basename(path_parquet)
#> [1] "financial_prices.parquet"

```

```

arrow::read_parquet(path_parquet) |>
  slice_head(n = 10)
#> # A tibble: 10 x 4
#>   date      chr_symbol num_return num_price
#>   <date>    <chr>         <dbl>    <dbl>
#> 1 2025-01-01 STOCK_01   -0.014847  98.526
#> 2 2025-01-02 STOCK_01    0.00017318 98.543
#> 3 2025-01-03 STOCK_01    0.0027754  98.817
#> 4 2025-01-04 STOCK_01    0.010886   99.899
#> 5 2025-01-05 STOCK_01    0.016885  101.60
#> 6 2025-01-06 STOCK_01    0.017499  103.39
#> 7 2025-01-07 STOCK_01    0.046333  108.30
#> 8 2025-01-08 STOCK_01   -0.017287  106.44
#> 9 2025-01-09 STOCK_01    0.0033907  106.80
#> 10 2025-01-10 STOCK_01   -0.0078836  105.96

```

複数の Parquet ファイルをまとめて扱う場合は Dataset を使用する。ここでは銘柄をパーティションとして保存する。

```

path_dataset <- file.path(tempdir(), "financial_dataset")

arrow::write_dataset(
  tbl_arrow_price,
  path = path_dataset,
  format = "parquet",
  partitioning = "chr_symbol",
  existing_data_behavior = "overwrite"
)

dataset_finance <- arrow::open_dataset(path_dataset)

dataset_finance
#> FileSystemDataset with 20 Parquet files

```



```
#> 4 columns
#> date: date32[day]
#> num_return: double
#> num_price: double
#> chr_symbol: string
```

Dataset に対しても `dplyr` に近い文法を使用できる。必要な銘柄、期間、列を指定してから `collect()` する。

```
tbl_arrow_selected <- dataset_finance |>
  filter(
    chr_symbol == "STOCK_01",
    date >= as.Date("2025-07-01")
  ) |>
  select(date, chr_symbol, num_price, num_return) |>
  collect()
```

```
tbl_arrow_selected
#> # A tibble: 184 x 4
#>   date      chr_symbol num_price num_return
#>   <date>    <chr>      <dbl>    <dbl>
#> 1 2025-07-01 STOCK_01      92.811  0.0061451
#> 2 2025-07-02 STOCK_01      95.183  0.025240
#> 3 2025-07-03 STOCK_01      97.579  0.024861
#> 4 2025-07-04 STOCK_01      97.833  0.0025990
#> 5 2025-07-05 STOCK_01      97.612 -0.0022665
#> 6 2025-07-06 STOCK_01      98.638  0.010456
#> 7 2025-07-07 STOCK_01      98.926  0.0029172
#> 8 2025-07-08 STOCK_01      98.611 -0.0031869
#> 9 2025-07-09 STOCK_01     100.28   0.016826
#> 10 2025-07-10 STOCK_01      99.207 -0.010806
#> # i 174 more rows
```

Arrow と Spark は役割が異なる。Arrow は、列指向形式、Parquet、Dataset、異なる言語間の高速なデータ交換を中心とし、1 台の PC でも利用しやすい。Spark は、複数のコンピュータから成るクラスターへデータと計算を分散するための処理基盤である。

データが 1 台の PC で扱える範囲では、Arrow と `dbplyr` が導入しやすい。データ量や計算量が 1 台のメモリと CPU を大きく超える場合には Spark が候補になる。本書では、実行環境を複雑にしないため、Arrow と `dbplyr` を実装対象とし、Spark は位置づけの説明にとどめる。

I-1.5 総合演習

最後に、`data/raw/market/daily_stock_returns.parquet` から読み込んだ日次リターン

を、読込み、整形、集計、反復処理、可視化の順に分析する。

```
analysis_data <- tbl_daily_stock_return |>
  pivot_longer(
    cols = c(firm1, firm2),
    names_to = "firm",
    values_to = "return"
  ) |>
  mutate(
    gross_return = 1 + return,
    direction = case_when(
      return > 0 ~ "positive",
      return < 0 ~ "negative",
      TRUE ~ "zero"
    )
  ) |>
  arrange(firm, date)

analysis_data
#> # A tibble: 42 x 5
#>   date      firm      return gross_return direction
#>   <date>    <chr>    <dbl>         <dbl> <chr>
#> 1 2020-04-01 firm1 -0.039482      0.96052 negative
#> 2 2020-04-02 firm1  0.0059787    1.0060  positive
#> 3 2020-04-03 firm1  0.055787      1.0558  positive
#> 4 2020-04-06 firm1  0.041935      1.0419  positive
#> 5 2020-04-07 firm1 -0.020193      0.97981 negative
#> 6 2020-04-08 firm1 -0.0022947    0.99771 negative
#> 7 2020-04-09 firm1  0.081707      1.0817  positive
#> 8 2020-04-10 firm1  0.034561      1.0346  positive
#> 9 2020-04-13 firm1  0.032601      1.0326  positive
#> 10 2020-04-14 firm1 -0.037663      0.96234 negative
#> # i 32 more rows
```

```
firm_summary <- analysis_data |>
  group_by(firm) |>
  summarise(
    observations = n(),
    mean_return = mean(return),
    standard_deviation = sd(return),
    minimum_return = min(return),
    maximum_return = max(return),
    positive_ratio = mean(return > 0),
    .groups = "drop"
  )
```

```

firm_summary
#> # A tibble: 2 x 7
#>   firm observations mean_return standard_deviation minimum_return
#>   <chr>          <int>      <dbl>          <dbl>          <dbl>
#> 1 firm1             21    0.014161          0.053840       -0.072071
#> 2 firm2             21    0.0080253          0.054390       -0.096212
#> # i 2 more variables: maximum_return <dbl>, positive_ratio <dbl>

```

```

cumulative_return_data <- analysis_data |>
  group_by(firm) |>
  mutate(
    cumulative_gross_return = cumprod(gross_return),
    cumulative_return = cumulative_gross_return - 1
  ) |>
  ungroup()

cumulative_return_data
#> # A tibble: 42 x 7
#>   date      firm      return gross_return direction cumulative_gross_return
#>   <date>    <chr>    <dbl>      <dbl> <chr>          <dbl>
#> 1 2020-04-01 firm1 -0.039482    0.96052 negative      0.96052
#> 2 2020-04-02 firm1  0.0059787    1.0060 positive      0.96626
#> 3 2020-04-03 firm1  0.055787    1.0558 positive      1.0202
#> 4 2020-04-06 firm1  0.041935    1.0419 positive      1.0629
#> 5 2020-04-07 firm1 -0.020193    0.97981 negative      1.0415
#> 6 2020-04-08 firm1 -0.0022947    0.99771 negative      1.0391
#> 7 2020-04-09 firm1  0.081707    1.0817 positive      1.1240
#> 8 2020-04-10 firm1  0.034561    1.0346 positive      1.1628
#> 9 2020-04-13 firm1  0.032601    1.0326 positive      1.2007
#> 10 2020-04-14 firm1 -0.037663    0.96234 negative      1.1555
#> # i 32 more rows
#> # i 1 more variable: cumulative_return <dbl>

```

```

firm_names <- unique(analysis_data$firm)

firm_names |>
  map_dfr(
    \(firm_name) {
      firm_returns <- analysis_data |>
        filter(firm == firm_name) |>
        pull(return)

      tibble(

```

```

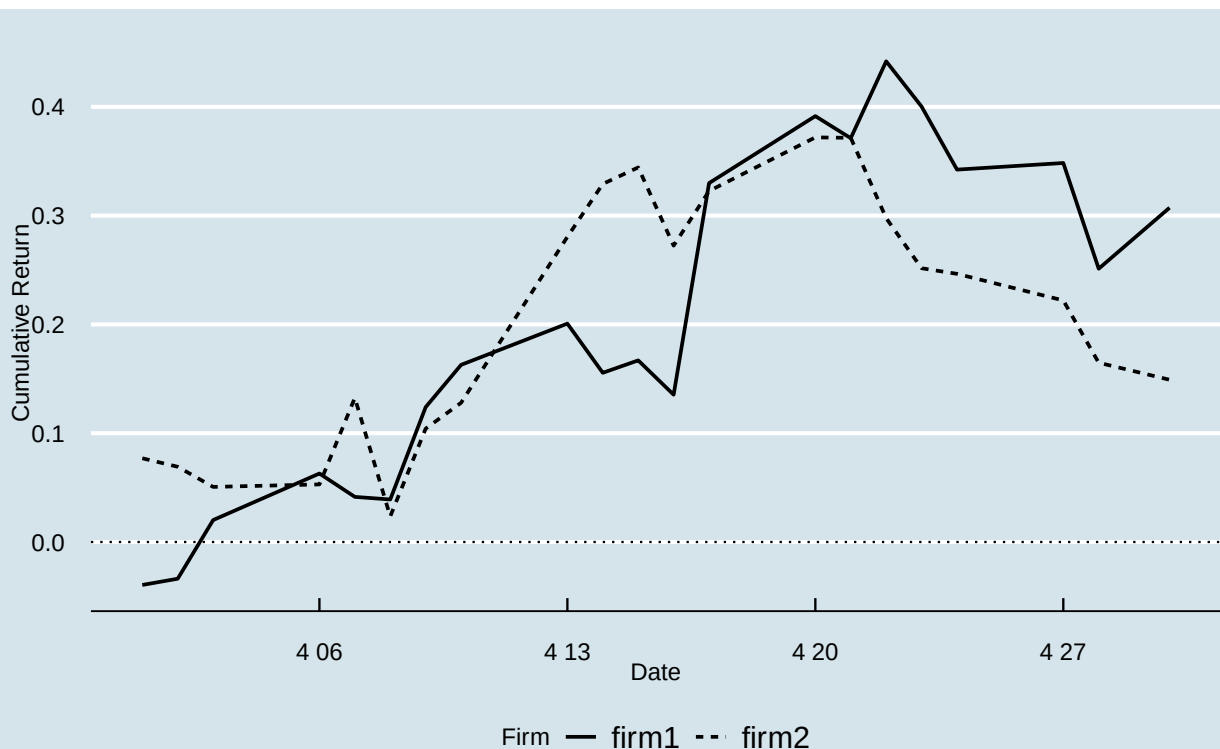
    firm = firm_name,
    mean_return = mean(firm_returns),
    volatility = sd(firm_returns),
    lower_5_percent = quantile(firm_returns, 0.05),
    upper_95_percent = quantile(firm_returns, 0.95)
  )
}
)
#> # A tibble: 2 x 5
#>   firm mean_return volatility lower_5_percent upper_95_percent
#>   <chr>      <dbl>      <dbl>         <dbl>         <dbl>
#> 1 firm1    0.014161    0.053840     -0.041378      0.081707
#> 2 firm2    0.0080253   0.054390     -0.053818      0.078843

```

```

p <- cumulative_return_data |>
  ggplot(aes(x = date, y = cumulative_return, linetype = firm)) +
  geom_line(linewidth = 0.8) +
  geom_hline(yintercept = 0, linetype = "dotted") +
  labs(
    x = "Date",
    y = "Cumulative Return",
    linetype = "Firm"
  )
p |> add_lagrange_theme_LFL()

```



I-1.6 まとめ

本章では、Base R の基本的なデータ構造と関数から出発し、記述統計、確率分布、tidyverse によるデータ分析までを一通り確認した。重要な点は次のとおりである。

- オブジェクト名には内容を表す snake_case を用いる。
- 同じ計算を複数の値へ適用するときは、まずベクトル化を検討する。
- 反復処理が必要な場合は `purrr::map()` 系関数を用いる。
- データ操作は `dplyr`、文字列処理は `stringr`、整形は `tidyr` を用いる。
- パイプ演算子は `|>` に統一する。
- グラフは `ggplot2` でデータ、変数、図形を分けて組み立てる。
- データファイルは絶対パスではなく、プロジェクトルートを基準に読み込む。

本章で扱った技術は、次の流れとしてつながる。

CSV・データベース・Parquet



dbplyr・Arrow



filter・select



collect



tibble・stringr・tidyr



nest



map



future_map



集計・モデル推定・可視化

dbplyr と Arrow は、必要なデータを絞り込む入口を担う。collect() 後は、tibble として文字列処理、整形、集計を行い、nest() で銘柄別・業種別・シナリオ別のデータを保持する。同じ分析は map() で反復し、計算量が多い場合は future_map() へ置き換える。可視化には ggplot2 を使い、本書では add_lagrange_theme_LFL() を標準テーマとする。

次章以降では、本章で学んだ操作を用いて、財務データと株式データの分析を進める。

第 I-2 章財務データの整理と分析

本章では、企業の財務データを読み込み、データ型、欠損値、年度構成および産業構成を確認する。

I-2.1 分析環境

I-2.2 財務データの読み込みと整備

```
financial_data_raw <-
  read_parquet_file_LFL(
    file.path(
      "raw",
      "fundamentals",
      "financial_statements.parquet"
    )
  )

show_table_LFL(financial_data_raw, n = 10)
```

```
#> # A tibble: 7,920 x 11
#>   year fir_ID ind_ID sales    OX    NFE    X    OA    FA    OL
#> * <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
#> 1  2015     1     1 5261.4 437.49 NA      286.64 13006.  3543.4 4373.0
#> 2  2016     1     1 5949.0 564.14 50.667  513.48 13866.  4642.2 4534.2
#> 3  2017     1     1 6505.1 691.18 29.543  661.64 13953.  7744.0 5111.2
#> 4  2018     1     1 6846.4 751.29 86.487  664.8  18818.  7284.7 5137.3
#> 5  2019     1     1 7572.2 958.53 298.05  660.48 18190  9735.1 5488.0
#> 6  2020     1     1 7537.6 778.37 -65.459 843.83 20463. 10274.  5371.4
#> 7  2015     2     1 3505.8  45.82  5.7511  40.07  2977.8  2258.3 1840.4
#> 8  2016     2     1 3491.3  51.25  1.8765  49.37  3184.8  1881.8 1769.2
#> 9  2017     2     1 3945.7  83.43  7.5279  75.9  3392.2  2425.2 1955.3
#> 10 2018     2     1 4139.3  93.4   6.8166  86.58  3569.3  2331.0 2022.1
#> # i 7,910 more rows
#> # i 1 more variable: F0 <dbl>
```

```

show_data_structure_LFL(financial_data_raw)
#> Rows: 7,920
#> Columns: 11
#> $ year      <dbl> 2015, 2016, 2017, 2018, 2019, 2020, 2015, 2016, 2017, 2018~
#> $ firm_ID   <dbl> 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 3, 4, 4~
#> $ industry_ID <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1~
#> $ sales     <dbl> 5261.40, 5948.96, 6505.06, 6846.38, 7572.24, 7537.63, 3505~
#> $ OX        <dbl> 437.49, 564.14, 691.18, 751.29, 958.53, 778.37, 45.82, 51.~
#> $ NFE       <dbl> NA, 50.667498, 29.543157, 86.486500, 298.049774, -65.45877~
#> $ X         <dbl> 286.64, 513.48, 661.64, 664.80, 660.48, 843.83, 40.07, 49.~
#> $ OA        <dbl> 13005.55, 13865.58, 13952.58, 18818.48, 18190.00, 20462.86~
#> $ FA        <dbl> 3543.43, 4642.16, 7743.99, 7284.72, 9735.13, 10274.25, 225~
#> $ OL        <dbl> 4372.96, 4534.22, 5111.22, 5137.28, 5487.96, 5371.38, 1840~
#> $ FO        <dbl> 2480.72, 3959.70, 6159.02, 10123.91, 11362.22, 13772.15, 2~

financial_data_overview <- financial_data_raw |>
  summarise(
    observations = n(),
    variables = ncol(financial_data_raw),
    first_year = min(year, na.rm = TRUE),
    last_year = max(year, na.rm = TRUE),
    firms = n_distinct(firm_ID),
    industries = n_distinct(industry_ID)
  )

show_table_LFL(financial_data_overview)
#> # A tibble: 1 x 6
#>   observations variables fir_year las_year firms industries
#>   <int>         <int>   <dbl>   <dbl> <int>         <int>
#> 1      7920          11    2015    2020  1515          10

```

列ごとの欠損数と欠損率を確認する。

```

missing_summary <- financial_data_raw |>
  summarise(
    across(
      everything(),
      list(
        missing = \(x) sum(is.na(x)),
        missing_rate = \(x) mean(is.na(x))
      ),
      .names = "{.col}__{.fn}"
    )
  ) |>
  pivot_longer(

```

```

  cols = everything(),
  names_to = c("variable", ".value"),
  names_sep = "__"
) |>
  arrange(desc(missing_rate), variable)

```

```

show_table_LFL(missing_summary)
#> # A tibble: 11 x 3
#>   variable    missing  mis_rate
#>   <chr>      <int>    <dbl>
#> 1 NFE              1 0.00012626
#> 2 FA                0 0
#> 3 FO                0 0
#> 4 OA                0 0
#> 5 OL                0 0
#> 6 OX                0 0
#> 7 X                 0 0
#> 8 firm_ID          0 0
#> 9 industry_ID      0 0
#> 10 sales            0 0
#> # i 1 more row

```

企業 ID と産業 ID は分類変数であるためファクター型へ変換する。年度は整数型へ変換し、前年度の値を正しく取得できるように企業 ID と年度の順に並べる。

```

financial_data <- financial_data_raw |>
  mutate(
    firm_ID = as.factor(firm_ID),
    industry_ID = as.factor(industry_ID),
    year = as.integer(year)
  ) |>
  arrange(firm_ID, year)

show_data_structure_LFL(financial_data)
#> Rows: 7,920
#> Columns: 11
#> $ year      <int> 2015, 2016, 2017, 2018, 2019, 2020, 2015, 2016, 2017, 2018~
#> $ firm_ID    <fct> 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 3, 4, 4~
#> $ industry_ID <fct> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1~
#> $ sales      <dbl> 5261.40, 5948.96, 6505.06, 6846.38, 7572.24, 7537.63, 3505~
#> $ OX         <dbl> 437.49, 564.14, 691.18, 751.29, 958.53, 778.37, 45.82, 51.~
#> $ NFE        <dbl> NA, 50.667498, 29.543157, 86.486500, 298.049774, -65.45877~
#> $ X          <dbl> 286.64, 513.48, 661.64, 664.80, 660.48, 843.83, 40.07, 49.~
#> $ OA         <dbl> 13005.55, 13865.58, 13952.58, 18818.48, 18190.00, 20462.86~
#> $ FA         <dbl> 3543.43, 4642.16, 7743.99, 7284.72, 9735.13, 10274.25, 225~

```



```
#> $ OL          <dbl> 4372.96, 4534.22, 5111.22, 5137.28, 5487.96, 5371.38, 1840~
#> $ FO          <dbl> 2480.72, 3959.70, 6159.02, 10123.91, 11362.22, 13772.15, 2~
```

企業と年度の組合せが重複していないことを確認する。

```
panel_key_duplicates <- financial_data |>
  count(
    firm_ID,
    year,
    name = "observations"
  ) |>
  filter(observations > 1)

show_table_LFL(panel_key_duplicates)
#> # A tibble: 0 x 3
#> # i 3 variables: fir_ID <fct>, year <int>, observations <int>
```

```
if (nrow(panel_key_duplicates) > 0) {
  stop(
    "firm_IDとyearの組合せに重複があります。",
    call. = FALSE
  )
}
```

I-2.3 標本構成と売上高分布

年度ごとの企業数と観測数を集計する。

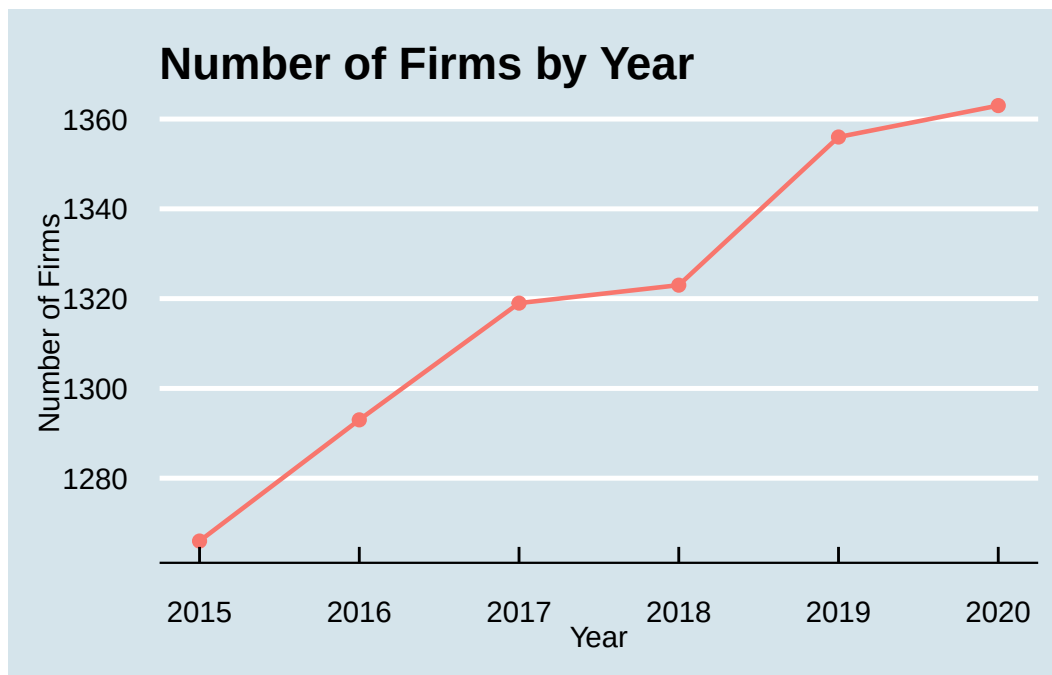
```
firms_by_year <- financial_data |>
  group_by(year) |>
  summarise(
    firms = n_distinct(firm_ID),
    observations = n(),
    .groups = "drop"
  ) |>
  arrange(year)

show_table_LFL(firms_by_year)
#> # A tibble: 6 x 3
#>   year firms observations
#>   <int> <int>         <int>
#> 1  2015  1266         1266
#> 2  2016  1293         1293
```

```
#> 3 2017 1319      1319
#> 4 2018 1323      1323
#> 5 2019 1356      1356
#> 6 2020 1363      1363
```

```
p_firms_by_year <- firms_by_year |>
  ggplot(aes(x = year, y = firms, colour = "Firms")) +
  geom_line(linewidth = 0.8) +
  geom_point(size = 2) +
  scale_x_continuous(breaks = firms_by_year$year) +
  guides(colour = "none") +
  labs(
    x = "Year",
    y = "Number of Firms",
    title = "Number of Firms by Year"
  )

add_lagrange_theme_LFL(p_firms_by_year)
```



最新年度について、産業ごとの企業数を集計する。

```
latest_year <-
  max(financial_data$year, na.rm = TRUE)

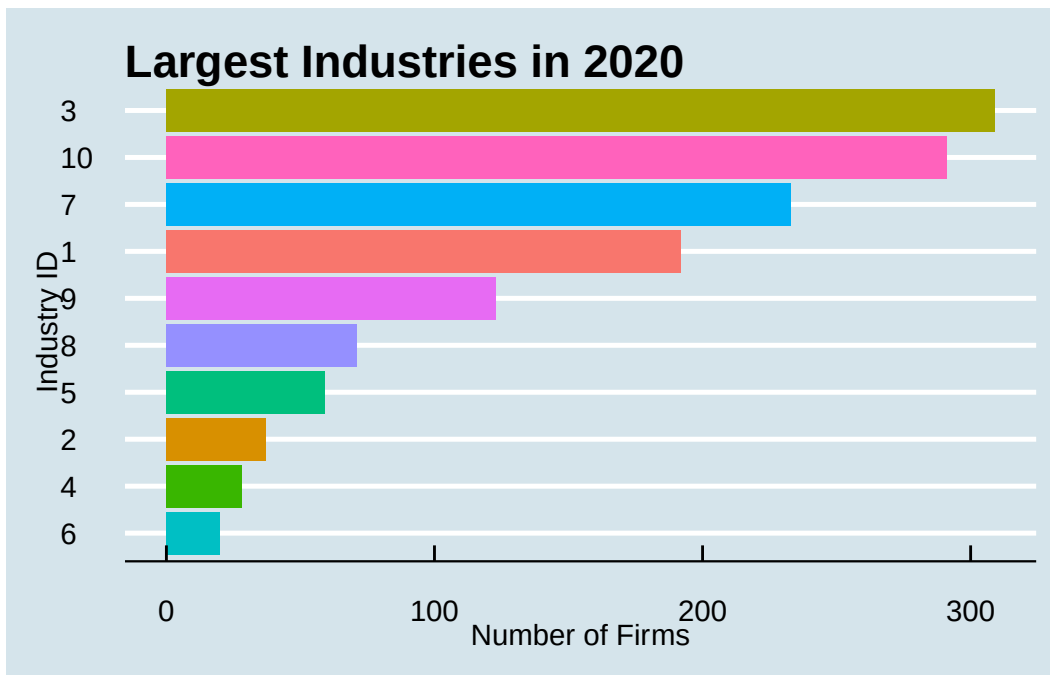
firms_by_industry <- financial_data |>
  filter(year == latest_year) |>
  count(
    industry_ID,
    name = "firms",
```

```
    sort = TRUE
  )

show_table_LFL(firms_by_industry, n = 20)
#> # A tibble: 10 x 2
#>   ind_ID firms
#>   <fct>  <int>
#> 1 3      309
#> 2 10     291
#> 3 7      233
#> 4 1      192
#> 5 9      123
#> 6 8       71
#> 7 5       59
#> 8 2       37
#> 9 4       28
#> 10 6      20
```

```
p_firms_by_industry <- firms_by_industry |>
  slice_head(n = 15) |>
  ggplot(
    aes(
      x = reorder(industry_ID, firms),
      y = firms,
      fill = industry_ID
    )
  ) +
  geom_col(show.legend = FALSE) +
  coord_flip() +
  labs(
    x = "Industry ID",
    y = "Number of Firms",
    title = paste("Largest Industries in", latest_year)
  )

add_lagrange_theme_LFL(p_firms_by_industry)
```



売上高は企業規模によって大きく異なるため、元の値と自然対数の両方を確認する。

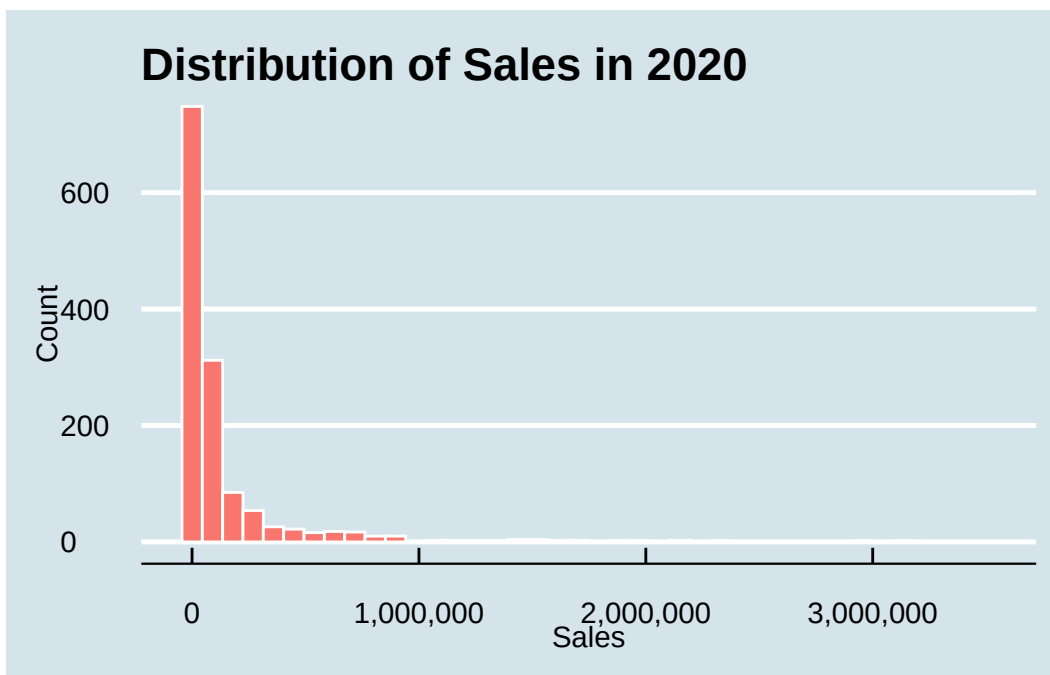
```
sales_summary <- financial_data |>
  summarise(
    observations = sum(!is.na(sales)),
    minimum = min(sales, na.rm = TRUE),
    first_quartile = quantile(sales, 0.25, na.rm = TRUE),
    median = median(sales, na.rm = TRUE),
    mean = mean(sales, na.rm = TRUE),
    third_quartile = quantile(sales, 0.75, na.rm = TRUE),
    maximum = max(sales, na.rm = TRUE)
  )

show_table_LFL(sales_summary)
#> # A tibble: 1 x 7
#>   observations minimum fir_quart median    mean thi_quart maximum
#>   <int>      <dbl>    <dbl>  <dbl>  <dbl>   <dbl>   <dbl>
#> 1      7920    205.34   16103. 40431. 166007. 118314. 3496433
```

```
p_sales_distribution <- financial_data |>
  filter(
    year == latest_year,
    !is.na(sales),
    sales >= 0
  ) |>
  ggplot(aes(x = sales, fill = "Sales")) +
  geom_histogram(
    bins = 40,
    colour = "white",
```

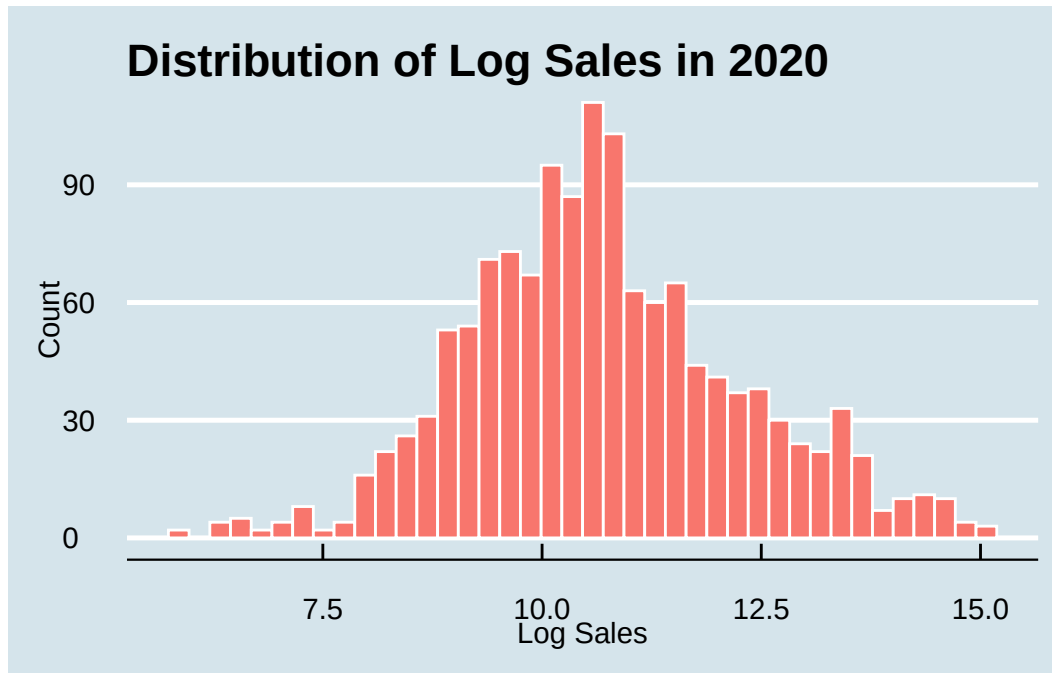
```
    show.legend = FALSE
  ) +
  scale_x_continuous(labels = scales::label_comma()) +
  labs(
    x = "Sales",
    y = "Count",
    title = paste("Distribution of Sales in", latest_year)
  )

add_lagrange_theme_LFL(p_sales_distribution)
```



```
p_log_sales_distribution <- financial_data |>
  filter(
    year == latest_year,
    !is.na(sales),
    sales > 0
  ) |>
  ggplot(aes(x = log(sales), fill = "Log Sales")) +
  geom_histogram(
    bins = 40,
    colour = "white",
    show.legend = FALSE
  ) +
  labs(
    x = "Log Sales",
    y = "Count",
    title = paste("Distribution of Log Sales in", latest_year)
  )
```

```
add_lagrange_theme_LFL(p_log_sales_distribution)
```



I-2.4 株主資本と ROE 分析

本章では、営業資産から営業負債を控除した純営業資産と、金融負債から金融資産を控除した純金融負債の差として株主資本を計算する。

$$BE_t = (OA_t - OL_t) - (FO_t - FA_t)$$

ROE は当期純利益を期首株主資本で割って計算する。

$$ROE_t = \frac{X_t}{BE_{t-1}}$$

ゼロ除算や無限大は欠損値へ置き換える。

```
divide_safely_LFL <- function(numerator, denominator) {
  result <- numerator / denominator
  replace(result, !is.finite(result), NA_real_)
}
```

```
financial_data <- financial_data |>
  mutate(
    NOA = OA - OL,
    NFO = FO - FA,
```

```

    BE = NOA - NFO
  ) |>
  group_by(firm_ID) |>
  arrange(year, .by_group = TRUE) |>
  mutate(
    lagged_BE = lag(BE),
    ROE = divide_safely_LFL(X, lagged_BE)
  ) |>
  ungroup()

financial_data |>
  select(
    firm_ID,
    industry_ID,
    year,
    X,
    BE,
    lagged_BE,
    ROE
  ) |>
  slice_head(n = 10) |>
  show_table_LFL(n = 10)
#> # A tibble: 10 x 7
#>   fir_ID ind_ID year      X      BE lag_BE      ROE
#>   <fct>  <fct> <int>  <dbl>   <dbl>  <dbl>   <dbl>
#> 1 1      1      2015 286.64 9695.3    NA     NA
#> 2 1      1      2016 513.48 10014.    9695.3 0.052962
#> 3 1      1      2017 661.64 10426.    10014. 0.066073
#> 4 1      1      2018 664.8  10842.    10426. 0.063762
#> 5 1      1      2019 660.48 11075.    10842. 0.060919
#> 6 1      1      2020 843.83 11594.    11075. 0.076193
#> 7 2      1      2015  40.07 1054.9     NA     NA
#> 8 2      1      2016  49.37 1081.6    1054.9 0.046800
#> 9 2      1      2017  75.9  1134.8    1081.6 0.070174
#> 10 2     1      2018  86.58 1183.7    1134.8 0.076295

```

ROE の要約統計量を確認する。

```

roe_summary <- financial_data |>
  summarise(
    observations = sum(!is.na(ROE)),
    mean = mean(ROE, na.rm = TRUE),
    median = median(ROE, na.rm = TRUE),
    standard_deviation = sd(ROE, na.rm = TRUE),
    first_percentile = quantile(ROE, 0.01, na.rm = TRUE),

```

```

    ninety_ninth_percentile = quantile(ROE, 0.99, na.rm = TRUE)
  )

show_table_LFL(roe_summary)
#> # A tibble: 1 x 6
#>   observations    mean  median sta_devia fir_perce nin_perce
#>   <int>      <dbl>   <dbl>   <dbl>    <dbl>    <dbl>
#> 1      6405 0.077713 0.072828 0.066274 -0.079541 0.26731

```

極端値の影響を避けるため、1 パーセンタイルから 99 パーセンタイルまでを表示する。

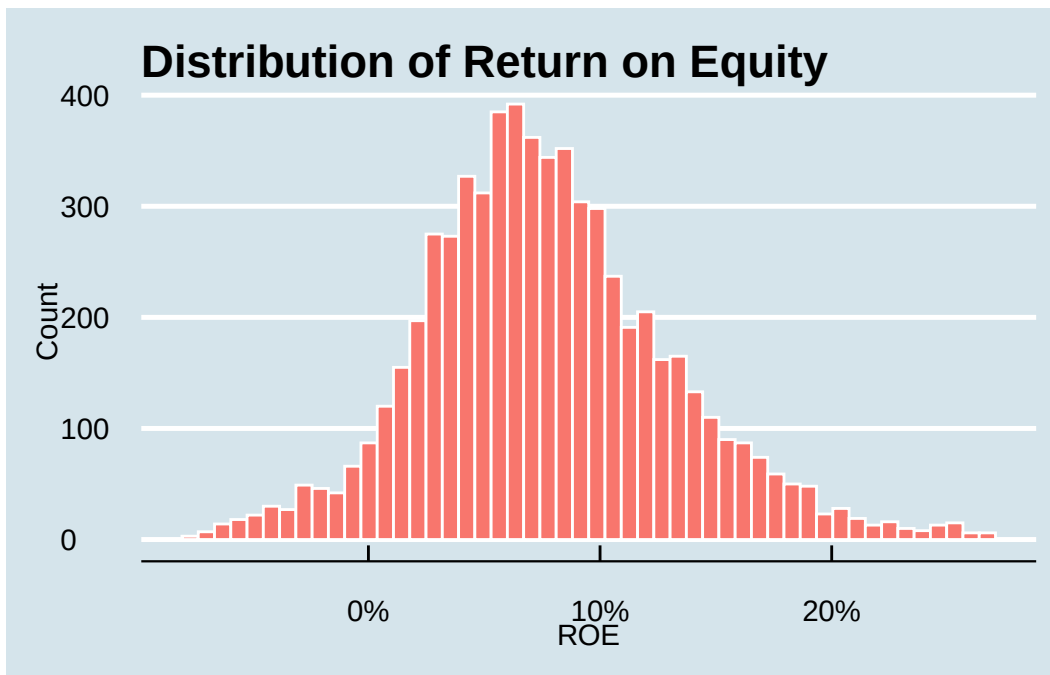
```

roe_limits <- quantile(
  financial_data$ROE,
  probs = c(0.01, 0.99),
  na.rm = TRUE
)

p_roe_distribution <- financial_data |>
  filter(
    !is.na(ROE),
    between(ROE, roe_limits[[1]], roe_limits[[2]])
  ) |>
  ggplot(aes(x = ROE, fill = "ROE")) +
  geom_histogram(
    bins = 50,
    colour = "white",
    show.legend = FALSE
  ) +
  scale_x_continuous(labels = scales::label_percent()) +
  labs(
    x = "ROE",
    y = "Count",
    title = "Distribution of Return on Equity"
  )

add_lagrange_theme_LFL(p_roe_distribution)

```

```

industry_roe_summary <- financial_data |>
  group_by(industry_ID) |>
  summarise(
    firms = n_distinct(firm_ID),
    observations = sum(!is.na(ROE)),
    mean_ROE = mean(ROE, na.rm = TRUE),
    median_ROE = median(ROE, na.rm = TRUE),
    .groups = "drop"
  ) |>
  filter(observations > 0) |>
  arrange(desc(mean_ROE))

show_table_LFL(industry_roe_summary, n = 20)
#> # A tibble: 10 x 5
#>   ind_ID firms observations  mea_ROE  med_ROE
#>   <fct> <int>         <int>    <dbl>    <dbl>
#> 1 2      40          186 0.10762  0.10283
#> 2 6      34          127 0.095142 0.091981
#> 3 8      80          349 0.093711 0.084671
#> 4 5      67          259 0.085703 0.084851
#> 5 9     136          531 0.083403 0.079794
#> 6 1     225          918 0.077285 0.075118
#> 7 3     327         1433 0.075489 0.070116
#> 8 10    314         1388 0.075273 0.070490
#> 9 4      34          138 0.073719 0.072702
#> 10 7     258         1076 0.067555 0.060409

```

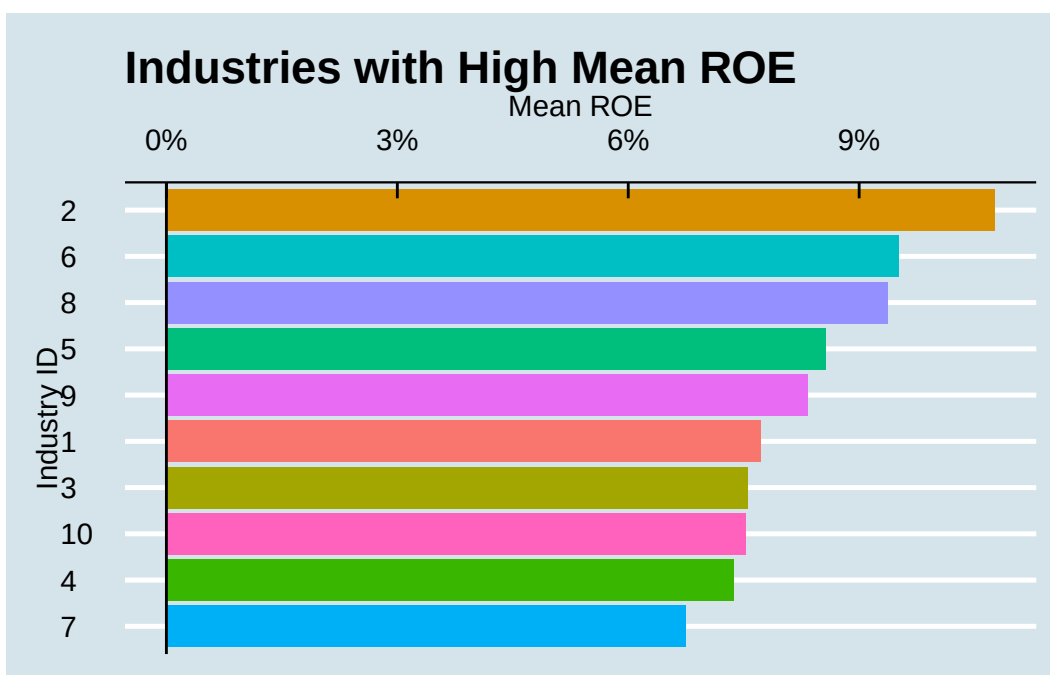
産業ごとの標本数が小さい場合、平均値は一部企業の影響を強く受ける。ここでは ROE の観測数が 50 以上の産業だけを比較する。

```

p_industry_roe <- industry_roe_summary |>
  filter(observations >= 50) |>
  slice_max(
    order_by = mean_ROE,
    n = 15,
    with_ties = FALSE
  ) |>
  ggplot(
    aes(
      x = reorder(industry_ID, mean_ROE),
      y = mean_ROE,
      fill = industry_ID
    )
  ) +
  geom_col(show.legend = FALSE) +
  coord_flip() +
  geom_hline(yintercept = 0) +
  labs(
    x = "Industry ID",
    y = "Mean ROE",
    title = "Industries with High Mean ROE"
  )

add_lagrange_theme_LFL(p_industry_roe) +
  scale_y_continuous(
    position = "right",
    labels = scales::label_percent()
  )

```



最新年度について、産業内で ROE が高い企業から順に順位を付ける。dense_rank() を使うと、同順位がある場合でも次の順位を飛ばさない。

```
latest_roe_ranking <- financial_data |>
  filter(
    year == latest_year,
    !is.na(ROE)
  ) |>
  group_by(industry_ID) |>
  mutate(
    ROE_rank = dense_rank(desc(ROE))
  ) |>
  ungroup() |>
  arrange(industry_ID, ROE_rank)
```

```
latest_roe_ranking |>
  select(
    industry_ID,
    ROE_rank,
    firm_ID,
    ROE
  ) |>
  slice_head(n = 20) |>
  show_table_LFL(n = 20)
```

```
#> # A tibble: 20 x 4
```

```
#>   ind_ID ROE_rank fir_ID    ROE
#>   <fct>    <int> <fct>   <dbl>
#> 1 1      1      1 8      0.38826
#> 2 1      2      2 90      0.33395
#> 3 1      3      225     0.30061
#> 4 1      4      81      0.23113
#> 5 1      5      171     0.21847
#> 6 1      6      138     0.20680
#> 7 1      7      66      0.20335
#> 8 1      8      153     0.19822
#> 9 1      9      181     0.19080
#> 10 1     10      65      0.18881
#> 11 1     11      92      0.17455
#> 12 1     12      195     0.15685
#> 13 1     13      13      0.15474
#> 14 1     14      23      0.15316
#> 15 1     15      148     0.14986
#> 16 1     16      214     0.14639
#> 17 1     17      115     0.14620
#> 18 1     18      201     0.14286
```

```
#> 19 1          19 50      0.14212
#> 20 1          20 143     0.14211
```

各産業で ROE が最も高い企業を抽出する。

```
top_firm_by_industry <- latest_roe_ranking |>
  group_by(industry_ID) |>
  slice_max(
    order_by = ROE,
    n = 1,
    with_ties = FALSE
  ) |>
  ungroup() |>
  select(
    industry_ID,
    firm_ID,
    ROE
  )

show_table_LFL(top_firm_by_industry, n = 20)
#> # A tibble: 10 x 3
#>   ind_ID fir_ID    ROE
#>   <fct> <fct>    <dbl>
#> 1 1      8      0.38826
#> 2 2     242     0.37500
#> 3 3     475     0.49754
#> 4 4     619     0.14913
#> 5 5     661     0.26731
#> 6 6     719     0.14220
#> 7 7     929     0.56418
#> 8 8    1042     0.25600
#> 9 9    1167     0.23462
#> 10 10   1380     0.24979
```

I-2.5 上級デュボン・モデル

上級デュボン・モデルでは、ROE を純営業資産利益率と財務レバレッジの効果へ分解する。

$$ROE_t = RNOA_t + FLEV_{t-1}(RNOA_t - NBC_t)$$

各構成要素を次のように定義する。

$$RNOA_t = \frac{OX_t}{NOA_{t-1}}$$

$$PM_t = \frac{OX_t}{Sales_t}$$

$$ATO_t = \frac{Sales_t}{NOA_{t-1}}$$

$$FLEV_{t-1} = \frac{NFO_{t-1}}{BE_{t-1}}$$

$$NBC_t = \frac{NFE_t}{NFO_{t-1}}$$

```
financial_data_dupont <- financial_data |>
  group_by(firm_ID) |>
  arrange(year, .by_group = TRUE) |>
  mutate(
    lagged_NOA = lag(NOA),
    lagged_NFO = lag(NFO),
    RNOA = divide_safely_LFL(OX, lagged_NOA),
    PM = divide_safely_LFL(OX, sales),
    ATO = divide_safely_LFL(sales, lagged_NOA),
    lagged_FLEV = divide_safely_LFL(lagged_NFO, lagged_BE),
    NBC = divide_safely_LFL(NFE, lagged_NFO),
    ROE_DuPont = RNOA + lagged_FLEV * (RNOA - NBC),
    DuPont_gap = ROE - ROE_DuPont
  ) |>
  ungroup()

financial_data_dupont |>
  select(
    firm_ID,
    year,
    ROE,
    RNOA,
    PM,
    ATO,
    lagged_FLEV,
    NBC,
    ROE_DuPont,
    DuPont_gap
  )
```

```

) |>
slice_head(n = 10) |>
show_table_LFL(n = 10)
#> # A tibble: 10 x 10
#>   fir_ID year      ROE      RNOA      PM      ATO lag_FLEV      NBC
#>   <fct> <int>    <dbl>    <dbl>    <dbl>    <dbl>    <dbl>    <dbl>
#> 1 1      2015 NA      NA      0.083151 NA      NA      NA
#> 2 1      2016 0.052962 0.065350 0.094830 0.68913 -0.10961 -0.047678
#> 3 1      2017 0.066073 0.074071 0.10625 0.69712 -0.068152 -0.043289
#> 4 1      2018 0.063762 0.084974 0.10974 0.77436 -0.15202 -0.054567
#> 5 1      2019 0.060919 0.070062 0.12658 0.55348 0.26187 0.10498
#> 6 1      2020 0.076193 0.061279 0.10326 0.59342 0.14692 -0.040231
#> 7 2      2015 NA      NA      0.013070 NA      NA      NA
#> 8 2      2016 0.046800 0.045056 0.014679 3.0693 0.078263 0.022729
#> 9 2      2017 0.070174 0.058935 0.021145 2.7872 0.30884 0.022536
#> 10 2      2018 0.076295 0.064999 0.022564 2.8807 0.26625 0.022561
#> # i 2 more variables: ROE_DuPon <dbl>, DuP_gap <dbl>

```

デュポン分解で再構成した ROE と、直接計算した ROE の差を確認する。

```

dupont_validation <- financial_data_dupont |>
  summarise(
    comparable_observations = sum(!is.na(DuPont_gap)),
    mean_absolute_gap = mean(abs(DuPont_gap), na.rm = TRUE),
    maximum_absolute_gap = max(abs(DuPont_gap), na.rm = TRUE)
  )

show_table_LFL(dupont_validation)
#> # A tibble: 1 x 3
#>   com_obser      mea_gap      max_gap
#>   <int>    <dbl>    <dbl>
#> 1     6405 0.00000036675 0.000020751

```

営業利益率 PM と純営業資産回転率 ATO は、企業の事業モデルを異なる側面から表す。産業別中央値を使って両者の関係を確認する。

```

industry_pm_ato <- financial_data_dupont |>
  group_by(industry_ID) |>
  summarise(
    observations = sum(complete.cases(PM, ATO)),
    median_PM = median(PM, na.rm = TRUE),
    median_ATO = median(ATO, na.rm = TRUE),
    median_RNOA = median(RNOA, na.rm = TRUE),
    .groups = "drop"
  )

```

```

) |>
filter(
  observations >= 50,
  is.finite(median_PM),
  is.finite(median_ATO)
)

show_table_LFL(industry_pm_ato, n = 20)
#> # A tibble: 10 x 5
#>   ind_ID observations    med_PM med_ATO med_RNOA
#>   <fct>          <int>    <dbl>  <dbl>   <dbl>
#> 1 1          918 0.025331  3.9378 0.083905
#> 2 2          186 0.037769  2.4106 0.10883
#> 3 3         1433 0.048140  1.8342 0.089713
#> 4 4          138 0.040086  1.7797 0.075919
#> 5 5          259 0.034620  2.5099 0.097564
#> 6 6          127 0.048194  1.2994 0.074109
#> 7 7         1076 0.031837  2.5163 0.080634
#> 8 8          349 0.074662  1.0662 0.063506
#> 9 9          531 0.042643  2.1790 0.098533
#> 10 10        1388 0.044615  1.9604 0.090718

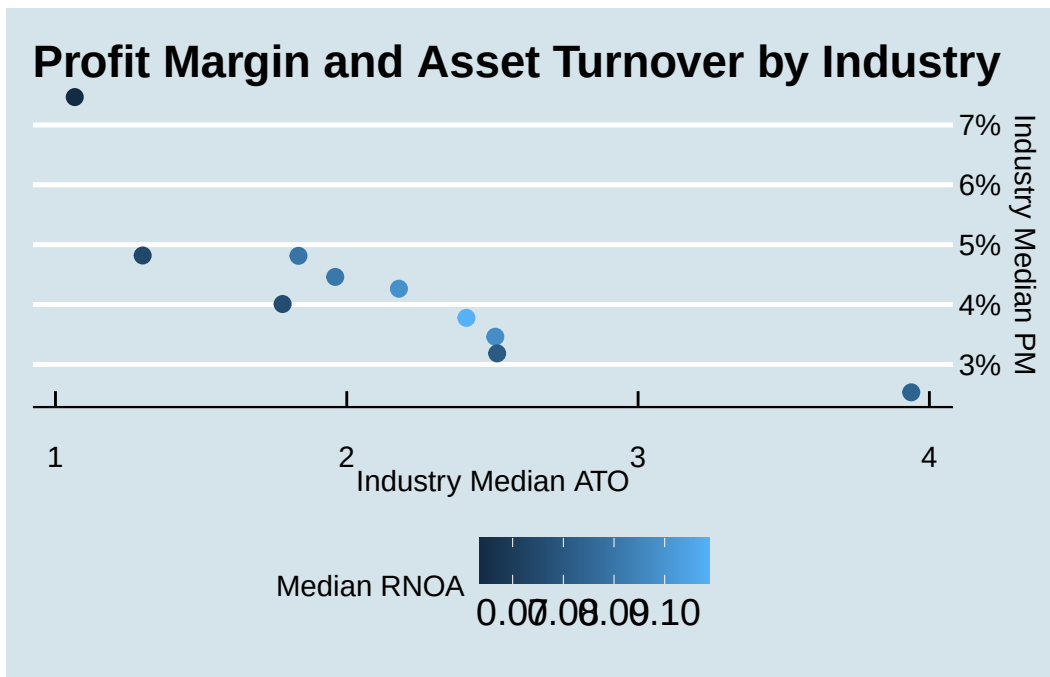
```

```

p_pm_ato_tradeoff <- industry_pm_ato |>
ggplot(
  aes(
    x = median_ATO,
    y = median_PM,
    colour = median_RNOA
  )
) +
geom_point(size = 2.5) +
labs(
  x = "Industry Median ATO",
  y = "Industry Median PM",
  colour = "Median RNOA",
  title = "Profit Margin and Asset Turnover by Industry"
)

add_lagrange_theme_LFL(p_pm_ato_tradeoff) +
  scale_y_continuous(
    position = "right",
    labels = scales::label_percent()
  )

```



主要産業について PM の分布を箱ひげ図で比較する。

```
major_industries <- firms_by_industry |>
  slice_head(n = 8) |>
  pull(industry_ID)

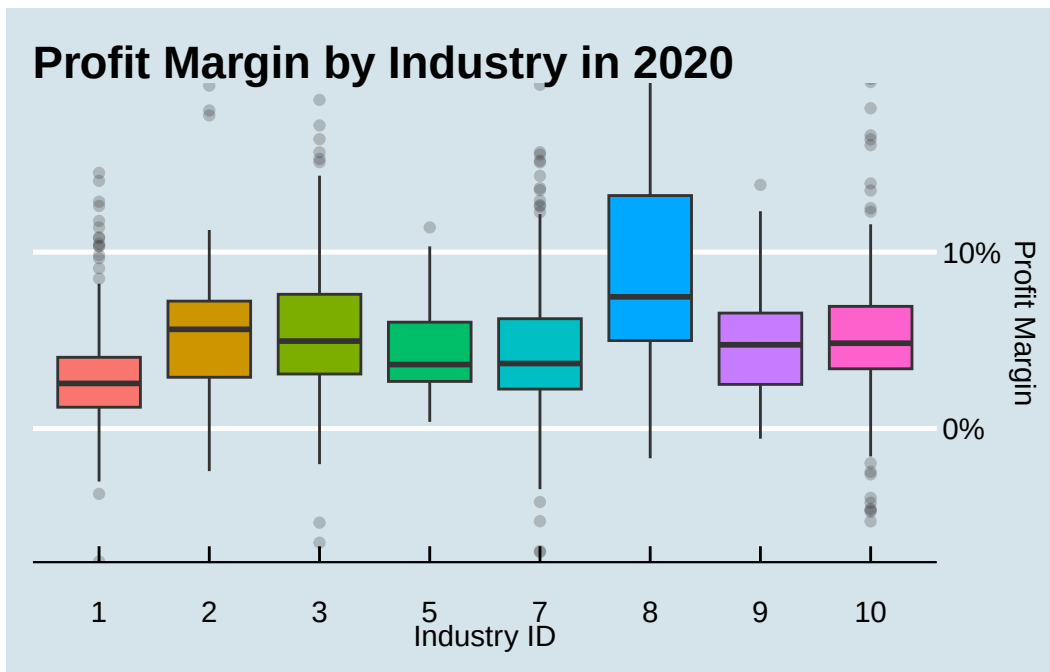
pm_limits <- quantile(
  financial_data_dupont$PM,
  probs = c(0.01, 0.99),
  na.rm = TRUE
)

p_pm_by_industry <- financial_data_dupont |>
  filter(
    year == latest_year,
    industry_ID %in% major_industries,
    !is.na(PM)
  ) |>
  ggplot(
    aes(
      x = industry_ID,
      y = PM,
      fill = industry_ID
    )
  ) +
  geom_boxplot(
    outlier.alpha = 0.25,
    show.legend = FALSE
  ) +
  coord_cartesian(ylim = pm_limits) +
```



```
labs(
  x = "Industry ID",
  y = "Profit Margin",
  title = paste(
    "Profit Margin by Industry in",
    latest_year
  )
)

add_lagrange_theme_LFL(p_pm_by_industry) +
  scale_y_continuous(
    position = "right",
    labels = scales::label_percent()
  )
```



I-2.6 財務指標の要約と中間データ

複数列に同じ集計を適用する場合は、列名のベクトルと `purrr::map_dfr()` を組み合わせる。

```
financial_metrics <- c(
  "sales",
  "BE",
  "ROE",
  "RNOA",
  "PM",
  "ATO"
)
```

```
metric_summary <- financial_metrics |>
  map_dfr(
    \(metric_name) {
      values <- financial_data_dupont[[metric_name]]
      finite_values <- values[is.finite(values)]

      tibble(
        metric = metric_name,
        observations = length(finite_values),
        mean = mean(finite_values),
        median = median(finite_values),
        standard_deviation = sd(finite_values),
        minimum = min(finite_values),
        maximum = max(finite_values)
      )
    }
  )

show_table_LFL(metric_summary)
#> # A tibble: 6 x 7
#>   metric observations      mean      median sta_devia  minimum  maximum
#>   <chr>          <int>    <dbl>    <dbl>    <dbl>    <dbl>    <dbl>
#> 1 sales            7920 166007.    40431.    3.8198e+5  205.34    3.4964e+6
#> 2 BE              7920 111517.    26882.    4.2062e+5  136.93    2.5448e+7
#> 3 ROE             6405   0.077713   0.072828  6.6274e-2  -0.70192  1.4055e+0
#> 4 RNOA            6405   0.11141   0.085868  1.2622e-1  -0.82036  1.8283e+0
#> 5 PM              7920   0.045534   0.040555  4.2223e-2  -0.20079  3.8289e-1
#> 6 ATO             6405   2.7051    2.1024    1.8684e+0   0.25825  9.8170e+0
```

第 3 章では、本章で計算した株主資本、前年度株主資本および ROE などを株式データと結合する。そのため、上級デュポン・モデルの計算結果を含む整理済みデータを `data/derived/fundamentals/financial_characteristics.parquet` へ保存する。

```
financial_output_path <-
  write_derived_parquet_file_LFL(
    financial_data_dupont,
    file.path(
      "fundamentals",
      "financial_characteristics.parquet"
    )
  )

tbl_financial_output <- tibble::tibble(
  output_file = relative_path_LFL(financial_output_path),
```

```

    size = base::file.info(financial_output_path)$size,
    modified = base::file.info(financial_output_path)$mtime
  )

show_table_LFL(tbl_financial_output)
#> # A tibble: 1 x 3
#>   out_file                                size modified
#>   <chr>                                <dbl> <dtm>
#> 1 data/derived/fundamentals/financial_characteristi~ 1357543 2026-09-25 09:27:59

```

保存した Parquet ファイルを読み直し、行数と列名が一致することを確認する。

```

financial_output_check <-
  read_derived_parquet_file_LFL(
    file.path(
      "fundamentals",
      "financial_characteristics.parquet"
    )
  )

stopifnot(
  nrow(financial_output_check) ==
    nrow(financial_data_dupont)
)

stopifnot(
  identical(
    names(financial_output_check),
    names(financial_data_dupont)
  )
)

output_summary <- tibble(
  output_file = relative_path_LFL(financial_output_path),
  rows = nrow(financial_output_check),
  columns = ncol(financial_output_check)
)

show_table_LFL(output_summary)
#> # A tibble: 1 x 3
#>   out_file                                rows columns
#>   <chr>                                <int>   <int>
#> 1 data/derived/fundamentals/financial_characteristics.parquet 7920     25

```

I-2.7 まとめ

本章では、財務データの構造、欠損値およびパネルデータのキーを確認した。次に、年度別・産業別の企業構成、売上高の分布、株主資本、ROE および産業内順位を計算した。最後に、ROE を RNOA、財務レバレッジおよび純借入コストへ分解し、第 3 章で利用する整理済みデータを `data/derived/fundamentals/financial_characteristics.parquet` へ保存した。

第 3 章では、株式データからリターンを計算し、本章で作成した財務データと結合する。

第 I-3 章株式データと時系列分析

本章では、月次株式データから株式リターン、超過リターンおよび時価総額を作成し、第 2 章で整理した年次財務データと結合する。さらに、月次リターンの分布、累積リターン、年次リターンおよび簿価時価比率との関係を確認し、第 4 章以降で利用する分析用データを保存する。

I-3.1 分析環境

I-3.2 株式データの読み込みと整備

```
stock_data_raw <-
read_parquet_file_LFL(
  file.path(
    "raw",
    "market",
    "stock_prices_and_returns.parquet"
  )
)

show_table_LFL(stock_data_raw, n = 10)
```

```
#> # A tibble: 95,040 x 9
#>   year month mon_ID fir_ID sto_price  DPS sha_outst adj_coeff      R_F
#> * <dbl> <dbl> <dbl> <dbl>    <dbl> <dbl>    <dbl>    <dbl>    <dbl>
#> 1  2015     1     1     1      954     0  2422000      1 0.00065068
#> 2  2015     2     2     1      960     0  2422000      1 0.00058341
#> 3  2015     3     3     1     1113     0  2422000      1 0.00061144
#> 4  2015     4     4     1     1081     0  2422000      1 0.00068482
#> 5  2015     5     5     1     1317     0  2422000      1 0.00073677
#> 6  2015     6     6     1     1366    29  2422000      1 0.00069540
#> 7  2015     7     7     1     1353     0  2422000      1 0.00049354
#> 8  2015     8     8     1     1209     0  2422000      1 0.00047106
#> 9  2015     9     9     1     1291     0  2422000      1 0.00055242
#> 10 2015    10    10     1     1407     0  2422000      1 0.00063803
#> # i 95,030 more rows
```

本章の計算に必要な列がすべて存在することを確認する。

```
required_stock_columns <- c(
  "year",
  "month",
  "month_ID",
  "firm_ID",
  "stock_price",
  "shares_outstanding",
  "DPS",
  "adjustment_coefficient",
  "R_F"
)

missing_stock_columns <-
  setdiff(required_stock_columns, names(stock_data_raw))

if (length(missing_stock_columns) > 0) {
  stop(
    "株式データに必要な列が不足しています: ",
    paste(missing_stock_columns, collapse = ", "),
    call. = FALSE
  )
}
```

```
show_data_structure_LFL(stock_data_raw)
#> Rows: 95,040
#> Columns: 9
#> $ year          <dbl> 2015, 2015, 2015, 2015, 2015, 2015, 2015, 2015, ~
#> $ month         <dbl> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 1, 2, 3, ~
#> $ month_ID      <dbl> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, ~
#> $ firm_ID       <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ~
#> $ stock_price   <dbl> 954, 960, 1113, 1081, 1317, 1366, 1353, 1209, 1~
#> $ DPS           <dbl> 0, 0, 0, 0, 0, 29, 0, 0, 0, 0, 0, 29, 0, 0, 0, ~
#> $ shares_outstanding <dbl> 2422000, 2422000, 2422000, 2422000, 2422000, 24~
#> $ adjustment_coefficient <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ~
#> $ R_F           <dbl> 0.00065068255, 0.00058340985, 0.00061144232, 0.~

stock_data_overview <- stock_data_raw |>
  summarise(
    observations = n(),
    variables = ncol(stock_data_raw),
    firms = n_distinct(firm_ID),
    first_year = min(year, na.rm = TRUE),
    last_year = max(year, na.rm = TRUE),
```

```

    first_month_ID = min(month_ID, na.rm = TRUE),
    last_month_ID = max(month_ID, na.rm = TRUE)
  )

```

```
show_table_LFL(stock_data_overview)
```

```
#> # A tibble: 1 x 7
```

```

#>   observations variables firms fir_year las_year fir_ID las_ID
#>   <int>         <int> <int>   <dbl>    <dbl>  <dbl>  <dbl>
#> 1      95040           9  1515    2015     2020      1    72

```

```
stock_missing_summary <- stock_data_raw |>
```

```

  summarise(
    across(
      everything(),
      list(
        missing = \(x) sum(is.na(x)),
        missing_rate = \(x) mean(is.na(x))
      ),
      .names = "{.col}__{.fn}"
    )
  ) |>

```

```
  ) |>
```

```

  pivot_longer(
    cols = everything(),
    names_to = c("variable", ".value"),
    names_sep = "__"
  ) |>

```

```

  arrange(desc(missing_rate), variable)

```

```
show_table_LFL(
```

```
  stock_missing_summary,
```

```
  n = nrow(stock_missing_summary)
```

```
)
```

```
#> # A tibble: 9 x 3
```

```

#>   variable                missing mis_rate
#>   <chr>                 <int>    <dbl>
#> 1 DPS                    0         0
#> 2 R_F                    0         0
#> 3 adjustment_coefficient  0         0
#> 4 firm_ID                0         0
#> 5 month                  0         0
#> 6 month_ID              0         0
#> 7 shares_outstanding     0         0
#> 8 stock_price            0         0
#> 9 year                   0         0

```

企業ごとの時系列計算を正しく行うため、企業 ID と月 ID の順に並べる。

```
stock_data <- stock_data_raw |>
  mutate(
    firm_ID = as.integer(firm_ID),
    year = as.integer(year),
    month = as.integer(month),
    month_ID = as.integer(month_ID),
    month_date = lubridate::make_date(year, month, 1)
  ) |>
  arrange(firm_ID, month_ID)

show_data_structure_LFL(stock_data)
#> Rows: 95,040
#> Columns: 10
#> $ year          <int> 2015, 2015, 2015, 2015, 2015, 2015, 2015, 2015, ~
#> $ month         <int> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 1, 2, 3, ~
#> $ month_ID      <int> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, ~
#> $ firm_ID       <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ~
#> $ stock_price   <dbl> 954, 960, 1113, 1081, 1317, 1366, 1353, 1209, 1~
#> $ DPS           <dbl> 0, 0, 0, 0, 0, 29, 0, 0, 0, 0, 0, 29, 0, 0, 0, ~
#> $ shares_outstanding <dbl> 2422000, 2422000, 2422000, 2422000, 2422000, 24~
#> $ adjustment_coefficient <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ~
#> $ R_F           <dbl> 0.00065068255, 0.00058340985, 0.00061144232, 0.~
#> $ month_date    <date> 2015-01-01, 2015-02-01, 2015-03-01, 2015-04-01~
```

firm_ID と month_ID の組合せが一意であることを確認する。

```
stock_key_duplicates <- stock_data |>
  count(
    firm_ID,
    month_ID,
    name = "observations"
  ) |>
  filter(observations > 1)

show_table_LFL(stock_key_duplicates)
#> # A tibble: 0 x 3
#> # i 3 variables: fir_ID <int>, mon_ID <int>, observations <int>
```

```
if (nrow(stock_key_duplicates) > 0) {
  stop(
    "firm_IDとmonth_IDの組合せに重複があります。",
    call. = FALSE
  )
}
```



```
}
```

同じ月 ID に複数の年月が対応していないことも確認する。

```
month_calendar_check <- stock_data |>
  distinct(
    month_ID,
    year,
    month,
    month_date
  ) |>
  count(
    month_ID,
    name = "calendar_rows"
  ) |>
  filter(calendar_rows > 1)

show_table_LFL(month_calendar_check)
#> # A tibble: 0 x 2
#> # i 2 variables: mon_ID <int>, cal_rows <int>

if (nrow(month_calendar_check) > 0) {
  stop(
    "同じmonth_IDに複数の年月が対応しています。",
    call. = FALSE
  )
}
```

年度別の企業数と観測数を確認する。

```
stock_observations_by_year <- stock_data |>
  group_by(year) |>
  summarise(
    firms = n_distinct(firm_ID),
    observations = n(),
    first_month_ID = min(month_ID),
    last_month_ID = max(month_ID),
    .groups = "drop"
  ) |>
  arrange(year)

show_table_LFL(
  stock_observations_by_year,
  n = nrow(stock_observations_by_year)
)
```

```
#> # A tibble: 6 x 5
#>   year firms observations fir_ID las_ID
#>   <int> <int>         <int> <int> <int>
#> 1  2015  1266         15192     1    12
#> 2  2016  1293         15516    13    24
#> 3  2017  1319         15828    25    36
#> 4  2018  1323         15876    37    48
#> 5  2019  1356         16272    49    60
#> 6  2020  1363         16356    61    72
```

I-3.3 月次リターンと時価総額

配当が支払われた観測と、調整係数が 1 と異なる観測を確認する。

```
dividend_observations <- stock_data |>
  filter(
    !is.na(DPS),
    DPS != 0
  ) |>
  select(
    firm_ID,
    year,
    month,
    month_ID,
    stock_price,
    DPS,
    adjustment_coefficient
  ) |>
  arrange(firm_ID, month_ID)

adjustment_observations <- stock_data |>
  filter(
    !is.na(adjustment_coefficient),
    !near(adjustment_coefficient, 1)
  ) |>
  select(
    firm_ID,
    year,
    month,
    month_ID,
    stock_price,
    DPS,
    adjustment_coefficient
```

```

) |>
  arrange(firm_ID, month_ID)

corporate_action_summary <- stock_data |>
  summarise(
    observations = n(),
    dividend_observations =
      sum(!is.na(DPS) & DPS != 0),
    adjusted_observations =
      sum(
        !is.na(adjustment_coefficient) &
        !near(adjustment_coefficient, 1)
      )
  )

show_table_LFL(dividend_observations, n = 20)
#> # A tibble: 14,250 x 7
#>   fir_ID  year month mon_ID sto_price  DPS adj_coeff
#>   <int> <int> <int>  <int>    <dbl> <dbl>    <dbl>
#> 1     1    2015     6     6    1366    29      1
#> 2     1    2015    12    12    1477    29      1
#> 3     1    2016     6    18    1696    40      1
#> 4     1    2016    12    24    2842    40      1
#> 5     1    2017     6    30    4317    43      1
#> 6     1    2017    12    36    3915    43      1
#> 7     1    2018     6    42    2986    43      1
#> 8     1    2018    12    48    2992    43      1
#> 9     1    2019     6    54    3608    53      1
#> 10    1    2019    12    60    4803    53      1
#> 11    1    2020     6    66    4698    56      1
#> 12    1    2020    12    72    3341    56      1
#> 13     2    2015     6     6    2167     3      1
#> 14     2    2015    12    12    2613     3      1
#> 15     2    2016     6    18    2395     7      1
#> 16     2    2016    12    24    3576     7      1
#> 17     2    2017     6    30    4966     7      1
#> 18     2    2017    12    36    5852     7      1
#> 19     2    2018     6    42    4415    12      1
#> 20     2    2018    12    48    4542    12      1
#> # i 14,230 more rows
show_table_LFL(adjustment_observations, n = 20)
#> # A tibble: 267 x 7
#>   fir_ID  year month mon_ID sto_price  DPS adj_coeff
#>   <int> <int> <int>  <int>    <dbl> <dbl>    <dbl>
#> 1     1    2017     4    28    4082     0     1.2

```

```

#> 2      9 2015      4      4      2655      0      1.2
#> 3     10 2015      2      2       757      0      1.2
#> 4     21 2019     12     60      3008     19      1.1
#> 5     27 2019      9     57     13750      0      0.1
#> 6     42 2016      5     17     47324      0      0.1
#> 7     48 2019      2     50     13232      0      0.1
#> 8     53 2019      4     52      1754      0      1.1
#> 9     54 2019      2     50      4015      0      1.1
#> 10    60 2019     12     60      1749      0      1.1
#> 11    66 2019     11     59     10140      0      0.1
#> 12    67 2018      1     37      2020      0      1.1
#> 13    72 2016      5     17      1876      0      1.1
#> 14    74 2017      6     30      1402     11      2
#> 15    80 2018      3     39      1393      0      1.2
#> 16    84 2017      4     28     10697      0      0.1
#> 17    90 2020      2     62    153396      0      0.1
#> 18    96 2015      4      4       805      0      1.2
#> 19   103 2020      6     66     44488     468      0.1
#> 20   104 2016      1     13      9191      0      0.1
#> # i 247 more rows
show_table_LFL(corporate_action_summary)
#> # A tibble: 1 x 3
#>   observations div_obser adj_obser
#>   <int>      <int>    <int>
#> 1    95040    14250     267

```

企業 i の月 t における時価総額 $ME_{i,t}$ は、株価 $P_{i,t}$ と発行済株式数 $N_{i,t}$ の積である。

$$ME_{i,t} = P_{i,t} N_{i,t}$$

```

stock_data <- stock_data |>
  mutate(
    ME = stock_price * shares_outstanding,
    ME = if_else(
      is.finite(ME) & ME > 0,
      ME,
      NA_real_
    )
  )

stock_data |>
  select(
    firm_ID,

```

```

    month_date,
    stock_price,
    shares_outstanding,
    ME
  ) |>
  slice_head(n = 10) |>
  show_table_LFL(n = 10)
#> # A tibble: 10 x 5
#>   fir_ID mon_date  sto_price sha_outst      ME
#>   <int> <date>      <dbl>    <dbl>    <dbl>
#> 1     1 2015-01-01      954    2422000 2310588000
#> 2     1 2015-02-01      960    2422000 2325120000
#> 3     1 2015-03-01     1113    2422000 2695686000
#> 4     1 2015-04-01     1081    2422000 2618182000
#> 5     1 2015-05-01     1317    2422000 3189774000
#> 6     1 2015-06-01     1366    2422000 3308452000
#> 7     1 2015-07-01     1353    2422000 3276966000
#> 8     1 2015-08-01     1209    2422000 2928198000
#> 9     1 2015-09-01     1291    2422000 3126802000
#> 10    1 2015-10-01     1407    2422000 3407754000

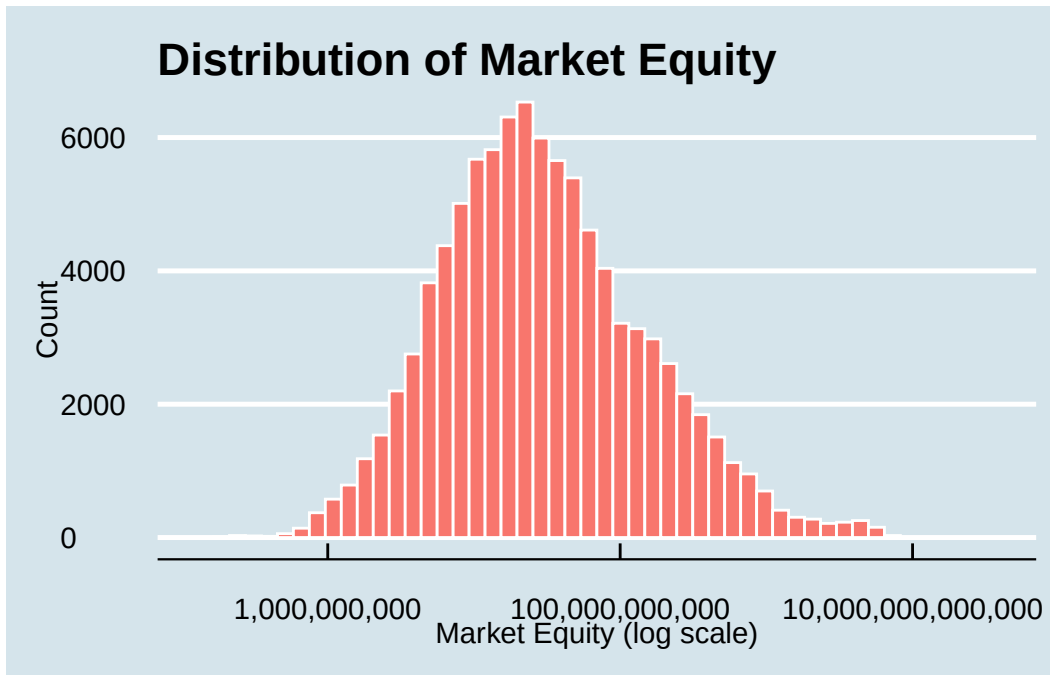
```

```

p_market_equity <- stock_data |>
  filter(
    is.finite(ME),
    ME > 0
  ) |>
  ggplot(aes(x = ME, fill = "Market Equity")) +
  geom_histogram(
    bins = 50,
    colour = "white",
    show.legend = FALSE
  ) +
  scale_x_log10(
    labels = scales::label_comma()
  ) +
  labs(
    x = "Market Equity (log scale)",
    y = "Count",
    title = "Distribution of Market Equity"
  )

add_lagrange_theme_LFL(p_market_equity)

```



企業ごとに前月株価を作成し、月 ID が連続している場合だけ利用する。

```
stock_data <- stock_data |>
  group_by(firm_ID) |>
  arrange(month_ID, .by_group = TRUE) |>
  mutate(
    lagged_stock_price = lag(stock_price),
    lagged_month_ID = lag(month_ID),
    consecutive_month =
      month_ID - lagged_month_ID == 1L,
    lagged_stock_price = if_else(
      consecutive_month,
      lagged_stock_price,
      NA_real_
    )
  ) |>
  ungroup()
```

月次株式リターン $R_{i,t}$ を、株価、配当および調整係数から計算する。

$$R_{i,t} = \frac{(P_{i,t} + D_{i,t})A_{i,t} - P_{i,t-1}}{P_{i,t-1}}$$

```
stock_data <- stock_data |>
  mutate(
    R = case_when(
      is.na(lagged_stock_price) ~ NA_real_,
      lagged_stock_price <= 0 ~ NA_real_,
      is.na(stock_price) ~ NA_real_,
```

```

is.na(DPS) ~ NA_real_,
is.na(adjustment_coefficient) ~ NA_real_,
TRUE ~ (
  (stock_price + DPS) *
  adjustment_coefficient -
  lagged_stock_price
) / lagged_stock_price
),
R = if_else(
  is.finite(R),
  R,
  NA_real_
)
)

```

```

stock_data |>
  select(
    firm_ID,
    month_date,
    lagged_stock_price,
    stock_price,
    DPS,
    adjustment_coefficient,
    R
  ) |>
  filter(!is.na(R)) |>
  slice_head(n = 15) |>
  show_table_LFL(n = 15)

```

```
#> # A tibble: 15 x 7
```

#>	fir_ID	mon_date	lag_price	sto_price	DPS	adj_coeff	R
#>	<int>	<date>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
#> 1	1	2015-02-01	954	960	0	1	0.0062893
#> 2	1	2015-03-01	960	1113	0	1	0.15937
#> 3	1	2015-04-01	1113	1081	0	1	-0.028751
#> 4	1	2015-05-01	1081	1317	0	1	0.21832
#> 5	1	2015-06-01	1317	1366	29	1	0.059226
#> 6	1	2015-07-01	1366	1353	0	1	-0.0095168
#> 7	1	2015-08-01	1353	1209	0	1	-0.10643
#> 8	1	2015-09-01	1209	1291	0	1	0.067825
#> 9	1	2015-10-01	1291	1407	0	1	0.089853
#> 10	1	2015-11-01	1407	1562	0	1	0.11016
#> 11	1	2015-12-01	1562	1477	29	1	-0.035851
#> 12	1	2016-01-01	1477	1561	0	1	0.056872
#> 13	1	2016-02-01	1561	1487	0	1	-0.047406
#> 14	1	2016-03-01	1487	1526	0	1	0.026227

```
#> 15      1 2016-04-01      1526      1542      0      1 0.010485
```

同じ月の無リスク金利はすべての企業に共通であることを確認し、月ごとの中央値で統一する。

```
risk_free_rate_by_month <- stock_data |>
  group_by(
    month_ID,
    year,
    month,
    month_date
  ) |>
  summarise(
    observations = n(),
    valid_rates = sum(is.finite(R_F)),
    distinct_rates =
      n_distinct(R_F[is.finite(R_F)]),
    R_F_month = {
      values <- R_F[is.finite(R_F)]

      if (length(values) == 0) {
        NA_real_
      } else {
        median(values)
      }
    },
    .groups = "drop"
  ) |>
  arrange(month_ID)

risk_free_rate_diagnostics <-
  risk_free_rate_by_month |>
  filter(distinct_rates > 1)

show_table_LFL(
  risk_free_rate_diagnostics,
  n = nrow(risk_free_rate_diagnostics)
)

#> # A tibble: 72 x 8
#>   mon_ID year month mon_date observations val_rates dis_rates R_month
#>   <int> <int> <int> <date>          <int>      <int>      <int>      <dbl>
#> 1     1   2015     1 2015-01-01         1266        1266         83 0.000082029
#> 2     2   2015     2 2015-02-01         1266        1266         83 0.000082029
#> 3     3   2015     3 2015-03-01         1266        1266         83 0.000082029
#> 4     4   2015     4 2015-04-01         1266        1266         83 0.000082029
#> 5     5   2015     5 2015-05-01         1266        1266         83 0.000082029
```


#>	6	6	2015	6 2015-06-01	1266	1266	83 0.000082029
#>	7	7	2015	7 2015-07-01	1266	1266	83 0.000082029
#>	8	8	2015	8 2015-08-01	1266	1266	83 0.000082029
#>	9	9	2015	9 2015-09-01	1266	1266	83 0.000082029
#>	10	10	2015	10 2015-10-01	1266	1266	83 0.000082029
#>	11	11	2015	11 2015-11-01	1266	1266	83 0.000082029
#>	12	12	2015	12 2015-12-01	1266	1266	83 0.000082029
#>	13	13	2016	1 2016-01-01	1293	1293	83 0.000082029
#>	14	14	2016	2 2016-02-01	1293	1293	83 0.000075959
#>	15	15	2016	3 2016-03-01	1293	1293	83 0.000075959
#>	16	16	2016	4 2016-04-01	1293	1293	83 0.000082029
#>	17	17	2016	5 2016-05-01	1293	1293	83 0.000082029
#>	18	18	2016	6 2016-06-01	1293	1293	83 0.000082029
#>	19	19	2016	7 2016-07-01	1293	1293	83 0.000082029
#>	20	20	2016	8 2016-08-01	1293	1293	83 0.000082029
#>	21	21	2016	9 2016-09-01	1293	1293	83 0.000082029
#>	22	22	2016	10 2016-10-01	1293	1293	83 0.000075959
#>	23	23	2016	11 2016-11-01	1293	1293	83 0.000075959
#>	24	24	2016	12 2016-12-01	1293	1293	83 0.000075959
#>	25	25	2017	1 2017-01-01	1319	1319	83 0.000075959
#>	26	26	2017	2 2017-02-01	1319	1319	83 0.000075959
#>	27	27	2017	3 2017-03-01	1319	1319	83 0.000075959
#>	28	28	2017	4 2017-04-01	1319	1319	83 0.000082029
#>	29	29	2017	5 2017-05-01	1319	1319	83 0.000075959
#>	30	30	2017	6 2017-06-01	1319	1319	83 0.000075959
#>	31	31	2017	7 2017-07-01	1319	1319	83 0.000075959
#>	32	32	2017	8 2017-08-01	1319	1319	83 0.000075959
#>	33	33	2017	9 2017-09-01	1319	1319	83 0.000075959
#>	34	34	2017	10 2017-10-01	1319	1319	83 0.000075959
#>	35	35	2017	11 2017-11-01	1319	1319	83 0.000075959
#>	36	36	2017	12 2017-12-01	1319	1319	83 0.000075959
#>	37	37	2018	1 2018-01-01	1323	1323	83 0.000075959
#>	38	38	2018	2 2018-02-01	1323	1323	83 0.000075959
#>	39	39	2018	3 2018-03-01	1323	1323	83 0.000082029
#>	40	40	2018	4 2018-04-01	1323	1323	83 0.000082029
#>	41	41	2018	5 2018-05-01	1323	1323	83 0.000082029
#>	42	42	2018	6 2018-06-01	1323	1323	83 0.000075959
#>	43	43	2018	7 2018-07-01	1323	1323	83 0.000075959
#>	44	44	2018	8 2018-08-01	1323	1323	83 0.000082029
#>	45	45	2018	9 2018-09-01	1323	1323	83 0.000082029
#>	46	46	2018	10 2018-10-01	1323	1323	83 0.000082029
#>	47	47	2018	11 2018-11-01	1323	1323	83 0.000082029
#>	48	48	2018	12 2018-12-01	1323	1323	83 0.000082029
#>	49	49	2019	1 2019-01-01	1356	1356	83 0.000075959
#>	50	50	2019	2 2019-02-01	1356	1356	83 0.000078994

```

#> 51      51  2019      3 2019-03-01      1356      1356      83 0.000082029
#> 52      52  2019      4 2019-04-01      1356      1356      83 0.000082029
#> 53      53  2019      5 2019-05-01      1356      1356      83 0.000082029
#> 54      54  2019      6 2019-06-01      1356      1356      83 0.000075959
#> 55      55  2019      7 2019-07-01      1356      1356      83 0.000082029
#> 56      56  2019      8 2019-08-01      1356      1356      83 0.000082029
#> 57      57  2019      9 2019-09-01      1356      1356      83 0.000082029
#> 58      58  2019     10 2019-10-01      1356      1356      83 0.000075959
#> 59      59  2019     11 2019-11-01      1356      1356      83 0.000075959
#> 60      60  2019     12 2019-12-01      1356      1356      83 0.000075959
#> 61      61  2020      1 2020-01-01      1363      1363      83 0.000082029
#> 62      62  2020      2 2020-02-01      1363      1363      83 0.000082029
#> 63      63  2020      3 2020-03-01      1363      1363      83 0.000082029
#> 64      64  2020      4 2020-04-01      1363      1363      83 0.000075959
#> 65      65  2020      5 2020-05-01      1363      1363      83 0.000075959
#> 66      66  2020      6 2020-06-01      1363      1363      83 0.000075959
#> 67      67  2020      7 2020-07-01      1363      1363      83 0.000075959
#> 68      68  2020      8 2020-08-01      1363      1363      83 0.000075959
#> 69      69  2020      9 2020-09-01      1363      1363      83 0.000075959
#> 70      70  2020     10 2020-10-01      1363      1363      83 0.000082029
#> 71      71  2020     11 2020-11-01      1363      1363      83 0.000082029
#> 72      72  2020     12 2020-12-01      1363      1363      83 0.000082029

```

```

stock_data <- stock_data |>
  select(-R_F) |>
  left_join(
    risk_free_rate_by_month |>
      select(
        month_ID,
        R_F = R_F_month
      ),
    by = "month_ID"
  )

missing_monthly_risk_free_rate <- stock_data |>
  distinct(month_ID, R_F) |>
  filter(!is.finite(R_F))

if (nrow(missing_monthly_risk_free_rate) > 0) {
  stop(
    "月次無リスク金利を作成できないmonth_IDがあります。",
    call. = FALSE
  )
}

```

月次超過リターン $R_{i,t}^e$ は、株式リターンから無リスク金利を差し引いて計算する。

$$R_{i,t}^e = R_{i,t} - R_{F,t}$$

```
stock_data <- stock_data |>
  mutate(
    Re = R - R_F,
    Re = if_else(
      is.finite(Re),
      Re,
      NA_real_
    )
  )
```

I-3.4 月次リターンの分析

```
calculate_skewness_LFL <- function(x) {
  values <- x[is.finite(x)]

  if (
    length(values) < 3 ||
    near(sd(values), 0)
  ) {
    return(NA_real_)
  }

  mean(
    ((values - mean(values)) /
     sd(values))^3
  )
}

calculate_kurtosis_LFL <- function(x) {
  values <- x[is.finite(x)]

  if (
    length(values) < 4 ||
    near(sd(values), 0)
  ) {
    return(NA_real_)
  }
}
```

```

mean(
  ((values - mean(values)) /
    sd(values))^4
)
}

```

```

monthly_return_summary <- stock_data |>
  select(R, Re) |>
  pivot_longer(
    cols = everything(),
    names_to = "return_type",
    values_to = "return"
  ) |>
  filter(is.finite(return)) |>
  group_by(return_type) |>
  summarise(
    observations = n(),
    mean = mean(return),
    median = median(return),
    standard_deviation = sd(return),
    minimum = min(return),
    first_quartile = quantile(return, 0.25),
    third_quartile = quantile(return, 0.75),
    maximum = max(return),
    skewness = calculate_skewness_LFL(return),
    kurtosis = calculate_kurtosis_LFL(return),
    .groups = "drop"
  )

show_table_LFL(monthly_return_summary)
#> # A tibble: 2 x 11
#>   ret_type observations      mean  median sta_devia minimum fir_quart thi_quart
#>   <chr>          <int>    <dbl>   <dbl>    <dbl>    <dbl>    <dbl>    <dbl>
#> 1 R              93525 0.015859 0.010332 0.091131 -0.38028 -0.040573 0.064808
#> 2 Re             93525 0.015779 0.010252 0.091131 -0.38036 -0.040652 0.064730
#> # i 3 more variables: maximum <dbl>, skewness <dbl>, kurtosis <dbl>

```

極端値の影響を抑えるため、1 パーセンタイルから 99 パーセンタイルまでを表示する。

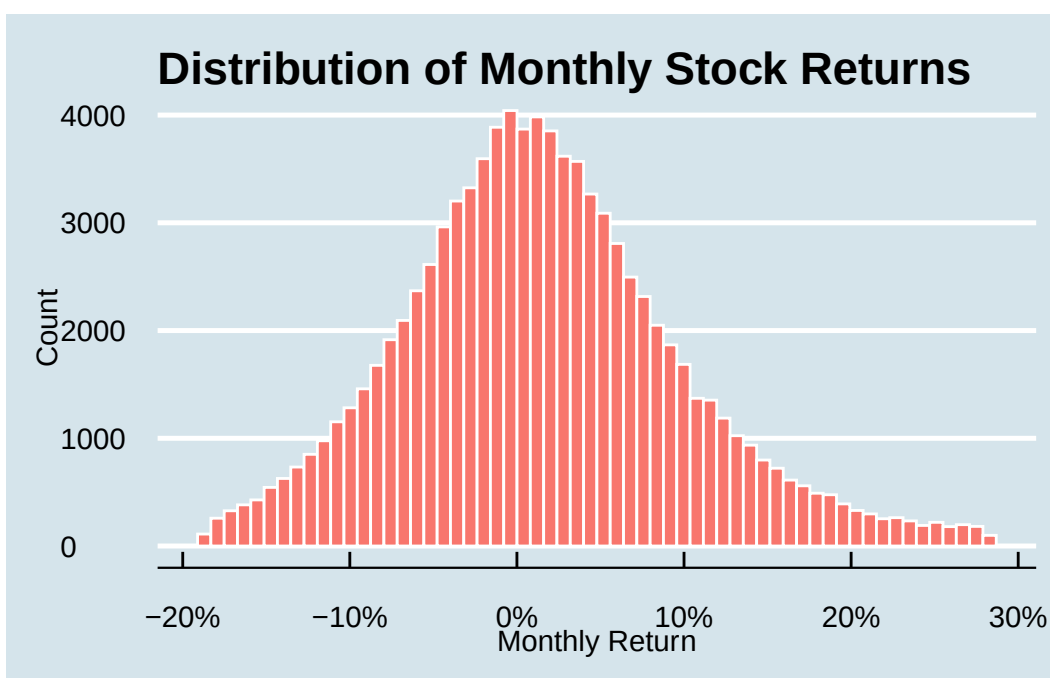
```

monthly_return_limits <- quantile(
  stock_data$R,
  probs = c(0.01, 0.99),
  na.rm = TRUE
)

```

```
p_monthly_return_distribution <- stock_data |>
  filter(
    is.finite(R),
    between(
      R,
      monthly_return_limits[[1]],
      monthly_return_limits[[2]]
    )
  ) |>
  ggplot(aes(x = R, fill = "Monthly Return")) +
  geom_histogram(
    bins = 60,
    colour = "white",
    show.legend = FALSE
  ) +
  scale_x_continuous(
    labels = scales::label_percent()
  ) +
  labs(
    x = "Monthly Return",
    y = "Count",
    title = "Distribution of Monthly Stock Returns"
  )

add_lagrange_theme_LFL(
  p_monthly_return_distribution
)
```



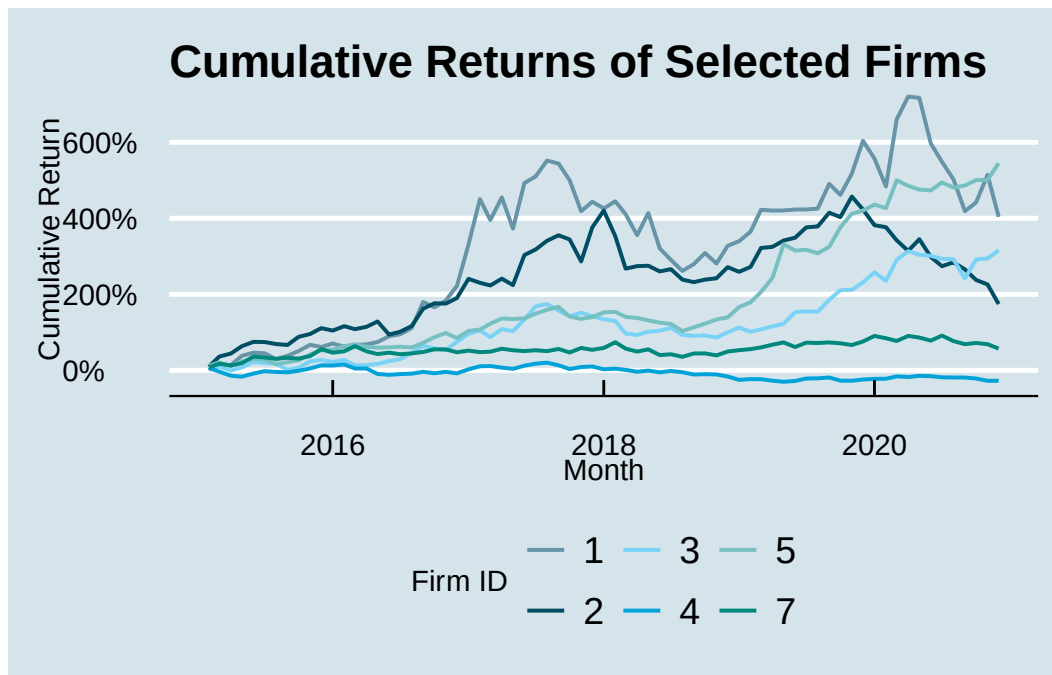
観測数が多い企業から 6 社を選び、月次リターンを複利で累積する。

```
sample_firm_ids <- stock_data |>
  filter(is.finite(R)) |>
  count(
    firm_ID,
    sort = TRUE
  ) |>
  slice_head(n = 6) |>
  pull(firm_ID)

cumulative_return_data <- stock_data |>
  filter(
    firm_ID %in% sample_firm_ids,
    is.finite(R)
  ) |>
  group_by(firm_ID) |>
  arrange(month_ID, .by_group = TRUE) |>
  mutate(
    cumulative_return = cumprod(1 + R) - 1
  ) |>
  ungroup()
```

```
p_cumulative_return <- cumulative_return_data |>
  ggplot(
    aes(
      x = month_date,
      y = cumulative_return,
      colour = factor(firm_ID)
    )
  ) +
  geom_line(linewidth = 0.7) +
  scale_y_continuous(
    labels = scales::label_percent()
  ) +
  labs(
    x = "Month",
    y = "Cumulative Return",
    colour = "Firm ID",
    title = "Cumulative Returns of Selected Firms"
  )

add_lagrange_theme_LFL(p_cumulative_return) +
  ggthemes::scale_colour_economist()
```



```

firm_return_summary <- sample_firm_ids |>
  map_dfr(
    \(firm_id) {
      firm_returns <- stock_data |>
        filter(
          firm_ID == firm_id,
          is.finite(R)
        ) |>
        pull(R)

      tibble(
        firm_ID = firm_id,
        observations = length(firm_returns),
        mean_return = mean(firm_returns),
        volatility = sd(firm_returns),
        minimum_return = min(firm_returns),
        maximum_return = max(firm_returns)
      )
    }
  )

show_table_LFL(firm_return_summary)
#> # A tibble: 6 x 6
#>   fir_ID observations  mea_retur volatility min_retur max_retur
#>   <int>      <int>      <dbl>      <dbl>      <dbl>      <dbl>
#> 1         1         71  0.029240    0.11544  -0.18157    0.33603
#> 2         2         71  0.018125    0.089135 -0.18825    0.24481
#> 3         3         71  0.023028    0.075369 -0.14460    0.16440
#> 4         4         71 -0.0030159  0.054766 -0.13816    0.11475

```

```
#> 5      5      71 0.028390 0.061602 -0.097372 0.25720
#> 6      7      71 0.0077876 0.053950 -0.096949 0.14549
```

I-3.5 年次データと財務データの結合

企業 i の年 t における年次リターンは、月次リターンを複利で集約する。

$$R_{i,t}^{annual} = \prod_{m=1}^{M_{i,t}} (1 + R_{i,t,m}) - 1$$

```
annual_stock_data <- stock_data |>
  group_by(
    firm_ID,
    year
  ) |>
  summarise(
    months = sum(is.finite(R)),
    R = if (
      sum(is.finite(R)) == 0
    ) {
      NA_real_
    } else {
      prod(1 + R[is.finite(R)]) - 1
    },
    Re = if (
      sum(is.finite(Re)) == 0
    ) {
      NA_real_
    } else {
      prod(1 + Re[is.finite(Re)]) - 1
    },
    ME = {
      values <- ME[is.finite(ME)]

      if (length(values) == 0) {
        NA_real_
      } else {
        dplyr::last(values)
      }
    },
    .groups = "drop"
  ) |>
```



```

arrange(firm_ID, year)

show_table_LFL(annual_stock_data, n = 20)
#> # A tibble: 7,920 x 6
#>   fir_ID year months      R      Re      ME
#>   <int> <int> <int>   <dbl>   <dbl>   <dbl>
#> 1     1   2015     11  0.61213  0.61073 3577294000
#> 2     1   2016     12  0.99727  0.99547 6883324000
#> 3     1   2017     12  0.68786  0.68637 11376990000
#> 4     1   2018     12 -0.21361 -0.21439 8694752000
#> 5     1   2019     12  0.64684  0.64533 13957518000
#> 6     1   2020     12 -0.28430 -0.28501 9708946000
#> 7     2   2015     11  1.1092   1.1074 4086732000
#> 8     2   2016     12  0.37523  0.37395 5592864000
#> 9     2   2017     12  0.64073  0.63928 9152528000
#> 10    2   2018     12 -0.21969 -0.22046 7103688000
#> 11    2   2019     12  0.40819  0.40688 9962680000
#> 12    2   2020     12 -0.47547 -0.47599 5194044000
#> 13    3   2015     11  0.27011  0.26898 70434234000
#> 14    3   2016     12  0.37526  0.37398 96064839000
#> 15    3   2017     12  0.38682  0.38558 132546843000
#> 16    3   2018     12 -0.17493 -0.17573 107515395000
#> 17    3   2019     12  0.65869  0.65717 174487833000
#> 18    3   2020     12  0.25382  0.25265 214032195000
#> 19    4   2015     11  0.12885  0.12784 14518674000
#> 20    4   2016     12 -0.18552 -0.18631 11592135000
#> # i 7,900 more rows

```

第 2 章で作成した `data/derived/fundamentals/financial_characteristics.parquet` を読み込む。

```

financial_data <-
  read_derived_parquet_file_LFL(
    file.path(
      "fundamentals",
      "financial_characteristics.parquet"
    )
  ) |>
  mutate(
    firm_ID = as.integer(firm_ID),
    year = as.integer(year)
  )

show_table_LFL(financial_data, n = 10)
#> # A tibble: 7,920 x 25

```

```

#>   year fir_ID ind_ID sales    OX    NFE    X    OA    FA    OL
#>   <int> <int> <fct>   <dbl> <dbl>   <dbl> <dbl>   <dbl> <dbl> <dbl>
#> 1  2015     1  1     5261.4 437.49  NA     286.64 13006.   3543.4 4373.0
#> 2  2016     1  1     5949.0 564.14  50.667  513.48 13866.   4642.2 4534.2
#> 3  2017     1  1     6505.1 691.18  29.543  661.64 13953.   7744.0 5111.2
#> 4  2018     1  1     6846.4 751.29  86.487  664.8  18818.   7284.7 5137.3
#> 5  2019     1  1     7572.2 958.53 298.05   660.48 18190    9735.1 5488.0
#> 6  2020     1  1     7537.6 778.37 -65.459  843.83 20463.   10274.  5371.4
#> 7  2015     2  1     3505.8 45.82   5.7511  40.07  2977.8   2258.3 1840.4
#> 8  2016     2  1     3491.3 51.25   1.8765  49.37  3184.8   1881.8 1769.2
#> 9  2017     2  1     3945.7 83.43   7.5279  75.9   3392.2   2425.2 1955.3
#> 10 2018     2  1     4139.3 93.4    6.8166  86.58  3569.3   2331.0 2022.1
#> # i 7,910 more rows
#> # i 15 more variables: FO <dbl>, NOA <dbl>, NFO <dbl>, BE <dbl>, lag_BE <dbl>,
#> #   ROE <dbl>, lag_NOA <dbl>, lag_NFO <dbl>, RNOA <dbl>, PM <dbl>, ATO <dbl>,
#> #   lag_FLEV <dbl>, NBC <dbl>, ROE_DuPon <dbl>, DuP_gap <dbl>

```

財務データの企業・年度キーが一意であることを確認する。

```

financial_key_duplicates <- financial_data |>
  count(
    firm_ID,
    year,
    name = "observations"
  ) |>
  filter(observations > 1)

show_table_LFL(financial_key_duplicates)
#> # A tibble: 0 x 3
#> # i 3 variables: fir_ID <int>, year <int>, observations <int>

if (nrow(financial_key_duplicates) > 0) {
  stop(
    "財務データのfirm_IDとyearに重複があります。",
    call. = FALSE
  )
}

```

```

annual_data <- annual_stock_data |>
  left_join(
    financial_data,
    by = c("firm_ID", "year")
  ) |>
  arrange(firm_ID, year)

```

```

stopifnot(
  nrow(annual_data) ==
    nrow(annual_stock_data)
)

annual_data_overview <- annual_data |>
  summarise(
    observations = n(),
    firms = n_distinct(firm_ID),
    first_year = min(year, na.rm = TRUE),
    last_year = max(year, na.rm = TRUE),
    financial_matches =
      sum(!is.na(BE))
  )

show_table_LFL(annual_data_overview)
#> # A tibble: 1 x 5
#>   observations firms fir_year las_year fin_match
#>   <int> <int>   <int>   <int>   <int>
#> 1     7920  1515     2015     2020     7920

```

```

monthly_data <- stock_data |>
  left_join(
    financial_data,
    by = c("firm_ID", "year")
  ) |>
  arrange(firm_ID, month_ID)

stopifnot(
  nrow(monthly_data) ==
    nrow(stock_data)
)

row_count_check <- tibble(
  stock_rows = nrow(stock_data),
  monthly_rows = nrow(monthly_data),
  annual_rows = nrow(annual_data)
)

show_table_LFL(row_count_check)
#> # A tibble: 1 x 3
#>   sto_rows mon_rows ann_rows
#>   <int>   <int>   <int>
#> 1   95040   95040    7920

```

最新年度について、売上高、当期純利益および時価総額の関係を確認する。

```
latest_analysis_year <- annual_data |>
  filter(
    is.finite(sales),
    sales > 0,
    is.finite(X),
    X > 0,
    is.finite(ME),
    ME > 0
  ) |>
  summarise(
    latest_year = max(year)
  ) |>
  pull(latest_year)
```

```
bubble_chart_data <- annual_data |>
  filter(
    year == latest_analysis_year,
    is.finite(sales),
    sales > 0,
    is.finite(X),
    X > 0,
    is.finite(ME),
    ME > 0
  )
```

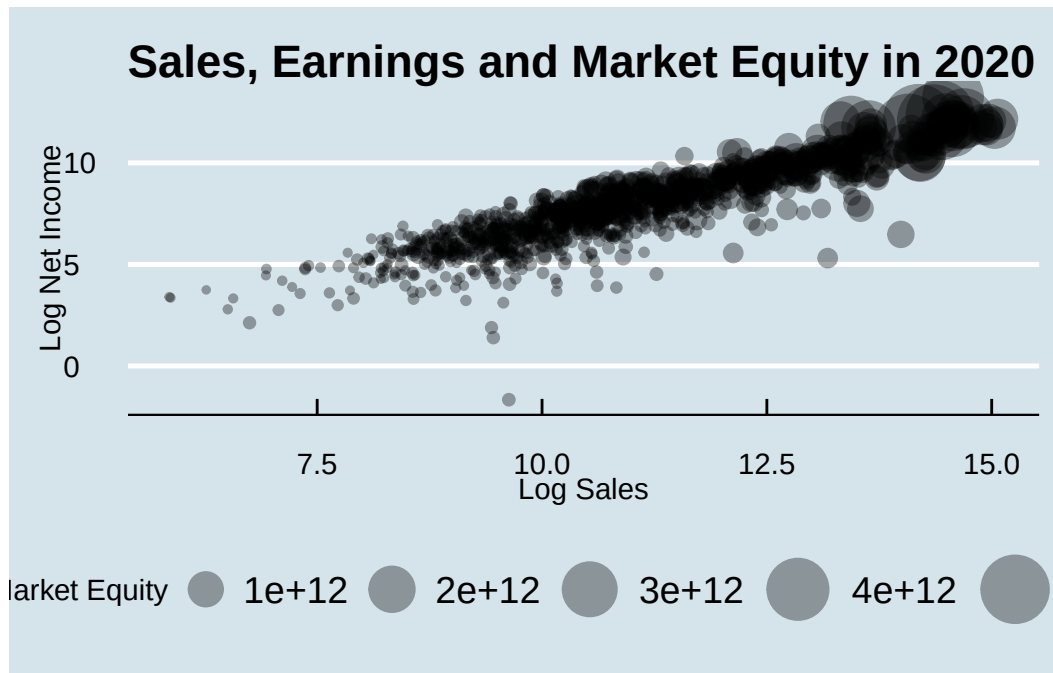
```
p_sales_earnings_market_equity <- bubble_chart_data |>
  ggplot(
    aes(
      x = log(sales),
      y = log(X),
      size = ME
    )
  ) +
  geom_point(alpha = 0.35) +
  scale_size(
    range = c(1, 12),
    name = "Market Equity"
  ) +
  labs(
    x = "Log Sales",
    y = "Log Net Income",
```

```

    title = paste(
      "Sales, Earnings and Market Equity in",
      latest_analysis_year
    )
  )

add_lagrange_theme_LFL(
  p_sales_earnings_market_equity
)

```



I-3.6 統計分析

観測数が最も多い企業を選び、月次超過リターンの平均が 0 と異なるかを検定する。

```

test_firm_id <- stock_data |>
  filter(is.finite(Re)) |>
  count(
    firm_ID,
    sort = TRUE
  ) |>
  slice_head(n = 1) |>
  pull(firm_ID)

test_firm_returns <- stock_data |>
  filter(
    firm_ID == test_firm_id,
    is.finite(Re)
  ) |>

```

```

pull(Re)

manual_t_value <- mean(test_firm_returns) /
(
  sd(test_firm_returns) /
  sqrt(length(test_firm_returns))
)

t_test_result <- t.test(
  test_firm_returns,
  mu = 0
)

t_test_summary <- broom::tidy(t_test_result) |>
mutate(
  firm_ID = test_firm_id,
  manual_t_value = manual_t_value,
  .before = 1
)

show_table_LFL(t_test_summary)
#> # A tibble: 1 x 10
#>   fir_ID man_value estimate statistic  p.value parameter  conf.low conf.high
#>   <int>    <dbl>    <dbl>    <dbl>    <dbl>    <dbl>    <dbl>    <dbl>
#> 1     1      2.1285 0.029161    2.1285 0.036817      70 0.0018367 0.056485
#> # i 2 more variables: method <chr>, alternative <chr>

```

前年度末の時価総額を作成し、期首株主資本を前年度末時価総額で割って簿価時価比率を計算する。

```

annual_data <- annual_data |>
  group_by(firm_ID) |>
  arrange(year, .by_group = TRUE) |>
  mutate(
    lagged_ME = lag(ME),
    lagged_BEME = lagged_BE / lagged_ME,
    lagged_BEME = if_else(
      is.finite(lagged_BEME) &
        lagged_BEME > 0,
      lagged_BEME,
      NA_real_
    )
  ) |>
  ungroup()

```

回帰に使用できる年度別観測数を確認する。

```
regression_observations_by_year <- annual_data |>
  filter(
    is.finite(Re),
    is.finite(lagged_BEME)
  ) |>
  count(
    year,
    name = "observations"
  ) |>
  arrange(year)

show_table_LFL(
  regression_observations_by_year,
  n = nrow(regression_observations_by_year)
)
#> # A tibble: 5 x 2
#>   year observations
#>   <int>         <int>
#> 1  2016         1241
#> 2  2017         1270
#> 3  2018         1268
#> 4  2019         1304
#> 5  2020         1322
```

```
eligible_regression_years <-
  regression_observations_by_year |>
  filter(observations >= 30)

if (nrow(eligible_regression_years) == 0) {
  stop(
    "回帰分析に必要な観測数を満たす年度がありません。",
    call. = FALSE
  )
}

regression_year <- eligible_regression_years |>
  summarise(year = max(year)) |>
  pull(year)

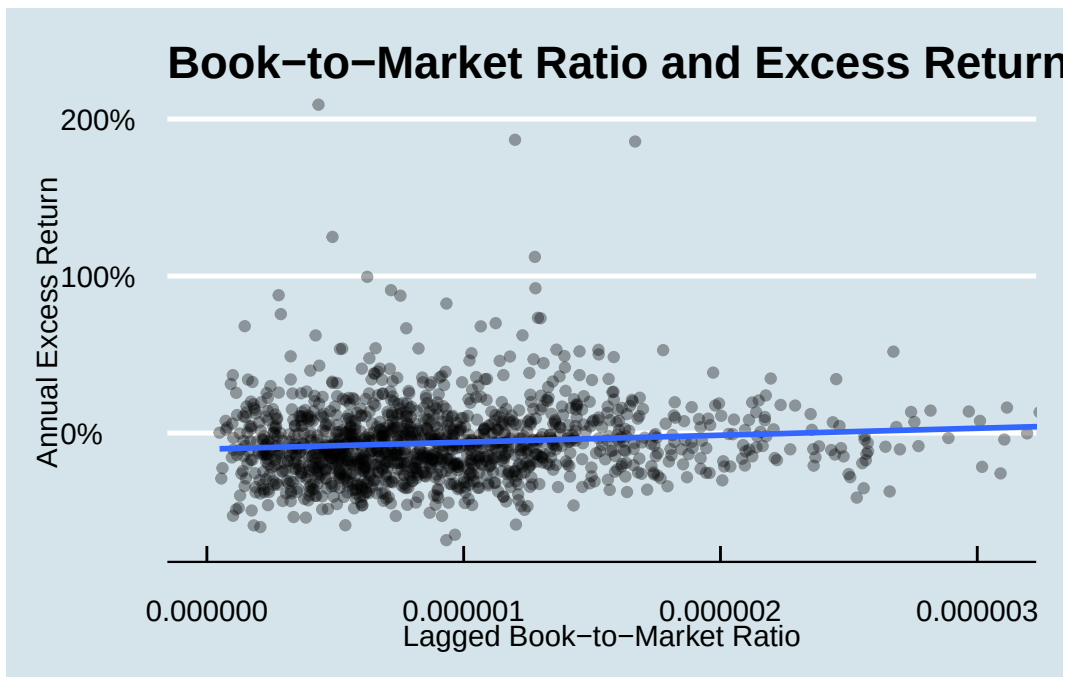
lm_sample_data <- annual_data |>
  filter(
    year == regression_year,
    is.finite(Re),
```

```
    is.finite(lagged_BEME)
  ) |>
select(
  firm_ID,
  year,
  Re,
  lagged_BEME
)
```

```
beme_plot_limit <- quantile(
  lm_sample_data$lagged_BEME,
  0.99,
  na.rm = TRUE
)

p_beme_return_regression <- lm_sample_data |>
ggplot(
  aes(
    x = lagged_BEME,
    y = Re
  )
) +
geom_point(alpha = 0.35) +
geom_smooth(
  method = "lm",
  se = FALSE
) +
coord_cartesian(
  xlim = c(0, beme_plot_limit)
) +
scale_y_continuous(
  labels = scales::label_percent()
) +
labs(
  x = "Lagged Book-to-Market Ratio",
  y = "Annual Excess Return",
  title = paste(
    "Book-to-Market Ratio and Excess Return in",
    regression_year
  )
)

add_lagrange_theme_LFL(
  p_beme_return_regression
)
```

```
lm_result <- lm(
  Re ~ lagged_BEME,
  data = lm_sample_data
)

show_table_LFL(
  broom::tidy(lm_result)
)

#> # A tibble: 2 x 5
#>   term          estimate  std.error statistic    p.value
#>   <chr>          <dbl>    <dbl>    <dbl>    <dbl>
#> 1 (Intercept)  -0.10237    0.012394  -8.2593 3.5232e-16
#> 2 lagged_BEME  44359.      10973.      4.0426 5.5911e- 5

show_table_LFL(
  broom::glance(lm_result)
)

#> # A tibble: 1 x 12
#>   r.squared adj.r.squared  sigma statistic    p.value    df logLik    AIC
#>   <dbl>      <dbl>    <dbl>    <dbl>    <dbl> <dbl> <dbl> <dbl>
#> 1  0.012230    0.011481  0.24523    16.343 0.000055911     1 -16.667 39.334
#> # i 4 more variables: BIC <dbl>, deviance <dbl>, df.residual <int>, nobs <int>
```

年度ごとの回帰を `nest()` と `purrr::map()` で実行する。

```
yearly_regression_models <- annual_data |>
  filter(
    is.finite(Re),
```

```
    is.finite(lagged_BEME)
  ) |>
group_by(year) |>
filter(n() >= 30) |>
ungroup() |>
nest(data = -year) |>
mutate(
  model = map(
    data,
    \(data) lm(
      Re ~ lagged_BEME,
      data = data
    )
  ),
  coefficients = map(
    model,
    broom::tidy
  ),
  fit = map(
    model,
    broom::glance
  )
)
```

```
yearly_regression_coefficients <-
  yearly_regression_models |>
  select(year, coefficients) |>
  unnest(coefficients) |>
  arrange(year, term)

yearly_regression_fit <-
  yearly_regression_models |>
  select(year, fit) |>
  unnest(fit) |>
  select(
    year,
    r.squared,
    adj.r.squared,
    sigma,
    statistic,
    p.value,
    nobs
  ) |>
  arrange(year)
```

```
show_table_LFL(
  yearly_regression_coefficients,
  n = nrow(yearly_regression_coefficients)
)
#> # A tibble: 10 x 6
#>   year term          estimate std.error statistic    p.value
#>   <int> <chr>          <dbl>      <dbl>    <dbl>    <dbl>
#> 1  2016 (Intercept)    0.056439    0.016563    3.4076 6.7644e- 4
#> 2  2016 lagged_BEME  72103.      12275.      5.8738 5.4661e- 9
#> 3  2017 (Intercept)    0.17989     0.015673   11.478 4.4094e-29
#> 4  2017 lagged_BEME  32467.      11890.      2.7306 6.4094e- 3
#> 5  2018 (Intercept)   -0.12679     0.015203   -8.3395 1.9259e-16
#> 6  2018 lagged_BEME  65283.      13067.      4.9959 6.6754e- 7
#> 7  2019 (Intercept)    0.28895     0.026406   10.942 1.0121e-26
#> 8  2019 lagged_BEME 100567.      19057.      5.2771 1.5363e- 7
#> 9  2020 (Intercept)   -0.10237     0.012394   -8.2593 3.5232e-16
#> 10 2020 lagged_BEME  44359.      10973.      4.0426 5.5911e- 5

show_table_LFL(
  yearly_regression_fit,
  n = nrow(yearly_regression_fit)
)
#> # A tibble: 5 x 7
#>   year r.squared adj.r.squared sigma statistic    p.value  nobs
#>   <int>    <dbl>      <dbl>    <dbl>    <dbl>    <dbl> <int>
#> 1  2016 0.027092    0.026306 0.28691   34.501 5.4661e-9 1241
#> 2  2017 0.0058459    0.0050619 0.28492    7.4562 6.4094e-3 1270
#> 3  2018 0.019334    0.018559 0.27721   24.959 6.6754e-7 1268
#> 4  2019 0.020940    0.020188 0.49819   27.847 1.5363e-7 1304
#> 5  2020 0.012230    0.011481 0.24523   16.343 5.5911e-5 1322
```

I-3.7 分析用データの保存

第4章以降で使用するため、月次データを `data/derived/returns/monthly_stock_returns.parquet`、年次データを `data/derived/fundamentals/annual_firm_characteristics.parquet` へ保存する。

```
output_data <- list(
  monthly = monthly_data,
  annual = annual_data
)

output_files <- c(
```

```

monthly = file.path(
  "returns",
  "monthly_stock_returns.parquet"
),
annual = file.path(
  "fundamentals",
  "annual_firm_characteristics.parquet"
)
)

output_paths <- output_data |>
  imap_chr(
    \(data, data_name)
      write_derived_parquet_file_LFL(
        data,
        output_files[[data_name]]
      )
    )

tibble::tibble(
  output = base::names(output_paths),
  path = relative_path_LFL(base::unname(output_paths))
) |>
  show_table_LFL()
#> # A tibble: 2 x 2
#>   output path
#>   <chr>   <chr>
#> 1 monthly data/derived/returns/monthly_stock_returns.parquet
#> 2 annual  data/derived/fundamentals/annual_firm_characteristics.parquet

```

保存したファイルを読み直し、行数と列名を検証する。

```

output_verification <- output_data |>
  imap_dfr(
    \(expected_data, data_name) {
      output_check <-
        read_derived_parquet_file_LFL(
          output_files[[data_name]]
        )

      stopifnot(
        nrow(output_check) ==
          nrow(expected_data)
      )
    }
  )

```

```

    stopifnot(
      identical(
        names(output_check),
        names(expected_data)
      )
    )

    tibble(
      data = data_name,
      output_file = relative_path_LFL(output_paths[[data_name]]),
      rows = nrow(output_check),
      columns = ncol(output_check),
      size_bytes =
        file.info(
          output_paths[[data_name]]
        )$size
    )
  }
)

show_table_LFL(output_verification)
#> # A tibble: 2 x 5
#>   data      out_file      rows columns siz_bytes
#>   <chr>    <chr>      <int>   <int>    <dbl>
#> 1 monthly data/derived/returns/monthly_stock_returns.pa~ 95040      39  4761273
#> 2 annual  data/derived/fundamentals/annual_firm_charact~  7920      31  1698508

```

I-3.8 まとめ

本章では、株価、配当および調整係数から月次リターンを作成し、無リスク金利を差し引いて超過リターンを計算した。さらに、月次リターンを年次リターンへ集約し、第2章で作成した財務データと結合した。最後に、簿価時価比率と年次超過リターンの関係を回帰分析で確認し、月次データを `data/derived/returns/monthly_stock_returns.parquet`、年次データを `data/derived/fundamentals/annual_firm_characteristics.parquet` として保存した。

第4章では、本章で作成した分析用データを利用して、ポートフォリオ分析へ進む。

第 I-4 章ファクターモデル分析

本章では、第 3 章で作成した月次株式データと年次財務データを利用し、市場ポートフォリオ、企業規模ポートフォリオ、簿価時価比率ポートフォリオ、SMB、HML を構築する。さらに、企業規模 10 分位ポートフォリオを対象として、CAPM と Fama–French の 3 ファクター・モデルを推定する。

本章の処理は、次の順序で進める。

1. 第 3 章の月次・年次データを読み込む
2. 前年度末の情報からポートフォリオを形成する
3. 市場ポートフォリオと市場超過リターンを計算する
4. 企業規模と簿価時価比率による単変量ソートを行う
5. 2×3 ポートフォリオから SMB と HML を構築する
6. CAPM と FF3 モデルを推定して比較する
7. 第 5 章以降で使用するファクター・データを保存する

I-4.1 分析環境

本章では、第 1 章から第 3 章と同じ共通ユーティリティを利用する。ファイル探索、Parquet 読み込み、表の表示、グラフのテーマ設定を各章で重複して定義しない。

I-4.2 入力データ

第 3 章で作成した次の 2 ファイルを読み込む。

```
data/derived/returns/monthly_stock_returns.parquet
data/derived/fundamentals/annual_firm_characteristics.parquet
```

`monthly_stock_returns.parquet` は月次データ、`annual_firm_characteristics.parquet` は年次データである。

```
tbl_monthly <-
  read_derived_parquet_file_LFL(
    file.path(
      "returns",
      "monthly_stock_returns.parquet"
    )
  ) |>
  mutate(
    firm_ID = as.integer(firm_ID),
    year = as.integer(year),
    month = as.integer(month),
    month_ID = as.integer(month_ID),
    month_date = as.Date(month_date)
  ) |>
  arrange(firm_ID, month_ID)

tbl_annual <-
  read_derived_parquet_file_LFL(
    file.path(
      "fundamentals",
      "annual_firm_characteristics.parquet"
    )
  ) |>
  mutate(
    firm_ID = as.integer(firm_ID),
    year = as.integer(year)
  ) |>
  arrange(firm_ID, year)

show_data_structure_LFL(tbl_monthly)
#> Rows: 95,040
#> Columns: 39
#> $ year                <int> 2015, 2015, 2015, 2015, 2015, 2015, 2015, 2015, ~
#> $ month               <int> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 1, 2, 3, ~
#> $ month_ID            <int> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, ~
#> $ firm_ID             <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ~
#> $ stock_price         <dbl> 954, 960, 1113, 1081, 1317, 1366, 1353, 1209, 1~
#> $ DPS                 <dbl> 0, 0, 0, 0, 0, 29, 0, 0, 0, 0, 0, 29, 0, 0, 0, ~
#> $ shares_outstanding  <dbl> 2422000, 2422000, 2422000, 2422000, 2422000, 24~
#> $ adjustment_coefficient <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ~
#> $ month_date          <date> 2015-01-01, 2015-02-01, 2015-03-01, 2015-04-01~
#> $ ME                 <dbl> 2310588000, 2325120000, 2695686000, 2618182000, ~
#> $ lagged_stock_price  <dbl> NA, 954, 960, 1113, 1081, 1317, 1366, 1353, 120~
#> $ lagged_month_ID    <int> NA, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, ~
#> $ consecutive_month  <lgl> NA, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, T~
```

```

#> $ R <dbl> NA, 0.006289308, 0.159375000, -0.028751123, 0.2~
#> $ R_F <dbl> 0.00008202927, 0.00008202927, 0.00008202927, 0.~
#> $ Re <dbl> NA, 0.006207279, 0.159292971, -0.028833152, 0.2~
#> $ industry_ID <fct> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,~
#> $ sales <dbl> 5261.40, 5261.40, 5261.40, 5261.40, 5261.40, 52~
#> $ OX <dbl> 437.49, 437.49, 437.49, 437.49, 437.49, 437.49,~
#> $ NFE <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~
#> $ X <dbl> 286.64, 286.64, 286.64, 286.64, 286.64, 286.64,~
#> $ OA <dbl> 13005.55, 13005.55, 13005.55, 13005.55, 13005.5~
#> $ FA <dbl> 3543.43, 3543.43, 3543.43, 3543.43, 3543.43, 35~
#> $ OL <dbl> 4372.96, 4372.96, 4372.96, 4372.96, 4372.96, 43~
#> $ FO <dbl> 2480.72, 2480.72, 2480.72, 2480.72, 2480.72, 24~
#> $ NOA <dbl> 8632.59, 8632.59, 8632.59, 8632.59, 8632.59, 86~
#> $ NFO <dbl> -1062.71, -1062.71, -1062.71, -1062.71, -1062.7~
#> $ BE <dbl> 9695.30, 9695.30, 9695.30, 9695.30, 9695.30, 96~
#> $ lagged_BE <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~
#> $ ROE <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~
#> $ lagged_NOA <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~
#> $ lagged_NFO <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~
#> $ RNOA <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~
#> $ PM <dbl> 0.08315087, 0.08315087, 0.08315087, 0.08315087,~
#> $ ATO <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~
#> $ lagged_FLEV <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~
#> $ NBC <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~
#> $ ROE_DuPont <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~
#> $ DuPont_gap <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~

show_data_structure_LFL(tbl_annual)

#> Rows: 7,920
#> Columns: 31
#> $ firm_ID <int> 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 3, 4, 4~
#> $ year <int> 2015, 2016, 2017, 2018, 2019, 2020, 2015, 2016, 2017, 2018~
#> $ months <int> 11, 12, 12, 12, 12, 12, 11, 12, 12, 12, 12, 12, 11, 12, 12~
#> $ R <dbl> 0.61213017, 0.99727265, 0.68786382, -0.21361287, 0.6468357~
#> $ Re <dbl> 0.61073273, 0.99547052, 0.68636684, -0.21438527, 0.6453315~
#> $ ME <dbl> 3577294000, 6883324000, 11376990000, 8694752000, 139575180~
#> $ industry_ID <fct> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1~
#> $ sales <dbl> 5261.40, 5948.96, 6505.06, 6846.38, 7572.24, 7537.63, 3505~
#> $ OX <dbl> 437.49, 564.14, 691.18, 751.29, 958.53, 778.37, 45.82, 51.~
#> $ NFE <dbl> NA, 50.667498, 29.543157, 86.486500, 298.049774, -65.45877~
#> $ X <dbl> 286.64, 513.48, 661.64, 664.80, 660.48, 843.83, 40.07, 49.~
#> $ OA <dbl> 13005.55, 13865.58, 13952.58, 18818.48, 18190.00, 20462.86~
#> $ FA <dbl> 3543.43, 4642.16, 7743.99, 7284.72, 9735.13, 10274.25, 225~
#> $ OL <dbl> 4372.96, 4534.22, 5111.22, 5137.28, 5487.96, 5371.38, 1840~
#> $ FO <dbl> 2480.72, 3959.70, 6159.02, 10123.91, 11362.22, 13772.15, 2~
#> $ NOA <dbl> 8632.59, 9331.36, 8841.36, 13681.20, 12702.04, 15091.48, 1~

```



```
#> $ NFO          <dbl> -1062.71, -682.46, -1584.97, 2839.19, 1627.09, 3497.90, 82~
#> $ BE           <dbl> 9695.30, 10013.82, 10426.33, 10842.01, 11074.95, 11593.58,~
#> $ lagged_BE     <dbl> NA, 9695.30, 10013.82, 10426.33, 10842.01, 11074.95, NA, 1~
#> $ ROE           <dbl> NA, 0.05296174, 0.06607269, 0.06376165, 0.06091859, 0.0761~
#> $ lagged_NOA    <dbl> NA, 8632.59, 9331.36, 8841.36, 13681.20, 12702.04, NA, 113~
#> $ lagged_NFO    <dbl> NA, -1062.71, -682.46, -1584.97, 2839.19, 1627.09, NA, 82.~
#> $ RNOA          <dbl> NA, 0.06535003, 0.07407066, 0.08497448, 0.07006184, 0.0612~
#> $ PM            <dbl> 0.083150872, 0.094830021, 0.106252671, 0.109735364, 0.1265~
#> $ ATO           <dbl> NA, 0.6891281, 0.6971181, 0.7743582, 0.5534778, 0.5934189,~
#> $ lagged_FLEV   <dbl> NA, -0.10961084, -0.06815181, -0.15201610, 0.26186934, 0.1~
#> $ NBC           <dbl> NA, -0.047677633, -0.043289214, -0.054566648, 0.104977044,~
#> $ ROE_DuPont    <dbl> NA, 0.05296097, 0.06607237, 0.06376199, 0.06091861, 0.0761~
#> $ DuPont_gap    <dbl> NA, 0.00000077332412, 0.00000031526680, -0.00000033567247,~
#> $ lagged_ME     <dbl> NA, 3577294000, 6883324000, 11376990000, 8694752000, 13957~
#> $ lagged_BEME   <dbl> NA, 0.0000027102329, 0.0000014547942, 0.0000009164401, 0.0~
```

```
tbl_input_overview <- tribble(
  ~data, ~observations, ~variables, ~firms, ~first_year, ~last_year,
  "monthly", nrow(tbl_monthly), ncol(tbl_monthly),
  n_distinct(tbl_monthly$firm_ID),
  min(tbl_monthly$year, na.rm = TRUE),
  max(tbl_monthly$year, na.rm = TRUE),
  "annual", nrow(tbl_annual), ncol(tbl_annual),
  n_distinct(tbl_annual$firm_ID),
  min(tbl_annual$year, na.rm = TRUE),
  max(tbl_annual$year, na.rm = TRUE)
)
```

```
show_table_LFL(tbl_input_overview)
```

```
#> # A tibble: 2 x 6
```

```
#>   data      observations variables firms fir_year las_year
#>   <chr>          <int>      <int> <int>   <int>   <int>
#> 1 monthly      95040         39  1515    2015    2020
#> 2 annual       7920         31  1515    2015    2020
```

分析を開始する前に、必要な列とパネルキーの一意性を確認する。

```
vec_required_monthly_columns <- c(
  "year", "month", "month_ID", "month_date",
  "firm_ID", "R", "R_F", "Re", "ME"
)

vec_required_annual_columns <- c(
  "year", "firm_ID", "ME", "BE", "lagged_BE"
```

```
)

vec_missing_monthly_columns <- setdiff(
  vec_required_monthly_columns,
  names(tbl_monthly)
)

vec_missing_annual_columns <- setdiff(
  vec_required_annual_columns,
  names(tbl_annual)
)

if (length(vec_missing_monthly_columns) > 0) {
  stop(
    "月次データに必要な列が不足しています: ",
    paste(vec_missing_monthly_columns, collapse = ", ")
  )
}

if (length(vec_missing_annual_columns) > 0) {
  stop(
    "年次データに必要な列が不足しています: ",
    paste(vec_missing_annual_columns, collapse = ", ")
  )
}

tbl_monthly_key_duplicates <- tbl_monthly |>
  count(firm_ID, month_ID, name = "observations") |>
  filter(observations > 1)

tbl_annual_key_duplicates <- tbl_annual |>
  count(firm_ID, year, name = "observations") |>
  filter(observations > 1)

if (nrow(tbl_monthly_key_duplicates) > 0) {
  stop("月次データのfirm_IDとmonth_IDの組合せに重複があります。")
}

if (nrow(tbl_annual_key_duplicates) > 0) {
  stop("年次データのfirm_IDとyearの組合せに重複があります。")
}

show_table_LFL(tbl_monthly_key_duplicates)
#> # A tibble: 0 x 3
#> # i 3 variables: fir_ID <int>, mon_ID <int>, observations <int>
```

```
show_table_LFL(tbl_annual_key_duplicates)
#> # A tibble: 0 x 3
#> # i 3 variables: fir_ID <int>, year <int>, observations <int>
```

次の関数は、欠損値や非有限値を除外して要約統計量と加重平均を計算する。

```
compute_median_safe <- function(x) {
  values <- x[is.finite(x)]
  if (length(values) == 0) return(NA_real_)
  median(values)
}

compute_mean_safe <- function(x) {
  values <- x[is.finite(x)]
  if (length(values) == 0) return(NA_real_)
  mean(values)
}

compute_weighted_mean_safe <- function(x, weight) {
  valid <- is.finite(x) & is.finite(weight) & weight > 0
  if (!any(valid)) return(NA_real_)
  sum(x[valid] * weight[valid]) / sum(weight[valid])
}

compute_significance_stars <- function(p_value) {
  case_when(
    is.na(p_value) ~ "",
    p_value < 0.01 ~ "***",
    p_value < 0.05 ~ "**",
    p_value < 0.10 ~ "*",
    TRUE ~ ""
  )
}
```

I-4.3 ポートフォリオ形成と市場リターン

年度 t の月次リターンに対して、年度 $t - 1$ 末に観測された時価総額と株主資本を使用する。これにより、将来情報をポートフォリオ形成に利用することを防ぐ。

```
if (!"lagged_ME" %in% names(tbl_annual)) {
  tbl_annual <- tbl_annual |>
  group_by(firm_ID) |>
  arrange(year, .by_group = TRUE) |>
```

```

    mutate(lagged_ME = lag(ME)) |>
    ungroup()
}

if (!"lagged_BEME" %in% names(tbl_annual)) {
  tbl_annual <- tbl_annual |>
    mutate(lagged_BEME = lagged_BE / lagged_ME)
}

tbl_annual <- tbl_annual |>
  mutate(
    lagged_ME = if_else(
      is.finite(lagged_ME) & lagged_ME > 0,
      lagged_ME,
      NA_real_
    ),
    lagged_BEME = if_else(
      is.finite(lagged_BEME) & lagged_BEME > 0,
      lagged_BEME,
      NA_real_
    )
  ) |>
  arrange(firm_ID, year)

tbl_lagged_variable_availability <- tbl_annual |>
  group_by(year) |>
  summarise(
    firms = n_distinct(firm_ID),
    valid_lagged_ME = sum(is.finite(lagged_ME)),
    valid_lagged_BEME = sum(is.finite(lagged_BEME)),
    valid_for_2x3 = sum(
      is.finite(lagged_ME) & is.finite(lagged_BEME)
    ),
    .groups = "drop"
  )

show_table_LFL(tbl_lagged_variable_availability)
#> # A tibble: 6 x 5
#>   year firms val_ME val_BEME val_2x3
#>   <int> <int> <int>    <int>    <int>
#> 1  2015  1266     0         0         0
#> 2  2016  1293  1241      1241      1241
#> 3  2017  1319  1270      1270      1270
#> 4  2018  1323  1268      1268      1268
#> 5  2019  1356  1304      1304      1304

```

```
#> 6 2020 1363 1322 1322 1322
```

年度 t の企業 i の市場ポートフォリオ・ウェイトは、前年度末時価総額から計算する。

$$w_{i,t}^M = \frac{ME_{i,t-1}}{\sum_j ME_{j,t-1}}$$

```
tbl_market_membership <- tbl_annual |>
  filter(is.finite(lagged_ME), lagged_ME > 0) |>
  group_by(year) |>
  mutate(market_weight = lagged_ME / sum(lagged_ME)) |>
  ungroup() |>
  select(year, firm_ID, lagged_ME, market_weight)

tbl_market_weight_check <- tbl_market_membership |>
  group_by(year) |>
  summarise(
    firms = n_distinct(firm_ID),
    weight_sum = sum(market_weight),
    minimum_weight = min(market_weight),
    maximum_weight = max(market_weight),
    .groups = "drop"
  )

if (any(abs(tbl_market_weight_check$weight_sum - 1) > 1e-10)) {
  stop("市場ポートフォリオの年度別ウェイト合計が1ではありません。")
}

show_table_LFL(tbl_market_weight_check)

#> # A tibble: 5 x 5
#>   year firms wei_sum   min_weigh max_weigh
#>   <int> <int>   <dbl>       <dbl>    <dbl>
#> 1  2016  1241     1 0.0000019019  0.040134
#> 2  2017  1270     1 0.0000015083  0.032973
#> 3  2018  1268     1 0.0000012609  0.042639
#> 4  2019  1304     1 0.0000021887  0.039396
#> 5  2020  1322     1 0.00000098548 0.056868
```

市場ポートフォリオの月次リターンは、前年度末時価総額による加重平均である。

$$R_{M,t} = \sum_i w_{i,t}^M R_{i,t}$$

```

tbl_market_returns <- tbl_monthly |>
  inner_join(
    tbl_market_membership,
    by = c("year", "firm_ID")
  ) |>
  group_by(month_ID, year, month, month_date) |>
  summarise(
    R_F = compute_median_safe(R_F),
    R_M = compute_weighted_mean_safe(R, lagged_ME),
    eligible_firms = n_distinct(firm_ID),
    valid_return_firms = sum(
      is.finite(R) & is.finite(lagged_ME) & lagged_ME > 0
    ),
    .groups = "drop"
  ) |>
  mutate(
    R_Me = R_M - R_F,
    return_coverage = valid_return_firms / eligible_firms
  ) |>
  arrange(month_ID)

tbl_invalid_market_months <- tbl_market_returns |>
  filter(!is.finite(R_F) | !is.finite(R_M) | !is.finite(R_Me))

if (nrow(tbl_invalid_market_months) > 0) {
  stop("市場リターンを作成できない月があります。")
}

show_table_LFL(tbl_market_returns)
#> # A tibble: 60 x 10
#>   mon_ID  year month mon_date      R_F      R_M eli_firms val_firms
#>   <int> <int> <int> <date>      <dbl>      <dbl>    <int>    <int>
#> 1     13  2016     1 2016-01-01 0.000082029  0.045835     1241     1241
#> 2     14  2016     2 2016-02-01 0.000075959  0.028423     1241     1241
#> 3     15  2016     3 2016-03-01 0.000075959 -0.057976     1241     1241
#> 4     16  2016     4 2016-04-01 0.000082029 -0.0010718     1241     1241
#> 5     17  2016     5 2016-05-01 0.000082029 -0.0064160     1241     1241
#> 6     18  2016     6 2016-06-01 0.000082029 -0.037075     1241     1241
#> 7     19  2016     7 2016-07-01 0.000082029  0.029853     1241     1241
#> 8     20  2016     8 2016-08-01 0.000082029  0.053757     1241     1241
#> 9     21  2016     9 2016-09-01 0.000082029  0.023650     1241     1241
#> 10    22  2016    10 2016-10-01 0.000075959 -0.012053     1241     1241
#> # i 50 more rows
#> # i 2 more variables: R_Me <dbl>, ret_cover <dbl>

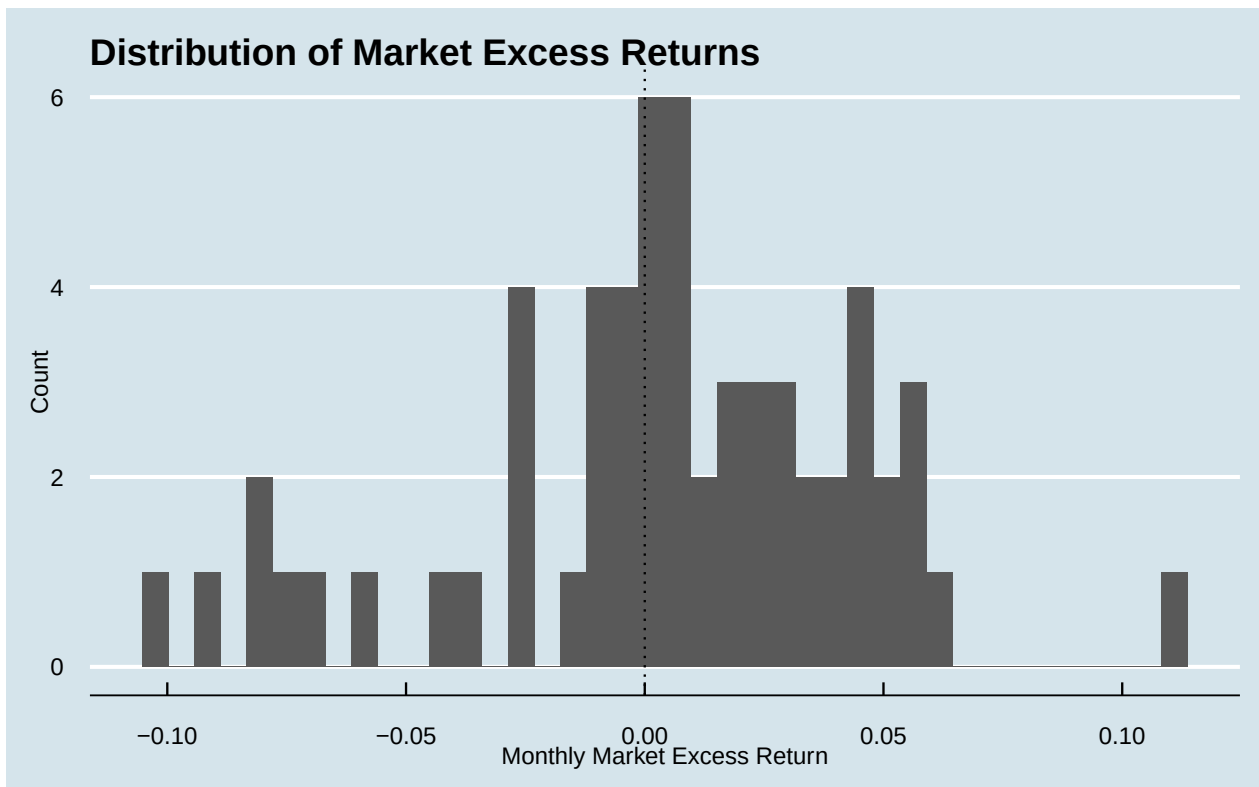
```

```
tbl_market_return_summary <- tbl_market_returns |>
  summarise(
    observations = n(),
    mean_market_return = mean(R_M),
    vec_mean_market_excess_return = mean(R_Me),
    market_volatility = sd(R_M),
    minimum_market_return = min(R_M),
    maximum_market_return = max(R_M),
    minimum_coverage = min(return_coverage)
  )

show_table_LFL(tbl_market_return_summary)
#> # A tibble: 1 x 7
#>   observations mea_retur vec_retur mar_volat min_retur max_retur min_cover
#>   <int>      <dbl>    <dbl>    <dbl>    <dbl>    <dbl>    <dbl>
#> 1         60 0.0041003 0.0040215 0.041498 -0.10244 0.11104      1
```

```
plot_market_excess_return <- tbl_market_returns |>
  ggplot(aes(x = R_Me)) +
  geom_histogram(bins = 40) +
  geom_vline(xintercept = 0, linetype = "dotted") +
  labs(
    x = "Monthly Market Excess Return",
    y = "Count",
    title = "Distribution of Market Excess Returns"
  )

show_plot_LFL(add_lagrange_theme_LFL(plot_market_excess_return))
```

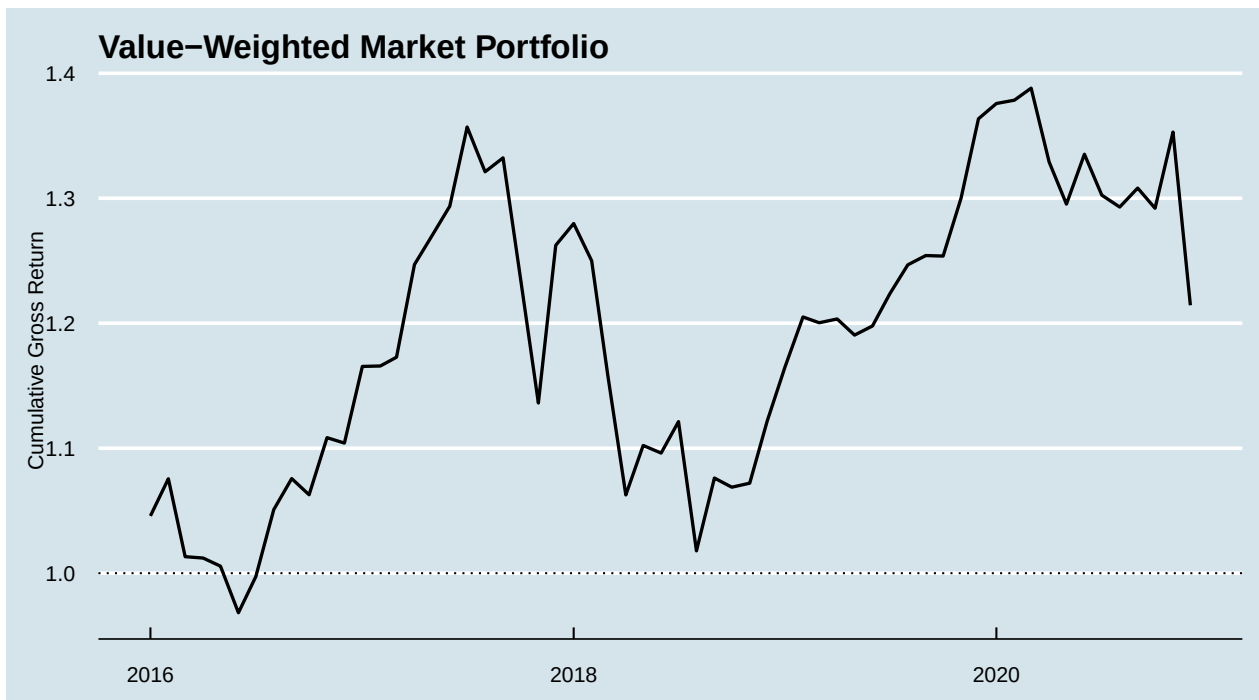


```

plot_cumulative_market_return <- tbl_market_returns |>
  arrange(month_ID) |>
  mutate(cumulative_gross_market_return = cumprod(1 + R_M)) |>
  ggplot(aes(x = month_date, y = cumulative_gross_market_return)) +
  geom_line(linewidth = 0.8) +
  geom_hline(yintercept = 1, linetype = "dotted") +
  labs(
    x = NULL,
    y = "Cumulative Gross Return",
    title = "Value-Weighted Market Portfolio"
  )
)

show_plot_LFL(add_lagrange_theme_LFL(plot_cumulative_market_return))

```

企業規模と簿価時価比率の両方を利用できる企業を共通の形成ユニバースとする。

```
tbl_formation_universe <- tbl_annual |>
  filter(
    is.finite(lagged_ME),
    lagged_ME > 0,
    is.finite(lagged_BEME),
    lagged_BEME > 0
  ) |>
  select(year, firm_ID, lagged_ME, lagged_BEME)

tbl_formation_breakpoints <- tbl_formation_universe |>
  group_by(year) |>
  summarise(
    firms = n_distinct(firm_ID),
    size_median = median(lagged_ME),
    BEME_30 = quantile(lagged_BEME, probs = 0.30, names = FALSE),
    BEME_70 = quantile(lagged_BEME, probs = 0.70, names = FALSE),
    .groups = "drop"
  )

tbl_formation <- tbl_formation_universe |>
  left_join(tbl_formation_breakpoints, by = "year") |>
  group_by(year) |>
  mutate(
    ME_rank10 = ntile(lagged_ME, 10),
    BEME_rank10 = ntile(lagged_BEME, 10)
  ) |>
  ungroup() |>
```

```

mutate(
  size_group = if_else(lagged_ME <= size_median, "S", "B"),
  BEME_group = case_when(
    lagged_BEME <= BEME_30 ~ "L",
    lagged_BEME <= BEME_70 ~ "N",
    TRUE ~ "H"
  ),
  FF_portfolio_type = factor(
    paste0(size_group, BEME_group),
    levels = c("SL", "SN", "SH", "BL", "BN", "BH")
  )
) |>
select(
  year, firm_ID, lagged_ME, lagged_BEME,
  ME_rank10, BEME_rank10,
  size_group, BEME_group, FF_portfolio_type
)

tbl_portfolio_membership_counts <- tbl_formation |>
  count(year, FF_portfolio_type, name = "firms") |>
  complete(
    year,
    FF_portfolio_type,
    fill = list(firms = 0)
  ) |>
  arrange(year, FF_portfolio_type)

tbl_empty_portfolios <- tbl_portfolio_membership_counts |>
  filter(firms == 0)

if (nrow(tbl_empty_portfolios) > 0) {
  stop("企業が割り当てられていない2×3ポートフォリオがあります。")
}

show_table_LFL(tbl_formation_breakpoints)
#> # A tibble: 5 x 5
#>   year firms   siz_media      BEM_30      BEM_70
#>   <int> <int>     <dbl>      <dbl>      <dbl>
#> 1  2016  1241 24564654000 0.00000076171 0.0000014304
#> 2  2017  1270 25133528500 0.00000070949 0.0000013738
#> 3  2018  1268 28268839000 0.00000061518 0.0000012038
#> 4  2019  1304 23853596500 0.00000073364 0.0000014045
#> 5  2020  1322 31878381000 0.00000057440 0.0000011304
show_table_LFL(tbl_portfolio_membership_counts)
#> # A tibble: 30 x 3

```

```
#>      year FF_type firms
#>    <int> <fct>  <int>
#>  1  2016 SL      140
#>  2  2016 SN      210
#>  3  2016 SH      271
#>  4  2016 BL      233
#>  5  2016 BN      286
#>  6  2016 BH      101
#>  7  2017 SL      141
#>  8  2017 SN      224
#>  9  2017 SH      270
#> 10  2017 BL      240
#> # i 20 more rows
```

I-4.4 単変量ポートフォリオ・ソート

企業規模 10 分位ごとに、等加重リターンと前年度末時価総額加重リターンを計算する。

```
tbl_size_decile_returns <- tbl_monthly |>
  inner_join(
    tbl_formation |>
      select(year, firm_ID, lagged_ME, ME_rank10),
    by = c("year", "firm_ID")
  ) |>
  group_by(
    month_ID, year, month, month_date, ME_rank10
  ) |>
  summarise(
    R_F = compute_median_safe(R_F),
    R_equal = compute_mean_safe(R),
    R_value = compute_weighted_mean_safe(R, lagged_ME),
    firms = n_distinct(firm_ID),
    valid_return_firms = sum(is.finite(R)),
    .groups = "drop"
  ) |>
  mutate(
    Re_equal = R_equal - R_F,
    Re_value = R_value - R_F
  ) |>
  arrange(month_ID, ME_rank10)

tbl_size_decile_coverage <- tbl_size_decile_returns |>
  group_by(month_ID) |>
```

```

summarise(
  portfolios = n_distinct(ME_rank10),
  .groups = "drop"
) |>
filter(portfolios != 10)

if (nrow(tbl_size_decile_coverage) > 0) {
  stop("10個の企業規模ポートフォリオがそろわない月があります。")
}

tbl_size_decile_summary <- tbl_size_decile_returns |>
  group_by(ME_rank10) |>
  summarise(
    months = n(),
    mean_equal_excess_return = mean(Re_equal, na.rm = TRUE),
    mean_value_excess_return = mean(Re_value, na.rm = TRUE),
    value_return_volatility = sd(Re_value, na.rm = TRUE),
    .groups = "drop"
  ) |>
  arrange(ME_rank10)

show_table_LFL(tbl_size_decile_summary)
#> # A tibble: 10 x 5
#>   ME_rank1 months mea_retur mea_retur val_volat
#>   <int>    <int>    <dbl>    <dbl>    <dbl>
#> 1      1      60 0.014931 0.015119 0.042085
#> 2      2      60 0.013581 0.013512 0.040856
#> 3      3      60 0.015247 0.015316 0.040102
#> 4      4      60 0.013073 0.012958 0.041626
#> 5      5      60 0.010989 0.010910 0.041165
#> 6      6      60 0.010277 0.010388 0.040414
#> 7      7      60 0.0067439 0.0065837 0.041338
#> 8      8      60 0.0051647 0.0054430 0.041428
#> 9      9      60 0.0046434 0.0045278 0.044503
#> 10     10      60 0.0037215 0.0029811 0.042314

```

```

plot_size_decile <- tbl_size_decile_summary |>
  select(
    ME_rank10,
    equal_weighted = mean_equal_excess_return,
    value_weighted = mean_value_excess_return
  ) |>
  pivot_longer(
    cols = c(equal_weighted, value_weighted),
    names_to = "weighting",

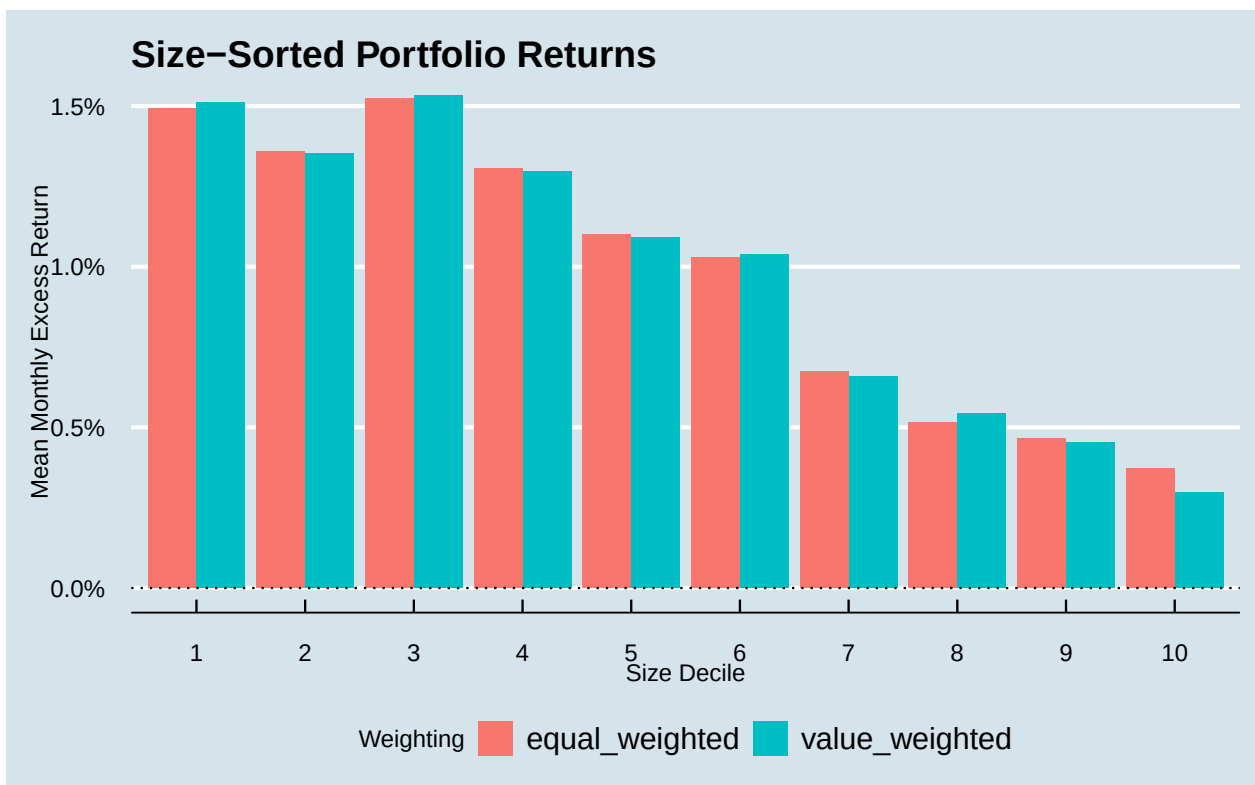
```

```

    values_to = "mean_excess_return"
  ) |>
  mutate(ME_rank10 = factor(ME_rank10, levels = 1:10)) |>
  ggplot(
    aes(
      x = ME_rank10,
      y = mean_excess_return,
      fill = weighting
    )
  ) +
  geom_col(position = "dodge") +
  geom_hline(yintercept = 0, linetype = "dotted") +
  scale_y_continuous(labels = scales::label_percent()) +
  labs(
    x = "Size Decile",
    y = "Mean Monthly Excess Return",
    fill = "Weighting",
    title = "Size-Sorted Portfolio Returns"
  )
)

show_plot_LFL(add_lagrange_theme_LFL(plot_size_decile))

```



```

tbl_beme_decile_returns <- tbl_monthly |>
  inner_join(
    tbl_formation |>
      select(year, firm_ID, lagged_ME, BEME_rank10),

```

```

    by = c("year", "firm_ID")
  ) |>
  group_by(
    month_ID, year, month, month_date, BEME_rank10
  ) |>
  summarise(
    R_F = compute_median_safe(R_F),
    R_equal = compute_mean_safe(R),
    R_value = compute_weighted_mean_safe(R, lagged_ME),
    firms = n_distinct(firm_ID),
    valid_return_firms = sum(is.finite(R)),
    .groups = "drop"
  ) |>
  mutate(
    Re_equal = R_equal - R_F,
    Re_value = R_value - R_F
  ) |>
  arrange(month_ID, BEME_rank10)

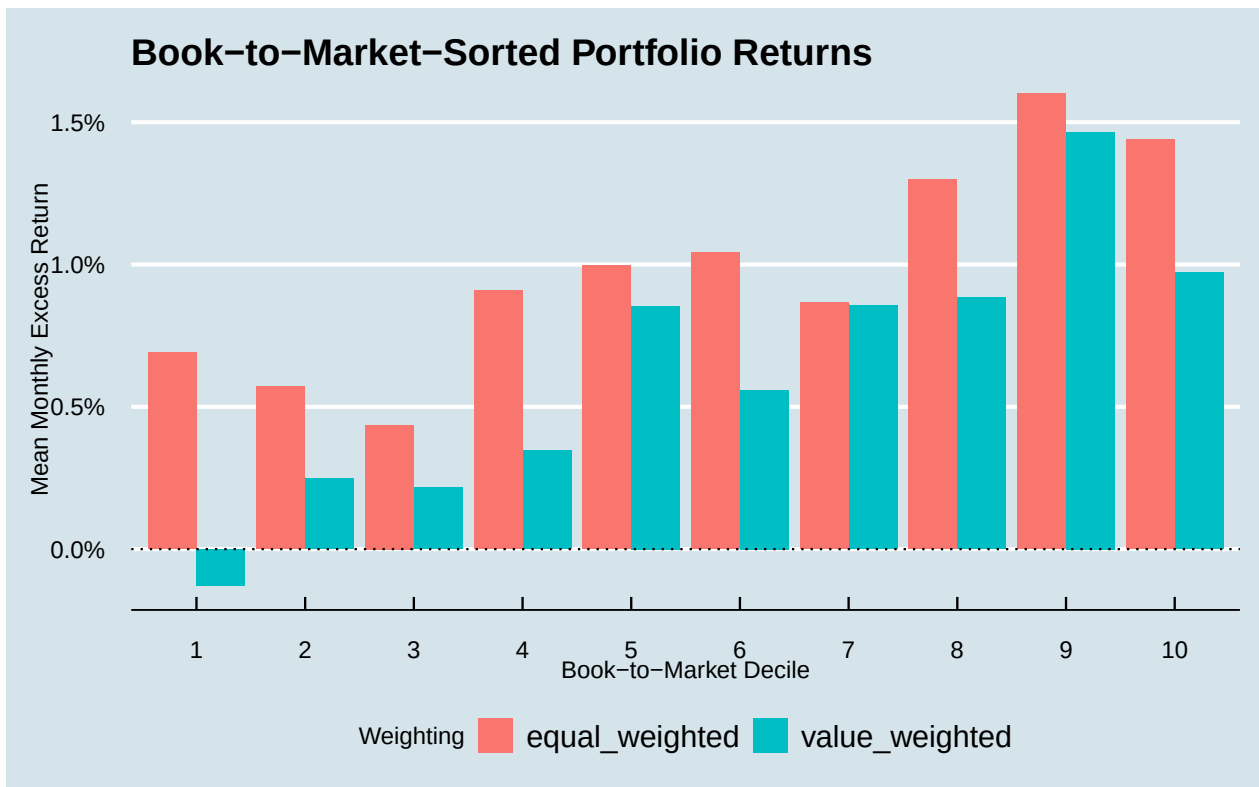
tbl_beme_decile_summary <- tbl_beme_decile_returns |>
  group_by(BEME_rank10) |>
  summarise(
    months = n(),
    mean_equal_excess_return = mean(Re_equal, na.rm = TRUE),
    mean_value_excess_return = mean(Re_value, na.rm = TRUE),
    value_return_volatility = sd(Re_value, na.rm = TRUE),
    .groups = "drop"
  ) |>
  arrange(BEME_rank10)

show_table_LFL(tbl_beme_decile_summary)
#> # A tibble: 10 x 5
#>   BEM_rank1 months mea_retur  mea_retur val_volat
#>   <int>   <int>    <dbl>    <dbl>    <dbl>
#> 1       1     60 0.0069080 -0.0012722 0.052048
#> 2       2     60 0.0057059 0.0024925 0.039802
#> 3       3     60 0.0043601 0.0021706 0.045255
#> 4       4     60 0.0090970 0.0034693 0.045192
#> 5       5     60 0.0099654 0.0085384 0.047651
#> 6       6     60 0.010433 0.0055909 0.042627
#> 7       7     60 0.0086709 0.0085607 0.044756
#> 8       8     60 0.012979 0.0088343 0.044844
#> 9       9     60 0.016017 0.014657 0.049615
#> 10      10     60 0.014389 0.0097273 0.043590

```

```
plot_beme_decile <- tbl_beme_decile_summary |>
  select(
    BEME_rank10,
    equal_weighted = mean_equal_excess_return,
    value_weighted = mean_value_excess_return
  ) |>
  pivot_longer(
    cols = c(equal_weighted, value_weighted),
    names_to = "weighting",
    values_to = "mean_excess_return"
  ) |>
  mutate(BEME_rank10 = factor(BEME_rank10, levels = 1:10)) |>
  ggplot(
    aes(
      x = BEME_rank10,
      y = mean_excess_return,
      fill = weighting
    )
  ) +
  geom_col(position = "dodge") +
  geom_hline(yintercept = 0, linetype = "dotted") +
  scale_y_continuous(labels = scales::label_percent()) +
  labs(
    x = "Book-to-Market Decile",
    y = "Mean Monthly Excess Return",
    fill = "Weighting",
    title = "Book-to-Market-Sorted Portfolio Returns"
  )

show_plot_LFL(add_lagrange_theme_LFL(plot_beme_decile))
```



I-4.5 CAPM

CAPM では、ポートフォリオ p の超過リターンを市場超過リターンで説明する。

$$R_{p,t}^e = \alpha_p + \beta_p R_{M,t}^e + \varepsilon_{p,t}$$

```
tbl_capm_data <- tbl_size_decile_returns |>
  select(month_ID, ME_rank10, Re = Re_value) |>
  inner_join(
    tbl_market_returns |>
      select(month_ID, R_Me),
    by = "month_ID"
  ) |>
  filter(is.finite(Re), is.finite(R_Me)) |>
  arrange(ME_rank10, month_ID)

tbl_capm_models <- tbl_capm_data |>
  group_by(ME_rank10) |>
  nest() |>
  mutate(
    model = map(data, \(data) lm(Re ~ R_Me, data = data)),
    coefficients = map(model, broom::tidy),
    fit = map(model, broom::glance)
  )
```



```
tbl_capm_results <- tbl_capm_models |>
  select(ME_rank10, coefficients) |>
  unnest(coefficients) |>
  mutate(significance = compute_significance_stars(p.value)) |>
  arrange(ME_rank10, term)

tbl_capm_fit <- tbl_capm_models |>
  select(ME_rank10, fit) |>
  unnest(fit) |>
  select(
    ME_rank10, r.squared, adj.r.squared,
    sigma, statistic, p.value, nobs
  ) |>
  arrange(ME_rank10)

tbl_capm_alpha <- tbl_capm_results |>
  filter(term == "(Intercept)") |>
  transmute(
    ME_rank10,
    tbl_capm_alpha = estimate,
    standard_error = std.error,
    statistic,
    p_value = p.value,
    significance
  )

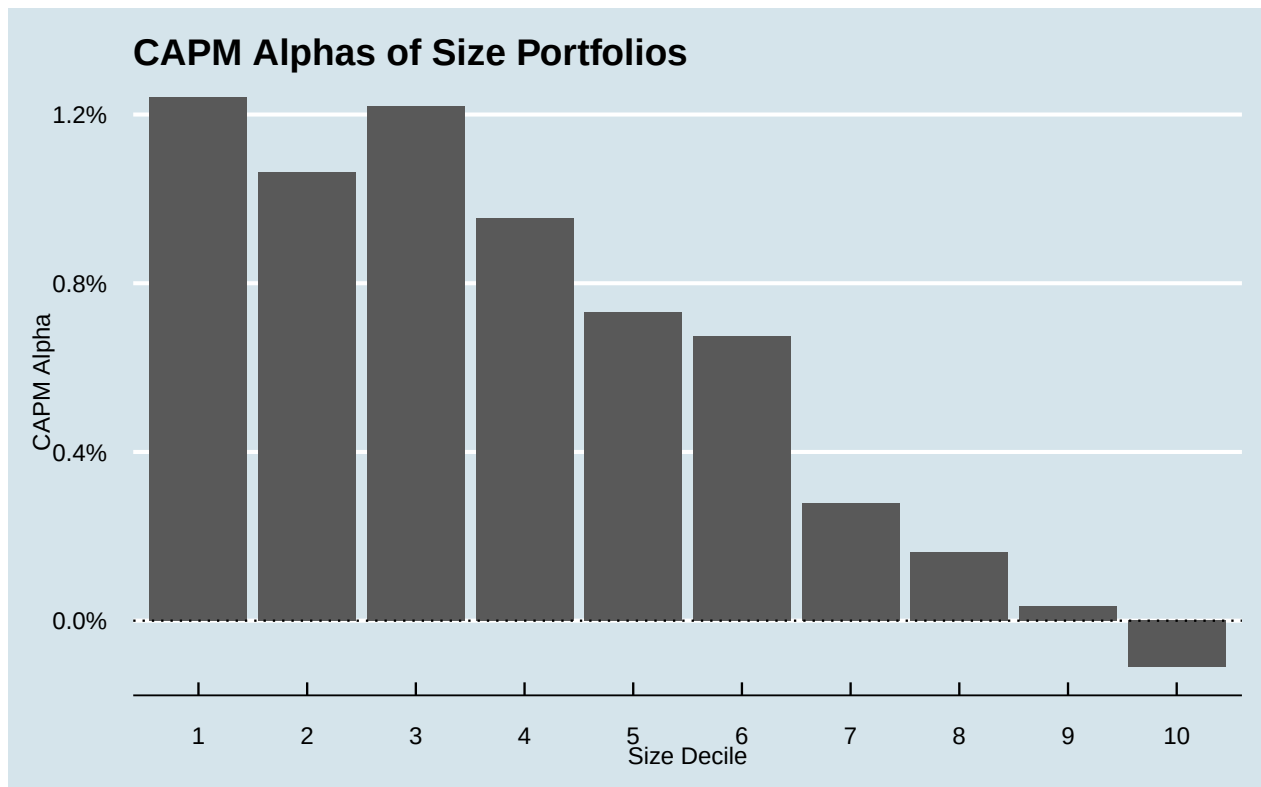
show_table_LFL(tbl_capm_results)
#> # A tibble: 20 x 7
#> # Groups:   ME_rank1 [10]
#>   ME_rank1 term          estimate std.error statistic    p.value significance
#>   <int> <chr>          <dbl>    <dbl>    <dbl>    <dbl> <chr>
#> 1     1 1 (Intercept) 0.012412 0.0041175 3.0144 3.8151e- 3 ***
#> 2     1 1 R_Me      0.67325 0.099584 6.7607 7.3884e- 9 ***
#> 3     2 2 (Intercept) 0.010641 0.0036808 2.8910 5.3962e- 3 ***
#> 4     2 2 R_Me      0.71390 0.089021 8.0195 5.6813e-11 ***
#> 5     3 3 (Intercept) 0.012195 0.0031256 3.9016 2.5116e- 4 ***
#> 6     3 3 R_Me      0.77614 0.075595 10.267 1.1583e-14 ***
#> 7     4 4 (Intercept) 0.0095370 0.0028845 3.3062 1.6262e- 3 ***
#> 8     4 4 R_Me      0.85081 0.069764 12.196 1.2013e-17 ***
#> 9     5 5 (Intercept) 0.0073101 0.0023212 3.1493 2.5865e- 3 ***
#> 10    5 5 R_Me      0.89511 0.056138 15.945 7.2596e-23 ***
#> # i 10 more rows

show_table_LFL(tbl_capm_fit)
#> # A tibble: 10 x 7
```

```
#> # Groups:   ME_rank1 [10]
#>   ME_rank1 r.squared adj.r.squared   sigma statistic   p.value  nobs
#>   <int>    <dbl>      <dbl>      <dbl>    <dbl>      <dbl> <int>
#> 1      1    0.44073    0.43109 0.031743    45.706 7.3884e- 9    60
#> 2      2    0.52580    0.51763 0.028376    64.312 5.6813e-11    60
#> 3      3    0.64507    0.63895 0.024096   105.41 1.1583e-14    60
#> 4      4    0.71944    0.71461 0.022238   148.73 1.2013e-17    60
#> 5      5    0.81424    0.81104 0.017894   254.23 7.2596e-23    60
#> 6      6    0.86156    0.85918 0.015166   360.96 1.3994e-26    60
#> 7      7    0.89634    0.89456 0.013423   501.54 3.1166e-30    60
#> 8      8    0.90073    0.89902 0.013165   526.25 8.8761e-31    60
#> 9      9    0.93827    0.93721 0.011152   881.65 9.0177e-37    60
#> 10     10    0.98847    0.98827 0.0045827 4972.0 6.5289e-58    60
```

```
plot_capm_alpha <- tbl_capm_alpha |>
  mutate(ME_rank10 = factor(ME_rank10, levels = 1:10)) |>
  ggplot(aes(x = ME_rank10, y = tbl_capm_alpha)) +
  geom_col() +
  geom_hline(yintercept = 0, linetype = "dotted") +
  scale_y_continuous(labels = scales::label_percent()) +
  labs(
    x = "Size Decile",
    y = "CAPM Alpha",
    title = "CAPM Alphas of Size Portfolios"
  )

show_plot_LFL(add_lagrange_theme_LFL(plot_capm_alpha))
```



```
tbl_capm_beta <- tbl_capm_results |>
  filter(term == "R_Me") |>
  transmute(
    ME_rank10,
    tbl_capm_beta = estimate,
    beta_p_value = p.value
  )

tbl_security_market_line <- tbl_size_decile_summary |>
  select(
    ME_rank10,
    mean_excess_return = mean_value_excess_return
  ) |>
  left_join(tbl_capm_beta, by = "ME_rank10")

vec_mean_market_excess_return <- mean(
  tbl_market_returns$R_Me,
  na.rm = TRUE
)

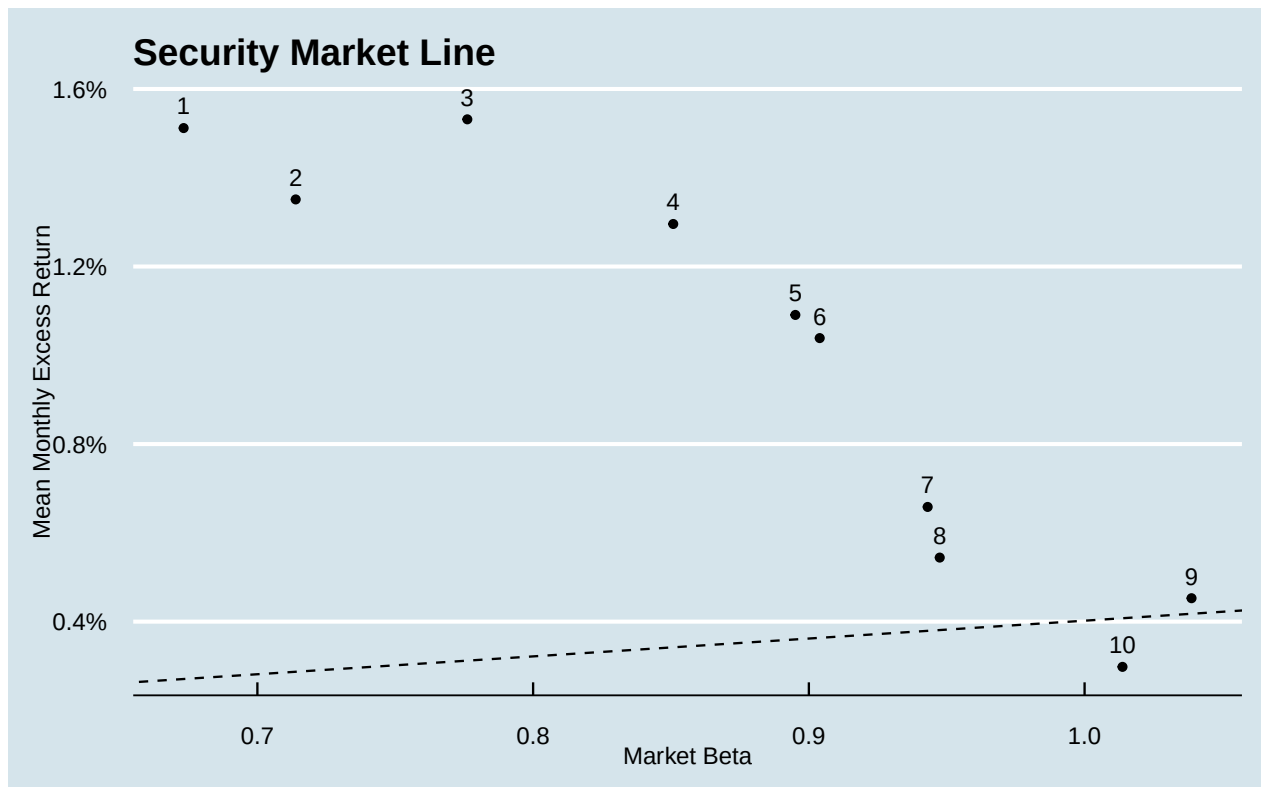
plot_security_market_line <- tbl_security_market_line |>
  ggplot(
    aes(
      x = tbl_capm_beta,
      y = mean_excess_return
    )
  ) +
```

```

geom_point() +
geom_text(aes(label = ME_rank10), nudge_y = 0.0005) +
geom_abline(
  intercept = 0,
  slope = vec_mean_market_excess_return,
  linetype = "dashed"
) +
scale_y_continuous(labels = scales::label_percent()) +
labs(
  x = "Market Beta",
  y = "Mean Monthly Excess Return",
  title = "Security Market Line"
)

show_plot_LFL(add_lagrange_theme_LFL(plot_security_market_line))

```



I-4.6 2×3 ポートフォリオと SMB・HML

企業規模を Small と Big の 2 群、簿価時価比率を Low、Neutral、High の 3 群に分ける。

ポートフォリオ	企業規模	簿価時価比率
SL	Small	Low
SN	Small	Neutral
SH	Small	High

ポートフォリオ	企業規模	簿価時価比率
BL	Big	Low
BN	Big	Neutral
BH	Big	High

```
tbl_ff_portfolio_returns <- tbl_monthly |>
  inner_join(
    tbl_formation |>
      select(
        year, firm_ID, lagged_ME,
        size_group, BEME_group, FF_portfolio_type
      ),
    by = c("year", "firm_ID")
  ) |>
  filter(!is.na(FF_portfolio_type)) |>
  group_by(
    month_ID, year, month, month_date,
    FF_portfolio_type, size_group, BEME_group
  ) |>
  summarise(
    R_F = compute_median_safe(R_F),
    R = compute_weighted_mean_safe(R, lagged_ME),
    firms = n_distinct(firm_ID),
    valid_return_firms = sum(is.finite(R)),
    .groups = "drop"
  ) |>
  mutate(Re = R - R_F) |>
  arrange(month_ID, FF_portfolio_type)

tbl_ff_portfolio_coverage <- tbl_ff_portfolio_returns |>
  group_by(month_ID) |>
  summarise(
    portfolios = n_distinct(FF_portfolio_type),
    .groups = "drop"
  ) |>
  filter(portfolios != 6)

if (nrow(tbl_ff_portfolio_coverage) > 0) {
  stop("6個のSize--BE/MEポートフォリオがそろわない月があります。")
}

tbl_ff_portfolio_summary <- tbl_ff_portfolio_returns |>
  group_by(
```

```

    FF_portfolio_type,
    size_group,
    BEME_group
  ) |>
  summarise(
    months = n(),
    mean_return = mean(R),
    mean_excess_return = mean(Re),
    volatility = sd(R),
    .groups = "drop"
  ) |>
  arrange(FF_portfolio_type)

```

```
show_table_LFL(tbl_ff_portfolio_summary)
```

```
#> # A tibble: 6 x 7
```

```

#>   FF_type siz_group BEM_group months  mea_retur  mea_retur volatility
#>   <fct>   <chr>    <chr>    <int>    <dbl>    <dbl>    <dbl>
#> 1 SL      S        L        60  0.010056  0.0099773  0.044267
#> 2 SN      S        N        60  0.013474  0.013395   0.041252
#> 3 SH      S        H        60  0.014002  0.013923   0.040505
#> 4 BL      B        L        60  0.00081235 0.00073351 0.043049
#> 5 BN      B        N        60  0.0063080  0.0062291  0.042621
#> 6 BH      B        H        60  0.010766  0.010687   0.046846

```

```

plot_ff_portfolio <- tbl_ff_portfolio_summary |>
  ggplot(
    aes(
      x = BEME_group,
      y = mean_excess_return,
      fill = size_group
    )
  ) +
  geom_col(position = "dodge") +
  geom_text(
    aes(label = FF_portfolio_type, group = size_group),
    position = position_dodge(width = 0.9),
    vjust = -0.5
  ) +
  geom_hline(yintercept = 0, linetype = "dotted") +
  scale_y_continuous(
    labels = scales::label_percent(),
    expand = expansion(mult = c(0.05, 0.15))
  ) +
  labs(
    x = "Book-to-Market Group",

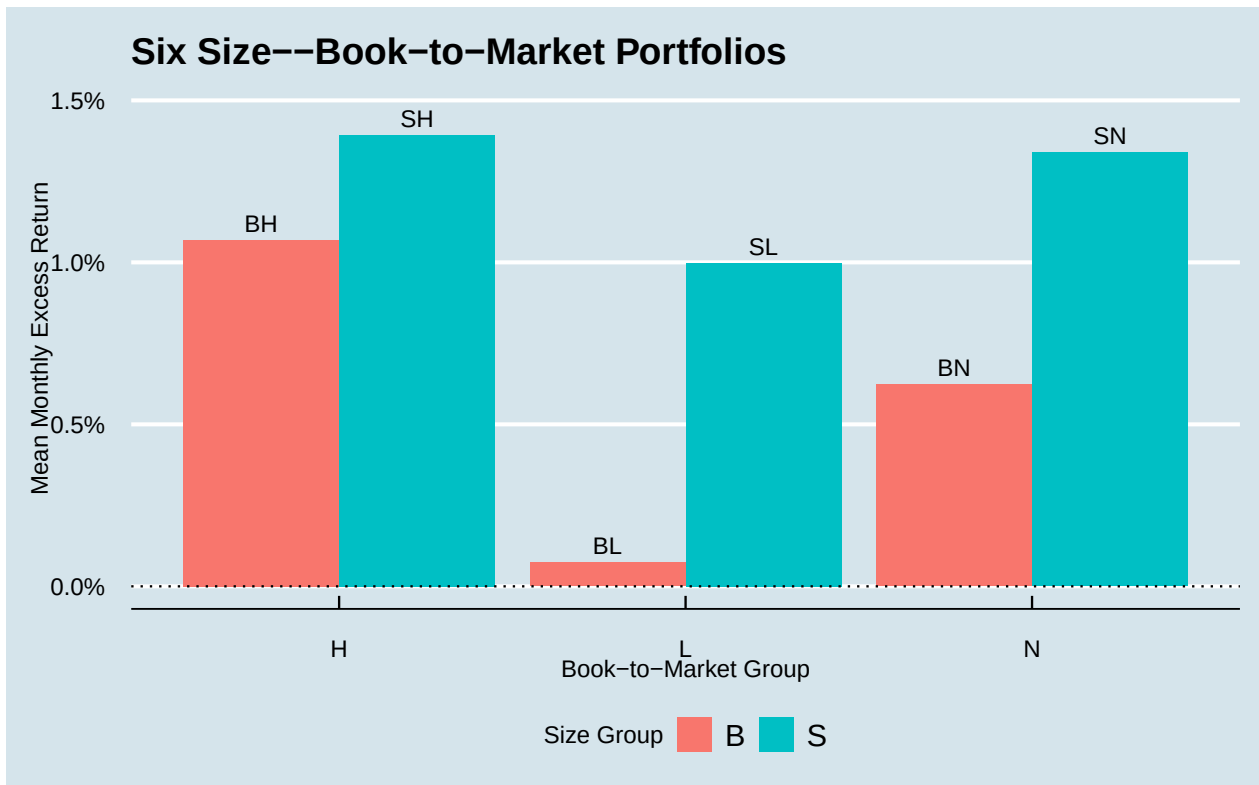
```

```

y = "Mean Monthly Excess Return",
fill = "Size Group",
title = "Six Size--Book-to-Market Portfolios"
)

```

```
show_plot_LFL(add_lagrange_theme_LFL(plot_ff_portfolio))
```



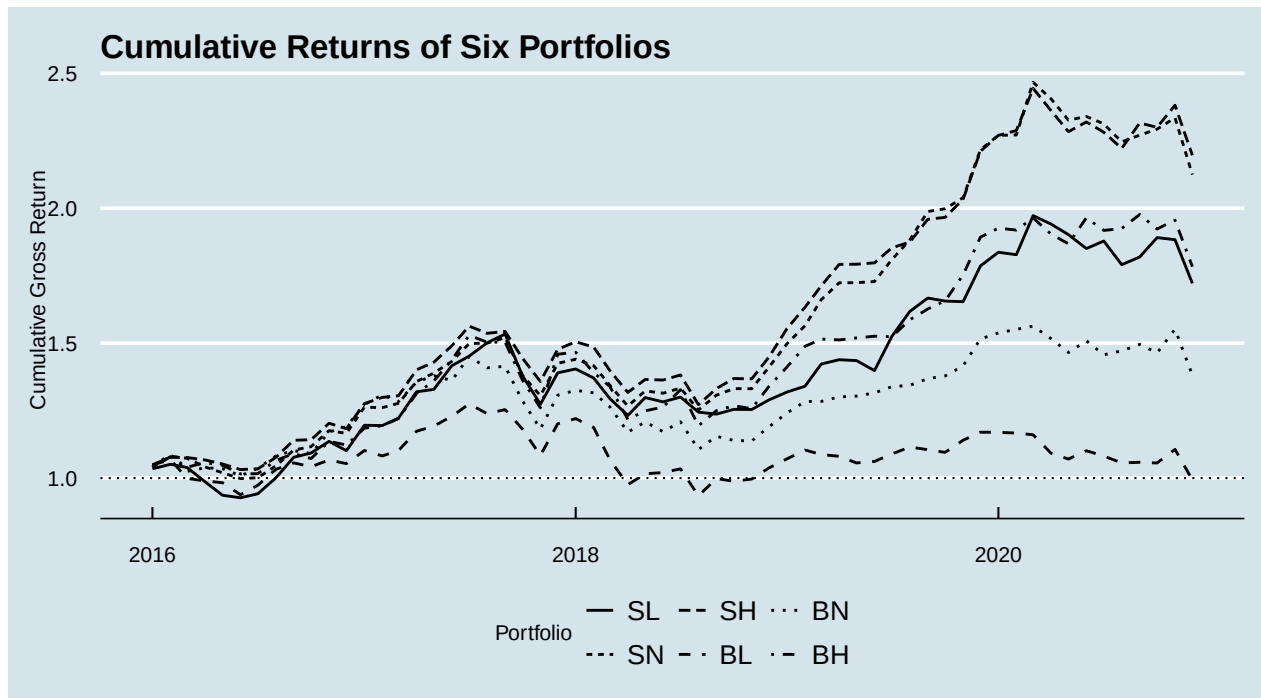
```

plot_ff_cumulative <- tbl_ff_portfolio_returns |>
  group_by(FF_portfolio_type) |>
  arrange(month_ID, .by_group = TRUE) |>
  mutate(cumulative_gross_return = cumprod(1 + R)) |>
  ungroup() |>
  ggplot(
    aes(
      x = month_date,
      y = cumulative_gross_return,
      linetype = FF_portfolio_type
    )
  ) +
  geom_line(linewidth = 0.7) +
  geom_hline(yintercept = 1, linetype = "dotted") +
  labs(
    x = NULL,
    y = "Cumulative Gross Return",
    linetype = "Portfolio",
    title = "Cumulative Returns of Six Portfolios"
  )

```

)

```
show_plot_LFL(add_lagrange_theme_LFL(plot_ff_cumulative))
```



SMB は Small ポートフォリオの平均リターンから Big ポートフォリオの平均リターンを引いて計算する。

$$SMB_t = \frac{SL_t + SN_t + SH_t}{3} - \frac{BL_t + BN_t + BH_t}{3}$$

HML は High ポートフォリオの平均リターンから Low ポートフォリオの平均リターンを引いて計算する。

$$HML_t = \frac{SH_t + BH_t}{2} - \frac{SL_t + BL_t}{2}$$

```
tbl_ff_portfolio_wide <- tbl_ff_portfolio_returns |>
  select(
    month_ID, year, month, month_date,
    FF_portfolio_type, R
  ) |>
  pivot_wider(
    names_from = FF_portfolio_type,
    values_from = R
  ) |>
  arrange(month_ID)
```



```
tbl_smb_hml <- tbl_ff_portfolio_wide |>
  mutate(
    SMB = (SL + SN + SH) / 3 - (BL + BN + BH) / 3,
    HML = (SH + BH) / 2 - (SL + BL) / 2
  ) |>
  select(
    month_ID, year, month, month_date,
    SL, SN, SH, BL, BN, BH, SMB, HML
  )

tbl_factor <- tbl_market_returns |>
  select(
    month_ID, year, month, month_date,
    R_F, R_M, R_Me,
    market_return_coverage = return_coverage
  ) |>
  inner_join(
    tbl_smb_hml,
    by = c("month_ID", "year", "month", "month_date")
  ) |>
  arrange(month_ID)

tbl_invalid_factor_rows <- tbl_factor |>
  filter(
    !if_all(
      c(R_F, R_M, R_Me, SMB, HML),
      is.finite
    )
  )

if (nrow(tbl_invalid_factor_rows) > 0) {
  stop("ファクター・データに欠損値または非有限値があります。")
}

show_table_LFL(tbl_factor)

#> # A tibble: 60 x 16
#>   mon_ID year month mon_date      R_F      R_M      R_Me mar_cover
#>   <int> <int> <int> <date>      <dbl>      <dbl>      <dbl>      <dbl>
#> 1     13  2016     1 2016-01-01 0.000082029  0.045835  0.045753         1
#> 2     14  2016     2 2016-02-01 0.000075959  0.028423  0.028347         1
#> 3     15  2016     3 2016-03-01 0.000075959 -0.057976 -0.058052         1
#> 4     16  2016     4 2016-04-01 0.000082029 -0.0010718 -0.0011539         1
#> 5     17  2016     5 2016-05-01 0.000082029 -0.0064160 -0.0064980         1
#> 6     18  2016     6 2016-06-01 0.000082029 -0.037075 -0.037157         1
#> 7     19  2016     7 2016-07-01 0.000082029  0.029853  0.029771         1
```

```
#> 8      20 2016      8 2016-08-01 0.000082029 0.053757 0.053675      1
#> 9      21 2016      9 2016-09-01 0.000082029 0.023650 0.023568      1
#> 10     22 2016     10 2016-10-01 0.000075959 -0.012053 -0.012129      1
#> # i 50 more rows
#> # i 8 more variables: SL <dbl>, SN <dbl>, SH <dbl>, BL <dbl>, BN <dbl>,
#> #   BH <dbl>, SMB <dbl>, HML <dbl>
```

```
vec_factor_names <- c("R_Me", "SMB", "HML")

tbl_factor_summary <- vec_factor_names |>
  map_dfr(
    \(factor_name) {
      values <- tbl_factor[[factor_name]]
      finite_values <- values[is.finite(values)]

      tibble(
        factor = factor_name,
        observations = length(finite_values),
        mean = mean(finite_values),
        standard_deviation = sd(finite_values),
        minimum = min(finite_values),
        median = median(finite_values),
        maximum = max(finite_values),
        t_value = mean(finite_values) /
          (sd(finite_values) / sqrt(length(finite_values)))
      )
    }
  )

tbl_factor_correlation <- tbl_factor |>
  select(R_Me, SMB, HML) |>
  cor(use = "complete.obs")

show_table_LFL(tbl_factor_summary)
#> # A tibble: 3 x 8
#>   factor observations      mean sta_devia  minimum  median  maximum t_value
#>   <chr>          <int>    <dbl>    <dbl>    <dbl>    <dbl>    <dbl>    <dbl>
#> 1 R_Me              60 0.0040215 0.041498 -0.10252 0.0060055 0.11097 0.75064
#> 2 SMB                60 0.0065488 0.022857 -0.038447 0.0055499 0.069390 2.2193
#> 3 HML                60 0.0069496 0.018058 -0.043572 0.0089560 0.035442 2.9811

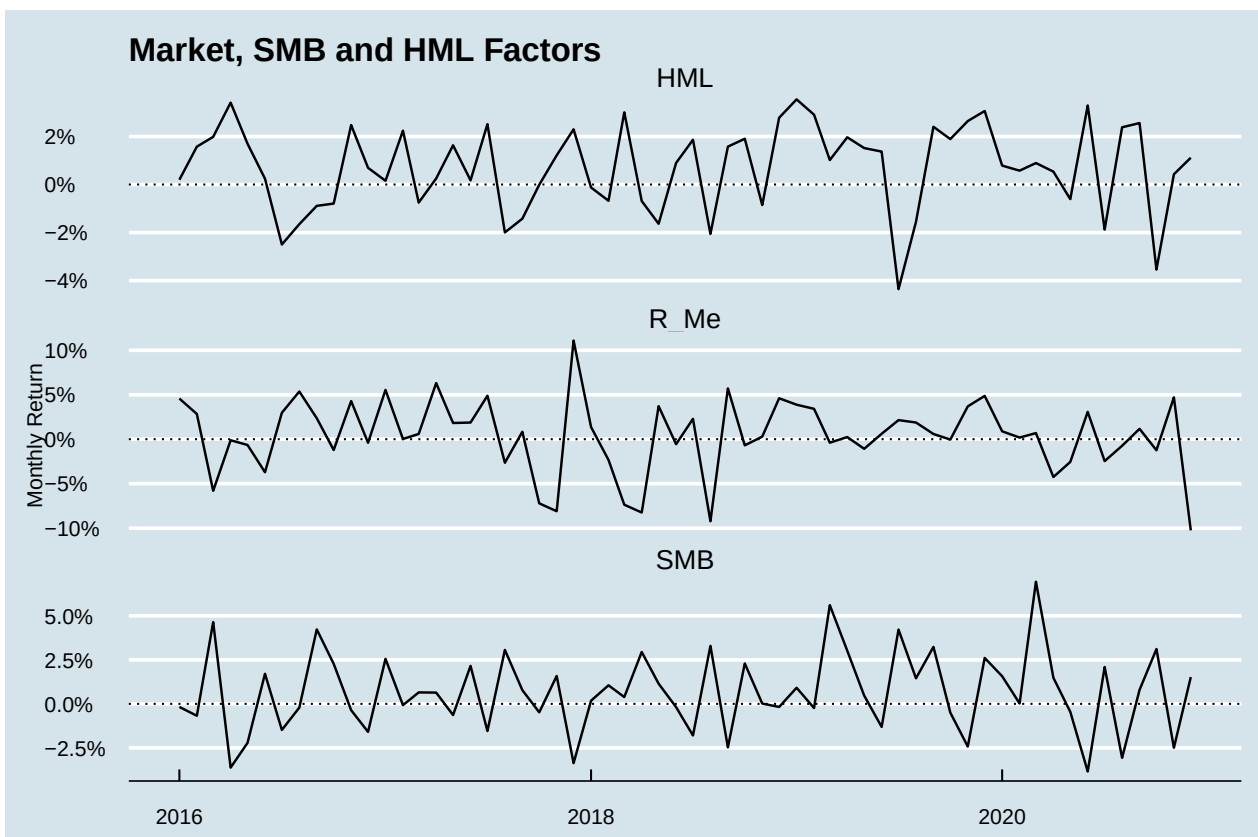
tbl_factor_correlation
#>           R_Me           SMB           HML
#> R_Me  1.0000000 -0.3431883  0.1702499
#> SMB  -0.3431883  1.0000000 -0.3806883
#> HML   0.1702499 -0.3806883  1.0000000
```

```

plot_factor_time_series <- tbl_factor |>
  select(month_date, R_Me, SMB, HML) |>
  pivot_longer(
    cols = c(R_Me, SMB, HML),
    names_to = "factor",
    values_to = "return"
  ) |>
  ggplot(aes(x = month_date, y = return)) +
  geom_line(linewidth = 0.6) +
  geom_hline(yintercept = 0, linetype = "dotted") +
  facet_wrap(
    vars(factor),
    ncol = 1,
    scales = "free_y"
  ) +
  scale_y_continuous(labels = scales::label_percent()) +
  labs(
    x = NULL,
    y = "Monthly Return",
    title = "Market, SMB and HML Factors"
  )
)

show_plot_LFL(add_lagrange_theme_LFL(plot_factor_time_series))

```



I-4.7 FF3 モデルと CAPM の比較

Fama–French の 3 ファクター・モデルは、ポートフォリオの超過リターンを市場、SMB、HML で説明する。

$$R_{p,t}^e = \alpha_p + \beta_{M,p} R_{M,t}^e + \beta_{SMB,p} SMB_t + \beta_{HML,p} HML_t + \varepsilon_{p,t}$$

```
tbl_ff3_data <- tbl_size_decile_returns |>
  select(month_ID, ME_rank10, Re = Re_value) |>
  inner_join(
    tbl_factor |>
      select(month_ID, R_Me, SMB, HML),
    by = "month_ID"
  ) |>
  filter(
    if_all(
      c(Re, R_Me, SMB, HML),
      is.finite
    )
  ) |>
  arrange(ME_rank10, month_ID)

tbl_ff3_models <- tbl_ff3_data |>
  group_by(ME_rank10) |>
  nest() |>
  mutate(
    model = map(
      data,
      \(data) lm(Re ~ R_Me + SMB + HML, data = data)
    ),
    coefficients = map(model, broom::tidy),
    fit = map(model, broom::glance)
  )

tbl_ff3_results <- tbl_ff3_models |>
  select(ME_rank10, coefficients) |>
  unnest(coefficients) |>
  mutate(significance = compute_significance_stars(p.value)) |>
  arrange(ME_rank10, term)

tbl_ff3_fit <- tbl_ff3_models |>
  select(ME_rank10, fit) |>
```

```

unnest(fit) |>
select(
  ME_rank10, r.squared, adj.r.squared,
  sigma, statistic, p.value, nobs
) |>
arrange(ME_rank10)

tbl_ff3_alpha <- tbl_ff3_results |>
filter(term == "(Intercept)") |>
transmute(
  ME_rank10,
  tbl_ff3_alpha = estimate,
  standard_error = std.error,
  statistic,
  p_value = p.value,
  significance
)

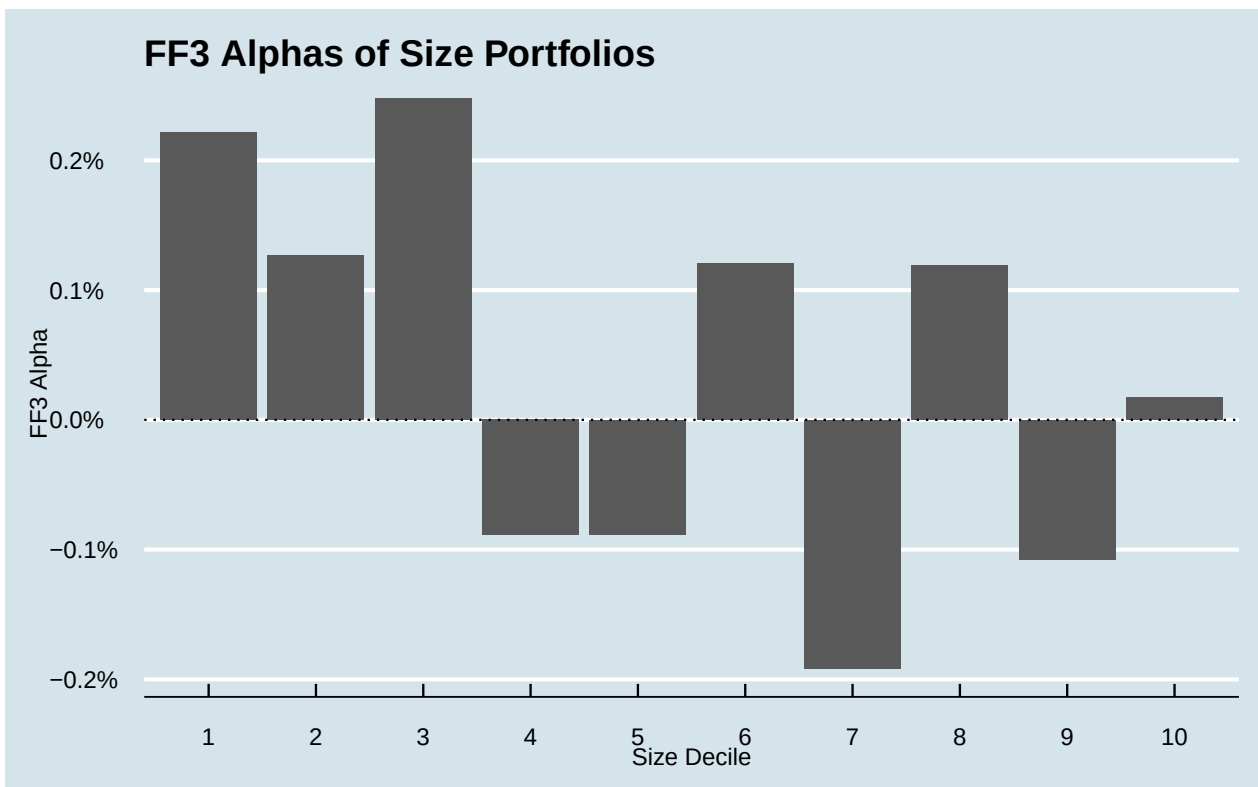
show_table_LFL(tbl_ff3_results)
#> # A tibble: 40 x 7
#> # Groups:   ME_rank1 [10]
#>   ME_rank1 term          estimate std.error statistic    p.value significance
#>   <int> <chr>          <dbl>    <dbl>    <dbl>    <dbl> <chr>
#> 1     1 1 (Intercept) 0.0022160 0.0019246  1.1514 2.5446e- 1 ""
#> 2     1 1 HML        0.037314 0.098061   0.38052 7.0500e- 1 ""
#> 3     1 1 R_Me       0.92776 0.042009  22.085 2.5525e-29 "****"
#> 4     1 1 SMB        1.3610 0.081276  16.746 1.8226e-23 "****"
#> 5     2 2 (Intercept) 0.0012670 0.0015741  0.80490 4.2428e- 1 ""
#> 6     2 2 HML        0.048745 0.080200   0.60779 5.4578e- 1 ""
#> 7     2 2 R_Me       0.94434 0.034357  27.486 3.3917e-34 "****"
#> 8     2 2 SMB        1.2382 0.066472  18.627 1.1450e-25 "****"
#> 9     3 3 (Intercept) 0.0024754 0.0014256  1.7364 8.7992e- 2 "*"
#> 10    3 3 HML        0.26825 0.072634   3.6931 5.0419e- 4 "****"
#> # i 30 more rows
show_table_LFL(tbl_ff3_fit)
#> # A tibble: 10 x 7
#> # Groups:   ME_rank1 [10]
#>   ME_rank1 r.squared adj.r.squared    sigma statistic    p.value  nobs
#>   <int>    <dbl>    <dbl>    <dbl>    <dbl>    <dbl> <int>
#> 1     1 0.91540 0.91087 0.012564 201.99 5.3609e-30 60
#> 2     2 0.93996 0.93674 0.010276 292.24 3.6748e-34 60
#> 3     3 0.94888 0.94615 0.0093063 346.51 4.0778e-36 60
#> 4     4 0.98279 0.98187 0.0056047 1066.2 2.3878e-49 60
#> 5     5 0.97989 0.97881 0.0059918 909.61 1.8726e-47 60
#> 6     6 0.95716 0.95487 0.0085857 417.09 2.9086e-38 60

```

```
#> 7      7  0.96821      0.96651 0.0075647      568.60 6.8782e-42      60
#> 8      8  0.95121      0.94860 0.0093927      363.94 1.1072e-36      60
#> 9      9  0.94869      0.94594 0.010347      345.12 4.5371e-36      60
#> 10     10 0.99594      0.99572 0.0027668      4581.1 6.5209e-67      60
```

```
plot_ff3_alpha <- tbl_ff3_alpha |>
  mutate(ME_rank10 = factor(ME_rank10, levels = 1:10)) |>
  ggplot(aes(x = ME_rank10, y = tbl_ff3_alpha)) +
  geom_col() +
  geom_hline(yintercept = 0, linetype = "dotted") +
  scale_y_continuous(labels = scales::label_percent()) +
  labs(
    x = "Size Decile",
    y = "FF3 Alpha",
    title = "FF3 Alphas of Size Portfolios"
  )
)
```

```
show_plot_LFL(add_lagrange_theme_LFL(plot_ff3_alpha))
```



```
tbl_model_alpha_comparison <- tbl_capm_alpha |>
  select(ME_rank10, tbl_capm_alpha) |>
  left_join(
    tbl_ff3_alpha |>
      select(ME_rank10, tbl_ff3_alpha),
    by = "ME_rank10"
  ) |>
```

```

pivot_longer(
  cols = c(tbl_capm_alpha, tbl_ff3_alpha),
  names_to = "model",
  values_to = "alpha"
)

tbl_model_fit_comparison <- tbl_capm_fit |>
  select(
    ME_rank10,
    CAPM_adjusted_R2 = adj.r.squared
  ) |>
  left_join(
    tbl_ff3_fit |>
      select(
        ME_rank10,
        FF3_adjusted_R2 = adj.r.squared
      ),
    by = "ME_rank10"
  )

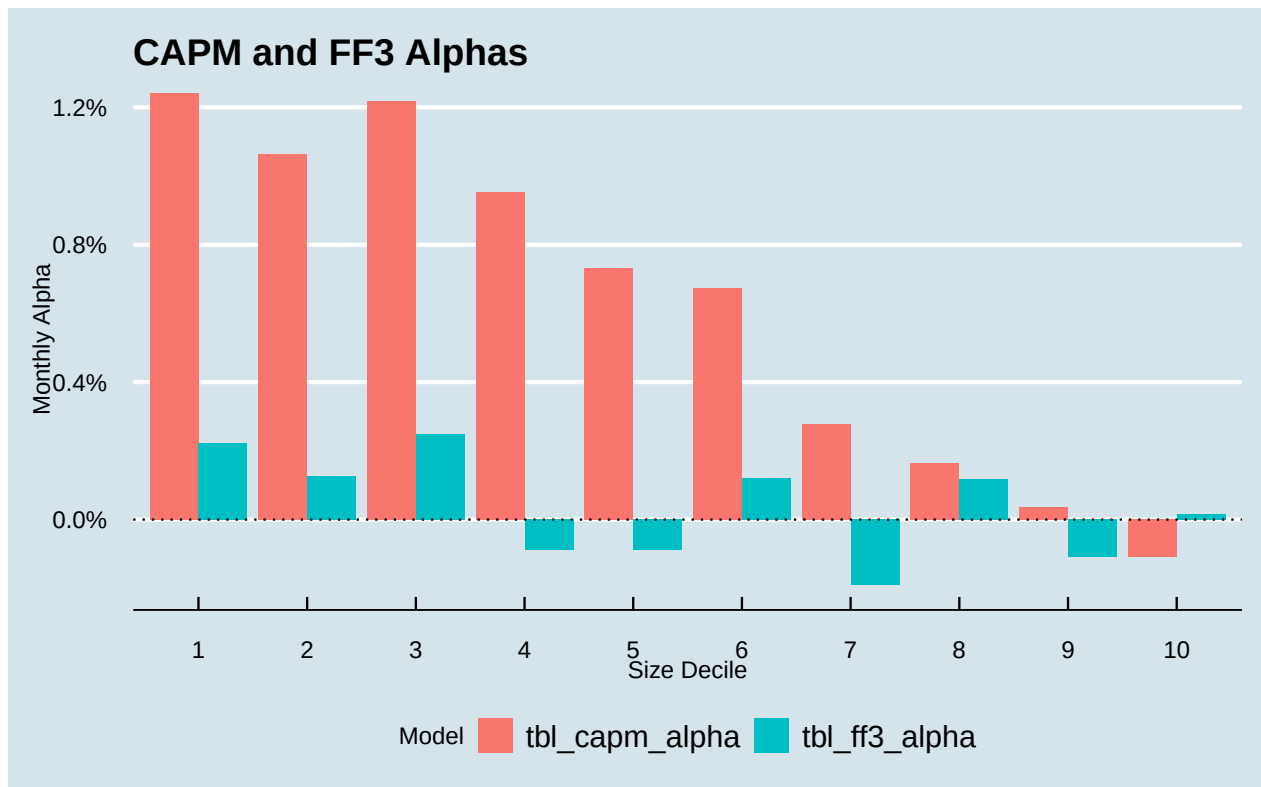
show_table_LFL(tbl_model_alpha_comparison)
#> # A tibble: 20 x 3
#> # Groups:   ME_rank1 [10]
#>   ME_rank1 model          alpha
#>   <int> <chr>          <dbl>
#> 1      1 tbl_capm_alpha 0.012412
#> 2      1 tbl_ff3_alpha 0.0022160
#> 3      2 tbl_capm_alpha 0.010641
#> 4      2 tbl_ff3_alpha 0.0012670
#> 5      3 tbl_capm_alpha 0.012195
#> 6      3 tbl_ff3_alpha 0.0024754
#> 7      4 tbl_capm_alpha 0.0095370
#> 8      4 tbl_ff3_alpha -0.00088852
#> 9      5 tbl_capm_alpha 0.0073101
#> 10     5 tbl_ff3_alpha -0.00088220
#> # i 10 more rows
show_table_LFL(tbl_model_fit_comparison)
#> # A tibble: 10 x 3
#> # Groups:   ME_rank1 [10]
#>   ME_rank1 CAP_R2  FF3_R2
#>   <int>   <dbl>   <dbl>
#> 1      1 0.43109 0.91087
#> 2      2 0.51763 0.93674
#> 3      3 0.63895 0.94615
#> 4      4 0.71461 0.98187

```

```
#> 5      5 0.81104 0.97881
#> 6      6 0.85918 0.95487
#> 7      7 0.89456 0.96651
#> 8      8 0.89902 0.94860
#> 9      9 0.93721 0.94594
#> 10     10 0.98827 0.99572
```

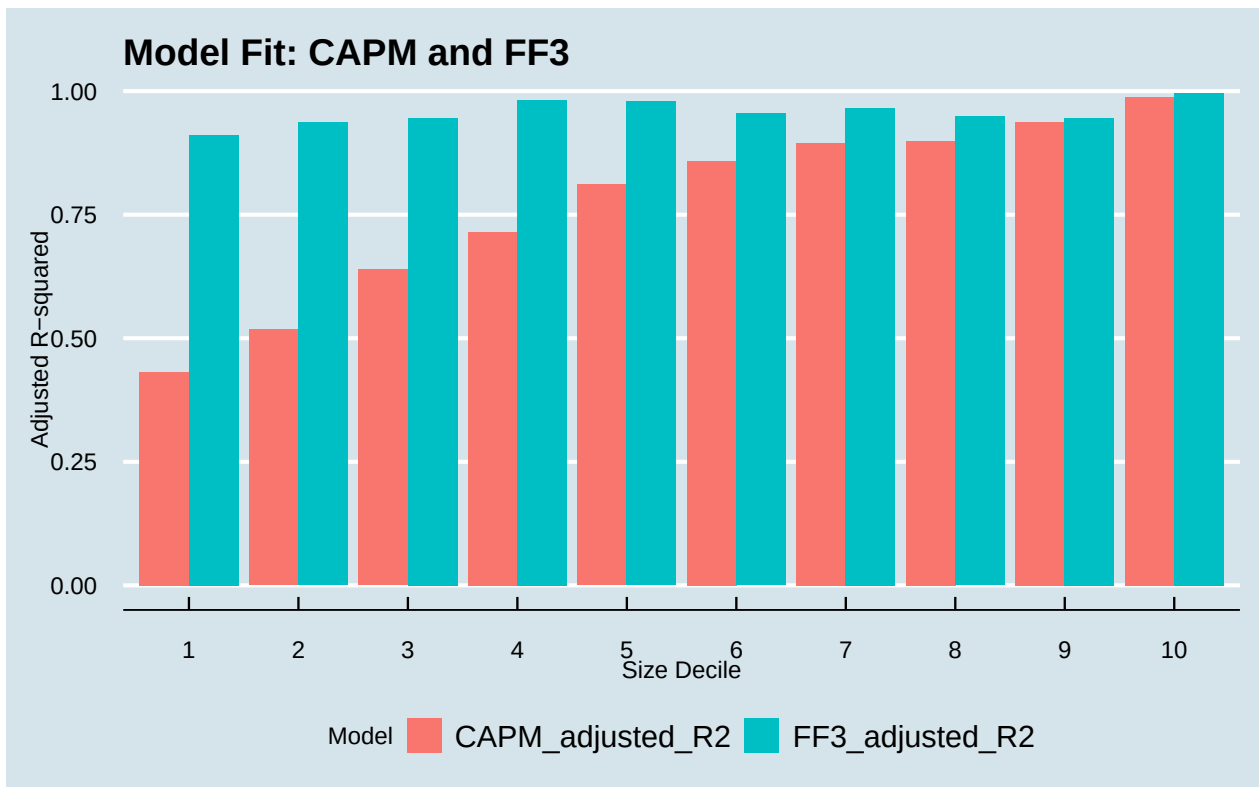
```
plot_model_alpha <- tbl_model_alpha_comparison |>
  mutate(ME_rank10 = factor(ME_rank10, levels = 1:10)) |>
  ggplot(
    aes(
      x = ME_rank10,
      y = alpha,
      fill = model
    )
  ) +
  geom_col(position = "dodge") +
  geom_hline(yintercept = 0, linetype = "dotted") +
  scale_y_continuous(labels = scales::label_percent()) +
  labs(
    x = "Size Decile",
    y = "Monthly Alpha",
    fill = "Model",
    title = "CAPM and FF3 Alphas"
  )

show_plot_LFL(add_lagrange_theme_LFL(plot_model_alpha))
```

```
plot_model_fit <- tbl_model_fit_comparison |>
  pivot_longer(
    cols = c(CAPM_adjusted_R2, FF3_adjusted_R2),
    names_to = "model",
    values_to = "adjusted_R2"
  ) |>
  mutate(ME_rank10 = factor(ME_rank10, levels = 1:10)) |>
  ggplot(
    aes(
      x = ME_rank10,
      y = adjusted_R2,
      fill = model
    )
  ) +
  geom_col(position = "dodge") +
  labs(
    x = "Size Decile",
    y = "Adjusted R-squared",
    fill = "Model",
    title = "Model Fit: CAPM and FF3"
  )

show_plot_LFL(add_lagrange_theme_LFL(plot_model_fit))
```



I-4.8 ファクター・データの保存

第5章以降で使用するファクター・データを、`data/derived/factors/fama_french_factors.parquet` に保存する。

```
tbl_factor_output <- tbl_factor |>
  select(
    month_ID, year, month, month_date,
    R_F, R_M, R_Me, SMB, HML,
    SL, SN, SH, BL, BN, BH,
    market_return_coverage
  )

vec_factor_output_path <-
  write_derived_parquet_file_LFL(
    tbl_factor_output,
    file.path(
      "factors",
      "fama_french_factors.parquet"
    )
  )

tbl_factor_output_check <-
  read_derived_parquet_file_LFL(
    file.path(
```

```

      "factors",
      "fama_french_factors.parquet"
    )
  )

stopifnot(
  nrow(tbl_factor_output_check) == nrow(tbl_factor_output),
  identical(names(tbl_factor_output_check), names(tbl_factor_output))
)

tbl_factor_output_summary <- tibble(
  output_file = relative_path_LFL(vec_factor_output_path),
  rows = nrow(tbl_factor_output_check),
  columns = ncol(tbl_factor_output_check),
  first_month = min(tbl_factor_output_check$month_ID),
  last_month = max(tbl_factor_output_check$month_ID)
)

show_table_LFL(tbl_factor_output_summary)
#> # A tibble: 1 x 5
#>   out_file                      rows columns fir_month las_month
#>   <chr>                        <int>   <int>   <int>    <int>
#> 1 data/derived/factors/fama_french_factors.pa~    60     16     13      72

```

I-4.9 まとめ

本章では、前年度末時価総額を用いて市場ポートフォリオと企業規模ポートフォリオを構築した。また、企業規模と簿価時価比率による独立 2×3 ソートから 6 ポートフォリオを作成し、SMB と HML を計算した。

企業規模 10 分位ポートフォリオに対して CAPM と FF3 モデルを推定し、アルファと調整済み決定係数を比較した。回帰の反復処理には `nest()` と `purrr::map()` を使用しており、明示的なループは使用していない。

作成したファクター・データは次の場所に保存される。

```
data/derived/factors/fama_french_factors.parquet
```

第 5 章では、このファクター・データを利用して、個別企業の資本コスト、共分散構造、平均分散ポートフォリオを分析する。

第 I-5 章資産価格モデルとポートフォリオ最適化

本章では、Fama-French の 3 ファクター・モデルを用いて企業別の期待リターンを推定し、その結果を配当成長率の分析とポートフォリオ最適化へ接続する。

前章までに作成した月次株式データ、年次企業データ、ファクターデータを利用し、企業固有风险と共通ファクターリスクから株式リターンの分散共分散行列を構築する。最後に、平均分散最適化を実行し、目標期待リターンの変化に応じた効率的フロンティアを確認する。

I-5.1 分析対象とデータの接続

本章では、プロジェクト共通の `utility` を読み込み、前章までに作成された派生データを利用する。パッケージの導入方法そのものは扱わず、分析に必要な処理から開始する。

```
tbl_monthly_raw <-  
  read_derived_parquet_file_LFL(  
    file.path(  
      "returns",  
      "monthly_stock_returns.parquet"  
    )  
  )  
tbl_annual_raw <-  
  read_derived_parquet_file_LFL(  
    file.path(  
      "fundamentals",  
      "annual_firm_characteristics.parquet"  
    )  
  )  
tbl_factor <-  
  read_derived_parquet_file_LFL(  
    file.path(  
      "factors",
```

```
    "fama_french_factors.parquet"  
  )  
)
```

企業別の時系列回帰には一定数の観測値が必要である。本章では、2020 年末まで上場が継続し、月次観測値が 36 か月以上存在する企業を推定対象とする。

```
tbl_monthly <- tbl_monthly_raw |>  
  group_by(firm_ID) |>  
  mutate(  
    is_public_2020 = max(month_ID, na.rm = TRUE) == 72,  
    n_observations = n()  
  ) |>  
  ungroup() |>  
  filter(  
    is_public_2020,  
    n_observations >= 36  
  ) |>  
  select(-n_observations)
```

年次データについても、月次データに含まれる企業と年度の組合せに限定する。

```
tbl_annual <- tbl_monthly |>  
  distinct(year, firm_ID) |>  
  left_join(  
    tbl_annual_raw,  
    by = c("year", "firm_ID")  
  )
```

月次株式データとファクターデータを month_ID で結合する。無リスク金利 R_F が月次株式データにも含まれる場合は、重複を避けるため、ファクターデータ側の値を利用する。

```
tbl_monthly_factor <- tbl_monthly |>  
  select(-any_of("R_F")) |>  
  left_join(  
    tbl_factor,  
    by = "month_ID"  
  )
```

I-5.2 FF3 モデルによる期待リターンの推定

企業 i の時点 t における超過リターンを、次の 3 ファクター・モデルで表す。

$$R_{i,t} - R_{F,t} = \beta_{i,M} R_{M,t}^e + \beta_{i,SMB} SMB_t + \beta_{i,HML} HML_t + \varepsilon_{i,t}$$

ここで、 $R_{M,t}^e$ は市場超過リターン、 SMB_t は企業規模ファクター、 HML_t は簿価時価比率ファクターである。本章では、入力データに含まれる超過リターン Re を被説明変数とし、定数項を置かずに企業別回帰を推定する。

```
tbl_ff3_models <- tbl_monthly_factor |>
  select(
    firm_ID,
    Re,
    R_Me,
    SMB,
    HML
  ) |>
  drop_na() |>
  group_by(firm_ID) |>
  nest(tbl_firm = -firm_ID) |>
  mutate(
    list_model = map(
      tbl_firm,
      \(tbl_data) {
        lm(
          Re ~ 0 + R_Me + SMB + HML,
          data = tbl_data
        )
      }
    ),
    tbl_coefficient = map(
      list_model,
      broom::tidy
    )
  ) |>
  ungroup()
```

企業別回帰から得られた係数を横持ち形式に変換し、市場、企業規模、簿価時価比率に対する感応度として整理する。

```
tbl_ff3_loadings <- tbl_ff3_models |>
  select(
    firm_ID,
    tbl_coefficient
  ) |>
  unnest(tbl_coefficient) |>
```

```

select(
  firm_ID,
  term,
  estimate
) |>
pivot_wider(
  names_from = term,
  values_from = estimate
) |>
rename(
  beta_market = R_Me,
  beta_size = SMB,
  beta_value = HML
)

show_table_head_LFL(tbl_ff3_loadings)
#> # A tibble: 6 x 4
#>   firm_ID beta_market beta_size beta_value
#>   <int>      <dbl>      <dbl>      <dbl>
#> 1     1      1.6191      2.6384      0.48152
#> 2     2      1.2405      0.50565     -0.52252
#> 3     3      0.69623      1.1439      0.33640
#> 4     4      0.37560      0.50193     -0.43245
#> 5     5      0.58264      1.5325      1.1194
#> 6     7      0.38022      0.92052     -0.29961

```

月次ファクターリターンの標本平均を 12 倍し、年率換算したファクター・リスクプレミアムを求める。

```

vec_factor_names <- c("R_Me", "SMB", "HML")

tbl_factor_risk_premium <- tbl_factor |>
  summarise(
    across(
      all_of(vec_factor_names),
      \(vec_return) {
        mean(vec_return, na.rm = TRUE) * 12
      }
    )
  ) |>
  pivot_longer(
    cols = everything(),
    names_to = "factor",
    values_to = "annual_risk_premium"
  )

```

```

vec_factor_risk_premium <- tbl_factor_risk_premium |>
  arrange(match(factor, vec_factor_names)) |>
  pull(annual_risk_premium)

names(vec_factor_risk_premium) <- vec_factor_names

show_table_LFL(tbl_factor_risk_premium)
#> # A tibble: 3 x 2
#>   factor ann_premi
#>   <chr>      <dbl>
#> 1 R_Me      0.048258
#> 2 SMB       0.078586
#> 3 HML       0.083395

```

FF3 モデルに基づく企業 i の期待超過リターンは、ファクター・ローディングとファクター・リスクプレミアムの積和として求められる。

$$E[R_i - R_F] = \beta_{i,M} E[R_M^e] + \beta_{i,SMB} E[SMB] + \beta_{i,HML} E[HML]$$

本章では、この期待超過リターンを企業の株式資本コストとして利用する。

```

tbl_cost_of_equity <- tbl_ff3_loadings |>
  mutate(
    cost_of_equity =
      beta_market * vec_factor_risk_premium[["R_Me"]] +
      beta_size * vec_factor_risk_premium[["SMB"]] +
      beta_value * vec_factor_risk_premium[["HML"]]
  )

show_table_head_LFL(tbl_cost_of_equity)
#> # A tibble: 6 x 5
#>   firm_ID beta_market beta_size beta_value cost_of_equity
#>   <int>      <dbl>      <dbl>      <dbl>      <dbl>
#> 1      1      1.6191      2.6384      0.48152      0.32563
#> 2      2      1.2405      0.50565     -0.52252      0.056027
#> 3      3      0.69623      1.1439      0.33640      0.15155
#> 4      4      0.37560      0.50193     -0.43245      0.021505
#> 5      5      0.58264      1.5325      1.1194      0.24190
#> 6      7      0.38022      0.92052     -0.29961      0.065702

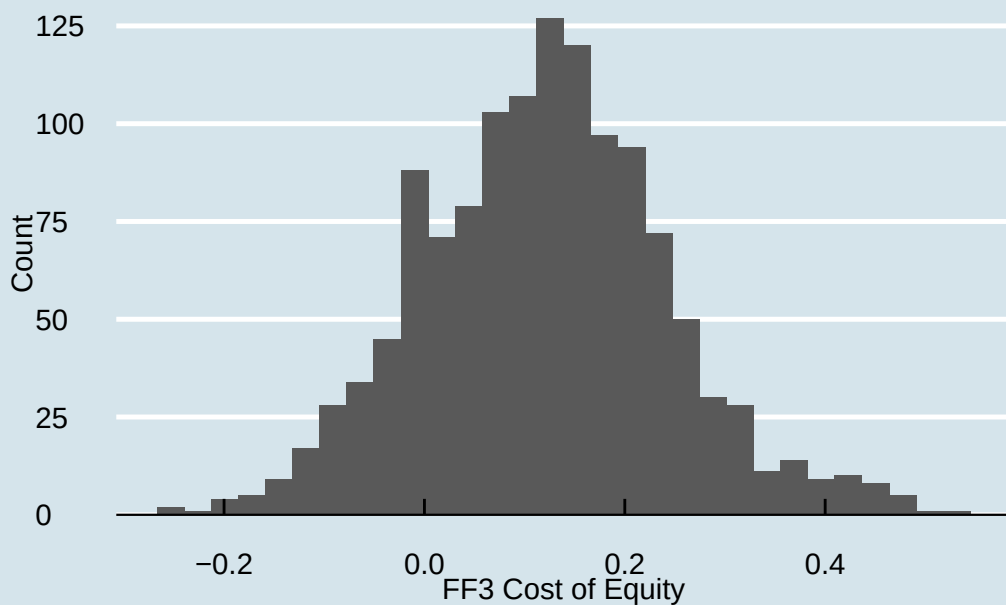
```


I-5.3 期待リターンと配当成長率

```
plot_cost_of_equity_distribution <- tbl_cost_of_equity |>
  ggplot(
    aes(x = cost_of_equity)
  ) +
  geom_histogram(bins = 30) +
  labs(
    x = "FF3 Cost of Equity",
    y = "Count"
  ) +
  scale_y_continuous(
    expand = expansion(mult = c(0, 0.05))
  )

plot_cost_of_equity_distribution <- add_lagrange_theme_LFL(
  plot_cost_of_equity_distribution
)

show_plot_LFL(plot_cost_of_equity_distribution)
```



```
tbl_firm_characteristics_2020 <- tbl_annual |>
  arrange(
    firm_ID,
    year
  ) |>
  group_by(firm_ID) |>
```

```

mutate(
  lagged_ME = lag(ME)
) |>
ungroup() |>
filter(year == 2020) |>
mutate(
  size_group = ntile(lagged_ME, 2),
  size_group = factor(
    size_group,
    levels = c(1, 2),
    labels = c("Small", "Large")
  )
) |>
inner_join(
  tbl_cost_of_equity,
  by = "firm_ID"
) |>
drop_na(
  lagged_ME,
  cost_of_equity,
  size_group
)

```

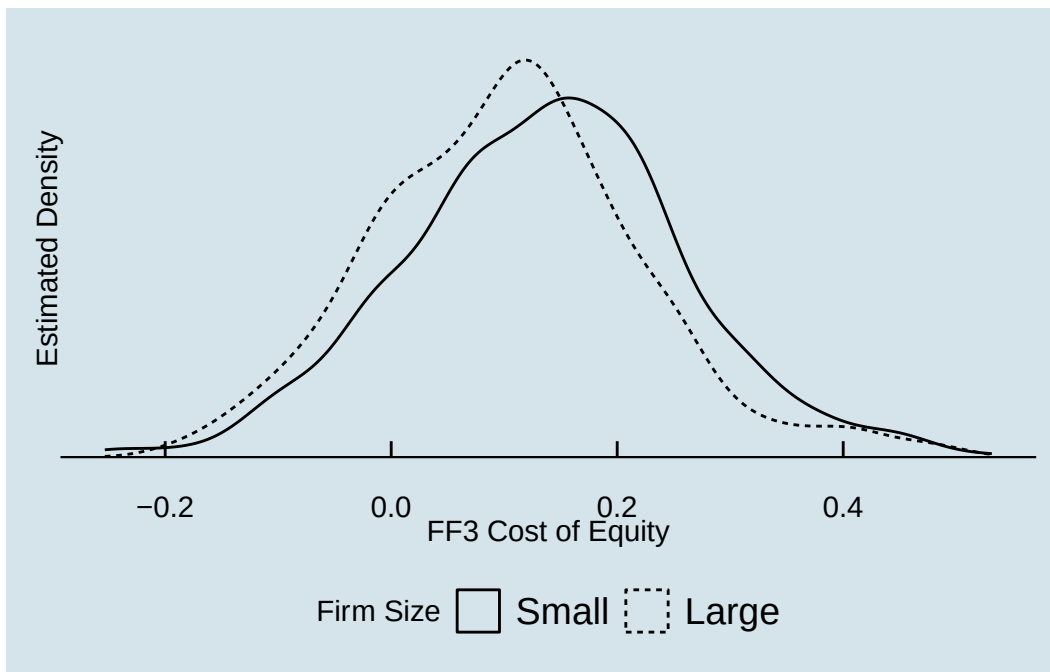
```

plot_cost_of_equity_by_size <- tbl_firm_characteristics_2020 |>
ggplot(
  aes(
    x = cost_of_equity,
    linetype = size_group
  )
) +
geom_density() +
labs(
  x = "FF3 Cost of Equity",
  y = "Estimated Density",
  linetype = "Firm Size"
) +
scale_y_continuous(
  expand = expansion(mult = c(0, 0.05)),
  breaks = NULL
)

plot_cost_of_equity_by_size <- add_lagrange_theme_LFL(
  plot_cost_of_equity_by_size
)

```

```
show_plot_LFL(plot_cost_of_equity_by_size)
```



```
tbl_dividend_yield_2020 <- tbl_monthly |>
  filter(year == 2020) |>
  group_by(firm_ID) |>
  summarise(
    annual_dividend = sum(
      DPS * shares_outstanding,
      na.rm = TRUE
    ),
    latest_ME = ME[which.max(month_ID)],
    .groups = "drop"
  ) |>
  mutate(
    dividend_yield = annual_dividend / latest_ME
  ) |>
  filter(
    is.finite(dividend_yield),
    dividend_yield > 0
  )

show_table_head_LFL(tbl_dividend_yield_2020)
#> # A tibble: 6 x 4
#>   firm_ID annual_dividend latest_ME dividend_yield
#>   <int>      <dbl>      <dbl>      <dbl>
#> 1     1    325472000  9708946000    0.033523
#> 2     2     37536000  5194044000    0.0072267
#> 3     3    4660110000 214032195000    0.021773
#> 4     4    101352000  8652927000    0.011713
```

```
#> 5      5      745488000 235170402000      0.0031700
#> 6      7      64848000  15909376000      0.0040761
```

株式資本コストを r_i 、配当利回りを y_i とすると、本章で用いる潜在配当成長率 g_i は次式から逆算される。

$$g_i = \frac{r_i - y_i}{1 + y_i}$$

```
tbl_implied_dividend_growth <- tbl_dividend_yield_2020 |>
  inner_join(
    tbl_cost_of_equity |>
      select(
        firm_ID,
        cost_of_equity
      ),
    by = "firm_ID"
  ) |>
  mutate(
    implied_dividend_growth =
      (cost_of_equity - dividend_yield) /
      (1 + dividend_yield)
  )

show_table_head_LFL(tbl_implied_dividend_growth)
#> # A tibble: 6 x 6
#>   firm_ID annual_dividend latest_ME dividend_yield cost_of_equity
#>   <int>      <dbl>      <dbl>      <dbl>      <dbl>
#> 1     1      325472000  9708946000      0.033523      0.32563
#> 2     2      37536000  5194044000      0.0072267      0.056027
#> 3     3     4660110000 214032195000      0.021773      0.15155
#> 4     4     101352000  8652927000      0.011713      0.021505
#> 5     5      745488000 235170402000      0.0031700      0.24190
#> 6     7      64848000  15909376000      0.0040761      0.065702
#> # i 1 more variable: implied_dividend_growth <dbl>
```

I-5.4 ファクターモデルによるリスク推定

```
tbl_factor_covariance <- compute_covariance_table_LFL(
  tbl_data = tbl_factor,
  vec_variables = vec_factor_names,
  annualization_factor = 12
)
```

```

)

matrix_factor_covariance <- convert_covariance_table_to_matrix_LFL(
  tbl_factor_covariance
)

show_table_LFL(tbl_factor_covariance)
#> # A tibble: 9 x 3
#>   var_row var_column covariance
#>   <chr>    <chr>         <dbl>
#> 1 R_Me    R_Me             0.020665
#> 2 R_Me    SMB             -0.0039063
#> 3 R_Me    HML              0.0015310
#> 4 SMB     R_Me             -0.0039063
#> 5 SMB     SMB              0.0062694
#> 6 SMB     HML             -0.0018855
#> 7 HML     R_Me              0.0015310
#> 8 HML     SMB             -0.0018855
#> 9 HML     HML              0.0039129

```

```

n_portfolio_firms <- 100L

tbl_investment_universe <- tbl_annual |>
  filter(year == 2020) |>
  inner_join(
    tbl_cost_of_equity |>
      select(
        firm_ID,
        cost_of_equity
      ),
    by = "firm_ID"
  ) |>
  slice_max(
    order_by = ME,
    n = n_portfolio_firms,
    with_ties = FALSE
  ) |>
  arrange(desc(ME)) |>
  select(
    firm_ID,
    ME,
    cost_of_equity
  )

vec_investment_firm_ids <- tbl_investment_universe |>

```

```
pull(firm_ID)

n_portfolio_firms <- length(vec_investment_firm_ids)

show_table_head_LFL(tbl_investment_universe)
#> # A tibble: 6 x 3
#>   firm_ID      ME cost_of_equity
#>   <int>    <dbl>         <dbl>
#> 1     790 2.7339e13         0.061424
#> 2     987 5.4441e12        -0.071023
#> 3    1237 5.2764e12         0.091188
#> 4    1057 5.1050e12        -0.058108
#> 5     988 4.9127e12        -0.10059
#> 6    1400 3.7669e12         0.20979
```

```
tbl_idiosyncratic_variance <- tbl_monthly_factor |>
  filter(
    firm_ID %in% vec_investment_firm_ids
  ) |>
  inner_join(
    tbl_ff3_loadings,
    by = "firm_ID"
  ) |>
  mutate(
    fitted_return =
      R_F +
      beta_market * R_Me +
      beta_size * SMB +
      beta_value * HML,
    residual_return = R - fitted_return
  ) |>
  group_by(firm_ID) |>
  summarise(
    idiosyncratic_variance =
      12 * var(
        residual_return,
        na.rm = TRUE
      ),
    .groups = "drop"
  ) |>
  right_join(
    tbl_investment_universe |>
      select(firm_ID),
    by = "firm_ID"
  ) |>
```

```

arrange(
  match(
    firm_ID,
    vec_investment_firm_ids
  )
)

show_table_head_LFL(tbl_idiosyncratic_variance)
#> # A tibble: 6 x 2
#>   firm_ID idiosyncratic_variance
#>   <int>          <dbl>
#> 1     790          0.055478
#> 2     987          0.036155
#> 3    1237          0.053695
#> 4    1057          0.045481
#> 5     988          0.12455
#> 6    1400          0.094833

```

企業別ファクター・ローディングを並べた行列を B 、企業固有リスクの対角行列を Σ_ϵ とすると、株式リターンの分散共分散行列は次式で構築できる。

$$\Sigma = B\Sigma_F B^\top + \Sigma_\epsilon$$

```

tbl_portfolio_loadings <- tbl_investment_universe |>
  select(firm_ID) |>
  left_join(
    tbl_ff3_loadings,
    by = "firm_ID"
  )

matrix_factor_loadings <- tbl_portfolio_loadings |>
  select(
    beta_market,
    beta_size,
    beta_value
  ) |>
  as.matrix()

vec_idiosyncratic_variance <- tbl_idiosyncratic_variance |>
  pull(idiosyncratic_variance)

matrix_idiosyncratic_covariance <- diag(
  vec_idiosyncratic_variance

```

```
)

matrix_return_covariance <-
  matrix_factor_loadings %*%
  matrix_factor_covariance %*%
  t(matrix_factor_loadings) +
  matrix_idiosyncratic_covariance

rownames(matrix_return_covariance) <- vec_investment_firm_ids
colnames(matrix_return_covariance) <- vec_investment_firm_ids
```

```
vec_expected_returns <- tbl_investment_universe |>
  pull(cost_of_equity)

names(vec_expected_returns) <- vec_investment_firm_ids

tibble(
  firm_ID = vec_investment_firm_ids,
  expected_return = vec_expected_returns
) |>
  show_table_head_LFL()

#> # A tibble: 6 x 2
#>   firm_ID expected_return
#>   <int>         <dbl>
#> 1     790         0.061424
#> 2     987        -0.071023
#> 3    1237         0.091188
#> 4    1057        -0.058108
#> 5     988        -0.10059
#> 6    1400         0.20979
```

I-5.5 平均分散最適化

目標期待リターンを μ_p^* とすると、ポートフォリオ・ウェイト w は次の問題から求められる。

$$\min_w \frac{1}{2} w^T \Sigma w$$

$$w^T \mu = \mu_p^*$$

$$w^T \mathbf{1} = 1$$

本章の基本設定では、空売りを禁止する不等式制約は追加しない。

```
target_return <- 0.10

list_mean_variance_portfolio <- compute_mean_variance_portfolio_LFL(
  matrix_covariance = matrix_return_covariance,
  vec_expected_returns = vec_expected_returns,
  target_return = target_return
)
```

```
tbl_optimal_portfolio <- create_portfolio_weight_table_LFL(
  list_optimization = list_mean_variance_portfolio,
  vec_asset_ids = vec_investment_firm_ids,
  target_return = target_return
) |>
  arrange(
    desc(abs(weight))
  )
```

```
show_table_head_LFL(
  tbl_optimal_portfolio,
  n = 10
)
```

```
#> # A tibble: 10 x 3
#>   asset_ID    weight target_return
#>   <int>    <dbl>         <dbl>
#> 1     208 -0.16960           0.1
#> 2    1473  0.075792          0.1
#> 3     727  0.075617          0.1
#> 4    1253  0.073272          0.1
#> 5     987  0.068849          0.1
#> 6     829  0.062825          0.1
#> 7     409 -0.059685          0.1
#> 8      33  0.057845          0.1
#> 9     765  0.057677          0.1
#> 10    709  0.057313          0.1
```

```
tbl_portfolio_summary <- compute_portfolio_summary_LFL(
  list_optimization = list_mean_variance_portfolio,
  target_return = target_return
) |>
  mutate(
```

```

    realized_expected_return = sum(
      list_mean_variance_portfolio$solution *
      vec_expected_returns
    ),
    return_constraint_error =
      realized_expected_return -
      target_return,
    weight_constraint_error =
      weight_sum -
      1
  )

show_table_LFL(tbl_portfolio_summary)
#> # A tibble: 1 x 8
#>   tar_retur por_retur por_varia por_risk wei_sum rea_retur   ret_error wei_error
#>   <dbl>     <dbl>     <dbl>   <dbl>   <dbl>   <dbl>     <dbl>   <dbl>
#> 1     0.1       0.1 0.0069899 0.083606     1     0.1 -2.7756e-17     0

```

I-5.6 効率的フロンティア

```

vec_target_returns <- seq(
  from = -0.10,
  to = 0.40,
  length.out = 100
)

```

```

tbl_efficient_frontier <- compute_efficient_frontier_LFL(
  matrix_covariance = matrix_return_covariance,
  vec_expected_returns = vec_expected_returns,
  vec_target_returns = vec_target_returns
)

show_table_head_LFL(
  tbl_efficient_frontier |>
  select(
    target_return,
    portfolio_risk
  )
)
#> # A tibble: 6 x 2
#>   target_return portfolio_risk
#>   <dbl>         <dbl>

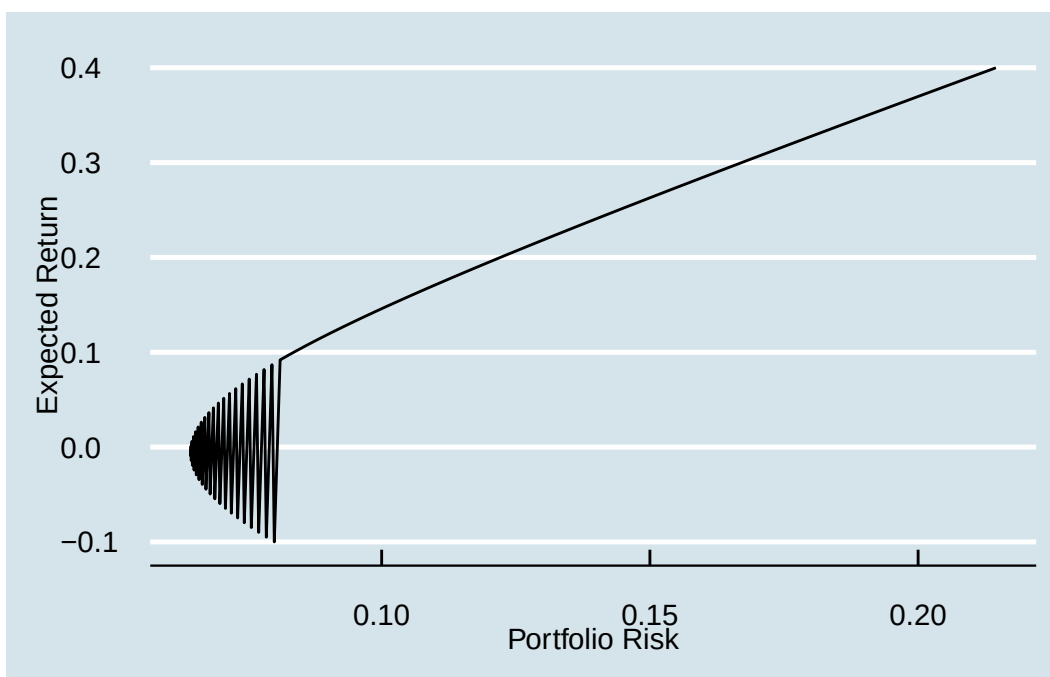
```

```
#> 1      -0.1      0.079982
#> 2      -0.094949 0.078497
#> 3      -0.089899 0.077068
#> 4      -0.084848 0.075697
#> 5      -0.079798 0.074389
#> 6      -0.074747 0.073145
```

```
plot_efficient_frontier <- tbl_efficient_frontier |>
  ggplot(
    aes(
      x = portfolio_risk,
      y = target_return
    )
  ) +
  geom_line() +
  labs(
    x = "Portfolio Risk",
    y = "Expected Return"
  )

plot_efficient_frontier <- add_lagrange_theme_LFL(
  plot_efficient_frontier
)

show_plot_LFL(plot_efficient_frontier)
```



I-5.7 まとめ

本章では、Fama-French の 3 ファクター・モデルを企業別に推定し、市場、企業規模、簿価時価比率に対するファクター・ローディングを求めた。ファクター・リスクプレミアムとの積和から株式資本コストを計算し、配当利回りと組み合わせることで潜在配当成長率を推定した。

さらに、共通ファクターの分散共分散行列と企業固有リスクから株式リターンの分散共分散行列を構築した。これを平均分散最適化へ接続し、単一の目標期待リターンに対応する最適ポートフォリオと、複数の目標期待リターンからなる効率的フロンティアを確認した。

第 I-6 章 イベント・スタディ

本章では、企業の決算発表を対象としてイベントスタディを行う。イベントスタディは、企業イベントの前後に観察された株式リターンから、通常時に予想されるリターンを差し引き、イベント固有の株価反応を測定する方法である。

本章では、次の処理を順番に行う。

1. 個別株リターン、市場リターン、イベント情報を読み込む
2. 決算サプライズを計算し、イベントを 5 群に分類する
3. 発表日の翌取引日をイベント日 0 として設定する
4. 推定期間を使ってイベントごとのマーケット・モデルを推定する
5. 異常リターンと累積異常リターンを計算する
6. AAR、CAAR、複数のイベント・ウィンドウについて統計的検定を行う
7. 決算サプライズと短期 CAR の関係を確認する
8. イベント単位と日次集計の結果を保存する

I-6.1 イベントスタディの基本設計

企業 i の日 t における実現リターンを $R_{i,t}$ 、市場リターンを $R_{M,t}$ とする。推定期間では、次のマーケット・モデルを推定する。

$$R_{i,t} = \alpha_i + \beta_i R_{M,t} + \varepsilon_{i,t}$$

推定された通常リターンは次式で表される。

$$\widehat{R}_{i,t} = \widehat{\alpha}_i + \widehat{\beta}_i R_{M,t}$$

異常リターン $AR_{i,t}$ は、実現リターンと通常リターンの差である。

$$AR_{i,t} = R_{i,t} - \widehat{R}_{i,t}$$

イベント期間の開始日から日 t までの累積異常リターン $CAR_{i,t}$ は、異常リターンを累積して計算する。

$$CAR_{i,t} = \sum_{\tau=T_1}^t AR_{i,\tau}$$

本章では、イベント日 0 を基準として次の期間を使用する。

期間	相対取引日	用途
推定期間	-130 日から-31 日	マーケット・モデルの推定
事前イベント期間	-30 日から-1 日	発表前の株価反応
イベント日	0 日	発表後最初の取引日
事後イベント期間	1 日から 30 日	発表後の株価反応

推定期間は 100 取引日、イベント期間は 61 取引日である。マーケット・モデルの推定には、最低 80 観測を必要とする。

```
estimation_start <- -130L
estimation_end <- -31L
event_start <- -30L
event_end <- 30L
minimum_estimation_observations <- 80L

tbl_window_parameter <- tribble(
  ~parameter, ~value,
  "estimation_start", estimation_start,
  "estimation_end", estimation_end,
  "event_start", event_start,
  "event_end", event_end,
  "minimum_estimation_observations", minimum_estimation_observations
)

tbl_window_parameter
#> # A tibble: 5 x 2
#>   parameter      value
#>   <chr>         <int>
#> 1 estimation_start    -130
#> 2 estimation_end      -31
#> 3 event_start        -30
```

```
#> 4 event_end 30
#> 5 minimum_estimation_observations 80
```

プロジェクトルートとデータディレクトリを確認する。

```
vec_project_candidate <- unique(c(
  Sys.getenv("QUARTO_PROJECT_DIR"),
  getwd(),
  dirname(getwd())
))

vec_project_candidate <- vec_project_candidate[
  nzchar(vec_project_candidate)
]

vec_project_match <- vec_project_candidate[
  file.exists(file.path(vec_project_candidate, "_quarto.yml"))
]

if (length(vec_project_match) == 0L) {
  stop("LagrangeFinanceのプロジェクトルートを確認できません。")
}

project_dir <- normalizePath(
  vec_project_match[[1]],
  winslash = "/",
  mustWork = TRUE
)

data_dir <- get_data_dir_LFL()

raw_event_data_dir <- file.path(
  data_dir,
  "raw",
  "events"
)

derived_event_data_dir <- file.path(
  data_dir,
  "derived",
  "events"
)

dir.create(
  derived_event_data_dir,
```

```

    recursive = TRUE,
    showWarnings = FALSE
  )

  tibble(
    directory = c(
      "project",
      "raw_events",
      "derived_events"
    ),
    path = base::c(
      ".",
      relative_path_LFL(raw_event_data_dir),
      relative_path_LFL(derived_event_data_dir)
    )
  )
#> # A tibble: 3 x 2
#>   directory      path
#>   <chr>         <chr>
#> 1 project      .
#> 2 raw_events   data/raw/events
#> 3 derived_events data/derived/events

```

I-6.2 入力データの読み込みと検証

本章では次の 3 ファイルを使用する。

```

data/raw/events/firm_returns.parquet
data/raw/events/market_returns.parquet
data/raw/events/earnings_events.parquet

```

ファイル名はデータの内容を表す名称に統一し、Parquet 形式で読み込む。

```

vec_input_file <- c(
  return = file.path(
    "raw",
    "events",
    "firm_returns.parquet"
  ),
  market = file.path(
    "raw",
    "events",
    "market_returns.parquet"
  )
)

```



```

),
event = file.path(
  "raw",
  "events",
  "earnings_events.parquet"
)
)

vec_input_path <- purrr::map_chr(
  vec_input_file,
  find_data_file_LFL
)

tibble(
  data_name = names(vec_input_path),
  file_name = basename(vec_input_path),
  path = relative_path_LFL(base::unname(vec_input_path))
)

#> # A tibble: 3 x 3
#>   data_name file_name      path
#>   <chr>      <chr>      <chr>
#> 1 return    firm_returns.parquet data/raw/events/firm_returns.parquet
#> 2 market    market_returns.parquet data/raw/events/market_returns.parquet
#> 3 event     earnings_events.parquet data/raw/events/earnings_events.parquet

```

```

tbl_return_raw <-
  read_parquet_file_LFL(
    vec_input_file[["return"]]
  )

tbl_market_raw <-
  read_parquet_file_LFL(
    vec_input_file[["market"]]
  )

tbl_event_raw <-
  read_parquet_file_LFL(
    vec_input_file[["event"]]
  )

```

```

validate_required_columns_LFL(
  tbl_return_raw,
  c("date", "firm_ID", "R"),
  "tbl_return_raw"
)

```

```
)  
  
validate_required_columns_LFL(  
  tbl_market_raw,  
  c("date", "R_M"),  
  "tbl_market_raw"  
)
```

```
validate_required_columns_LFL(  
  tbl_event_raw,  
  c(  
    "event_date",  
    "firm_ID",  
    "earnings_forecast",  
    "realized_earnings",  
    "lagged_ME"  
  ),  
  "tbl_event_raw"  
)
```

```
tbl_return <- tbl_return_raw |>  
  transmute(  
    date = as.Date(date),  
    firm_ID = as.integer(firm_ID),  
    R = as.numeric(R)  
  ) |>  
  arrange(  
    firm_ID,  
    date  
  )
```

```
tbl_market <- tbl_market_raw |>  
  transmute(  
    date = as.Date(date),  
    R_M = as.numeric(R_M)  
  ) |>  
  arrange(date)
```

```
tbl_event <- tbl_event_raw |>  
  transmute(  
    event_date = as.Date(event_date),  
    firm_ID = as.integer(firm_ID),  
    earnings_forecast = as.numeric(  
      earnings_forecast  
    ),  
  )
```

```
    realized_earnings = as.numeric(
      realized_earnings
    ),
    lagged_ME = as.numeric(lagged_ME)
  ) |>
  arrange(
    event_date,
    firm_ID
  )
```

```
validate_unique_keys_LFL(
  tbl_return,
  c("firm_ID", "date"),
  "tbl_return"
)
```

```
validate_unique_keys_LFL(
  tbl_market,
  "date",
  "tbl_market"
)
```

```
validate_unique_keys_LFL(
  tbl_event,
  c("firm_ID", "event_date"),
  "tbl_event"
)
```

```
tbl_missing_summary <- bind_rows(
  summarize_missing_values_LFL(
    tbl_return,
    "return"
  ),
  summarize_missing_values_LFL(
    tbl_market,
    "market"
  ),
  summarize_missing_values_LFL(
    tbl_event,
    "event"
  )
)
```

```
tbl_missing_summary
```

```
#> # A tibble: 10 x 5
#>   data_name observations missing_ratio variable      missing_values
#>   <chr>          <int>          <dbl> <chr>          <int>
#> 1 return          1610400            0 date              0
#> 2 return          1610400            0 firm_ID           0
#> 3 return          1610400            0 R                 0
#> 4 market           1220            0 date              0
#> 5 market           1220            0 R_M               0
#> 6 event            3960            0 event_date        0
#> 7 event            3960            0 firm_ID           0
#> 8 event            3960            0 earnings_forecast 0
#> 9 event            3960            0 realized_earnings 0
#> 10 event           3960            0 lagged_ME         0
```

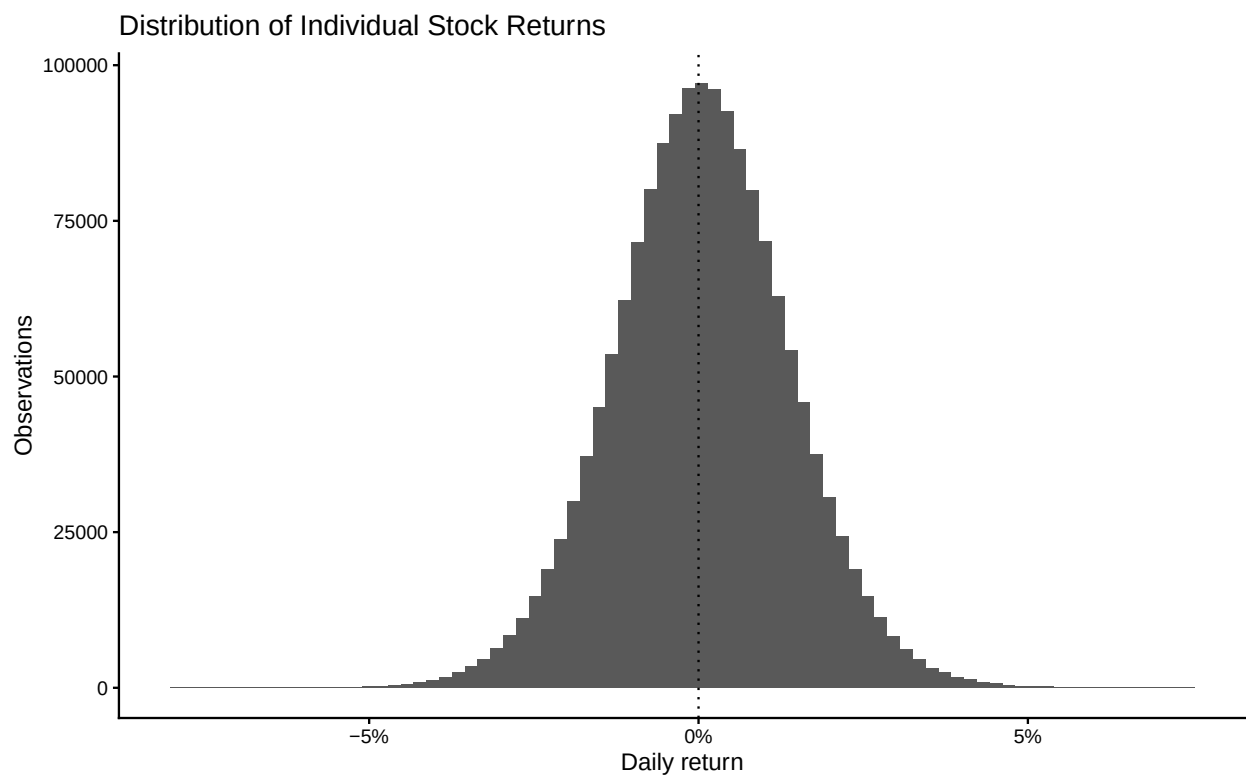
```
tbl_input_overview <- tribble(
  ~data_name, ~observations, ~firms, ~first_date, ~last_date,
  "individual returns",
  nrow(tbl_return),
  n_distinct(tbl_return$firm_ID),
  min(tbl_return$date, na.rm = TRUE),
  max(tbl_return$date, na.rm = TRUE),
  "market returns",
  nrow(tbl_market),
  NA_integer_,
  min(tbl_market$date, na.rm = TRUE),
  max(tbl_market$date, na.rm = TRUE),
  "events",
  nrow(tbl_event),
  n_distinct(tbl_event$firm_ID),
  min(tbl_event$event_date, na.rm = TRUE),
  max(tbl_event$event_date, na.rm = TRUE)
)
```

```
tbl_input_overview
#> # A tibble: 3 x 5
#>   data_name      observations firms first_date last_date
#>   <chr>          <int> <int> <date>    <date>
#> 1 individual returns    1610400  1320 2016-01-04 2020-12-30
#> 2 market returns        1220    NA 2016-01-04 2020-12-30
#> 3 events                3960  1320 2017-01-04 2020-04-07
```

```
plot_return_distribution <- tbl_return |>
  filter(is.finite(R)) |>
  ggplot(aes(x = R)) +
```

```
geom_histogram(bins = 80) +  
geom_vline(  
  xintercept = 0,  
  linetype = "dotted"  
) +  
scale_x_continuous(  
  labels = scales::label_percent()  
) +  
labs(  
  x = "Daily return",  
  y = "Observations",  
  title = "Distribution of Individual Stock Returns"  
) +  
theme_classic()
```

plot_return_distribution



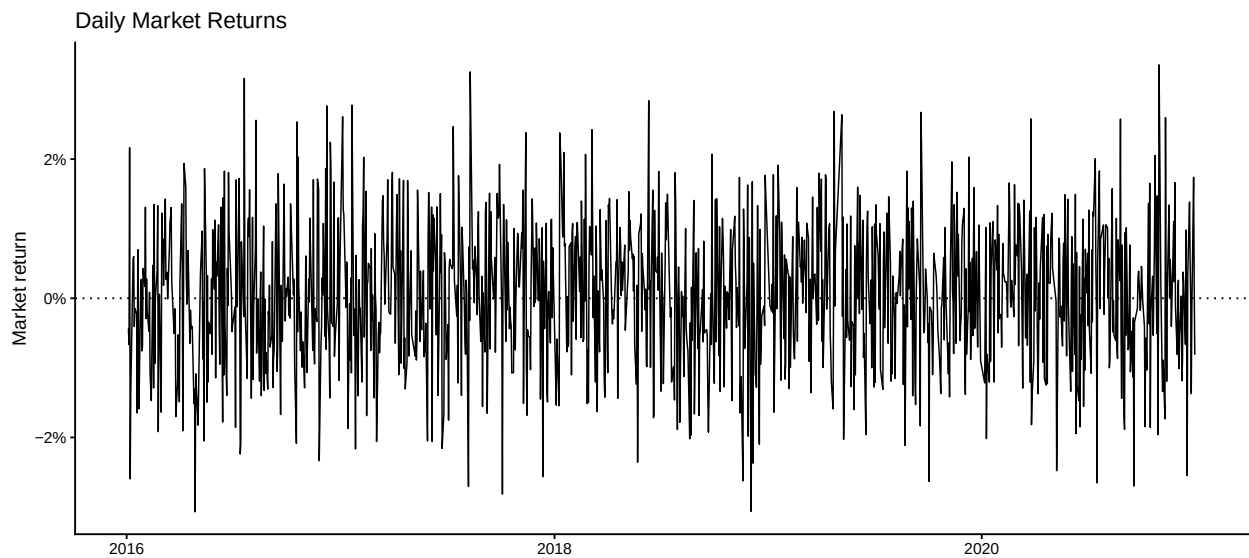
```
plot_market_return_series <- tbl_market |>  
ggplot(aes(x = date, y = R_M)) +  
geom_line(linewidth = 0.4) +  
geom_hline(  
  yintercept = 0,  
  linetype = "dotted"  
) +  
scale_y_continuous(  
  labels = scales::label_percent()  
)
```

```

) +
labs(
  x = NULL,
  y = "Market return",
  title = "Daily Market Returns"
) +
theme_classic()

plot_market_return_series

```



I-6.3 決算サプライズとイベント分類

決算サプライズは、実現利益と予想利益の差を期首時価総額で標準化して計算する。

$$Surprise_i = \frac{RealizedEarnings_i - ForecastEarnings_i}{ME_{i,-1}}$$

正の値は実現利益が予想を上回ったことを、負の値は予想を下回ったことを表す。

```

tbl_event <- compute_earnings_surprise_LFL(
  tbl_event = tbl_event,
  number_of_groups = 5L
)

tbl_event |>
  select(
    event_ID,
    firm_ID,
    event_date,

```

```

    event_year,
    earnings_surprise,
    event_strength
  ) |>
  slice_head(n = 20)
#> # A tibble: 20 x 6
#>   event_ID firm_ID event_date event_year earnings_surprise event_strength
#>   <int>    <int> <date>      <dbl>          <dbl> <ord>
#> 1         1      39 2017-01-04    2017        -0.0081327 1
#> 2         2      57 2017-01-04    2017        -0.010049 1
#> 3         3      73 2017-01-04    2017        -0.0020232 4
#> 4         4     109 2017-01-04    2017        -0.0043961 3
#> 5         5     111 2017-01-04    2017        -0.0044873 3
#> 6         6     231 2017-01-04    2017        -0.014399 1
#> 7         7     944 2017-01-04    2017        -0.0068979 2
#> 8         8    1227 2017-01-04    2017        -0.0011248 5
#> 9         9     422 2017-01-05    2017        -0.0048408 3
#> 10        10    1203 2017-01-05    2017        -0.00086568 5
#> 11        11     346 2017-01-06    2017        -0.0058582 2
#> 12        12     590 2017-01-06    2017        -0.010344 1
#> 13        13     644 2017-01-06    2017        -0.0064889 2
#> 14        14    1153 2017-01-06    2017        -0.0021003 4
#> 15        15     191 2017-01-10    2017        -0.0034658 4
#> 16        16     353 2017-01-10    2017        -0.010249 1
#> 17        17     566 2017-01-10    2017        -0.0043876 3
#> 18        18     322 2017-01-11    2017        -0.0046897 3
#> 19        19    1050 2017-01-11    2017        -0.0024991 4
#> 20        20     426 2017-01-12    2017        -0.0010089 5

```

```

tbl_event_strength_summary <- tbl_event |>
  group_by(
    event_year,
    event_strength
  ) |>
  summarise(
    events = n(),
    mean_surprise = mean(
      earnings_surprise,
      na.rm = TRUE
    ),
    median_surprise = median(
      earnings_surprise,
      na.rm = TRUE
    ),
    .groups = "drop"
  )

```

```
) |>
  arrange(
    event_year,
    event_strength
  )
```

```
tbl_event_strength_summary
```

```
#> # A tibble: 20 x 5
```

```
#>   event_year event_strength events mean_surprise median_surprise
#>   <dbl> <ord>          <int>      <dbl>          <dbl>
#> 1  2017 1             214    -0.010271      -0.0098319
#> 2  2017 2             214    -0.0064294     -0.0063866
#> 3  2017 3             214    -0.0044249     -0.0043693
#> 4  2017 4             213    -0.0028679     -0.0028917
#> 5  2017 5             213    -0.00062223    -0.00081805
#> 6  2018 1             263    -0.0099427     -0.0092589
#> 7  2018 2             263    -0.0066866     -0.0066374
#> 8  2018 3             263    -0.0048770     -0.0048670
#> 9  2018 4             263    -0.0031921     -0.0031796
#> 10 2018 5             263    -0.00098501    -0.0011702
#> 11 2019 1             262    -0.0098017     -0.0094002
#> 12 2019 2             262    -0.0064618     -0.0064241
#> 13 2019 3             261    -0.0045936     -0.0045890
#> 14 2019 4             261    -0.0029579     -0.0029594
#> 15 2019 5             261    -0.00088716    -0.0010823
#> 16 2020 1              54    -0.010749     -0.010333
#> 17 2020 2              54    -0.0065058     -0.0065211
#> 18 2020 3              54    -0.0043621     -0.0044412
#> 19 2020 4              54    -0.0026712     -0.0026349
#> 20 2020 5              54    -0.00051141    -0.00069626
```

```
plot_earnings_surprise_distribution <- tbl_event |>
  filter(is.finite(earnings_surprise)) |>
  ggplot(
    aes(
      x = earnings_surprise,
      linetype = event_strength
    )
  ) +
  geom_density(linewidth = 0.8) +
  geom_vline(
    xintercept = 0,
    linetype = "dotted"
  ) +
  labs(
```

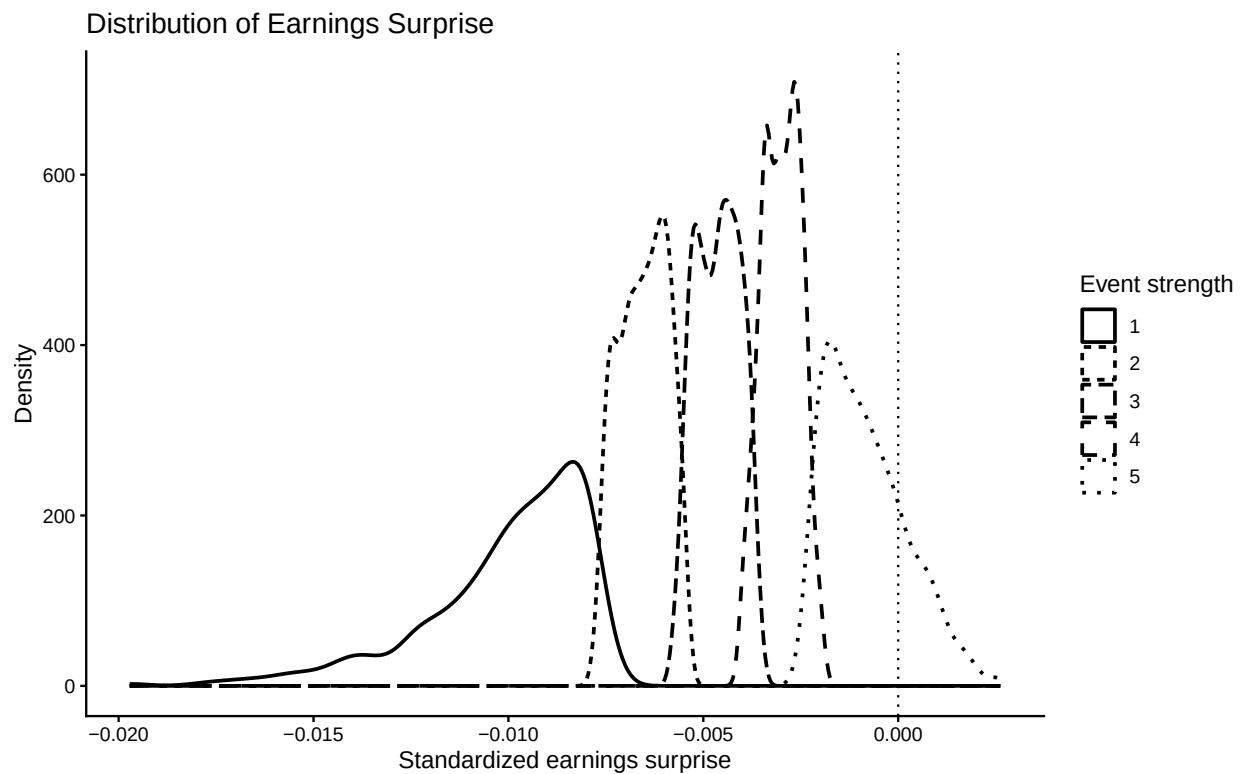


```

x = "Standardized earnings surprise",
y = "Density",
linetype = "Event strength",
title = "Distribution of Earnings Surprise"
) +
theme_classic()

```

```
plot_earnings_surprise_distribution
```



```

plot_surprise_by_strength <- tbl_event |>
  filter(is.finite(earnings_surprise)) |>
  ggplot(
    aes(
      x = event_strength,
      y = earnings_surprise
    )
  ) +
  geom_boxplot(
    outlier.alpha = 0.25
  ) +
  geom_hline(
    yintercept = 0,
    linetype = "dotted"
  ) +
  labs(
    x = "Event strength",

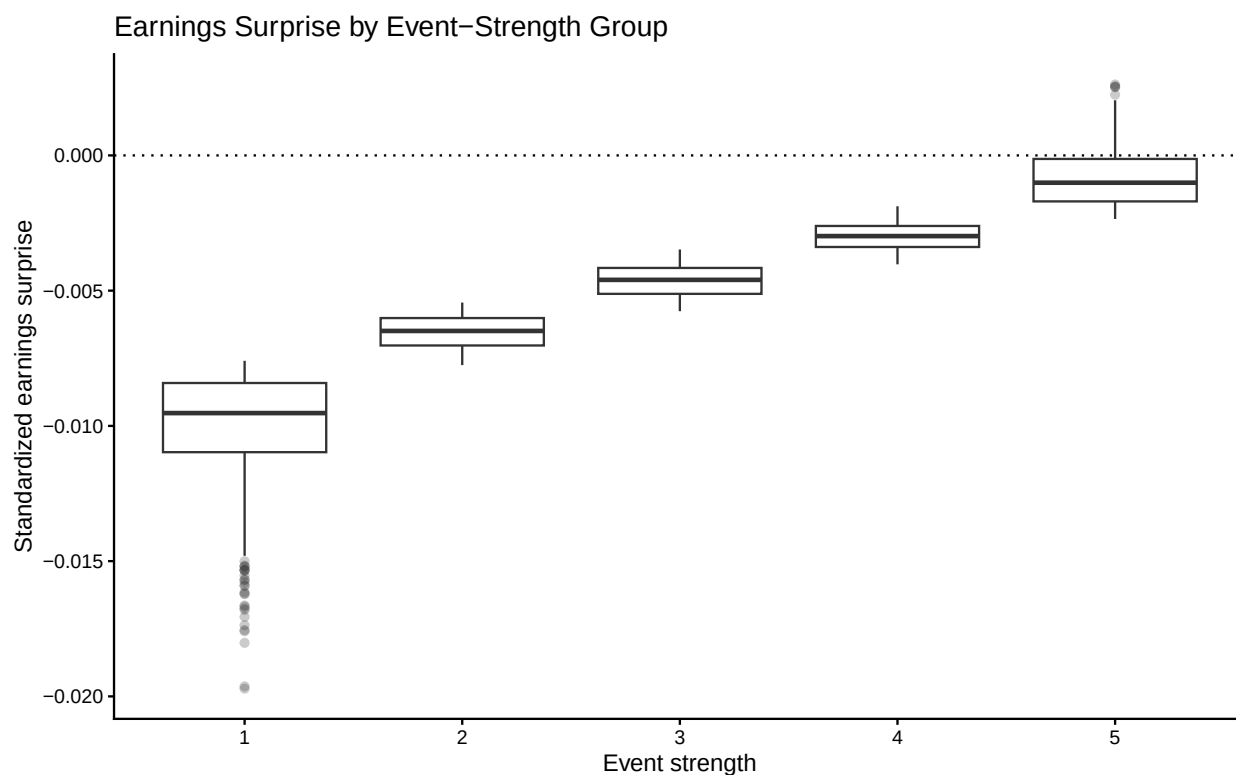
```

```

y = "Standardized earnings surprise",
title = "Earnings Surprise by Event-Strength Group"
) +
theme_classic()

```

```
plot_surprise_by_strength
```



```

plot_events_by_year <- tbl_event |>
  count(
    event_year,
    event_strength,
    name = "events"
  ) |>
  ggplot(
    aes(
      x = factor(event_year),
      y = events,
      fill = event_strength
    )
  ) +
  geom_col(
    position = "dodge"
  ) +
  labs(
    x = "Event year",
    y = "Events",

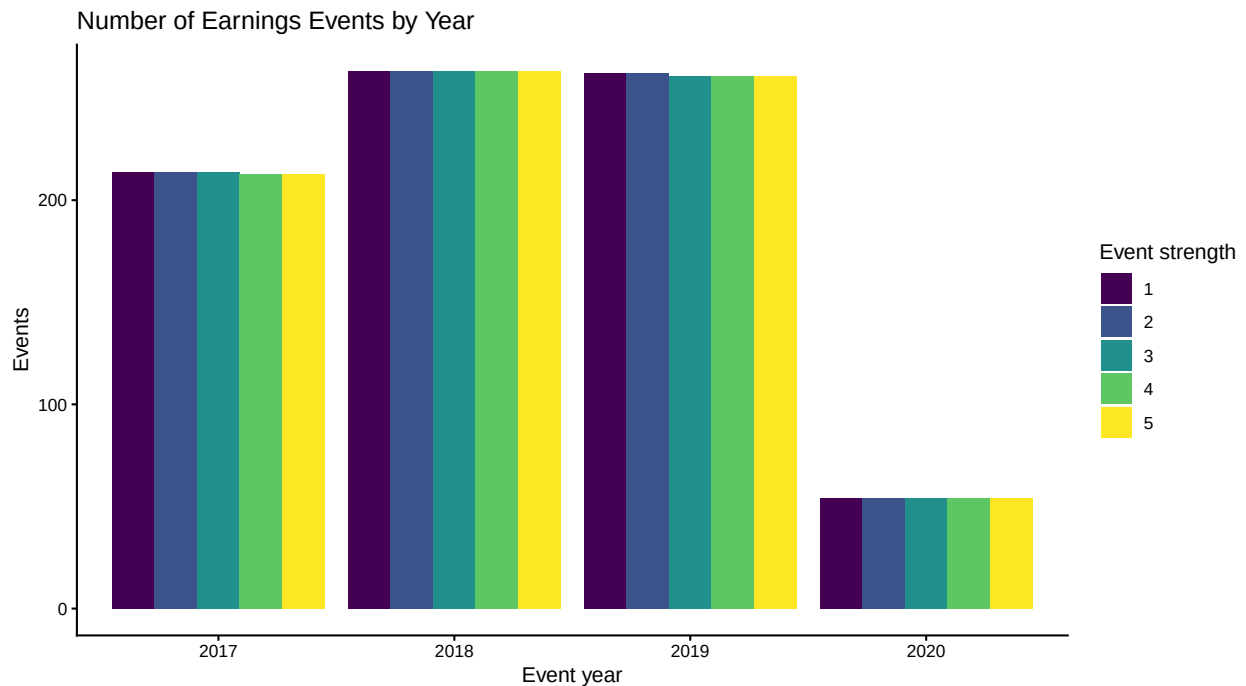
```

```

    fill = "Event strength",
    title = "Number of Earnings Events by Year"
  ) +
  theme_classic()

```

```
plot_events_by_year
```



I-6.4 イベント日とイベント・パネル

市場リターンの日付を取引日カレンダーとして使用する。

```

tbl_trading_calendar <- tbl_market |>
  distinct(date) |>
  arrange(date) |>
  mutate(
    date_ID = row_number()
  ) |>
  select(
    date_ID,
    date
  )

```

```

tbl_market_with_ID <- tbl_market |>
  inner_join(
    tbl_trading_calendar,
    by = "date"
  ) |>

```

```

select(
  date_ID,
  date,
  R_M
)

tbl_return_market <- tbl_return |>
  inner_join(
    tbl_market_with_ID,
    by = "date"
  ) |>
  select(
    firm_ID,
    date_ID,
    R,
    R_M
  ) |>
  arrange(
    firm_ID,
    date_ID
  )

```

決算発表は取引終了後に行われると仮定する。そのため、発表日が取引日であっても、その次の取引日をイベント日 0 とする。

```

tbl_event <- assign_event_day_zero_LFL(
  tbl_event = tbl_event,
  tbl_trading_calendar = tbl_trading_calendar
)

tbl_event |>
  select(
    event_ID,
    firm_ID,
    event_date,
    event_trade_date,
    event_trade_date_ID,
    event_strength,
    earnings_surprise
  ) |>
  slice_head(n = 20)
#> # A tibble: 20 x 7
#>   event_ID firm_ID event_date event_trade_date event_trade_date_ID
#>   <int>    <int> <date>      <date>                <int>
#> 1         1      39 2017-01-04 2017-01-05                247

```

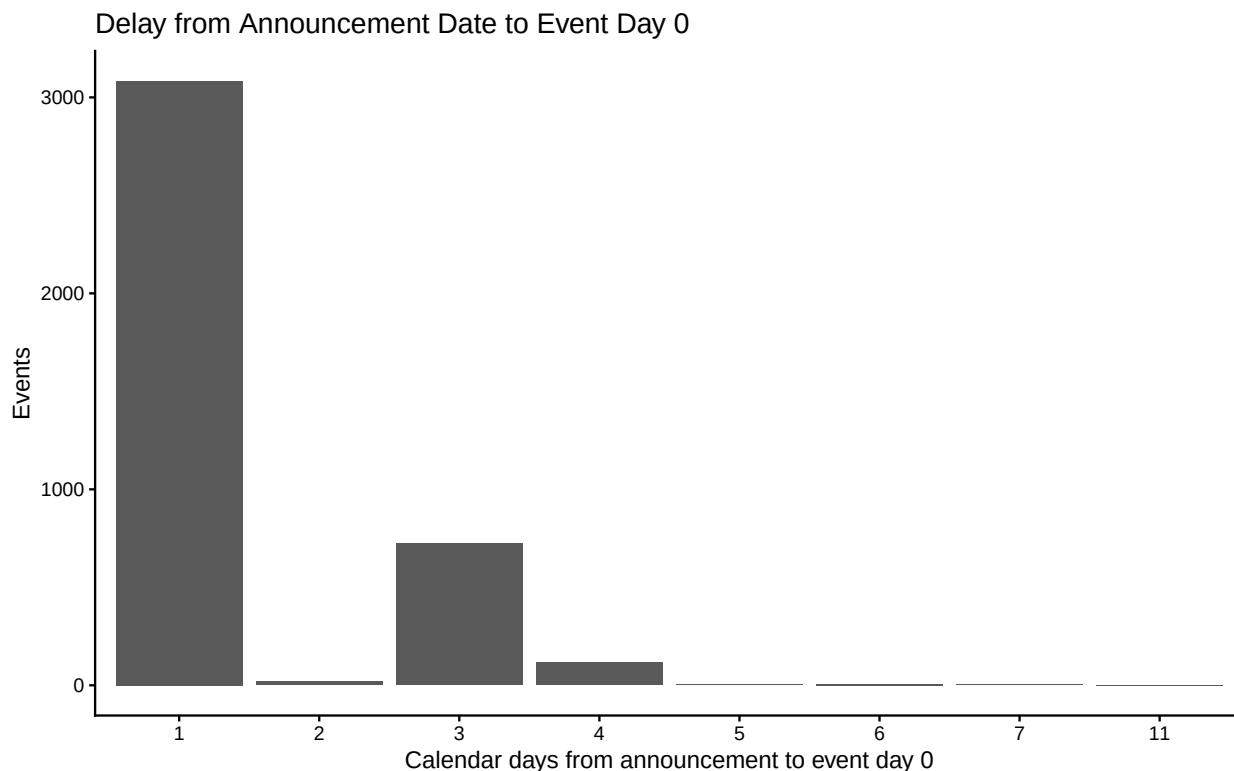
```
#> 2      2      57 2017-01-04 2017-01-05      247
#> 3      3      73 2017-01-04 2017-01-05      247
#> 4      4     109 2017-01-04 2017-01-05      247
#> 5      5     111 2017-01-04 2017-01-05      247
#> 6      6     231 2017-01-04 2017-01-05      247
#> 7      7     944 2017-01-04 2017-01-05      247
#> 8      8    1227 2017-01-04 2017-01-05      247
#> 9      9     422 2017-01-05 2017-01-06      248
#> 10     10    1203 2017-01-05 2017-01-06      248
#> 11     11     346 2017-01-06 2017-01-10      249
#> 12     12     590 2017-01-06 2017-01-10      249
#> 13     13     644 2017-01-06 2017-01-10      249
#> 14     14    1153 2017-01-06 2017-01-10      249
#> 15     15     191 2017-01-10 2017-01-11      250
#> 16     16     353 2017-01-10 2017-01-11      250
#> 17     17     566 2017-01-10 2017-01-11      250
#> 18     18     322 2017-01-11 2017-01-12      251
#> 19     19    1050 2017-01-11 2017-01-12      251
#> 20     20     426 2017-01-12 2017-01-13      252
#> # i 2 more variables: event_strength <ord>, earnings_surprise <dbl>
```

```
tbl_event_gap <- tbl_event |>
  mutate(
    calendar_day_gap = as.integer(
      event_trade_date - event_date
    )
  )

plot_event_date_gap <- tbl_event_gap |>
  count(
    calendar_day_gap,
    name = "events"
  ) |>
  ggplot(
    aes(
      x = factor(calendar_day_gap),
      y = events
    )
  ) +
  geom_col() +
  labs(
    x = "Calendar days from announcement to event day 0",
    y = "Events",
    title = "Delay from Announcement Date to Event Day 0"
  ) +
```

```
theme_classic()
```

```
plot_event_date_gap
```



```
tbl_event_panel <- create_event_study_panel_LFL(
  tbl_event = tbl_event,
  tbl_trading_calendar = tbl_trading_calendar,
  tbl_return_market = tbl_return_market,
  estimation_start = estimation_start,
  event_end = event_end
)
```

```
tbl_event_panel |>
```

```
  select(
    event_ID,
    firm_ID,
    relative_day,
    date,
    R,
    R_M
  ) |>
```

```
  slice_head(n = 20)
```

```
#> # A tibble: 20 x 6
```

```
#>   event_ID firm_ID relative_day date          R          R_M
#>   <int>   <int>      <int> <date>      <dbl>      <dbl>
#> 1       1       39      -130 2016-06-23 0.0098506 0.018087
```

```
#> 2      1      39      -129 2016-06-24  0.012580  0.0092862
#> 3      1      39      -128 2016-06-27  0.015076  0.0031098
#> 4      1      39      -127 2016-06-28  0.015788  0.00067260
#> 5      1      39      -126 2016-06-29 -0.0033987 -0.0048162
#> 6      1      39      -125 2016-06-30 -0.0071048 -0.0038061
#> 7      1      39      -124 2016-07-01  0.014684 -0.0032365
#> 8      1      39      -123 2016-07-04 -0.0069849 -0.0012511
#> 9      1      39      -122 2016-07-05 -0.023656 -0.018547
#> 10     1      39      -121 2016-07-06  0.011629  0.017017
#> 11     1      39      -120 2016-07-07 -0.0064901 -0.0011435
#> 12     1      39      -119 2016-07-08 -0.0062219 -0.0012174
#> 13     1      39      -118 2016-07-11  0.0055502  0.017228
#> 14     1      39      -117 2016-07-12  0.0064255  0.0060624
#> 15     1      39      -116 2016-07-13 -0.027040 -0.022380
#> 16     1      39      -115 2016-07-14 -0.014094 -0.020934
#> 17     1      39      -114 2016-07-15  0.0064607  0.0081064
#> 18     1      39      -113 2016-07-19 -0.010573 -0.0026187
#> 19     1      39      -112 2016-07-20  0.024173  0.031602
#> 20     1      39      -111 2016-07-21 -0.00044361 -0.00024792
```

```
estimation_window_length <-
  estimation_end -
  estimation_start + 1L

event_window_length <-
  event_end -
  event_start + 1L

tbl_event_coverage <- tbl_event_panel |>
  group_by(
    event_ID,
    firm_ID,
    event_strength
  ) |>
  summarise(
    valid_estimation_days = sum(
      between(
        relative_day,
        estimation_start,
        estimation_end
      ) &
      is.finite(R) &
      is.finite(R_M)
    ),
    valid_event_days = sum(
```

```

    between(
      relative_day,
      event_start,
      event_end
    ) &
    is.finite(R) &
    is.finite(R_M)
  ),
  .groups = "drop"
)

tbl_event_coverage |>
  summarise(
    events = n(),
    complete_estimation_window = sum(
      valid_estimation_days ==
        estimation_window_length
    ),
    usable_estimation_window = sum(
      valid_estimation_days >=
        minimum_estimation_observations
    ),
    complete_event_window = sum(
      valid_event_days ==
        event_window_length
    )
  )

#> # A tibble: 1 x 4
#>   events complete_estimation_window usable_estimation_wi~1 complete_event_window
#>   <int>                <int>                <int>                <int>
#> 1   3960                3960                3960                3960
#> # i abbreviated name: 1: usable_estimation_window

```

```

plot_estimation_coverage <- tbl_event_coverage |>
  ggplot(
    aes(
      x = valid_estimation_days
    )
  ) +
  geom_histogram(
    binwidth = 5,
    boundary = 0
  ) +
  geom_vline(
    xintercept =

```

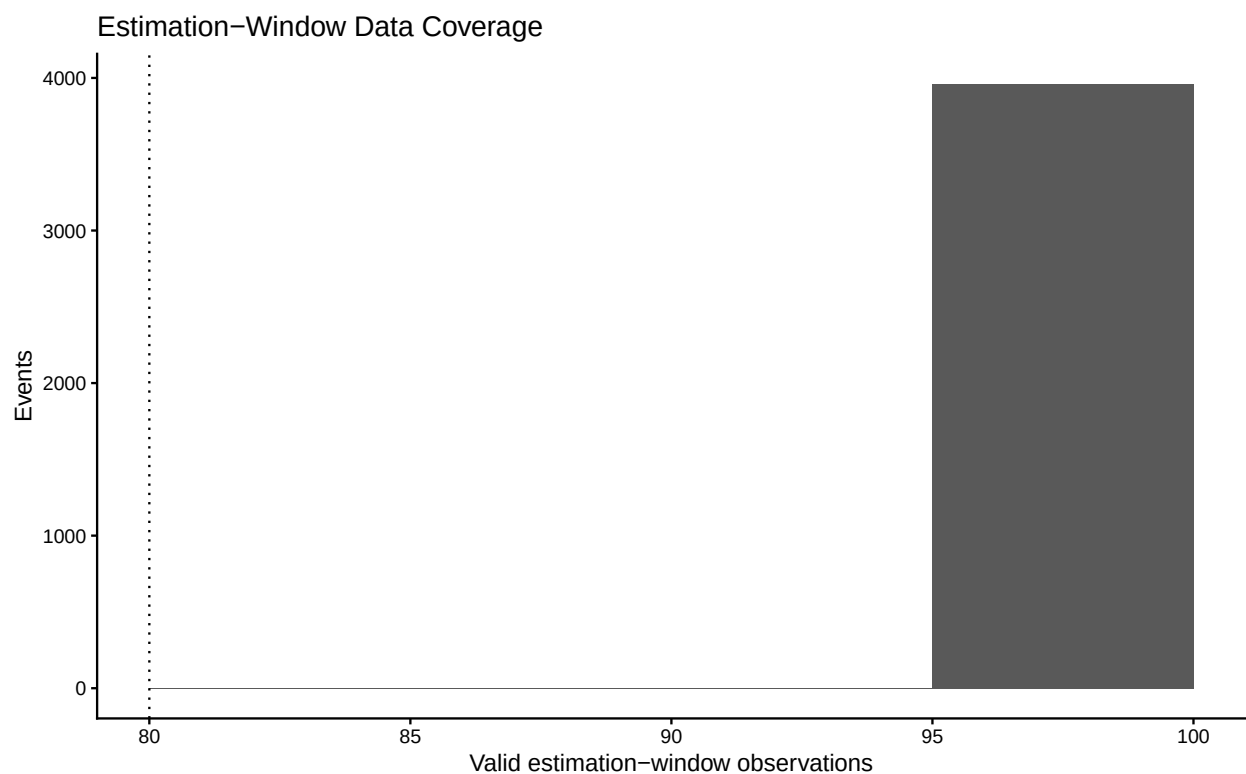


```

    minimum_estimation_observations,
    linetype = "dotted"
) +
labs(
  x = "Valid estimation-window observations",
  y = "Events",
  title = "Estimation-Window Data Coverage"
) +
theme_classic()

```

plot_estimation_coverage



I-6.5 マーケット・モデルの推定

```

tbl_market_model <- estimate_market_models_LFL(
  tbl_event_panel = tbl_event_panel,
  estimation_start = estimation_start,
  estimation_end = estimation_end,
  minimum_observations =
    minimum_estimation_observations
)

tbl_market_model |>
  slice_head(n = 20)

```

```
#> # A tibble: 20 x 13
#>   event_ID firm_ID event_date event_trade_date event_year event_strength
#>   <int>   <int> <date>      <date>          <dbl> <ord>
#> 1       1     39 2017-01-04 2017-01-05          2017 1
#> 2       2     57 2017-01-04 2017-01-05          2017 1
#> 3       3     73 2017-01-04 2017-01-05          2017 4
#> 4       4    109 2017-01-04 2017-01-05          2017 3
#> 5       5    111 2017-01-04 2017-01-05          2017 3
#> 6       6    231 2017-01-04 2017-01-05          2017 1
#> 7       7    944 2017-01-04 2017-01-05          2017 2
#> 8       8   1227 2017-01-04 2017-01-05          2017 5
#> 9       9    422 2017-01-05 2017-01-06          2017 3
#> 10      10   1203 2017-01-05 2017-01-06          2017 5
#> 11      11    346 2017-01-06 2017-01-10          2017 2
#> 12      12    590 2017-01-06 2017-01-10          2017 1
#> 13      13    644 2017-01-06 2017-01-10          2017 2
#> 14      14   1153 2017-01-06 2017-01-10          2017 4
#> 15      15    191 2017-01-10 2017-01-11          2017 4
#> 16      16    353 2017-01-10 2017-01-11          2017 1
#> 17      17    566 2017-01-10 2017-01-11          2017 3
#> 18      18    322 2017-01-11 2017-01-12          2017 3
#> 19      19   1050 2017-01-11 2017-01-12          2017 4
#> 20      20    426 2017-01-12 2017-01-13          2017 5
#> # i 7 more variables: earnings_surprise <dbl>, alpha <dbl>, beta <dbl>,
#> #   sigma_AR <dbl>, r_squared <dbl>, adjusted_r_squared <dbl>,
#> #   estimation_observations <int>
```

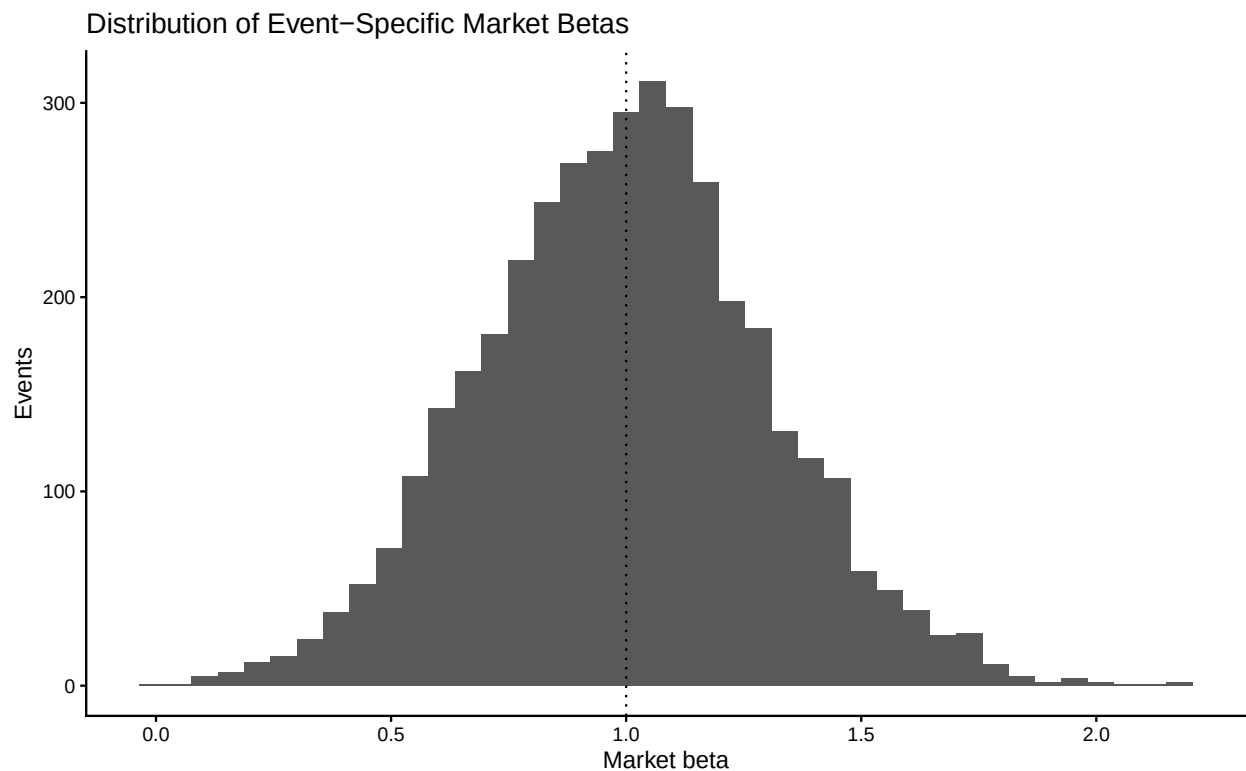
```
tbl_market_model_summary <- tbl_market_model |>
  summarise(
    estimated_events = n(),
    mean_alpha = mean(alpha),
    mean_beta = mean(beta),
    median_beta = median(beta),
    minimum_beta = min(beta),
    maximum_beta = max(beta),
    mean_r_squared = mean(r_squared),
    median_r_squared = median(r_squared),
    mean_sigma_AR = mean(sigma_AR),
    minimum_observations =
      min(estimation_observations),
    maximum_observations =
      max(estimation_observations)
  )

tbl_market_model_summary
```

```
#> # A tibble: 1 x 11
#>   estimated_events mean_alpha mean_beta median_beta minimum_beta maximum_beta
#>   <int>          <dbl>    <dbl>    <dbl>      <dbl>      <dbl>
#> 1       3960 -0.000017343  0.99751    1.0048   -0.0083393    2.1776
#> # i 5 more variables: mean_r_squared <dbl>, median_r_squared <dbl>,
#> #   mean_sigma_AR <dbl>, minimum_observations <int>, maximum_observations <int>
```

```
plot_market_beta <- tbl_market_model |>
  ggplot(aes(x = beta)) +
  geom_histogram(bins = 40) +
  geom_vline(
    xintercept = 1,
    linetype = "dotted"
  ) +
  labs(
    x = "Market beta",
    y = "Events",
    title = "Distribution of Event-Specific Market Betas"
  ) +
  theme_classic()
```

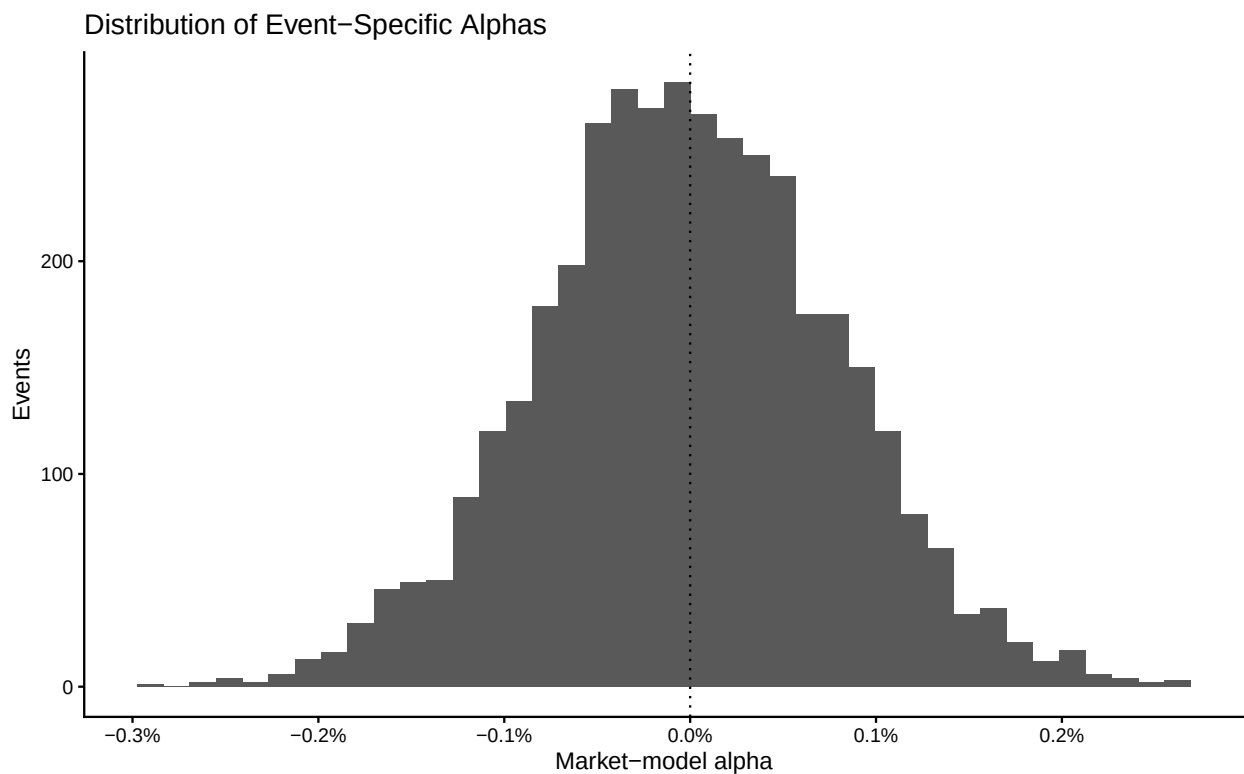
```
plot_market_beta
```



```
plot_market_alpha <- tbl_market_model |>
  ggplot(aes(x = alpha)) +
  geom_histogram(bins = 40) +
```

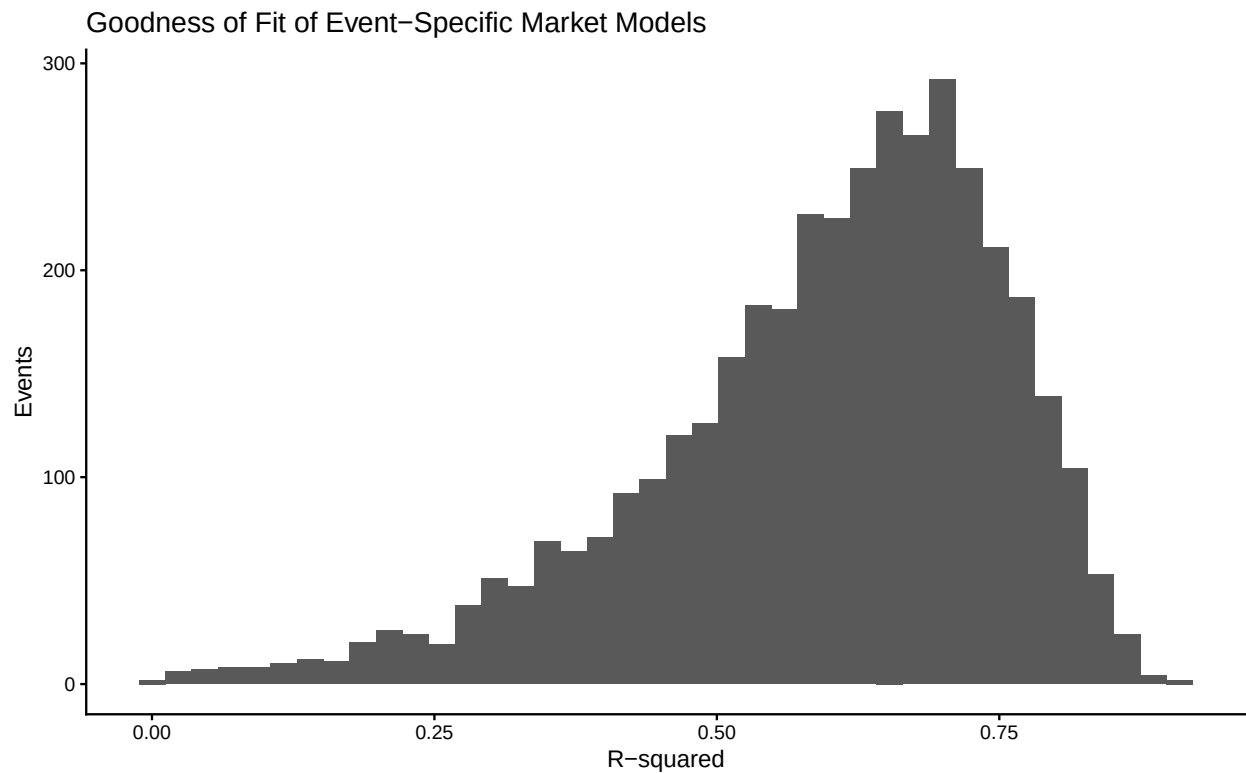
```
geom_vline(
  xintercept = 0,
  linetype = "dotted"
) +
scale_x_continuous(
  labels = scales::label_percent()
) +
labs(
  x = "Market-model alpha",
  y = "Events",
  title = "Distribution of Event-Specific Alphas"
) +
theme_classic()
```

plot_market_alpha



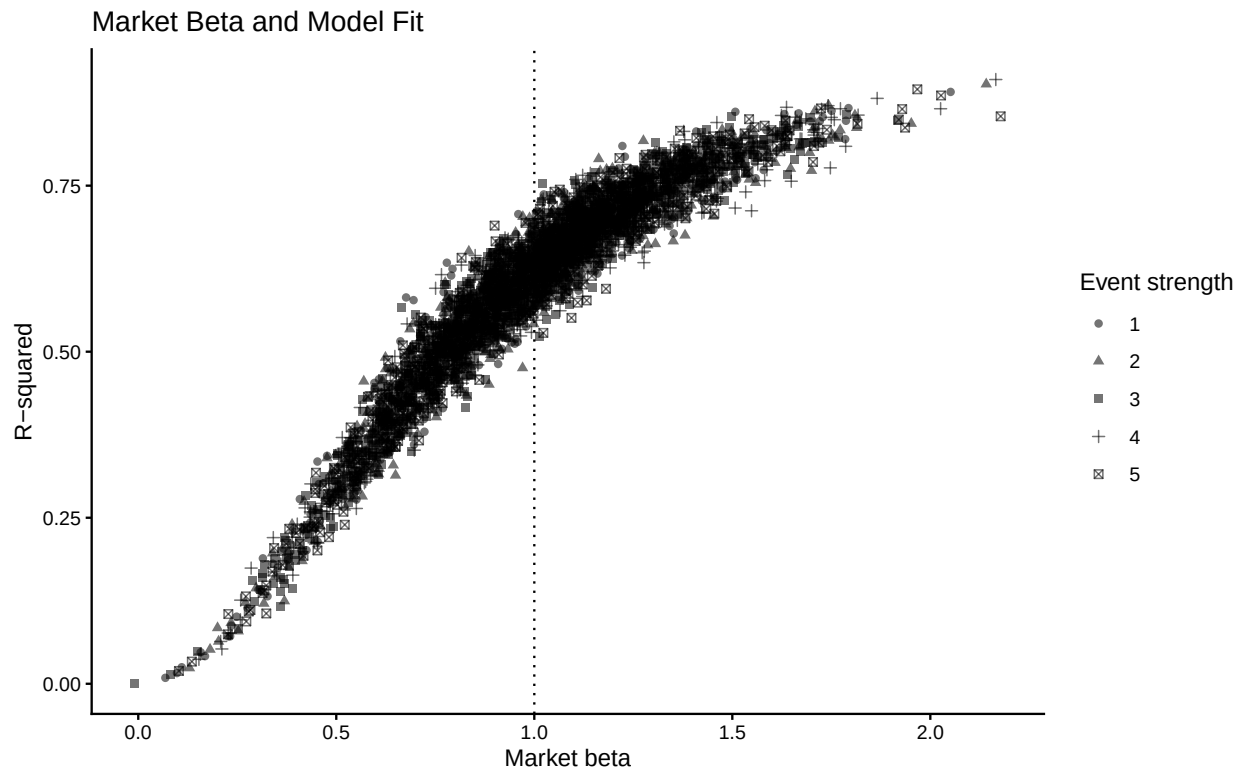
```
plot_market_model_r_squared <- tbl_market_model |>
ggplot(aes(x = r_squared)) +
geom_histogram(bins = 40) +
labs(
  x = "R-squared",
  y = "Events",
  title = "Goodness of Fit of Event-Specific Market Models"
) +
theme_classic()
```

```
plot_market_model_r_squared
```



```
plot_beta_r_squared <- tbl_market_model |>
  ggplot(
    aes(
      x = beta,
      y = r_squared,
      shape = event_strength
    )
  ) +
  geom_point(alpha = 0.55) +
  geom_vline(
    xintercept = 1,
    linetype = "dotted"
  ) +
  labs(
    x = "Market beta",
    y = "R-squared",
    shape = "Event strength",
    title = "Market Beta and Model Fit"
  ) +
  theme_classic()

plot_beta_r_squared
```



I-6.6 異常リターンと累積異常リターン

```
tbl_abnormal_return <- compute_abnormal_returns_LFL(
  tbl_event_panel = tbl_event_panel,
  tbl_market_model = tbl_market_model,
  event_start = event_start,
  event_end = event_end,
  require_complete_window = TRUE
)

tbl_abnormal_return |>
  select(
    event_ID,
    firm_ID,
    event_strength,
    relative_day,
    date,
    R,
    R_M,
    normal_return,
    AR,
    CAR
  ) |>
  slice_head(n = 20)
#> # A tibble: 20 x 10
```

```

#>      event_ID firm_ID event_strength relative_day date          R          R_M
#>      <int>    <int> <ord>          <int> <date>        <dbl>        <dbl>
#> 1           1      39 1             -30 2016-11-18 -0.0013155 -1.4725e-3
#> 2           1      39 1             -29 2016-11-21  0.0037198 -3.3404e-3
#> 3           1      39 1             -28 2016-11-22  0.021021  1.7138e-2
#> 4           1      39 1             -27 2016-11-24  0.0030893  1.5701e-2
#> 5           1      39 1             -26 2016-11-25 -0.0062645 -2.3346e-2
#> 6           1      39 1             -25 2016-11-28 -0.0040561 -6.5350e-3
#> 7           1      39 1             -24 2016-11-29  0.012325  -4.1437e-3
#> 8           1      39 1             -23 2016-11-30 -0.0096488  2.9259e-3
#> 9           1      39 1             -22 2016-12-01  0.014048  4.7329e-4
#> 10          1      39 1             -21 2016-12-02  0.012192  1.0758e-2
#> 11          1      39 1             -20 2016-12-05  0.0097831 -5.4821e-3
#> 12          1      39 1             -19 2016-12-06  0.026133  1.8634e-2
#> 13          1      39 1             -18 2016-12-07 -0.0044072 -7.3350e-3
#> 14          1      39 1             -17 2016-12-08  0.015023  2.7649e-2
#> 15          1      39 1             -16 2016-12-09  0.0074484 -1.4362e-3
#> 16          1      39 1             -15 2016-12-12  0.011444  -6.4222e-3
#> 17          1      39 1             -14 2016-12-13 -0.011043  -1.4330e-2
#> 18          1      39 1             -13 2016-12-14  0.019488  2.2428e-2
#> 19          1      39 1             -12 2016-12-15  0.0054953  1.8578e-2
#> 20          1      39 1             -11 2016-12-16 -0.0066719 -3.4403e-3
#> # i 3 more variables: normal_return <dbl>, AR <dbl>, CAR <dbl>

```

イベント強度 g に属するイベント数を N_g とすると、平均異常リターンは次式で表される。

$$AAR_{g,t} = \frac{1}{N_g} \sum_{i \in g} AR_{i,t}$$

平均累積異常リターンは次式で表される。

$$CAAR_{g,t} = \frac{1}{N_g} \sum_{i \in g} CAR_{i,t}$$

```

significance_stars <- function(p_value) {
  case_when(
    is.na(p_value) ~ "",
    p_value < 0.01 ~ "***",
    p_value < 0.05 ~ "**",
    p_value < 0.10 ~ "*",
    TRUE ~ ""
  )
}

```

```
tbl_daily_statistics <- tbl_abnormal_return |>
  group_by(
    relative_day,
    event_strength
  ) |>
  summarise(
    events = n_distinct(event_ID),
    AAR = mean(AR),
    CAAR = mean(CAR),
    standard_deviation_AR = sd(AR),
    standard_deviation_CAR = sd(CAR),
    .groups = "drop"
  ) |>
  mutate(
    standard_error_AAR =
      standard_deviation_AR /
      sqrt(events),
    standard_error_CAAR =
      standard_deviation_CAR /
      sqrt(events),
    t_AAR = if_else(
      is.finite(standard_error_AAR) &
        standard_error_AAR > 0,
      AAR / standard_error_AAR,
      NA_real_
    ),
    t_CAAR = if_else(
      is.finite(standard_error_CAAR) &
        standard_error_CAAR > 0,
      CAAR / standard_error_CAAR,
      NA_real_
    ),
    p_AAR = if_else(
      is.finite(t_AAR),
      2 * pt(
        -abs(t_AAR),
        df = events - 1L
      ),
      NA_real_
    ),
    p_CAAR = if_else(
      is.finite(t_CAAR),
      2 * pt(
        -abs(t_CAAR),
        df = events - 1L
      )
    )
  )
```



```

    ),
    NA_real_
  ),
  significance_AAR =
    significance_stars(p_AAR),
  significance_CAAR =
    significance_stars(p_CAAR)
) |>
arrange(
  event_strength,
  relative_day
)

tbl_daily_statistics
#> # A tibble: 305 x 15
#>   relative_day event_strength events      AAR      CAAR
#>   <int> <ord>      <int>    <dbl>    <dbl>
#> 1      -30 1          793 -0.00033279 -0.00033279
#> 2      -29 1          793  0.00034601  0.000013216
#> 3      -28 1          793 -0.000030469 -0.000017253
#> 4      -27 1          793  0.00024231  0.00022506
#> 5      -26 1          793  0.00042018  0.00064524
#> 6      -25 1          793  0.00020818  0.00085342
#> 7      -24 1          793  0.00053366  0.0013871
#> 8      -23 1          793 -0.000028460  0.0013586
#> 9      -22 1          793 -0.00014238  0.0012162
#> 10     -21 1          793 -0.00019155  0.0010247
#> # i 295 more rows
#> # i 10 more variables: standard_deviation_AR <dbl>,
#> #   standard_deviation_CAR <dbl>, standard_error_AAR <dbl>,
#> #   standard_error_CAAR <dbl>, t_AAR <dbl>, t_CAAR <dbl>, p_AAR <dbl>,
#> #   p_CAAR <dbl>, significance_AAR <chr>, significance_CAAR <chr>

```

```

plot_caar_by_strength <- tbl_daily_statistics |>
  ggplot(
    aes(
      x = relative_day,
      y = CAAR,
      linetype = event_strength
    )
  ) +
  annotate(
    "rect",
    xmin = -1,
    xmax = 1,

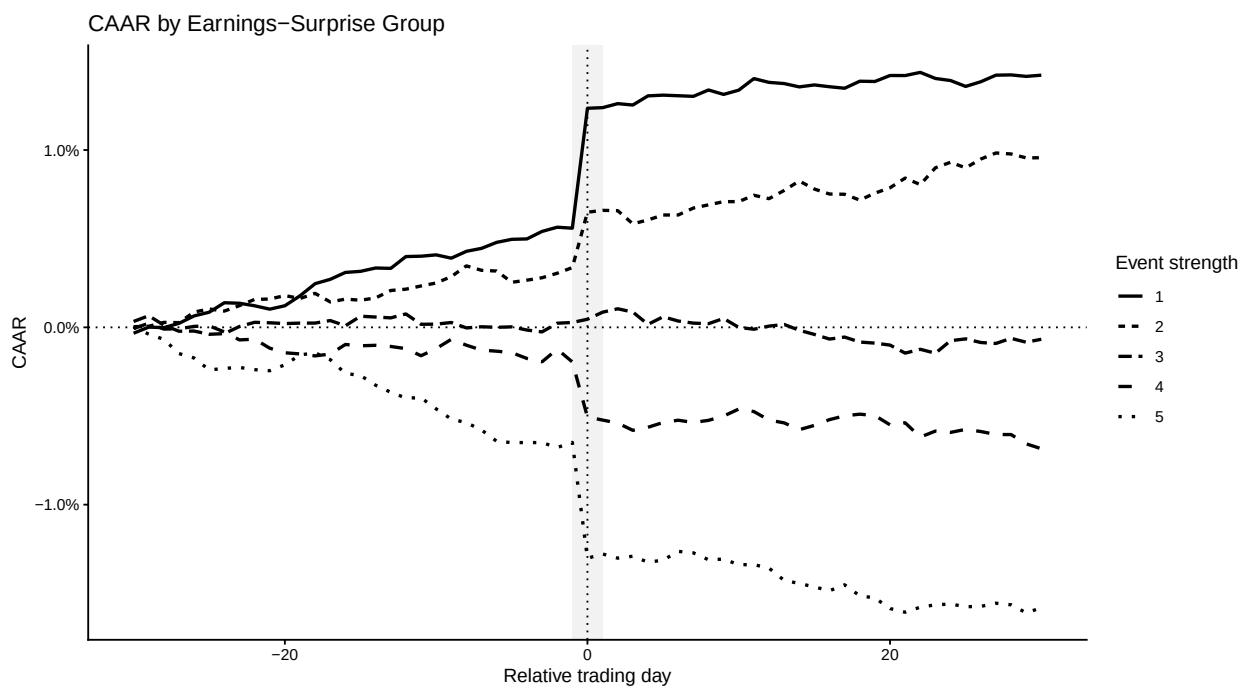
```

```

  ymin = -Inf,
  ymax = Inf,
  alpha = 0.08
) +
geom_line(linewidth = 0.9) +
geom_hline(
  yintercept = 0,
  linetype = "dotted"
) +
geom_vline(
  xintercept = 0,
  linetype = "dotted"
) +
scale_y_continuous(
  labels = scales::label_percent()
) +
labs(
  x = "Relative trading day",
  y = "CAAR",
  linetype = "Event strength",
  title = "CAAR by Earnings-Surprise Group"
) +
theme_classic()

```

plot_caar_by_strength



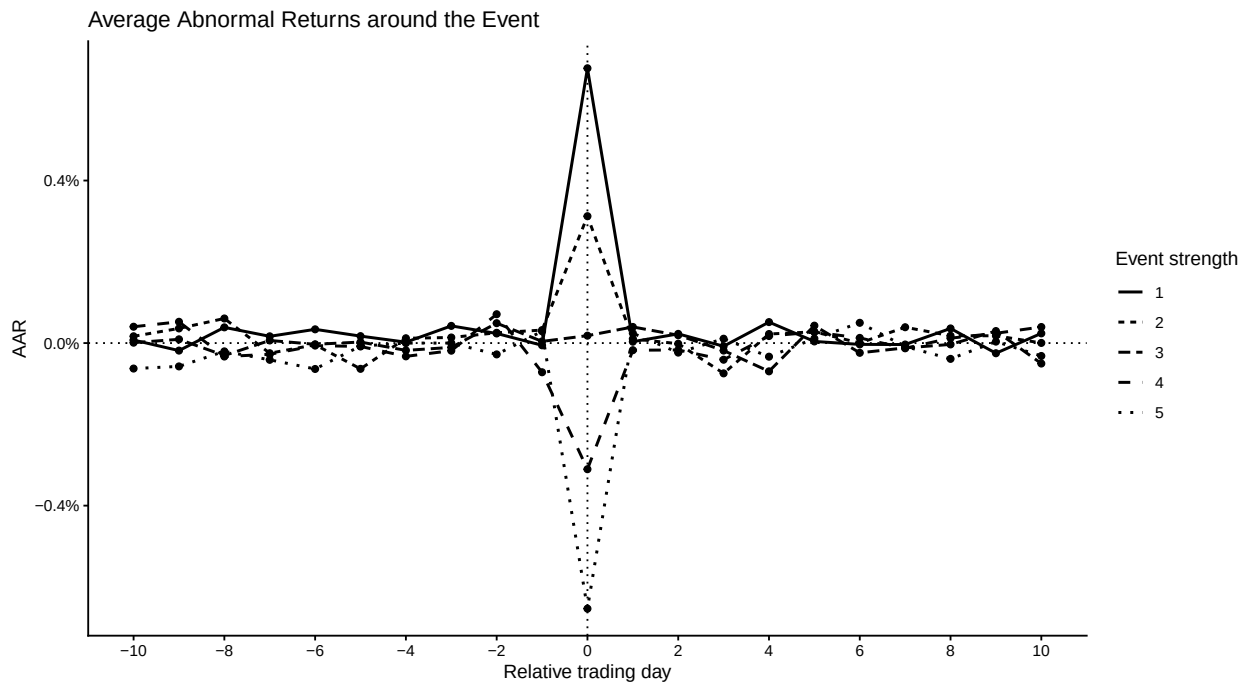
```

plot_aar_around_event <- tbl_daily_statistics |>
  filter(

```

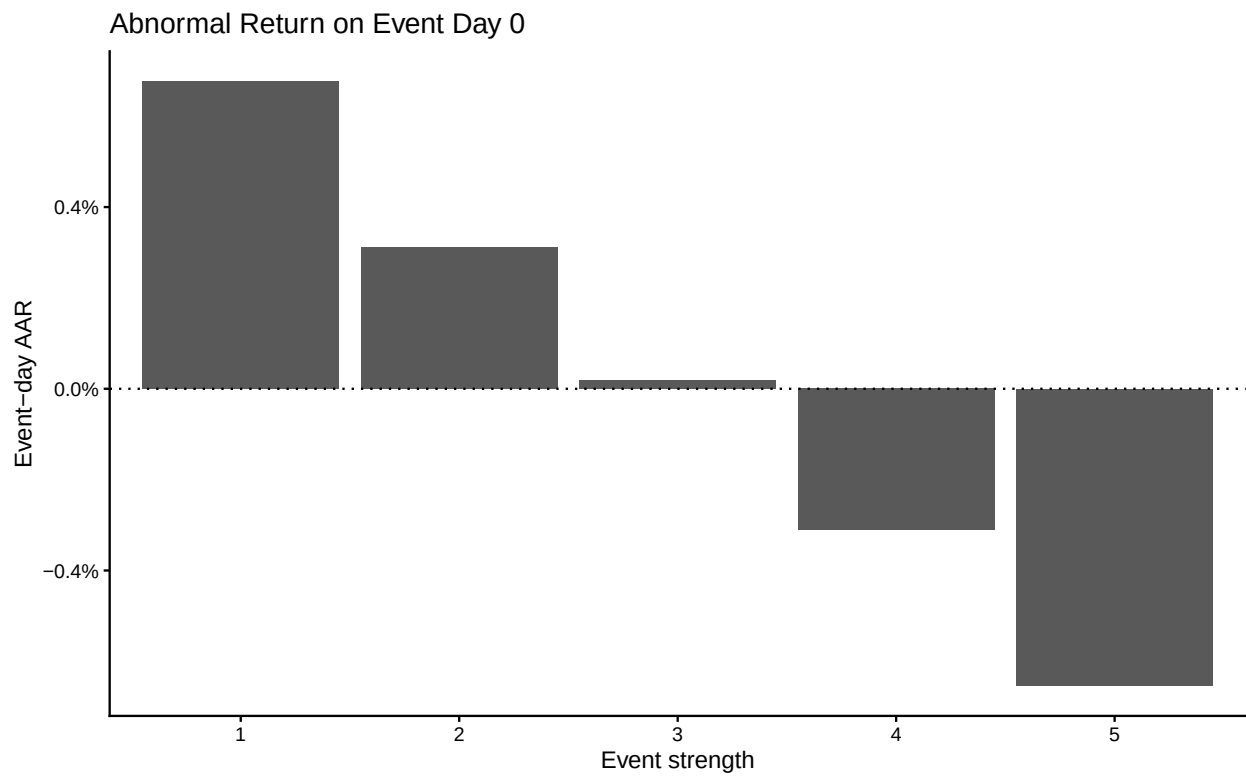
```
    between(
      relative_day,
      -10L,
      10L
    )
  ) |>
  ggplot(
    aes(
      x = relative_day,
      y = AAR,
      linetype = event_strength
    )
  ) +
  geom_line(linewidth = 0.8) +
  geom_point() +
  geom_vline(
    xintercept = 0,
    linetype = "dotted"
  ) +
  geom_hline(
    yintercept = 0,
    linetype = "dotted"
  ) +
  scale_x_continuous(
    breaks = seq(-10, 10, 2)
  ) +
  scale_y_continuous(
    labels = scales::label_percent()
  ) +
  labs(
    x = "Relative trading day",
    y = "AAR",
    linetype = "Event strength",
    title = "Average Abnormal Returns around the Event"
  ) +
  theme_classic()

plot_aar_around_event
```



```
plot_event_day_aar <- tbl_daily_statistics |>
  filter(relative_day == 0L) |>
  ggplot(
    aes(
      x = event_strength,
      y = AAR
    )
  ) +
  geom_col() +
  geom_hline(
    yintercept = 0,
    linetype = "dotted"
  ) +
  scale_y_continuous(
    labels = scales::label_percent()
  ) +
  labs(
    x = "Event strength",
    y = "Event-day AAR",
    title = "Abnormal Return on Event Day 0"
  ) +
  theme_classic()

plot_event_day_aar
```



```
plot_post_event_caar <- tbl_daily_statistics |>
```

```
  filter(
```

```
    between(
```

```
      relative_day,
```

```
      0L,
```

```
      30L
```

```
    )
```

```
  ) |>
```

```
  ggplot(
```

```
    aes(
```

```
      x = relative_day,
```

```
      y = CAAR,
```

```
      linetype = event_strength
```

```
    )
```

```
  ) +
```

```
  geom_line(linewidth = 0.9) +
```

```
  geom_hline(
```

```
    yintercept = 0,
```

```
    linetype = "dotted"
```

```
  ) +
```

```
  scale_y_continuous(
```

```
    labels = scales::label_percent()
```

```
  ) +
```

```
  labs(
```

```
    x = "Trading days after event",
```

```
    y = "CAAR",
```

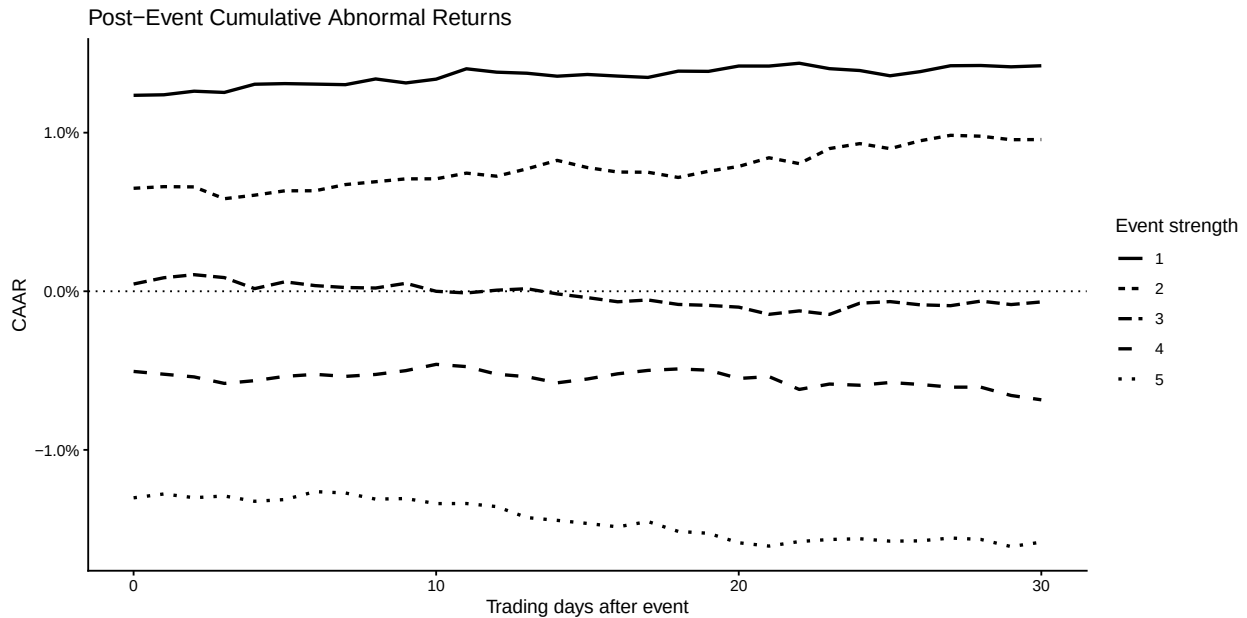
```
    linetype = "Event strength",
```

```

  title = "Post-Event Cumulative Abnormal Returns"
) +
theme_classic()

```

```
plot_post_event_caar
```



```

plot_ar_distribution_by_strength <- tbl_abnormal_return |>
  filter(
    between(
      relative_day,
      -1L,
      1L
    )
  ) |>
  ggplot(
    aes(
      x = AR,
      linetype = event_strength
    )
  ) +
  geom_density(linewidth = 0.8) +
  geom_vline(
    xintercept = 0,
    linetype = "dotted"
  ) +
  scale_x_continuous(
    labels = scales::label_percent()
  ) +
  labs(
    x = "Abnormal return",

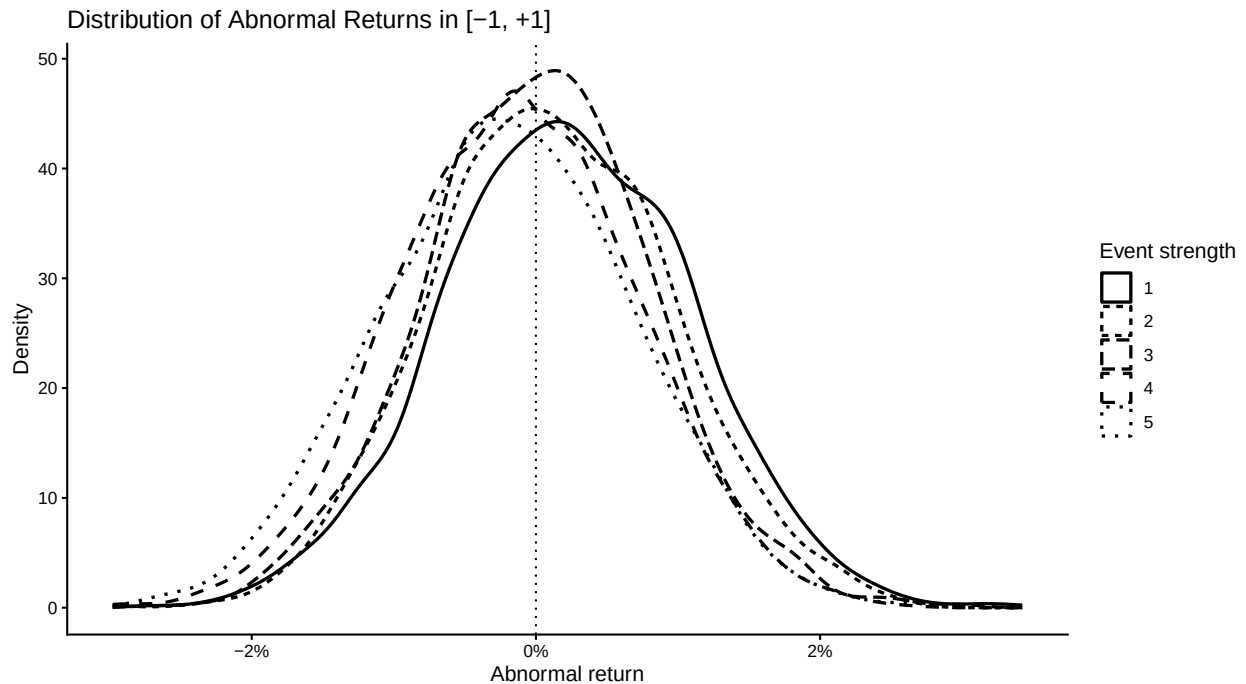
```

```

y = "Density",
linetype = "Event strength",
title = "Distribution of Abnormal Returns in [-1, +1]"
) +
theme_classic()

```

```
plot_ar_distribution_by_strength
```



I-6.7 イベント・ウィンドウ分析

```

tbl_summary_window <- tribble(
  ~window_ID, ~window_label, ~start_day, ~end_day,
  "m1_p1", "[ -1, +1]", -1L, 1L,
  "0_p1", "[0, +1]", 0L, 1L,
  "m5_p5", "[ -5, +5]", -5L, 5L,
  "m10_p10", "[ -10, +10]", -10L, 10L,
  "m30_p30", "[ -30, +30]", -30L, 30L
)

```

```

tbl_summary_window
#> # A tibble: 5 x 4
#>   window_ID window_label start_day end_day
#>   <chr>      <chr>         <int>  <int>
#> 1 m1_p1     [ -1, +1]           -1      1
#> 2 0_p1      [0, +1]            0      1
#> 3 m5_p5     [ -5, +5]          -5      5

```

```
#> 4 m10_p10    [-10, +10]        -10      10
#> 5 m30_p30    [-30, +30]        -30      30
```

```
tbl_event_window_return <-
  compute_event_window_returns_LFL(
    tbl_abnormal_return = tbl_abnormal_return,
    tbl_window = tbl_summary_window
  )

tbl_event_window_return |>
  slice_head(n = 20)
#> # A tibble: 20 x 14
#>   event_ID firm_ID event_date event_trade_date event_year event_strength
#>   <int>    <int> <date>      <date>          <dbl> <ord>
#> 1         1      39 2017-01-04 2017-01-05          2017 1
#> 2         2      57 2017-01-04 2017-01-05          2017 1
#> 3         3      73 2017-01-04 2017-01-05          2017 4
#> 4         4     109 2017-01-04 2017-01-05          2017 3
#> 5         5     111 2017-01-04 2017-01-05          2017 3
#> 6         6     231 2017-01-04 2017-01-05          2017 1
#> 7         7     944 2017-01-04 2017-01-05          2017 2
#> 8         8    1227 2017-01-04 2017-01-05          2017 5
#> 9         9     422 2017-01-05 2017-01-06          2017 3
#> 10        10    1203 2017-01-05 2017-01-06          2017 5
#> 11        11     346 2017-01-06 2017-01-10          2017 2
#> 12        12     590 2017-01-06 2017-01-10          2017 1
#> 13        13     644 2017-01-06 2017-01-10          2017 2
#> 14        14    1153 2017-01-06 2017-01-10          2017 4
#> 15        15     191 2017-01-10 2017-01-11          2017 4
#> 16        16     353 2017-01-10 2017-01-11          2017 1
#> 17        17     566 2017-01-10 2017-01-11          2017 3
#> 18        18     322 2017-01-11 2017-01-12          2017 3
#> 19        19    1050 2017-01-11 2017-01-12          2017 4
#> 20        20     426 2017-01-12 2017-01-13          2017 5
#> # i 8 more variables: earnings_surprise <dbl>, window_ID <chr>,
#> #   window_label <chr>, start_day <int>, end_day <int>, expected_days <int>,
#> #   CAR_window <dbl>, observed_days <int>
```

```
tbl_event_window_summary <- tbl_event_window_return |>
  group_by(
    window_ID,
    window_label,
    start_day,
    end_day,
```



```

    event_strength
  ) |>
  summarise(
    events = n_distinct(event_ID),
    mean_CAR = mean(CAR_window),
    median_CAR = median(CAR_window),
    standard_deviation_CAR =
      sd(CAR_window),
    positive_CAR_ratio =
      mean(CAR_window > 0),
    .groups = "drop"
  ) |>
  mutate(
    standard_error_CAR =
      standard_deviation_CAR /
      sqrt(events),
    t_value = if_else(
      is.finite(standard_error_CAR) &
        standard_error_CAR > 0,
      mean_CAR /
        standard_error_CAR,
      NA_real_
    ),
    p_value = if_else(
      is.finite(t_value),
      2 * pt(
        -abs(t_value),
        df = events - 1L
      ),
      NA_real_
    ),
    significance =
      significance_stars(p_value)
  ) |>
  arrange(
    window_ID,
    event_strength
  )

```

```
tbl_event_window_summary
```

```
#> # A tibble: 25 x 14
```

```

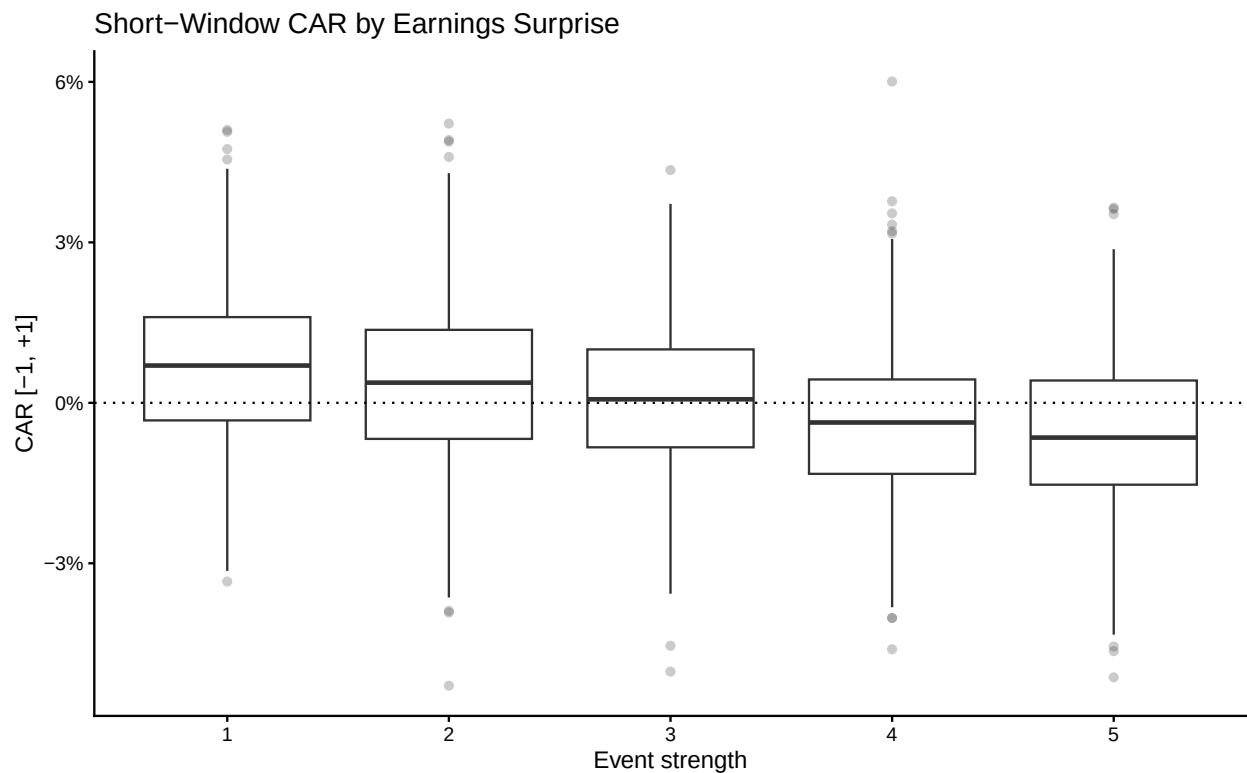
#>   window_ID window_label start_day end_day event_strength events   mean_CAR
#>   <chr>      <chr>         <int>  <int> <ord>          <int>    <dbl>
#> 1 0_p1      [0, +1]         0      1 1              793 0.0067977
#> 2 0_p1      [0, +1]         0      1 2              793 0.0032266

```

```
#> 3 0_p1      [0, +1]          0      1 3          792 0.00057929
#> 4 0_p1      [0, +1]          0      1 4          791 -0.0032795
#> 5 0_p1      [0, +1]          0      1 5          791 -0.0062931
#> 6 m10_p10  [-10, +10]       -10     10 1          793 0.0093638
#> 7 m10_p10  [-10, +10]       -10     10 2          793 0.0047579
#> 8 m10_p10  [-10, +10]       -10     10 3          792 -0.00017347
#> 9 m10_p10  [-10, +10]       -10     10 4          791 -0.0030068
#> 10 m10_p10 [-10, +10]       -10     10 5          791 -0.0094161
#> # i 15 more rows
#> # i 7 more variables: median_CAR <dbl>, standard_deviation_CAR <dbl>,
#> #   positive_CAR_ratio <dbl>, standard_error_CAR <dbl>, t_value <dbl>,
#> #   p_value <dbl>, significance <chr>
```

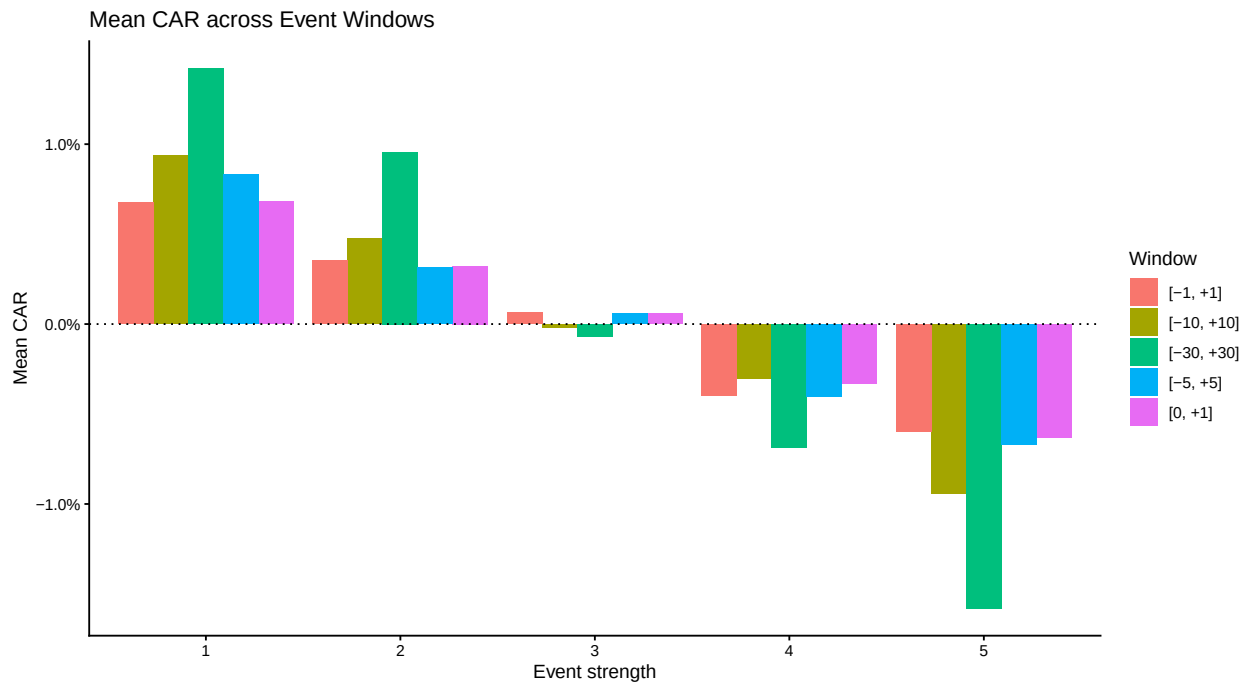
```
plot_short_window_car <- tbl_event_window_return |>
  filter(window_ID == "m1_p1") |>
  ggplot(
    aes(
      x = event_strength,
      y = CAR_window
    )
  ) +
  geom_boxplot(
    outlier.alpha = 0.25
  ) +
  geom_hline(
    yintercept = 0,
    linetype = "dotted"
  ) +
  scale_y_continuous(
    labels = scales::label_percent()
  ) +
  labs(
    x = "Event strength",
    y = "CAR [-1, +1]",
    title = "Short-Window CAR by Earnings Surprise"
  ) +
  theme_classic()
```

```
plot_short_window_car
```



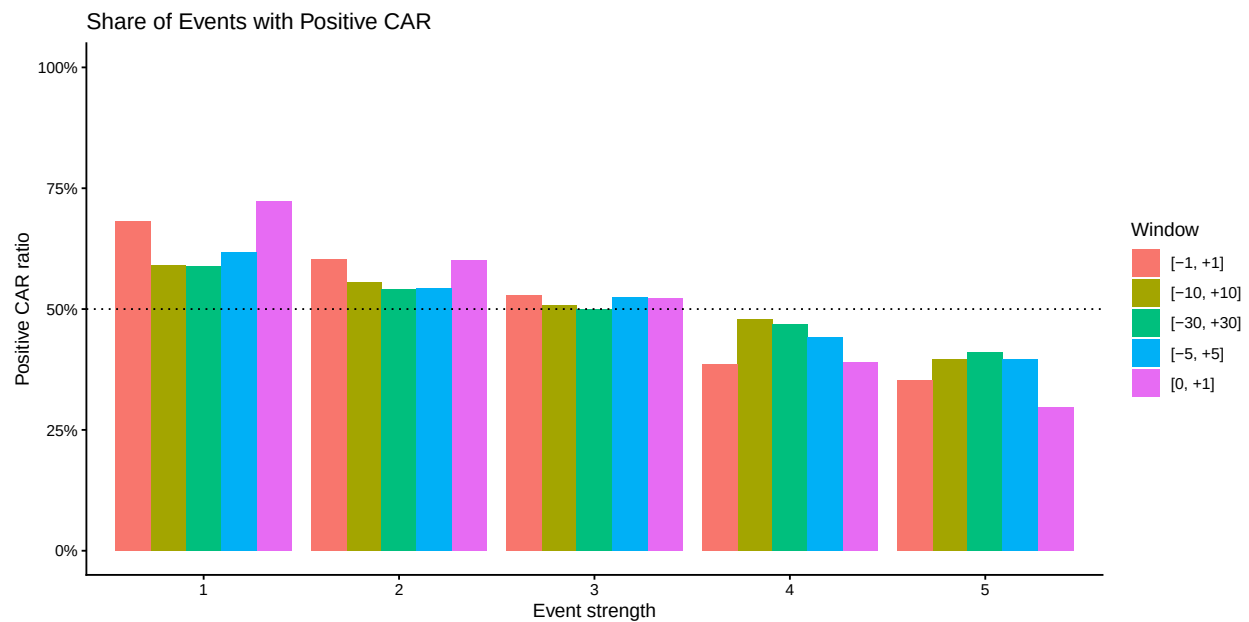
```
plot_window_mean_car <- tbl_event_window_summary |>
  ggplot(
    aes(
      x = event_strength,
      y = mean_CAR,
      fill = window_label
    )
  ) +
  geom_col(
    position = "dodge"
  ) +
  geom_hline(
    yintercept = 0,
    linetype = "dotted"
  ) +
  scale_y_continuous(
    labels = scales::label_percent()
  ) +
  labs(
    x = "Event strength",
    y = "Mean CAR",
    fill = "Window",
    title = "Mean CAR across Event Windows"
  ) +
  theme_classic()

plot_window_mean_car
```



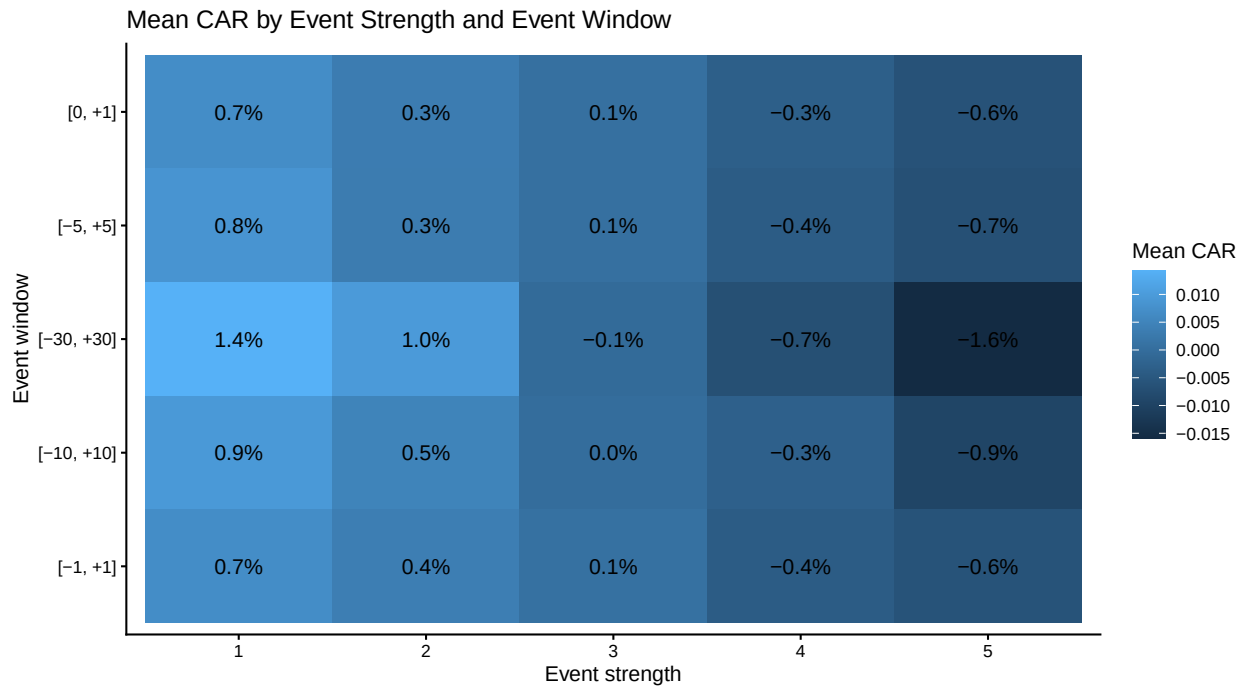
```
plot_positive_car_ratio <- tbl_event_window_summary |>
  ggplot(
    aes(
      x = event_strength,
      y = positive_CAR_ratio,
      fill = window_label
    )
  ) +
  geom_col(
    position = "dodge"
  ) +
  geom_hline(
    yintercept = 0.5,
    linetype = "dotted"
  ) +
  scale_y_continuous(
    labels = scales::label_percent(),
    limits = c(0, 1)
  ) +
  labs(
    x = "Event strength",
    y = "Positive CAR ratio",
    fill = "Window",
    title = "Share of Events with Positive CAR"
  ) +
  theme_classic()

plot_positive_car_ratio
```



```
plot_window_car_heatmap <- tbl_event_window_summary |>
  ggplot(
    aes(
      x = event_strength,
      y = window_label,
      fill = mean_CAR
    )
  ) +
  geom_tile() +
  geom_text(
    aes(
      label = scales::percent(
        mean_CAR,
        accuracy = 0.1
      )
    )
  ) +
  labs(
    x = "Event strength",
    y = "Event window",
    fill = "Mean CAR",
    title = "Mean CAR by Event Strength and Event Window"
  ) +
  theme_classic()

plot_window_car_heatmap
```



I-6.8 決算サプライズと短期 CAR

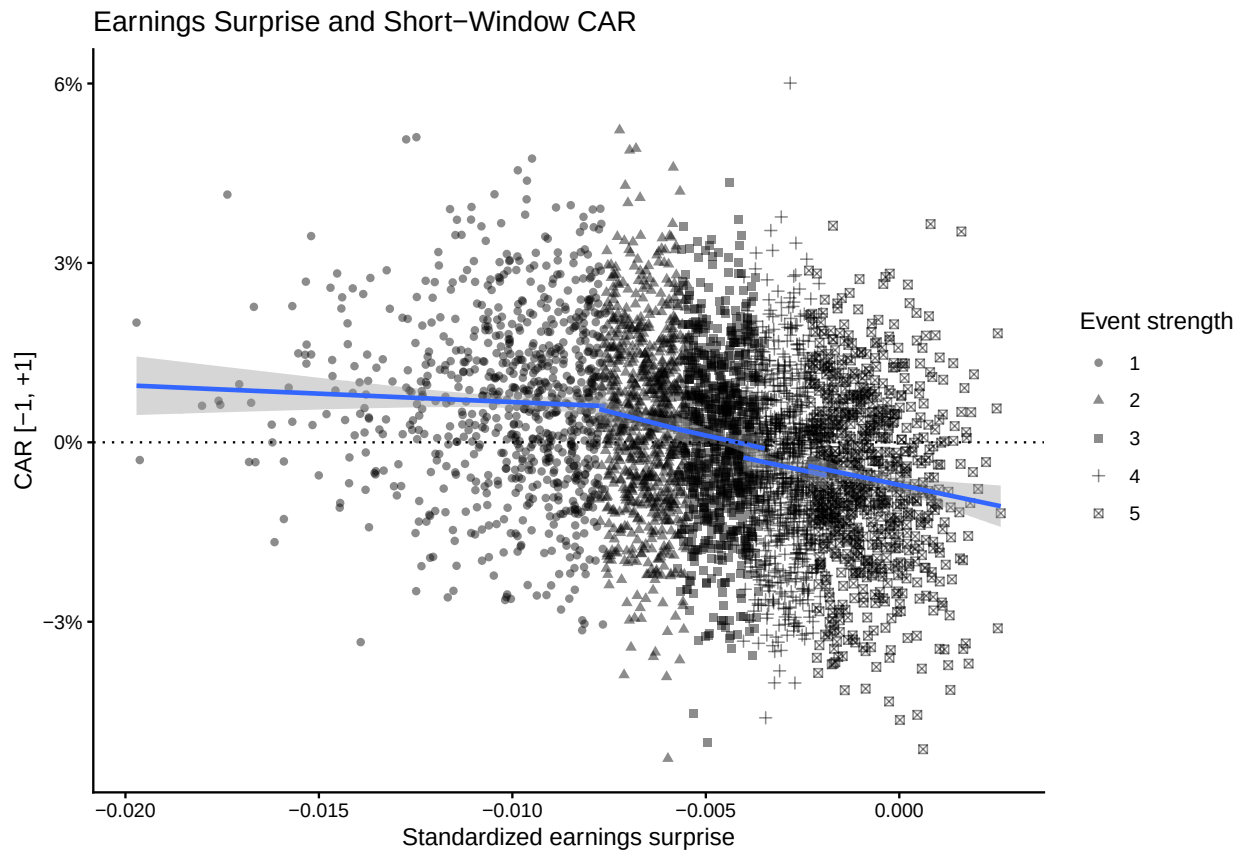
```
tbl_surprise_regression <- tbl_event_window_return |>
  filter(
    window_ID == "m1_p1",
    is.finite(earnings_surprise),
    is.finite(CAR_window)
  )

model_surprise_car <- lm(
  CAR_window ~ earnings_surprise,
  data = tbl_surprise_regression
)

broom::tidy(model_surprise_car)
#> # A tibble: 2 x 5
#>   term                estimate std.error statistic    p.value
#>   <chr>                <dbl>    <dbl>    <dbl>    <dbl>
#> 1 (Intercept)        -0.0066774 0.00041568 -16.064 2.6646e-56
#> 2 earnings_surprise -1.3714    0.069096 -19.848 1.1933e-83
broom::glance(model_surprise_car)
#> # A tibble: 1 x 12
#>   r.squared adj.r.squared sigma statistic    p.value    df logLik      AIC
#>   <dbl>      <dbl>    <dbl>    <dbl>    <dbl> <dbl> <dbl>    <dbl>
#> 1  0.090519    0.090289 0.014511    393.93 1.1933e-83     1 11144. -22282.
#> # i 4 more variables: BIC <dbl>, deviance <dbl>, df.residual <int>, nobs <int>
```

```
plot_surprise_car <- tbl_surprise_regression |>
  ggplot(
    aes(
      x = earnings_surprise,
      y = CAR_window,
      shape = event_strength
    )
  ) +
  geom_point(alpha = 0.45) +
  geom_smooth(
    method = "lm",
    se = TRUE
  ) +
  geom_hline(
    yintercept = 0,
    linetype = "dotted"
  ) +
  scale_y_continuous(
    labels = scales::label_percent()
  ) +
  labs(
    x = "Standardized earnings surprise",
    y = "CAR [-1, +1]",
    shape = "Event strength",
    title = "Earnings Surprise and Short-Window CAR"
  ) +
  theme_classic()

plot_surprise_car
```



```
plot_surprise_car_by_year <- tbl_surprise_regression |>
  ggplot(
    aes(
      x = earnings_surprise,
      y = CAR_window
    )
  ) +
  geom_point(alpha = 0.35) +
  geom_smooth(
    method = "lm",
    se = FALSE
  ) +
  facet_wrap(
    vars(event_year),
    scales = "free_x"
  ) +
  geom_hline(
    yintercept = 0,
    linetype = "dotted"
  ) +
  scale_y_continuous(
    labels = scales::label_percent()
  ) +
  labs(
    x = "Standardized earnings surprise",
```

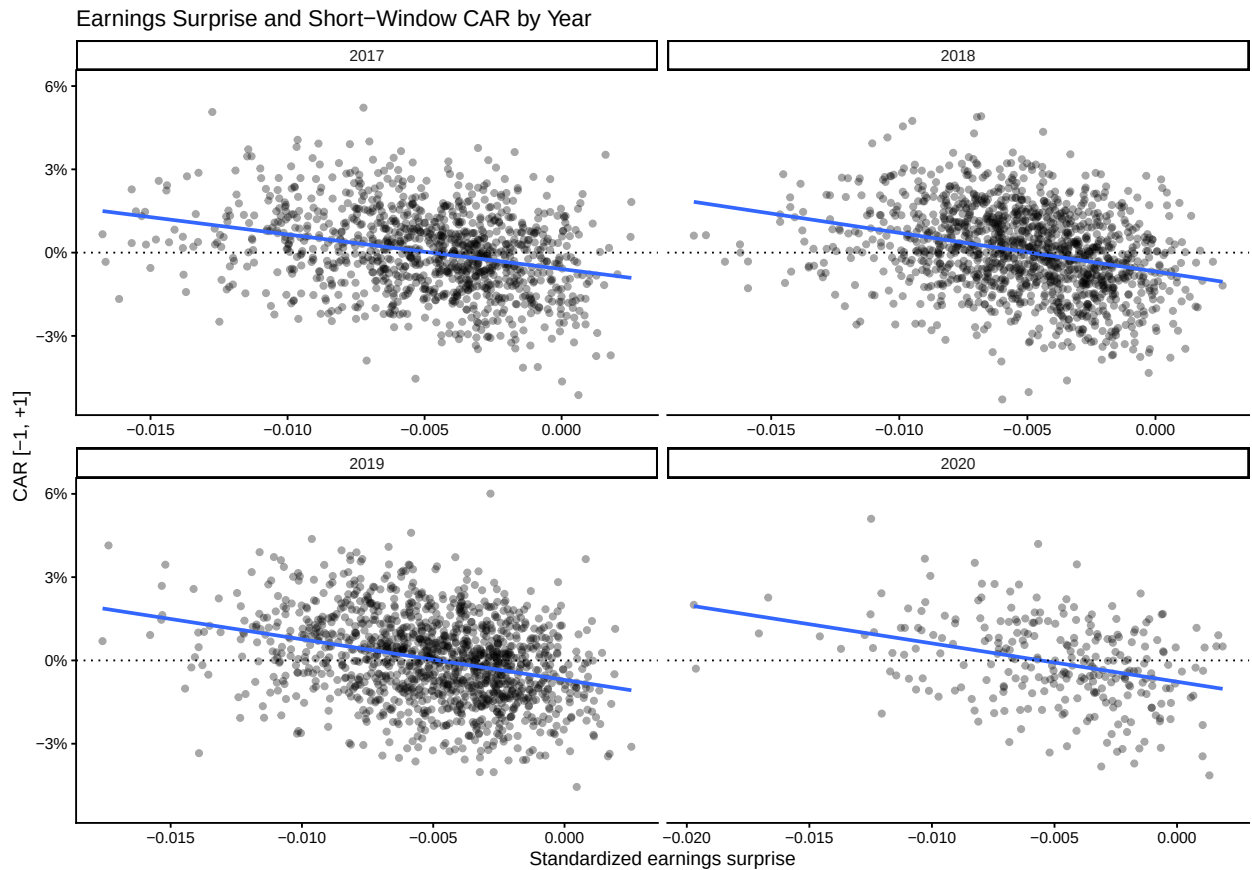


```

y = "CAR [-1, +1]",
title = "Earnings Surprise and Short-Window CAR by Year"
) +
theme_classic()

```

```
plot_surprise_car_by_year
```



I-6.9 分析結果の保存

```

tbl_event_window_wide <- tbl_event_window_return |>
  select(
    event_ID,
    window_ID,
    CAR_window
  ) |>
  pivot_wider(
    names_from = window_ID,
    values_from = CAR_window,
    names_prefix = "CAR_"
  )

tbl_event_output <- tbl_market_model |>

```

```
semi_join(  
  tbl_abnormal_return |>  
    distinct(event_ID),  
  by = "event_ID"  
) |>  
left_join(  
  tbl_event_window_wide,  
  by = "event_ID"  
) |>  
arrange(event_ID)
```

```
tbl_event_daily_output <- tbl_abnormal_return |>  
  select(  
    event_ID,  
    firm_ID,  
    event_date,  
    event_trade_date,  
    event_year,  
    event_strength,  
    earnings_surprise,  
    relative_day,  
    date,  
    R,  
    R_M,  
    normal_return,  
    AR,  
    CAR,  
    alpha,  
    beta,  
    sigma_AR,  
    r_squared,  
    estimation_observations  
  )  
  
tbl_event_summary_output <- tbl_event_daily_output |>  
  arrange(  
    event_strength,  
    relative_day  
  )
```

```
list_output <- list(  
  event_results = tbl_event_output,  
  event_daily = tbl_event_daily_output,  
  event_summary = tbl_event_summary_output
```

```

)

vec_output_file <- c(
  event_results = file.path(
    "events",
    "event_study_results.parquet"
  ),
  event_daily = file.path(
    "events",
    "event_study_daily.parquet"
  ),
  event_summary = file.path(
    "events",
    "event_study_summary.parquet"
  )
)

vec_output_path <- purrr::imap_chr(
  list_output,
  \(tbl_output, output_name)
  write_derived_parquet_file_LFL(
    tbl_output,
    vec_output_file[[output_name]]
  )
)

tibble(
  output_name = names(vec_output_path),
  path = relative_path_LFL(base::unname(vec_output_path))
)

#> # A tibble: 3 x 2
#>   output_name    path
#>   <chr>         <chr>
#> 1 event_results data/derived/events/event_study_results.parquet
#> 2 event_daily   data/derived/events/event_study_daily.parquet
#> 3 event_summary data/derived/events/event_study_summary.parquet

```

```

tbl_output_verification <- purrr::imap_dfr(
  list_output,
  \(tbl_expected, output_name) {
    output_file <- vec_output_file[[output_name]]
    output_path <- vec_output_path[[output_name]]

    tbl_check <-
      read_derived_parquet_file_LFL(

```

```

        output_file
    )

    stopifnot(
        nrow(tbl_check) ==
        nrow(tbl_expected)
    )

    stopifnot(
        identical(
            names(tbl_check),
            names(tbl_expected)
        )
    )

    tibble(
        output_name = output_name,
        output_file = output_path,
        rows = nrow(tbl_check),
        columns = ncol(tbl_check),
        size_bytes =
            file.info(output_path)$size
    )
}
)

tbl_output_verification
#> # A tibble: 3 x 5
#>   output_name output_file rows columns size_bytes
#>   <chr>       <chr>      <int>  <int>    <dbl>
#> 1 event_results C:/AnalyticFin/Projects/LagrangeFinan~ 3960    18    467503
#> 2 event_daily   C:/AnalyticFin/Projects/LagrangeFinan~ 241560   19    9222492
#> 3 event_summary C:/AnalyticFin/Projects/LagrangeFinan~ 305     15     33564

```

I-6.10 まとめ

本章では、決算発表の翌取引日をイベント日 0 として、マーケット・モデルによるイベントスタディを実施した。推定期間にはイベント日の 130 取引日前から 31 取引日前までの 100 日を使用し、イベント期間には 30 取引日前から 30 取引日後までの 61 日を使用した。

決算サプライズは、実現利益と予想利益の差を期首時価総額で標準化し、年度ごとに 5 群へ分類した。イベント別にマーケット・モデルを推定し、通常リターン、異常リターン、累積異常リターンを計算した。さらに、日次の AAR と CAAR、複数イベント・ウィンドウの CAR、正の

CAR となったイベントの割合、決算サプライズと短期 CAR の回帰関係を確認した。

反復処理には `purrr` を利用し、明示的なループは使用していない。データ探索、入力検証、イベント日 0 の設定、イベント・パネルの作成、マーケット・モデルの推定、AR・CAR の計算、イベント・ウィンドウ集計は、`utility_LFL.R` の共通関数へ分離した。

結果は次の 3 ファイルに保存される。

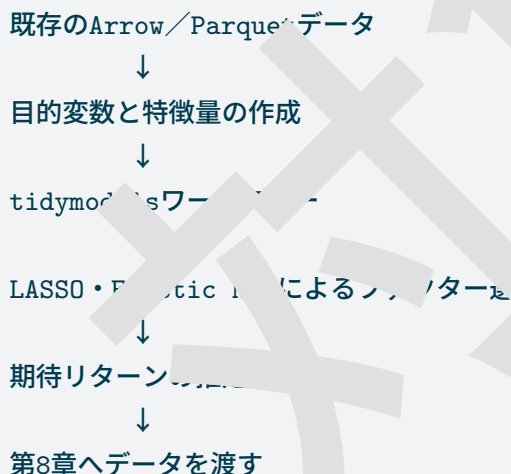
```
data/derived/events/event_study_results.parquet
data/derived/events/event_study_daily.parquet
data/derived/events/event_study_summary.parquet
```

第 1-7 章機械学習によるファクター選定

本章では、第 2 章から第 6 章までに作成した財務データ、株式リターン・ファクター・データおよびイベントスタディ結果を利用し、翌期リターンの予測に有効なファクターを機械学習によって選定する。

分析には `tidymodels` を用いる。目的は複雑なモデルを競わせることではなく、データ準備、前処理、学習、評価、予測という一連の流れを簡単に利用可能なワークフローとして理解することである。

本章では、次の順序で処理を進める。



1-7.1 機械学習によるファクター分析

第 4 章では、企業規模と簿価時価比率から SMB と HML を構築した。第 5 章では、それらのファクターを使って企業別の期待リターンを推定した。

本章では、あらかじめ少数の変数だけを選ぶのではなく、財務、価格、リスク、市場環境に関する複数の候補から、翌期リターンの予測に有効な変数を選定する。

LASSO は、回帰係数に罰則を加え、一部の係数を 0 にする。

$$\min_{\beta} \left[\frac{1}{N} \sum_{i=1}^N (y_i - x_i^{\top} \beta)^2 + \lambda \sum_{j=1}^p |\beta_j| \right]$$

係数が 0 にならなかった変数を、予測に利用されたファクターとして確認できる。

I-7.2 分析データの作成

```
tbl_financial <- read_derived_parquet_file_LFL(
  file.path(
    "fundamentals",
    "annual_firm_characteristics.parquet"
  )
)
```

```
tbl_monthly <- read_derived_parquet_file_LFL(
  file.path(
    "returns",
    "monthly_stock_returns.parquet"
  )
)
```

```
tbl_factor <- read_derived_parquet_file_LFL(
  file.path(
    "factors",
    "fama_french_factors.parquet"
  )
)
```

```
tbl_input_overview <- tribble(
  ~data, ~observations, ~variables,
  "annual_firm_characteristics",
  nrow(tbl_financial),
  ncol(tbl_financial),
  "monthly_stock_returns",
  nrow(tbl_monthly),
  ncol(tbl_monthly),
  "fama_french_factors",
  nrow(tbl_factor),
  ncol(tbl_factor)
)
```

```
show_table_LFL(tbl_input_overview)
#> # A tibble: 3 x 3
#>   data                                observations variables
#>   <chr>                                <int>         <int>
#> 1 annual_firm_characteristics          7920           31
#> 2 monthly_stock_returns              95040           39
#> 3 fama_french_factors                 60            16
```

年度 t の情報から、年度 $t + 1$ の超過リターンを予測する。

$$y_{i,t} = R_{i,t+1}^e$$

```
tbl_target <- tbl_financial |>
  mutate(
    firm_ID = as.integer(firm_ID),
    year = as.integer(year)
  ) |>
  arrange(firm_ID, year) |>
  group_by(firm_ID) |>
  mutate(
    forward_excess_return = forward_excess_return,
    prediction_year = year + 1L
  ) |>
  ungroup()
```

```
tbl_price_moments <- tbl_moments |>
  mutate(
    firm_ID = as.integer(firm_ID),
    year = as.integer(year)
  ) |>
  filter(is.finite(forward_excess_return)) |>
  group_by(firm_ID, year) |>
  summarise(
    momentum = prod(1 + R) - 1,
    volatility = sd(R),
    downside_risk = sqrt(mean(pmin(R, 0)^2)),
    average_market_equity = mean(ME, na.rm = TRUE),
    .groups = "drop"
  )
```



```
tbl_market_features <- tbl_factor |>
  group_by(year) |>
  summarise(
    market_premium = prod(1 + R_Me) - 1,
    smb_return = prod(1 + SMB) - 1,
    hml_return = prod(1 + HML) - 1,
    market_volatility = sd(R_Me),
    .groups = "drop"
  )
```

```
vec_candidate_features <- c(
  "ROE",
  "BE",
  "ME",
  "lagged_BE",
  "lagged_ME",
  "lagged_BEME",
  "sales",
  "X",
  "NOA",
  "NFO"
)

vec_available_features <- intersect(
  vec_candidate_features,
  names(tbl_target)
)

tbl_ml <- tbl_target %>%
  select(
    firm_ID,
    year,
    prediction_year,
    forward_excess_return,
    any_of("industry_ID"),
    all_of(vec_available_features)
  ) |>
  left_join(
    tbl_price_features,
    by = c("firm_ID", "year")
  ) |>
  left_join(
    tbl_market_features,
    by = "year"
  ) |>
```

```

mutate(
  across(
    where(is.numeric),
    \(x) replace(x, !is.finite(x), NA_real_)
  )
) |>
filter(!is.na(forward_excess_return)) |>
arrange(year, firm_ID)

show_table_LFL(tbl_ml, n = 20)
#> # A tibble: 6,405 x 23
#>   fir_ID year pre_year for_retur ind_ID ROE ME lag_BE
#>   <dbl> <dbl>   <dbl>      <dbl> <fct>  <dbl>   <dbl>   <dbl>
#> 1     1     2015     2016  0.99547     1    NA      93620.  70434234000 NA
#> 2     2     2015     2016  0.37395     1    NA      1054.  408000000 NA
#> 3     3     2015     2016  0.37398     1    NA      93620.  70434234000 NA
#> 4     4     2015     2016 -0.18631     1    NA      2285.  14518674000 NA
#> 5     5     2015     2016  0.20702     1    NA      587.  9856474000 NA
#> 6     6     2015     2016  0.11198     1    NA      7571.9  6428920000 NA
#> 7     7     2015     2016 -0.05555     1    NA      12970.  6125536000 NA
#> 8     9     2015     2016  0.47111     1    NA      12151.  3983774000 NA
#> 9    10     2015     2016  0.15444     1    NA      14306.  5485326000 NA
#> 10   11     2015     2016  0.47444     1    NA      195685659000 NA
#> 11   12     2015     2016  0.19678     1    NA      489.68  935820000 NA
#> 12   13     2015     2016  0.15519     1    NA      98895.  157028040000 NA
#> 13   14     2015     2016 -0.15666     1    NA      3888.3  12791475000 NA
#> 14   15     2015     2016 -0.20041     1    NA      5282.7  7087944000 NA
#> 15   16     2015     2016  0.19678     1    NA      25353  27627851000 NA
#> 16   17     2015     2016  0.22344     1    NA      212026.  111711138000 NA
#> 17   18     2015     2016  0.00093     1    NA      14701.  42414944000 NA
#> 18   20     2015     2016  0.19678     1    NA      14702.  19779450000 NA
#> 19   21     2015     2016  1.00095     1    NA      145766.  74118912000 NA
#> 20   22     2015     2016  0.16118     1    NA      10149.  9317574000 NA
#> # i 6,385 more rows
#> # i 14 more variables: lag_ME <dbl>, lag_BEME <dbl>, sales <dbl>, X <dbl>,
#> #   NOA <dbl>, Nfu <dbl>, momentum <dbl>, volatility <dbl>, dow_risk <dbl>,
#> #   ave_equit <dbl>, mar_premi <dbl>, smb_retur <dbl>, hml_retur <dbl>,
#> #   mar_volat <dbl>

```

I-7.3 tidymodels による学習

tidymodels では、前処理とモデルを別々に管理し、それらを workflow() でまとめる。

```
recipe
  ↓
model specification
  ↓
workflow
  ↓
resampling
  ↓
tuning
  ↓
prediction
```

```
factor_recipe <- recipes::recipe(
  forward_excess_return ~ .,
  data = tbl_ml
) |>
  recipes::update_role(
    firm_ID,
    year,
    prediction_year,
    new_role = "id"
  ) |>
  recipes::step_unknown(
    recipes::all_nominal_predictors()
  ) |>
  recipes::step_novel(
    recipes::all_nominal_predictors()
  ) |>
  recipes::step_dummy(
    recipes::all_nominal_predictors()
  ) |>
  recipes::step_center(
    recipes::all_numeric_predictors()
  ) |>
  recipes::step_zv(
    recipes::all_predictors()
  ) |>
  recipes::step_normalize(
    recipes::all_numeric_predictors()
  )
```

ランダム分割ではなく、年度の順序を維持して学習期間と評価期間を分ける。

```

vec_analysis_years <- sort(unique(tbl_ml$year))

split_year <- vec_analysis_years[
  max(
    2L,
    floor(length(vec_analysis_years) * 0.7)
  )
]

tbl_training <- tbl_ml |>
  filter(year <= split_year)

tbl_testing <- tbl_ml |>
  filter(year > split_year)

tbl_split_overview <- tribble(
  ~sample, ~observations, ~first_year, ~last_year,
  "training",
  nrow(tbl_training),
  min(tbl_training$year),
  max(tbl_training$year),
  "testing",
  nrow(tbl_testing),
  min(tbl_testing$year),
  max(tbl_testing$year)
)

show_table_LFJ(tbl_split_overview)
#> # A tibble: 2 x 4
#>   sample observations first_year last_year
#>   <dbl>         <int>    <dbl>    <dbl>
#> 1 training      2779     2015     2017
#> 2 testing       2620     2018     2019

```

I-7.4 ファクター選定

LASSO、Ridge、Elastic Net は、同じ `glmnet` エンジンで表現できる。

- `mixture = 1` : LASSO
- `mixture = 0` : Ridge
- `0 < mixture < 1` : Elastic Net

```
elastic_net_spec <- parsnip::linear_reg(
  penalty = tune::tune(),
  mixture = tune::tune()
) |>
  parsnip::set_engine("glmnet")

elastic_net_workflow <- workflows::workflow() |>
  workflows::add_recipe(factor_recipe) |>
  workflows::add_model(elastic_net_spec)
```

本ドラフトでは、まず小さなグリッドを使って処理全体が動くことを確認する。

```
tbl_parameter_grid <- tidyr::crossing(
  penalty = c(0.0001, 0.001, 0.01, 0.1),
  mixture = c(0, 0.5, 1)
)
```

```
set.seed(1234)

validation_folds <- rsample::vfold_cv(
  tbl_training,
  v = min(5L, nrow(tbl_training))
)
```

```
factor_tuning <- tune::tune_grid(
  elastic_net_workflow,
  resamples = validation_folds,
  grid = tbl_parameter_grid,
  metrics = yardstick::rmse,
  yardstick::rmse,
  yardstick::rmse
)
)
```

```
best_parameters <- tune::select_best(
  factor_tuning,
  metric = "rmse"
)
```

```
show_table_LFL(best_parameters)
#> # A tibble: 1 x 3
#>   penalty mixture .config
#>   <dbl>   <dbl> <chr>
#> 1  0.001     0.5 pre0_mod05_post0
```

```
final_factor_workflow <- tune::finalize_workflow(
  elastic_net_workflow,
  best_parameters
)

final_factor_fit <- parsnip::fit(
  final_factor_workflow,
  data = tbl_training
)
```

選択された変数を係数から確認する。

```
tbl_selected_factors <- final_factor_fit |>
  workflows::extract_fit_parsnip() |>
  broom::tidy() |>
  filter(
    term != "(Intercept)",
    estimate != 0
  ) |>
  arrange(desc(abs(estimate)))

show_table_LFL(tbl_selected_factors, 30)

#> # A tibble: 22 x 3
#>   term                estimate p_value
#>   <chr>              <dbl>     <dbl>
#> 1 volatility          0.52925     0.001
#> 2 lagged_ME           0.01087     0.001
#> 3 momentum            0.03888     0.001
#> 4 market_return       0.27678     0.001
#> 5 hml_return          -0.02659     0.001
#> 6 hml_return          -0.02574     0.001
#> 7 market_volatility  -0.02509     0.001
#> 8 industry_ID_X7       0.02235     0.001
#> 9 ME                  -0.01900     0.001
#> 10 industry_ID_X10     0.01759     0.001
#> 11 industry_ID_X5     -0.01514     0.001
#> 12 lagged_ME          0.01513     0.001
#> 13 X                  -0.01124     0.001
#> 14 ROE                 -0.00997     0.001
#> 15 industry_ID_X2      0.00946     0.001
#> 16 industry_ID_X6     -0.00920     0.001
#> 17 industry_ID_X3      0.00882     0.001
#> 18 industry_ID_X9     -0.00587     0.001
#> 19 NOA                 0.00490     0.001
```

```
#> 20 lagged_BE      -0.0034932  0.001
#> 21 BE              0.0023084  0.001
#> 22 industry_ID_X8  0.0015864  0.001
```

I-7.5 期待リターンの推定

```
tbl_test_predictions <- predict(
  final_factor_fit,
  new_data = tbl_testing
) |>
  bind_cols(
    tbl_testing |>
      select(
        firm_ID,
        year,
        prediction_year,
        forward_excess_return
      )
  ) |>
  rename(
    predicted_return = .prediction_year,
    realized_return = forward_excess_return
  )
```

```
tbl_prediction_metrics <- yardstick::metrics_set(
  yardstick::rmse,
  yardstick::mae,
  yardstick::rsq
)(
  tbl_test_predictions,
  truth = realized_return,
  estimate = predicted_return
)
```

```
show_table_LFL(tbl_prediction_metrics)
#> # A tibble: 3 x 3
#>   .metric .estimator .estimate
#>   <chr>   <chr>      <dbl>
#> 1 rmse    standard      3.9625
#> 2 mae     standard      3.0002
#> 3 rsq     standard      0.25561
```

```

final_factor_fit_all <- parsnip::fit(
  final_factor_workflow,
  data = tbl_ml
)

tbl_security_scores <- predict(
  final_factor_fit_all,
  new_data = tbl_ml
) |>
bind_cols(
  tbl_ml |>
  select(
    firm_ID,
    year,
    prediction_year,
    forward_excess_return
  )
) |>
rename(
  expected_return = .pred,
  realized_return = forward_excess_return
) |>
group_by(prediction_year) |>
mutate(
  score_rank = percent_rank(expected_return),
  score_decile = ntile(expected_return, 10)
) |>
ungroup()

```

I-7.6 ポートフォリオ候補の作成

ここでは最適化を行わずに、予測値によって銘柄を並べ、分位ポートフォリオを作成する。

```

tbl_decile_returns <- tbl_security_scores |>
  group_by(
    prediction_year,
    score_decile
  ) |>
  summarise(
    securities = n(),
    average_expected_return = mean(
      expected_return,
      na.rm = TRUE
    )
  )

```



```

),
average_realized_return = mean(
  realized_return,
  na.rm = TRUE
),
.groups = "drop"
)

```

```
show_table_LFL(tbl_decile_returns)
```

```

#> # A tibble: 50 x 5
#>   pre_year sco_decil securities ave_retur ave_retur
#>   <dbl>    <int>    <int>    <dbl>    <dbl>
#> 1  2016        1      125  0.040431 0.0093272
#> 2  2016        2      124  0.079922 0.110018
#> 3  2016        3      124  0.10226  0.010000
#> 4  2016        4      124  0.11709  0.098003
#> 5  2016        5      124  0.13332  0.11710
#> 6  2016        6      124  0.14901  0.14950
#> 7  2016        7      124  0.16400  0.21376
#> 8  2016        8      124  0.16600  0.22106
#> 9  2016        9      124  0.19540  0.20003
#> 10 2016       10      124  0.19000  0.10087
#> # i 40 more rows

```

最上位 10 分位と最下位 10 分位の差を確認する

```

tbl_long_short <- tbl_decile_returns %>%
  filter(score_decile %in% c(1L, 10L)) |>
  select(
    pre_year,
    score_decile,
    average_realized_return
  ) |>
  pivot_wider(
    names_from = score_decile,
    values_from = average_realized_return,
    names_prefix = "decile_"
  ) |>
  mutate(
    long_short_return =
      decile_10 - decile_1
  )

```

```
show_table_LFL(tbl_long_short)
```

```
#> # A tibble: 5 x 4
```

```
#>   pre_year      dec_1    dec_10 lon_retur
#>   <dbl>      <dbl>    <dbl>    <dbl>
#> 1    2016  0.0093272 0.21087    0.20154
#> 2    2017  0.041990 0.30981    0.26782
#> 3    2018 -0.18027  0.11281    0.29308
#> 4    2019  0.17750  0.64451    0.46701
#> 5    2020 -0.17070  0.024575   0.19527
```

I-7.7 第 8 章へのデータ保存

第 8 章では、期待リターン、過去リターン、投資対象銘柄を使って制約付きポートフォリオを構築する。

```
tbl_expected_returns <- tbl_security_scores |>
  transmute(
    rebalance_year = prediction_year,
    firm_ID,
    expected_return,
    score_rank,
    score_decile
  )

tbl_asset_return_history <- tbl_monthly_returns |>
  transmute(
    firm_ID = as.integer(firm_ID),
    month_ID = as.integer(month_ID),
    month_date = as.Date(month_date),
    return = R
  ) |>
  filter(is.finite(return))

tbl_investment_universe <- tbl_expected_returns |>
  left_join(
    tbl_financial |>
      transmute(
        rebalance_year = as.integer(year) + 1L,
        firm_ID = as.integer(firm_ID),
        market_equity = ME
      ),
    by = c(
      "rebalance_year", "firm_ID"
    )
  ) |>
  filter(
    is.finite(expected_return),
```

```

    is.finite(market_equity),
    market_equity > 0
)

```

```

tbl_chapter_07_outputs <- list(
  expected_returns = tbl_expected_returns,
  security_scores = tbl_security_scores,
  factor_selection = tbl_selected_factors,
  asset_return_history = tbl_asset_return_history,
  investment_universe = tbl_investment_universe
)

```

```

vec_chapter_07_files <- c(
  expected_returns = file.path(
    "portfolios",
    "expected_returns.parquet"
  ),
  security_scores = file.path(
    "factors",
    "security_scores.parquet"
  ),
  factor_selection = file.path(
    "factors",
    "factor_selection.parquet"
  ),
  asset_return_history = file.path(
    "portfolios",
    "asset_return_history.parquet"
  ),
  investment_universe = file.path(
    "portfolios",
    "investment_universe.parquet"
  )
)

```

```

vec_chapter_07_paths <- tbl_chapter_07_outputs |>
  imap_chr(
    \(data, name) {
      write_derived_parquet_file_LFL(
        data,
        vec_chapter_07_files[[name]]
      )
    }
  )

```

```
tibble::tibble(  
  output = base::names(vec_chapter_07_paths),  
  path = relative_path_LFL(base::unname(vec_chapter_07_paths))  
) |>  
  show_table_LFL()  
#> # A tibble: 5 x 2  
#>   output          path  
#>   <chr>         <chr>  
#> 1 expected_returns data/derived/portfolios/expected_returns.parquet  
#> 2 security_scores  data/derived/factors/security_scores.parquet  
#> 3 factor_selection data/derived/factors/factor_selection.parquet  
#> 4 asset_return_history data/derived/portfolios/asset_return_history.parquet  
#> 5 investment_universe data/derived/portfolios/investment_universe.parquet
```

本章では、既存の財務データ、株式リターンおよびファクター・データから機械学習用データを作成し、tidymodels のワークフローによってファクター選定と期待リターン推定を行った。

第 8 章では、この期待リターンを制約付きポートフォリオ最適化へ接続する。

第 I-8 章制約付きポートフォリオ最適化とバックテスト

本章では、これまでに作成した株式リターン履歴を用いて、制約付きポートフォリオ最適化を実装する。第 5 章で確認した平均分散法を出発点とし、最小分散、最大リターン、最大シャープレシオ、最小期待ショートフォール、リスク・パブリッシング、目標リターン、目標リスクへと拡張する。

計算には GaussQuant のポートフォリオ API を用いる。これにより、最適化問題そのものを本章内で再実装するのではなく、投資ユニバース、リターンデータ、制約条件、最適化手法を分離して記述できる。

株式リターン履歴



投資ユニバース



`portfolio_spec GQI`

制約条件

`portfolio_optimizer GQI()`



最適ウェイト



Efficient Frontier



バックテスト

I-8.1 ポートフォリオ最適化の基本

銘柄数を N 、ポートフォリオ・ウェイトを w 、期待リターンを μ 、分散共分散行列を Σ とする。

$$\mu_p = w^\top \mu, \quad \sigma_p^2 = w^\top \Sigma w$$

完全投資かつロングオンリー、銘柄別上限を $w_{\max}$ とすると、

$$\sum_{i=1}^N w_i = 1, \quad 0 \leq w_i \leq w_{\max}$$

である。GaussQuant では、これらを目的関数の内部に直接書き込まず、ポートフォリオ仕様と制約条件として登録する。

I-8.2 投資ユニバースとリターンデータ

```
tbl_expected_returns <- read_derived_parquet_file_LFL(
  file.path("portfolios", "expected_returns.parquet")
)

tbl_return_history <- read_derived_parquet_file_LFL(
  file.path("portfolios", "asset_return_history.parquet")
)

tbl_investment_universe <- read_derived_parquet_file_LFL(
  file.path("portfolios", "investment_universe.parquet")
)
```

```
tbl_input_overview <- tibble::tibble(
  ~data_observations, ~data_variables,
  "expected_returns", nrow(tbl_expected_returns), ncol(tbl_expected_returns),
  "asset_return_history", nrow(tbl_return_history), ncol(tbl_return_history),
  "investment_universe", nrow(tbl_investment_universe),
  ~ ncol(tbl_investment_universe)
)
```

```
show_table_LFL(tbl_input_overview)
```

```
#> # A tibble: 3 x 3
#>   data               observations variables
#>   <chr>              <int>      <int>
#> 1 expected_returns      6405         5
#> 2 asset_return_history  93525        4
#> 3 investment_universe   6405         6
```

```

rebalance_year <- tbl_investment_universe |>
  summarise(rebalance_year = min(rebalance_year, na.rm = TRUE)) |>
  pull(rebalance_year)

tbl_universe_one_year <- tbl_investment_universe |>
  filter(rebalance_year == !!rebalance_year) |>
  arrange(desc(market_equity)) |>
  slice_head(n = 20)

vec_assets <- tbl_universe_one_year |>
  pull(firm_ID) |>
  as.character()

```

`portfolio_optimize_GQL()` は long 形式のリターン履歴から `firm_ID` を資産名、`month_ID` を時点として利用する。

```

tbl_returns_GQL <- tbl_return_history |>
  filter(firm_ID %in% as.integer(vec_assets)) |>
  transmute(
    date = month_ID,
    asset = as.character(firm_ID),
    return = as.numeric(return)
  ) |>
  arrange(date, asset)

```

第 7 章で推定した期待リターンは投資ユニークな形式や結果比較に利用できる。一方、現行の `portfolio_optimize_GQL()` は入力されたリターン履歴から標本平均と分散共分散行列を推定する。ここでは、この GaussQuant の計算仕様に合わせてリターン履歴を直接入力する。

I-8.3 ポートフォリオ仕様と制約条件

```

portfolio_spec <- GaussQuant::portfolio_spec_GQL(
  assets = vec_assets
)

maximum_weight <- 0.20

portfolio_spec <- portfolio_spec |>
  GaussQuant::portfolio_constraint_GQL(
    type = "weight_sum",
    min_sum = 1,
    max_sum = 1
  )

```

```

) |>
GaussQuant::portfolio_constraint_GQL(
  type = "box",
  min = 0,
  max = maximum_weight
)

```

```

tbl_portfolio_constraints <- portfolio_spec$constraints |>
  select(constraint_id, type, enabled)

```

```
show_table_LFL(tbl_portfolio_constraints)
```

```

#> # A tibble: 2 x 3
#>   con_id type      enabled
#>   <int> <chr>    <lgl>
#> 1     1 1 weight_sum TRUE
#> 2     2 2 box      TRUE

```

I-8.4 基本的なポートフォリオ最適化

最小分散

```

result_minimum_variance <- GaussQuant::portfolio_optimize_GQL(
  tbl_returns = tbl_returns_GQL,
  portfolio = portfolio_spec,
  method = "minimum_variance"
)

tbl_minimum_variance_weights <- result_minimum_variance$weights |>
  arrange(desc(weight))

show_table_LFL(tbl_minimum_variance_weights)
#> # A tibble: 20 x 2
#>   asset weight
#>   <chr>   <dbl>
#> 1  987    0.2
#> 2 1253   0.12876
#> 3  472   0.12851
#> 4 1205   0.12772
#> 5  710   0.11864
#> 6  725   0.10204
#> 7  996   0.096426

```



```
#> 8 1408 0.031944
#> 9 721 0.029151
#> 10 1237 0.019244
#> # i 10 more rows
```

$$\max_w \left[w^\top \mu - \frac{\gamma}{2} w^\top \Sigma w \right]$$

```
result_mean_variance <- GaussQuant::portfolio_optimize_GQL(
  tbl_returns = tbl_returns_GQL,
  portfolio = portfolio_spec,
  method = "mean_variance",
  risk_aversion = 5
)
```

```
result_maximum_return <- GaussQuant::portfolio_optimize_GQL(
  tbl_returns = tbl_returns_GQL,
  portfolio = portfolio_spec,
  method = "maximum_return"
)
```

$$R = \frac{\mu}{\sigma}$$

```
risk_free_rate <- 0

result_minimum_sharpe <- GaussQuant::portfolio_optimize_GQL(
  tbl_returns = tbl_returns_GQL,
  portfolio = portfolio_spec,
  method = "minimum_sharpe",
  risk_free_rate = risk_free_rate
)
```

I-8.5 期待ショートフォールとリスク・バジエッティング

損失を L 、信頼水準を α とする。期待ショートフォールは、市場 VaR を超えるテール損失の平均として扱う。

```
confidence_level <- 0.95

result_minimum_ES <- GaussQuant::portfolio_optimize_GQL(
```

```
tbl_returns = tbl_returns_GQL,
portfolio = portfolio_spec,
method = "minimum_ES",
confidence_level = confidence_level
)
```

期待ショートフォールは次章の市場リスク計測で詳しく扱う。本章では、分散だけでなく損失分布のテールを目的関数にできることを確認する。

ポートフォリオ分散への資産 i の寄与は、

$$RC_i = w_i(\Sigma w)_i$$

で表される。ここでは各資産の目標リスク寄与を等しく設定する。

```
risk_budget <- rep(
  1 / length(vec_assets),
  length(vec_assets)
)

result_risk_budget <- GaussQu + port + risk_budget + GQL
tbl_returns = tbl_returns_GQL
portfolio = portfolio_spec,
method = "risk_budget",
risk_budget = risk_budget
)
```

I-8.6 目標リターンと目標リスク

```
minimum_return <- result_minimum_variance$statistics |>
  filter(metric == "expected_return") |>
  pull(value)

maximum_return <- result_maximum_return$statistics |>
  filter(metric == "expected_return") |>
  pull(value)

target_return <- mean(c(minimum_return, maximum_return))
```

```
result_target_return <- GaussQuant::portfolio_optimize_GQL(
  tbl_returns = tbl_returns_GQL,
  portfolio = portfolio_spec,
  method = "target_return",
  target_return = target_return
)
```

```
minimum_volatility <- result_minimum_variance$statistics |>
  filter(metric == "volatility") |>
  pull(value)
```

```
maximum_return_volatility <- result_maximum_return$statistics |>
  filter(metric == "volatility") |>
  pull(value)
```

```
target_volatility <- mean(
  c(minimum_volatility, maximum_return_volatility)
)
```

```
result_target_risk <- GaussQuant::portfolio_optimize_GQL(
  tbl_returns = tbl_returns_GQL,
  portfolio = portfolio_spec,
  method = "target_risk",
  target_volatility = target_volatility
)
```

I-8.7 最適化手法の比較

```
lst_optimization_results <- list(
  minimum_variance = result_minimum_variance,
  mean_variance = result_mean_variance,
  maximum_return = result_maximum_return,
  maximum_sharpe = result_maximum_sharpe,
  minimum_ES = result_minimum_ES,
  risk_budget = result_risk_budget,
  target_return = result_target_return,
  target_risk = result_target_risk
)
```

```
tbl_optimization_comparison <- purrr::imap_dfr(
  lst_optimization_results,
  \(result, method_name) {
```

```

result$statistics |>
  mutate(method = method_name) |>
  select(method, metric, value)
}
) |>
pivot_wider(
  names_from = metric,
  values_from = value
)

show_table_LFL(tbl_optimization_comparison)
#> # A tibble: 8 x 6
#>   method      exp_return volatility  variance sha_ratio  min_short
#>   <chr>      <dbl>      <dbl>    <dbl>    <dbl>    <dbl>
#> 1 minimum_variance 0.011530  0.036210 0.00131  0.3111  0.00003
#> 2 mean_variance    0.020610  0.042072 0.001770 0.4895  0.06563
#> 3 maximum_return   0.025119  0.058955 0.003475 0.5111  0.083785
#> 4 maximum_sharpe   0.020657  0.049693 0.002469 0.41570 0.081022
#> 5 minimum_ES       0.016009  0.045921 0.002108 0.34862 0.068515
#> 6 risk_budget      0.012763  0.027500 0.000328 0.29845 0.080010
#> 7 target_return    0.018311  0.017130 0.001407 0.47333 0.064364
#> 8 target_risk      0.018411  0.041111 0.002115 0.38782 0.077334

```

各手法は同じ「最適ポートフォリオ」を求めているわけではない。最小分散は分散、最大リターンは期待リターン、最小 ES はテール損失、リスク・バジレットはリスク寄与の割合をそれぞれ最適化している。

I-8.8 Efficient Frontier

目標期待リターンを指定しながら最小分散ポートフォリオを求めることで Efficient Frontier を構成できる。ここでは、GaussQuant の `target_return` 最適化を繰り返し、専用の `portfolio_frontier` 関数 `QL()` を導入する前段階として最適化 API と Efficient Frontier の関係を確認する。

```

vec_target_returns <- seq(
  minimum_return,
  maximum_return,
  length.out = 31
)[1:30]

```

最大リターン点そのものは `result_maximum_return` で別途計算済みである。したがって、`target_return` によるフロンティア計算では最大リターン境界を 1 点だけ除外し、境界上での

数値的な退化を避ける。

```
lst_frontier_results <- purrr::map(
  vec_target_returns,
  \(target_value) {
    GaussQuant::portfolio_optimize_GQL(
      tbl_returns = tbl_returns_GQL,
      portfolio = portfolio_spec,
      method = "target_return",
      target_return = target_value
    )
  }
)

tbl_efficient_frontier <- purrr::map2_dfr(
  lst_frontier_results,
  vec_target_returns,
  \(result, target_value) {
    result$statistics |>
      filter(
        metric %in% c(
          "expected_return",
          "volatility",
          "sharpe_ratio"
        )
      ) |>
      select(metric, value) |>
      pivot_wider(
        names_from = metric,
        values_from = value
      )
      mutate(
        target_return = target_value,
        .before = 1
      )
  }
)
```

```
plot_efficient_frontier <- ggplot(
  tbl_efficient_frontier,
  aes(
    x = volatility,
    y = expected_return
  )
) +
```

```

geom_line() +
geom_point() +
labs(
  x = "Volatility",
  y = "Expected Return"
)

show_plot_LFL(plot_efficient_frontier)

```

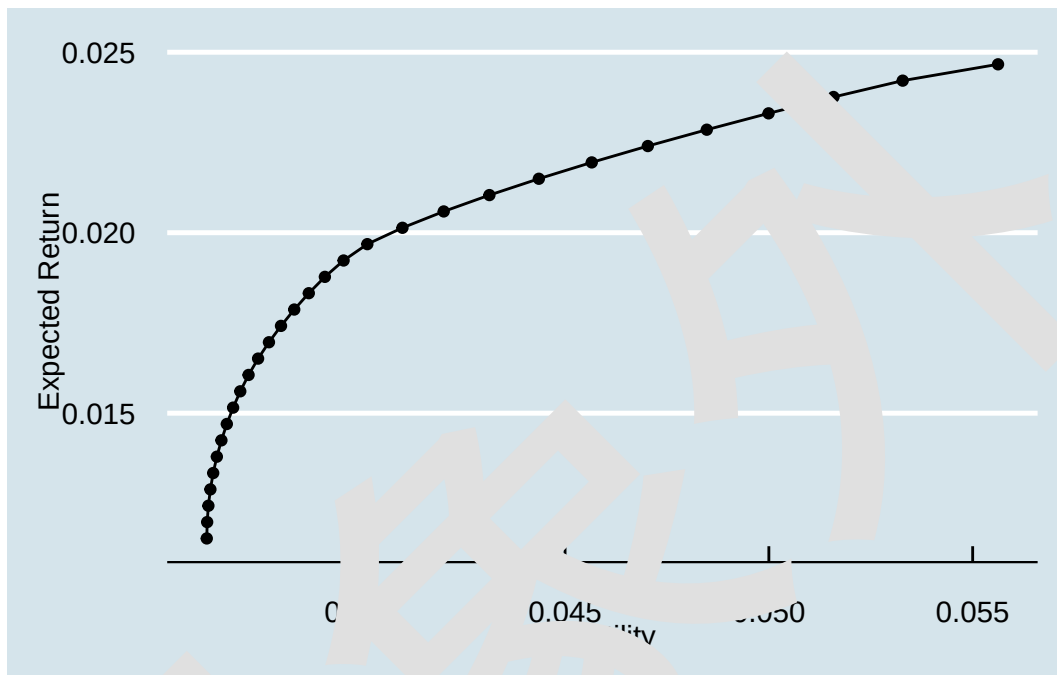


Figure 8-1 制約付きポートフォリオの Efficient Frontier

最小分散ポートフォリオと最大シャープレシオポートフォリオが重なる。

```

tbl_key_portfolio <- bind_rows(
  result_minimum_variance$statistics |>
  pivot_wider(names_from = metric, values_from = value) |>
  mutate(portfolio = "Minimum Variance"),
  result_maximum_sharpe$statistics |>
  pivot_wider(names_from = metric, values_from = value) |>
  mutate(portfolio = "Maximum Sharpe"),
  result_maximum_return$statistics |>
  pivot_wider(names_from = metric, values_from = value) |>
  mutate(portfolio = "Maximum Return")
)

```

```

plot_efficient_frontier_key_points <- ggplot(
  tbl_efficient_frontier,
  aes(

```

```

    x = volatility,
    y = expected_return
  )
) +
  geom_line() +
  geom_point() +
  geom_point(
    data = tbl_key_portfolios,
    aes(x = volatility, y = expected_return),
    size = 3
  ) +
  geom_text(
    data = tbl_key_portfolios,
    aes(
      x = volatility,
      y = expected_return,
      label = portfolio
    ),
    nudge_y = 0.00005,
    check_overlap = TRUE
  ) +
  labs(
    x = "Volatility",
    y = "Expected Return"
  )
)

show_plot_LFL(plot_efficient_frontier, y_points,

```

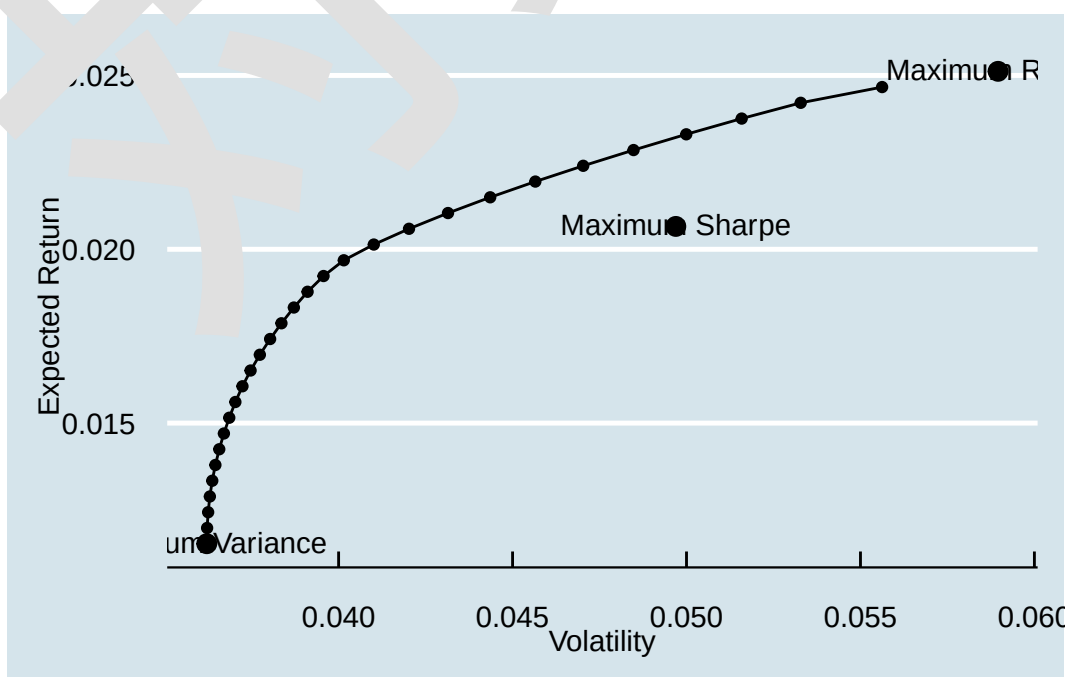
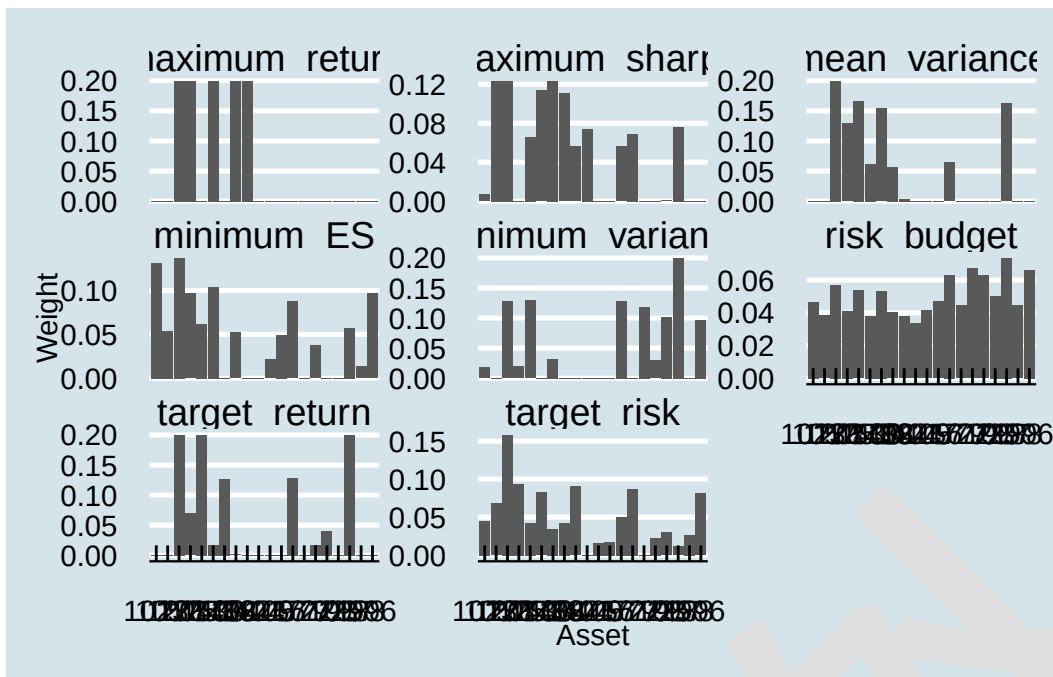


Figure3: Efficient Frontier と代表的ポートフォリオ

I-8.9 最適ウェイトの比較

```
tbl_weight_comparison <- purrr::imap_dfr(  
  lst_optimization_results,  
  \(result, method_name) {  
    result$weights |>  
      mutate(method = method_name)  
  }  
)
```

```
plot_weight_comparison <- ggplot(  
  tbl_weight_comparison,  
  aes(  
    x = asset,  
    y = weight  
  )  
) +  
  geom_col() +  
  facet_wrap(  
    vars(method),  
    scales = "free_y"  
  ) +  
  labs(  
    x = "Asset",  
    y = "Weight"  
  ) +  
  theme(  
    axis_text.x = element_text(  
      angle = 90,  
      hjust = 1  
    )  
  )  
)  
  
show_plot_LFL(plot_weight_comparison)
```

I-8.10 アウト・オブ・サンプル評価

最適化の結果は推定期間内の目的関数で評価すべきではない。推定期間で求めたウェイトをその後のリターンへ適用し、アウト・オブ・サンプル評価する。

```
tbl_next_year_returns <- tbl_returns %>%
  filter(
    firm_ID %in% as.integer(vec_asset),
    lubridate::year(month_date) == as.integer(year) + 1
  ) |>
  transmute(
    month_date = month_date,
    asset = as.character(firm_ID),
    return = as.numeric(return)
  )
```

最大シャープレシオ・ポートフォリオを例として評価する。

```
tbl_backtest <- tbl_next_year_returns |>
  inner_join(
    result_maximum_sharpe$weights,
    by = "asset"
  ) |>
  group_by(month_date) |>
  summarise(
    portfolio_return = sum(
      weight * return,
      na.rm = TRUE
    )
  )
```

```

),
.groups = "drop"
) |>
arrange(month_date) |>
mutate(
  wealth_index = cumprod(
    1 + portfolio_return
  )
)

```

```

plot_backtest <- ggplot(
  tbl_backtest,
  aes(
    x = month_date,
    y = wealth_index
  )
) +
geom_line() +
labs(
  x = NULL,
  y = "Wealth Index"
)
show_plot_LFL(plot_backtest)

```

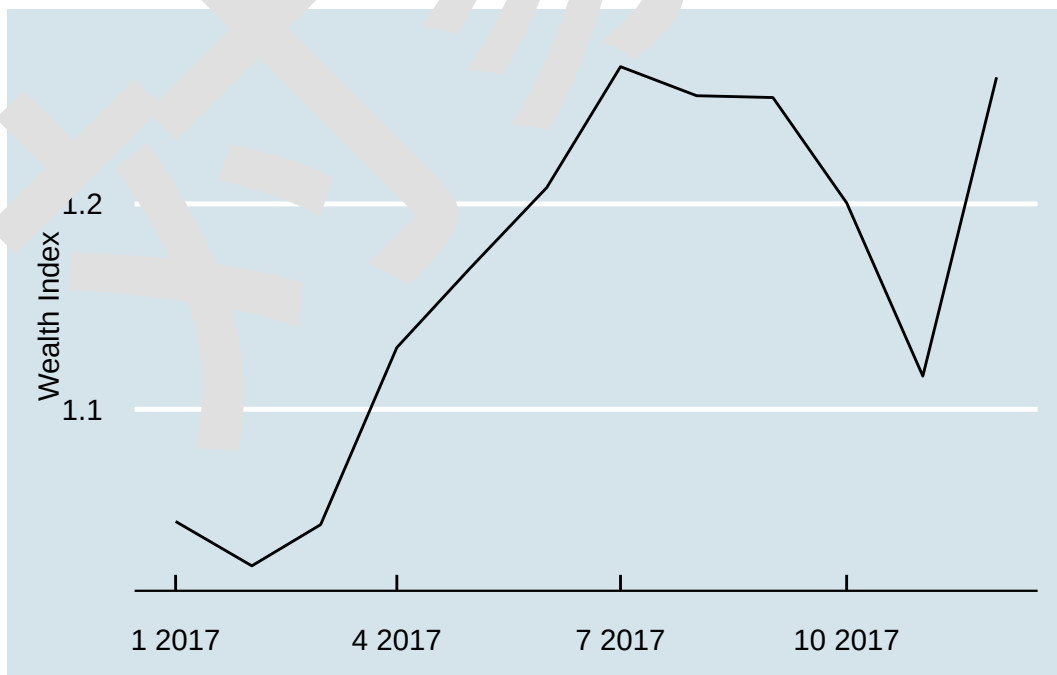


Figure4: 最大シャープレシオ・ポートフォリオのアウト・オブ・サンプル推移

```
tbl_backtest_summary <- tbl_backtest |>
  summarise(
    observations = n(),
    cumulative_return = prod(1 + portfolio_return) - 1,
    mean_return = mean(portfolio_return),
    volatility = sd(portfolio_return),
    minimum_return = min(portfolio_return),
    maximum_return = max(portfolio_return)
  )

show_table_LFL(tbl_backtest_summary)
#> # A tibble: 1 x 6
#>   observations cum_retur mea_retur volatility min_retur max_retur
#>   <int>      <dbl>    <dbl>      <dbl>    <dbl>    <dbl>
#> 1      12    0.26165  0.020890   0.054658 -0.070265  0.13034
```

I-8.11 本章のまとめ

本章では、ポートフォリオ最適化を GaussQuant の統一 API で実装した。最適化ロジックを章内に直接記述する方法から、ポートフォリオ仕様、制約条件、最適化手法を分離する構成へ変更した。

最小分散、平均分散、最大リターン、最大シャープレシオに加えて、期待ショートフォール、リスク・バジューティング、目標リターン、目標リスクを同じ API で扱うことで、ポートフォリオ最適化を一般的なリスク配分問題として捉えることができる。

また、目標リターンを連続的に変化させることで Efficient Frontier を構成し、最小分散点と最大シャープレシオ点を同一平面上で確認した。

次章では、ポートフォリオの損失分布そのものに焦点を移し、市場 VaR と期待ショートフォールを中心に市場リスク計測を扱う。

第 I-9 章市場リスクと市場 VaR

本章では、市場リスク管理の中心的な尺度である市場 Value at Risk（市場 VaR）を扱う。最初に、市場 VaR を中心として発展してきた市場リスク規制を、バーゼル II、バーゼル 2.5、Fundamental Review of the Trading Book (FRTB) という流れから整理する。そのうえで、分散共分散法、ヒストリカル法、モンテカルロ法による市場 VaR を実装し、期待ショートフォール (ES)、リスク寄与度、バックテストへ進む。

本章の目的は FRTB の標準的方式を再現することではない。FRTB を市場リスク管理の歴史的背景として位置づけ、市場 VaR と ES の考え方を、R によるデータ処理とシミュレーションを通して理解することにある。

I-9.1 FRTB とは何か

1990 年代以降、金融機関の市場リスク管理では市場 VaR が広く利用されるようになった。市場 VaR は、一定の保有期間と信頼水準のもとで、ポートフォリオに生じ得る損失を一つの数値として表現する。この簡潔さから、市場 VaR は内部リスク管理だけでなく、規制上の市場リスク計測にも組み込まれていった。

しかし、2007 年から 2008 年にかけて進行した世界金融危機は、従来の市場リスク管理の限界を明らかにした。特にトレーディング勘定では、危機前に想定されていた水準を大きく上回る損失が発生した。市場流動性の急激な低下、信用スプレッドの拡大、証券化商品の価格下落、複数のリスク要因の同時変動などが重なり、平常時の過去データを中心に推定されたリスク尺度だけでは、危機時の損失を十分に捉えられない場合があることが認識された。

さらに、大規模金融機関の経営危機は金融システム全体へ波及し得るため、各国では金融システム安定化のための公的支援が実施された。この経験は「Too Big To Fail」をめぐる議論を強め、個別金融機関の内部管理だけでなく、金融システム全体の安定性を意識した規制改革を促すことになった。

危機への比較的早い対応が、一般にバーゼル 2.5 と呼ばれる市場リスク規制の強化である。従来の市場 VaR に加えて、深刻な市場ストレス期を反映した stressed VaR が導入され、トレー

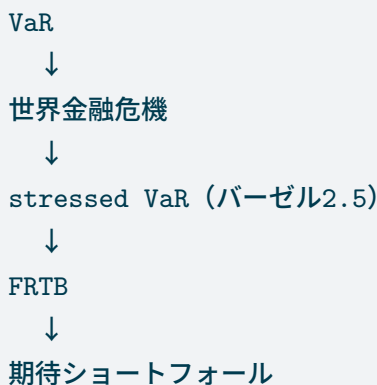
ディング勘定の信用リスクについても追加的な資本賦課が強化された。

ただし、これは危機への対応の終点ではなかった。トレーディング勘定と銀行勘定の境界、流動性の異なるリスク要因の扱い、内部モデルの比較可能性、市場 VaR が損失分布の裾の大きさを直接表現しないことなど、より構造的な問題が残った。

そこでバーゼル銀行監督委員会は、市場リスク規制を根本から見直す FRTB を策定した。2016 年に市場リスクの最低所要自己資本に関する基準が公表され、2019 年に改訂された最終基準が公表された。

FRTB の重要な特徴の一つが、内部モデル方式において中心的なリスク尺度を市場 VaR から期待ショートフォールへ移したことである。市場市場 VaR が損失分布のある分位点を示すのに対し、ES はその分位点を越えた領域の損失を平均する。そのため、極端な市場変動が発生した場合の損失の大きさをより直接的に捉えることができる。

したがって、市場リスク計測の発展は概念的には次のように整理できる。



日本ではバーゼル III 最終化に伴う市場リスク規制が段階的に導入され、国際統一基準金融機関および内部モデルを採用する国内基準金融機関には 2024 年 3 月 31 日から、内部モデルを採用しない国内基準金融機関等には 2025 年 3 月 31 日から適用されている。

本章では FRTB の標準的方式の詳細な資本計算には立ち入らない。ここからは、FRTB に至る市場リスク管理の基礎となった市場 VaR そのものを理解する。

I-9.2 市場 VaR とは何か

時点 t のポートフォリオ価値を V_t とすると、1 期間の損益は

$$\text{PnL}_{t+1} = V_{t+1} - V_t$$

である。市場リスク管理では損失を正の値として扱うため、損失変数を

$$L_{t+1} = -\text{PnL}_{t+1} = V_t - V_{t+1}$$

と定義する。

信頼水準 α における市場 VaR は、損失分布の α 分位点として

$$\text{VaR}_\alpha = \inf \{l : P(L \leq l) \geq \alpha\}$$

と定義される。

例えば 95% 市場 VaR が 1 億円であるとは、モデルと観測期間を前提としたとき、損失が 1 億円以下となる確率が 95% であることを意味する。残り 5% の領域では 1 億円を超える損失が発生し得る。市場 VaR そのものは、その 5% の領域で損失がどこまで拡大するかを示さない。

I-9.3 市場データとポートフォリオ損益

本章では第 8 章と同じ株式リターン履歴を利用する。データは Parquet 形式で保存されているため、Arrow を通して読み込む。

```
tbl_return_history <- read_derived_parquet_file_LFL(
  file.path("portfolios", "asset_return_history.parquet")
)

tbl_investment_universe <- read_derived_parquet_file_LFL(
  file.path("portfolios", "investment_universe.parquet")
)
```

最初のリバランス年について時価総額上位 20 銘柄を選択し、ここでは市場リスク尺度そのものに焦点を当てるため、等ウェイト・ポートフォリオを構成する。

```
risk_year <- tbl_investment_universe |>
  summarise(risk_year = min(rebalance_year, na.rm = TRUE)) |>
  pull(risk_year)

vec_assets <- tbl_investment_universe |>
  filter(rebalance_year == !!risk_year) |>
  arrange(desc(market_equity)) |>
  slice_head(n = 20) |>
  pull(firm_ID)

tbl_portfolio_returns <- tbl_return_history |>
```

```

filter(firm_ID %in% vec_assets) |>
mutate(asset = as.character(firm_ID)) |>
group_by(month_ID) |>
summarise(
  portfolio_return = mean(return, na.rm = TRUE),
  .groups = "drop"
) |>
arrange(month_ID)

show_table_LFL(tbl_portfolio_returns)
#> # A tibble: 71 x 2
#>   mon_ID   por_retur
#>   <int>     <dbl>
#> 1     2  0.10549
#> 2     3  0.091547
#> 3     4  0.063221
#> 4     5  0.077893
#> 5     6  0.10997
#> 6     7 -0.0023825
#> 7     8  0.00030174
#> 8     9  0.0060773
#> 9    10 -0.029288
#> 10    11  0.080602
#> # i 61 more rows

```

損失率を

$$L_t = -R_{p,t}$$

とする。

```

tbl_portfolio_losses <- tbl_portfolio_returns |>
  mutate(loss = -portfolio_return)

```

I-9.4 分散共分散法による市場 VaR

最も基本的な市場 VaR 計算法は、ポートフォリオ・リターンが正規分布に従うと仮定する分散共分散法である。

ポートフォリオ・リターンを

$$R_p \sim N(\mu_p, \sigma_p^2)$$

とすると、信頼水準 α の市場 VaR は

$$\text{VaR}_\alpha = -\mu_p + \Phi^{-1}(\alpha)\sigma_p$$

で与えられる。

```
confidence_level <- 0.95

mean_return <- mean(
  tbl_portfolio_returns$portfolio_return,
  na.rm = TRUE
)

volatility <- sd(
  tbl_portfolio_returns$portfolio_return,
  na.rm = TRUE
)

gaussian_var <- -mean_return +
  qnorm(confidence_level) * volatility

tbl_gaussian_var <- tibble(
  method = "Gaussian",
  confidence_level = confidence_level,
  VaR = gaussian_var
)

show_table_LFL(tbl_gaussian_var)
#> # A tibble: 1 x 3
#>   method   con_level   VaR
#>   <chr>       <dbl>   <dbl>
#> 1 Gaussian     0.95 0.064805
```

分散共分散法は高速で解釈しやすい。一方、正規分布という仮定によって、歪度やファットテールを十分に表現できない場合がある。

I-9.5 ヒストリカル法による市場 VaR

ヒストリカル市場 VaR では、特定の確率分布を仮定せず、実際に観測された損失分布から分位点を求める。

$$\text{VaR}_\alpha = Q_\alpha(L)$$

```
historical_var <- quantile(
  tbl_portfolio_losses$loss,
  probs = confidence_level,
  na.rm = TRUE,
  names = FALSE,
  type = 7
)

tbl_historical_var <- tibble(
  method = "Historical",
  confidence_level = confidence_level,
  VaR = historical_var
)

show_table_LFL(tbl_historical_var)
#> # A tibble: 1 x 3
#>   method      con_level    VaR
#>   <chr>         <dbl>    <dbl>
#> 1 Historical      0.95 0.066008
```

ヒストリカル法には分布仮定が不要という利点がある。しかし、将来発生する損失が過去の観測範囲に強く依存するという制約がある。

I-9.6 モンテカルロ法による市場 VaR

モンテカルロ法では、確率モデルから多数の市場シナリオを生成し、それぞれのシナリオでポートフォリオ損益を計算する。

ここでは正規分布モデルを用いる。シミュレーション規模が大きくなることを想定し、`furrr`により並列化できる構造とする。

```
future::plan(
  future::multisession,
```

```
workers = max(
  1,
  future::availableCores() - 1
)
)
```

```
set_random_seed_LFL(20260918)

n_scenarios <- 100000
n_batches <- 20
n_per_batch <- n_scenarios / n_batches

lst_simulated_returns <- furrr::future_map(
  seq_len(n_batches),
  \(batch_id) {
    tibble(
      simulated_return = rnorm(
        n = n_per_batch,
        mean = mean_return,
        sd = volatility
      )
    )
  },
  .options = furrr::furrr_options(
    seed = TRUE
  )
)

tbl_monte_carlo <- bind_rows(
  lst_simulated_returns
) |>
mutate(
  loss = -simulated_return
)
```

```
monte_carlo_var <- quantile(
  tbl_monte_carlo$loss,
  probs = confidence_level,
  na.rm = TRUE,
  names = FALSE
)

tbl_monte_carlo_var <- tibble(
  method = "Monte Carlo",
```

```

    confidence_level = confidence_level,
    VaR = monte_carlo_var
)

show_table_LFL(tbl_monte_carlo_var)
#> # A tibble: 1 x 3
#>   method      con_level    VaR
#>   <chr>         <dbl>    <dbl>
#> 1 Monte Carlo    0.95 0.064715

```

計算終了後は並列バックエンドを通常状態へ戻す。

I-9.7 3つの市場 VaR の比較

```

tbl_var_comparison <- bind_rows(
  tbl_gaussian_var,
  tbl_historical_var,
  tbl_monte_carlo_var
)

show_table_LFL(tbl_var_comparison)
#> # A tibble: 3 x 3
#>   method      con_level    VaR
#>   <chr>         <dbl>    <dbl>
#> 1 Gaussian    0.95 0.064805
#> 2 Historical  0.95 0.066008
#> 3 Monte Carlo 0.95 0.064715

```

```

plot_var_comparison <- tbl_var_comparison |>
  ggplot(
    aes(
      x = method,
      y = VaR
    )
  ) +
  geom_col() +
  labs(
    x = NULL,
    y = "市場VaR"
  )

show_plot_LFL(plot_var_comparison)

```

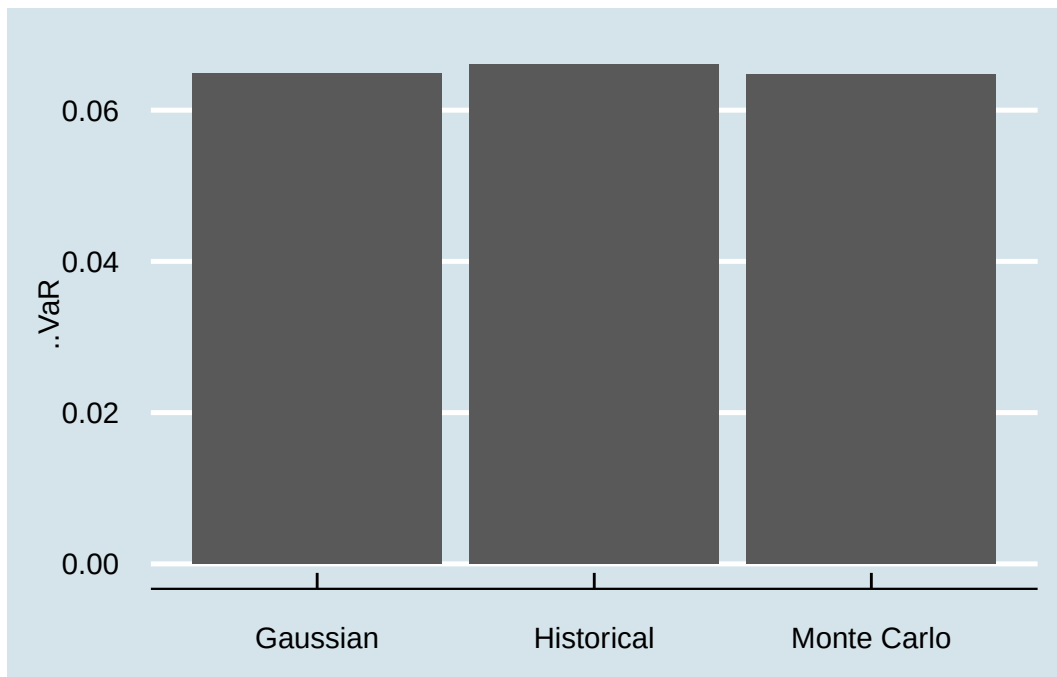


Figure5: 市場 VaR 計算法の比較

同じ信頼水準でも、市場 VaR は損失分布の推定方法によって異なる。したがって、市場 VaR は単なる一つの統計量ではなく、データ、モデル、保有期間、信頼水準と一体として解釈する必要がある。

I-9.8 期待ショートフォール

市場 VaR の重要な限界は、市場 VaR を超えた領域の損失の大きさを表現しないことである。

期待ショートフォール (ES) は、損失が市場 VaR を超えるテール領域の平均損失として直感的に理解できる。

$$ES_{\alpha} = E[L \mid L \geq \text{VaR}_{\alpha}]$$

実証分布では次のように計算できる。

```
historical_es <- tbl_portfolio_losses |>
  filter(loss >= historical_var) |>
  summarise(
    ES = mean(loss, na.rm = TRUE)
  ) |>
  pull(ES)

tbl_var_es <- tibble(
  measure = c("VaR", "期待ショートフォール"),
```

```
value = c(
  historical_var,
  historical_es
)
)

show_table_LFL(tbl_var_es)
#> # A tibble: 2 x 2
#>   measure          value
#>   <chr>          <dbl>
#> 1 VaR            0.066008
#> 2 期待ショートフォール 0.087030
```

```
plot_var_es <- tbl_var_es |>
  ggplot(
    aes(
      x = measure,
      y = value
    )
  ) +
  geom_col() +
  labs(
    x = NULL,
    y = "Loss"
  )

show_plot_LFL(plot_var_es)
```

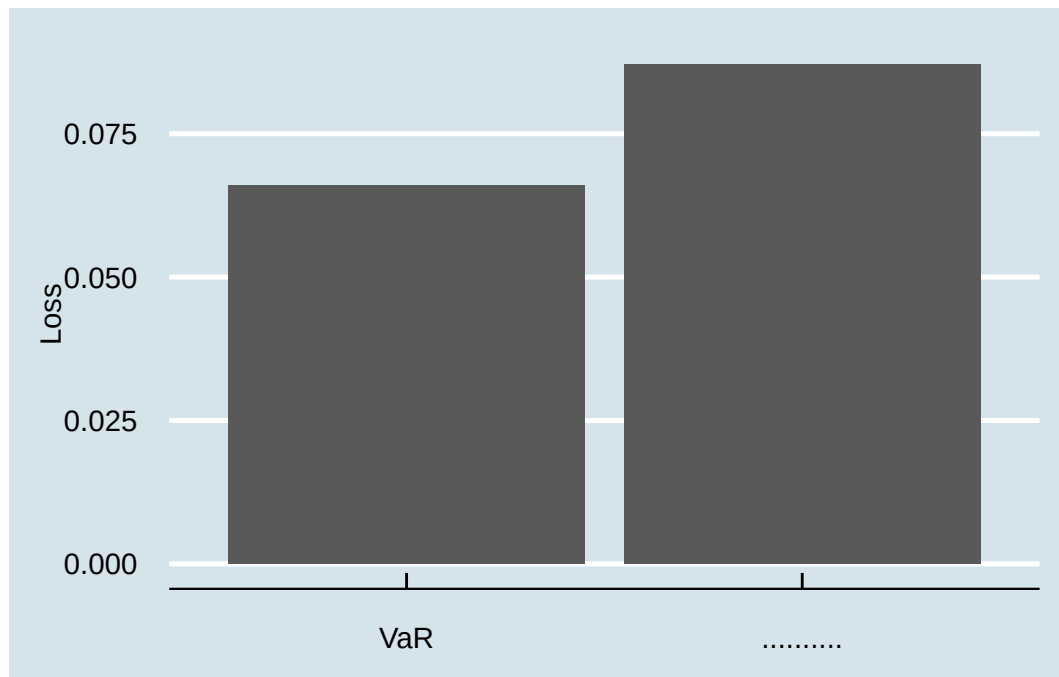


Figure6: 市場 VaR と期待ショートフォール

通常、同一の信頼水準では ES は市場 VaR 以上となる。市場 VaR がテール領域への入口を示すのに対し、ES はその先で平均的にどの程度の損失が生じるかを表す。

I-9.9 市場 VaR のリスク寄与度

ポートフォリオ全体の市場 VaR だけでは、どの資産がリスクを生み出しているかは分からない。分散共分散法では、市場 VaR を各資産の寄与に分解する考え方へ発展させることができる。

ポートフォリオ・ボラティリティを

$$\sigma_p = \sqrt{w^\top \Sigma w}$$

とすると、資産 i のボラティリティに対する限界寄与は

$$\frac{\partial \sigma_p}{\partial w_i} = \frac{(\Sigma w)_i}{\sigma_p}$$

である。

成分リスク寄与は

$$RC_i = w_i \frac{(\Sigma w)_i}{\sigma_p}$$

となり、

$$\sum_i RC_i = \sigma_p$$

が成立する。この考え方は、第 8 章のリスク・バジェットティングと直接つながっている。

I-9.10 市場 VaR バックテスト

市場 VaR モデルは計算して終わりではない。実際の損失が市場 VaR を超えた回数を観測し、モデルが想定した信頼水準と整合しているかを確認する必要がある。

時点 t の損失を L_t 、市場 VaR を VaR_t とすると、超過指標を

$$I_t = \mathbf{1}(L_t > \text{VaR}_t)$$

と定義する。

ここでは説明を単純にするため、全期間から求めたヒストリカル市場 VaR に対する超過回数を確認する。

```
tbl_var_backtest <- tbl_portfolio_losses |>
  mutate(
    VaR = historical_var,
    exception = loss > VaR
  )

tbl_var_backtest_summary <- tbl_var_backtest |>
  summarise(
    observations = n(),
    exceptions = sum(exception, na.rm = TRUE),
    exception_rate = mean(exception, na.rm = TRUE),
    expected_exception_rate =
      1 - confidence_level
  )

show_table_LFL(
  tbl_var_backtest_summary
)
```

```
#> # A tibble: 1 x 4
#>   observations exceptions exc_rate exp_rate
#>   <int>         <int>    <dbl>    <dbl>
#> 1         71           4 0.056338 0.050000
```

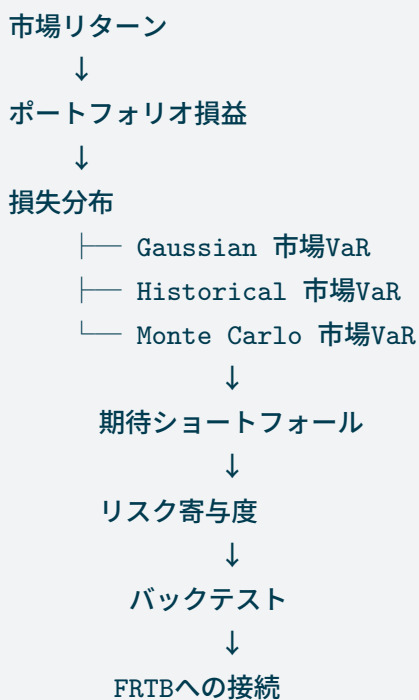
実務的なバックテストでは、各時点で利用可能であった過去データのみを使って市場 VaR を推定し、翌期の実現損失と比較する。したがって、ローリング・ウィンドウによる VaR 推定へ拡張することが重要である。

I-9.11 FRTB と期待ショートフォール

世界金融危機後の市場リスク規制は、単に市場 VaR のパラメータを変更するだけではなく、市場ストレス、流動性、モデル化可能性、トレーディング勘定の境界などを含めて再設計された。

FRTB の内部モデル方式では期待ショートフォールが中心的な市場リスク尺度となる。これは「市場 VaR が誤っていて ES が正しい」という単純な置き換えではない。市場 VaR は現在でもリスク管理やバックテストを理解するうえで重要な基礎概念であり、ES を理解するためにも市場 VaR の分位点という考え方が出発点となる。

本章で扱った流れは次のように整理できる。



第 10 章では、市場価格の変動ではなく企業のデフォルトによって生じる信用損失を扱う。One-Factor Gaussian モデルを用いて関連したデフォルトを生成し、ポートフォリオ損失分布から信用 VaR を計算する。

市場 VaR と信用 VaR は対象とするリスクが異なる一方、

リスク要因 → 損失シナリオ → 損失分布 → テールリスク

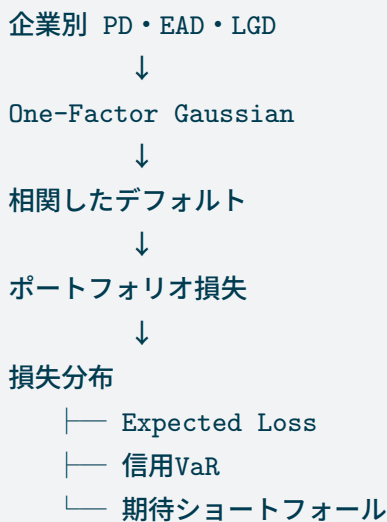
という計算構造を共有している。この構造はさらに第 11 章の CLO トランシェ評価へつながっていく。

第 I-10 章信用リスクと信用 VaR

第 9 章では、市場価格の変動からポートフォリオ損失分布を構築し、市場 VaR と期待ショートフォール（ES）によってテールリスクを計測した。本章では同じ考え方を信用リスクへ拡張する。

市場リスクでは株価や金利などの市場変数が損益を生み出す。信用リスクでは、企業のデフォルトと回収率が損失を生み出す。さらに重要なのは、企業のデフォルトが互いに独立ではないことである。景気悪化のような共通要因によって、多数の企業の信用状態が同時に悪化する可能性がある。

本章では One-Factor Gaussian モデルを使ってこの依存関係を表現する。



この損失分布は、第 11 章でそのまま CLO のトランシェ損失へ変換する。

I-10.1 信用リスクとポートフォリオ損失

企業 i の信用損失は、基本的に Exposure at Default（EAD）、Loss Given Default（LGD）、デフォルト指標の積として表せる。

$$L_i = EAD_i LGD_i \mathbf{1}_{\{Default_i\}}$$

ポートフォリオ全体の損失は

$$L = \sum_{i=1}^N EAD_i LGD_i \mathbf{1}_{\{Default_i\}}$$

となる。

企業 i のデフォルト確率を PD_i とすると、期待損失は

$$EL_i = PD_i EAD_i LGD_i$$

であり、ポートフォリオでは

$$EL = \sum_{i=1}^N PD_i EAD_i LGD_i$$

となる。

しかし、リスク管理で問題となるのは平均的な損失だけではない。多数の企業が同時にデフォルトした場合に、損失分布の裾がどこまで広がるかを把握する必要がある。

I-10.2 デフォルト相関

仮にすべての企業のデフォルトが独立なら、十分に分散されたポートフォリオでは個別企業のデフォルトリスクの多くを分散できる。

実際には、企業は共通のマクロ経済環境にさらされている。

景気後退、金利上昇、資金調達環境の悪化などは、複数企業の信用状態へ同時に影響する。そのため、信用ポートフォリオではデフォルト間の依存関係をモデル化する必要がある。

本章では、各企業の信用状態を一つの共通因子と企業固有因子に分解する。

I-10.3 One-Factor Gaussian モデル

企業 i の潜在的な資産状態を

$$X_i = \sqrt{\rho_i} Z + \sqrt{1 - \rho_i} \epsilon_i$$

とする。

ここで、

$$Z \sim N(0, 1)$$

はすべての企業に共通するシステムティック・ファクター、

$$\epsilon_i \sim N(0, 1)$$

は企業固有のファクターである。

Z とすべての ϵ_i は互いに独立とする。

ρ_i は企業 i の潜在変数が共通因子から受ける影響の強さを表す。共通の ρ を仮定した場合、異なる企業 i, j の潜在変数の相関は

$$\text{Corr}(X_i, X_j) = \rho$$

となる。

企業 i は

$$X_i < \Phi^{-1}(PD_i)$$

となったときデフォルトすると定義する。

この構造により、各企業の無条件デフォルト確率を PD_i に保ちながら、共通因子を通じたデフォルト依存を表現できる。

I-10.4 条件付きデフォルト確率

共通因子 $Z = z$ が与えられたとき、企業 i の条件付きデフォルト確率は

$$PD_i(z) = \Phi \left(\frac{\Phi^{-1}(PD_i) - \sqrt{\rho_i} z}{\sqrt{1 - \rho_i}} \right)$$

となる。

z が大きく低下すると、条件付き PD は上昇する。つまり、共通因子が悪化したシナリオでは多数企業のデフォルト確率が同時に上昇する。

これが One-Factor Gaussian モデルにおけるデフォルト依存の中心である。

I-10.5 信用ポートフォリオの作成

まず、モデルの動きを明確にするため、100 社からなる仮想的な信用ポートフォリオを作成する。

```
set_random_seed_LFL(20260918)

n_obligors <- 100L

tbl_credit_portfolio <- tibble(
  obligor_ID = seq_len(n_obligors),
  PD = seq(
    0.005,
    0.05,
    length.out = n_obligors
  ),
  EAD = 100,
  LGD = 0.60,
  rho = 0.20
) |>
mutate(
  expected_loss = PD * EAD * LGD
)

show_table_LFL(tbl_credit_portfolio)
#> # A tibble: 100 x 6
#>   obl_ID      PD    EAD   LGD   rho exp_loss
#>   <int>    <dbl> <dbl> <dbl> <dbl>   <dbl>
#> 1     1 0.005    100  0.6  0.2  0.3
#> 2     2 0.0054545 100  0.6  0.2 0.32727
#> 3     3 0.0059091 100  0.6  0.2 0.35455
#> 4     4 0.0063636 100  0.6  0.2 0.38182
#> 5     5 0.0068182 100  0.6  0.2 0.40909
#> 6     6 0.0072727 100  0.6  0.2 0.43636
#> 7     7 0.0077273 100  0.6  0.2 0.46364
#> 8     8 0.0081818 100  0.6  0.2 0.49091
#> 9     9 0.0086364 100  0.6  0.2 0.51818
#> 10    10 0.0090909 100  0.6  0.2 0.54545
#> # i 90 more rows
```

ポートフォリオの総 EAD と期待損失を確認する。

```
tbl_credit_portfolio_summary <- tbl_credit_portfolio |>
  summarise(
    obligors = n(),
    total_EAD = sum(EAD),
    expected_loss = sum(expected_loss),
    expected_loss_rate =
      expected_loss / total_EAD
  )

show_table_LFL(
  tbl_credit_portfolio_summary
)
```

```
#> # A tibble: 1 x 4
#>   obligors tot_EAD exp_loss exp_rate
#>   <int>    <dbl>   <dbl>   <dbl>
#> 1     100  10000    165   0.0165
```

I-10.6 共通因子と条件付き PD

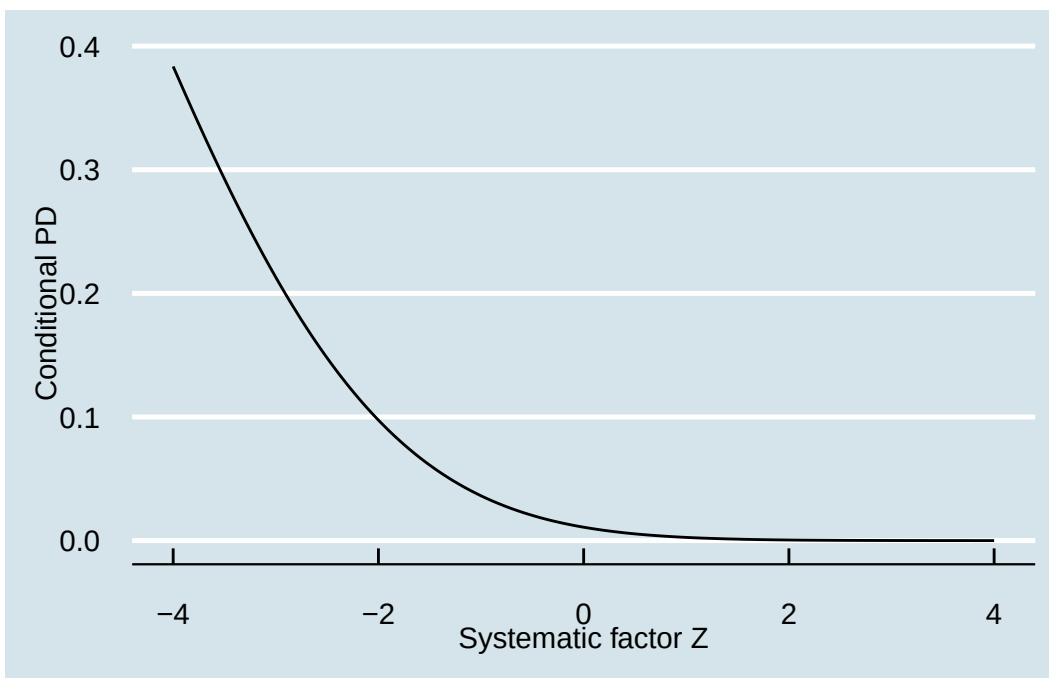
代表的な企業について、共通因子 Z の変化が条件付き PD へどのように影響するかを確認する。

```
representative_PD <- 0.02
representative_rho <- 0.20

tbl_conditional_PD <- tibble(
  systematic_factor = seq(
    -4,
    4,
    length.out = 200
  )
) |>
mutate(
  conditional_PD = pnorm(
    (
      qnorm(representative_PD) -
      sqrt(representative_rho) *
      systematic_factor
    ) /
    sqrt(1 - representative_rho)
  )
)
```

```
plot_conditional_PD <- tbl_conditional_PD |>
  ggplot(
    aes(
      x = systematic_factor,
      y = conditional_PD
    )
  ) +
  geom_line() +
  labs(
    x = "Systematic factor Z",
    y = "Conditional PD"
  )

show_plot_LFL(plot_conditional_PD)
```



共通因子が悪化するほど条件付き PD が急速に上昇する。信用ポートフォリオのテール損失は、この共通因子の極端な悪化によって生じる。

I-10.7 モンテカルロ法による相関デフォルト

One-Factor Gaussian モデルをモンテカルロ法で実装する。

各シナリオで一つの共通因子 Z を生成し、各企業について独立な固有因子 ϵ_i を生成する。

```
future::plan(
  future::multisession,
  workers = max(
```

```

    1,
    future::availableCores() - 1
  )
)

```

シミュレーションはバッチ単位で並列化する。乱数は `furrr_options(seed = TRUE)` により再現可能とする。

```

set_random_seed_LFL(20260918)

n_scenarios <- 100000L
n_batches <- 20L
n_per_batch <- n_scenarios / n_batches

vec_PD <- tbl_credit_portfolio$PD
vec_EAD <- tbl_credit_portfolio$EAD
vec_LGD <- tbl_credit_portfolio$LGD
vec_rho <- tbl_credit_portfolio$rho

lst_credit_losses <- furrr::future_map(
  seq_len(n_batches),
  \(batch_id) {
    purrr::map_dbl(
      seq_len(n_per_batch),
      \(scenario_id) {
        systematic_factor <- rnorm(1)

        vec_idiosyncratic <- rnorm(
          n_obligors
        )

        vec_latent_variable <-
          sqrt(vec_rho) *
            systematic_factor +
          sqrt(1 - vec_rho) *
            vec_idiosyncratic

        vec_default <-
          vec_latent_variable <
            qnorm(vec_PD)

        sum(
          vec_EAD *
            vec_LGD *
            vec_default

```



```

    )
  }
)
},
.options = furrr::furrr_options(
  seed = TRUE
)
)

tbl_credit_losses <- tibble(
  loss = unlist(
    lst_credit_losses,
    use.names = FALSE
  )
)

```

I-10.8 信用損失分布

シミュレーションされた信用損失分布を確認する。

```

tbl_credit_loss_summary <- tbl_credit_losses |>
  summarise(
    scenarios = n(),
    mean_loss = mean(loss),
    standard_deviation = sd(loss),
    minimum_loss = min(loss),
    maximum_loss = max(loss)
  )

show_table_LFL(
  tbl_credit_loss_summary
)
#> # A tibble: 1 x 5
#>   scenarios mea_loss sta_devia min_loss max_loss
#>   <int>     <dbl>    <dbl>    <dbl>    <dbl>
#> 1   100000    164.37    216.42         0    2820

```

理論上の期待損失とモンテカルロ平均を比較する。

```

theoretical_expected_loss <-
  sum(
    tbl_credit_portfolio$PD *
    tbl_credit_portfolio$EAD *

```

```

    tbl_credit_portfolio$LGD
  )

monte_carlo_expected_loss <-
  mean(tbl_credit_losses$loss)

tbl_expected_loss_comparison <- tibble(
  method = c(
    "Analytical",
    "Monte Carlo"
  ),
  expected_loss = c(
    theoretical_expected_loss,
    monte_carlo_expected_loss
  )
)

show_table_LFL(
  tbl_expected_loss_comparison
)
#> # A tibble: 2 x 2
#>   method      exp_loss
#>   <chr>      <dbl>
#> 1 Analytical    165
#> 2 Monte Carlo  164.37

```

両者が十分に近ければ、各企業の無条件 PD がシミュレーション上でも再現されていることを確認できる。

```

plot_credit_loss_distribution <- tbl_credit_losses |>
  ggplot(
    aes(x = loss)
  ) +
  geom_histogram(
    bins = 40
  ) +
  labs(
    x = "Portfolio loss",
    y = "Frequency"
  )

show_plot_LFL(
  plot_credit_loss_distribution
)

```

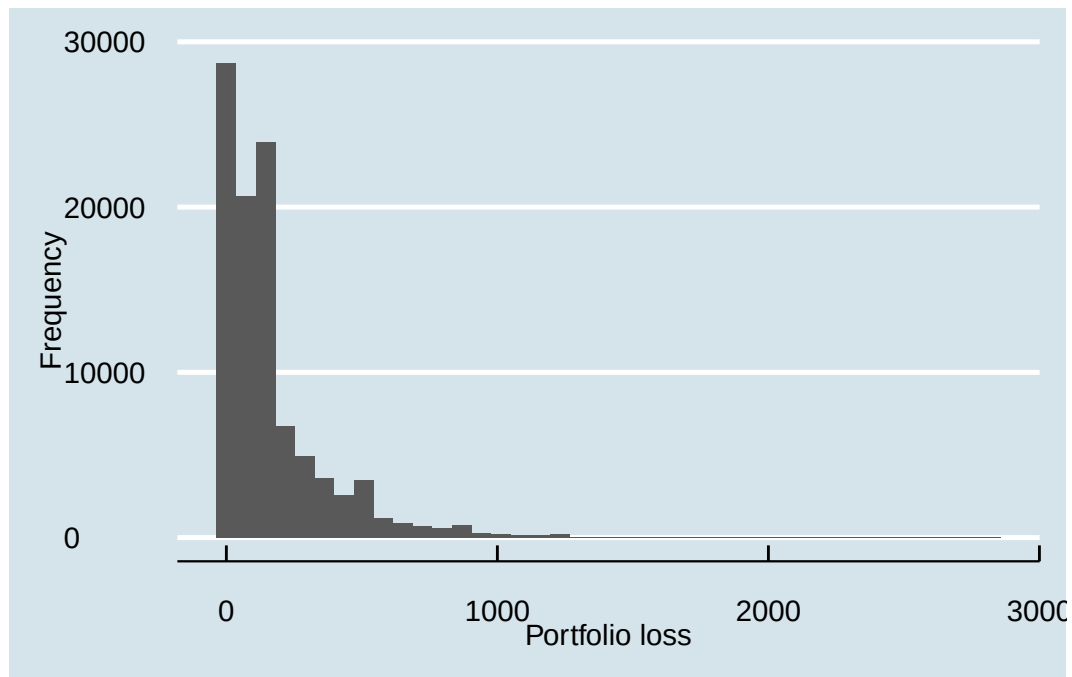


Figure7: One-Factor Gaussian モデルによる信用損失分布

信用損失分布は一般に左右対称ではない。通常時にはデフォルト件数が少ない一方、共通因子が大きく悪化すると多数企業が同時にデフォルトし、右側に長いテールを持つ損失分布が形成される。

I-10.9 信用 VaR

第 9 章と同様に、信用損失分布の分位点から信用 VaR を計算する。

$$\text{VaR}_{\alpha}^{\text{credit}} = Q_{\alpha}(L)$$

ここでは 95%、99%、99.9% の 3 つの信頼水準を比較する。

```
vec_confidence_levels <- c(
  0.95,
  0.99,
  0.999
)

tbl_credit_var <- tibble(
  confidence_level =
    vec_confidence_levels
) |>
mutate(
  Credit_VaR = purrr::map_dbl(
```

```

    confidence_level,
    \(confidence_level) {
      quantile(
        tbl_credit_losses$loss,
        probs = confidence_level,
        names = FALSE,
        type = 7
      )
    }
  )
)

show_table_LFL(tbl_credit_var)
#> # A tibble: 3 x 2
#>   con_level Cre_VaR
#>   <dbl>    <dbl>
#> 1   0.95      600
#> 2   0.99     1020
#> 3   0.999    1680

```

信用リスクでは市場リスクより長い保有期間と高い信頼水準が用いられることが多く、99.9%という極端な分位点を見ることで、共通因子が大きく悪化した場合のポートフォリオ損失を確認できる。

I-10.10 Expected Loss と Unexpected Loss

信用リスクでは Expected Loss (EL) と Unexpected Loss (UL) を区別することが重要である。

Expected Loss は平均的に予想される信用損失であり、

$$EL = E[L]$$

である。

一方、ある信頼水準における信用 VaR と EL との差を、テール側の予期せぬ損失を表す一つの尺度として

$$UL_{\alpha} = \text{VaR}_{\alpha}^{\text{credit}} - EL$$

と置くことができる。

```
tbl_credit_risk_measures <- tbl_credit_var |>
  mutate(
    expected_loss =
      theoretical_expected_loss,
    unexpected_loss =
      Credit_VaR -
      expected_loss
  )

show_table_LFL(
  tbl_credit_risk_measures
)
#> # A tibble: 3 x 4
#>   con_level Cre_VaR exp_loss une_loss
#>   <dbl>    <dbl>    <dbl>    <dbl>
#> 1     0.95      600      165      435
#> 2     0.99     1020      165      855
#> 3     0.999    1680      165     1515
```

ここでの $\text{VaR}_\alpha^{\text{credit}} - EL$ は本章で損失分布のテールを比較するための定義である。標準偏差として定義される Unexpected Loss とは区別する必要がある。

I-10.11 信用期待ショートフォール

市場リスクと同様に、信用損失についても信用 VaR を超えた領域を ES で評価できる。

```
tbl_credit_es <- tbl_credit_var |>
  mutate(
    Credit_ES = purrr::map2_dbl(
      confidence_level,
      Credit_VaR,
      \(confidence_level, credit_var) {
        tbl_credit_losses |>
          filter(loss >= credit_var) |>
          summarise(
            ES = mean(loss)
          ) |>
          pull(ES)
      }
    )
  )

show_table_LFL(tbl_credit_es)
```

```
#> # A tibble: 3 x 3
#>   con_level Cre_VaR  Cre_ES
#>   <dbl>    <dbl>    <dbl>
#> 1    0.95      600  830.48
#> 2    0.99     1020 1271.1
#> 3    0.999    1680 1935.6
```

信用損失分布では同じ損失額に確率質量が集中する離散性があるため、実証的な $\text{loss} \geq \text{Credit_VaR}$ によるテール平均は、連続分布における ES の定義と完全には一致しない場合がある。本章ではシミュレーション損失分布のテールの大きさを直感的に確認する尺度として用いる。

I-10.12 相関がテールリスクへ与える影響

One-Factor Gaussian モデルの重要なパラメータが ρ である。

個々の企業の PD、EAD、LGD を変えなくても、共通因子への依存度が高くなれば同時デフォルトが増え、損失分布のテールは厚くなる。

この関係を確認するため、

$$\rho = 0, 0.10, 0.20, 0.40$$

を比較する。

```
simulate_credit_loss_LFL <- function(
  rho_value,
  n_scenarios = 20000L
) {
  purrr::map_dbl(
    seq_len(n_scenarios),
    \(scenario_id) {
      systematic_factor <- rnorm(1)

      vec_idiosyncratic <- rnorm(
        n_obligors
      )

      vec_latent_variable <-
        sqrt(rho_value) *
          systematic_factor +
        sqrt(1 - rho_value) *
```

```
      vec_idiosyncratic

      vec_default <-
        vec_latent_variable <
          qnorm(vec_PD)

      sum(
        vec_EAD *
        vec_LGD *
        vec_default
      )
    }
  )
}
```

```
set_random_seed_LFL(20260918)

vec_rho_scenarios <- c(
  0,
  0.10,
  0.20,
  0.40
)

tbl_correlation_sensitivity <- tibble(
  rho = vec_rho_scenarios
) |>
mutate(
  losses = purrr::map(
    rho,
    \(rho_value) {
      simulate_credit_loss_LFL(
        rho_value = rho_value
      )
    }
  ),
  expected_loss = purrr::map_dbl(
    losses,
    mean
  ),
  VaR_99 = purrr::map_dbl(
    losses,
    \(losses) {
      quantile(
        losses,
```

```

      probs = 0.99,
      names = FALSE
    )
  }
),
VaR_999 = purrr::map_dbl(
  losses,
  \(losses) {
    quantile(
      losses,
      probs = 0.999,
      names = FALSE
    )
  }
)
)

show_table_LFL(
  tbl_correlation_sensitivity |>
  select(
    -losses
  )
)

#> # A tibble: 4 x 4
#>   rho exp_loss VaR_99 VaR_999
#>   <dbl>   <dbl> <dbl>   <dbl>
#> 1  0     164.18  420    540
#> 2  0.1   166.06  720   1080
#> 3  0.2   163.89 1020   1680
#> 4  0.4   167.13 1680  3060.1

```

```

plot_correlation_sensitivity <-
  tbl_correlation_sensitivity |>
  select(
    rho,
    VaR_99,
    VaR_999
  ) |>
  pivot_longer(
    cols = starts_with("VaR"),
    names_to = "confidence_level",
    values_to = "Credit_VaR"
  ) |>
  ggplot(
    aes(

```



```

    x = rho,
    y = Credit_VaR,
    linetype = confidence_level
  )
) +
geom_line() +
geom_point() +
labs(
  x = "Asset correlation",
  y = "Credit VaR",
  linetype = NULL
)

show_plot_LFL(
  plot_correlation_sensitivity
)

```

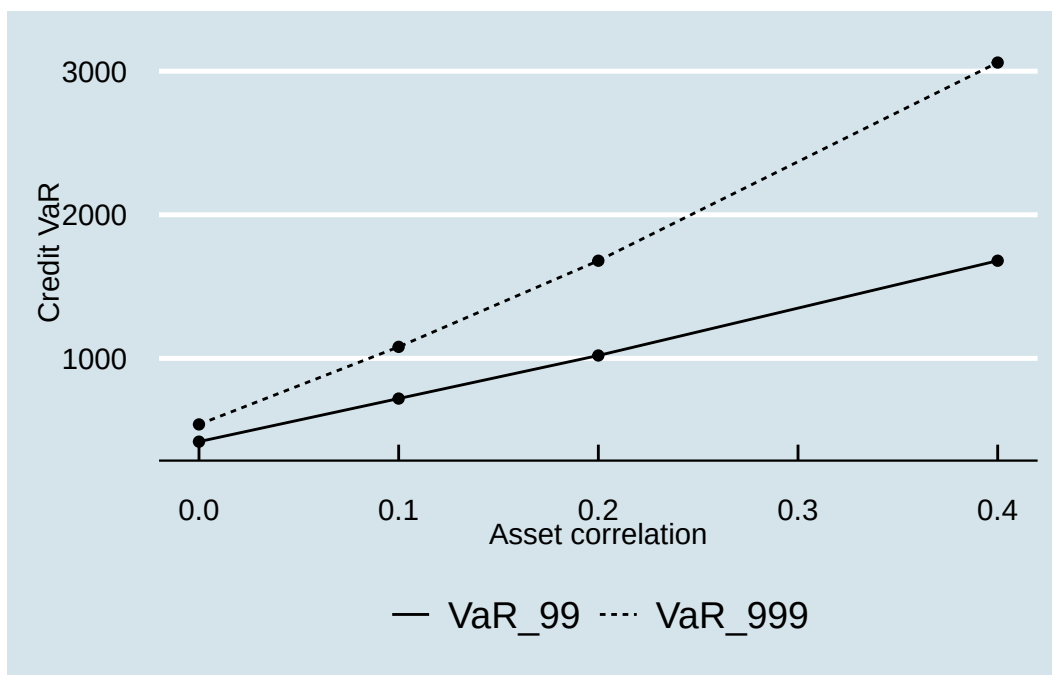


Figure8: 資産相関と信用 VaR

相関の上昇は必ずしも期待損失を大きく変化させない。一方で、同時デフォルトが発生しやすくなるため、高い信頼水準の信用 VaR は大きく上昇する。

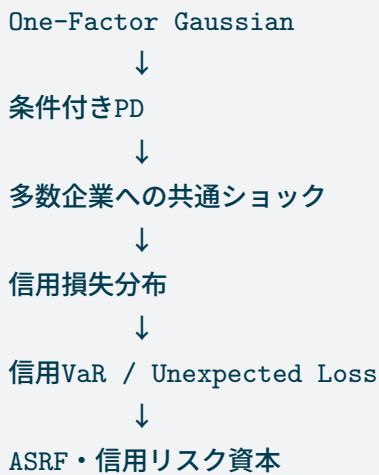
これは信用ポートフォリオにおいて、個別企業の PD だけでなくデフォルト依存が重要であることを示している。

I-10.13 Basel の ASRF モデルとの関係

One-Factor Gaussian モデルは単なるシミュレーション手法ではない。バーゼルの信用リスク内部格付手法（IRB）の理論的背景には、単一のシステムティック・ファクターによって債務者間の依存を表現する Asymptotic Single Risk Factor（ASRF）の考え方がある。

十分に細分化された大規模ポートフォリオでは、企業固有リスクが分散され、ポートフォリオ損失は主として共通因子によって決まる。この極限的な考え方により、信用ポートフォリオのテール損失を解析的な資本関数へ結びつけることができる。

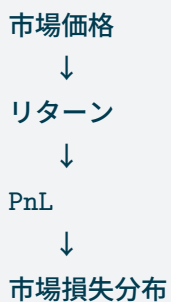
本章のモンテカルロ・シミュレーションは、この構造を有限の 100 社ポートフォリオとして直接観察している。



I-10.14 市場 VaR と信用 VaR

第 9 章と本章を比較すると、市場リスクと信用リスクでは損失を生み出すメカニズムが異なる。

市場リスクでは、



であった。

信用リスクでは、

共通因子 + 企業固有因子



デフォルト



EAD × LGD



信用損失分布

となる。

しかし最終的な計算構造は共通している。

リスク要因 → シナリオ → 損失 → 損失分布 → テールリスク

この統一的な見方が、異なるリスクを一つの計算基盤で扱うための出発点となる。

I-10.15 第 11 章への接続—CLO のトランシェ

本章では One-Factor Gaussian モデルからポートフォリオ全体の信用損失 L を生成した。

第 11 章では、この損失をポートフォリオ全体で負担するのではなく、Attachment Point A と Detachment Point D によって複数のトランシェへ分割する。

トランシェ損失は

$$L_{A,D} = \min \{ \max(L - A, 0), D - A \}$$

で表すことができる。

したがって、第 11 章で必要となる信用モデルを新しく作り直す必要はない。

第10章

One-Factor Gaussian



Portfolio Loss L



第11章

Attachment / Detachment



Tranche Loss



Premium Leg / Loss Leg



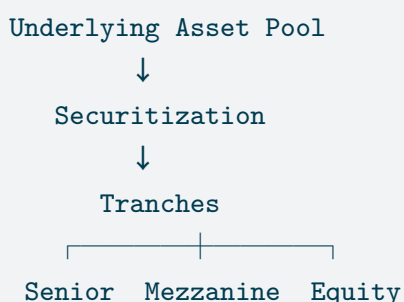
CLO Tranche Pricing

本章で構築した損失分布が、そのまま CLO のエクイティ、メザニン、シニア・トランシェの評価へつながる。

第 I-11 章 CLO と信用ポートフォリオのトランシェ評価

第 10 章では、One-Factor Gaussian モデルを用いて企業間のデフォルト依存を表現し、信用ポートフォリオの損失分布を構築した。本章では、そのポートフォリオ損失を複数の投資家へ異なる優先順位で配分する「トランチ (tranche)」という仕組みを考える。

トランチは単なる格付け区分ではない。証券化商品のキャッシュフローや損失を、異なる優先順位を持つ複数のクラスへ分割する仕組みである。



ただし、何を分割するかは商品によって異なる。CLO では主として信用損失と支払順位が重要となる。一方、住宅ローンを原資産とする MBS/CMO では、期限前償還によって生じる元本キャッシュフローのタイミングもトランチ構造によって再配分される。

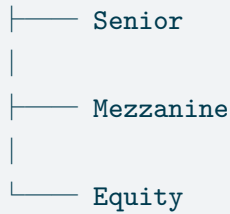
この違いを理解することが、CLO 評価を理解する第一歩である。

I-11.1 トランチとは何か

フランス語の *tranche* は「切り分けられた部分」を意味する。金融商品では、一つの資産プールから生じるキャッシュフローや損失を、優先順位の異なる複数の証券へ分割する。

単純化すると、





という構造になる。

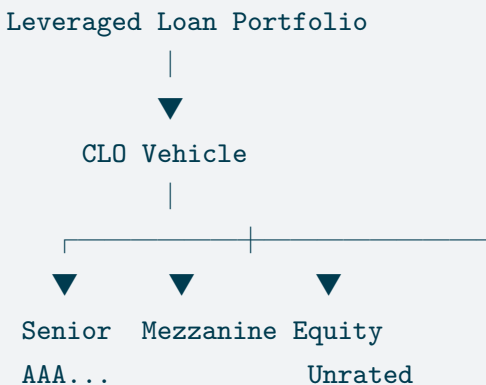
Senior は支払いの優先順位が高く、損失を負担する順位は遅い。Equity は支払い順位が最も低い一方、信用損失を最初に吸収する。Mezzanine はその中間に位置する。

したがって、同じ資産プールから発行された証券であっても、各トランチのリスクと期待収益は大きく異なる。

I-11.2 CLO の信用トランチ

Collateralized Loan Obligation (CLO) は、主として企業向けローンのポートフォリオを裏付けとして、複数の Debt Tranche と Equity Tranche を発行する証券化商品である。

典型的な構造を単純化すると、

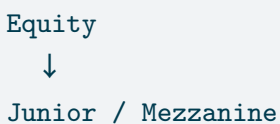


となる。

Debt Tranche には格付けが付与され、Senior から Junior へ信用リスクが段階的に高くなる。Equity Tranche は通常、最も劣後し、最初に損失を吸収する。

したがって、

損失の吸収順位



↓

Senior

となる。

一方、利息や元本などのキャッシュフローは原則として上位の Debt Tranche から優先的に支払われ、最後に残ったキャッシュフローが Equity へ配分される。この優先順位を Payment Waterfall という。

Collateral Cash Flow

↓

Fees / Expenses

↓

Senior Debt

↓

Mezzanine / Junior Debt

↓

Equity

CLO を理解するには「格付けの異なる債券が並んでいる」と考えるだけでは不十分である。格付けの差を生み出している経済的な構造は、**同じ信用ポートフォリオの損失をどの順位で負担するか**にある。

I-11.3 MBS では何をトランチングするのか

Mortgage-Backed Securities (MBS) も証券化商品であるが、住宅ローンには企業向けローンとは異なる重要な特徴がある。

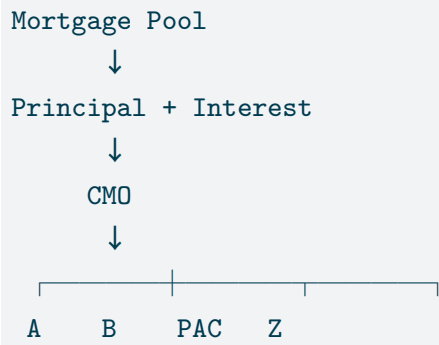
借り手が住宅ローンを期限前に返済できることである。

特に金利が低下すると、借り手はより低い金利の住宅ローンへ借り換えるインセンティブを持つ。その結果、MBS 投資家には予定より早く元本が返済される可能性がある。

これが Prepayment Risk（期限前償還リスク）である。

単純な Pass-Through MBS では、住宅ローン・プールから発生した元本と利息を投資家へ比例配分する。

より複雑な Collateralized Mortgage Obligation (CMO) では、この元本キャッシュフローを複数のトランチへ異なる順位で配分する。



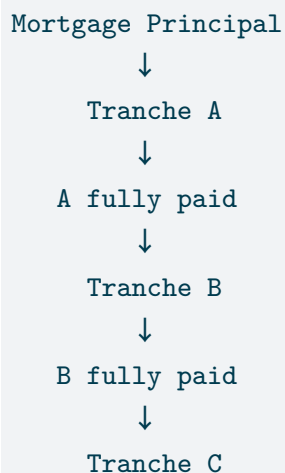
したがって、CLO の信用トランチと MBS/CMO のキャッシュフロー・トランチは、同じ「トランチ」という言葉を使っているにもかかわらず中心となるリスクが異なる。

項目	CLO	MBS / CMO
主な原資産	企業向けローン	住宅ローン
主要なリスク	デフォルト・回収率・相関	金利・期限前償還
トランチングの中心	信用損失と支払順位	元本・利息の配分時期
代表的な構造	Senior / Mezzanine / Equity	Sequential / PAC / Support / Z
投資家が注目するもの	Loss Waterfall	Prepayment / Cash-flow Waterfall

I-11.4 MBS の Sequential Pay

CMO の最も基本的な考え方の一つが Sequential Pay である。

複数のトランチ A、B、C がある場合、住宅ローンから返済された元本をまず A へ集中して配分する。A が全額償還された後に B へ、その後 C へ配分する。



これにより、同じ住宅ローン・プールから、平均残存期間や期限前償還感応度の異なる証券を作ることができる。

I-11.5 PAC と Support Tranche

Planned Amortization Class (PAC) は、一定の期限前償還速度の範囲内で、比較的安定した元本償還スケジュールを目指すトランチである。

その安定性を支えるのが Support Tranche である。

実際の期限前償還が想定より速い、あるいは遅い場合、その変動を Support Tranche が優先的に吸収する。

Prepayment Variability



Support Tranche



PAC Cash Flowを安定化

つまりトランチングとは、信用損失だけを分割する技術ではない。キャッシュフローのタイミングに関するリスクそのものを投資家間で再配分する技術でもある。

I-11.6 Z-Tranche—早期に元本償還されないトランチ

CMO には、さらに特徴的な Z-Tranche (Accrual Tranche) が存在する。

Z-Tranche は初期期間に現金利息を受け取らず、その利息相当額を元本へ加算する。上位トランチが償還されている間、Z-Tranche への元本支払いは後順位となる。

単純化すると、

初期期間

Mortgage Cash Flow



A → B → C の償還



Z-Tranche

現金利息を受け取らず

利息を元本へ加算

その後、

A / B / C 償還後
↓
Z-Tranche
利息・元本の支払い

へ移る。

したがって、Z-Tranche は「期限前償還が存在しない」という意味ではない。原資産の住宅ローンでは期限前償還が発生しているが、ウォーターフォールによってその元本キャッシュフローが先に他のトランチへ配分されるため、Z-Tranche は初期には直接の元本償還を受けにくい構造になっている。

この区別は重要である。

原資産で期限前償還が起きることと、あるトランチがその期限前償還による元本を直ちに受け取ることは同じではない。

トランチングによって、原資産に存在するリスクを消すことはできない。しかし、そのリスクをどの投資家が、どの時点で、どの程度負担するかを組み替えることはできる。

I-11.7 CLO と MBS に共通する考え方

ここまでを見ると、CLO と MBS/CMO は異なる商品に見える。

しかし数学的な発想には共通点がある。

原資産プール
↓
リスクのあるキャッシュフロー
↓
優先順位を設定
↓
複数のトランチへ配分

CLO では主として、

Portfolio Credit Loss → Tranche Loss

を考える。

MBS/CMO では主として、

Mortgage Cash Flow → Tranche Cash Flow

を考える。

つまり、証券化の本質は原資産のリスクを消去することではなく、異なる性質を持つ証券へ再配分することにある。

I-11.8 Attachment Point と Detachment Point

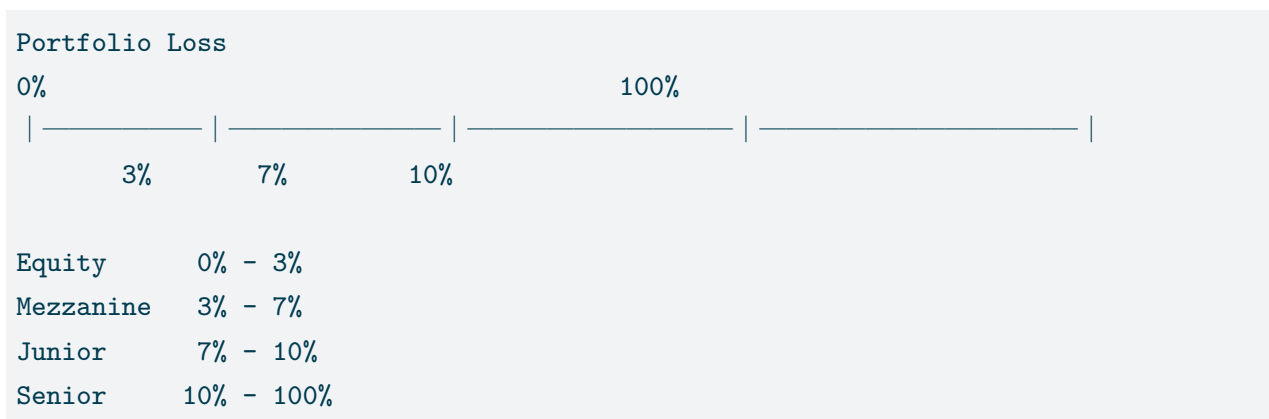
ここから CLO の信用ランチを数理的に表現する。

ポートフォリオ損失率を L とする。

あるランチがポートフォリオ損失率 A から損失を負担し始め、 D で全額毀損するとする。

A を Attachment Point、 D を Detachment Point という。

たとえば、



とする。

3%-7% ランチでは、ポートフォリオ損失が 3% に達するまでは損失を負担しない。

損失が 3% を超えるとランチ損失が発生し、7% に達するとランチ元本はすべて失われる。

I-11.9 トランチ損失

Attachment Point A 、Detachment Point D のランチ損失は、

$$L_{A,D} = \min \{ \max(L - A, 0), D - A \}$$

で表される。

トランチ元本 $D - A$ に対する損失率として表すなら、

$$TL_{A,D} = \frac{\min\{\max(L - A, 0), D - A\}}{D - A}$$

となる。

```
tranche_loss_LFL <- function(
  portfolio_loss,
  attachment,
  detachment
) {
  pmin(
    pmax(
      portfolio_loss - attachment,
      0
    ),
    detachment - attachment
  )
}
```

単純な損失率を使ってトランチ構造を確認する。

```
tbl_tranche_example <- tibble(
  portfolio_loss = seq(
    0,
    0.15,
    by = 0.001
  )
) |>
mutate(
  Equity = tranche_loss_LFL(
    portfolio_loss,
    0.00,
    0.03
  ),
  Mezzanine = tranche_loss_LFL(
    portfolio_loss,
    0.03,
    0.07
  ),
  Junior = tranche_loss_LFL(
    portfolio_loss,
```

```

    0.07,
    0.10
  ),
  Senior = tranche_loss_LFL(
    portfolio_loss,
    0.10,
    1.00
  )
)

show_table_LFL(
  tbl_tranche_example
)
#> # A tibble: 151 x 5
#>   por_loss Equity Mezzanine Junior Senior
#>   <dbl>   <dbl>     <dbl>   <dbl>   <dbl>
#> 1     0     0         0     0     0
#> 2  0.001  0.001         0     0     0
#> 3  0.002  0.002         0     0     0
#> 4  0.003  0.003         0     0     0
#> 5  0.004  0.004         0     0     0
#> 6  0.005  0.005         0     0     0
#> 7  0.006  0.006         0     0     0
#> 8  0.007  0.007         0     0     0
#> 9  0.008  0.008         0     0     0
#> 10 0.009  0.009         0     0     0
#> # i 141 more rows

```

```

plot_tranche_loss <- tbl_tranche_example |>
  pivot_longer(
    cols = -portfolio_loss,
    names_to = "tranche",
    values_to = "tranche_loss"
  ) |>
  ggplot(
    aes(
      x = portfolio_loss,
      y = tranche_loss,
      linetype = tranche
    )
  ) +
  geom_line() +
  labs(
    x = "Portfolio Loss",
    y = "Tranche Loss",

```

```
linetype = NULL
)

show_plot_LFL(plot_tranche_loss)
```

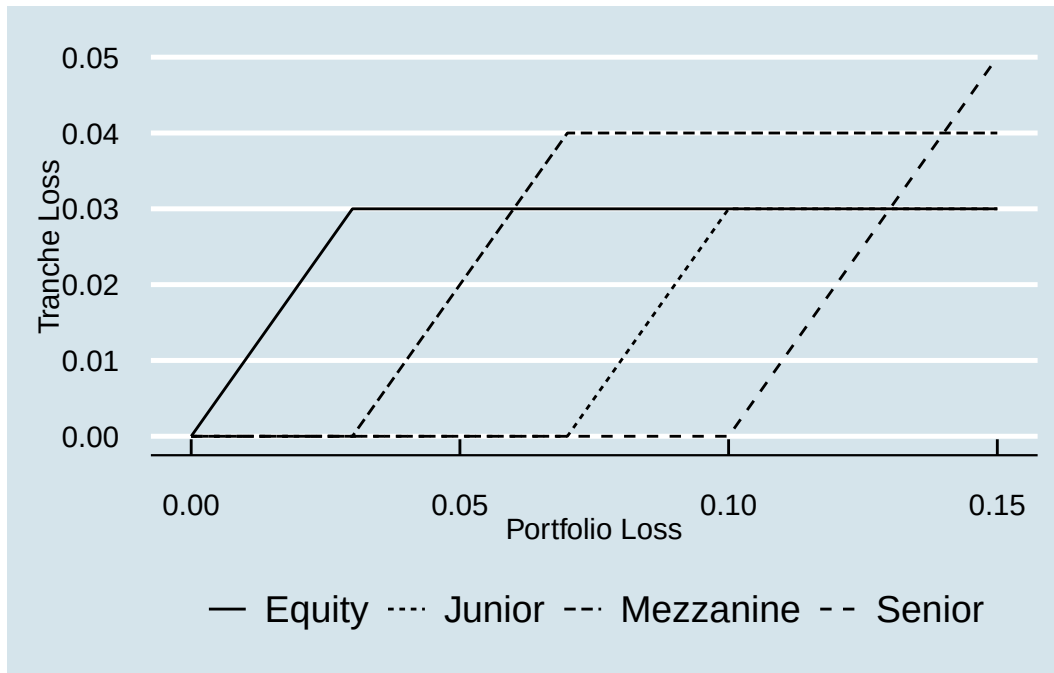


Figure9: ポートフォリオ損失と各トランチの損失

この図から、同じポートフォリオ損失でもトランチによって損失発生タイミングが大きく異なることが分かる。

I-11.10 第 10 章の信用損失分布をトランチへ配分する

第 10 章では One-Factor Gaussian モデルによって

$$L = \sum_i EAD_i LGD_i \mathbf{1}_{\{Default_i\}}$$

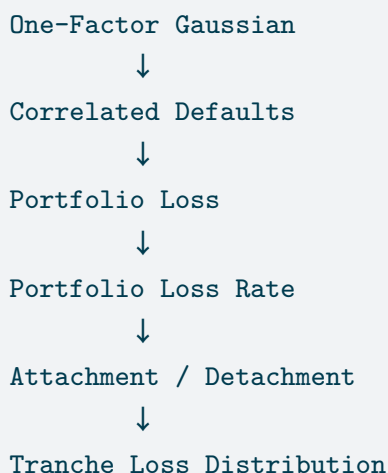
をシミュレーションした。

総 EAD を EAD_{total} とすれば、

$$l = \frac{L}{EAD_{\text{total}}}$$

によってポートフォリオ損失率へ変換できる。

この l を Attachment / Detachment 構造へ通せば、各トランチの損失分布が得られる。



つまり、第 10 章の信用 VaR モデルと第 11 章の CLO モデルは別々のモデルではない。

同じ信用損失エンジンの出力を、異なる優先順位へ配分しているだけである。

I-11.11 Expected Tranche Loss

シナリオ s におけるトランチ損失を

$$L_{A,D}^{(s)}$$

とすれば、Expected Tranche Loss は

$$ETL_{A,D} = E[L_{A,D}]$$

である。

Equity は小さなポートフォリオ損失から損失を受けるため ETL が大きい。

Senior は Attachment Point が高いため、通常の信用環境では損失を受けない。しかし極端な同時デフォルトが発生すると損失が生じる。

したがって Senior Tranche の評価では、第 10 章で確認したデフォルト相関と損失分布のテールが特に重要となる。

I-11.12 Premium Leg と Loss Leg

信用トランチの価格評価では、大きく二つのキャッシュフローを考える。

Protection Buyer が支払うプレミアムを Premium Leg、

信用損失発生時に Protection Seller が負担する損失を Loss Leg とする。

概念的には、

$$PV_{\text{Premium}} = s \sum_t DF(t) E[\text{OutstandingTranche}_t] \Delta t$$

であり、

$$PV_{\text{Loss}} = \sum_t DF(t) E[\Delta L_{A,D}(t)]$$

となる。

Fair Spread s^* は、

$$PV_{\text{Premium}} = PV_{\text{Loss}}$$

を満たすスプレッドである。

したがって、

$$s^* = \frac{PV_{\text{Loss}}}{\sum_t DF(t) E[\text{OutstandingTranche}_t] \Delta t}$$

となる。

ここで初めて、信用損失分布が「価格」へ変換される。

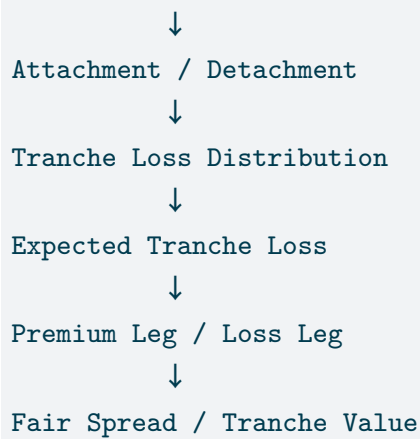
I-11.13 CLO 評価の計算構造

第 10 章と本章をまとめると、CLO トランチ評価の計算構造は次のようになる。

```

PD / EAD / LGD / rho
    ↓
One-Factor Gaussian
    ↓
Conditional Default
    ↓
Portfolio Loss Distribution

```

この構造は、信用ポートフォリオのリスク計測と証券化商品の価格評価が、同じ損失分布を基礎としていることを示している。

第 10 章では損失分布の分位点を取ることで信用 VaR を求めた。

本章では同じ損失分布を Attachment / Detachment によって切り分け、各トランチのキャッシュフローへ変換する。

$$\text{同じ Loss Distribution} \begin{cases} \rightarrow \text{信用 VaR} \\ \rightarrow \text{CLO Tranche Pricing} \end{cases}$$

これが、第 I 部におけるポートフォリオ・リスクモデルから証券化商品評価までの接続である。

第Ⅱ部 金融工学 無裁定モデル

第 II-1 章債券評価と金利リスクの基礎

金融では、異なる時点に発生する金額をそのまま比較することはできない。今日受け取る 100 円と 1 年後に受け取る 100 円では、運用可能期間や金利が異なるため、経済的な価値も異なるからである。そこで、時点の異なるキャッシュフローを評価する場合には、将来の金額を現在価値へ割り引くか、現在の金額を将来価値へ換算し、同一時点の価値にそろえる必要がある。金融商品の価格評価では、通常、将来のすべてのキャッシュフローを評価時点の現在価値へ換算する。

金融商品の評価では、理論的な価格式を理解するだけでなく、市場で定められたマーケットコンベンションを正確に理解し、それを計算システムとして実装することが重要である。特に債券は、利払日、日数計算規則、経過利子、受渡日、価格表示、利回り表示、端数処理などが市場慣行として規格化されている。これらを正確に反映しなければ、理論的に正しい価格式を用いても実際の市場で用いられる単価や利回りを再現できない。

日本の債券市場では、長年にわたり価格そのものだけでなく利回りを中心として取引条件を表現する慣行があり、とりわけ単利利回りが広く用いられてきた。単利利回りは、将来の各キャッシュフローを支払時点ごとに割り引く複利評価と比べると簡易な指標であるが、市場慣行として重要な役割を果たしている。このため、本章では市場で用いられる単利利回りから債券単価を求め、その単価を基準として複利利回りを逆算する流れを出発点とする。

金融市場では、すべての満期についてゼロクーポン金利が直接観測されるわけではない。TONAR OIS 市場で主として観測されるのは、各満期において固定レグと変動レグの現在価値を等しくするパーレートである。金融商品の評価では、このパーレートからディスカウントファクターを導出し、スポットレートおよびフォワードレートからなる金利期間構造へ変換する必要がある。

そこで本章では、まず パーレート → ディスカウントファクター → スポットレート → フォワードレートという導出関係を確認する。さらに、順イールド、逆イールドおよびフラットカーブについて、三つの金利の大小関係を直観と数式の両面から理解する。その後、同一の 40 年債を一貫して用い、単利利回り → 債券単価 → 複利利回り → フラットイールド評価 → 連続複利による再評価 → Z スプレッド → アセットスワップ → 金利感応度へと評価を展開する。最

後に、クリーンプライスから複利利回りを逆算する問題を求根問題として捉え、複数の数値解法を比較する。

II-1.1 イールドカーブの基本

イールドカーブは、横軸に満期、縦軸に金利をとり、金利の期間構造を表したものである。金利は市場全体に一つだけ存在する数字ではなく、1 年、5 年、10 年、30 年といった期間ごとに異なる。長期金利が短期金利より高い右上がりの形状を順イールド、低い右下がりの形状を逆イールド、満期による差が小さい形状をフラットカーブという。

同じ市場の金利期間構造にも、パーレート、スポットレート、フォワードレートという複数の表現がある。これらは別々の金利ではなく、同一のイールドカーブを異なる角度から見たものである。

パーレートは、債券であれば価格を額面と等しくし、スワップであれば契約時の価値をゼロにする固定金利である。市場で観測されるクーポン付き商品の金利を表す。

スポットレートは、評価時点から特定の将来時点までのゼロクーポン金利である。満期まで途中の利払いがない一つのキャッシュフローを割り引くための金利であり、金融商品の各キャッシュフローを個別に評価する基礎となる。

フォワードレートは、将来の特定期間に対応する金利である。スポットレートが評価時点から満期までの平均的な金利を表すのに対し、フォワードレートは将来の局所的な金利を表す。

満期 T_n の TONAR OIS パーレートを S_n 、固定レッグの各計算期間を α_i 、ディスカウントファクターを $DF(T_i)$ とする。固定レッグの現在価値は、想定元本を 1 とすれば、

$$PV_{\text{fixed}} = S_n \sum_{i=1}^n \alpha_i DF(T_i)$$

と表される。一方、標準的な OIS の変動レッグの現在価値は、概念的に

$$PV_{\text{float}} = 1 - DF(T_n)$$

である。パーレートでは両者が等しいため、

$$S_n = \frac{1 - DF(T_n)}{\sum_{i=1}^n \alpha_i DF(T_i)}$$

となる。TONAR OIS の市場クォートは、この固定レグと変動レグを釣り合わせるパーレートである。

上式を満期 T_n のディスカウントファクターについて解くと、

$$DF(T_n) = \frac{1 - S_n \sum_{i=1}^{n-1} \alpha_i DF(T_i)}{1 + S_n \alpha_n}$$

となる。短い満期で求めたディスカウントファクターを使って、次の満期の値を順番に求めていく手続きがブートストラップである。本章では導出の意味を理解することに重点を置き、実際のマーケットコンベンションと補間を含むカーブ構築は第 II-2 章で扱う。

ディスカウントファクターは、将来の 1 単位の金額を評価時点の価値へ変換する倍率である。時点 t_i のキャッシュフローを CF_i とすれば、金融商品の現在価値は、

$$PV = \sum_{i=1}^n CF_i DF(t_i)$$

となる。連続複利のスポットレートを $z(T)$ とすると、

$$DF(T) = \exp\{-z(T)T\}$$

したがって、

$$z(T) = -\frac{\log DF(T)}{T}$$

である。スポットレートは、評価時点から満期までの期間全体を表す平均的な金利と捉えることができる。

インスタント・フォワードレートを $f(T)$ とすると、

$$DF(T) = \exp\left\{-\int_0^T f(u) du\right\}$$

である。したがって、スポットレートは、現在から満期までのフォワードレートの平均として、

$$z(T) = \frac{1}{T} \int_0^T f(u) du$$

と表される。また、

$$f(T) = -\frac{\partial \log DF(T)}{\partial T} = z(T) + T \frac{\partial z(T)}{\partial T}$$

である。この微分関係により、イールドカーブの傾きとスポットレート、フォワードレートの大小関係が結びつく。

II-1.2 イールドカーブの形状と三つの金利

スポットレートカーブが右上がりなら、 $z'(T) > 0$ である。したがって、

$$f(T) = z(T) + Tz'(T) > z(T)$$

となる。スポットレートが現在から満期までの平均勾配、フォワードレートが満期付近の局所的な勾配だと考えれば、上り坂が次第に急になるとき、局所的な勾配がそれまでの平均を上回ることは直観的にも明らかである。

パーレートとスポットレートの関係は、途中で利払いのある債券と、満期まで何も支払わないゼロクーポン債を比べると理解しやすい。額面を 1、満期を 2 年、年 1 回払いのクーポンを c とすれば、パー債は、

$$1 = cDF(1) + (1 + c)DF(2)$$

を満たすため、

$$c = \frac{1 - DF(2)}{DF(1) + DF(2)}$$

となる。順イールドでは、パー債は相対的に低い短期金利で評価される 1 年後のクーポンを先に回収できる。この途中キャッシュフローの現在価値があるため、価格を額面 1 にするクーポン率は、すべての収益を 2 年後に受け取るゼロクーポン債の 2 年スポットレートより低くてよい。

したがって、滑らかで単調な順イールドでは、典型的に、

$$\text{パーレート} < \text{スポットレート} < \text{フォワードレート}$$

となる。

スポットレートカーブが右下がりなら、 $z'(T) < 0$ であり、

$$f(T) = z(T) + Tz'(T) < z(T)$$

となる。スポットレートには、それ以前の高い短期金利が平均として残るため、満期付近のフォワードレートはスポットレートより低い。

また、パー債が途中で受け取るクーポンには短期側の高い金利が反映される。このため、途中利払いのないゼロクーポン債のスポットレートより、価格を額面にするパーレートの方が高くなる。したがって、滑らかで単調な逆イールドでは、典型的に、

$$\text{パーレート} > \text{スポットレート} > \text{フォワードレート}$$

となる。

スポットレートが満期によらず一定なら、 $z'(T) = 0$ なので、 $f(T) = z(T)$ となる。複利方式と支払頻度をそろえた理想的な条件では、

$$\text{パーレート} = \text{スポットレート} = \text{フォワードレート}$$

となる。

以下では、TONAR OIS 市場で観測されるパーレートと、そこから構築したスポットレートおよびインスタント・フォワードレートを一枚のグラフに重ねる。

```
tonar_benchmark <- GaussQuant::tonar_curve_benchmark_GQL()

tbl_tonar_curve <- tonar_benchmark$validation$quote_repricing |>
  dplyr::select(
    pillar_date,
    market_quote,
    discount_factor,
    zero_rate,
    inst_fwd_rate
  ) |>
  dplyr::filter(!is.na(pillar_date)) |>
```

```
dplyr::distinct(pillar_date, .keep_all = TRUE) |>
dplyr::arrange(pillar_date)

tbl_tonar_rates_long <- tbl_tonar_curve |>
  dplyr::select(
    pillar_date,
    market_quote,
    zero_rate,
    inst_fwd_rate
  ) |>
  tidyr::pivot_longer(
    cols = c(
      market_quote,
      zero_rate,
      inst_fwd_rate
    ),
    names_to = "curve_type",
    values_to = "rate"
  ) |>
  dplyr::mutate(
    curve_type = dplyr::recode(
      curve_type,
      market_quote = "パーレート",
      zero_rate = "スポットレート",
      inst_fwd_rate = "フォワードレート"
    )
  ) |>
  dplyr::filter(!is.na(rate))

yield_curve_plot <- tbl_tonar_rates_long |>
  ggplot2::ggplot(
    ggplot2::aes(
      x = pillar_date,
      y = 100 * rate,
      color = curve_type,
      linetype = curve_type
    )
  ) +
  ggplot2::geom_line(linewidth = 0.8) +
  ggplot2::geom_point(size = 1.8) +
  ggplot2::scale_x_date(
    date_breaks = "5 years",
    date_labels = "%Y"
  ) +
  ggplot2::scale_color_manual(
```

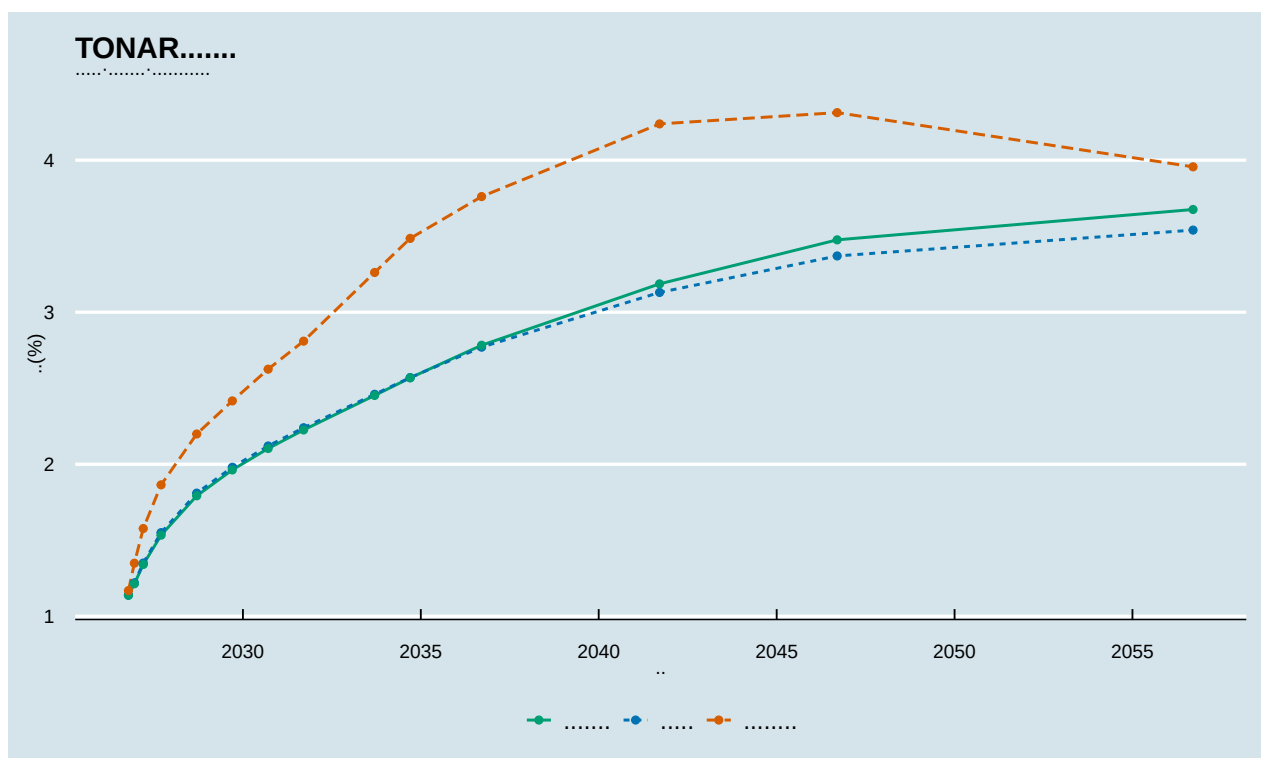


```

values = c(
  "パーレート" = "#0072B2",
  "スポットレート" = "#009E73",
  "フォワードレート" = "#D55E00"
)
) +
ggplot2::labs(
  title = "TONARイールドカーブ",
  subtitle = "パーレート・スポットレート・フォワードレートの比較",
  x = "満期",
  y = "金利 (%)",
  color = NULL,
  linetype = NULL
) +
ggplot2::theme(
  legend.position = "bottom"
)

show_plot_LFL(yield_curve_plot)

```



パーレートは複数の期間金利を反映するため比較的滑らかであり、スポットレートは評価時点から各満期までの平均的な金利を表す。フォワードレートは満期付近の局所的な金利とカーブの傾きを反映するため、三曲線の中で最も変動しやすい。

次の数値例では、年1回複利のスポットレートを滑らかに上昇または低下させ、対応するディスカウントファクター、パーレートおよび1年フォワードレートを計算する。

```

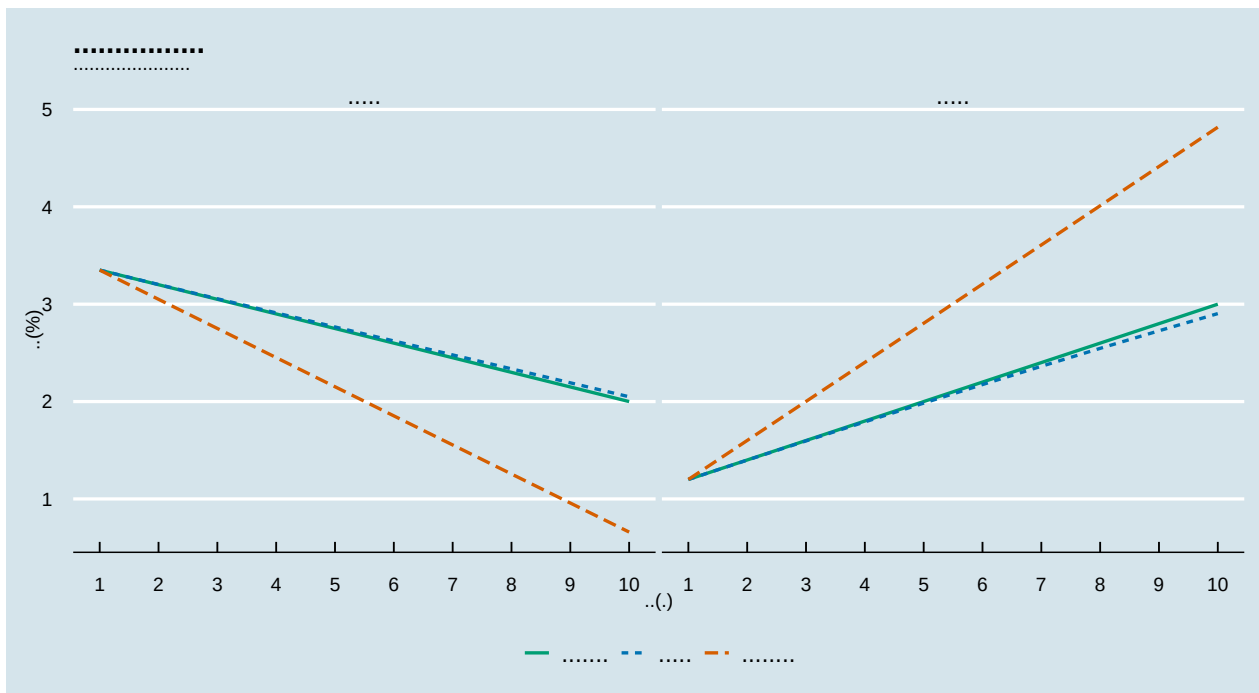
tbl_curve_shapes <- tidyr::crossing(
  curve_shape = c("順イールド", "逆イールド"),
  maturity = 1:10
) |>
dplyr::group_by(curve_shape) |>
dplyr::mutate(
  spot_rate = dplyr::if_else(
    curve_shape == "順イールド",
    0.01 + 0.002 * maturity,
    0.035 - 0.0015 * maturity
  ),
  discount_factor = (1 + spot_rate)^(-maturity),
  previous_discount = dplyr::lag(
    discount_factor,
    default = 1
  ),
  forward_rate = (
    previous_discount /
    discount_factor
  ) - 1,
  par_rate = (
    1 - discount_factor
  ) / cumsum(discount_factor)
) |>
dplyr::ungroup() |>
dplyr::select(
  curve_shape,
  maturity,
  par_rate,
  spot_rate,
  forward_rate
) |>
tidyr::pivot_longer(
  cols = c(
    par_rate,
    spot_rate,
    forward_rate
  ),
  names_to = "curve_type",
  values_to = "rate"
) |>
dplyr::mutate(
  curve_type = dplyr::recode(
    curve_type,
    par_rate = "パーレート",

```

```
    spot_rate = "スポットレート",
    forward_rate = "フォワードレート"
  )
)

curve_shape_plot <- tbl_curve_shapes |>
  ggplot2::ggplot(
    ggplot2::aes(
      x = maturity,
      y = 100 * rate,
      color = curve_type,
      linetype = curve_type
    )
  ) +
  ggplot2::geom_line(linewidth = 0.9) +
  ggplot2::facet_wrap(~curve_shape) +
  ggplot2::scale_color_manual(
    values = c(
      "パーレート" = "#0072B2",
      "スポットレート" = "#009E73",
      "フォワードレート" = "#D55E00"
    )
  ) +
  ggplot2::scale_x_continuous(breaks = 1:10) +
  ggplot2::labs(
    title = "イールドカーブの形状と三つの金利",
    subtitle = "滑らかで単調な順イールドと逆イールドの模式例",
    x = "満期 (年)",
    y = "金利 (%)",
    color = NULL,
    linetype = NULL
  ) +
  ggplot2::theme(
    legend.position = "bottom"
  )

show_plot_LFL(curve_shape_plot)
```



この大小関係は、複利方式と支払頻度をそろえ、金利期間構造が滑らかかつ単調に上昇または下降する典型的な状況を想定している。複雑なカーブ形状や異なる市場慣行のもとですべての満期について機械的に成立する恒等式ではない。しかし、パーレートは途中キャッシュフローを含む金利、スポットレートは満期までの平均的な金利、フォワードレートは満期付近の局所的な金利であるという違いを理解すれば、三つのカーブの位置関係を直観的に読むことができる。

II-1.3 債券とキャッシュフロー

固定利付債は、あらかじめ定められた利払日にクーポンを支払い、満期日に最終クーポンと元本を償還する。したがって、債券評価の出発点は将来キャッシュフローの日付と金額を正確に確定することである。

本章では、額面 100、年率 7.50%、半年払い、受渡日を 2022 年 4 月 1 日、償還日を 2062 年 4 月 1 日とする残存 40 年の固定利付債を一貫して用いて計算を確認する。

```
jp_calendar <- QuantLib::Japan()

trade_date_chr <- "2022-03-30"
issue_date_chr <- "2022-04-01"
maturity_date_chr <- "2062-04-01"

face_amount <- 100
coupon_rate <- 0.07500
settlement_days <- 2L

dc_30 <- GaussQuant::day_counter_GQL(
```

```
"Thirty360",
convention = "BondBasis"
)

cmpd_compounded <- QuantLib::Compounding_Compounded_get()
cmpd_simple <- QuantLib::Compounding_Simple_get()
freq_semiannual <- QuantLib::Frequency_Semiannual_get()

trade_date <- GaussQuant::date_GQL(trade_date_chr)
GaussQuant::set_eval_date_GQL(trade_date)

settlement_date <- GaussQuant::advance_days_GQL(
  calendar_obj = jp_calendar,
  date_obj = trade_date,
  n_days = settlement_days
)

settlement_date_chr <- as.character(settlement_date)

bond_obj <- GaussQuant::fixed_rate_bond_GQL(
  issue_date = issue_date_chr,
  maturity_date = maturity_date_chr,
  face_amount = face_amount,
  coupon_rate = coupon_rate,
  settlement_days = settlement_days,
  calendar = jp_calendar,
  accrual_day_counter = dc_30,
  payment_convention = "Unadjusted",
  schedule_convention = "Unadjusted",
  maturity_convention = "Unadjusted",
  date_generation = "Backward",
  end_of_month = FALSE,
  schedule_frequency = GaussQuant::period_months_GQL(6)
)

tbl_bond_setup <- tibble::tribble(
  ~item, ~value,
  "trade_date", trade_date_chr,
  "settlement_date", settlement_date_chr,
  "issue_date", issue_date_chr,
  "maturity_date", maturity_date_chr,
  "face_amount", as.character(face_amount),
  "coupon_rate_pct", sprintf("%.4f", 100 * coupon_rate),
  "frequency", "Semiannual"
)
```

```
GaussQuant::show_tbl_GQH(
  tbl_bond_setup,
  "固定利付債の計算条件"
)
#> # A tibble: 7 x 2
#>   item      value
#>   <chr>    <chr>
#> 1 trade_date 2022-03-30
#> 2 settlement_date 2022-04-01
#> 3 issue_date 2022-04-01
#> 4 maturity_date 2062-04-01
#> 5 face_amount 100
#> 6 coupon_rate_pct 7.5000
#> 7 frequency Semiannual
```

```
tbl_bond_cashflow <- GaussQuant::bond_cashflow_table_GQL(
  bond = bond_obj
)

GaussQuant::show_tbl_GQH(
  tbl_bond_cashflow,
  "債券キャッシュフロー",
  n = 30
)
#> # A tibble: 30 x 2
#>   date      amount
#>   <chr>    <dbl>
#> 1 2022-10-01 3.7500
#> 2 2023-04-01 3.7500
#> 3 2023-10-01 3.7500
#> 4 2024-04-01 3.7500
#> 5 2024-10-01 3.7500
#> 6 2025-04-01 3.7500
#> 7 2025-10-01 3.7500
#> 8 2026-04-01 3.7500
#> 9 2026-10-01 3.7500
#> 10 2027-04-01 3.7500
#> # i 20 more rows
```

各キャッシュフローを正確な支払日に配置することが、価格、利回り、デュレーションおよびスプレッド計算の共通基盤となる。

債券計算では、日数計算規則、休日カレンダー、営業日調整、受渡日、利払頻度、価格表示、利

回り表示および端数処理を一体として扱う必要がある。計算式だけが一致していても、マーケットコンベンションが異なれば市場で用いられる値とは一致しない。

代表的な日数計算規則には Actual/365 Fixed、Actual/360、Actual/Actual ICMA、Actual/Actual ISDA、30/360 Bond Basis などがある。債券のクーポン計算と金利カーブの時間軸では、同じ日数計算規則を用いるとは限らない。この区別は、後続章で金利カーブを構築するときにも重要となる。

II-1.4 債券単価と市場利回り

日本の公社債市場では単利利回りが広く用いられてきた。額面 100 円、年間クーポン額を C 、償還価格を R 、単価を P 、残存年数を T とすると、単利の最終利回り Y は

$$Y = \frac{C + \frac{R - P}{T}}{P} \times 100$$

で表される。

逆に、単利利回り $y = Y/100$ から単価を求めると、

$$P = \frac{C + \frac{R}{T}}{y + \frac{1}{T}}$$

となる。実務では残存日数、残存年数および単価について所定の端数処理を適用するため、計算システムでは式だけでなく丸め・切り捨て規則まで実装する必要がある。以下では、受渡日を 2022 年 4 月 1 日に統一し、単利利回り 6.515% から日本市場の計算規則に従って求めた単価を、本章以降のクリーンプライスとして一貫して使用する。

```
yen_coupon_rate_pct <- 7.50
yen_simple_rate_pct <- 6.515
yen_redemption <- 100

yen_remaining_years <- GaussQuant::bond_year_fraction_YEN_GQL(
  start_date = settlement_date_chr,
  end_date = maturity_date_chr
)

yen_simple_price <- GaussQuant::bond_price_from_simple_rate_YEN_GQL(
  start_date = settlement_date_chr,
  end_date = maturity_date_chr,
```

```

    coupon_rate = yen_coupon_rate_pct,
    simple_rate = yen_simple_rate_pct,
    redemption = yen_redemption
  )

yen_simple_rate_recalculated <- GaussQuant::bond_simple_rate_from_price_YEN_GQL(
  start_date = settlement_date_chr,
  end_date = maturity_date_chr,
  coupon_rate = yen_coupon_rate_pct,
  price = yen_simple_price,
  redemption = yen_redemption
)

tbl_yen_simple <- tibble::tribble(
  ~metric, ~value,
  "remaining_years", yen_remaining_years,
  "simple_rate_pct", yen_simple_rate_pct,
  "clean_price_from_simple_rate", yen_simple_price,
  "simple_rate_from_price_pct", yen_simple_rate_recalculated
)

GaussQuant::show_tbl_GQH(
  tbl_yen_simple,
  "単利利回りから求めたクリーンプライス"
)

#> # A tibble: 4 x 2
#>   metric                                value
#>   <chr>                                <dbl>
#> 1 remaining_years                      40
#> 2 simple_rate_pct                      6.515
#> 3 clean_price_from_simple_rate 110.93
#> 4 simple_rate_from_price_pct        6.5150

```

債券市場で表示される単価は通常クリーンプライスである。受渡時には、直前利払日から受渡日までに発生した経過利子を加える。

$$\text{Dirty Price} = \text{Clean Price} + \text{Accrued Interest}$$

クリーンプライスは市場での価格比較に適している一方、実際の受渡金額および将来キャッシュフローとの現在価値比較にはダーティプライスを用いる。したがって、利回りやスプレッドを逆算するときには、どの価格を入力としているかを明確にする必要がある。

II-1.5 利回りによる債券評価

単利利回りは、クーポン収入と償還差損益を残存期間で年率化し、現在の単価に対する収益率として表す。計算が簡潔である一方、各キャッシュフローの発生時点を個別に割り引くものではない。

複利利回りでは、各キャッシュフローをその支払時点まで複利で割り引く。市場単価を P 、キャッシュフローを CF_i 、支払時点を t_i とすると、複利利回り y は概念的に

$$P = \sum_{i=1}^n \frac{CF_i}{(1 + y/m)^{mt_i}}$$

を満たす利回りである。ここで m は年間複利回数である。

本章では、ここで用いる市場単価を任意に与えるのではなく、前節の単利利回りから求めたクリーンプライスとする。したがって、計算の流れは**単利利回り → クリーンプライス → 複利利回り**となる。

```
clean_price_input <- yen_simple_price

yen_compound_yield_pct <- GaussQuant::bond_compound_yield_from_price_YEN_GQL(
  start_date = settlement_date_chr,
  end_date = maturity_date_chr,
  coupon_rate = yen_coupon_rate_pct,
  price = clean_price_input,
  redemption = yen_redemption,
  coupon_frequency = 2,
  compound_frequency = 2
)

yen_compound_price <- GaussQuant::bond_price_from_compound_yield_YEN_GQL(
  start_date = settlement_date_chr,
  end_date = maturity_date_chr,
  coupon_rate = yen_coupon_rate_pct,
  compound_yield = yen_compound_yield_pct,
  redemption = yen_redemption,
  coupon_frequency = 2,
  compound_frequency = 2
)

compound_yield <- yen_compound_yield_pct / 100
```

```
tbl_compound_yield <- tibble::tribble(
  ~item, ~value,
  "simple_yield_pct", sprintf("%.8f", yen_simple_rate_pct),
  "clean_price_from_simple_rate", sprintf("%.8f", clean_price_input),
  "compound_yield_pct", sprintf("%.8f", yen_compound_yield_pct),
  "price_from_compound_yield", sprintf("%.8f", yen_compound_price)
)

GaussQuant::show_tbl_GQH(
  tbl_compound_yield,
  "単利利回り → 単価 → 複利利回り"
)

#> # A tibble: 4 x 2
#>   item                                value
#>   <chr>                             <chr>
#> 1 simple_yield_pct                 6.51500000
#> 2 clean_price_from_simple_rate 110.92600000
#> 3 compound_yield_pct              6.71047990
#> 4 price_from_compound_yield      110.92600000
```

フラットイールド評価では、満期ごとに異なるゼロレートを用いるのではなく、同一の利回りをすべての将来キャッシュフローに適用する。これは YTM による債券評価を金利カーブの形式で表現したものと考えることができる。

```
flat_yield_handle <- GaussQuant::bond_flat_forward_handle_GQL(
  settlement_date = settlement_date_chr,
  rate = compound_yield,
  day_counter = dc_30,
  compounding = cmpd_compounded,
  frequency = freq_semiannual
)

flat_engine <- QuantLib::DiscountingBondEngine(flat_yield_handle)

QuantLib::Instrument_setPricingEngine(
  bond_obj,
  flat_engine
)

#> NULL

flat_dirty_price <- QuantLib::Instrument_NPV(bond_obj)

tbl_flat_yield <- tibble::tribble(
  ~item, ~value,
```

```

    "flat_yield_pct", sprintf("%.8f", 100 * compound_yield),
    "dirty_price", sprintf("%.8f", flat_dirty_price)
)

GaussQuant::show_tbl_GQH(
  tbl_flat_yield,
  "フラットイールドによる債券評価"
)

#> # A tibble: 2 x 2
#>   item          value
#>   <chr>         <chr>
#> 1 flat_yield_pct 6.71047990
#> 2 dirty_price    110.92600032

```

離散複利で表された利回りは、同じ経済価値を保ったまま連続複利へ変換できる。半年複利 y と等価な連続複利 r_c は、

$$r_c = 2 \log \left(1 + \frac{y}{2} \right)$$

である。

```

continuous_yield <- 2 * log1p(compound_yield / 2)

tbl_compounding_compare <- tibble::tribble(
  ~yield_type, ~yield_pct,
  "半年複利", 100 * compound_yield,
  "連続複利", 100 * continuous_yield
)

GaussQuant::show_tbl_GQH(
  tbl_compounding_compare,
  "複利方式による利回り表示の比較"
)

#> # A tibble: 2 x 2
#>   yield_type yield_pct
#>   <chr>       <dbl>
#> 1 半年複利    6.7105
#> 2 連続複利    6.6004

```

ここまでの計算は、単利利回り、複利利回り、フラットイールドおよび連続複利が同じ債券を異なる方法で表現していることを示している。特に、単利利回りから市場単価を得た後、その単価を複利評価体系へ変換することが、Z スプレッドやアセットスワップへ進むための接続点

となる。

II-1.6 債券の相対価値評価

単一の YTM は債券全体を一つの利回りで表現する。これに対して金利カーブによる評価では、各キャッシュフローをその支払時点に対応する割引係数で評価する。

この考え方によって、債券固有の利回りを市場の基準金利カーブから分離し、スプレッドとして評価できる。

Z スプレッドは、基準となるゼロカーブの各年限へ一律に加算したときに、割引後キャッシュフローの合計が市場価格と一致する一定スプレッドである。

```
z_spread <- 50 / 10000

zspread_result <- GaussQuant::bond_npv_with_zspread_GQL(
  bond = bond_obj,
  base_curve_handle = flat_yield_handle,
  z_spread = z_spread,
  compounding = compd_compounded,
  frequency = freq_semiannual,
  day_counter = dc_30
)

tbl_zspread <- tibble::tribble(
  ~item, ~value,
  "z_spread_bp", sprintf("%.4f", 10000 * z_spread),
  "npv_with_zspread", sprintf("%.8f", zspread_result$npv)
)

GaussQuant::show_tbl_GQH(
  tbl_zspread,
  "Zスプレッドによる債券評価"
)

#> # A tibble: 2 x 2
#>   item          value
#>   <chr>        <chr>
#> 1 z_spread_bp    50.0000
#> 2 npv_with_zspread 103.77912413
```

アセットスワップでは、固定利付債のキャッシュフローを金利スワップの変動金利キャッシュフローへ変換し、債券をスワップ市場の基準金利と比較する。したがって、単純な「債券利回り－スワップレート」ではなく、債券価格、経過利子、固定クーポン、スワップの割引係数およ

び変動金利レグの現在価値を同じ評価時点で整合させる必要がある。

本章ではアセットスワップの位置づけと計算構造を確認し、実際の OIS・IRS カーブ構築およびマルチカーブ評価は第 II-3 章で扱う。第 II-3 章で構築したスワップカーブを用いることで、本節の債券評価を実際の IRS 市場との相対価値評価へ接続できる。

債券の複利利回りと同年限のスワップレートとの差は相対価値を見るための直感的な指標となるが、これはアセットスワップ・スプレッドそのものではない。アセットスワップではキャッシュフロー単位の現在価値を一致させる必要がある。

この区別を明確にすることで、単利利回りから始まった債券評価を、金利スワップと共通の割引評価体系へ接続できる。

II-1.7 債券の金利感応度

デュレーションは、債券キャッシュフローの回収時点を現在価値で加重平均した指標である。債券の残存期間だけではなく、クーポンと利回りにも依存する。

修正デュレーションは、利回り変化に対する価格の一次感応度を表す。

$$\frac{\Delta P}{P} \approx -D_{\text{mod}} \Delta y$$

```
# VBA定義に合わせ、利回り Y は「%」単位で扱う。
# 例: 6.71047990% は Y = 6.71047990。
#
# P      : 単価 (円 / 額面100円)
# dP/dY  : 利回り1%変化あたりの単価変化 (円)
# BPV    : -dP/dY。数値はそのまま「銭 / bp」と一致する。
#
# 一般的な Modified Duration (利回りを小数で扱う定義) は
#   D_mod = -100 / P * dP/dY
# となる。
```

```
compute_bond_price_LRL <- function(
  compound_yield_pct,
  coupon_rate_pct,
  remaining_years
) {
  if (compound_yield_pct == 0) {
    return(
      face_amount +
      coupon_rate_pct * remaining_years
    )
  }
}
```

```

    )
  }

  discount_factor_half_year <- (
    1 + compound_yield_pct / 200
  )^(-1)

  face_amount * (
    coupon_rate_pct / compound_yield_pct *
    (
      1 - discount_factor_half_year^(
        2 * remaining_years
      )
    ) +
    discount_factor_half_year^(2 * remaining_years)
  )
}

compute_bond_price_derivative_LRL <- function(
  compound_yield_pct,
  coupon_rate_pct,
  remaining_years
) {
  if (compound_yield_pct == 0) {
    return(
      -(
        coupon_rate_pct * remaining_years *
        (2 * remaining_years + 1) / 400 +
        remaining_years
      )
    )
  }

  discount_factor_half_year <- (
    1 + compound_yield_pct / 200
  )^(-1)

  -100 * coupon_rate_pct / compound_yield_pct^2 *
  (
    1 - discount_factor_half_year^(
      2 * remaining_years
    )
  ) -
  (1 - coupon_rate_pct / compound_yield_pct) *
  remaining_years *

```

```

    discount_factor_half_year^(
      2 * remaining_years + 1
    )
  }

compute_bond_price_second_derivative_LRL <- function(
  compound_yield_pct,
  coupon_rate_pct,
  remaining_years
) {
  if (compound_yield_pct == 0) {
    return(
      remaining_years *
      (2 * remaining_years + 1) / 20000 *
      (
        coupon_rate_pct *
        (remaining_years + 1) / 3 +
        100
      )
    )
  }

  discount_factor_half_year <- (
    1 + compound_yield_pct / 200
  )^(-1)

  remaining_years *
  (2 * remaining_years + 1) / 200 *
  (1 - coupon_rate_pct / compound_yield_pct) *
  discount_factor_half_year^(
    2 * remaining_years + 2
  ) -
  2 * coupon_rate_pct * remaining_years /
  compound_yield_pct^2 *
  discount_factor_half_year^(
    2 * remaining_years + 1
  ) +
  200 * coupon_rate_pct / compound_yield_pct^3 *
  (
    1 - discount_factor_half_year^(
      2 * remaining_years
    )
  )
}

```

```

compute_bond_price_example_LRL <- function(
  compound_yield_pct
) {
  compute_bond_price_LRL(
    compound_yield_pct = compound_yield_pct,
    coupon_rate_pct = yen_coupon_rate_pct,
    remaining_years = yen_remaining_years
  )
}

compute_bond_price_derivative_example_LRL <- function(
  compound_yield_pct
) {
  compute_bond_price_derivative_LRL(
    compound_yield_pct = compound_yield_pct,
    coupon_rate_pct = yen_coupon_rate_pct,
    remaining_years = yen_remaining_years
  )
}

compute_bond_price_second_derivative_example_LRL <- function(
  compound_yield_pct
) {
  compute_bond_price_second_derivative_LRL(
    compound_yield_pct = compound_yield_pct,
    coupon_rate_pct = yen_coupon_rate_pct,
    remaining_years = yen_remaining_years
  )
}

tbl_vba_zero_yield <- tibble::tribble(
  ~item, ~value,
  "price_Y0", compute_bond_price_example_LRL(0),
  "dP_dY_Y0", compute_bond_price_derivative_example_LRL(0),
  "BPV_sen_per_bp_Y0", -compute_bond_price_derivative_example_LRL(0),
  "d2P_dY2_Y0", compute_bond_price_second_derivative_example_LRL(0),
  "modified_duration_Y0",
    -100 * compute_bond_price_derivative_example_LRL(0) /
      compute_bond_price_example_LRL(0),
  "convexity_Y0",
    10000 * compute_bond_price_second_derivative_example_LRL(0) /
      compute_bond_price_example_LRL(0)
)

```



```
GaussQuant::show_tbl_GQH(
  tbl_vba_zero_yield,
  "VBA定義：利回り0%の場合"
)

#> # A tibble: 6 x 2
#>   item          value
#>   <chr>        <dbl>
#> 1 price_Y0      400
#> 2 dP_dY_Y0    -100.75
#> 3 BPV_sen_per_bp_Y0 100.75
#> 4 d2P_dY2_Y0    32.805
#> 5 modified_duration_Y0 25.188
#> 6 convexity_Y0  820.12

formula_price <- compute_bond_price_example_LRL(
  yen_compound_yield_pct
)
formula_price_derivative <-
  compute_bond_price_derivative_example_LRL(
    yen_compound_yield_pct
  )
formula_price_second_derivative <-
  compute_bond_price_second_derivative_example_LRL(
    yen_compound_yield_pct
  )

bpv_sen_per_bp <- -formula_price_derivative
bpv_yen_per_bp <- bpv_sen_per_bp / 100

modified_duration <- (
  -100 *
    formula_price_derivative /
    formula_price
)

convexity_value <- (
  10000 *
    formula_price_second_derivative /
    formula_price
)

formula_modified_duration_pct_unit <- (
  -formula_price_derivative /
    formula_price
)
```

```

formula_convexity_pct_unit <- (
  formula_price_second_derivative /
  formula_price
)

tbl_yen_risk <- GaussQuant::bond_risk_from_compound_yield_YEN_GQL(
  start_date = settlement_date_chr,
  end_date = maturity_date_chr,
  coupon_rate = yen_coupon_rate_pct,
  compound_yield = yen_compound_yield_pct,
  redemption = yen_redemption,
  coupon_frequency = 2,
  compound_frequency = 2,
  accrued_interest = 0
)

tbl_risk_compare <- tibble::tribble(
  ~item, ~analytical_formula, ~gaussquant,
  "price", formula_price, yen_compound_price,
  "modified_duration", modified_duration,
  tbl_yen_risk$modified_duration[[1]],
  "convexity", convexity_value,
  tbl_yen_risk$convexity[[1]],
  "bpv_yen_per_bp", bpv_yen_per_bp,
  tbl_yen_risk$bpv_yen[[1]],
  "bpv_sen_per_bp", bpv_sen_per_bp,
  100 * tbl_yen_risk$bpv_yen[[1]]
)

GaussQuant::show_tbl_GQH(
  tbl_risk_compare,
  "VBA定義とGaussQuantによる金利リスクの照合"
)

#> # A tibble: 5 x 3
#>   item          analytical_formula gaussquant
#>   <chr>                <dbl>         <dbl>
#> 1 price                110.93         110.93
#> 2 modified_duration     13.651          13.651
#> 3 convexity             321.17          321.17
#> 4 bpv_yen_per_bp         0.15142         0.15142
#> 5 bpv_sen_per_bp        15.142          15.142

tbl_vba_derivatives <- tibble::tribble(
  ~item, ~value,

```

```

"dP_dY_yen_per_1pct", formula_price_derivative,
"d2P_dY2_yen_per_pct2", formula_price_second_derivative,
"modified_duration_per_pct", formula_modified_duration_pct_unit,
"convexity_per_pct2", formula_convexity_pct_unit,
"BPV_sen_per_bp", bpv_sen_per_bp
)

GaussQuant::show_tbl_GQH(
  tbl_vba_derivatives,
  "VBAの%単位による微分量"
)

#> # A tibble: 5 x 2
#>   item                value
#>   <chr>              <dbl>
#> 1 dP_dY_yen_per_1pct    -15.142
#> 2 d2P_dY2_yen_per_pct2   3.5627
#> 3 modified_duration_per_pct 0.13651
#> 4 convexity_per_pct2      0.032117
#> 5 BPV_sen_per_bp        15.142

```

デュレーションが価格・利回り曲線の一次の傾きを表すのに対し、コンベクシティは二次の曲率を表す。利回り変化が大きくなるほど、一次近似だけでは価格変化を十分に説明できなくなる。

本章では VBA の定義に合わせ、複利利回り Y を **% 単位** で扱う。したがって、 $Y = 6.71047990$ は 6.71047990% を意味する。

単価 P を「円／額面 100 円」で表すと、

$$\text{BPV} = -\frac{\partial P}{\partial Y}$$

と定義できる。この $-\partial P / \partial Y$ は「円／1%」であるが、1bp = 0.01%、1円 = 100 銭であるため、数値はそのまま **銭／bp** と一致する。

本例では、

$$-\frac{\partial P}{\partial Y} \approx 15.1421$$

であるため、BPV は約 15.1421 銭／bp、すなわち 0.151421 円／bp である。

一般的な Modified Duration は利回りを小数で扱う定義なので、

$$D_{\text{mod}} = -\frac{100}{P} \frac{\partial P}{\partial Y}$$

となる。VBA で % 単位の微分値を扱う場合には、この 100 倍の換算を明示する必要がある。

II-1.8 数値解法による複利利回りの計算

複利利回りから債券価格を求める計算は直接評価である。一方、市場で観測されたクリーンプライスから複利利回りを求めるには、

$$f(y) = P_{\text{model}}(y) - P_{\text{market}} = 0$$

を満たす y を求める必要がある。これは典型的な非線形方程式の求根問題である。

数値解法の違いを明確にするため、ここでも本章の冒頭から使用している**残存 40 年の固定利付債**をそのまま用いる。年 2 回払い、額面 100、クーポン 7.50%、単利利回り 6.515% であり、単利利回りから求めたクリーンプライスを共通の市場価格とする。その同じクリーンプライスから複利利回りを逆算し、各数値解法の精度と計算時間を比較する。

40 年債では半年ごとのキャッシュフローが 80 回存在するため、短期債よりも反復計算の差が表れやすい。実務上も、超長期国債や長期社債の評価では、価格計算と利回り計算を大量に繰り返すため、求根アルゴリズムの速度と安定性が重要となる。

```
benchmark_face <- face_amount
benchmark_coupon_rate_pct <- 100 * coupon_rate
benchmark_simple_yield_pct <- yen_simple_rate_pct
benchmark_years <- 40
benchmark_frequency <- 2
benchmark_periods <- benchmark_years * benchmark_frequency
benchmark_redemption <- yen_redemption

# 第II-1章で一貫して使用している40年債のクリーンプライス
benchmark_market_price <- clean_price_input

benchmark_coupon <- (
  benchmark_face *
  benchmark_coupon_rate_pct / 100 /
  benchmark_frequency
)

tbl_solver_benchmark <- tibble::tribble(
```

```

~item, ~value,
"remaining_years", benchmark_years,
"coupon_rate_pct", benchmark_coupon_rate_pct,
"simple_yield_pct", benchmark_simple_yield_pct,
"coupon_frequency", benchmark_frequency,
"cashflow_count", benchmark_periods,
"clean_price_from_simple_yield", benchmark_market_price
)

GaussQuant::show_tbl_GQH(
  tbl_solver_benchmark,
  "40年債の数値解法ベンチマーク条件"
)

#> # A tibble: 6 x 2
#>   item                                value
#>   <chr>                             <dbl>
#> 1 remaining_years                   40
#> 2 coupon_rate_pct                   7.5
#> 3 simple_yield_pct                   6.515
#> 4 coupon_frequency                   2
#> 5 cashflow_count                    80
#> 6 clean_price_from_simple_yield 110.93

```

同じ 40 年債について、複利利回り y から計算される価格関数を定義する。

```

# 数値解法でもVBAと同じく、複利利回りは%単位で扱う。
# 例: 6.71047990% -> yield_pct = 6.71047990

compute_bond_price_benchmark_LRL <- function(
  compound_yield_pct
) {
  compute_bond_price_LRL(
    compound_yield_pct = compound_yield_pct,
    coupon_rate_pct = benchmark_coupon_rate_pct,
    remaining_years = benchmark_years
  )
}

compute_bond_yield_objective_LRL <- function(
  compound_yield_pct
) {
  compute_bond_price_benchmark_LRL(
    compound_yield_pct
  ) -
  benchmark_market_price
}

```

```

}

compute_bond_yield_derivative_LRL <- function(
  compound_yield_pct
) {
  compute_bond_price_derivative_LRL(
    compound_yield_pct = compound_yield_pct,
    coupon_rate_pct = benchmark_coupon_rate_pct,
    remaining_years = benchmark_years
  )
}

```

ニュートン・ラフソン法は、現在の推定値における関数値と微分値から次の推定値を求める。

$$y_{k+1} = y_k - \frac{f(y_k)}{f'(y_k)}$$

ここでは VBA と同じく Y を % 単位で扱う。つまり、6.71047990% は $Y = 6.71047990$ とする。小数表記の 0.067104799 ではない点が重要である。

半年複利の割引係数を

$$\alpha = \left(1 + \frac{Y}{200}\right)^{-1}$$

とおく。

残存年数を T 、年 2 回払いのクーポン率を C (%）、額面を 100 とすると、 $Y \neq 0$ の場合の債券単価は

$$P = 100 \left\{ \frac{C}{Y} (1 - \alpha^{2T}) + \alpha^{2T} \right\}.$$

また、 $Y = 0$ の場合は

$$P = 100 + CT$$

となる。

この価格関数を、% 単位の利回り Y で微分する。VBA では BPV を

$$\text{BPV} = -\frac{\partial P}{\partial Y}$$

と定義している。

ここで $\partial P / \partial Y$ の単位は「円 / 1 percentage point」である。すなわち、利回りが 1.00% 変化したときの、額面 100 円あたりの単価変化を表す。

一方、

$$1\text{bp} = 0.01\%$$

であり、

$$1\text{円} = 100\text{銭}$$

である。

したがって、利回り変化を bp、単価変化を銭で表すと、

$$\Delta p(\text{銭}) = \Delta P(\text{円}) \times 100$$

である。

また、

$$\Delta Y(\%) = \frac{\Delta y(\text{bp})}{100}$$

なので、

$$\Delta p(\text{銭}) \approx \frac{\partial P}{\partial Y} \Delta Y(\%) \times 100$$

に代入すると、

$$\Delta p(\text{銭}) \approx \frac{\partial P}{\partial Y} \frac{\Delta y(\text{bp})}{100} \times 100$$

となる。したがって、

$$\Delta p(\text{銭}) \approx \frac{\partial P}{\partial Y} \Delta y(\text{bp})$$

であり、1bp の変化に対しては、

$$\text{BPV}(\text{銭/bp}) = -\frac{\partial P}{\partial Y}$$

となる。

つまり、% 単位で計算した $-\partial P/\partial Y$ の数値そのものが、銭/bp で表した BPV になる。

例えば本章の 40 年債では、

$$-\frac{\partial P}{\partial Y} \approx 15.1421$$

なので、

$$\text{BPV} \approx 15.1421 \text{ 銭/bp}$$

である。円単位では、

$$15.1421 \text{ 銭} = 0.151421 \text{ 円}$$

なので、

$$\text{BPV} \approx 0.151421 \text{ 円/bp}$$

となる。

同様に、Convexity では価格関数の 2 階微分を用いる。VBA の定義では Y を % 単位で扱うため、

$$\text{Convexity}_{\%} = \frac{1}{P + \text{Acc}} \frac{\partial^2 P}{\partial Y^2}$$

である。

ただし、一般的な Convexity は利回りを小数単位で扱う。そのため、% 単位の 2 階微分から通常の Convexity へ換算する場合には、

$$\text{Convexity} = \frac{10000}{P + \text{Acc}} \frac{\partial^2 P}{\partial Y^2}$$

となる。1 階微分では 100 倍、2 階微分では $100^2 = 10000$ 倍になる点が重要である。

また、利回り変化を bp、単価変化を銭で表した 2 次近似では、

$$\Delta p(\text{銭}) \approx \frac{\partial P}{\partial Y} \Delta y(\text{bp}) + \frac{1}{2} \frac{1}{100} \frac{\partial^2 P}{\partial Y^2} \{\Delta y(\text{bp})\}^2.$$

したがって、bp・銭単位で 2 次項を使う場合には、2 階微分を

$$\frac{\partial^2 p}{\partial y(\text{bp})^2} = \frac{1}{100} \frac{\partial^2 P}{\partial Y^2}$$

として扱う。

VBA では $Y = 0$ を別ケースとして明示的に扱う。半年複利の割引係数は

$$\alpha = \left(1 + \frac{Y}{200}\right)^{-1}$$

であり、 $Y = 0$ では $\alpha = 1$ となる。

債券単価は、

$$P = 100 + CT$$

となる。

1 階微分は、

$$\left. \frac{\partial P}{\partial Y} \right|_{Y=0} = - \left\{ \frac{CT(2T+1)}{400} + T \right\}$$

である。

したがって、

$$\text{BPV}_{Y=0} = - \left. \frac{\partial P}{\partial Y} \right|_{Y=0} = \frac{CT(2T+1)}{400} + T$$

となり、この数値は VBA の単位規約ではそのまま銭／bp である。

2 階微分は、

$$\left. \frac{\partial^2 P}{\partial Y^2} \right|_{Y=0} = \frac{T(2T+1)}{20000} \left\{ \frac{C(T+1)}{3} + 100 \right\}$$

である。

したがって、 $Y = 0$ の Modified Duration は、

$$D_{\text{mod}, Y=0} = - \frac{100}{P + \text{Acc}} \left. \frac{\partial P}{\partial Y} \right|_{Y=0}$$

Convexity は、

$$\text{Convexity}_{Y=0} = \frac{10000}{P + \text{Acc}} \left. \frac{\partial^2 P}{\partial Y^2} \right|_{Y=0}$$

となる。

なお、受渡日が利払日で経過利子 $\text{Acc} = 0$ の場合は、上式の $P + \text{Acc}$ は単に P となる。

$Y \neq 0$ では、

$$P = 100 \left\{ \frac{C}{Y} (1 - \alpha^{2T}) + \alpha^{2T} \right\}, \quad \alpha = \left(1 + \frac{Y}{200} \right)^{-1}$$

を直接微分する。

VBA で使用する 1 階微分は、

$$\frac{\partial P}{\partial Y} = -100 \frac{C}{Y^2} (1 - \alpha^{2T}) - \left(1 - \frac{C}{Y} \right) T \alpha^{2T+1}$$

である。

2 階微分は、

$$\frac{\partial^2 P}{\partial Y^2} = \frac{T(2T+1)}{200} \left(1 - \frac{C}{Y}\right) \alpha^{2T+2} - 2 \frac{CT}{Y^2} \alpha^{2T+1} + 200 \frac{C}{Y^3} (1 - \alpha^{2T})$$

となる。

これにより、価格 P 、BPV、Modified Duration、Convexity をすべて同一の価格関数と同一の % 単位規約から計算できる。

さらに、一般的な Modified Duration は利回りを小数単位で定義するため、VBA の % 単位微分との関係は

$$D_{\text{mod}} = -\frac{100}{P + \text{Acc}} \frac{\partial P}{\partial Y}$$

である。受渡日が利払日で経過利子が 0 なら、

$$D_{\text{mod}} = -\frac{100}{P} \frac{\partial P}{\partial Y}$$

となる。

ニュートン・ラフソン法では、この解析微分 $\partial P / \partial Y$ をそのまま傾きとして使用する。価格方程式

$$f(Y) = P(Y) - P_{\text{market}} = 0$$

に対して、

$$Y_{n+1} = Y_n - \frac{P(Y_n) - P_{\text{market}}}{\partial P / \partial Y}$$

と更新する。この式では Y 自体が % 単位なので、途中で 10^{-4} や 100 を追加で掛ける必要はない。

ニュートン・ラフソン法は、VBAと同じ解析微分 dP/dY をそのまま使用する。
 # Y の単位は%なので、更新式にも追加の1e-4換算は不要。

```
compute_bond_yield_newton_LRL <- function(  
  initial_yield_pct = benchmark_simple_yield_pct,
```

```

tolerance = 1e-12,
max_iter = 100L
) {
  purrr::reduce(
    seq_len(max_iter),
    .init = list(
      yield_pct = initial_yield_pct,
      converged = FALSE
    ),
    .f = function(state, iteration) {
      if (state$converged) {
        return(state)
      }

      objective_value <- compute_bond_yield_objective_LRL(
        state$yield_pct
      )
      price_derivative <- compute_bond_yield_derivative_LRL(
        state$yield_pct
      )

      next_yield_pct <- (
        state$yield_pct -
        objective_value / price_derivative
      )

      list(
        yield_pct = next_yield_pct,
        converged = abs(objective_value) < tolerance
      )
    }
  )$yield_pct
}

yield_newton_result <- compute_bond_yield_newton_LRL()

tbl_newton_check <- tibble::tribble(
  ~item, ~value,
  "initial_yield_pct", benchmark_simple_yield_pct,
  "compound_yield_pct", yield_newton_result,
  "price", compute_bond_price_benchmark_LRL(yield_newton_result),
  "dP_dY", compute_bond_yield_derivative_LRL(yield_newton_result),
  "BPV_sen_per_bp", -compute_bond_yield_derivative_LRL(yield_newton_result),
  "modified_duration",
  -100 *

```

```

    compute_bond_yield_derivative_LRL(yield_newton_result) /
    compute_bond_price_benchmark_LRL(yield_newton_result)
)

GaussQuant::show_tbl_GQH(
  tbl_newton_check,
  "ニュートン・ラフソン法：VBA解析微分による複利利回り計算"
)
#> # A tibble: 6 x 2
#>   item          value
#>   <chr>        <dbl>
#> 1 initial_yield_pct    6.515
#> 2 compound_yield_pct   6.7105
#> 3 price              110.93
#> 4 dP_dY             -15.142
#> 5 BPV_sen_per_bp      15.142
#> 6 modified_duration   13.651

```

二分法は、根を挟む区間を半分ずつ狭めていく方法である。収束速度はニュートン・ラフソン法より遅いが、適切に根を挟む区間を設定すれば安定して解を求められる。

```

compute_bond_yield_bisection_LRL <- function(
  lower = -1,
  upper = 20,
  tolerance = 1e-12,
  max_iter = 200L
) {
  result <- purrr::reduce(
    seq_len(max_iter),
    .init = list(
      lower = lower,
      upper = upper,
      root = NA_real_,
      converged = FALSE
    ),
    .f = function(state, iteration) {
      if (state$converged) {
        return(state)
      }

      midpoint <- (
        state$lower + state$upper
      ) / 2

      objective_lower <- compute_bond_yield_objective_LRL(

```

```

    state$lower
  )
  objective_midpoint <- compute_bond_yield_objective_LRL(
    midpoint
  )

  next_state <- if (
    sign(objective_lower) == sign(objective_midpoint)
  ) {
    list(
      lower = midpoint,
      upper = state$upper
    )
  } else {
    list(
      lower = state$lower,
      upper = midpoint
    )
  }

  list(
    lower = next_state$lower,
    upper = next_state$upper,
    root = midpoint,
    converged = abs(objective_midpoint) < tolerance ||
      abs(state$upper - state$lower) < tolerance
  )
}
)

result$root
}

yield_bisection_result <- compute_bond_yield_bisection_LRL()

```

R の `uniroot()` は Brent 法系の区間求根アルゴリズムを用いる。二分法の堅牢性と補間による高速な収束を組み合わせた実用的な方法である。

```

compute_bond_yield_brent_LRL <- function() {
  stats::uniroot(
    compute_bond_yield_objective_LRL,
    interval = c(-1, 20),
    tol = 1e-12
  )$root
}

```

```
yield_brent_result <- compute_bond_yield_brent_LRL()
```

規則的な固定利付債では、

$$x = \frac{1}{1 + Y/200}$$

と置くことで、価格方程式を x の多項式として表現できる。この場合、R の `polyroot()` を利用してすべての複素根を求め、経済的に意味のある実根を選択できる。

40 年・半年払いでは 80 次多項式となるため、短期債の場合よりも多項式求根法の計算負荷が明確になる。

```
compute_bond_yield_polyroot_LRL <- function() {
  poly_coefficients <- c(
    -benchmark_market_price,
    rep(
      benchmark_coupon,
      benchmark_periods - 1L
    ),
    benchmark_coupon + benchmark_face
  )

  poly_roots <- polyroot(poly_coefficients)

  valid_poly_roots <- poly_roots[
    abs(Im(poly_roots)) < 1e-10 &
    Re(poly_roots) > 0
  ]

  yield_candidates <- (
    1 / Re(valid_poly_roots) - 1
  ) * 200

  yield_candidates[
    which.min(
      abs(
        purrr::map_dbl(
          yield_candidates,
          compute_bond_yield_objective_LRL
        )
      )
    )
  ]
}
```

```

    ]
  }

yield_polyroot_result <- compute_bond_yield_polyroot_LRL()

```

GaussQuant についても同じ 40 年債、同じクリーンプライスを用いる。比較対象をそろえるため、評価日を発行日とし、半年払い・30/360 Bond Basis で規則的な 40 年債を構築する。

```

# 第II-1章で使用している同一の40年債オブジェクトをそのまま利用する
GaussQuant::set_eval_date_GQL(trade_date)

compute_bond_yield_gaussquant_LRL <- function() {
  100 * GaussQuant::bond_yield_from_clean_safe_GQL(
    bond = bond_obj,
    clean_price = benchmark_market_price,
    day_counter = dc_30,
    compounding = cmpd_compounded,
    frequency = freq_semiannual
  )
}

yield_gaussquant_result <- compute_bond_yield_gaussquant_LRL()

```

最後に、計算結果だけでなく計算時間も同時に比較する。一回だけの実行時間は OS や他プロセスの影響を受けやすいため、各解法を同じ回数だけ繰り返し、総経過時間と 1 回当たり平均時間を計測する。

ここでは各解法を 500 回実行する。比較の目的は絶対的な速度ランキングではなく、同一 PC・同一 R セッション・同一 40 年債という条件で、アルゴリズムごとの相対的な計算コストを確認することである。

```

tbl_yield_reference <- tibble::tribble(
  ~yield_type, ~yield_pct,
  "市場慣行の単利利回り", benchmark_simple_yield_pct,
  "単価から逆算した複利利回り", yen_compound_yield_pct
)

GaussQuant::show_tbl_GQH(
  tbl_yield_reference,
  "単利利回りと複利利回りの基準値"
)

#> # A tibble: 2 x 2
#>   yield_type          yield_pct
#>   <chr>              <dbl>

```



```
#> 1 市場慣行の単利利回り      6.515
#> 2 単価から逆算した複利利回り  6.7105

#| label: yield-solver-timing

benchmark_repetitions <- 500L

compute_bond_yield_benchmark_LRL <- function(
  method,
  solver,
  result,
  price_function = NULL
) {
  timing <- system.time(
    purrr::map_dbl(
      seq_len(benchmark_repetitions),
      ~ solver()
    )
  )

  price_error <- if (is.null(price_function)) {
    NA_real_
  } else {
    price_function(result) -
      benchmark_market_price
  }

  tibble::tibble(
    method = method,
    yield = result,
    price_error = price_error,
    repetitions = benchmark_repetitions,
    elapsed_ms = unname(timing[["elapsed"]]) * 1000,
    average_us = (
      unname(timing[["elapsed"]]) *
        1e6 /
        benchmark_repetitions
    )
  )
}

tbl_solver_timing <- dplyr::bind_rows(
  compute_bond_yield_benchmark_LRL(
    "ニュートン・ラフソン法",
    compute_bond_yield_newton_LRL,
```

```

    yield_newton_result,
    compute_bond_price_benchmark_LRL
  ),
  compute_bond_yield_benchmark_LRL(
    "二分法",
    compute_bond_yield_bisection_LRL,
    yield_bisection_result,
    compute_bond_price_benchmark_LRL
  ),
  compute_bond_yield_benchmark_LRL(
    "Brent法 (uniroot) ",
    compute_bond_yield_brent_LRL,
    yield_brent_result,
    compute_bond_price_benchmark_LRL
  ),
  compute_bond_yield_benchmark_LRL(
    "多項式求根法 (polyroot) ",
    compute_bond_yield_polyroot_LRL,
    yield_polyroot_result,
    compute_bond_price_benchmark_LRL
  ),
  compute_bond_yield_benchmark_LRL(
    "GaussQuant",
    compute_bond_yield_gaussquant_LRL,
    yield_gaussquant_result,
    NULL
  )
) |>
dplyr::arrange(.data$average_us)

tbl_solver_timing_display <- tbl_solver_timing |>
dplyr::transmute(
  method,
  compound_yield_pct = sprintf("%.6f", yield),
  calculation_time_ms = sprintf(
    "%.2f",
    elapsed_ms / benchmark_repetitions
  )
)
GaussQuant::show_tbl_GQH(
  tbl_solver_timing_display,
  "40年債：同一複利利回りを求める数値解法の比較"
)
#> # A tibble: 5 x 3
#>   method                                compound_yield_pct calculation_time_ms

```

#>	<chr>	<chr>	<chr>
#> 1	Brent法 (uniroot)	6.710480	0.04
#> 2	ニュートン・ラフソン法	6.710480	0.10
#> 3	二分法	6.710480	0.28
#> 4	多項式求根法 (polyroot)	6.710480	0.70
#> 5	GaussQuant	6.710479	0.98

以上の結果から、本例では Brent 法 → ニュートン・ラフソン法 → 二分法 → polyroot の順に高速な結果となった。Brent 法は二分法の安定性と補間法的高速性を組み合わせた手法であり、本例のように価格関数が利回りに対して滑らかで単調な問題では特に有効である。一方、ニュートン・ラフソン法は解析微分を必要とするものの高い収束性能を持ち、二分法は計算速度では劣るものの安定性に優れる。polyroot は規則的なキャッシュフローでは利用できるが、本例の 40 年債では 80 次多項式となるため計算負荷が大きくなる。

GaussQuant も同じ 40 年債、同じクリーンプライスを入力としている。ただし、単純な価格式だけを解く自作アルゴリズムとは異なり、債券オブジェクト、利払スケジュール、日数計算規則、価格種別などのマーケットコンベンションを含めて処理する。そのため、速度差には求根法そのものの差だけでなく、実務的な債券評価処理のオーバーヘッドも含まれる。

したがって、数値解法の選択では単純な計算速度だけでなく、収束速度、安定性、微分計算の可否、マーケットコンベンションへの対応を含めて評価することが重要である。この考え方は、後続章で扱う金利カーブのブートストラップ、インプライド・ボラティリティ、モデル・キャリブレーションにも共通する。

第 II-2 章金利カーブ構築と債券先物の金利感応度

債券、金利スワップおよび金利オプションの評価では、将来キャッシュフローを現在価値へ換算するための金利期間構造が必要となる。金利期間構造は、満期ごとの金利を並べた単なる曲線ではない。市場で観測される預金金利、債券価格、FRA、OIS レートおよびスワップレートなどから、各将来日の割引係数を整合的に推定した結果である。

同じ期間構造は、割引係数、ゼロ金利またはフォワード金利として表現できる。これらは相互に変換できるが、表現方法によってカーブの特徴の見え方は異なる。割引係数は現在価値計算に直接用いられ、ゼロ金利は評価日から各満期までの平均的な金利水準を示し、フォワード金利は将来の各区間に対応する限界的な金利を示す。

本章では、まず日本国債の長期時系列を用いて、イールドカーブが年限方向と時間方向の双方に変化することを確認する。次に、ゼロ金利ノードから単純なカーブを構築し、割引係数、ゼロ金利およびフォワード金利の関係を整理する。そのうえで補間方式の違い、預金と債券またはスワップを用いたブートストラップ、Implied Term Structure、金利シフトおよびカーブ検証へ進む。最後に、構築した金利期間構造が日本国債先物の受渡適格銘柄と CTD (Cheapest to Deliver) の選択へどのように影響するかを考察する。

II-2.1 イールドカーブおよび補間

第 II-1 章で述べたとおり、パーレート（スワップレート）、スポットレート（ゼロレート）、フォワードレートおよびディスカウントファクターの計算は、将来キャッシュフローを現在価値へ換算するための基礎であり、資本市場取引における最も重要な計算の一つである。

しかし、スポットレートやフォワードレートを市場から直接観測することはできない。実際に観測されるのは、国債価格、国債の複利利回り、預金金利、OIS レートおよびスワップレートなど、取引可能な金融商品の価格またはレートである。したがって、これらの市場データから期間構造を構築する際には、異常値を適切に処理するとともに、限られた年限（tenor）の観測

値をどのように補間し、安定したイールドカーブを構築するかが重要となる。

補間方法の選択は、単にグラフの見た目を滑らかにするためのものではない。補間されたカーブから割引係数、ゼロ金利およびフォワード金利が導出されるため、補間アルゴリズムの違いは金融商品の価格評価とリスク計測にも影響する。

そこで、まず実際の市場金利が年限方向と時間方向にどのように変化するかを確認する。以下では、財務省「[Interest Rate Historical Data](#)」の日本国債複利利回りを用いる。各観測日の離散的な年限データを自然スプラインで補間し、イールドカーブの長期的な変化を可視化する。また、同じデータを年限ごとの時系列へ組み替え、各 tenor の金利水準が時間とともにどのように推移したかを確認する。

財務省の CSV は、観測日を行、年限を列とする wide 形式で提供される。ここでは、すべての列をいったん文字列として読み込み、`pivot_longer()` によって観測日、年限および複利利回りからなる long 形式へ変換する。年限を一つの変数へまとめることで、年限方向のイールドカーブと時間方向の金利時系列を同じデータから描画できる。

```
jgb_yield_url <- paste0(
  "https://www.mof.go.jp/english/policy/jgbs/",
  "reference/interest_rate/historical/jgbcme_all.csv"
)

tbl_jgb_yield_raw <- readr::read_csv(
  jgb_yield_url,
  skip = 1,
  col_types = readr::cols(
    .default = readr::col_character()
  ),
  na = c("", "-"),
  show_col_types = FALSE
)

tbl_jgb_yield <- tbl_jgb_yield_raw |>
  dplyr::rename(
    date_text = Date
  ) |>
  tidyr::pivot_longer(
    cols = -date_text,
    names_to = "tenor",
    values_to = "yield_text"
  ) |>
  dplyr::mutate(
    date = readr::parse_date(
      date_text,
```

```

    format = "%Y/%m/%d"
  ),
  tenor_years = readr::parse_number(tenor),
  compound_yield_pct = readr::parse_double(
    yield_text,
    na = c("", "-")
  )
) |>
dplyr::filter(
  !is.na(date),
  !is.na(tenor_years),
  !is.na(compound_yield_pct),
  date >= as.Date("1984-01-01")
) |>
dplyr::select(
  date,
  tenor,
  tenor_years,
  compound_yield_pct
) |>
dplyr::arrange(
  date,
  tenor_years
)

GaussQuant::show_tbl_GQH(
  tbl_jgb_yield,
  "日本国債の年限別複利利回り",
  n = 20
)

#> # A tibble: 20 x 4
#>   date      tenor tenor_years compound_yield_pct
#>   <date>    <chr>      <dbl>          <dbl>
#> 1 1984-01-04 1Y          1          6.261
#> 2 1984-01-04 2Y          2          6.391
#> 3 1984-01-04 3Y          3          6.832
#> 4 1984-01-04 4Y          4          7.064
#> 5 1984-01-04 5Y          5          7.169
#> 6 1984-01-04 6Y          6          7.129
#> 7 1984-01-04 7Y          7          7.163
#> 8 1984-01-04 8Y          8          7.25
#> 9 1984-01-04 9Y          9          7.347
#> 10 1984-01-05 1Y          1          6.265
#> 11 1984-01-05 2Y          2          6.408
#> 12 1984-01-05 3Y          3          6.861

```

```
#> 13 1984-01-05 4Y      4      7.091
#> 14 1984-01-05 5Y      5      7.196
#> 15 1984-01-05 6Y      6      7.157
#> 16 1984-01-05 7Y      7      7.19
#> 17 1984-01-05 8Y      8      7.276
#> 18 1984-01-05 9Y      9      7.372
#> 19 1984-01-06 1Y      1      6.307
#> 20 1984-01-06 2Y      2      6.457
```

市場で観測される国債利回りは、1年、2年、5年、10年などの離散的な年限に限られる。そこで、観測日ごとに年限方向の自然スプライン補間を行い、0.1年刻みの連続的なカーブへ変換する。元の観測値を点、補間値を線として描くことで、観測された情報と補間によって推定された情報を区別する。

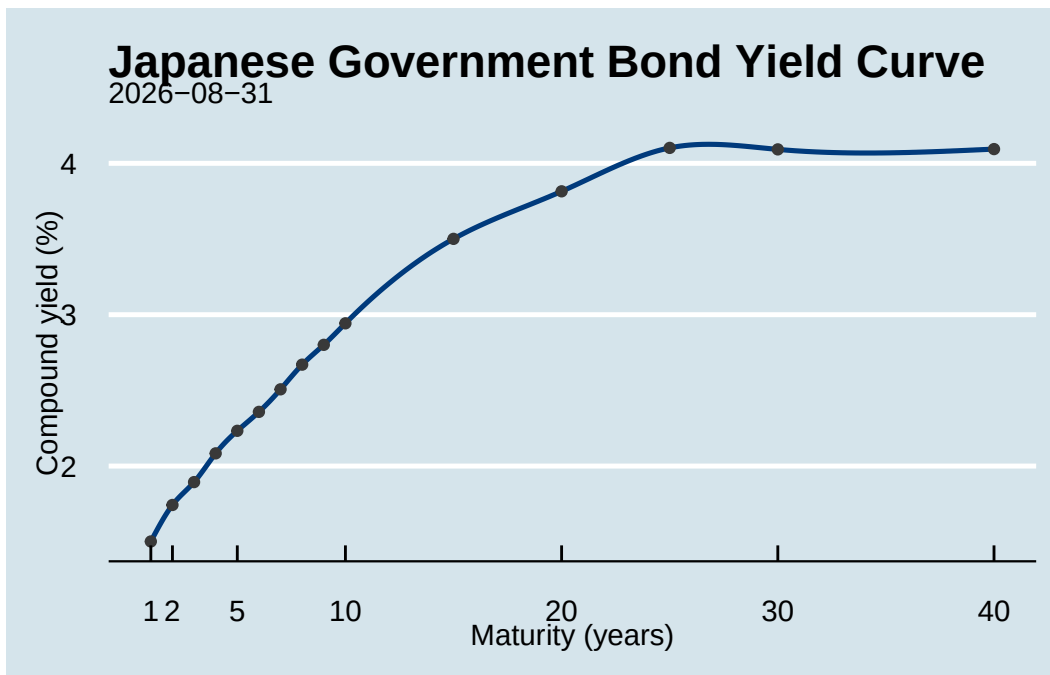
```
tbl_jgb_yield_spline <- tbl_jgb_yield |>
  dplyr::group_by(date) |>
  tidyr::nest() |>
  dplyr::mutate(
    tbl_spline = purrr::map(
      data,
      \(tbl_date) {
        spline_result <- stats::spline(
          x = tbl_date$tenor_years,
          y = tbl_date$compound_yield_pct,
          xout = seq(
            min(tbl_date$tenor_years),
            max(tbl_date$tenor_years),
            by = 0.1
          ),
          method = "natural"
        )

        tibble::tibble(
          tenor_years = spline_result$x,
          compound_yield_pct = spline_result$y
        )
      }
    )
  ) |>
  dplyr::select(
    date,
    tbl_spline
  ) |>
  tidyr::unnest(tbl_spline)
```

```
latest_jgb_date <- max(
  tbl_jgb_yield$date,
  na.rm = TRUE
)

jgb_yield_curve_plot <- tbl_jgb_yield_spline |>
  dplyr::filter(date == latest_jgb_date) |>
  ggplot2::ggplot(
    ggplot2::aes(
      x = tenor_years,
      y = compound_yield_pct
    )
  ) +
  ggplot2::geom_line(
    linewidth = 0.9,
    colour = "#003A7C"
  ) +
  ggplot2::geom_point(
    data = tbl_jgb_yield |>
      dplyr::filter(date == latest_jgb_date),
    size = 1.5,
    colour = "#393939"
  ) +
  ggplot2::scale_x_continuous(
    breaks = c(1, 2, 5, 10, 20, 30, 40)
  ) +
  ggplot2::labs(
    title = "Japanese Government Bond Yield Curve",
    subtitle = format(latest_jgb_date, "%Y-%m-%d"),
    x = "Maturity (years)",
    y = "Compound yield (%)"
  )

show_plot_LFL(jgb_yield_curve_plot)
```

日次データをそのままアニメーション化するとフレーム数が過大になるため、各四半期の最終観測日を抽出する。これにより、市場の長期的な局面変化を保ちながら、再現可能な計算量に抑える。

```
tbl_jgb_animation_dates <- tbl_jgb_yield |>
  dplyr::distinct(date) |>
  dplyr::mutate(
    year = as.integer(format(date, "%Y")),
    quarter = paste0(
      "Q",
      (as.integer(format(date, "%m")) - 1) %/% 3 + 1
    )
  ) |>
  dplyr::group_by(
    year,
    quarter
  ) |>
  dplyr::slice_max(
    order_by = date,
    n = 1,
    with_ties = FALSE
  ) |>
  dplyr::ungroup() |>
  dplyr::select(date)

tbl_jgb_yield_animation <- tbl_jgb_yield |>
  dplyr::semi_join(
    tbl_jgb_animation_dates,
    by = "date"
```

```

)

tbl_jgb_spline_animation <- tbl_jgb_yield_spline |>
  dplyr::semi_join(
    tbl_jgb_animation_dates,
    by = "date"
  )

tbl_jgb_time_series <- tbl_jgb_yield_animation |>
  dplyr::filter(
    tenor_years %in% c(1, 2, 5, 10, 20, 30, 40)
  ) |>
  dplyr::mutate(
    tenor = factor(
      tenor,
      levels = unique(tenor[order(tenor_years)])
    )
  )

```

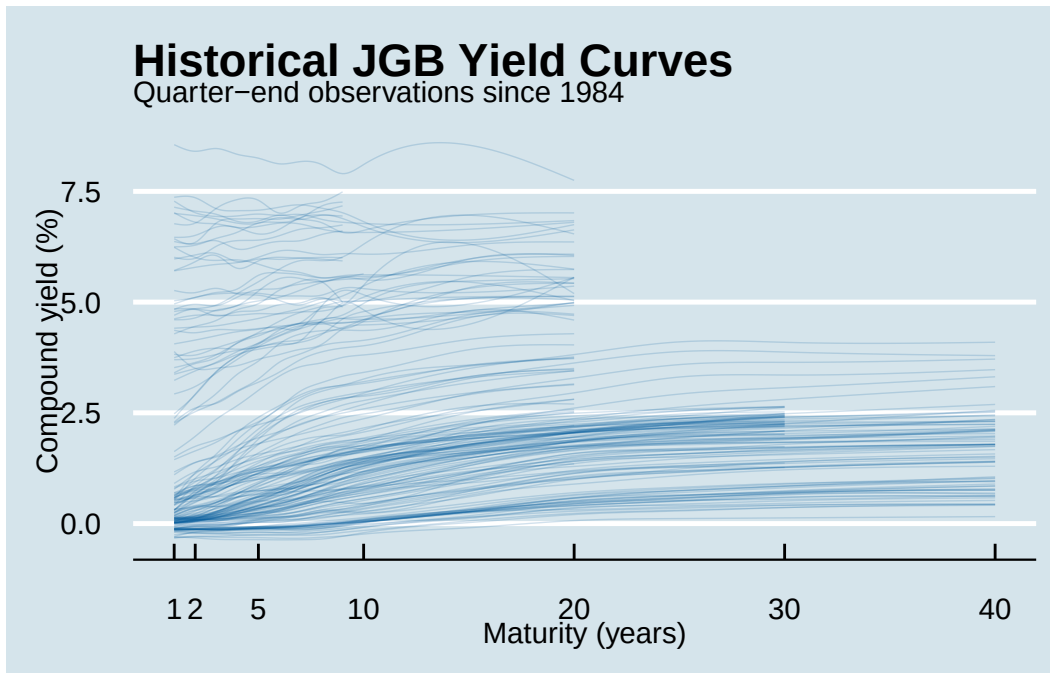
まず、四半期ごとのイールドカーブを静止画で重ね、全期間の形状の範囲を確認する。図の表示には本書共通の `show_plot_LFL()` を用いる。

```

jgb_yield_curve_history_plot <- tbl_jgb_spline_animation |>
  ggplot2::ggplot(
    ggplot2::aes(
      x = tenor_years,
      y = compound_yield_pct,
      group = date
    )
  ) +
  ggplot2::geom_line(
    linewidth = 0.25,
    alpha = 0.18,
    colour = "#005A9C"
  ) +
  ggplot2::scale_x_continuous(
    breaks = c(1, 2, 5, 10, 20, 30, 40)
  ) +
  ggplot2::labs(
    title = "Historical JGB Yield Curves",
    subtitle = "Quarter-end observations since 1984",
    x = "Maturity (years)",
    y = "Compound yield (%)"
  )

```

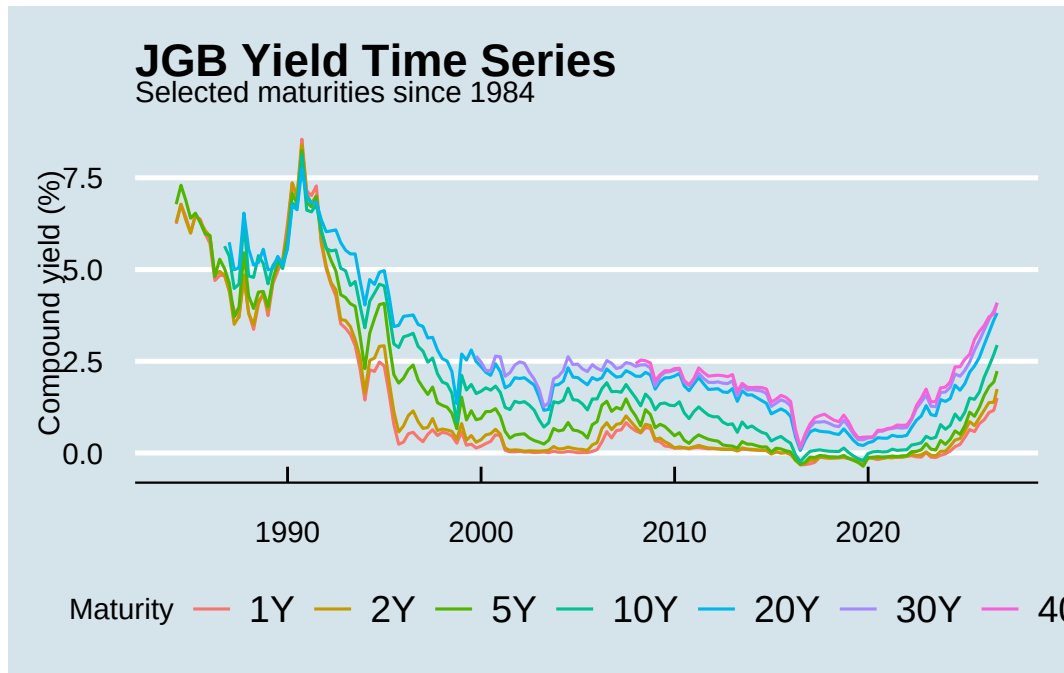
```
show_plot_LFL(jgb_yield_curve_history_plot)
```



次に、同じデータを年限別の時系列として描く。ある日付を固定して横方向に見たものがイールドカーブであり、ある年限を固定して時間方向に見たものが金利時系列である。

```
jgb_yield_time_series_plot <- tbl_jgb_time_series |>
  ggplot2::ggplot(
    ggplot2::aes(
      x = date,
      y = compound_yield_pct,
      colour = tenor
    )
  ) +
  ggplot2::geom_line(
    linewidth = 0.55
  ) +
  ggplot2::labs(
    title = "JGB Yield Time Series",
    subtitle = "Selected maturities since 1984",
    x = NULL,
    y = "Compound yield (%)",
    colour = "Maturity",
    linetype = "Maturity"
  ) +
  ggplot2::guides(
    colour = ggplot2::guide_legend(nrow = 1),
    linetype = ggplot2::guide_legend(nrow = 1)
  )
```

```
show_plot_LFL(jgb_yield_time_series_plot)
```



HTML 出力では、`gganimate` と `gifski` が利用可能な場合に二つのアニメーションを生成する。

```
jgb_yield_animation_file <- file.path(
  "images",
  "jgb_yield_curve_animation.gif"
)

if (!file.exists(jgb_yield_animation_file)) {
  dir.create(
    "images",
    recursive = TRUE,
    showWarnings = FALSE
  )
}

jgb_yield_curve_animation_plot <- tbl_jgb_spline_animation |>
  ggplot2::ggplot(
    ggplot2::aes(
      x = tenor_years,
      y = compound_yield_pct
    )
  ) +
  ggplot2::geom_line(
    linewidth = 0.8,
    colour = "#003A7C"
  ) +
  ggplot2::geom_point(
```

```
    data = tbl_jgb_yield_animation,
    size = 1.5,
    colour = "#393939"
  ) +
  ggplot2::scale_x_continuous(
    breaks = c(1, 2, 5, 10, 20, 30, 40),
    limits = range(
      tbl_jgb_spline_animation$tenor_years,
      na.rm = TRUE
    )
  ) +
  ggplot2::scale_y_continuous(
    limits = range(
      tbl_jgb_spline_animation$compound_yield_pct,
      na.rm = TRUE
    )
  ) +
  ggplot2::labs(
    title = "JGB Yield Curve",
    subtitle = "Date: {closest_state}",
    x = "Maturity (years)",
    y = "Compound yield (%)"
  ) +
  ggplot2::theme(
    axis.text.x = ggplot2::element_text(size = 20),
    axis.text.y = ggplot2::element_text(size = 20),
    axis.title.x = ggplot2::element_text(size = 22),
    axis.title.y = ggplot2::element_text(size = 22)
  ) +
  gganimate::transition_states(
    date,
    transition_length = 1,
    state_length = 0,
    wrap = FALSE
  )

jgb_yield_curve_animation <- gganimate::animate(
  jgb_yield_curve_animation_plot,
  width = 900,
  height = 560,
  fps = 8,
  duration = 24,
  renderer = gganimate::gifski_renderer()
)
```

```
gganimate::anim_save(  
  filename = jgb_yield_animation_file,  
  animation = jgb_yield_curve_animation  
)  
}
```

```
jgb_yield_time_series_animation_file <- file.path(  
  "images",  
  "jgb_yield_time_series_animation.gif"  
)  
  
if (!file.exists(jgb_yield_time_series_animation_file)) {  
  dir.create(  
    "images",  
    recursive = TRUE,  
    showWarnings = FALSE  
  )  
  
jgb_yield_time_series_animation_plot <- tbl_jgb_time_series |>  
  ggplot2::ggplot(  
    ggplot2::aes(  
      x = date,  
      y = compound_yield_pct,  
      colour = tenor  
    )  
  ) +  
  ggplot2::geom_line(  
    linewidth = 1.2  
  ) +  
  ggplot2::scale_x_date(  
    limits = range(  
      tbl_jgb_time_series$date,  
      na.rm = TRUE  
    )  
  ) +  
  ggplot2::scale_y_continuous(  
    limits = range(  
      tbl_jgb_time_series$compound_yield_pct,  
      na.rm = TRUE  
    )  
  ) +  
  ggplot2::labs(  
    title = "JGB Yield Time Series",  
    subtitle = "Observed through {frame_along}",  
    x = NULL,
```

```

    y = "Compound yield (%)",
    colour = "Maturity",
    linetype = "Maturity"
  ) +
  ggplot2::guides(
    colour = ggplot2::guide_legend(nrow = 1),
    linetype = ggplot2::guide_legend(nrow = 1)
  ) +
  ggplot2::theme(
    axis.text.x = ggplot2::element_text(size = 20),
    axis.text.y = ggplot2::element_text(size = 20),
    axis.title.x = ggplot2::element_text(size = 22),
    axis.title.y = ggplot2::element_text(size = 22)
  ) +
  gganimate::transition_reveal(date)

jgb_yield_time_series_animation <- gganimate::animate(
  jgb_yield_time_series_animation_plot,
  width = 900,
  height = 560,
  fps = 8,
  duration = 24,
  renderer = gganimate::gifski_renderer()
)

gganimate::anim_save(
  filename = jgb_yield_time_series_animation_file,
  animation = jgb_yield_time_series_animation
)
}

```

二つの図は、同じ市場データを異なる方向から観察したものである。イールドカーブは一時点における年限間の相対関係を示し、金利時系列は各年限の水準が金融政策、景気および市場環境の変化を受けて推移する様子を示す。この二つの視点を区別することが、補間、ブートストラップおよび金利リスクを理解する出発点となる。

II-2.2 金利カーブの基礎

評価日を 0、将来時点を t とし、時点 t に 1 を受け取るキャッシュフローの現在価値を $D(0, t)$ とする。 $D(0, t)$ は割引係数である。連続複利のゼロ金利を $z(0, t)$ とすると、両者の関係は次式で表される。

$$D(0, t) = \exp[-z(0, t)t]$$

したがって、割引係数からゼロ金利を求めると次式となる。

$$z(0, t) = -\frac{\log D(0, t)}{t}$$

二つの時点 t_1 と t_2 の間に適用される連続複利のフォワード金利を $f(t_1, t_2)$ とすると、次式で求められる。

$$f(t_1, t_2) = -\frac{\log [D(0, t_2)/D(0, t_1)]}{t_2 - t_1}$$

割引係数、ゼロ金利およびフォワード金利は同じ期間構造を異なる方法で表している。ただし、補間方法の違いは、とくにフォワード金利の形状に強く表れる。

最初に、評価日と複数の将来日について連続複利のゼロ金利を設定する。ここで用いる数値は、カーブの構造を確認するための例示値である。

```
evaluation_date_chr <- "2022-08-19"

GaussQuant::set_eval_date_GQL(evaluation_date_chr)

tbl_zero_nodes <- tibble::tribble(
  ~tenor, ~date, ~zero_rate,
  "0D", as.Date("2022-08-19"), 0.0005,
  "1M", as.Date("2022-09-19"), 0.0006,
  "3M", as.Date("2022-11-21"), 0.0008,
  "6M", as.Date("2023-02-20"), 0.0011,
  "1Y", as.Date("2023-08-21"), 0.0018,
  "2Y", as.Date("2024-08-19"), 0.0032,
  "3Y", as.Date("2025-08-19"), 0.0048,
  "5Y", as.Date("2027-08-19"), 0.0075,
  "10Y", as.Date("2032-08-19"), 0.0115
)

GaussQuant::show_tbl_GQH(
  tbl_zero_nodes,
  "Zero-rate nodes",
  n = 20
)

#> # A tibble: 9 x 3
#>   tenor date          zero_rate
```



```
#>   <chr> <date>      <dbl>
#> 1 0D    2022-08-19  0.0005
#> 2 1M    2022-09-19  0.0006
#> 3 3M    2022-11-21  0.0008
#> 4 6M    2023-02-20  0.0011
#> 5 1Y    2023-08-21  0.0018
#> 6 2Y    2024-08-19  0.0032
#> 7 3Y    2025-08-19  0.0048
#> 8 5Y    2027-08-19  0.0075
#> 9 10Y   2032-08-19  0.0115
```

`build_zero_curve_GQL()` は、日付とゼロ金利の組から QuantLib の ZeroCurve を構築する。ここではカーブの時間軸に Actual/365 Fixed を用いる。

```
zero_curve <- GaussQuant::build_zero_curve_GQL(
  date_chr = as.character(tbl_zero_nodes$date),
  zero_rates = tbl_zero_nodes$zero_rate,
  day_counter = "Actual365Fixed"
)

zero_curve_handle <- GaussQuant::yield_curve_handle_GQL(
  zero_curve
)

tbl_zero_curve <- GaussQuant::curve_tbl_GQH(
  curve = zero_curve,
  tenors = c(
    "1D", "1W", "1M", "3M", "6M",
    "1Y", "2Y", "3Y", "5Y", "7Y", "10Y"
  )
)

GaussQuant::show_tbl_GQH(
  tbl_zero_curve,
  "Zero curve",
  n = 20
)

#> # A tibble: 11 x 4
#>   tenor date      discount zero_rate
#>   <chr> <date>      <dbl>      <dbl>
#> 1 1D    2022-08-20  1.00000 0.00050323
#> 2 1W    2022-08-26  0.99999 0.00052258
#> 3 1M    2022-09-19  0.99995 0.00060000
#> 4 3M    2022-11-19  0.99980 0.00079365
#> 5 6M    2023-02-19  0.99945 0.0010967
```

```
#> 6 1Y    2023-08-19  0.99821 0.0017923
#> 7 2Y    2024-08-19  0.99361 0.0032000
#> 8 3Y    2025-08-19  0.98569 0.0048000
#> 9 5Y    2027-08-19  0.96317 0.0075
#> 10 7Y   2029-08-19  0.93824 0.0091004
#> 11 10Y  2032-08-19  0.89128 0.0115
```

カーブから取得したゼロ金利は連続複利表示である。入力ノードの間では QuantLib の補間規則に従って金利が補間される。

カーブを細かい時間間隔へ展開し、割引係数、ゼロ金利および隣接区間のフォワード金利を計算する。

```
tbl_zero_curve_grid <- GaussQuant::curve_grid_tbl_GQL(
  curve = zero_curve,
  n = 240L,
  extrapolate = TRUE
) |>
dplyr::mutate(
  next_time = dplyr::lead(time),
  next_discount = dplyr::lead(discount_factor),
  forward_rate = dplyr::if_else(
    !is.na(next_time) &
    next_time > time &
    discount_factor > 0 &
    next_discount > 0,
    -log(next_discount / discount_factor) /
      (next_time - time),
    NA_real_
  )
)

zero_grid_show_index <- unique(
  round(
    seq(
      from = 1,
      to = nrow(tbl_zero_curve_grid),
      length.out = 12
    )
  )
)

GaussQuant::show_tbl_GQH(
  tbl_zero_curve_grid |>
```

```

dplyr::slice(zero_grid_show_index) |>
dplyr::select(
  curve_date,
  time,
  discount_factor,
  zero_rate,
  forward_rate
),
"Zero-curve grid",
n = 20
)
#> # A tibble: 12 x 5
#>   curve_date      time discount_factor zero_rate forward_rate
#>   <date>         <dbl>          <dbl>      <dbl>      <dbl>
#> 1 2022-08-19     0            1          0          0.00054839
#> 2 2023-07-21  0.92055        0.99845  0.0016808  0.0030308
#> 3 2024-06-06  1.8          0.99477  0.0029154  0.0055000
#> 4 2025-05-08  2.7205        0.98824  0.0043485  0.0087671
#> 5 2026-04-09  3.6411        0.97960  0.0056618  0.010636
#> 6 2027-03-12  4.5644        0.96896  0.0069082  0.013126
#> 7 2028-01-26  5.4411        0.95819  0.0078503  0.012233
#> 8 2028-12-28  6.3644        0.94681  0.0085881  0.013707
#> 9 2029-11-29  7.2849        0.93433  0.0093238  0.015178
#> 10 2030-10-31  8.2055        0.92077  0.010059   0.016652
#> 11 2031-09-17  9.0849        0.90685  0.010762   0.018057
#> 12 2032-08-19 10.008        0.89128  0.0115     NA

```

```

discount_factor_plot <- tbl_zero_curve_grid |>
  ggplot2::ggplot(
    ggplot2::aes(
      x = curve_date,
      y = discount_factor
    )
  ) +
  ggplot2::geom_line() +
  ggplot2::labs(
    title = "Discount factor curve",
    x = NULL,
    y = "Discount factor"
  )

```

```

zero_rate_plot <- tbl_zero_curve_grid |>
  ggplot2::ggplot(
    ggplot2::aes(
      x = curve_date,

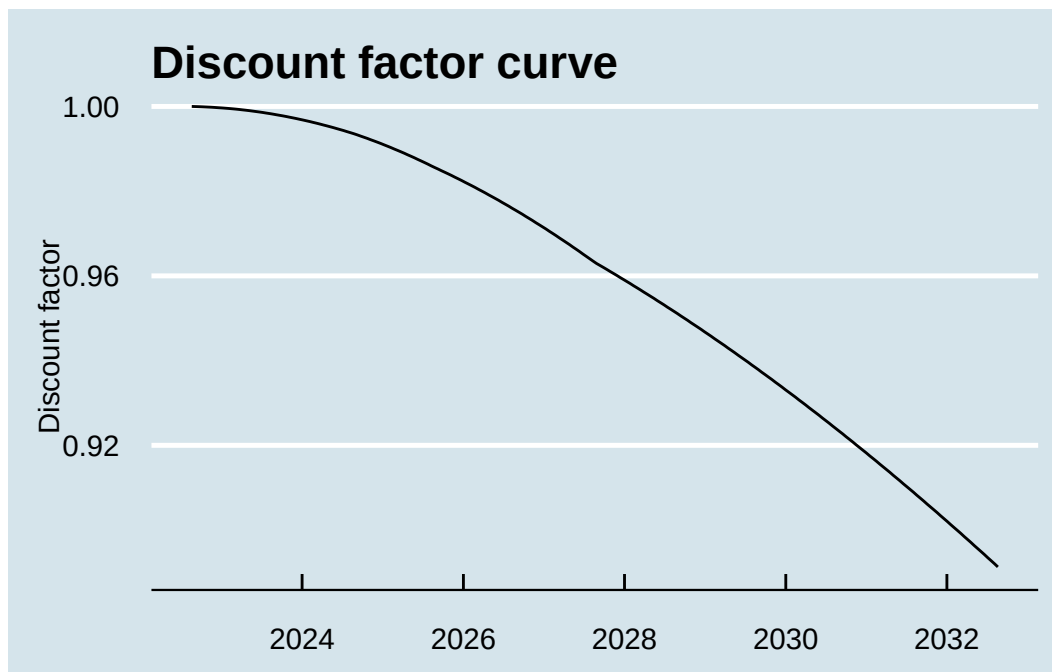
```

```

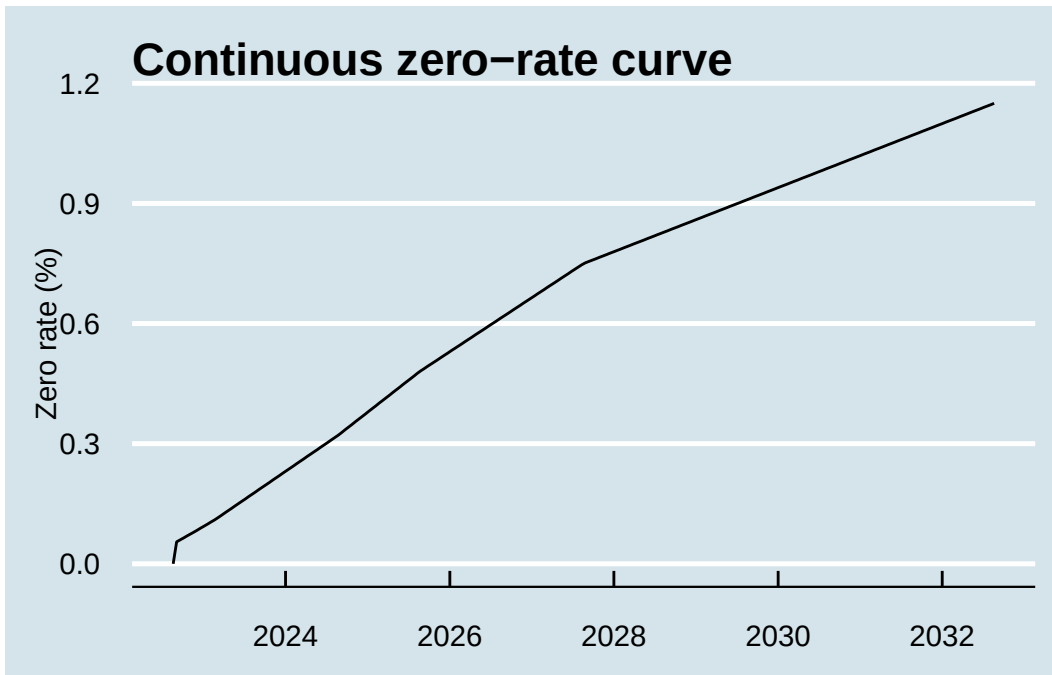
    y = 100 * zero_rate
  )
) +
ggplot2::geom_line() +
ggplot2::labs(
  title = "Continuous zero-rate curve",
  x = NULL,
  y = "Zero rate (%)"
)

forward_rate_plot <- tbl_zero_curve_grid |>
  dplyr::filter(!is.na(forward_rate)) |>
  ggplot2::ggplot(
    ggplot2::aes(
      x = curve_date,
      y = 100 * forward_rate
    )
  ) +
  ggplot2::geom_line() +
  ggplot2::labs(
    title = "One-step forward-rate curve",
    x = NULL,
    y = "Forward rate (%)"
  )
show_plot_LFL(discount_factor_plot)

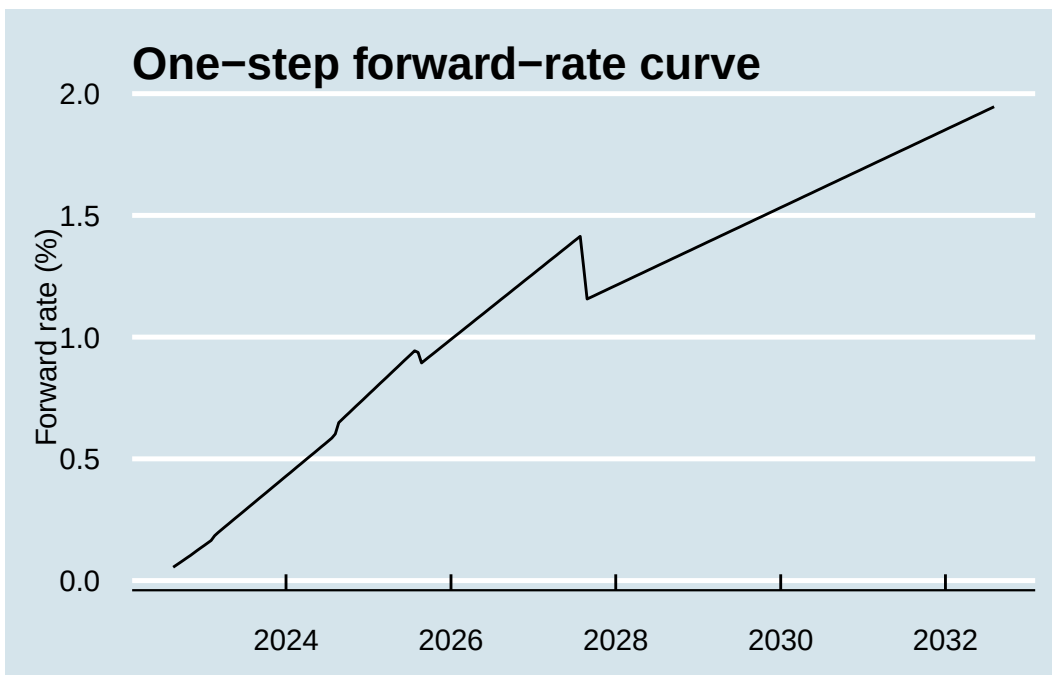
```



```
show_plot_LFL(zero_rate_plot)
```



```
show_plot_LFL(forward_rate_plot)
```



割引係数は将来の 1 単位を現在価値へ換算する倍率である。正の金利環境では、満期が長くなるほど一般に小さくなる。ゼロ金利は評価日から各満期までの平均的な金利水準を示す。一方、フォワード金利は隣接する二時点間の金利を表すため、補間方法やノード配置の影響を強く受ける。

II-2.3 補間と参照日

市場では限られた満期の金利または価格しか観測できないため、ノード間の値を補間する必要がある。補間対象には、ゼロ金利、割引係数、対数割引係数またはフォワード金利などがある。

ここでは、同じゼロ金利ノードから次の二つのカーブを構築する。

- ZeroCurve：ゼロ金利を基礎とするカーブ
- DiscountCurve：ノードのゼロ金利を割引係数へ変換し、割引係数を基礎とするカーブ

```
discount_curve <- GaussQuant::build_discount_curve_GQL(
  nodes = tbl_zero_nodes |>
    dplyr::select(
      date,
      zero_rate
    ),
  day_counter = "Actual365Fixed"
)

discount_curve_handle <- GaussQuant::yield_curve_handle_GQL(
  discount_curve
)

comparison_max_time <- min(
  as.numeric(zero_curve$maxTime()),
  as.numeric(discount_curve$maxTime())
)

comparison_time_grid <- seq(
  from = 0,
  to = comparison_max_time,
  length.out = 300
)

comparison_reference_date <- as.Date(
  GaussQuant::iso_GQL(
    zero_curve$referenceDate()
  )
)

tbl_interpolation <- tibble::tibble(
  time = comparison_time_grid,
  date = comparison_reference_date +
    round(comparison_time_grid * 365)
) |>
  dplyr::mutate(
    ql_date = purrr::map(
      as.character(date),
      GaussQuant::date_GQL
    ),
    zero_curve_discount = purrr::map_dbl(
      ql_date,
```

```
  ~ GaussQuant::curve_discount_safe_GQL(
    zero_curve,
    .x
  )
),
discount_curve_discount = purrr::map_dbl(
  ql_date,
  ~ GaussQuant::curve_discount_safe_GQL(
    discount_curve,
    .x
  )
)
) |>
dplyr::select(
  -ql_date
) |>
tidyr::pivot_longer(
  cols = c(
    zero_curve_discount,
    discount_curve_discount
  ),
  names_to = "interpolation",
  values_to = "discount_factor"
) |>
dplyr::mutate(
  interpolation = dplyr::recode(
    interpolation,
    zero_curve_discount = "ZeroCurve",
    discount_curve_discount = "DiscountCurve"
  )
) |>
dplyr::group_by(
  interpolation
) |>
dplyr::arrange(
  interpolation,
  time
) |>
dplyr::mutate(
  zero_rate = dplyr::if_else(
    time > 0 &
      discount_factor > 0,
    -log(discount_factor) /
      time,
    tbl_zero_nodes$zero_rate[[1]]
  )
)
```

```

),
next_time = dplyr::lead(
  time
),
next_discount = dplyr::lead(
  discount_factor
),
forward_rate = dplyr::if_else(
  !is.na(next_time) &
  next_time > time &
  discount_factor > 0 &
  next_discount > 0,
  -log(
    next_discount /
    discount_factor
  ) / (
    next_time -
    time
  ),
  NA_real_
)
) |>
dplyr::ungroup()

```

```

interpolation_zero_plot <- tbl_interpolation |>
ggplot2::ggplot(
  ggplot2::aes(
    x = time,
    y = 100 * zero_rate,
    linetype = interpolation
  )
) +
ggplot2::geom_line() +
ggplot2::labs(
  title = "Zero rates under different interpolation targets",
  x = "Time from reference date",
  y = "Zero rate (%)",
  linetype = NULL
)

interpolation_forward_plot <- tbl_interpolation |>
dplyr::filter(!is.na(forward_rate)) |>
ggplot2::ggplot(
  ggplot2::aes(
    x = time,

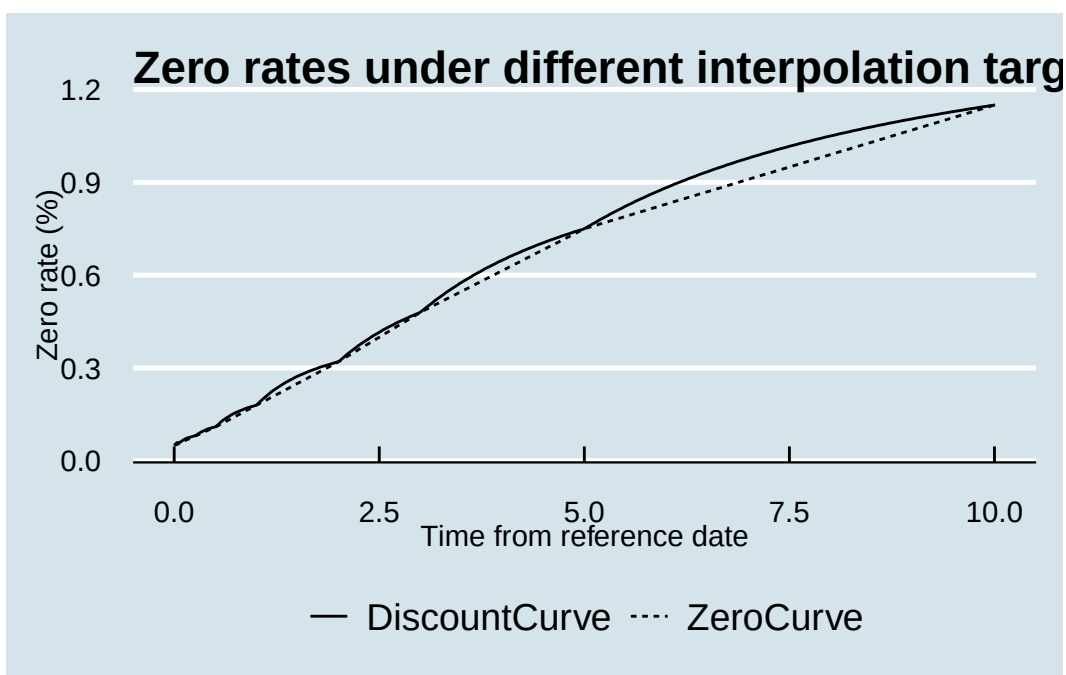
```



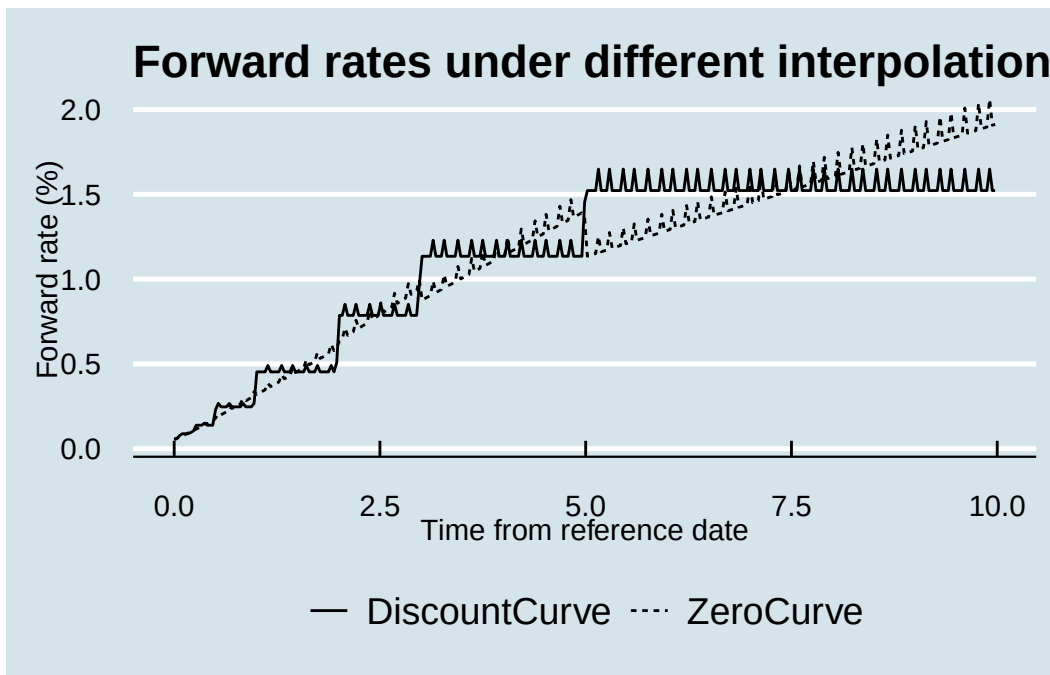
```

    y = 100 * forward_rate,
    linetype = interpolation
  )
) +
ggplot2::geom_line() +
ggplot2::labs(
  title = "Forward rates under different interpolation targets",
  x = "Time from reference date",
  y = "Forward rate (%)",
  linetype = NULL
)
show_plot_LFL(interpolation_zero_plot)

```



```
show_plot_LFL(interpolation_forward_plot)
```



両カーブは入力ノードではほぼ同じ情報を再現するが、ノード間では異なる形状を示す。ゼロ金利のグラフでは差が小さく見えても、フォワード金利へ変換すると差が拡大することがある。したがって、カーブの妥当性はゼロ金利だけでなく、フォワード金利も確認する必要がある。

カーブの参照日は、割引係数が 1 となる時点であり、時間軸の起点である。日付ノードから構築したカーブでは、通常、最初のノード日が固定参照日となる。

次の確認では、QuantLib の評価日を変更した後も、すでに構築済みの ZeroCurve の参照日が変わらないことを確認する。

```
reference_date_before <- as.Date(
  GaussQuant::iso_GQL(
    zero_curve$referenceDate()
  )
)

GaussQuant::set_eval_date_GQL(
  "2022-09-19"
)

reference_date_after <- as.Date(
  GaussQuant::iso_GQL(
    zero_curve$referenceDate()
  )
)

tbl_reference_date <- tibble::tribble(
  ~item, ~value,
  "curve_reference_date_before", as.character(reference_date_before),
```

```

    "evaluation_date_after_change", "2022-09-19",
    "curve_reference_date_after", as.character(reference_date_after)
)

GaussQuant::show_tbl_GQH(
  tbl_reference_date,
  "Fixed reference date"
)

#> # A tibble: 3 x 2
#>   item                value
#>   <chr>              <chr>
#> 1 curve_reference_date_before 2022-08-19
#> 2 evaluation_date_after_change 2022-09-19
#> 3 curve_reference_date_after 2022-08-19

GaussQuant::set_eval_date_GQL(
  evaluation_date_chr
)

```

固定参照日のカーブは、評価日を変更しても参照日が変化しない。一方、決済日数と休日カレンダーから参照日を決定する Moving Reference Date 型のカーブでは、評価日の変更に応じて参照日も移動する。評価日を進めるシナリオ分析では、既存カーブを固定したまま使うのか、市場データから新しいカーブを再構築するのかを区別する必要がある。

II-2.4 ブートストラップ

市場ではゼロ金利を直接観測できる満期は限られている。そのため、短期預金、国債価格およびスワップレートなどから、割引係数を順次推定するブートストラップを行う。

短期部分では預金金利から最初の割引係数を求め、その先では固定利付債の市場価格と理論価格が一致するようにカーブを延長する。GaussQuant の `build_bond_discount_curve_GQL()` は、短期預金と複数の固定利付債を用いて PiecewiseFlatForward カーブを構築する。

```

GaussQuant::set_eval_date_GQL(
  "2008-09-15"
)

bond_curve <- GaussQuant::build_bond_discount_curve_GQL(
  settlement_date = "2008-09-18",
  fixing_days = 3,
  settlement_days = 3
)

```

```
tbl_bond_curve <- GaussQuant::curve_tbl_GQH(
  curve = bond_curve,
  tenors = c(
    "1M", "3M", "6M", "1Y",
    "2Y", "3Y", "5Y", "7Y",
    "10Y", "15Y", "20Y"
  )
)
```

```
GaussQuant::show_tbl_GQH(
  tbl_bond_curve,
  "Deposit and bond bootstrap curve",
  n = 30
)
```

```
#> # A tibble: 11 x 4
#>   tenor date      discount zero_rate
#>   <chr> <date>      <dbl>    <dbl>
#> 1 1M    2008-10-18  0.99921 0.0096148
#> 2 3M    2008-12-18  0.99761 0.0096148
#> 3 6M    2009-03-18  0.99286 0.014471
#> 4 1Y    2009-09-18  0.98097 0.019229
#> 5 2Y    2010-09-18  0.95732 0.021818
#> 6 3Y    2011-09-18  0.92904 0.024541
#> 7 5Y    2013-09-18  0.85971 0.030237
#> 8 7Y    2015-09-18  0.78217 0.035102
#> 9 10Y   2018-09-18  0.67867 0.038764
#> 10 15Y  2023-09-18  0.53179 0.042103
#> 11 20Y  2028-09-18  0.41668 0.043772
```

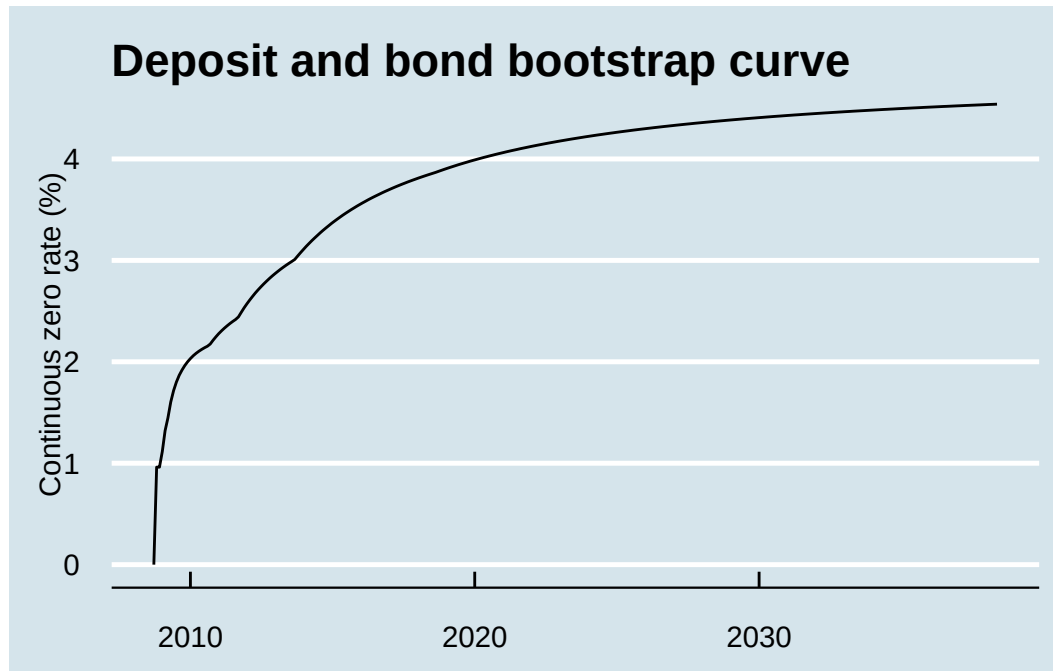
```
tbl_bond_curve_grid <- GaussQuant::curve_grid_tbl_GQL(
  curve = bond_curve,
  n = 300L,
  extrapolate = TRUE
)
```

```
bond_curve_plot <- tbl_bond_curve_grid |>
  ggplot2::ggplot(
    ggplot2::aes(
      x = curve_date,
      y = 100 * zero_rate
    )
  ) +
  ggplot2::geom_line() +
  ggplot2::labs(
```

```

    title = "Deposit and bond bootstrap curve",
    x = NULL,
    y = "Continuous zero rate (%)"
  )
show_plot_LFL(bond_curve_plot)

```



債券価格を用いるブートストラップでは、各債券のクーポン日、満期日、経過利子および日数計算規則がカーブ構築に影響する。債券のクーポン計算には商品固有の日数計算規則を用いる一方、期間構造の時間軸には単純で加法的な日数計算規則を用いる必要がある。

次に、短期部分を預金金利、中長期部分を固定対変動金利スワップの市場レートから構築する。スワップの市場レートは、固定レグと変動レグの現在価値を等しくするパー・スワップレートである。

GaussQuant の `build_swap_curve_GQL()` は、短期預金と複数満期のスワップレートを用いて `PiecewiseFlatForward` カーブを構築する。

```

swap_curve <- GaussQuant::build_swap_curve_GQL(
  settlement_date = "2008-09-18",
  fixing_days = 3
)

swap_curve_handle <- GaussQuant::yield_curve_handle_GQL(
  swap_curve
)

tbl_swap_curve <- GaussQuant::curve_tbl_GQH(
  curve = swap_curve,

```

```

tenors = c(
  "1W", "1M", "3M", "6M",
  "1Y", "2Y", "3Y", "5Y",
  "7Y", "10Y", "15Y"
)
)

GaussQuant::show_tbl_GQH(
  tbl_swap_curve,
  "Deposit and swap bootstrap curve",
  n = 30
)

#> # A tibble: 11 x 4
#>   tenor date      discount zero_rate
#>   <chr> <date>      <dbl>    <dbl>
#> 1 1W    2008-09-25  0.99916  0.044079
#> 2 1M    2008-10-18  0.99733  0.032579
#> 3 3M    2008-12-18  0.99197  0.032440
#> 4 6M    2009-03-18  0.98327  0.034085
#> 5 1Y    2009-09-18  0.96714  0.033440
#> 6 2Y    2010-09-18  0.94376  0.028954
#> 7 3Y    2011-09-18  0.90902  0.031805
#> 8 5Y    2013-09-18  0.83739  0.035499
#> 9 7Y    2015-09-18  0.76276  0.038692
#> 10 10Y   2018-09-18  0.66310  0.041086
#> 11 15Y   2023-09-18  0.52138  0.043421

```

```

comparison_tenors <- c(
  "1M", "3M", "6M", "1Y",
  "2Y", "3Y", "5Y", "7Y",
  "10Y", "15Y"
)

tbl_bootstrap_comparison <- dplyr::bind_rows(
  GaussQuant::curve_tbl_GQH(
    curve = bond_curve,
    tenors = comparison_tenors
  ) |>
  dplyr::mutate(
    curve = "Deposit + bond"
  ),
  GaussQuant::curve_tbl_GQH(
    curve = swap_curve,
    tenors = comparison_tenors
  ) |>

```

```

    dplyr::mutate(
      curve = "Deposit + swap"
    )
)

GaussQuant::show_tbl_GQH(
  tbl_bootstrap_comparison,
  "Bootstrap curve comparison",
  n = 30
)

#> # A tibble: 20 x 5
#>   tenor date      discount zero_rate curve
#>   <chr> <date>      <dbl>      <dbl> <chr>
#> 1 1M    2008-10-18  0.99921 0.0096148 Deposit + bond
#> 2 3M    2008-12-18  0.99761 0.0096148 Deposit + bond
#> 3 6M    2009-03-18  0.99286 0.014471  Deposit + bond
#> 4 1Y    2009-09-18  0.98097 0.019229  Deposit + bond
#> 5 2Y    2010-09-18  0.95732 0.021818  Deposit + bond
#> 6 3Y    2011-09-18  0.92904 0.024541  Deposit + bond
#> 7 5Y    2013-09-18  0.85971 0.030237  Deposit + bond
#> 8 7Y    2015-09-18  0.78217 0.035102  Deposit + bond
#> 9 10Y   2018-09-18  0.67867 0.038764  Deposit + bond
#> 10 15Y  2023-09-18  0.53179 0.042103  Deposit + bond
#> 11 1M    2008-10-18  0.99733 0.032579  Deposit + swap
#> 12 3M    2008-12-18  0.99197 0.032440  Deposit + swap
#> 13 6M    2009-03-18  0.98327 0.034085  Deposit + swap
#> 14 1Y    2009-09-18  0.96714 0.033440  Deposit + swap
#> 15 2Y    2010-09-18  0.94376 0.028954  Deposit + swap
#> 16 3Y    2011-09-18  0.90902 0.031805  Deposit + swap
#> 17 5Y    2013-09-18  0.83739 0.035499  Deposit + swap
#> 18 7Y    2015-09-18  0.76276 0.038692  Deposit + swap
#> 19 10Y   2018-09-18  0.66310 0.041086  Deposit + swap
#> 20 15Y  2023-09-18  0.52138 0.043421  Deposit + swap

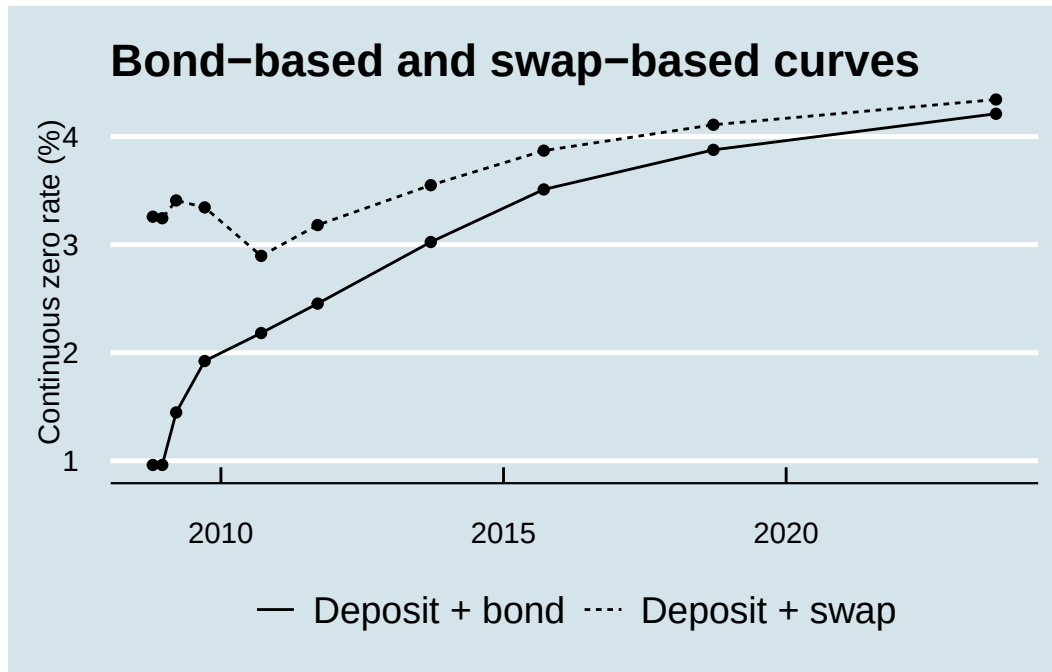
bootstrap_zero_plot <- tbl_bootstrap_comparison |>
  ggplot2::ggplot(
    ggplot2::aes(
      x = date,
      y = 100 * zero_rate,
      linetype = curve
    )
  ) +
  ggplot2::geom_line() +
  ggplot2::geom_point() +
  ggplot2::labs(

```

```

title = "Bond-based and swap-based curves",
x = NULL,
y = "Continuous zero rate (%)",
linetype = NULL
)
show_plot_LFL(bootstrap_zero_plot)

```



債券カーブとスワップカーブは、異なる市場商品から構築されるため一致するとは限らない。両者の差には、市場の流動性、商品固有の信用・担保条件、日数計算規則および観測時点の違いが含まれる。

FRA は、将来の一定期間に適用される金利を直接観測する商品として、預金とスワップの間を接続するために用いられる。本章の実装では預金とスワップを用いるが、FRA を追加する場合も、各市場商品の理論価格が市場価格と一致するように同じブートストラップの枠組みへ組み込む。

II-2.5 カーブ操作と検証

Implied Term Structure は、既存カーブが示す将来の割引関係を保ったまま、参照日を将来へ移した期間構造である。元のカーブの参照日を t_0 、新しい参照日を t_1 、将来日を t_2 とすると、Implied Term Structure の割引係数は概念的に次式で表される。

$$D_{t_1}(t_2) = \frac{D_{t_0}(t_2)}{D_{t_0}(t_1)}$$


```

implied_reference_date <- GaussQuant::date_GQL(
  "2009-09-18"
)

implied_constructor <- get0(
  "ImpliedTermStructure",
  envir = asNamespace("QuantLib"),
  inherits = FALSE
)

implied_curve <- if (is.null(implied_constructor)) {
  NULL
} else {
  tryCatch(
    implied_constructor(
      swap_curve_handle,
      implied_reference_date
    ),
    error = function(e) NULL
  )
}

tbl_implied_curve <- if (is.null(implied_curve)) {
  tibble::tibble(
    status = "ImpliedTermStructure is not available in the installed QuantLib
    ↪ binding."
  )
} else {
  GaussQuant::curve_tbl_GQH(
    curve = implied_curve,
    tenors = c(
      "1M", "3M", "6M", "1Y",
      "2Y", "3Y", "5Y", "10Y"
    )
  )
}

GaussQuant::show_tbl_GQH(
  tbl_implied_curve,
  "Implied term structure",
  n = 20
)

#> # A tibble: 8 x 4
#>   tenor date          discount zero_rate
#>   <chr> <date>          <dbl>      <dbl>

```

```
#> 1 1M      2009-10-18  0.99799  0.024471
#> 2 3M      2009-12-18  0.99392  0.024471
#> 3 6M      2010-03-18  0.98794  0.024471
#> 4 1Y      2010-09-18  0.97583  0.024471
#> 5 2Y      2011-09-18  0.93990  0.030989
#> 6 3Y      2012-09-18  0.90209  0.034338
#> 7 5Y      2014-09-18  0.82636  0.038145
#> 8 10Y     2019-09-18  0.65344  0.042551
```

Implied Term Structure は、将来時点から見た割引係数を現在のカーブから機械的に導くものであり、将来の市場カーブを予測したものではない。評価日を将来へ進めたシナリオで市場リスクを評価する場合には、カーブの単純な時間経過と市場金利の変動を区別する必要がある。

金利リスク分析では、基準カーブに一定のスプレッドを加えたカーブを用いる。すべての満期へ同じスプレッドを加える変化を平行シフトという。

ここでは、預金・スワップカーブへ 50 ベーシスポイントを加え、ゼロ金利と割引係数の変化を確認する。

```
parallel_spread <- 50 / 10000

spreaded_curve_bundle <- GaussQuant::make_zero_spreaded_curve_GQL(
  base_curve_handle = swap_curve_handle,
  spread_rate = parallel_spread,
  day_counter = QuantLib::Actual365Fixed(),
  extrapolate = TRUE
)

spreaded_curve <- spreaded_curve_bundle$curve

tbl_parallel_shift <- dplyr::bind_rows(
  GaussQuant::curve_tbl_GQH(
    curve = swap_curve,
    tenors = comparison_tenors
  ) |>
  dplyr::mutate(
    curve = "Base curve"
  ),
  GaussQuant::curve_tbl_GQH(
    curve = spreaded_curve,
    tenors = comparison_tenors
  ) |>
  dplyr::mutate(
    curve = "Base curve + 50bp"
```

```

    )
  )

tbl_parallel_shift_wide <- tbl_parallel_shift |>
  dplyr::select(
    tenor,
    date,
    curve,
    discount,
    zero_rate
  ) |>
  tidyr::pivot_wider(
    names_from = curve,
    values_from = c(
      discount,
      zero_rate
    )
  ) |>
  dplyr::mutate(
    observed_zero_shift_bp =
      10000 * (
        .data$`zero_rate_Base curve + 50bp` -
        .data$`zero_rate_Base curve`
      )
  )

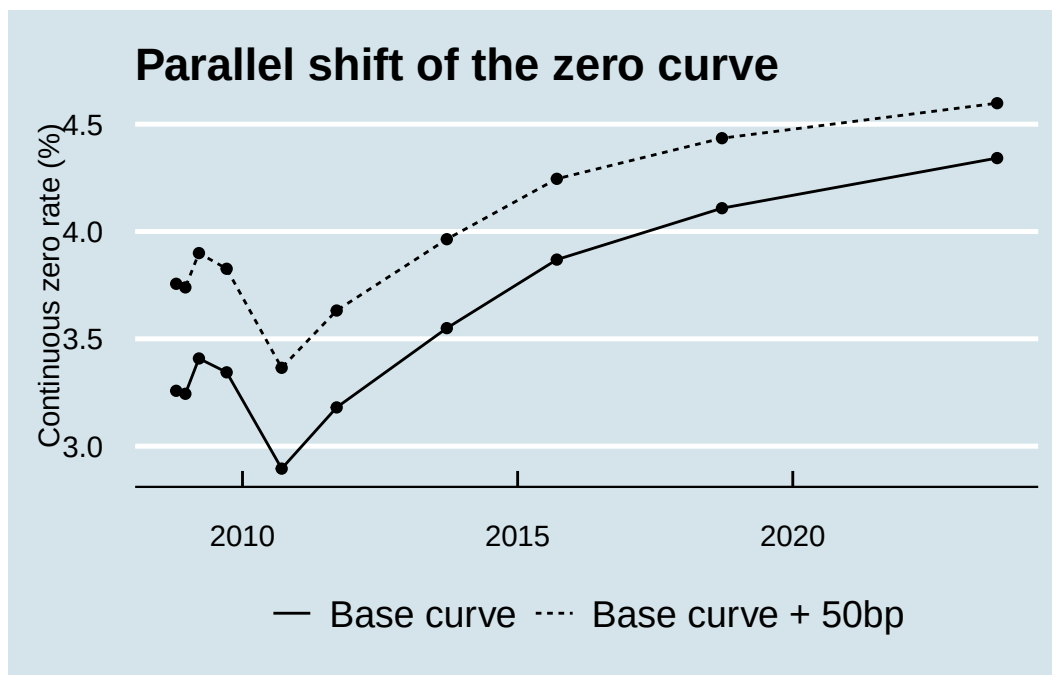
GaussQuant::show_tbl_GQH(
  tbl_parallel_shift_wide,
  "Parallel curve shift",
  n = 30
)

#> # A tibble: 10 x 7
#>   tenor date      `discount_Base curve` `discount_Base curve + 50bp`
#>   <chr> <date>                <dbl>                <dbl>
#> 1 1M    2008-10-18                0.99733                0.99693
#> 2 3M    2008-12-18                0.99197                0.99074
#> 3 6M    2009-03-18                0.98327                0.98088
#> 4 1Y    2009-09-18                0.96714                0.96249
#> 5 2Y    2010-09-18                0.94376                0.93494
#> 6 3Y    2011-09-18                0.90902                0.89679
#> 7 5Y    2013-09-18                0.83739                0.82022
#> 8 7Y    2015-09-18                0.76276                0.74293
#> 9 10Y   2018-09-18                0.66310                0.64182
#> 10 15Y  2023-09-18                0.52138                0.50176
#> # i 3 more variables: `zero_rate_Base curve` <dbl>,

```

```
#> # `zero_rate_Base curve + 50bp` <dbl>, observed_zero_shift_bp <dbl>
```

```
parallel_shift_plot <- tbl_parallel_shift |>
  ggplot2::ggplot(
    ggplot2::aes(
      x = date,
      y = 100 * zero_rate,
      linetype = curve
    )
  ) +
  ggplot2::geom_line() +
  ggplot2::geom_point() +
  ggplot2::labs(
    title = "Parallel shift of the zero curve",
    x = NULL,
    y = "Continuous zero rate (%)",
    linetype = NULL
  )
show_plot_LFL(parallel_shift_plot)
```



正の平行シフトでは割引率が上昇し、将来キャッシュフローの割引係数は低下する。そのため、通常の固定キャッシュフロー商品の現在価値も低下する。

実際の市場変動は完全な平行移動とは限らない。短期だけが動く変化、長期だけが動く変化、曲線の傾きが変化するスティーピングまたはフラットニングも生じる。年限別の Key Rate Shift では、特定年限のノードを動かし、周辺年限へ補間しながら再評価する。これらは平行シフトより細かなリスク分解である。

構築したカーブについて、少なくとも次の点を確認する必要がある。

- 参照日と最終日が想定どおりであること
- 割引係数が正であること
- 欠損値や計算不能な区間がないこと
- ゼロ金利とフォワード金利に不自然な跳びがないこと
- 入力商品の理論価格または理論レートが市場値へ再現されること

まず、例示的なゼロ金利ノードが二つのカーブでどの程度再現されるかを確認する。

```
tbl_node_recovery <- tbl_zero_nodes |>
  dplyr::mutate(
    ql_date = purrr::map(
      as.character(date),
      GaussQuant::date_GQL
    ),
    time = as.numeric(
      date -
        as.Date(evaluation_date_chr)
    ) / 365,
    zero_curve_discount = purrr::map_dbl(
      ql_date,
      ~ GaussQuant::curve_discount_safe_GQL(
        zero_curve,
        .x
      )
    ),
    discount_curve_discount = purrr::map_dbl(
      ql_date,
      ~ GaussQuant::curve_discount_safe_GQL(
        discount_curve,
        .x
      )
    ),
    zero_curve_implied_rate = dplyr::if_else(
      time > 0,
      -log(zero_curve_discount) /
        time,
      zero_rate
    ),
    discount_curve_implied_rate = dplyr::if_else(
      time > 0,
      -log(discount_curve_discount) /
        time,
      zero_rate
    )
  )
```

```

),
zero_curve_error =
  zero_curve_implied_rate -
  zero_rate,
discount_curve_error =
  discount_curve_implied_rate -
  zero_rate
) |>
dplyr::select(
  tenor,
  date,
  zero_rate,
  zero_curve_implied_rate,
  discount_curve_implied_rate,
  zero_curve_error,
  discount_curve_error
)

GaussQuant::show_tbl_GQH(
  tbl_node_recovery,
  "Node recovery check",
  n = 20
)

#> # A tibble: 9 x 7
#>   tenor date      zero_rate zero_curve_implied_rate discount_curve_implied_rate
#>   <chr> <date>      <dbl>          <dbl>          <dbl>
#> 1 0D    2022-08-19    0.0005          0.0005          0.0005
#> 2 1M    2022-09-19    0.0006          0.00060000     0.00060000
#> 3 3M    2022-11-21    0.0008          0.00080000     0.00080000
#> 4 6M    2023-02-20    0.0011          0.0011000     0.0011000
#> 5 1Y    2023-08-21    0.0018          0.0018000     0.0018000
#> 6 2Y    2024-08-19    0.0032          0.0032000     0.0032000
#> 7 3Y    2025-08-19    0.0048          0.0048000     0.0048000
#> 8 5Y    2027-08-19    0.0075          0.0075         0.0075
#> 9 10Y   2032-08-19    0.0115          0.0115         0.0115
#> # i 2 more variables: zero_curve_error <dbl>, discount_curve_error <dbl>

```

続いて、各カーブを密なグリッドへ展開し、割引係数の範囲と欠損値を確認する。

```

tbl_curve_validation <- dplyr::bind_rows(
  GaussQuant::curve_grid_tbl_GQL(
    zero_curve,
    n = 200L,
    extrapolate = TRUE
  ) |>

```

```
dplyr::mutate(
  curve = "ZeroCurve"
),
GaussQuant::curve_grid_tbl_GQL(
  discount_curve,
  n = 200L,
  extrapolate = TRUE
) |>
dplyr::mutate(
  curve = "DiscountCurve"
),
GaussQuant::curve_grid_tbl_GQL(
  bond_curve,
  n = 200L,
  extrapolate = TRUE
) |>
dplyr::mutate(
  curve = "Deposit + bond"
),
GaussQuant::curve_grid_tbl_GQL(
  swap_curve,
  n = 200L,
  extrapolate = TRUE
) |>
dplyr::mutate(
  curve = "Deposit + swap"
),
GaussQuant::curve_grid_tbl_GQL(
  spreaded_curve,
  n = 200L,
  extrapolate = TRUE
) |>
dplyr::mutate(
  curve = "Deposit + swap + 50bp"
)
) |>
dplyr::group_by(curve) |>
dplyr::summarise(
  first_date = min(
    curve_date,
    na.rm = TRUE
  ),
  last_date = max(
    curve_date,
    na.rm = TRUE
  )
)
```

```

    ),
    min_discount = min(
      discount_factor,
      na.rm = TRUE
    ),
    max_discount = max(
      discount_factor,
      na.rm = TRUE
    ),
    nonpositive_discount_count = sum(
      discount_factor <= 0,
      na.rm = TRUE
    ),
    missing_discount_count = sum(
      is.na(discount_factor)
    ),
    missing_zero_rate_count = sum(
      is.na(zero_rate)
    ),
    .groups = "drop"
  )

GaussQuant::show_tbl_GQH(
  tbl_curve_validation,
  "Curve validation summary",
  n = 20
)

#> # A tibble: 5 x 8
#>   curve   first_date last_date  min_discount max_discount nonpositive_discount~1
#>   <chr>   <date>     <date>      <dbl>        <dbl>             <int>
#> 1 Deposi~ 2008-09-18 2038-05-15    0.26019         1               0
#> 2 Deposi~ 2008-09-18 2023-09-18    0.52138         1               0
#> 3 Deposi~ 2008-09-18 2023-09-18    0.50176         1               0
#> 4 Discou~ 2022-08-19 2032-08-19    0.89128         1               0
#> 5 ZeroCu~ 2022-08-19 2032-08-19    0.89128         1               0
#> # i abbreviated name: 1: nonpositive_discount_count
#> # i 2 more variables: missing_discount_count <int>,
#> #   missing_zero_rate_count <int>

```

割引係数が正であることは、裁定のない期間構造を確認するための基本条件である。ただし、正の割引係数だけでは十分ではない。入力商品の再評価、補間区間のフォワード金利、外挿方法、日数計算規則および参照日の設定も確認する必要がある。

II-2.6 日本国債先物と CTD

日本の長期国債先物は、実在する特定銘柄ではなく、クーポン 6%、償還期限 10 年の標準物を取引対象とする。満期時には、取引所が定めた受渡適格銘柄の中から売方が実際に受け渡す銘柄を選択する。各銘柄ではクーポンと残存期間が異なるため、標準物との価値の差を Conversion Factor (CF) で調整する。

受渡価格の基本部分は次式で表される。

$$\text{Invoice Price} = F \times CF$$

ここで F は国債先物価格、 CF は Conversion Factor である。実際の受渡金額では、さらに受渡日の経過利子加わる。

CTD (Cheapest-to-Deliver) は、単に現物価格が最も安い銘柄ではない。現物価格、Conversion Factor、保有期間中のクーポン、経過利子、資金調達費用を合わせて比較する必要がある。本節では、2026 年 9 月 10 日を評価日、2026 年 12 月 20 日を受渡日として、長期国債先物 2026 年 12 月限の受渡適格銘柄を比較する。

以下では 373 回債から 383 回債までを比較する。BB 価格は本章の計算確認用に置いた 2026 年 9 月 10 日時点の値であり、最終稿では日本相互証券の確認値へ差し替える。

```
valuation_date_chr <- "2026-09-10"
delivery_date_chr <- "2026-12-20"
futures_price <- 125.71
funding_rate <- 0.0123

GaussQuant::set_eval_date_GQL(valuation_date_chr)

dc_jgb <- QuantLib::Actual365Fixed()
calendar_jp <- QuantLib::Japan()

cmpd_compounded <- "Compounded"
freq_semiannual <- "Semiannual"

tbl_jgb_deliverable <- tibble::tribble(
  ~issue, ~issue_date, ~coupon_pct, ~maturity_date, ~conversion_factor, ~bb_price,
  373L, "2023-12-20", 0.6, "2033-12-20", 0.695006, 87.620,
  374L, "2024-03-20", 0.8, "2034-03-20", 0.697881, 88.333,
  375L, "2024-06-20", 1.1, "2034-06-20", 0.707520, 89.828,
```

```

376L, "2024-09-20", 0.9, "2034-09-20", 0.687561, 87.828,
377L, "2024-12-20", 1.2, "2034-12-20", 0.698533, 89.484,
378L, "2025-03-20", 1.4, "2035-03-20", 0.704059, 90.486,
379L, "2025-06-20", 1.5, "2035-06-20", 0.703762, 90.774,
380L, "2025-09-20", 1.7, "2035-09-20", 0.710535, 91.904,
381L, "2025-12-20", 2.1, "2035-12-20", 0.731806, 94.703,
382L, "2026-03-20", 2.4, "2036-03-20", 0.747221, 96.808,
383L, "2026-06-20", 2.7, "2036-06-20", 0.763657, 98.966
)

GaussQuant::show_tbl_GQH(
  tbl_jgb_deliverable,
  "JGB futures deliverable basket",
  n = 20
)

#> # A tibble: 11 x 6
#>   issue issue_date coupon_pct maturity_date conversion_factor bb_price
#>   <int> <chr>         <dbl> <chr>                <dbl>    <dbl>
#> 1   373 2023-12-20         0.6 2033-12-20            0.69501  87.62
#> 2   374 2024-03-20         0.8 2034-03-20            0.69788  88.333
#> 3   375 2024-06-20         1.1 2034-06-20            0.70752  89.828
#> 4   376 2024-09-20         0.9 2034-09-20            0.68756  87.828
#> 5   377 2024-12-20         1.2 2034-12-20            0.69853  89.484
#> 6   378 2025-03-20         1.4 2035-03-20            0.70406  90.486
#> 7   379 2025-06-20         1.5 2035-06-20            0.70376  90.774
#> 8   380 2025-09-20         1.7 2035-09-20            0.71054  91.904
#> 9   381 2025-12-20         2.1 2035-12-20            0.73181  94.703
#> 10  382 2026-03-20         2.4 2036-03-20            0.74722  96.808
#> 11  383 2026-06-20         2.7 2036-06-20            0.76366  98.966

```

各受渡適格銘柄を GaussQuant の `fixed_rate_bond_GQL()` で構築する。BB 価格から半年複利利回りを逆算し、その利回りを用いて Modified Duration を計算する。本章では以後、単に Duration と記す場合も Modified Duration を意味する。

```

tbl_jgb_market <- tbl_jgb_deliverable |>
  dplyr::mutate(
    bond = purrr::pmap(
      list(issue_date, maturity_date, coupon_pct),
      \(issue_date, maturity_date, coupon_pct) {
        GaussQuant::fixed_rate_bond_GQL(
          issue_date = issue_date,
          maturity_date = maturity_date,
          coupon_rate = coupon_pct / 100,
          face_amount = 100,

```

```

    settlement_days = 0L,
    frequency = GaussQuant::period_months_GQL(6),
    calendar = calendar_jp,
    accrual_day_counter = dc_jgb,
    payment_convention = "Following",
    schedule_convention = "Unadjusted",
    maturity_convention = "Unadjusted"
  )
}
),
compound_yield = purrr::map2_dbl(
  bond,
  bb_price,
  \(bond, clean_price) {
    GaussQuant::bond_yield_from_clean_price_GQL(
      bond = bond,
      clean_price = clean_price,
      day_counter = dc_jgb,
      compounding = cmpd_compounded,
      frequency = freq_semiannual,
      settlement_date = valuation_date_chr
    )
  }
),
duration = purrr::map2_dbl(
  bond,
  compound_yield,
  \(bond, ytm) {
    GaussQuant::bond_duration_GQL(
      bond = bond,
      ytm = ytm,
      day_counter = dc_jgb,
      compounding = cmpd_compounded,
      frequency = freq_semiannual
    )
  }
)
)

```

```

tbl_jgb_market_display <- tbl_jgb_market |>
  dplyr::transmute(
    issue,
    coupon_pct,
    maturity_date,

```

```

    bb_price,
    compound_yield_pct = 100 * compound_yield,
    duration,
    conversion_factor
)

GaussQuant::show_tbl_GQH(
  tbl_jgb_market_display,
  "JGB compound yield and modified duration",
  n = 20
)

#> # A tibble: 11 x 7
#>   issue coupon_pct maturity_date bb_price compound_yield_pct duration
#>   <int>      <dbl> <chr>          <dbl>          <dbl>      <dbl>
#> 1   373        0.6 2033-12-20      87.62          2.4677      7.0261
#> 2   374        0.8 2034-03-20      88.333         2.5094      7.1857
#> 3   375        1.1 2034-06-20      89.828         2.5494      7.3436
#> 4   376        0.9 2034-09-20      87.828         2.5879      7.6087
#> 5   377        1.2 2034-12-20      89.484         2.6206      7.7566
#> 6   378        1.4 2035-03-20      90.486         2.6537      7.8760
#> 7   379        1.5 2035-06-20      90.774         2.6863      8.0886
#> 8   380        1.7 2035-09-20      91.904         2.7166      8.1938
#> 9   381        2.1 2035-12-20      94.703         2.7502      8.2991
#> 10  382        2.4 2036-03-20      96.808         2.7826      8.3440
#> 11  383        2.7 2036-06-20      98.966         2.8213      8.4848
#> # i 1 more variable: conversion_factor <dbl>

```

クーポンが高いほど、同一償還であればキャッシュフローを早く回収するため Duration は短くなる。一方、償還が遠いほど Duration は長くなる。実際の受渡適格銘柄では、この二つの効果が同時に働く。

Gross Basis は、現物クリーン価格と CF で調整した先物価格との差である。

$$\text{Gross Basis}_i = P_i - F \times CF_i$$

現物債を評価日から受渡日まで保有すると、クーポン収入と経過利子の変化を受け取る一方、現物購入資金には Funding Cost が発生する。本章では短期資金調達金利を 1.23% とし、単純化した Cash-and-Carry で比較する。

$$\text{Carry} = \text{Coupon Income} + (AI_T - AI_0) - \text{Funding Cost}$$

$$\text{Net Basis} = \text{Gross Basis} - \text{Carry}$$

```

valuation_date <- as.Date(valuation_date_chr)
delivery_date <- as.Date(delivery_date_chr)
carry_days <- as.numeric(delivery_date - valuation_date)
carry_year_fraction <- carry_days / 365

tbl_jgb_current <- tbl_jgb_market |>
  dplyr::mutate(
    accrued_start = purrr::map_dbl(
      bond,
      \(x) x$accruedAmount(GaussQuant::date_GQL(valuation_date_chr))
    ),
    accrued_end = purrr::map_dbl(
      bond,
      \(x) x$accruedAmount(GaussQuant::date_GQL(delivery_date_chr))
    ),
    coupon_income = purrr::map_dbl(
      bond,
      \(x) {
        GaussQuant::bond_cashflow_table_GQL(x) |>
          dplyr::mutate(date = as.Date(date)) |>
          dplyr::filter(date > valuation_date, date <= delivery_date) |>
          dplyr::summarise(value = sum(amount, na.rm = TRUE)) |>
          dplyr::pull(value)
      }
    ),
    dirty_price = bb_price + accrued_start,
    futures_invoice = futures_price * conversion_factor,
    gross_basis = bb_price - futures_invoice,
    funding_cost = dirty_price * funding_rate * carry_year_fraction,
    carry = coupon_income + accrued_end - accrued_start - funding_cost,
    net_basis = gross_basis - carry,
    implied_repo = (
      futures_invoice + accrued_end + coupon_income - dirty_price
    ) / dirty_price / carry_year_fraction
  ) |>
  dplyr::mutate(
    ctd = net_basis == min(net_basis, na.rm = TRUE)
  )

tbl_jgb_current_display <- tbl_jgb_current |>

```

```
dplyr::transmute(
  issue,
  coupon_pct,
  maturity_date,
  bb_price,
  compound_yield_pct = 100 * compound_yield,
  duration,
  conversion_factor,
  gross_basis,
  net_basis,
  implied_repo_pct = 100 * implied_repo,
  ctd
)

GaussQuant::show_tbl_GQH(
  tbl_jgb_current_display,
  "JGB basis, implied repo and CTD",
  n = 20
)

#> # A tibble: 11 x 11
#>   issue coupon_pct maturity_date bb_price compound_yield_pct duration
#>   <int>      <dbl> <chr>          <dbl>          <dbl>      <dbl>
#> 1   373        0.6 2033-12-20      87.62          2.4677      7.0261
#> 2   374        0.8 2034-03-20      88.333         2.5094      7.1857
#> 3   375        1.1 2034-06-20      89.828         2.5494      7.3436
#> 4   376        0.9 2034-09-20      87.828         2.5879      7.6087
#> 5   377        1.2 2034-12-20      89.484         2.6206      7.7566
#> 6   378        1.4 2035-03-20      90.486         2.6537      7.8760
#> 7   379        1.5 2035-06-20      90.774         2.6863      8.0886
#> 8   380        1.7 2035-09-20      91.904         2.7166      8.1938
#> 9   381        2.1 2035-12-20      94.703         2.7502      8.2991
#> 10  382        2.4 2036-03-20      96.808         2.7826      8.3440
#> 11  383        2.7 2036-06-20      98.966         2.8213      8.4848
#> # i 5 more variables: conversion_factor <dbl>, gross_basis <dbl>,
#> #   net_basis <dbl>, implied_repo_pct <dbl>, ctd <lgl>
```

Net Basis が最小となる銘柄は、同じ前提で比較したときに売方が最も安く受け渡せる銘柄である。Implied Repo Rate で比較すれば、最も高い収益率を与える銘柄が同じ CTD となることを確認できる。

ここで重要なのは、CTD が単純に「価格が低い銘柄」や「Duration が短い銘柄」と定義されているわけではないことである。Conversion Factor は 6% 標準物を基準にしているため、市場金利の水準とイールドカーブの形状が変わると、受渡適格銘柄間の相対価値も変化する。

II-2.7 イールドカーブ変動と最割安銘柄影響

長期国債先物の標準物は、クーポン 6%、償還期限 10 年である。市場金利が 6% を下回り、かつイールドカーブが右上がりである場合、短い Duration 側の銘柄が相対的に受渡しやすくなる傾向がある。

一方、標準物の 6% を上回る高金利環境で、短期金利が長期金利を上回る逆イールドを作ると、相対価値は反対方向へ動き得る。ここでは、その CTD Switching を同じ受渡適格銘柄で確認する。

シナリオでは、長期金利を 8%、短期 Funding Rate を 9% とする。

$$9\%_{\text{short}} > 8\%_{\text{long}} > 6\%_{\text{standard coupon}}$$

現在の順イールドから、標準物 6% を上回る逆イールドへ変化するイメージを次の図に示す。

```
tbl_curve_scenario <- tibble::tribble(
  ~tenor_years, ~scenario, ~yield_pct,
  0.25, "Current market", 1.23,
  2.00, "Current market", 1.55,
  5.00, "Current market", 2.20,
  7.00, "Current market", 2.55,
  10.0, "Current market", 2.90,
  0.25, "Stress: inverted", 9.00,
  2.00, "Stress: inverted", 8.80,
  5.00, "Stress: inverted", 8.50,
  7.00, "Stress: inverted", 8.25,
  10.0, "Stress: inverted", 8.00
)

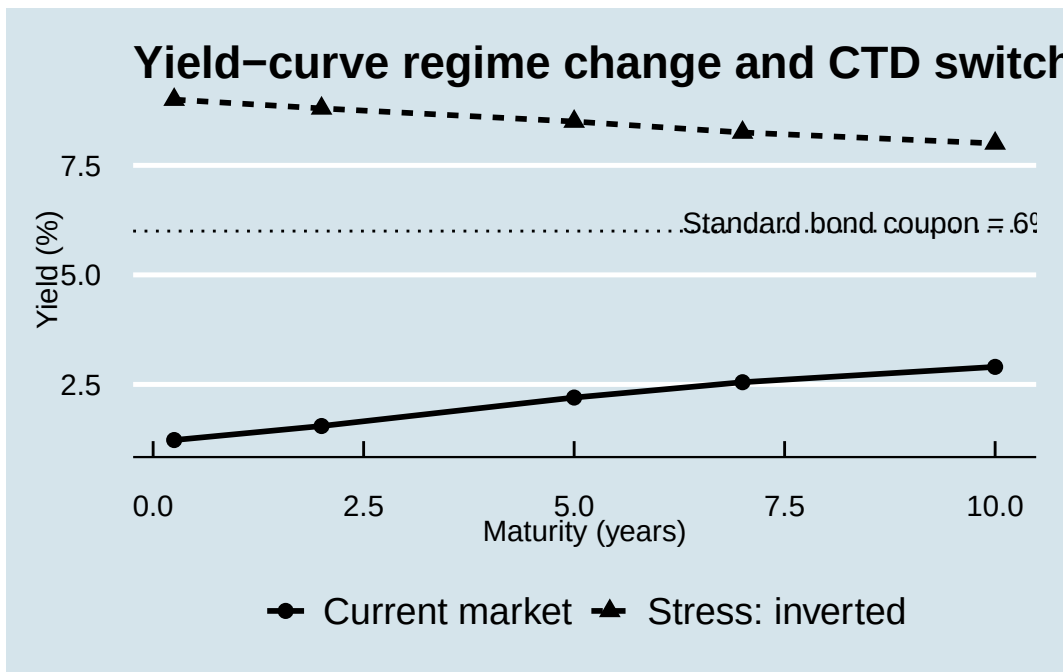
curve_switch_plot <- tbl_curve_scenario |>
  ggplot2::ggplot(
    ggplot2::aes(
      x = tenor_years,
      y = yield_pct,
      linetype = scenario,
      shape = scenario
    )
  ) +
  ggplot2::geom_line(linewidth = 1) +
  ggplot2::geom_point(size = 2.5) +
  ggplot2::geom_hline(
```

```

    yintercept = 6,
    linetype = "dotted"
  ) +
  ggplot2::annotate(
    "text",
    x = 8.5,
    y = 6.2,
    label = "Standard bond coupon = 6%"
  ) +
  ggplot2::labs(
    title = "Yield-curve regime change and CTD switching",
    x = "Maturity (years)",
    y = "Yield (%)",
    linetype = NULL,
    shape = NULL
  )

show_plot_LFL(curve_switch_plot)

```



先物価格を任意の X と置くのではなく、6% クーポン・10 年の標準物を 8% 複利で評価した価格をシナリオの先物価格とする。これにより、現物債と先物を同じ金利環境で比較できる。

```

stress_yield <- 0.08
stress_funding_rate <- 0.09

standard_bond <- GaussQuant::fixed_rate_bond_GQL(
  issue_date = valuation_date_chr,
  maturity_date = "2036-09-10",
  coupon_rate = 0.06,

```



```

    face_amount = 100,
    settlement_days = 0L,
    frequency = GaussQuant::period_months_GQL(6),
    calendar = calendar_jp,
    accrual_day_counter = dc_jgb,
    payment_convention = "Following",
    schedule_convention = "Unadjusted",
    maturity_convention = "Unadjusted"
)

stress_futures_price <- GaussQuant::bond_clean_price_from_yield_safe_GQL(
  bond = standard_bond,
  yield_rate = stress_yield,
  day_counter = dc_jgb,
  compounding = cmpd_compounded,
  frequency = freq_semiannual,
  settlement_date = GaussQuant::date_GQL(valuation_date_chr)
)

tbl_standard_bond <- tibble::tribble(
  ~item, ~value,
  "standard_coupon_pct", 6,
  "standard_maturity_years", 10,
  "compound_yield_pct", 100 * stress_yield,
  "futures_price", stress_futures_price,
  "short_funding_rate_pct", 100 * stress_funding_rate
)

GaussQuant::show_tbl_GQH(
  tbl_standard_bond,
  "Stress scenario standard bond",
  n = 20
)

#> # A tibble: 5 x 2
#>   item                value
#>   <chr>              <dbl>
#> 1 standard_coupon_pct      6
#> 2 standard_maturity_years 10
#> 3 compound_yield_pct       8
#> 4 futures_price          86.398
#> 5 short_funding_rate_pct    9

```

6% クーポンの 10 年標準物を 8% 複利で評価すると、価格はおおむね 86 円台となる。単純な近似 $(6 - 8) \times 10$ でも 100 円から大きく低下する方向を確認できるが、実際の価格は各クーポン

と元本を割り引いて求める。

同じ 373 回債から 383 回債について、複利利回りを一律 8% へ変更し、GaussQuant で時価と Modified Duration を再計算する。Conversion Factor は変更しない。

```
tbl_jgb_stress <- tbl_jgb_market |>
  dplyr::mutate(
    stress_price = purrr::map_dbl(
      bond,
      \(x) {
        GaussQuant::bond_clean_price_from_yield_safe_GQL(
          bond = x,
          yield_rate = stress_yield,
          day_counter = dc_jgb,
          compounding = compd_compounded,
          frequency = freq_semiannual,
          settlement_date = GaussQuant::date_GQL(valuation_date_chr)
        )
      }
    ),
    stress_duration = purrr::map_dbl(
      bond,
      \(x) {
        GaussQuant::bond_duration_GQL(
          bond = x,
          ytm = stress_yield,
          day_counter = dc_jgb,
          compounding = compd_compounded,
          frequency = freq_semiannual
        )
      }
    ),
    accrued_start = purrr::map_dbl(
      bond,
      \(x) x$accruedAmount(GaussQuant::date_GQL(valuation_date_chr))
    ),
    accrued_end = purrr::map_dbl(
      bond,
      \(x) x$accruedAmount(GaussQuant::date_GQL(delivery_date_chr))
    ),
    coupon_income = purrr::map_dbl(
      bond,
      \(x) {
        GaussQuant::bond_cashflow_table_GQL(x) |>
          dplyr::mutate(date = as.Date(date)) |>
          dplyr::filter(date > valuation_date, date <= delivery_date) |>
```

```

      dplyr::summarise(value = sum(amount, na.rm = TRUE)) |>
      dplyr::pull(value)
    }
  ),
  stress_dirty_price = stress_price + accrued_start,
  futures_invoice = stress_futures_price * conversion_factor,
  gross_basis = stress_price - futures_invoice,
  funding_cost = stress_dirty_price * stress_funding_rate * carry_year_fraction,
  carry = coupon_income + accrued_end - accrued_start - funding_cost,
  net_basis = gross_basis - carry,
  implied_repo = (
    futures_invoice + accrued_end + coupon_income - stress_dirty_price
  ) / stress_dirty_price / carry_year_fraction
) |>
dplyr::mutate(
  ctd = net_basis == min(net_basis, na.rm = TRUE)
)

tbl_jgb_stress_display <- tbl_jgb_stress |>
dplyr::transmute(
  issue,
  coupon_pct,
  maturity_date,
  compound_yield_pct = 100 * stress_yield,
  price = stress_price,
  duration = stress_duration,
  conversion_factor,
  gross_basis,
  net_basis,
  implied_repo_pct = 100 * implied_repo,
  ctd
)

GaussQuant::show_tbl_GQH(
  tbl_jgb_stress_display,
  "JGB stress scenario: 8% long yield and 9% short funding",
  n = 20
)

#> # A tibble: 11 x 11
#>   issue coupon_pct maturity_date compound_yield_pct price duration
#>   <int>      <dbl> <chr>                <dbl>   <dbl>      <dbl>
#> 1   373        0.6 2033-12-20                8 59.745    6.7912
#> 2   374        0.8 2034-03-20                8 59.847    6.9200
#> 3   375        1.1 2034-06-20                8 60.595    7.0519

```

```
#> 4 376 0.9 2034-09-20 8 58.510 7.3094
#> 5 377 1.2 2034-12-20 8 59.385 7.4281
#> 6 378 1.4 2035-03-20 8 59.755 7.5019
#> 7 379 1.5 2035-06-20 8 59.548 7.7019
#> 8 380 1.7 2035-09-20 8 60.019 7.7536
#> 9 381 2.1 2035-12-20 8 61.852 7.8234
#> 10 382 2.4 2036-03-20 8 63.127 7.7996
#> 11 383 2.7 2036-06-20 8 64.495 7.9174
#> # i 5 more variables: conversion_factor <dbl>, gross_basis <dbl>,
#> # net_basis <dbl>, implied_repo_pct <dbl>, ctd <lgl>
```

二つの市場環境を比較する。

```
tbl_ctd_switch <- dplyr::bind_rows(
  tbl_jgb_current |>
    dplyr::filter(ctd) |>
    dplyr::transmute(
      scenario = "Current market",
      issue,
      coupon_pct,
      maturity_date,
      duration,
      gross_basis,
      net_basis,
      implied_repo_pct = 100 * implied_repo
    ),
  tbl_jgb_stress |>
    dplyr::filter(ctd) |>
    dplyr::transmute(
      scenario = "Stress: 9% short / 8% long",
      issue,
      coupon_pct,
      maturity_date,
      duration = stress_duration,
      gross_basis,
      net_basis,
      implied_repo_pct = 100 * implied_repo
    )
)

GaussQuant::show_tbl_GQH(
  tbl_ctd_switch,
  "CTD switching summary",
  n = 20
)
```

```
#> # A tibble: 2 x 8
#>   scenario      issue coupon_pct maturity_date duration gross_basis net_basis
#>   <chr>      <int>    <dbl> <chr>          <dbl>      <dbl>    <dbl>
#> 1 Current market    373      0.6 2033-12-20    7.0261    0.25080  0.38345
#> 2 Stress: 9% shor~  383      2.7 2036-06-20    7.9174   -1.4830 -0.60881
#> # i 1 more variable: implied_repo_pct <dbl>
```

現在のように標準物 6% より市場金利が低く、イールドカーブが右上がりである環境では、短い Duration 側が相対的に受渡しやすくなる。一方、長期金利 8%、短期金利 9% という逆イールドでは、標準物 6% を上回る金利水準とカーブの傾きが同時に反転し、長い Duration 側へ CTD が移る可能性がある。

重要なのは、CTD を「Duration が最小の銘柄」または「Duration が最大の銘柄」と機械的に決めるのではないことである。最終判定は Net Basis 最小、または Implied Repo Rate 最大で行う。Duration は、金利水準と Conversion Factor の関係から相対価値がどちら側へ偏りやすいかを理解するための重要な手掛かりである。

II-2.8 まとめ

本章では、財務省の日本国債複利利回りを `pivot_longer()` で long 形式へ変換し、年限方向のイールドカーブと時間方向の金利時系列を可視化した。離散的な観測点へ自然スプライン補間を適用することで、補間値は市場で直接観測された値ではなく、選択したアルゴリズムから推定された値であることを確認した。

続いて、割引係数、ゼロ金利およびフォワード金利を通じて金利期間構造を整理し、補間対象の違い、ブートストラップ、Implied Term Structure、カーブシフトおよび検証を扱った。とくにフォワード金利は、ゼロ金利では目立たない補間差を拡大して示すことがある。このため、カーブの妥当性は単一の表現だけで判断せず、入力商品の再現性と複数のカーブ表現を組み合わせる必要がある。

後半では、日本の長期国債先物を対象として、6% クーポン・10 年の標準物、Conversion Factor、Gross Basis、Carry、Net Basis、Implied Repo Rate および CTD の関係を確認した。現物債の価格から複利利回りを逆算し、GaussQuant を用いて Modified Duration を求めることで、クーポンと償還期限が金利感応度へ与える影響も同時に確認した。

さらに、現在の低金利・順イールド環境から、長期 8%、短期 9% の高金利・逆イールド環境へ市場条件を変化させ、同じ受渡適格銘柄を再評価した。Conversion Factor が固定されていても、市場金利の水準とイールドカーブの形状が変化すれば、Gross Basis、Net Basis および Implied Repo Rate の順位が変化し、CTD がスイッチし得る。

この結果は、国債先物の CTD が単なる銘柄属性ではなく、標準物の設計、市場のイールドカーブ、現物価格、資金調達コストおよび受渡条件の相対関係から決まることを示している。

第 II-3 章 OIS スワップとマルチカーブ評価

II-3.1 IBOR 改革と OIS の基礎

2007 年から 2009 年にかけての世界金融危機、とりわけ 2008 年 9 月のリーマン・ブラザーズ破綻は、金利デリバティブの評価方法を大きく変化させた。それ以前の市場では、LIBOR は信用リスクをほとんど含まない基準金利として扱われ、同一の LIBOR カーブから将来の変動金利を予測すると同時に、将来キャッシュフローの現在価値を計算するシングルカーブ評価が広く用いられていた。

しかし、LIBOR は銀行間の無担保資金調達金利である。したがって、LIBOR には将来の短期金利に対する市場予想だけでなく、借り手銀行の信用リスク、資金流動性リスクおよび借入期間に応じたプレミアムが含まれる。金融危機時に LIBOR と OIS 金利の差が急拡大したことにより、LIBOR を信用リスクのない割引金利として扱うことができないことが明らかになった。

この認識を背景として、デリバティブ市場では中央清算および担保契約の利用が拡大した。担保付取引では、担保に付される翌日物金利に近い OIS カーブを用いて将来キャッシュフローを割り引くことが経済的に整合的である。一方、LIBOR、TIBOR または EURIBORなどを参照する契約の将来変動金利は、それぞれの参照金利に対応するフォワードカーブから予測する必要がある。その結果、将来金利を予測するカーブと現在価値を求めるディスカウントカーブを分離するマルチカーブ評価が定着した。

ここで、「担保付取引への移行」と「翌日物指標の性質」は区別する必要がある。米ドルの SOFR は米国債レポ取引に基づく有担保翌日物金利であるのに対し、日本円の TONA は無担保コール市場の翌日物金利である。いずれも翌日物のリスク・フリー・レートに近い指標として用いられるが、基礎となる市場の担保性は同一ではない。

さらに、LIBOR の不正操作および虚偽報告が明らかになり、銀行の判断に大きく依存する金利指標の信頼性が損なわれた。金融安定理事会は 2014 年、既存 IBOR の強化と、実取引に基づく代替的なリスク・フリー・レートの育成を提言した。2017 年には、英国の金融行為規制機構

である FCA が、2021 年末以降も銀行へ LIBOR の呈示を求め続けることはないとの方針を示した。

代表性のある円、ポンド、ユーロおよびスイスフラン LIBOR の多くは 2021 年末に終了し、米ドル LIBOR の銀行パネルも 2023 年 6 月 30 日に終了した。経過措置として残された合成 LIBOR も 2024 年 9 月 30 日に最終公表され、すべての LIBOR 設定が恒久的に終了した。

LIBOR に代わる主要な指標として、日本円では TONA、米ドルでは SOFR が採用された。これらの翌日物金利を参照する OIS では、通常、計算期間中の日次金利を後決めで複利計算する。そのため、LIBOR のように期間開始時点で適用金利が確定する取引とは異なり、期間全体の金利と利息額が確定するのは期末近くとなる。日次金利の公表時刻、Fixing が利用可能になるまでの時間差、計算終了日から支払日までの事務処理期間は、新たなオペレーショナル・リスクとして管理する必要がある。

以上の IBOR 改革によって、金利デリバティブ評価は、単一の LIBOR カーブですべてを処理する方法から、翌日物金利を基礎とする OIS カーブを割引に用い、参照金利ごとに将来金利を予測する方法へ移行した。

本章では、この歴史的変化を踏まえ、日本円の TONA および米ドルの SOFR を参照する OIS を取り上げる。まず市場レートから OIS カーブを構築し、固定レグと翌日物変動レグを評価する。次に、変動レグを日次金利へ分解し、Fixing の確定前後で評価額がどのように変化するかを確認する。最後に、将来金利を予測するカーブとキャッシュフローを割り引くカーブを分離し、マルチカーブ評価の基本構造を示す。

ユーロ市場では、OIS の翌日物参照金利として EONIA (Euro Overnight Index Average) が長く用いられてきた。世界金融危機後、担保付デリバティブの評価に OIS カーブを用いる実務が定着すると、EONIA を基礎とする OIS カーブは、ユーロ建てデリバティブのディスカウントカーブとして重要な役割を担うようになった。

一方、LIBOR をはじめとする金利指標改革は、ユーロの翌日物金利にも及んだ。EONIA から、より広範な実取引に基づく翌日物金利である€ STR (Euro Short-Term Rate) への移行が進められ、現在のユーロ OIS 市場では€ STR が中心的な基準金利となっている。

したがって、本章では EONIA を「金融危機後に OIS discounting が定着した時代を理解するための歴史的ベンチマーク」、€ STR を「現在のユーロ OIS 市場を表す基準金利」として位置づける。後半のケーススタディでは 2012 年の EONIA 市場データを用いて当時の OIS カーブ構築を再現し、ここでは最新版 GaussQuant を用いて現在の€ STR カーブを確認する。

```
estr_benchmark <- GaussQuant::estr_curve_benchmark_GQL()

tbl_estr_curve <- estr_benchmark$validation$quote_repricing |>
```



```

dplyr::select(
  quote_id,
  market_quote,
  implied_quote,
  quote_error,
  pillar_date,
  discount_factor,
  zero_rate,
  inst_fwd_rate
) |>
dplyr::filter(!is.na(pillar_date)) |>
dplyr::arrange(pillar_date)

GaussQuant::show_tbl_GQH(
  tbl_estr_curve,
  "€ STR OIS curve validation",
  n = 20
)

#> # A tibble: 15 x 8
#>   quote_id market_quote implied_quote quote_error pillar_date discount_factor
#>   <chr>         <dbl>         <dbl>         <dbl> <date>         <dbl>
#> 1 ESTR-1M      0.0243      0.024300  6.4680e-13 2026-10-14      0.99771
#> 2 ESTR-3M      0.0251      0.025100 -7.5746e-13 2026-12-14      0.99343
#> 3 ESTR-6M      0.0269      0.026900 -3.4153e-14 2027-03-15      0.98632
#> 4 ESTR-1Y      0.0297      0.029700 -1.9065e-14 2027-09-14      0.97051
#> 5 ESTR-2Y      0.0313      0.031300 -9.6728e-15 2028-09-14      0.93905
#> 6 ESTR-3Y      0.0315      0.031500 -6.4393e-15 2029-09-14      0.90961
#> 7 ESTR-4Y      0.0316      0.031600 -4.9405e-15 2030-09-16      0.88095
#> 8 ESTR-5Y      0.0317      0.031700 -3.9899e-15 2031-09-15      0.85322
#> 9 ESTR-7Y      0.0321      0.032100  9.3266e-14 2033-09-14      0.79851
#> 10 ESTR-8Y     0.0324      0.032400  1.0443e-13 2034-09-14      0.77129
#> 11 ESTR-10Y    0.0329      0.032900 -7.8756e-15 2036-09-15      0.71881
#> 12 ESTR-15Y    0.034       0.034000  1.8215e-14 2041-09-16      0.59813
#> 13 ESTR-20Y    0.0344      0.034400 -3.1850e-15 2046-09-14      0.49979
#> 14 ESTR-30Y    0.0334      0.033400  3.9690e-15 2056-09-14      0.37102
#> 15 ESTR-50Y    0.0305      0.030500  4.1456e-14 2076-09-14      0.24265
#> # i 2 more variables: zero_rate <dbl>, inst_fwd_rate <dbl>

estr_curve_plot <- tbl_estr_curve |>
dplyr::filter(!is.na(zero_rate)) |>
ggplot2::ggplot(
  ggplot2::aes(
    x = pillar_date,
    y = 100 * zero_rate
  )
)

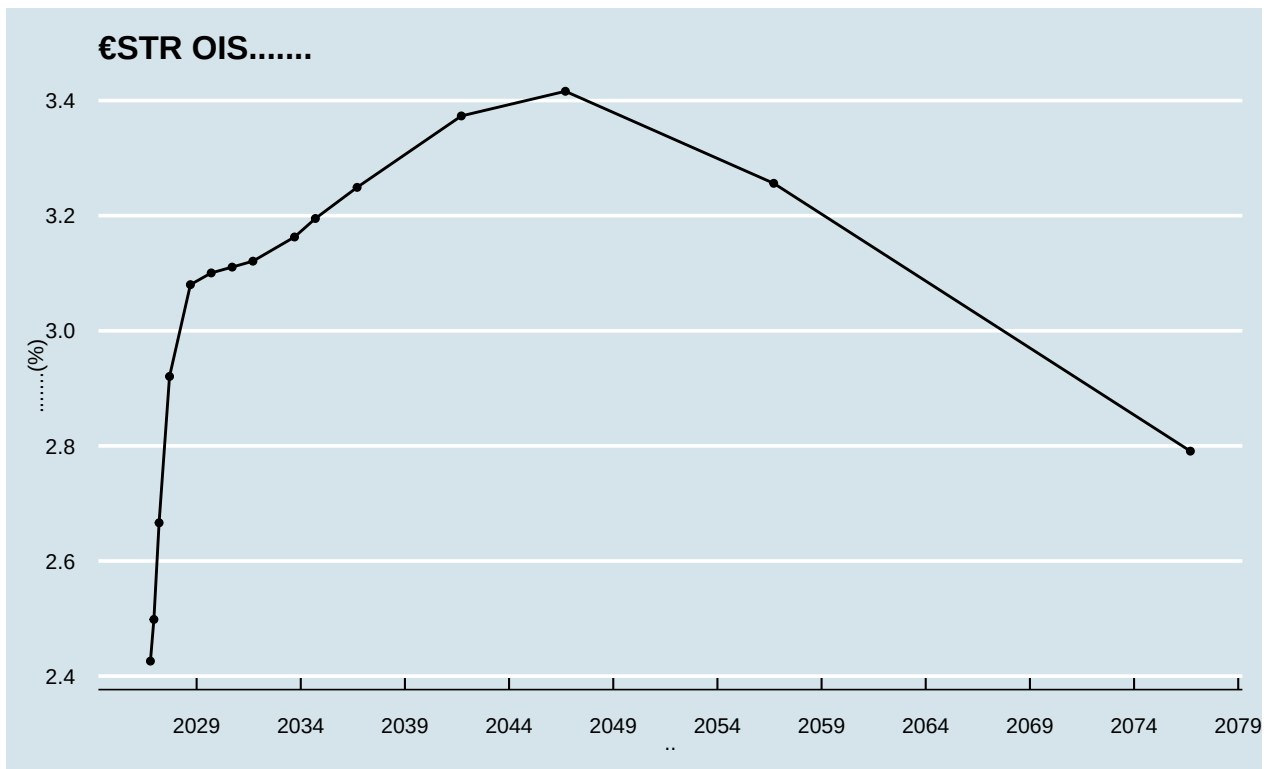
```

```

) +
ggplot2::geom_line(linewidth = 0.7) +
ggplot2::geom_point(size = 1.5) +
ggplot2::scale_x_date(
  date_breaks = "5 years",
  date_labels = "%Y"
) +
ggplot2::labs(
  title = "€ STR OISスポットカーブ",
  x = "満期",
  y = "スポットレート (%)"
)

show_plot_LFL(estr_curve_plot)

```



ここで表示する€ STR カーブと、後半で扱う 2012 年の EONIA カーブは観測時点が異なるため、金利水準そのものを直接比較するものではない。両者を通じて確認するのは、ユーロ OIS 市場の基準金利が EONIA から€ STR へ移行しても、市場クォートから期間構造を構築し、OIS 評価の基礎となるディスカウントカーブを得るという基本的な枠組みが引き継がれていることである。

OIS (Overnight Index Swap) は、あらかじめ定めた固定金利と、計算期間中の翌日物金利を複利で積み上げた変動金利を交換する金利スワップである。元本そのものは交換せず、通常は固定レグと変動レグの利息差額を決済する。

日次金利を r_i 、その金利が適用される日数を d_i 、日数計算の年間基準を B とすると、計算期

間全体の複利収益率は概念的に次式で表される。

$$R_{\text{comp}} = \prod_{i=1}^n \left(1 + r_i \frac{d_i}{B} \right) - 1$$

元本を N とすると、変動レグの利息額は次式となる。

$$I_{\text{overnight}} = NR_{\text{comp}}$$

固定金利を K 、固定レグの年率換算期間を α とすると、固定レグの利息額は次式となる。

$$I_{\text{fixed}} = NK\alpha$$

固定金利を支払う OIS の価値は、変動レグの現在価値から固定レグの現在価値を差し引いて求める。

$$PV_{\text{OIS}} = PV_{\text{overnight}} - PV_{\text{fixed}}$$

契約締結時に両レグの現在価値を等しくする固定金利がフェア OIS レートである。

以下では、カーブを表形式で確認するための補助関数を定義する。これは計算ロジックを新たに実装するものではなく、QuantLib のカーブから割引係数と連続複利ゼロ金利を取り出す表示関数である。

```
curve_tbl_local <- function(curve_obj, n = 200, extrapolate = TRUE) {
  if (extrapolate) {
    QuantLib::TermStructure_enableExtrapolation(curve_obj)
  }

  reference_date <- as.Date(
    GaussQuant::iso_GQL(curve_obj$referenceDate())
  )

  curve_times <- seq(
    from = 0,
    to = curve_obj$maxTime(),
    length.out = n
  )

  tibble::tibble(time = curve_times) |>
  dplyr::mutate(
```

```

discount_factor = purrr::map_dbl(
  time,
  function(x) curve_obj$discount(x)
),
zero_rate = dplyr::if_else(
  time > 0,
  -log(discount_factor) / time,
  0
),
date = reference_date + round(time * 365)
) |>
dplyr::select(
  date,
  time,
  discount_factor,
  zero_rate
)
}

```

II-3.2 TONA カーブと短期 OIS 評価

TONA は、日本の無担保コール市場における翌日物金利を基礎とする円のリスク・フリー・レートである。本節では、2022 年 8 月 19 日の例示的な TONA および OIS 市場レートから、円 OIS カーブを構築する。

短期部分には翌日物金利や短期 OIS を使用し、中長期部分には各満期の OIS スワップレートを使用する。ブートストラップでは、各市場商品の理論価格が市場価格と一致するように、満期ごとの割引係数を順次決定する。

```

trade_date_tona <- "2022-08-19"

GaussQuant::set_eval_date_GQL(trade_date_tona)

tona_quotes <- tibble::tribble(
  ~as_of_date, ~currency, ~instrument, ~kind, ~tenor, ~rate,
  as.Date(trade_date_tona), "JPY", "TONA", "depo", "1D", -0.0000900,
  as.Date(trade_date_tona), "JPY", "TONA", "ois", "1W", -0.0001261,
  as.Date(trade_date_tona), "JPY", "TONA", "ois", "2W", -0.0001469,
  as.Date(trade_date_tona), "JPY", "TONA", "ois", "1M", -0.0001807,
  as.Date(trade_date_tona), "JPY", "TONA", "ois", "3M", -0.0001919,
  as.Date(trade_date_tona), "JPY", "TONA", "ois", "6M", -0.0001043,
  as.Date(trade_date_tona), "JPY", "TONA", "ois", "9M", 0.0000022,
  as.Date(trade_date_tona), "JPY", "TONA", "ois", "12M", 0.0001250,

```

```

as.Date(trade_date_tona), "JPY", "TONA", "ois", "18M", 0.0003125,
as.Date(trade_date_tona), "JPY", "TONA", "ois", "2Y", 0.0004875,
as.Date(trade_date_tona), "JPY", "TONA", "ois", "3Y", 0.0007375,
as.Date(trade_date_tona), "JPY", "TONA", "ois", "4Y", 0.0009479,
as.Date(trade_date_tona), "JPY", "TONA", "ois", "5Y", 0.0011854,
as.Date(trade_date_tona), "JPY", "TONA", "ois", "7Y", 0.0019146
)

GaussQuant::show_tbl_GQH(
  tona_quotes,
  "TONA market quotes",
  n = 20
)

#> # A tibble: 14 x 6
#>   as_of_date currency instrument kind tenor      rate
#>   <date>      <chr>      <chr>      <chr> <chr>      <dbl>
#> 1 2022-08-19 JPY      TONA      depo  1D      -0.00009
#> 2 2022-08-19 JPY      TONA      ois    1W      -0.0001261
#> 3 2022-08-19 JPY      TONA      ois    2W      -0.0001469
#> 4 2022-08-19 JPY      TONA      ois    1M      -0.0001807
#> 5 2022-08-19 JPY      TONA      ois    3M      -0.0001919
#> 6 2022-08-19 JPY      TONA      ois    6M      -0.0001043
#> 7 2022-08-19 JPY      TONA      ois    9M       0.0000022
#> 8 2022-08-19 JPY      TONA      ois   12M      0.000125
#> 9 2022-08-19 JPY      TONA      ois   18M      0.0003125
#> 10 2022-08-19 JPY      TONA      ois   2Y      0.0004875
#> 11 2022-08-19 JPY      TONA      ois   3Y      0.0007375
#> 12 2022-08-19 JPY      TONA      ois   4Y      0.0009479
#> 13 2022-08-19 JPY      TONA      ois   5Y      0.0011854
#> 14 2022-08-19 JPY      TONA      ois   7Y      0.0019146

```

```

tona_curve_envs <- GaussQuant::ir_build_ois_curve_envs_GQH(
  quotes = tona_quotes,
  trade_date = trade_date_tona,
  verbose = TRUE
)

tona_curve_env <- tona_curve_envs[["JPY::TONA"]]
tona_curve <- tona_curve_env$curve

tona_curve_display_tbl <- curve_tbl_local(
  curve_obj = tona_curve,
  n = 200
)

```

```
GaussQuant::show_tbl_GQH(
  tona_curve_display_tbl,
  "TONA OIS curve",
  n = 12
)
#> # A tibble: 12 x 4
#>   date          time discount_factor  zero_rate
#>   <date>        <dbl>          <dbl>      <dbl>
#> 1 2022-08-23 0          1          0
#> 2 2022-09-05 0.035203    1.0000 -0.00014504
#> 3 2022-09-18 0.070407    1.0000 -0.00017306
#> 4 2022-10-01 0.10561      1.0000 -0.00018278
#> 5 2022-10-13 0.14081      1.0000 -0.00018668
#> 6 2022-10-26 0.17602      1.0000 -0.00018902
#> 7 2022-11-08 0.21122      1.0000 -0.00019057
#> 8 2022-11-21 0.24642      1.0000 -0.00019169
#> 9 2022-12-04 0.28163      1.0000 -0.00017512
#> 10 2022-12-17 0.31683      1.0000 -0.00015741
#> 11 2022-12-29 0.35203      1.0001 -0.00014325
#> 12 2023-01-11 0.38724      1.0001 -0.00013166

tona_curve_check_tbl <- tibble::tibble(
  reference_date = GaussQuant::iso_GQL(
    tona_curve$referenceDate()
  ),
  discount_2022_08_23 = tona_curve$discount(
    GaussQuant::date_GQL("2022-08-23")
  )
)

GaussQuant::show_tbl_GQH(
  tona_curve_check_tbl,
  "TONA curve checks",
  n = 20
)
#> # A tibble: 1 x 2
#>   reference_date discount_2022_08_23
#>   <chr>          <dbl>
#> 1 2022-08-23      1
```

割引係数 $P(0, T)$ は、将来時点 T の 1 円を評価日時点の価値へ換算する係数である。満期が長くなるほど、通常は割引係数が低下する。ゼロ金利は、各満期の割引係数を単一の金利として表したものである。

次に、TONA カーブを用いて短期円 OIS を評価する。評価対象は、2022 年 8 月 23 日から 8 月 30 日までの 7 日間、元本 1,000 万円、固定金利マイナス 0.015% の固定金利支払い OIS である。

OIS の固定レグと翌日物レグを確認し、NPV とフェアレートを表示する。

```
jpy_ois_short <- GaussQuant::trade_ois_swap_py_GQH(  
  curve_env = tona_curve_env,  
  effective = "2022-08-23",  
  maturity = "2022-08-30",  
  notional = 10000000,  
  fixed_rate = -0.00015,  
  pay_receive = "pay",  
  verbose = TRUE  
)  
  
GaussQuant::ir_show_swap_summary_GQH(  
  trade_env = jpy_ois_short,  
  title = "JPY short OIS summary"  
)  
  
jpy_ois_fixed_cf_tbl <- GaussQuant::swap_fixed_leg_table_GQL(  
  jpy_ois_short$swap  
)  
  
jpy_ois_overnight_cf_tbl <- GaussQuant::ois_overnight_leg_table_GQL(  
  jpy_ois_short$swap  
)  
  
GaussQuant::show_tbl_GQH(  
  jpy_ois_fixed_cf_tbl,  
  "TONA fixed leg cashflows",  
  n = 10  
)  
#> # A tibble: 1 x 2  
#>   date      amount  
#>   <chr>      <dbl>  
#> 1 2022-08-30 -28.767  
  
GaussQuant::show_tbl_GQH(  
  jpy_ois_overnight_cf_tbl,  
  "TONA overnight leg cashflows",  
  n = 10  
)  
#> # A tibble: 1 x 2  
#>   date      amount  
#>   <chr>      <dbl>
```

```
#> 1 2022-08-30 -24.184
```

単一期間の OIS では、開始日の割引係数を $P(0, T_0)$ 、終了日の割引係数を $P(0, T_1)$ 、固定レグの支払日の割引係数を $P(0, T_p)$ 、固定レグの年率換算期間を α とすると、フェアレートは概念的に次式で求められる。

$$K_{\text{fair}} = \frac{P(0, T_0) - P(0, T_1)}{\alpha P(0, T_p)}$$

```
jpy_effective_df <- tona_curve$discount(
  GaussQuant::date_GQL("2022-08-23")
)

jpy_maturity_df <- tona_curve$discount(
  GaussQuant::date_GQL("2022-08-30")
)

jpy_payment_df <- tona_curve$discount(
  GaussQuant::date_GQL("2022-09-01")
)

jpy_annuity <- (7 / 365) * jpy_payment_df

jpy_swap_rate_hand <- (
  jpy_effective_df - jpy_maturity_df
) / jpy_annuity

jpy_fair_rate_check_tbl <- tibble::tibble(
  effective_discount_factor = jpy_effective_df,
  maturity_discount_factor = jpy_maturity_df,
  payment_discount_factor = jpy_payment_df,
  annuity = jpy_annuity,
  fair_swap_rate_hand = jpy_swap_rate_hand
)

GaussQuant::show_tbl_GQH(
  jpy_fair_rate_check_tbl,
  "TONA fair-rate hand check",
  n = 20
)

#> # A tibble: 1 x 5
#>   effective_discount_fa~1 maturity_discount_fa~2 payment_discount_fac~3 annuity
#>   <dbl>                <dbl>                <dbl>    <dbl>
#> 1             1             1.0000             1.0000 0.019178
```



```
#> # i abbreviated names: 1: effective_discount_factor,
#> #   2: maturity_discount_factor, 3: payment_discount_factor
#> # i 1 more variable: fair_swap_rate_hand <dbl>
```

II-3.3 日次複利と Fixing

OIS の変動レグでは、計算期間中の翌日物金利を日ごとに複利で積み上げる。休日をまたぐ金利は、次の営業日まで複数日分に適用されるため、各行の適用日数を確認する必要がある。

次のコードでは、変動レグを日次単位に分解し、割引係数から日次フォワード金利を求める。

```
jpy_ois_daily <- GaussQuant::trade_ois_swap_py_GQH(
  curve_env = tona_curve_env,
  effective = "2022-08-23",
  maturity = "2022-08-30",
  notional = 10000000,
  fixed_rate = -0.00015,
  pay_receive = "pay",
  fixed_schedule_tenor = "1Y",
  floating_schedule_tenor = "1D",
  pay_lag = 0,
  verbose = TRUE
)

GaussQuant::set_eval_date_GQL(trade_date_tona)

jpy_daily_float_cf_tbl <- GaussQuant::ois_daily_forward_table_GQH(
  swap = jpy_ois_daily$swap,
  curve = tona_curve,
  index = jpy_ois_daily$index,
  eval_date = trade_date_tona,
  notional = 10000000
)

GaussQuant::show_tbl_GQH(
  jpy_daily_float_cf_tbl,
  "TONA daily forward leg",
  n = 20
)

#> # A tibble: 5 x 9
#>   fixing_date next_date   days fixing_before_eval fixing_on_eval fixing_value
#>   <chr>         <chr>   <int> <lgl>                <lgl>                <dbl>
#> 1 2022-08-23 2022-08-24     1 FALSE                FALSE                -0.000090000
```

```
#> 2 2022-08-24 2022-08-25 1 FALSE FALSE -0.00013212
#> 3 2022-08-25 2022-08-26 1 FALSE FALSE -0.00013212
#> 4 2022-08-26 2022-08-29 3 FALSE FALSE -0.00013212
#> 5 2022-08-29 2022-08-30 1 FALSE FALSE -0.00013212
#> # i 3 more variables: forward_rate_from_df <dbl>, applied_rate <dbl>,
#> # amount <dbl>
```

各日の単利フォワード金利は、開始日の割引係数を $P(0, t_i)$ 、終了日の割引係数を $P(0, t_{i+1})$ 、年率換算期間を δ_i とすると、次式で求められる。

$$f_i = \frac{P(0, t_i)/P(0, t_{i+1}) - 1}{\delta_i}$$

日次フォワード金利を用いて複利収益率と利息額を確認する。

```
daily_compounding_result <- if (
  nrow(jpy_daily_float_cf_tbl) >= 2 &&
  all(c("days", "rate") %in% names(jpy_daily_float_cf_tbl))
) {
  daily_compounded_return <- (
    1 +
    jpy_daily_float_cf_tbl$days[-1] /
    365 *
    jpy_daily_float_cf_tbl$rate[-1]
  ) |>
  prod() -
  1

  tibble::tibble(
    compounded_return = daily_compounded_return,
    floating_interest = daily_compounded_return * 10000000,
    equivalent_annual_rate = daily_compounded_return * 365 / 7
  )
} else {
  tibble::tibble(
    compounded_return = NA_real_,
    floating_interest = NA_real_,
    equivalent_annual_rate = NA_real_
  )
}

GaussQuant::show_tbl_GQH(
  daily_compounding_result,
  "TONA daily compounding result",
```

```

  n = 20
)
#> # A tibble: 1 x 3
#>   compounded_return floating_interest equivalent_annual_rate
#>   <dbl>             <dbl>             <dbl>
#> 1             NA             NA             NA

```

評価日より前の翌日物金利には、実際に公表された Fixing を使用する。評価日より後の金利は、OIS カーブから推定したフォワード金利を使用する。

したがって、評価途中の OIS 変動レグは、次の二つから構成される。

- 評価日までに確定した日次 Fixing の複利部分
- 評価日より後のフォワード金利による予測部分

後決め複利では、期間終了まで変動レグ全体の利息額が確定しない。Fixing の公表時刻と支払処理の間に必要な時間を確保するため、実務では Lookback、Observation Shift、Lockout または Payment Delay などの規則が用いられることがある。

次の例では、2022 年 8 月 23 日および 24 日の TONA Fixing を登録し、2022 年 8 月 25 日時点で再評価する。

```

jpy_fixings_tbl <- tibble::tribble(
  ~date, ~currency, ~instrument, ~fixing,
  "2022-08-23", "JPY", "TONA", -0.000090,
  "2022-08-24", "JPY", "TONA", -0.000085
)

GaussQuant::set_eval_date_GQL(trade_date_tona)

jpy_overnight_before_tbl <- GaussQuant::ois_overnight_leg_table_GQL(
  jpy_ois_short$swap
)

GaussQuant::ir_apply_fixings_GQH(
  index_obj = jpy_ois_short$index,
  fixings_tbl = jpy_fixings_tbl,
  currency = "JPY",
  instrument = "TONA"
)

GaussQuant::set_eval_date_GQL("2022-08-25")

jpy_overnight_after_tbl <- GaussQuant::ois_overnight_leg_table_GQL(

```

```

    jpy_ois_short$swap
  )

GaussQuant::ir_show_fixing_before_after_GQH(
  before_tbl = jpy_overnight_before_tbl,
  after_tbl = jpy_overnight_after_tbl,
  title = "TONA fixing before / after",
  n = 10
)

#> # A tibble: 1 x 3
#>   row_id amount_before amount_after
#>   <int>      <dbl>      <dbl>
#> 1      1      -24.184      -24.183

GaussQuant::ir_show_swap_summary_GQH(
  trade_env = jpy_ois_short,
  title = "TONA short OIS summary after fixings"
)

jpy_fixing_diag_tbl <- GaussQuant::ir_fixing_table_GQH(
  swap = jpy_ois_daily$swap,
  curve = tona_curve,
  index = jpy_ois_short$index
)

GaussQuant::show_tbl_GQH(
  jpy_fixing_diag_tbl,
  "TONA daily fixing decomposition",
  n = 20
)

#> # A tibble: 1 x 7
#>       i fixing_date fixing_value pay_date   amount    df    pv
#>   <int> <chr>      <dbl> <chr>      <dbl> <dbl> <dbl>
#> 1     1 2022-08-29  -0.00014154 2022-08-30 -24.183 1.0000 -24.183

```

II-3.4 SOFR とマルチカーブ評価

SOFR は、米国債を担保とする翌日物レポ取引に基づく米ドルのリスク・フリー・レートである。TONA が無担保翌日物市場に基づくのに対し、SOFR は有担保取引に基づく点が異なる。

OIS 評価の基本構造は TONA と同じであり、通貨、休日カレンダー、日数計算規則、市場クォートおよび翌日物指数が異なる。

```

trade_date_sofr <- "2023-09-26"

GaussQuant::set_eval_date_GQL(trade_date_sofr)

sofr_quotes <- tibble::tribble(
  ~as_of_date, ~currency, ~instrument, ~kind, ~tenor, ~rate,
  as.Date(trade_date_sofr), "USD", "SOFR", "depo", "1D", 0.0531,
  as.Date(trade_date_sofr), "USD", "SOFR", "ois", "1M", 0.0532,
  as.Date(trade_date_sofr), "USD", "SOFR", "ois", "3M", 0.0538,
  as.Date(trade_date_sofr), "USD", "SOFR", "ois", "6M", 0.0546,
  as.Date(trade_date_sofr), "USD", "SOFR", "ois", "1Y", 0.0545,
  as.Date(trade_date_sofr), "USD", "SOFR", "ois", "2Y", 0.0501,
  as.Date(trade_date_sofr), "USD", "SOFR", "ois", "3Y", 0.0467
)

sofr_curve_envs <- GaussQuant::ir_build_ois_curve_envs_GQH(
  quotes = sofr_quotes,
  trade_date = trade_date_sofr,
  verbose = TRUE
)

sofr_curve_env <- sofr_curve_envs[["USD::SOFR"]]
sofr_curve <- sofr_curve_env$curve

GaussQuant::show_tbl_GQH(
  curve_tbl_local(sofr_curve),
  "SOFR OIS curve",
  n = 12
)

#> # A tibble: 12 x 4
#>   date          time discount_factor zero_rate
#>   <date>        <dbl>          <dbl>      <dbl>
#> 1 2023-09-28 0          1          0
#> 2 2023-10-04 0.015299    0.99919 0.053075
#> 3 2023-10-09 0.030597    0.99838 0.053075
#> 4 2023-10-15 0.045896    0.99757 0.053075
#> 5 2023-10-20 0.061195    0.99676 0.053075
#> 6 2023-10-26 0.076494    0.99595 0.053075
#> 7 2023-11-01 0.091792    0.99514 0.053092
#> 8 2023-11-06 0.10709     0.99432 0.053170
#> 9 2023-11-12 0.12239     0.99351 0.053228
#> 10 2023-11-17 0.13769     0.99269 0.053273
#> 11 2023-11-23 0.15299     0.99188 0.053309
#> 12 2023-11-28 0.16829     0.99106 0.053339

```

```

sofr_ois_trade <- GaussQuant::trade_ois_swap_py_GQH(
  curve_env = sofr_curve_env,
  effective = "2023-09-28",
  maturity = "2025-09-28",
  notional = 10000000,
  fixed_rate = 0.05,
  pay_receive = "pay",
  verbose = TRUE
)

GaussQuant::ir_show_swap_summary_GQH(
  trade_env = sofr_ois_trade,
  title = "SOFR OIS summary"
)

sofr_overnight_cf_tbl <- GaussQuant::ois_overnight_leg_table_GQL(
  sofr_ois_trade$swap
)

GaussQuant::show_tbl_GQH(
  sofr_overnight_cf_tbl,
  "SOFR overnight leg cashflows",
  n = 20
)

#> # A tibble: 2 x 2
#>   date      amount
#>   <chr>      <dbl>
#> 1 2024-09-30 557111.
#> 2 2025-09-29 459522.

```

金融危機以前のシングルカーブ評価では、同一の金利カーブを用いて、将来変動金利の予測とキャッシュフローの割引を行っていた。

マルチカーブ評価では、二つの役割を分離する。

- フォワードカーブ参照金利の将来 Fixing または期間フォワード金利を予測する。
- ディスカウントカーブ担保契約に対応する金利を用いて、将来キャッシュフローを現在価値へ割り引く。

将来変動金利を F_j 、年率換算期間を α_j 、割引係数を D_j 、元本を N とすると、変動レグの現在価値は概念的に次式で表される。

$$PV_{\text{floating}} = N \sum_{j=1}^m \alpha_j F_j D_j$$

ここで、 F_j はフォワードカーブから求め、 D_j はディスカウントカーブから求める。

以下では、マルチカーブの効果だけを明確にするため、EURIBOR 6 か月物を参照する一般的なバニラスワップを用いる。同じフォワードカーブを使用しながら、割引カーブを同一にした場合と分離した場合の NPV を比較する。

```
GaussQuant::set_eval_date_GQL("2023-09-26")

multi_curve_dates <- c(
  "2023-09-26",
  "2024-09-26",
  "2025-09-26",
  "2026-09-28",
  "2028-09-26"
)

forecast_zero_rates <- c(
  0.0450,
  0.0470,
  0.0460,
  0.0445,
  0.0420
)

discount_zero_rates <- c(
  0.0400,
  0.0415,
  0.0410,
  0.0400,
  0.0380
)

forecast_curve <- GaussQuant::build_zero_curve_GQL(
  date_chr = multi_curve_dates,
  zero_rates = forecast_zero_rates,
  day_counter = "Actual365Fixed"
)

discount_curve <- GaussQuant::build_zero_curve_GQL(
  date_chr = multi_curve_dates,
  zero_rates = discount_zero_rates,
```

```

    day_counter = "Actual365Fixed"
  )

QuantLib::TermStructure_enableExtrapolation(forecast_curve)
#> NULL
QuantLib::TermStructure_enableExtrapolation(discount_curve)
#> NULL

forecast_handle <- GaussQuant::yield_curve_handle_GQL(
  forecast_curve
)

discount_handle <- GaussQuant::yield_curve_handle_GQL(
  discount_curve
)

multi_curve_comparison_tbl <- tibble::tibble(
  date = as.Date(multi_curve_dates)
) |>
  dplyr::mutate(
    forecast_discount_factor = purrr::map_dbl(
      as.character(date),
      function(x) {
        forecast_curve$discount(
          GaussQuant::date_GQL(x)
        )
      }
    ),
    ois_discount_factor = purrr::map_dbl(
      as.character(date),
      function(x) {
        discount_curve$discount(
          GaussQuant::date_GQL(x)
        )
      }
    )
  )

GaussQuant::show_tbl_GQH(
  multi_curve_comparison_tbl,
  "Forecast and discount curves",
  n = 20
)

#> # A tibble: 5 x 3
#>   date          forecast_discount_factor ois_discount_factor

```


#>	<date>	<dbl>	<dbl>
#> 1	2023-09-26	1	1
#> 2	2024-09-26	0.95396	0.95924
#> 3	2025-09-26	0.91199	0.92117
#> 4	2026-09-28	0.87471	0.88663
#> 5	2028-09-26	0.81040	0.82679

```
single_curve_swap <- GaussQuant::make_vanilla_swap_GQL(
  effective_date = "2023-09-28",
  maturity_date = "2025-09-28",
  fixed_rate = 0.045,
  notional = 10000000,
  forecast_handle = forecast_handle,
  discount_handle = forecast_handle,
  swap_type = "payer",
  index = "Euribor6M",
  fixed_tenor = GaussQuant::period_years_GQL(1),
  floating_tenor = GaussQuant::period_months_GQL(6),
  calendar = QuantLib::TARGET()
)
```

```
multi_curve_swap <- GaussQuant::make_vanilla_swap_GQL(
  effective_date = "2023-09-28",
  maturity_date = "2025-09-28",
  fixed_rate = 0.045,
  notional = 10000000,
  forecast_handle = forecast_handle,
  discount_handle = discount_handle,
  swap_type = "payer",
  index = "Euribor6M",
  fixed_tenor = GaussQuant::period_years_GQL(1),
  floating_tenor = GaussQuant::period_months_GQL(6),
  calendar = QuantLib::TARGET()
)
```

```
single_curve_summary <- GaussQuant::swap_summary_GQL(
  single_curve_swap
)
```

```
multi_curve_summary <- GaussQuant::swap_summary_GQL(
  multi_curve_swap
)
```

```
curve_valuation_comparison_tbl <- dplyr::bind_rows(
  single_curve_summary |>
```

```

    dplyr::mutate(valuation_method = "single_curve"),
  multi_curve_summary |>
    dplyr::mutate(valuation_method = "multi_curve")
) |>
  dplyr::relocate(valuation_method)

GaussQuant::show_tbl_GQH(
  curve_valuation_comparison_tbl,
  "Single-curve and multi-curve valuation",
  n = 20
)
#> # A tibble: 2 x 6
#>   valuation_method    npv fixed_leg_npv floating_leg_npv fair_rate fair_spread
#>   <chr>             <dbl>      <dbl>          <dbl>      <dbl>      <dbl>
#> 1 single_curve      40292.    -840562.        880855.    0.047157  -0.0020999
#> 2 multi_curve      39436.    -847103.        886539.    0.047095  -0.0020420

```

両方のスワップは同じフォワードカーブから将来 EURIBOR を予測するが、シングルカーブ評価ではそのカーブを割引にも使用し、マルチカーブ評価では別のディスカウントカーブを使用する。割引係数が異なるため、同じ将来キャッシュフローであっても NPV およびフェアレートが変化する。

II-3.5 ケーススタディ 1：EONIA カーブと OIS 評価

本ケーススタディでは、2012 年 12 月 11 日の EONIA 市場データから OIS カーブを構築する。短期部分には翌日物預金と短期 OIS を用い、その先には日付指定 OIS と長期 OIS を用いる。

カーブ構築後、年末をまたぐ市場金利に含まれる流動性効果を分離し、年末ジャンプを組み込んだ最終的な EONIA カーブを作成する。さらに、カーブ構築に使用した 5 年 OIS の市場レートを契約固定金利として OIS を構築し、NPV とフェアレートを確認する。

評価日は 2012 年 12 月 11 日とする。市場データには、翌日物預金、短期 OIS、日付指定 OIS および長期 OIS を使用する。

```

case_evaluation_date <- "2012-12-11"

GaussQuant::set_eval_date_GQL(
  case_evaluation_date
)

case_eonia_quotes <- GaussQuant::ametrano_bianchetti_eonia_quotes_GQL()

```

```
GaussQuant::show_tbl_GQH(
  case_eonia_quotes,
  "EONIA market quotes",
  n = 40
)
#> # A tibble: 30 x 9
#>   quote_id instrument_type rate_pct    rate fixing_days tenor_n tenor_unit
#>   <chr>      <chr>          <dbl>    <dbl>    <int>   <int> <chr>
#> 1 DEP-ON-0D deposit          0.04    0.0004      0       1 Days
#> 2 DEP-ON-1D deposit          0.04    0.0004      1       1 Days
#> 3 DEP-ON-2D deposit          0.04    0.0004      2       1 Days
#> 4 OIS-1W    spot_ois          0.07    0.0007      2       1 Weeks
#> 5 OIS-2W    spot_ois          0.069   0.00069     2       2 Weeks
#> 6 OIS-3W    spot_ois          0.078   0.00078     2       3 Weeks
#> 7 OIS-1M    spot_ois          0.074   0.00074     2       1 Months
#> 8 OIS-ECB-1 dated_ois          0.046   0.00046     NA      NA <NA>
#> 9 OIS-ECB-2 dated_ois          0.016   0.00016     NA      NA <NA>
#> 10 OIS-ECB-3 dated_ois        -0.007  -0.00007     NA      NA <NA>
#> # i 20 more rows
#> # i 2 more variables: start_date <chr>, end_date <chr>
```

`rate_pct` はパーセント表示の市場レートであり、`rate` は QuantLib へ渡す小数表示の金利である。

預金には 1 日の期間を設定し、評価日からスポット日までを接続するため、Fixing 日数を 0 日、1 日および 2 日とした。短期 OIS と長期 OIS にはスポット日から満期までの期間を指定し、日付指定 OIS には開始日と終了日を直接指定する。

次に、市場レートから EONIA カーブを構築する。

最初に対数割引係数を 3 次補間するカーブと、区分的に一定のフォワード金利を用いるカーブを構築する。後者を用いて年末部分のフォワード金利を確認し、年末ジャンプを推定する。

```
case_eonia_benchmark <- GaussQuant::eonia_curve_benchmark_GQL(
  quotes = case_eonia_quotes,
  evaluation_date = case_evaluation_date
)

case_eonia_curve <- case_eonia_benchmark$final$curve
case_eonia_handle <- case_eonia_benchmark$final$curve_handle

case_jump_summary <- case_eonia_benchmark$jump_summary

GaussQuant::show_tbl_GQH(
```

```

case_jump_summary,
"EONIA year-end jump",
n = 20
)
#> # A tibble: 1 x 7
#>   original_forward clean_forward      t12 jump_time jump_rate
#>   <dbl>          <dbl>    <dbl>    <dbl>    <dbl>
#> 1      0.00081531    0.00067122 0.038356 0.0054795 0.0010086
#> # i 2 more variables: jump_discount_factor <dbl>, jump_date <date>

```

年末ジャンプの推定では、2012 年 12 月 24 日から 2013 年 1 月 7 日までのフォワード金利を確認する。次に、2013 年 1 月 3 日の区分フォワード金利を前後のノードの平均へ置き換え、年末要因を含まないフォワードカーブを作成する。

元のフォワード金利と調整後のフォワード金利の差を、2012 年 12 月 31 日から 2013 年 1 月 2 日までの期間へ換算し、年末ジャンプを求める。

年末ジャンプを推定する過程で得られたフォワード金利を確認する。

```

case_benchmark_check_tbl <- tibble::tribble(
  ~metric, ~reference_value_pct, ~r_value_pct,
  "Original two-week forward", 0.082,
  100 * case_jump_summary$original_forward[[1L]],
  "Adjusted two-week forward", 0.067,
  100 * case_jump_summary$clean_forward[[1L]],
  "Year-end jump", 0.101,
  100 * case_jump_summary$jump_rate[[1L]]
) |>
dplyr::mutate(
  difference_pct =
    r_value_pct -
    reference_value_pct
)

GaussQuant::show_tbl_GQH(
  case_benchmark_check_tbl,
  "EONIA benchmark comparison",
  n = 20
)
#> # A tibble: 3 x 4
#>   metric                reference_value_pct r_value_pct difference_pct
#>   <chr>                  <dbl>          <dbl>          <dbl>
#> 1 Original two-week forward      0.082      0.081531     -0.00046933
#> 2 Adjusted two-week forward      0.067      0.067122      0.00012164

```

```
#> 3 Year-end jump          0.101    0.10086    -0.00013673

stopifnot(
  all(
    abs(case_benchmark_check_tbl$difference_pct) <= 0.0005
  )
)
```

基準値はパーセント単位で小数第 3 位まで表示されている。そのため、照合では表示桁の半分に相当する 0.0005 パーセントポイントを許容範囲とする。

構築した最終カーブを用いて、各 Rate Helper のインプライド・レートを求める。

```
case_eonia_validation <- case_eonia_benchmark$final_validation

case_eonia_repricing_tbl <- case_eonia_validation$quote_repricing |>
  dplyr::mutate(
    quote_error_bp = 10000 * quote_error
  )

GaussQuant::show_tbl_GQH(
  case_eonia_repricing_tbl,
  "EONIA quote repricing",
  n = 40
)

#> # A tibble: 30 x 10
#>   quote_id instrument_type market_quote implied_quote quote_error pillar_date
#>   <chr>      <chr>          <dbl>         <dbl>         <dbl> <date>
#> 1 DEP-ON-0D deposit          0.0004      0.00040000    2.6205e-15 2012-12-12
#> 2 DEP-ON-1D deposit          0.0004      0.00040000    2.6205e-15 2012-12-13
#> 3 DEP-ON-2D deposit          0.0004      0.00040000    2.6205e-15 2012-12-14
#> 4 OIS-1W    spot_ois          0.0007      0.00070000    4.5565e-13 2012-12-20
#> 5 OIS-2W    spot_ois          0.00069     0.00069000    1.0801e-14 2012-12-27
#> 6 OIS-3W    spot_ois          0.00078     0.00078000   -1.3611e-15 2013-01-03
#> 7 OIS-1M    spot_ois          0.00074     0.00074000    1.8503e-15 2013-01-14
#> 8 OIS-ECB-1 dated_ois        0.00046     0.00046000    6.5201e-12 2013-02-13
#> 9 OIS-ECB-2 dated_ois        0.00016     0.00016000    8.4594e-12 2013-03-13
#> 10 OIS-ECB-3 dated_ois       -0.00007    -0.000070000   9.1291e-12 2013-04-10
#> # i 20 more rows
#> # i 4 more variables: discount_factor <dbl>, zero_rate <dbl>,
#> #   inst_fwd_rate <dbl>, quote_error_bp <dbl>

stopifnot(
  max(
    abs(case_eonia_repricing_tbl$quote_error),
```

```

    na.rm = TRUE
  ) < 1e-8
)

```

各商品のインプライド・レートと市場レートが一致することにより、預金、短期 OIS、日付指定 OIS および長期 OIS が同じ期間構造上で再現されていることを確認できる。

市場データに含まれる 5 年 OIS を取り上げる。契約固定金利には、カーブ構築に用いた 5 年 OIS の市場レートを使用する。

```

case_ois_tenor <- QuantLib::Period(
  5,
  "Years"
)

case_ois_fixed_rate <- case_eonia_quotes |>
  dplyr::filter(
    quote_id == "OIS-5Y"
  ) |>
  dplyr::pull(
    rate
  )

case_eonia_index <- QuantLib::Eonia(
  case_eonia_handle
)

case_ois_contract_tbl <- tibble::tibble(
  evaluation_date = as.Date(case_evaluation_date),
  index = "EONIA",
  tenor = "5Y",
  settlement_days = 2L,
  contract_fixed_rate = case_ois_fixed_rate
)

GaussQuant::show_tbl_GQH(
  case_ois_contract_tbl,
  "Five-year EONIA OIS contract",
  n = 20
)

#> # A tibble: 1 x 5
#>   evaluation_date index tenor settlement_days contract_fixed_rate
#>   <date>         <chr> <chr>          <int>          <dbl>
#> 1 2012-12-11     EONIA 5Y              2            0.00456

```

最初に、QuantLib の MakeOIS を直接使用して OIS を構築する。

```
case_ois_builder_quantlib <- QuantLib::MakeOIS(  
  swapTenor = case_ois_tenor,  
  overnightIndex = case_eonia_index,  
  fixedRate = case_ois_fixed_rate  
)  
  
case_ois_quantlib <- QuantLib::MakeOIS_makeOIS(  
  case_ois_builder_quantlib  
)
```

次に、同じ期間、同じ EONIA インデックスおよび同じ固定金利を用いて OIS を構築する。

```
case_ois_gaussquant <- GaussQuant::build_ois_GQL(  
  swap_tenor = case_ois_tenor,  
  overnight_index = case_eonia_index,  
  fixed_rate = case_ois_fixed_rate  
)
```

両方の OIS について、NPV、固定レグの現在価値、翌日物レグの現在価値、フェアレートおよびフェアスプレッドを確認する。

```
case_ois_quantlib_summary <- GaussQuant::swap_summary_GQL(  
  case_ois_quantlib  
)  
  
case_ois_gaussquant_summary <- GaussQuant::swap_summary_GQL(  
  case_ois_gaussquant  
)  
  
case_ois_comparison_tbl <- dplyr::bind_rows(  
  case_ois_quantlib_summary |>  
    dplyr::mutate(  
      calculation_route = "QuantLib"  
    ),  
  case_ois_gaussquant_summary |>  
    dplyr::mutate(  
      calculation_route = "GaussQuant"  
    )  
) |>  
  dplyr::relocate(  
    calculation_route  
  )
```

```

GaussQuant::show_tbl_GQH(
  case_ois_comparison_tbl,
  "Five-year EONIA OIS valuation",
  n = 20
)
#> # A tibble: 2 x 6
#>   calculation_route      npv fixed_leg_npv floating_leg_npv fair_rate
#>   <chr>              <dbl>      <dbl>          <dbl>      <dbl>
#> 1 QuantLib          -1.6546e-13    -0.022951      0.022951  0.0045600
#> 2 GaussQuant        -1.6546e-13    -0.022951      0.022951  0.0045600
#> # i 1 more variable: fair_spread <dbl>

case_ois_metrics <- c(
  "npv",
  "fixed_leg_npv",
  "floating_leg_npv",
  "fair_rate",
  "fair_spread"
)

case_quantlib_values <- unlist(
  case_ois_quantlib_summary |>
    dplyr::select(
      dplyr::all_of(case_ois_metrics)
    ),
  use.names = FALSE
)

case_gaussquant_values <- unlist(
  case_ois_gaussquant_summary |>
    dplyr::select(
      dplyr::all_of(case_ois_metrics)
    ),
  use.names = FALSE
)

stopifnot(
  isTRUE(
    all.equal(
      case_quantlib_values,
      case_gaussquant_values,
      tolerance = 1e-12
    )
  )
)

```


カーブ構築に使用した 5 年 OIS の市場レートと、構築後のカーブから求めたフェアレートを比較する。

```
case_ois_rate_check_tbl <- tibble::tibble(
  market_fixed_rate = case_ois_fixed_rate,
  calculated_fair_rate =
    case_ois_gaussquant_summary$fair_rate[[1L]],
  difference_bp = 10000 * (
    case_ois_gaussquant_summary$fair_rate[[1L]] -
    case_ois_fixed_rate
  ),
  npv = case_ois_gaussquant_summary$npv[[1L]]
)

GaussQuant::show_tbl_GQH(
  case_ois_rate_check_tbl,
  "Five-year EONIA OIS rate check",
  n = 20
)

#> # A tibble: 1 x 4
#>   market_fixed_rate calculated_fair_rate difference_bp      npv
#>   <dbl>             <dbl>             <dbl>      <dbl>
#> 1      0.00456         0.0045600      -3.2876e-10 -1.6546e-13

stopifnot(
  abs(
    case_ois_gaussquant_summary$fair_rate[[1L]] -
    case_ois_fixed_rate
  ) < 1e-8
)
```

5 年 OIS はカーブ構築に使用した商品の一つであるため、市場固定金利とカーブから再計算したフェアレートは一致し、NPV はゼロに近い値となる。

最後に、5 年 OIS の固定レグと翌日物レグのキャッシュフローを確認する。

```
case_ois_fixed_leg_tbl <- GaussQuant::leg_to_cashflow_tbl_GQL(
  case_ois_gaussquant$fixedLeg()
)

case_ois_overnight_leg_tbl <- GaussQuant::leg_to_cashflow_tbl_GQL(
  case_ois_gaussquant$overnightLeg()
)

GaussQuant::show_tbl_GQH(
```

```

case_ois_fixed_leg_tbl,
"Five-year EONIA OIS fixed leg",
n = 20
)
#> # A tibble: 5 x 2
#>   date          amount
#>   <chr>         <dbl>
#> 1 2013-12-13 0.0046233
#> 2 2014-12-15 0.0046487
#> 3 2015-12-14 0.0046107
#> 4 2016-12-13 0.0046233
#> 5 2017-12-13 0.0046233

GaussQuant::show_tbl_GQH(
  case_ois_overnight_leg_tbl,
  "Five-year EONIA OIS overnight leg",
  n = 20
)
#> # A tibble: 5 x 2
#>   date          amount
#>   <chr>         <dbl>
#> 1 2013-12-13 0.000014593
#> 2 2014-12-15 0.00071766
#> 3 2015-12-14 0.0031409
#> 4 2016-12-13 0.0072966
#> 5 2017-12-13 0.012153

```

固定レグと翌日物レグは反対方向のキャッシュフローを持ち、市場固定金利がフェアレートと一致する場合には、両レグの現在価値が相殺される。

II-3.6 ケーススタディ 2：市場レート変動と評価損益

第 9 節では、2012 年 12 月 11 日の EONIA 市場データから期間構造を構築し、5 年 OIS の市場固定金利とフェアレートが一致することを確認した。

本ケーススタディでは、評価日と OIS の約定条件を変更せず、カーブ構築に使用する市場レートだけを上下へ 10 ベーシスポイント変化させる。各シナリオについて EONIA カーブを再構築し、第 9 節と同じ 5 年 OIS を再評価する。

比較するシナリオは、次の三つである。

- EONIA 市場レートを一律に 10 ベーシスポイント低下
- 第 9 節と同じ基準市場レート

- EONIA 市場レートを一律に 10 ベーシスポイント上昇

契約固定金利には、第 9 節で設定した 5 年 OIS の市場レート 0.456% を引き続き使用する。したがって、各シナリオの NPV の差は、市場レートの変化による評価損益を表す。

最初に、基準となる EONIA 市場レートへ、マイナス 10 ベーシスポイント、ゼロおよびプラス 10 ベーシスポイントのシフトを加える。

ここでは構築済みのゼロカーブを直接動かすのではなく、預金、短期 OIS、日付指定 OIS および長期 OIS の市場レートを変化させる。その後、それぞれの市場データから EONIA カーブを改めて構築する。

```
case_rate_scenario_tbl <- tibble::tribble(
  ~scenario, ~shift_bp,
  "10bp低下", -10,
  "基準",      0,
  "10bp上昇",  10
) |>
dplyr::mutate(
  quotes = purrr::map(
    shift_bp,
    function(shift_bp) {
      case_eonia_quotes |>
        dplyr::mutate(
          rate = rate + shift_bp / 10000,
          rate_pct = 100 * rate
        )
    }
  )
)
```

10 ベーシスポイントは、小数表示の金利では 0.001、パーセント表示では 0.1 パーセントポイントに相当する。

市場レートの変化を確認するため、翌日物預金、1 か月 OIS、5 年 OIS および 30 年 OIS を取り出す。

```
case_rate_scenario_quote_tbl <- purrr::map2_dfr(
  case_rate_scenario_tbl$scenario,
  case_rate_scenario_tbl$quotes,
  function(scenario, quotes) {
    quotes |>
      dplyr::filter(
        quote_id %in% c(
          "DEP-ON-OD",

```

```

      "OIS-1M",
      "OIS-5Y",
      "OIS-30Y"
    )
  ) |>
  dplyr::transmute(
    scenario = scenario,
    quote_id = quote_id,
    instrument_type = instrument_type,
    rate_pct = rate_pct,
    rate = rate
  )
}
)

GaussQuant::show_tbl_GQH(
  case_rate_scenario_quote_tbl,
  "EONIA market-rate scenarios",
  n = 20
)
#> # A tibble: 12 x 5
#>   scenario quote_id instrument_type rate_pct    rate
#>   <chr>    <chr>      <chr>          <dbl>   <dbl>
#> 1 10bp低下 DEP-ON-OD deposit        -0.06  -0.0006
#> 2 10bp低下 OIS-1M    spot_ois        -0.026 -0.00026
#> 3 10bp低下 OIS-5Y    spot_ois         0.356  0.00356
#> 4 10bp低下 OIS-30Y    spot_ois         1.938  0.01938
#> 5 基準     DEP-ON-OD deposit         0.04   0.0004
#> 6 基準     OIS-1M    spot_ois         0.074  0.00074
#> 7 基準     OIS-5Y    spot_ois         0.456  0.00456
#> 8 基準     OIS-30Y    spot_ois         2.038  0.02038
#> 9 10bp上昇 DEP-ON-OD deposit         0.14   0.0014
#> 10 10bp上昇 OIS-1M    spot_ois         0.174  0.00174
#> 11 10bp上昇 OIS-5Y    spot_ois         0.556  0.00556
#> 12 10bp上昇 OIS-30Y    spot_ois         2.138  0.02138

```

各商品のレートは、基準値から一律に 10 ベーシスポイント低下または上昇している。評価日、Fixing 日数、期間、開始日および終了日などの市場慣行は変更しない。

各シナリオの市場データから、年末ジャンプを含む EONIA カーブを再構築する。

```

case_rate_scenario_tbl <- case_rate_scenario_tbl |>
  dplyr::mutate(
    benchmark = purrr::map(
      quotes,

```

```

function(quotes) {
  GaussQuant::eonia_curve_benchmark_GQL(
    quotes = quotes,
    evaluation_date = case_evaluation_date
  )
}
),
curve = purrr::map(
  benchmark,
  ~ .x$final$curve
),
curve_handle = purrr::map(
  benchmark,
  ~ .x$final$curve_handle
),
max_quote_error = purrr::map_dbl(
  benchmark,
  function(benchmark) {
    max(
      abs(
        benchmark$final_validation$
          quote_repricing$
          quote_error
      ),
      na.rm = TRUE
    )
  }
)
)

```

```

case_curve_validation_tbl <- case_rate_scenario_tbl |>
  dplyr::select(
    scenario,
    shift_bp,
    max_quote_error
  )

```

```

GaussQuant::show_tbl_GQH(
  case_curve_validation_tbl,
  "EONIA scenario-curve validation",
  n = 20
)
#> # A tibble: 3 x 3
#>   scenario shift_bp max_quote_error
#>   <chr>      <dbl>      <dbl>

```

```
#> 1 10bp低下      -10      9.8407e-12
#> 2 基準           0      9.8573e-12
#> 3 10bp上昇       10      9.8710e-12

stopifnot(
  all(
    case_curve_validation_tbl$max_quote_error < 1e-8
  )
)
```

すべてのシナリオについて、Rate Helper のインプライド・レートが入力した市場レートと一致することを確認する。これにより、評価損益の比較に使用する各カーブが、それぞれの市場データを再現していることが分かる。

第 9 節と同じ 5 年 OIS を、各シナリオの EONIA カーブへ接続する。

契約期間は 5 年、契約固定金利は 0.456% のままとし、市場カーブだけを変更する。

```
case_rate_scenario_tbl <- case_rate_scenario_tbl |>
  dplyr::mutate(
    eonia_index = purrr::map(
      curve_handle,
      QuantLib::Eonia
    ),
    ois = purrr::map(
      eonia_index,
      function(eonia_index) {
        GaussQuant::build_ois_GQL(
          swap_tenor = case_ois_tenor,
          overnight_index = eonia_index,
          fixed_rate = case_ois_fixed_rate
        )
      }
    ),
    valuation = purrr::map(
      ois,
      GaussQuant::swap_summary_GQL
    )
  )
```

各シナリオの評価結果を一つの表へまとめる。

```
case_ois_scenario_result_tbl <- case_rate_scenario_tbl |>
  dplyr::select(
```

```
    scenario,
    shift_bp,
    valuation
  ) |>
  tidyr::unnest(
    cols = valuation
  )

case_base_fair_rate <- case_ois_scenario_result_tbl |>
  dplyr::filter(
    scenario == "基準"
  ) |>
  dplyr::pull(
    fair_rate
  )

case_base_npv <- case_ois_scenario_result_tbl |>
  dplyr::filter(
    scenario == "基準"
  ) |>
  dplyr::pull(
    npv
  )

case_ois_scenario_result_tbl <- case_ois_scenario_result_tbl |>
  dplyr::mutate(
    contract_fixed_rate = case_ois_fixed_rate,
    fair_rate_change_bp = 10000 * (
      fair_rate -
      case_base_fair_rate
    ),
    payer_valuation_change =
      npv -
      case_base_npv,
    receiver_npv = -npv,
    receiver_valuation_change =
      -(
        npv -
        case_base_npv
      )
  ) |>
  dplyr::select(
    scenario,
    shift_bp,
    contract_fixed_rate,
```

```

    fair_rate,
    fair_rate_change_bp,
    payer_npv = npv,
    payer_valuation_change,
    receiver_npv,
    receiver_valuation_change
  )

GaussQuant::show_tbl_GQH(
  case_ois_scenario_result_tbl,
  "Five-year EONIA OIS scenario valuation",
  n = 20
)

#> # A tibble: 3 x 9
#>   scenario shift_bp contract_fixed_rate fair_rate fair_rate_change_bp
#>   <chr>      <dbl>          <dbl>      <dbl>          <dbl>
#> 1 10bp低下    -10            0.00456 0.0035600        -10.000
#> 2 基準         0             0.00456 0.0045600         0
#> 3 10bp上昇     10            0.00456 0.0055600        10.0000
#> # i 4 more variables: payer_npv <dbl>, payer_valuation_change <dbl>,
#> #   receiver_npv <dbl>, receiver_valuation_change <dbl>

```

`payer_npv` は固定金利を支払うポジションの NPV であり、`receiver_npv` は固定金利を受け取る反対ポジションの NPV である。

`payer_valuation_change` と `receiver_valuation_change` は、それぞれのシナリオの NPV から基準シナリオの NPV を差し引いた評価損益である。

基準シナリオは、第 9 節で使用した市場データと約定条件をそのまま使用している。そのため、基準シナリオの NPV とフェアレートは、第 9 節の計算結果と一致する。

```

case_base_scenario_tbl <- case_ois_scenario_result_tbl |>
  dplyr::filter(
    scenario == "基準"
  )

case_base_check_tbl <- tibble::tibble(
  metric = c(
    "NPV",
    "Fair rate"
  ),
  chapter9_value = c(
    case_ois_gaussquant_summary$npv[[1L]],
    case_ois_gaussquant_summary$fair_rate[[1L]]
  )
)

```



```

),
chapter10_value = c(
  case_base_scenario_tbl$payer_npv[[1L]],
  case_base_scenario_tbl$fair_rate[[1L]]
)
) |>
dplyr::mutate(
  difference =
    chapter10_value -
    chapter9_value
)

GaussQuant::show_tbl_GQH(
  case_base_check_tbl,
  "Base-scenario consistency check",
  n = 20
)

#> # A tibble: 2 x 4
#>   metric      chapter9_value chapter10_value difference
#>   <chr>          <dbl>          <dbl>          <dbl>
#> 1 NPV          -1.6546e-13      -1.6546e-13          0
#> 2 Fair rate     4.5600e- 3       4.5600e- 3          0

stopifnot(
  isTRUE(
    all.equal(
      case_base_check_tbl$chapter9_value,
      case_base_check_tbl$chapter10_value,
      tolerance = 1e-12
    )
  )
)

```

この照合により、第 9 節の基準評価と、第 10 節の基準シナリオが同じ市場データ、評価日および約定条件に基づいていることを確認できる。

市場レートが低下すると、5 年 OIS のフェアレートも低下する。この場合、固定金利支払側は、低下後の市場水準より高い 0.456% を支払うため、既存契約の価値は低下する。

一方、市場レートが上昇すると、固定金利支払側は市場水準より低い 0.456% を支払う契約を保有することになるため、既存契約の価値は上昇する。

固定金利受取側では、この関係が逆になる。固定金利支払側の利益は固定金利受取側の損失となり、両者の NPV と評価損益は符号が反対になる。

上下 10 ベーシスポイントの結果から、固定金利支払側について、1 ベーシスポイント当たりの平均的な NPV 変化を求める。

```
case_npv_down <- case_ois_scenario_result_tbl |>
  dplyr::filter(
    scenario == "10bp低下"
  ) |>
  dplyr::pull(
    payer_npv
  )

case_npv_up <- case_ois_scenario_result_tbl |>
  dplyr::filter(
    scenario == "10bp上昇"
  ) |>
  dplyr::pull(
    payer_npv
  )

case_ois_bpv_tbl <- tibble::tibble(
  position = "Fixed-rate payer",
  npv_10bp_down = case_npv_down,
  npv_base = case_base_npv,
  npv_10bp_up = case_npv_up,
  average_npv_change_per_bp =
    (
      case_npv_up -
      case_npv_down
    ) / 20
)

GaussQuant::show_tbl_GQH(
  case_ois_bpv_tbl,
  "Five-year EONIA OIS rate sensitivity",
  n = 20
)

#> # A tibble: 1 x 5
#>   position      npv_10bp_down      npv_base npv_10bp_up average_npv_change_pe~1
#>   <chr>          <dbl>          <dbl>      <dbl>          <dbl>
#> 1 Fixed-rate payer -0.0050484 -1.6546e-13  0.0050178      0.00050331
#> # i abbreviated name: 1: average_npv_change_per_bp
```

average_npv_change_per_bp は、マイナス 10 ベーシスポイントからプラス 10 ベーシスポイントまでの NPV 変化を 20 で除した値である。

本ケーススタディでは名目元本を 1 として OIS を構築しているため、この値も単位元本当たりの評価変化を表す。実際の取引では、これに取引元本を乗じることによって、ポジション全体の金利感応度を求めることができる。

II-3.7 まとめ

本章では、世界金融危機と IBOR 改革を背景として、金利デリバティブの評価がシングルカーブ方式からマルチカーブ方式へ移行した経緯を確認した。LIBOR などの IBOR には銀行の信用リスクや資金流動性リスクが含まれるため、担保付取引の割引金利としてそのまま使用することは適切ではない。そこで、担保契約に対応する OIS カーブを割引に用い、参照金利ごとのフォワードカーブから将来金利を予測する評価方法が定着した。

OIS では、固定金利と、計算期間中の日々の翌日物金利を複利で積み上げた変動金利を交換する。評価日以前の翌日物金利には確定した Fixing を使用し、評価日以後の金利には期間構造から求めた予測値を使用する。TONA と SOFR はいずれも翌日物金利であるが、その基礎となる市場や担保性は異なるため、対象通貨の市場慣行を正しく設定する必要がある。

マルチカーブ評価では、将来の変動金利を求めるフォワードカーブと、将来キャッシュフローを現在価値へ変換するディスカウントカーブを分離する。同じ将来キャッシュフローであっても、使用する割引カーブが異なれば NPV やフェアレートも変化する。したがって、金利スワップの評価では、参照インデックスがどのカーブへ接続され、価格計算エンジンがどのカーブを割引に使用しているかを明確にする必要がある。

ケーススタディでは、期間構造ハンドルを翌日物金利インデックスへ接続し、MakeOIS を用いて OIS を構築した。さらに、NPV、フェアレート、固定レグおよび翌日物レグのキャッシュフローを確認し、OIS の契約条件と市場カーブが評価結果へどのように反映されるかを示した。

また、契約固定金利を変更せずに市場カーブを上下へ平行シフトし、OIS の評価損益を比較した。契約締結後も固定金利は変化しないが、市場カーブの変動によって将来の翌日物金利予測と割引係数が変化するため、OIS の NPV は日々変動する。固定金利支払側と固定金利受取側では、同じ市場変動に対する評価損益の符号が反対になる。

実際のリスク管理では、カーブ全体の平行シフトだけでなく、短期、中期および長期の金利を個別に変化させる必要がある。また、Fixing の確定、休日カレンダー、日数計算規則、支払ラグ、予測カーブおよび割引カーブの設定を総合的に確認することが、OIS と金利スワップを正しく評価するための基本となる。

II-3.8 参考資料

- Financial Conduct Authority, [The future of LIBOR](#), 27 July 2017.
- Financial Conduct Authority, [The end of LIBOR](#), 1 October 2024.
- Financial Stability Board, [Reforming Major Interest Rate Benchmarks](#), 22 July 2014.
- Bank of Japan, [Uncollateralized Overnight Call Rate](#).

第 II-4 章株価・為替と幾何ブラウン運動モデル

1973 年に公表された Black-Scholes-Merton モデルは、株式オプションの理論価格を求めるための基本モデルとして、金融工学の発展に大きな影響を与えた。このモデルでは、株価が幾何ブラウン運動に従い、将来の株価が対数正規分布に従うと仮定する。

一方、金利先物、債券先物、商品先物およびフォワード金利を原資産とするオプションでは、現物価格よりも満期に対応する先物価格またはフォワード価格を直接扱う方が自然である。Black は 1976 年、Black-Scholes-Merton モデルの考え方を先物オプションへ適用した。現在、この評価方法は Black-76 モデルまたは単に Black モデルと呼ばれている。

Black-Scholes-Merton モデルでは、現物価格、無リスク金利および配当利回りから将来のフォワード価格を求める。これに対し Black-76 モデルでは、観測されたフォワード価格または先物価格を直接入力し、満期時の Payoff を割り引いて現在価値を求める。

したがって、Black-76 と Black-Scholes-Merton は無関係な別のモデルではない。両者は、原資産価格が正の値を取り、その変化率が正規分布に従うという共通の仮定を持つ。現物価格を出発点とするか、フォワード価格を出発点とするかが異なる。

本章では、まず Black-76 による先物・フォワードオプションの解析評価を行い、オプション価格と Greeks を確認する。次に、同じ確率過程から Monte Carlo パスを生成し、満期 Payoff の割引期待値が解析解へ近づくことを確認する。

その後、Black-Scholes-Merton モデルによる株式オプション評価へ戻り、European オプションの解析解と二項 Tree を比較する。さらに、American オプションの早期行使価値を、近似解、二項 Tree および Longstaff-Schwartz 法によって評価する。

Black モデルは、原資産価格を正の値に限定する対数正規モデルである。この性質は株式や先物価格の評価には都合がよい一方、金利がゼロまたはマイナスとなる市場では制約となる。本章では、この制約を持たない Normal モデルを取り上げ、Black モデルとの違いを確認する。

II-4.1 ブラック – ショールズ – マートンとブラック 76

リスク中立測度の下で、株価 S_t は次の確率微分方程式に従うと仮定する。

$$dS_t = (r - q)S_t dt + \sigma S_t dW_t$$

ここで、 r は無リスク金利、 q は配当利回り、 σ はボラティリティ、 W_t はブラウン運動である。

満期 T のフォワード価格は次式となる。

$$F_0 = S_0 e^{(r-q)T}$$

したがって、Black–Scholes–Merton モデルは現物価格を出発点として、金利と配当を通じてフォワード価格を内生的に求めるモデルとみることができる。

Black-76 では、フォワード価格 F_0 を直接入力する。European call の価格は次式で与えられる。

$$C = D(0, T) [F_0 N(d_1) - K N(d_2)]$$

European put の価格は次式となる。

$$P = D(0, T) [K N(-d_2) - F_0 N(-d_1)]$$

ここで、

$$d_1 = \frac{\log(F_0/K) + \frac{1}{2}\sigma^2 T}{\sigma\sqrt{T}}$$

$$d_2 = d_1 - \sigma\sqrt{T}$$

であり、 K は行使価格、 $D(0, T)$ は満期までの割引係数、 $N(\cdot)$ は標準正規分布の累積分布関数である。

金利が決定論的である場合、同一満期の先物価格とフォワード価格は一致するため、Black-76 は先物オプションとフォワードオプションの双方に用いられる。

本節では、債券先物オプションを意識した価格表示を使用する。フォワード価格は 107 と 32 分の 6.75、行使価格は 107.5、ボラティリティは 5.2%、無リスク金利は 5% とする。

Black-76 では、現物価格ではなく、オプション満期に対応するフォワード価格または先物価格を原資産として使用する。

```
black_input_tbl <- tibble::tribble(
  ~input,      ~value,
  "trade_date", "2023-07-21",
  "maturity_date", "2023-08-26",
  "forward_price", as.character(107 + 6.75 / 32),
  "strike_price",  as.character(107.5),
  "volatility",    as.character(0.052),
  "risk_free_rate", as.character(0.05)
)

trade_date <- GaussQuant::date_GQL("2023-07-21")
maturity_date <- GaussQuant::date_GQL("2023-08-26")
forward_price <- 107 + 6.75 / 32
strike_price <- 107.5
volatility <- 0.052
risk_free_rate <- 0.05
day_counter <- QuantLib::Actual365Fixed()

call_payoff <- QuantLib::PlainVanillaPayoff(
  QuantLib::Option_Call_get(),
  strike_price
)

GaussQuant::set_eval_date_GQL(trade_date)

maturity <- day_counter$yearFraction(
  trade_date,
  maturity_date
)

GaussQuant::show_tbl_GQH(
  black_input_tbl,
  "Black-76 input assumptions",
  n = 20
)

#> # A tibble: 6 x 2
#>   input      value
#>   <chr>      <chr>
#> 1 trade_date 2023-07-21
```

```
#> 2 maturity_date 2023-08-26
#> 3 forward_price 107.2109375
#> 4 strike_price 107.5
#> 5 volatility 0.052
#> 6 risk_free_rate 0.05
```

II-4.2 ブラック 76 による解析評価

BlackCalculator は、フォワード価格、行使価格、総標準偏差および割引係数から、オプション価格と Greeks を直接計算する。

QuantLib へ渡される総標準偏差は、単なる年率ボラティリティではなく、

$$\sigma\sqrt{T}$$

である。

本節では、次の Greeks を確認する。

- **Delta**：フォワード価格の変化に対するオプション価格の感応度
- **Gamma**：フォワード価格の変化に対する Delta の感応度
- **Vega**：ボラティリティの変化に対するオプション価格の感応度
- **Theta**：時間経過に対するオプション価格の感応度

Black-76 の Delta はフォワード価格に対する Delta であり、株式オプションの Spot Delta とは区別する必要がある。

```
black_bundle <- GaussQuant::black_process_bundle_GQL(
  valuation_date = trade_date,
  forward = forward_price,
  risk_free_rate = risk_free_rate,
  volatility = volatility,
  day_counter = day_counter
)

discount_factor <- black_bundle$discount_curve$discount(
  maturity_date
)

black_calculator_tbl <- GaussQuant::black_calculator_table_GQL(
  payoff = call_payoff,
  forward = forward_price,
```



```

    volatility = volatility,
    maturity = maturity,
    discount_factor = discount_factor
)

GaussQuant::show_tbl_GQH(
  black_calculator_tbl,
  "Black calculator summary",
  n = 20
)

#> # A tibble: 6 x 2
#>   metric      value
#>   <chr>      <dbl>
#> 1 npv        0.56160
#> 2 delta       0.43558
#> 3 gamma       0.22397
#> 4 vega       13.203
#> 5 theta      -3.4524
#> 6 theta_per_day -0.0094587

```

前節では Black 公式を直接使用した。本節では、同じ入力条件から Black process を構築し、European option へ解析的 Pricing Engine を設定する。

数式を直接評価する方法と、確率過程および Pricing Engine を通じて評価する方法が、同一の理論価格を与えることを確認する。

```

european_call <- QuantLib::VanillaOption(
  call_payoff,
  QuantLib::EuropeanExercise(maturity_date)
)

analytic_engine <- QuantLib::AnalyticEuropeanEngine(
  black_bundle$process
)

european_call$setPricingEngine(
  analytic_engine
)

#> NULL

analytic_european_tbl <- GaussQuant::option_metric_table_GQL(
  option = european_call,
  process = black_bundle$process
)

```

```

analytic_npv <- analytic_european_tbl |>
  dplyr::filter(metric == "npv") |>
  dplyr::pull(value)

GaussQuant::show_tbl_GQH(
  analytic_european_tbl,
  "Analytic European call",
  n = 20
)
#> # A tibble: 7 x 2
#>   metric      value
#>   <chr>      <dbl>
#> 1 npv        0.56160
#> 2 delta       0.43558
#> 3 gamma       0.22397
#> 4 vega       13.203
#> 5 theta      -3.4524
#> 6 theta_per_day -0.0094587
#> 7 implied_volatility 0.051993

```

II-4.3 ブラック – ショールズ – マートンとヨーロピアン・オプション

ここからは現物株価を原資産とする Black–Scholes–Merton モデルを扱う。現物価格 100、行使価格 100、満期 1 年、ボラティリティ 20%、無リスク金利 5%、配当利回り 0% とする。

Black–Scholes–Merton process では、現物価格に加えて、無リスク金利カーブ、配当利回りカーブおよびボラティリティカーブを設定する。

```

bsm_input_tbl <- tibble::tribble(
  ~input,      ~value,
  "trade_date", "2024-03-01",
  "maturity",   "1Y",
  "spot_price", as.character(100),
  "strike_price", as.character(100),
  "volatility",  as.character(0.20),
  "risk_free_rate", as.character(0.05),
  "dividend_rate", as.character(0)
)

trade_date_bsm <- GaussQuant::date_GQL("2024-03-01")

```

```
calendar_bsm <- QuantLib::TARGET()

maturity_date_bsm <- calendar_bsm$advance(
  trade_date_bsm,
  1L,
  "Years"
)

spot_price_bsm <- 100
strike_price_bsm <- 100
volatility_bsm <- 0.20
risk_free_rate_bsm <- 0.05
dividend_rate_bsm <- 0

GaussQuant::set_eval_date_GQL(
  trade_date_bsm
)

bsm_bundle <- GaussQuant::bsm_process_bundle_GQL(
  valuation_date = trade_date_bsm,
  spot = spot_price_bsm,
  risk_free_rate = risk_free_rate_bsm,
  dividend_rate = dividend_rate_bsm,
  volatility = volatility_bsm,
  day_counter = day_counter,
  calendar = calendar_bsm
)

GaussQuant::show_tbl_GQH(
  bsm_input_tbl,
  "Black-Scholes-Merton assumptions",
  n = 20
)

#> # A tibble: 7 x 2
#>   input      value
#>   <chr>      <chr>
#> 1 trade_date 2024-03-01
#> 2 maturity   1Y
#> 3 spot_price 100
#> 4 strike_price 100
#> 5 volatility  0.2
#> 6 risk_free_rate 0.05
#> 7 dividend_rate 0
```

European オプションは満期時にのみ行使できるため、Black–Scholes–Merton モデルの解析解

を利用できる。

二項 Tree では、連続時間の価格過程を有限個の時間ステップへ分割する。ステップ数を増やすほど、理論上は連続時間モデルの解析値へ近づく。

ここでは、25、100 および 500 ステップの Cox–Ross–Rubinstein Tree を解析解と比較する。

```

european_put <- GaussQuant::make_european_option_GQL(
  spot = spot_price_bsm,
  strike = strike_price_bsm,
  maturity_date = maturity_date_bsm,
  option_type = "put",
  valuation_date = trade_date_bsm,
  risk_free_rate = risk_free_rate_bsm,
  dividend_yield = dividend_rate_bsm,
  volatility = volatility_bsm,
  day_counter = day_counter,
  calendar = calendar_bsm
)

european_engines <- list(
  crr_25 = QuantLib::BinomialCRRVanillaEngine(
    bsm_bundle$process,
    25L
  ),
  analytic = QuantLib::AnalyticEuropeanEngine(
    bsm_bundle$process
  ),
  crr_100 = QuantLib::BinomialCRRVanillaEngine(
    bsm_bundle$process,
    100L
  ),
  crr_500 = QuantLib::BinomialCRRVanillaEngine(
    bsm_bundle$process,
    500L
  )
)

european_comparison_tbl <- purrr::imap_dfr(
  european_engines,
  function(engine, method) {
    european_put$setPricingEngine(engine)

    tibble::tibble(
      method = method,
      npv = european_put$NPV()
    )
  }
)

```

```

    )
  }
)

analytic_put_npv <- european_comparison_tbl |>
  dplyr::filter(method == "analytic") |>
  dplyr::pull(npv)

european_comparison_tbl <- european_comparison_tbl |>
  dplyr::mutate(
    difference_from_analytic = npv - analytic_put_npv
  )

GaussQuant::show_tbl_GQH(
  european_comparison_tbl,
  "European put engine comparison",
  n = 20
)

#> # A tibble: 4 x 3
#>   method      npv difference_from_analytic
#>   <chr>      <dbl>                <dbl>
#> 1 crr_25    5.6546                0.071974
#> 2 analytic 5.5826                  0
#> 3 crr_100   5.5629               -0.019666
#> 4 crr_500   5.5786               -0.0039372

```

II-4.4 モンテカルロ評価

リスク中立評価では、オプション価格は満期 Payoff の割引期待値として表される。

$$V_0 = D(0, T) E^Q [\max(F_T - K, 0)]$$

Monte Carlo 法では、多数の将来パスを生成し、各パスの満期 Payoff を求め、その平均値を割り引く。

European call には解析解があるため、ここで Monte Carlo 法を使う目的は、解析解に代わる高度な方法を示すことではない。確率過程から生成したパスの期待値が Black 公式の価格へ近づくことを確認し、後に解析解を持たない商品へ応用する準備をすることにある。

```

mc_setting_tbl <- tibble::tribble(
  ~setting, ~value,
  "steps", 12,

```

```

  "paths", 20000,
  "seed", 1
)

mc_path_tbl <- GaussQuant::generate_path_table_GQL(
  process = black_bundle$process,
  maturity = maturity,
  n_steps = 12L,
  n_paths = 200L,
  seed = 1L,
  sequence = "pseudo"
)

mc_result <- GaussQuant::terminal_option_mc_GQL(
  path_tbl = mc_path_tbl,
  strike = strike_price,
  discount_factor = discount_factor,
  option_type = "call"
)

mc_comparison_tbl <- tibble::tribble(
  ~method, ~npv, ~standard_error,
  "Black analytic", analytic_npv, NA_real_,
  "Pseudo-random Monte Carlo", mc_result$summary$npv[[1]],
  ↪ mc_result$summary$standard_error[[1]]
) |>
  dplyr::mutate(
    difference_from_analytic = npv - analytic_npv
  )

GaussQuant::show_tbl_GQH(
  mc_setting_tbl,
  "Monte Carlo settings",
  n = 20
)

#> # A tibble: 3 x 2
#>   setting value
#>   <chr>    <dbl>
#> 1 steps      12
#> 2 paths    20000
#> 3 seed         1

GaussQuant::show_tbl_GQH(
  mc_comparison_tbl,
  "Analytic and Monte Carlo comparison",

```

```

  n = 20
)
#> # A tibble: 2 x 4
#>   method                npv standard_error difference_from_analytic
#>   <chr>                <dbl>         <dbl>                <dbl>
#> 1 Black analytic      0.56160          NA                  0
#> 2 Pseudo-random Monte Carlo 0.47698      0.059721          -0.084615

```

標準誤差は、有限本数のパスを用いたことによる Monte Carlo 推定値の不確実性を表す。解析値との差が標準誤差と同程度であれば、シミュレーション結果は統計的に整合的と判断できる。

Black process では、フォワード価格は対数正規分布に従う。したがって、各パスの価格は常に正であり、満期価格の分布は左右対称にはならない。

次のグラフでは、前節と同じオプション満期までの 120 本のパスと、その満期価格分布を表示する。

```

path_plot_tbl <- GaussQuant::generate_path_table_GQL(
  process = black_bundle$process,
  maturity = maturity,
  n_steps = 12L,
  n_paths = 120L,
  seed = 1L,
  sequence = "pseudo"
)

path_plot <- ggplot2::ggplot(
  path_plot_tbl,
  ggplot2::aes(
    x = time,
    y = price,
    group = path_id
  )
) +
  ggplot2::geom_line(
    alpha = 0.45,
    linewidth = 0.4
  ) +
  ggplot2::labs(
    title = "Black model Monte Carlo paths",
    x = "Year",
    y = "Forward price"
  )

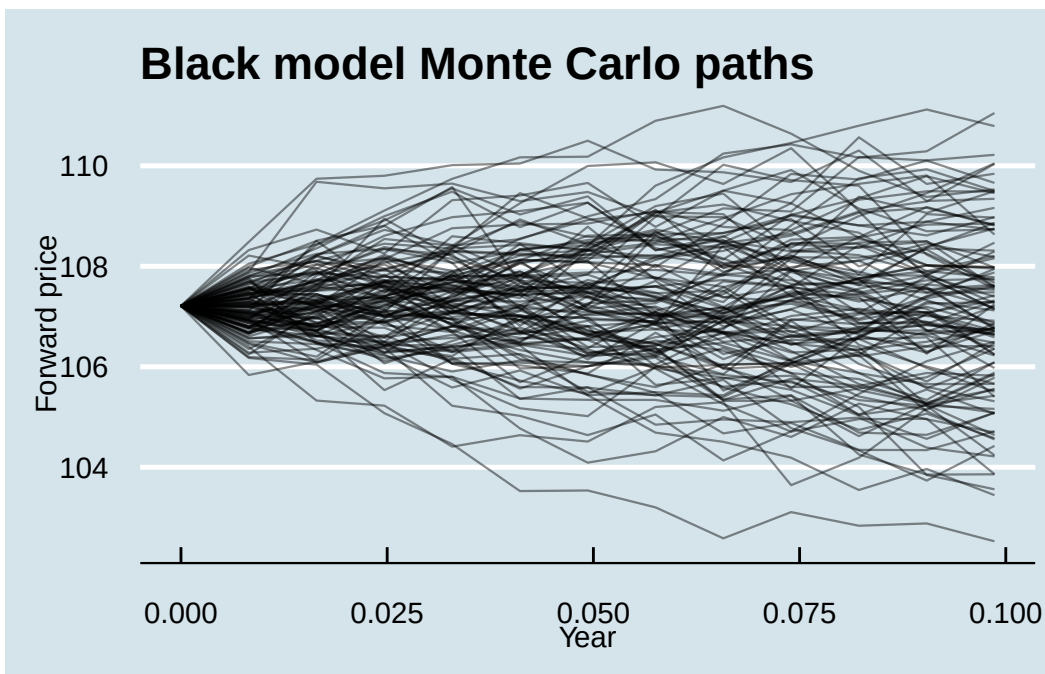
```

```

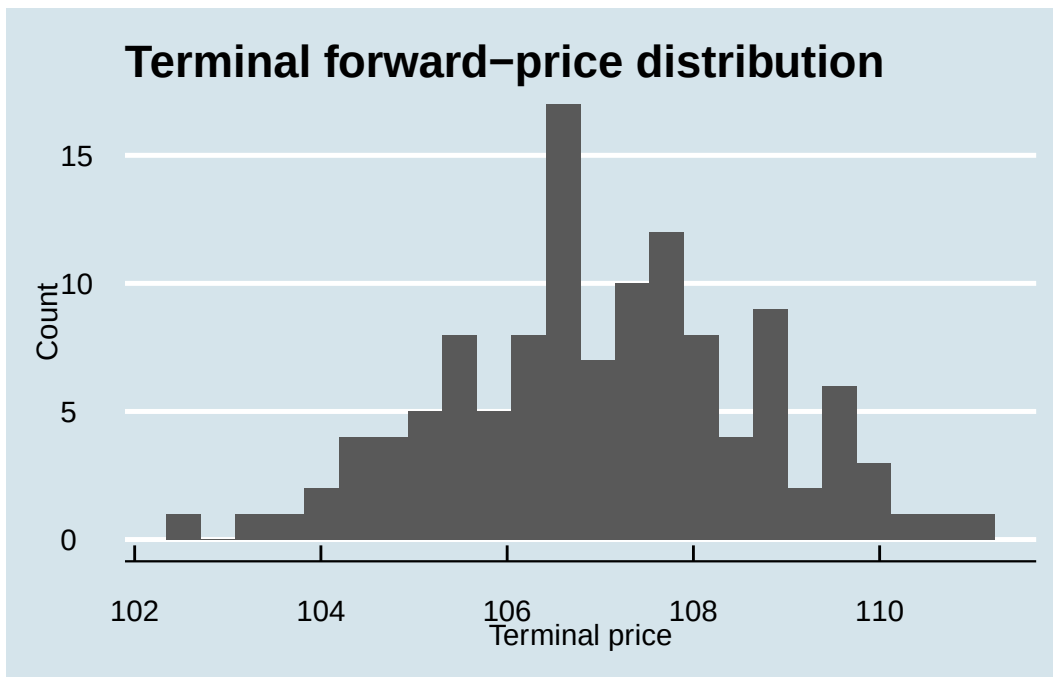
terminal_plot_tbl <- path_plot_tbl |>
  dplyr::group_by(path_id) |>
  dplyr::slice_max(
    order_by = step,
    n = 1L,
    with_ties = FALSE
  ) |>
  dplyr::ungroup()

terminal_plot <- ggplot2::ggplot(
  terminal_plot_tbl,
  ggplot2::aes(x = price)
) +
  ggplot2::geom_histogram(
    bins = 24
  ) +
  ggplot2::labs(
    title = "Terminal forward-price distribution",
    x = "Terminal price",
    y = "Count"
  )
show_plot_LFL(path_plot)

```



```
show_plot_LFL(terminal_plot)
```

Black process のフォワード価格は、時間幅 Δt ごとに次式で更新できる。

$$F_{t+\Delta t} = F_t \exp \left[-\frac{1}{2} \sigma^2 \Delta t + \sigma \sqrt{\Delta t} Z \right]$$

ここで、 Z は標準正規乱数である。フォワード測度の下ではフォワード価格のドリフトがゼロであるため、指数部には Ito 補正項である $-\frac{1}{2} \sigma^2 \Delta t$ が現れる。

QuantLib の PathGenerator が使用する Gaussian increment を取り出し、同じ乱数から手計算したパスと比較する。

```
n_steps_check <- 3L
seed_check <- 1L

generated_path_tbl <- GaussQuant::generate_path_table_GQL(
  process = black_bundle$process,
  maturity = maturity,
  n_steps = n_steps_check,
  n_paths = 1L,
  seed = seed_check,
  sequence = "pseudo"
)

gaussian_rng_check <- QuantLib::GaussianRandomSequenceGenerator(
  QuantLib::UniformRandomSequenceGenerator(
    n_steps_check,
    QuantLib::UniformRandomGenerator(seed_check)
  )
)
```

```

gaussian_increment <- gaussian_rng_check$nextSequence()$value() |>
  as.numeric()

time_grid_check <- generated_path_tbl |>
  dplyr::arrange(step) |>
  dplyr::pull(time)

dt_check <- diff(time_grid_check)

growth_factor <- exp(
  -0.5 * volatility^2 * dt_check +
  volatility * sqrt(dt_check) * gaussian_increment
)

hand_price <- forward_price * c(
  1,
  cumprod(growth_factor)
)

one_path_check_tbl <- generated_path_tbl |>
  dplyr::arrange(step) |>
  dplyr::transmute(
    step = step,
    time = time,
    generated_price = price,
    hand_price = hand_price,
    difference = price - hand_price
  )

GaussQuant::show_tbl_GQH(
  one_path_check_tbl,
  "One-path hand calculation",
  n = 20
)

#> # A tibble: 4 x 5
#>   step    time generated_price hand_price difference
#>   <int>  <dbl>          <dbl>      <dbl>      <dbl>
#> 1     0  0          107.21      107.21      0
#> 2     1 0.032877      106.99      106.99      0
#> 3     2 0.065753      109.82      109.82      0
#> 4     3 0.098630      110.42      110.42 -1.4211e-14

```

差が数値誤差の範囲に収まれば、PathGenerator が Black 過程の離散化式に従っていることを確認できる。

II-4.5 アメリカン put の早期行使価値

American オプションは満期までの任意の時点で行使できる。したがって、その価値は同じ条件の European オプション以上となる。

$$V_{\text{American}} \geq V_{\text{European}}$$

両者の差は早期行使権の価値である。

American put には一般的な閉形式の解析解がないため、二項 Tree または近似法を用いる。本節では、次の方法を比較する。

- 25 ステップ CRR Tree
- Barone–Adesi–Whaley 近似
- Bjerk Sund–Stensland 近似
- 500 ステップ CRR Tree

500 ステップ CRR Tree を比較基準とする。

```
american_put <- GaussQuant::make_american_option_GQL(  
  spot = spot_price_bsm,  
  strike = strike_price_bsm,  
  maturity_date = maturity_date_bsm,  
  option_type = "put",  
  valuation_date = trade_date_bsm,  
  risk_free_rate = risk_free_rate_bsm,  
  dividend_yield = dividend_rate_bsm,  
  volatility = volatility_bsm,  
  day_counter = day_counter,  
  calendar = calendar_bsm,  
  steps = 500L  
)  
  
american_engines <- list(  
  crr_25 = QuantLib::BinomialCRRVanillaEngine(  
    bsm_bundle$process,  
    25L  
  ),  
  barone_adesi_whaley = QuantLib::BaroneAdesiWhaleyApproximationEngine(  
    bsm_bundle$process  
  ),  
  bjerk_sund_stensland = QuantLib::BjerkSundStenslandApproximationEngine(  
    bsm_bundle$process,  
    500L  
  )  
)
```

```

    bsm_bundle$process
  ),
  crr_500 = QuantLib::BinomialCRRVanillaEngine(
    bsm_bundle$process,
    500L
  )
)

american_comparison_tbl <- purrr::imap_dfr(
  american_engines,
  function(engine, method) {
    american_put$setPricingEngine(engine)

    tibble::tibble(
      method = method,
      npv = american_put$NPV()
    )
  }
)

crr_500_american_npv <- american_comparison_tbl |>
  dplyr::filter(method == "crr_500") |>
  dplyr::pull(npv)

american_comparison_tbl <- american_comparison_tbl |>
  dplyr::mutate(
    difference_from_crr_500 = npv - crr_500_american_npv
  )

GaussQuant::show_tbl_GQH(
  american_comparison_tbl,
  "American put engine comparison",
  n = 20
)

#> # A tibble: 4 x 3
#>   method                npv difference_from_crr_500
#>   <chr>                <dbl>                <dbl>
#> 1 crr_25                6.1595                0.058390
#> 2 barone_adesi_whaley  6.1100                0.0089221
#> 3 bjerksund_stensland  5.9951               -0.10604
#> 4 crr_500                6.1011                0

```

II-4.6 準モンテカルロとロングスタッフ – シュワルツ法

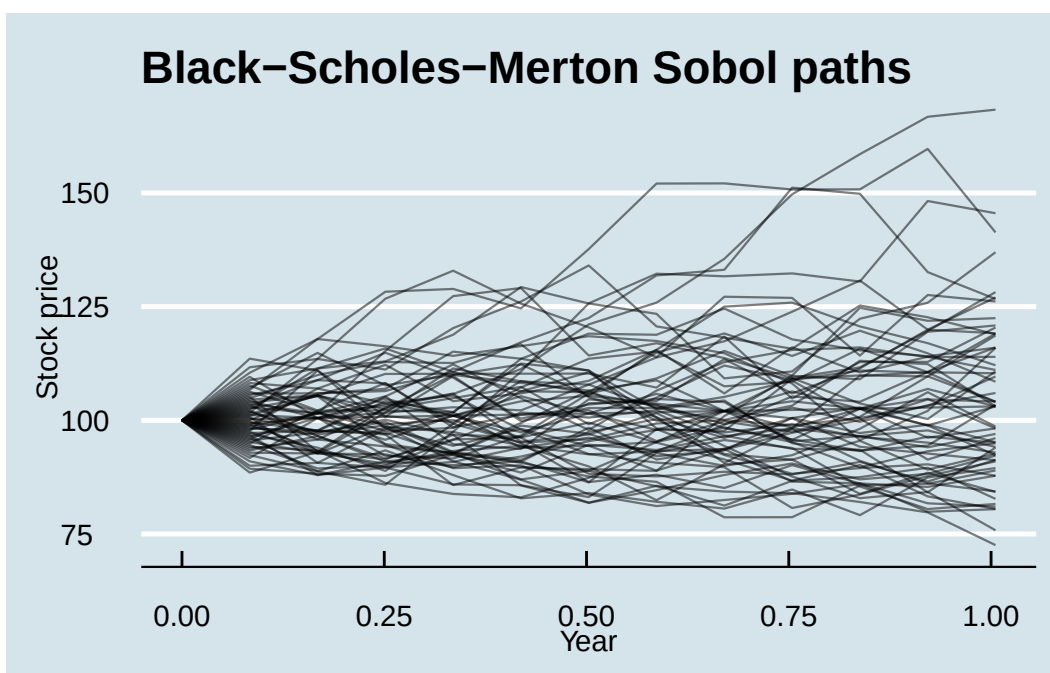
擬似乱数は乱数生成器によって確率的な点列を作る。これに対し Sobol 系列は、積分領域をできるだけ均等に覆うように構成された低乖離列である。

Sobol 系列を用いる準 Monte Carlo 法では、通常の擬似乱数より少ないパスで安定した数値積分が得られる場合がある。ただし、パス同士は独立な確率標本ではないため、通常の標準誤差の解釈をそのまま適用できない点に注意する。

次のコードでは、Black–Scholes–Merton process から 64 本の Sobol パスを生成する。

```
maturity_bsm <- day_counter$yearFraction(  
  trade_date_bsm,  
  maturity_date_bsm  
)  
  
sobol_display_tbl <- GaussQuant::generate_path_table_GQL(  
  process = bsm_bundle$process,  
  maturity = maturity_bsm,  
  n_steps = 12L,  
  n_paths = 64L,  
  seed = 1L,  
  sequence = "sobol"  
)  
  
sobol_plot <- ggplot2::ggplot(  
  sobol_display_tbl,  
  ggplot2::aes(  
    x = time,  
    y = price,  
    group = path_id  
  )  
) +  
  ggplot2::geom_line(  
    alpha = 0.5,  
    linewidth = 0.4  
  ) +  
  ggplot2::labs(  
    title = "Black-Scholes-Merton Sobol paths",  
    x = "Year",  
    y = "Stock price"  
  )
```

```
GaussQuant::show_tbl_GQH(
  sobol_display_tbl,
  "Sobol path table",
  n = 20
)
#> # A tibble: 20 x 4
#>   path_id step    time price
#>   <int> <int>   <dbl> <dbl>
#> 1      1     0 0      100
#> 2      1     1 0.083790 100.25
#> 3      1     2 0.16758 100.50
#> 4      1     3 0.25137 100.76
#> 5      1     4 0.33516 101.01
#> 6      1     5 0.41895 101.26
#> 7      1     6 0.50274 101.52
#> 8      1     7 0.58653 101.78
#> 9      1     8 0.67032 102.03
#> 10     1     9 0.75411 102.29
#> 11     1    10 0.83790 102.55
#> 12     1    11 0.92169 102.80
#> 13     1    12 1.0055 103.06
#> 14     2     0 0      100
#> 15     2     1 0.083790 104.24
#> 16     2     2 0.16758 100.50
#> 17     2     3 0.25137 104.77
#> 18     2     4 0.33516 101.01
#> 19     2     5 0.41895 105.30
#> 20     2     6 0.50274 101.52
show_plot_LFL(sobol_plot)
```



American オプションでは、各行使可能時点において、即時行使価値と保有を継続する価値を比較する必要がある。

put の即時行使価値は次式である。

$$I(S_t) = \max(K - S_t, 0)$$

Longstaff-Schwartz 法では、将来受け取る割引 Cashflow を原資産価格の関数として回帰し、条件付き継続価値を推定する。本章では、次の 2 次多項式を使用する。

$$\widehat{C}(S_t) = \beta_0 + \beta_1 S_t + \beta_2 S_t^2$$

即時行使価値が推定継続価値を上回る場合、その時点で行使する。

本節では、2,048 本の Sobol パスから American put を評価し、500 ステップ CRR Tree と比較する。

```
lsm_path_tbl <- GaussQuant::generate_path_table_GQL(
  process = bsm_bundle$process,
  maturity = maturity_bsm,
  n_steps = 12L,
  n_paths = 2048L,
  seed = 1L,
  sequence = "sobol"
)

lsm_result <- GaussQuant::lsm_american_put_GQL(
  path_tbl = lsm_path_tbl,
  strike = strike_price_bsm,
  risk_free_rate = risk_free_rate_bsm
)

lsm_npv <- lsm_result$summary |>
  dplyr::filter(metric == "lsm_npv") |>
  dplyr::pull(value)

lsm_comparison_tbl <- tibble::tribble(
  ~method, ~npv,
  "Longstaff-Schwartz Sobol", lsm_npv,
  "CRR Tree 500 steps", crr_500_american_npv
) |>
  dplyr::mutate(
    difference_from_crr_500 = npv - crr_500_american_npv
  )
```

```

)

exercise_summary_tbl <- lsm_result$exercise |>
  dplyr::count(
    exercise_step,
    exercise_time,
    name = "paths"
  ) |>
  dplyr::arrange(exercise_step)

GaussQuant::show_tbl_GQH(
  lsm_result$summary,
  "Longstaff-Schwartz summary",
  n = 20
)
#> # A tibble: 4 x 2
#>   metric      value
#>   <chr>      <dbl>
#> 1 lsm_npv      6.2925
#> 2 standard_error 0.16653
#> 3 paths      2048
#> 4 steps       12

GaussQuant::show_tbl_GQH(
  lsm_comparison_tbl,
  "Longstaff-Schwartz and CRR comparison",
  n = 20
)
#> # A tibble: 2 x 3
#>   method      npv difference_from_crr_500
#>   <chr>      <dbl>          <dbl>
#> 1 Longstaff-Schwartz Sobol 6.2925      0.19139
#> 2 CRR Tree 500 steps 6.1011      0

GaussQuant::show_tbl_GQH(
  exercise_summary_tbl,
  "Longstaff-Schwartz exercise timing",
  n = 20
)
#> # A tibble: 11 x 3
#>   exercise_step exercise_time paths
#>   <int>      <dbl> <int>
#> 1         2      0.16758    28
#> 2         3      0.25137    52
#> 3         4      0.33516    57

```



```
#> 4          5      0.41895    109
#> 5          6      0.50274    107
#> 6          7      0.58653    100
#> 7          8      0.67032     87
#> 8          9      0.75411     38
#> 9         10      0.83790     78
#> 10        11      0.92169     96
#> 11        12      1.0055    1296

GaussQuant::show_tbl_GQH(
  lsm_result$diagnostics,
  "Longstaff-Schwartz diagnostics",
  n = 20
)
#> # A tibble: 11 x 6
#>   step      time in_the_money_paths exercised_paths mean_immediate_value
#>   <int>    <dbl>          <int>          <int>          <dbl>
#> 1     11 0.92169           931           575           5.6825
#> 2     10 0.83790           918           459           5.5258
#> 3      9 0.75411           932           355           5.2815
#> 4      8 0.67032           937           392           5.0842
#> 5      7 0.58653           945           338           4.8180
#> 6      6 0.50274           946           282           4.5454
#> 7      5 0.41895           939           193           4.2509
#> 8      4 0.33516           959           105           3.8461
#> 9      3 0.25137           970            69           3.4077
#> 10     2 0.16758           963            28           2.8755
#> 11     1 0.083790          989             0           2.1136
#> # i 1 more variable: mean_realized_continuation <dbl>
```

Longstaff-Schwartz 法の価格は、パス数、時間ステップ、基底関数および回帰誤差に依存する。したがって、二項 Tree との差が完全にゼロになることを目的とするのではなく、早期行使判断を Monte Carlo パス上でどのように近似するかを確認する。

II-4.7 まとめ

Black-76 と Black-Scholes-Merton は、原資産が対数正規分布に従うという共通の仮定を持つ。Black-Scholes-Merton は現物価格から出発し、Black-76 はフォワード価格または先物価格を直接入力する。

Black-76 では、Black 公式による直接計算と、Black process を用いた解析的 Pricing Engine が同じ価格を与えることを確認した。さらに、Monte Carlo 法によって満期 Payoff の割引期待

値を求め、解析値と比較した。

Black–Scholes–Merton モデルでは、European put の解析解と CRR Tree を比較した。American put では、早期行使権を評価するため、近似 Engine、CRR Tree および Longstaff–Schwartz 法を比較した。

解析解、Tree、Monte Carlo および回帰法は、互いに競合する単純な代替手法ではない。Payoff の構造、早期行使の有無、モデルの次元および必要な計算精度に応じて使い分ける必要がある。

Black モデルの対数正規仮定は、価格が正である市場では扱いやすい。一方、金利がゼロまたはマイナスとなる場合には直接適用できない。この問題が、Normal モデルや Shifted Black モデルを用いる理由となる。

```
chapter04_summary_tbl <- tibble::tribble(
  ~section,          ~main_result,
  "Black-76",        "Black calculator and analytic European pricing",
  "Pseudo-random Monte Carlo", "Discounted terminal payoff and standard error",
  "European put",    "Analytic and CRR Tree comparison",
  "American put",    "Approximation engines and CRR Tree comparison",
  "Sobol",           "Low-discrepancy path generation",
  "Longstaff-Schwartz", "Regression-based early-exercise approximation"
)

GaussQuant::show_tbl_GQH(
  chapter04_summary_tbl,
  "Chapter II-4 summary",
  n = 20
)

#> # A tibble: 6 x 2
#>   section          main_result
#>   <chr>          <chr>
#> 1 Black-76      Black calculator and analytic European pricing
#> 2 Pseudo-random Monte Carlo Discounted terminal payoff and standard error
#> 3 European put  Analytic and CRR Tree comparison
#> 4 American put  Approximation engines and CRR Tree comparison
#> 5 Sobol         Low-discrepancy path generation
#> 6 Longstaff-Schwartz Regression-based early-exercise approximation
```

第 II-5 章信用リスクとクレジットデリバティブ

前章までの評価では、将来の市場価格、金利またはボラティリティの変動を扱った。本章では、契約相手または参照企業が債務を履行できなくなる信用リスクを検討する。

Credit Default Swap (CDS) は、参照企業などに信用事由が発生した場合に、プロテクションの売り手が買い手へ損失補償を支払う契約である。プロテクションの買い手は、その対価として契約期間中に定期的な Premium を支払う。

したがって、CDS は二つのレグから構成される。

- Premium Leg：プロテクション買い手が定期的な Spread を支払う。
- Protection Leg：信用事由が発生した場合に、プロテクション売り手が損失を補償する。

プロテクション買い手から見た CDS 価値は、次のように表される。

$$V_{\text{CDS}} = V_{\text{protection}} + V_{\text{premium}}$$

Premium Leg は支払いであるため通常は負、Protection Leg は受取りであるため正となる。

CDS 評価には、割引カーブだけでなく、将来のデフォルト確率を表す Default Probability Curve が必要である。本章では、Hazard Rate から生存確率とデフォルト確率を求め、Premium Leg と Protection Leg を評価する。

まず、フラット割引率とフラット Hazard Rate を使用して、MidPoint 法による CDS 評価と手計算を比較する。次に、標準 CDS2015 契約を ISDA CDS Engine で評価し、Fair spread、Fair upfront および Accrual rebate を確認する。

II-5.1 信用リスクとクレジットデリバティブの基礎

デフォルトが発生する確率時刻を τ とする。時点 t まで企業がデフォルトせずに存続する確率を、生存確率と呼ぶ。

$$Q(t) = P(\tau > t)$$

時点 0 から t までの累積デフォルト確率は、

$$P(\tau \leq t) = 1 - Q(t)$$

である。

Hazard Rate $\lambda(t)$ は、時点 t まで存続しているという条件の下で、その直後にデフォルトする瞬間的な強度を表す。

$$\lambda(t) = \lim_{\Delta t \rightarrow 0} \frac{P(t < \tau \leq t + \Delta t \mid \tau > t)}{\Delta t}$$

Hazard Rate が一定のフラットモデルでは、生存確率は次式となる。

$$Q(t) = \exp(-\lambda t)$$

累積デフォルト確率は、

$$1 - Q(t) = 1 - \exp(-\lambda t)$$

である。

Hazard Rate は、通常の意味での「一年間のデフォルト確率」そのものではない。たとえば Hazard Rate が 2% であっても、1 年累積デフォルト確率は、

$$1 - e^{-0.02}$$

であり、厳密には 2% と一致しない。

デフォルトが発生した場合でも、債権価値のすべてが失われるとは限らない。デフォルト後に回収できる元本比率を Recovery Rate R とする。

損失率 Loss Given Default は、

$$LGD = 1 - R$$

である。

Notional を N とすると、Protection Leg が補償する損失額は概ね、

$$N(1 - R)$$

となる。

Recovery Rate が高いほどデフォルト時の損失は小さくなり、同じデフォルト確率であれば CDS Spread は低くなる。

連続的な Premium 支払い、一定 Hazard Rate、一定 Recovery Rate および単純な割引を仮定すると、CDS Spread s と Hazard Rate λ には次の近似関係がある。

$$s \approx (1 - R)\lambda$$

したがって、

$$\lambda \approx \frac{s}{1 - R}$$

となる。

この近似は CDS の信用水準を理解するために有用である。ただし、実際の評価では、四半期ごとの支払い、Accrual on default、割引、標準日付およびカーブ補間が影響するため、厳密な Hazard Rate とは一致しない。

II-5.2 フラットカーブによるクレジットデリバティブ評価

1 年物 CDS を考える。

- 評価日：2022 年 9 月 19 日
- 満期：2023 年 9 月 20 日

- Notional : 10,000,000
- 割引率 : 10%
- 市場 Spread : 130bp
- 契約 Coupon : 100bp
- Recovery Rate : 40%

市場 Spread は 130bp であるが、契約 Coupon は 100bp である。このため、契約時点の CDS 価値はゼロにならない。

```
trade_date_flat <- GaussQuant::date_GQL("2022-09-19")
maturity_date_flat <- GaussQuant::date_GQL("2023-09-20")

GaussQuant::set_eval_date_GQL(trade_date_flat)

notional_flat <- 10000000
risk_free_rate_flat <- 0.10
market_spread_flat <- 130 / 10000
contract_coupon_flat <- 100 / 10000
recovery_rate_flat <- 0.40

actual_365 <- QuantLib::Actual365Fixed()
actual_360 <- QuantLib::Actual360()
weekend_calendar <- QuantLib::WeekendsOnly()

flat_cds_input_tbl <- tibble::tribble(
  ~input,      ~value,
  "trade_date", "2022-09-19",
  "maturity_date", "2023-09-20",
  "notional",    as.character(notional_flat),
  "risk_free_rate", as.character(risk_free_rate_flat),
  "market_spread", as.character(market_spread_flat),
  "contract_coupon", as.character(contract_coupon_flat),
  "recovery_rate",  as.character(recovery_rate_flat)
)

GaussQuant::show_tbl_GQH(
  flat_cds_input_tbl,
  "Flat CDS input assumptions",
  n = 20
)

#> # A tibble: 7 x 2
#>   input      value
#>   <chr>      <chr>
#> 1 trade_date 2022-09-19
#> 2 maturity_date 2023-09-20
```

```
#> 3 notional      10000000
#> 4 risk_free_rate 0.1
#> 5 market_spread  0.013
#> 6 contract_coupon 0.01
#> 7 recovery_rate  0.4
```

市場 Spread 130bp と Recovery Rate 40% から、フラット Hazard Rate を近似する。

```
hazard_rate_approximation <- market_spread_flat /
  (1 - recovery_rate_flat)

hazard_approximation_tbl <- tibble::tribble(
  ~metric,          ~value,
  "market_spread",  market_spread_flat,
  "recovery_rate",  recovery_rate_flat,
  "loss_given_default", 1 - recovery_rate_flat,
  "hazard_rate_approx", hazard_rate_approximation
)

probability_time_grid <- seq(
  0,
  5,
  by = 0.25
)

flat_probability_tbl <- tibble::tibble(
  time = probability_time_grid
) |>
  dplyr::mutate(
    survival_probability = exp(
      -hazard_rate_approximation * time
    ),
    cumulative_default_probability = 1 -
      survival_probability
  )

flat_probability_plot_tbl <- flat_probability_tbl |>
  tidyr::pivot_longer(
    cols = c(
      survival_probability,
      cumulative_default_probability
    ),
    names_to = "probability_type",
    values_to = "probability"
  )
```

```

flat_probability_plot <- ggplot2::ggplot(
  flat_probability_plot_tbl,
  ggplot2::aes(
    x = time,
    y = probability,
    linetype = probability_type
  )
) +
  ggplot2::geom_line(
    linewidth = 0.7
  ) +
  ggplot2::scale_y_continuous(
    labels = scales::percent_format(
      accuracy = 0.1
    )
  ) +
  ggplot2::labs(
    title = "Flat-hazard survival and default probabilities",
    x = "Time (years)",
    y = "Probability",
    linetype = "Probability"
  )

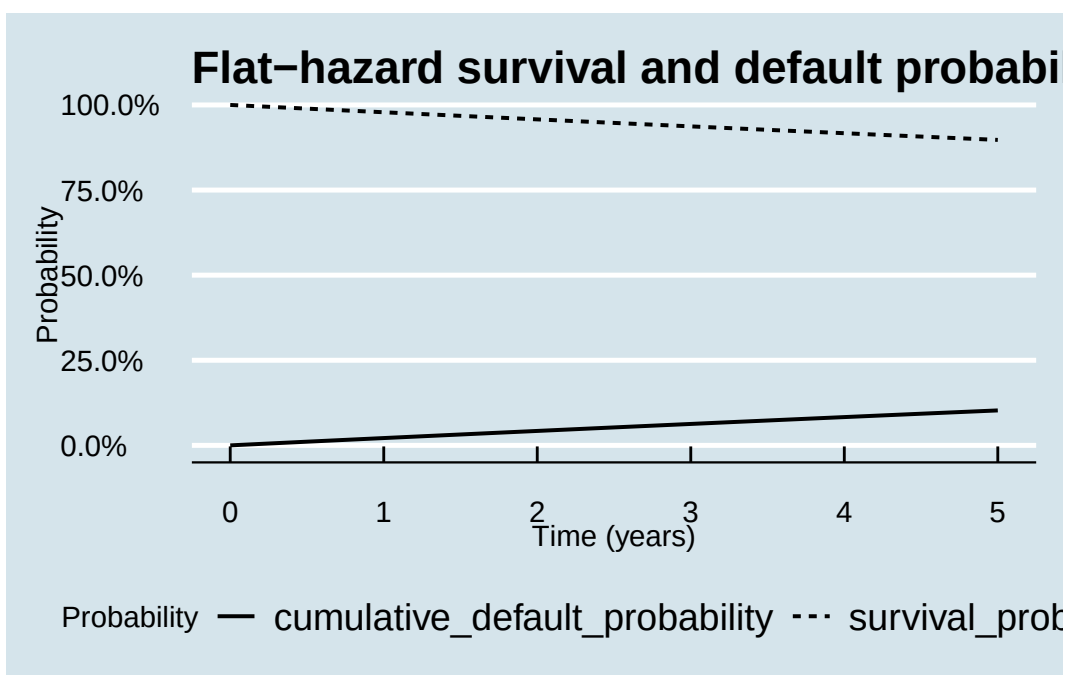
GaussQuant::show_tbl_GQH(
  hazard_approximation_tbl,
  "Flat hazard-rate approximation",
  n = 20
)
#> # A tibble: 4 x 2
#>   metric      value
#>   <chr>      <dbl>
#> 1 market_spread 0.013
#> 2 recovery_rate 0.4
#> 3 loss_given_default 0.6
#> 4 hazard_rate_approx 0.021667

GaussQuant::show_tbl_GQH(
  flat_probability_tbl,
  "Flat-hazard probability curve",
  n = 30
)
#> # A tibble: 21 x 3
#>   time survival_probability cumulative_default_probability
#>   <dbl>          <dbl>          <dbl>

```



```
#> 1 0 1 0
#> 2 0.25 0.99460 0.0054020
#> 3 0.5 0.98923 0.010775
#> 4 0.75 0.98388 0.016119
#> 5 1 0.97857 0.021434
#> 6 1.25 0.97328 0.026720
#> 7 1.5 0.96802 0.031978
#> 8 1.75 0.96279 0.037207
#> 9 2 0.95759 0.042408
#> 10 2.25 0.95242 0.047581
#> # i 11 more rows
show_plot_LFL(flat_probability_plot)
```



Premium は四半期ごとに支払われるものとする。単純な教育用例として、評価日の翌日から満期までの Forward Schedule を作成する。

```
cds_schedule_flat <- QuantLib::Schedule(
  trade_date_flat + 1L,
  maturity_date_flat,
  GaussQuant::period_months_GQL(3),
  weekend_calendar,
  "Following",
  "Unadjusted",
  "Forward",
  FALSE
)

discount_curve_flat <- GaussQuant::flat_curve_GQL(
  reference_date = trade_date_flat,
```

```

    rate = risk_free_rate_flat,
    day_counter = actual_365,
    compounding = "Continuous"
)

discount_curve_handle_flat <- QuantLib::YieldTermStructureHandle(
  discount_curve_flat
)

hazard_curve_flat <- QuantLib::FlatHazardRate(
  trade_date_flat,
  GaussQuant::quote_handle_GQL(
    hazard_rate_approximation
  ),
  actual_365
)

hazard_curve_handle_flat <- QuantLib::DefaultProbabilityTermStructureHandle(
  hazard_curve_flat
)

flat_curve_check_tbl <- tibble::tribble(
  ~metric, ~value,
  "discount_factor_at_maturity", discount_curve_flat$discount(
    maturity_date_flat
  ),
  "survival_probability_at_maturity", hazard_curve_flat$survivalProbability(
    maturity_date_flat
  ),
  "default_probability_to_maturity", 1 -
    hazard_curve_flat$survivalProbability(
      maturity_date_flat
    ),
  "hazard_rate_at_maturity", hazard_curve_flat$hazardRate(
    maturity_date_flat
  )
)

GaussQuant::show_tbl_GQH(
  GaussQuant::schedule_table_GQL(
    cds_schedule_flat
  ),
  "CDS schedule: flat-curve example",
  n = 20
)

```

```
#> # A tibble: 5 x 2
#>   period schedule_date
#>   <int> <date>
#> 1     1 2022-09-20
#> 2     2 2022-12-20
#> 3     3 2023-03-20
#> 4     4 2023-06-20
#> 5     5 2023-09-20

GaussQuant::show_tbl_GQH(
  flat_curve_check_tbl,
  "Flat discount and hazard-curve check",
  n = 20
)
#> # A tibble: 4 x 2
#>   metric                                value
#>   <chr>                                <dbl>
#> 1 discount_factor_at_maturity          0.90459
#> 2 survival_probability_at_maturity 0.97851
#> 3 default_probability_to_maturity    0.021492
#> 4 hazard_rate_at_maturity             0.021667
```

MidPoint 法では、各 Premium 期間中に発生するデフォルトを、その期間の midpoint で発生したものとして近似する。

100bp Coupon の CDS を作成し、GaussQuant の MidPoint CDS Engine で評価する。

```
cds_flat <- QuantLib::CreditDefaultSwap(
  GaussQuant::cds_buyer_side_GQL(),
  notional_flat,
  contract_coupon_flat,
  cds_schedule_flat,
  "Following",
  actual_360
)

midpoint_engine_flat <- GaussQuant::cds_midpoint_engine_GQL(
  probability_handle = hazard_curve_handle_flat,
  recovery_rate = recovery_rate_flat,
  discount_handle = discount_curve_handle_flat
)

cds_flat$setPricingEngine(
  midpoint_engine_flat
)
```

```

#> NULL

cds_flat_summary_tbl <- GaussQuant::cds_summary_GQL(
  cds_flat
)

cds_flat_leg_check_tbl <- tibble::tribble(
  ~metric,          ~value,
  "coupon_leg_npv",  cds_flat$couponLegNPV(),
  "default_leg_npv", cds_flat$defaultLegNPV(),
  "sum_of_two_legs", cds_flat$couponLegNPV() +
    cds_flat$defaultLegNPV(),
  "reported_npv",    GaussQuant::cds_npv_GQL(
    cds_flat
  ),
  "difference",      GaussQuant::cds_npv_GQL(
    cds_flat
  ) -
    (
      cds_flat$couponLegNPV() +
      cds_flat$defaultLegNPV()
    )
)

GaussQuant::show_tbl_GQH(
  cds_flat_summary_tbl,
  "CDS summary: MidPoint engine with flat curves",
  n = 20
)
#> # A tibble: 4 x 2
#>   metric          value
#>   <chr>          <dbl>
#> 1 npv            28116.
#> 2 fair_spread     0.012983
#> 3 coupon_leg_npv -94246.
#> 4 default_leg_npv 122362.

GaussQuant::show_tbl_GQH(
  cds_flat_leg_check_tbl,
  "CDS two-leg NPV check",
  n = 20
)
#> # A tibble: 5 x 2
#>   metric          value
#>   <chr>          <dbl>

```

```
#> 1 coupon_leg_npv -94246.
#> 2 default_leg_npv 122362.
#> 3 sum_of_two_legs 28116.
#> 4 reported_npv 28116.
#> 5 difference 0
```

プロテクション買い手から見て、Premium Leg は負、Protection Leg は正となる。契約 Coupon が Fair Spread より低ければ、プロテクション買い手に有利となり、CDS NPV は一般に正となる。

GaussQuant の `cds_cashflow_table_GQL()` を利用し、各 Premium 期間の割引係数、生存確率、デフォルト確率および期間中点の情報を確認する。

```
cds_cashflow_flat_tbl <- GaussQuant::cds_cashflow_table_GQL(
  cds_schedule = cds_schedule_flat,
  protection_start_date = cds_flat$protectionStartDate(),
  coupon_rate = contract_coupon_flat,
  hazard_curve = hazard_curve_flat,
  discount_curve = discount_curve_flat,
  trade_date = trade_date_flat,
  notional = notional_flat
)

GaussQuant::show_tbl_GQH(
  cds_cashflow_flat_tbl,
  "CDS cash-flow and probability table",
  n = 20
)

#> # A tibble: 5 x 13
#>   payment_date coupon_rate accrual_start accrual_end accrual_days
#>   <chr>          <dbl> <chr>          <chr>          <dbl>
#> 1 2022-09-20      NA    <NA>          2022-09-20      NA
#> 2 2022-12-20      0.01 2022-09-20    2022-12-20      91
#> 3 2023-03-20      0.01 2022-12-20    2023-03-20      90
#> 4 2023-06-20      0.01 2023-03-20    2023-06-20      92
#> 5 2023-09-20      0.01 2023-06-20    2023-09-20      92
#> # i 8 more variables: accrual_year_fraction_365 <dbl>, coupon_amount <dbl>,
#> #   discount_factor <dbl>, survival_probability <dbl>,
#> #   default_probability <dbl>, midpoint_date <chr>,
#> #   midpoint_discount_factor <dbl>, midpoint_survival_probability <dbl>
```

II-5.3 Premium Leg と Protection Leg

Premium Leg は、参照企業が各支払日まで存続した場合に支払われる。

単純化した現在価値は次式で表される。

$$V_{\text{premium}} \approx -Ns \sum_i \alpha_i D(0, t_i) Q(t_i)$$

ここで、 N は Notional、 s は契約 Coupon、 α_i は利息計算期間、 $D(0, t_i)$ は割引係数、 $Q(t_i)$ は生存確率である。

```
premium_leg_hand_flat <- -sum(
  cds_cashflow_flat_tbl$coupon_amount *
  cds_cashflow_flat_tbl$discount_factor *
  cds_cashflow_flat_tbl$midpoint_survival_probability,
  na.rm = TRUE
)
```

この近似では期間中点の生存確率を使用している。実際の CDS では、期間中にデフォルトした場合の経過 Premium である Accrual on default も考慮される。

期間 $(t_{i-1}, t_i]$ にデフォルトする確率を、

$$\Delta PD_i = Q(t_{i-1}) - Q(t_i)$$

とする。

Protection Leg は、各期間の中点 m_i で損失補償が支払われると近似する。

$$V_{\text{protection}} \approx N(1 - R) \sum_i D(0, m_i) \Delta PD_i$$

```
protection_leg_per_unit_hand_flat <- (
  1 - recovery_rate_flat
) *
sum(
  cds_cashflow_flat_tbl$midpoint_discount_factor *
  cds_cashflow_flat_tbl$default_probability,
  na.rm = TRUE
)
```

```
protection_leg_hand_flat <- notional_flat *
  protection_leg_per_unit_hand_flat
```

Risky PV01 は、CDS Spread を 1 単位支払う Premium Leg の現在価値に相当する。

$$RPV01 \approx \sum_i \alpha_i D(0, t_i) Q(t_i)$$

Fair Spread は、Premium Leg と Protection Leg の価値を等しくし、Upfront なしの CDS NPV をゼロにする Running Spread である。

$$s_{\text{fair}} = \frac{V_{\text{protection}}/N}{RPV01}$$

```
risky_pv01_hand_flat <- sum(
  (
    cds_cashflow_flat_tbl$accrual_days /
    360
  ) *
  cds_cashflow_flat_tbl$discount_factor *
  cds_cashflow_flat_tbl$midpoint_survival_probability,
  na.rm = TRUE
)

fair_spread_hand_flat <- protection_leg_per_unit_hand_flat /
  risky_pv01_hand_flat

cds_npv_hand_flat <- premium_leg_hand_flat +
  protection_leg_hand_flat

flat_hand_check_tbl <- tibble::tribble(
  ~metric,          ~hand_value,          ~quantlib_value,
  "coupon_leg_npv", premium_leg_hand_flat, cds_flat$couponLegNPV(),
  "default_leg_npv", protection_leg_hand_flat, cds_flat$defaultLegNPV(),
  "cds_npv",        cds_npv_hand_flat,    GaussQuant::cds_npv_GQL(
    cds_flat
  ),
  "fair_spread",    fair_spread_hand_flat,
  ↪ GaussQuant::cds_fair_spread_GQL(
    cds_flat
  )
) |>
```

```

dplyr::mutate(
  difference = hand_value -
    quantlib_value
)

flat_hand_summary_tbl <- tibble::tribble(
  ~metric,          ~value,
  "risky_pv01",      risky_pv01_hand_flat,
  "protection_leg_per_unit", protection_leg_per_unit_hand_flat,
  "hand_fair_spread", fair_spread_hand_flat,
  "quantlib_fair_spread", GaussQuant::cds_fair_spread_GQL(
    cds_flat
  ),
  "fair_spread_difference", fair_spread_hand_flat -
    GaussQuant::cds_fair_spread_GQL(
      cds_flat
    )
)

GaussQuant::show_tbl_GQH(
  flat_hand_check_tbl,
  "CDS hand calculation and MidPoint engine comparison",
  n = 20
)

#> # A tibble: 4 x 4
#>   metric          hand_value quantlib_value    difference
#>   <chr>          <dbl>          <dbl>          <dbl>
#> 1 coupon_leg_npv -94244.         -94246.         2.0964
#> 2 default_leg_npv 122362.         122362.         0
#> 3 cds_npv        28118.         28116.         2.0964
#> 4 fair_spread     0.012984         0.012983 0.00000028881

GaussQuant::show_tbl_GQH(
  flat_hand_summary_tbl,
  "CDS hand-calculation summary",
  n = 20
)

#> # A tibble: 5 x 2
#>   metric          value
#>   <chr>          <dbl>
#> 1 risky_pv01      0.94244
#> 2 protection_leg_per_unit 0.012236
#> 3 hand_fair_spread    0.012984
#> 4 quantlib_fair_spread 0.012983
#> 5 fair_spread_difference 0.00000028881

```


手計算と MidPoint Engine は近い値となるが、厳密には一致しない。差には Accrual on default、日付規則、Protection 開始日および QuantLib 内部の計算規約が含まれる。

II-5.4 Hazard Rate 推定と感応度

市場 Spread を契約 Coupon として設定した CDS は、契約開始時点で NPV がゼロとなるように Hazard Rate を調整できる。

ここでは、市場 Spread 130bp の CDS について、MidPoint Engine の NPV がゼロとなるフラット Hazard Rate を `uniroot()` で推定する。

```
cds_market_flat <- QuantLib::CreditDefaultSwap(  
  GaussQuant::cds_buyer_side_GQL(),  
  notional_flat,  
  market_spread_flat,  
  cds_schedule_flat,  
  "Following",  
  actual_360  
)  
  
cds_market_npv_at_hazard <- function(hazard_rate) {  
  engine <- GaussQuant::cds_midpoint_engine_GQL(  
    recovery_rate = recovery_rate_flat,  
    hazard_rate = hazard_rate,  
    discount_rate = risk_free_rate_flat,  
    reference_date = trade_date_flat,  
    day_counter = "Actual365Fixed"  
  )  
  
  cds_market_flat$setPricingEngine(  
    engine  
  )  
  
  GaussQuant::cds_npv_GQL(  
    cds_market_flat  
  )  
}  
  
hazard_calibration_result <- stats::uniroot(  
  cds_market_npv_at_hazard,  
  interval = c(  
    1e-8,  
    0.50  
  ),  
)
```

```

  tol = 1e-10
)

hazard_rate_calibrated <- hazard_calibration_result$root

calibrated_midpoint_engine <- GaussQuant::cds_midpoint_engine_GQL(
  recovery_rate = recovery_rate_flat,
  hazard_rate = hazard_rate_calibrated,
  discount_rate = risk_free_rate_flat,
  reference_date = trade_date_flat,
  day_counter = "Actual365Fixed"
)

cds_market_flat$setPricingEngine(
  calibrated_midpoint_engine
)

#> NULL

hazard_calibration_tbl <- tibble::tribble(
  ~metric,          ~value,
  "market_spread",  market_spread_flat,
  "simple_hazard_approximation", hazard_rate_approximation,
  "calibrated_hazard_rate",    hazard_rate_calibrated,
  "zero_npv_check",           GaussQuant::cds_npv_GQL(
                                cds_market_flat
                              ),
  "fair_spread_check",        GaussQuant::cds_fair_spread_GQL(
                                cds_market_flat
                              )
)

GaussQuant::show_tbl_GQH(
  hazard_calibration_tbl,
  "Flat hazard calibration from a zero-NPV CDS",
  n = 20
)

#> # A tibble: 5 x 2
#>   metric                                value
#>   <chr>                                <dbl>
#> 1 market_spread                        0.013
#> 2 simple_hazard_approximation 0.021667
#> 3 calibrated_hazard_rate        0.021695
#> 4 zero_npv_check                0.00000036661
#> 5 fair_spread_check             0.013000

```

単純近似 $\lambda \approx s/(1 - R)$ とカリブレーション結果は近いが、支払頻度と割引の影響により差が生じる。

単純近似の下では、Spread が高いほど Hazard Rate は高くなる。また、同じ Spread で Recovery Rate を高く仮定すると、デフォルト時損失が小さくなるため、その Spread を説明するために必要な Hazard Rate は高くなる。

```
credit_sensitivity_input_tbl <- tidyr::crossing(
  market_spread_bp = c(
    50,
    100,
    200
  ),
  recovery_rate = c(
    0.20,
    0.40,
    0.60
  )
)

credit_sensitivity_tbl <- credit_sensitivity_input_tbl |>
  dplyr::mutate(
    market_spread = market_spread_bp /
      10000,
    loss_given_default = 1 -
      recovery_rate,
    hazard_rate_approx = market_spread /
      loss_given_default,
    survival_probability_5y = exp(
      -5 * hazard_rate_approx
    ),
    cumulative_default_probability_5y = 1 -
      survival_probability_5y
  )

GaussQuant::show_tbl_GQH(
  credit_sensitivity_tbl,
  "Spread and recovery-rate sensitivity",
  n = 30
)

#> # A tibble: 9 x 7
#>   market_spread_bp recovery_rate market_spread loss_given_default
#>   <dbl>           <dbl>         <dbl>           <dbl>
#> 1           50         0.2         0.005           0.8
#> 2           50         0.4         0.005           0.6
```

```
#> 3          50          0.6          0.005          0.4
#> 4         100          0.2          0.01          0.8
#> 5         100          0.4          0.01          0.6
#> 6         100          0.6          0.01          0.4
#> 7         200          0.2          0.02          0.8
#> 8         200          0.4          0.02          0.6
#> 9         200          0.6          0.02          0.4
#> # i 3 more variables: hazard_rate_approx <dbl>, survival_probability_5y <dbl>,
#> #   cumulative_default_probability_5y <dbl>
```

Recovery Rate と Hazard Rate は、CDS Spread だけから完全に独立して識別できない。実務では Recovery Rate を外生的に仮定し、その仮定の下で Default Probability Curve を構築することが多い。

II-5.5 標準 CDS と ISDA カーブ

標準 CDS では、満期を単純に取引日から 5 年後とするのではなく、CDS IMM 日と CDS2015 ルールに従って決定する。

次の 5 年物 CDS を考える。

- 取引日：2022 年 8 月 19 日
- 市場 Spread：30.426bp
- 契約 Coupon：100bp
- Notional：10,000,000
- Recovery Rate：40%
- 外生的な OIS 割引率：0.25%

第 II-3 章では OIS カーブの構築を扱った。本章の目的は信用カーブと CDS 評価であるため、割引カーブの動学やブートストラップを繰り返さず、フラット OIS 割引率を外生的に与える。

```
trade_date_isda <- GaussQuant::date_GQL(
  "2022-08-19"
)

GaussQuant::set_eval_date_GQL(
  trade_date_isda
)

cds_tenor_isda <- GaussQuant::period_years_GQL(
  5L
)
```

```

maturity_date_isda <- QuantLib::cdsMaturity(
  trade_date_isda,
  cds_tenor_isda,
  "CDS2015"
)

market_spread_isda <- 30.426 / 10000
contract_coupon_isda <- 100 / 10000
notional_isda <- 10000000
recovery_rate_isda <- 0.40
ois_discount_rate_isda <- 0.0025

isda_cds_input_tbl <- tibble::tribble(
  ~input,      ~value,
  "trade_date", "2022-08-19",
  "cds_tenor",  "5Y",
  "maturity_date", GaussQuant::iso_GQL(
    maturity_date_isda
  ),
  "market_spread_bp", as.character(
    market_spread_isda *
    10000
  ),
  "contract_coupon_bp", as.character(
    contract_coupon_isda *
    10000
  ),
  "notional", as.character(
    notional_isda
  ),
  "recovery_rate", as.character(
    recovery_rate_isda
  ),
  "ois_discount_rate", as.character(
    ois_discount_rate_isda
  )
)

GaussQuant::show_tbl_GQH(
  isda_cds_input_tbl,
  "ISDA CDS input assumptions",
  n = 20
)

#> # A tibble: 8 x 2

```

```
#>   input      value
#>   <chr>    <chr>
#> 1 trade_date 2022-08-19
#> 2 cds_tenor   5Y
#> 3 maturity_date 2027-06-20
#> 4 market_spread_bp 30.426
#> 5 contract_coupon_bp 100
#> 6 notional    10000000
#> 7 recovery_rate 0.4
#> 8 ois_discount_rate 0.0025
```

CDS2015 ルールにより、Premium 支払日は標準化された四半期日へ調整される。

```
cds_calendar_isda <- QuantLib::WeekendsOnly()

cds_schedule_isda <- QuantLib::Schedule(
  trade_date_isda,
  maturity_date_isda,
  GaussQuant::period_months_GQL(3),
  cds_calendar_isda,
  "Following",
  "Unadjusted",
  "CDS2015",
  FALSE
)

GaussQuant::show_tbl_GQH(
  GaussQuant::schedule_table_GQL(
    cds_schedule_isda
  ),
  "CDS schedule: 5Y CDS2015 example",
  n = 30
)

#> # A tibble: 21 x 2
#>   period schedule_date
#>   <int> <date>
#> 1     1 2022-06-20
#> 2     2 2022-09-20
#> 3     3 2022-12-20
#> 4     4 2023-03-20
#> 5     5 2023-06-20
#> 6     6 2023-09-20
#> 7     7 2023-12-20
#> 8     8 2024-03-20
#> 9     9 2024-06-20
```

```
#> 10      10 2024-09-20
#> # i 11 more rows
```

```
discount_curve_isda <- GaussQuant::flat_curve_GQL(
  reference_date = trade_date_isda,
  rate = ois_discount_rate_isda,
  day_counter = actual_365,
  compounding = "Continuous"
)

discount_curve_handle_isda <- QuantLib::YieldTermStructureHandle(
  discount_curve_isda
)

isda_discount_check_tbl <- tibble::tribble(
  ~metric, ~value,
  "ois_discount_rate", ois_discount_rate_isda,
  "discount_factor_at_maturity", discount_curve_isda$discount(
    maturity_date_isda
  )
)

GaussQuant::show_tbl_GQH(
  isda_discount_check_tbl,
  "ISDA discount-curve check",
  n = 20
)

#> # A tibble: 2 x 2
#>   metric value
#>   <chr>   <dbl>
#> 1 ois_discount_rate 0.0025
#> 2 discount_factor_at_maturity 0.98798
```

市場 Spread 30.426bp を Running Spread とする標準 CDS を作成し、ISDA Engine の NPV がゼロとなるフラット Hazard Rate を推定する。

```
market_cds_isda <- GaussQuant::make_cds_GQL(
  maturity_date = maturity_date_isda,
  running_spread = market_spread_isda,
  notional = notional_isda,
  side = "buyer",
  trade_date = trade_date_isda,
  coupon_tenor = "3M",
  day_counter = "Actual360",
```

```

    date_generation_rule = "CDS2015"
  )

isda_market_npv_at_hazard <- function(hazard_rate) {
  probability_handle <- GaussQuant::flat_hazard_rate_GQL(
    hazard_rate = hazard_rate,
    reference_date = trade_date_isda,
    day_counter = "Actual365Fixed"
  )

  engine <- GaussQuant::cds_isda_engine_GQL(
    hazard_curve_handle = probability_handle,
    recovery_rate = recovery_rate_isda,
    discount_curve_handle = discount_curve_handle_isda
  )

  market_cds_isda$setPricingEngine(
    engine
  )

  GaussQuant::cds_npv_GQL(
    market_cds_isda
  )
}

isda_hazard_calibration_result <- stats::uniroot(
  isda_market_npv_at_hazard,
  interval = c(
    1e-8,
    0.50
  ),
  tol = 1e-10
)

hazard_rate_isda <- isda_hazard_calibration_result$root

hazard_curve_isda <- QuantLib::FlatHazardRate(
  trade_date_isda,
  GaussQuant::quote_handle_GQL(
    hazard_rate_isda
  ),
  actual_365
)

hazard_curve_handle_isda <- QuantLib::DefaultProbabilityTermStructureHandle(

```



```

  hazard_curve_isda
)

isda_hazard_check_tbl <- tibble::tribble(
  ~metric,          ~value,
  "market_spread",  market_spread_isda,
  "recovery_rate",  recovery_rate_isda,
  "implied_hazard_rate", hazard_rate_isda,
  "simple_hazard_approximation", market_spread_isda /
    (
      1 -
      recovery_rate_isda
    ),
  "survival_probability_at_maturity",
  hazard_curve_isda$survivalProbability(
    maturity_date_isda
  ),
  "default_probability_to_maturity",
  1 -
  hazard_curve_isda$survivalProbability(
    maturity_date_isda
  )
)

GaussQuant::show_tbl_GQH(
  isda_hazard_check_tbl,
  "ISDA implied hazard-rate check",
  n = 20
)

#> # A tibble: 6 x 2
#>   metric                                value
#>   <chr>                                <dbl>
#> 1 market_spread                        0.0030426
#> 2 recovery_rate                        0.4
#> 3 implied_hazard_rate                  0.0051398
#> 4 simple_hazard_approximation          0.005071
#> 5 survival_probability_at_maturity    0.97544
#> 6 default_probability_to_maturity    0.024562

```

市場 Spread と Recovery Rate から得られる単純近似と、ISDA Engine でゼロ NPV となる Hazard Rate には差が生じる。標準 CDS の日付、Accrual on default、割引および数値積分が評価へ反映されるためである。

II-5.6 ISDA Engine と評価比較

契約 Coupon を標準 Coupon 100bp へ固定した 5 年物 CDS を評価する。

市場 Spread は 30.426bp であり、契約 Coupon 100bp より低い。プロテクション買い手は市場水準より高い Premium を支払うため、Running Coupon だけで評価すると一般に負の NPV となる。

```
isda_engine <- GaussQuant::cds_isda_engine_GQL(
  hazard_curve_handle = hazard_curve_handle_isda,
  recovery_rate = recovery_rate_isda,
  discount_curve_handle = discount_curve_handle_isda
)

cds_isda <- GaussQuant::make_cds_GQL(
  maturity_date = maturity_date_isda,
  running_spread = contract_coupon_isda,
  notional = notional_isda,
  side = "buyer",
  trade_date = trade_date_isda,
  coupon_tenor = "3M",
  day_counter = "Actual360",
  date_generation_rule = "CDS2015",
  pricing_engine = isda_engine
)

cds_isda_summary_base_tbl <- GaussQuant::cds_summary_GQL(
  cds_isda
)

GaussQuant::show_tbl_GQH(
  cds_isda_summary_base_tbl,
  "ISDA CDS base summary",
  n = 20
)

#> # A tibble: 4 x 2
#>   metric      value
#>   <chr>      <dbl>
#> 1 npv      -3.3496e+5
#> 2 fair_spread  3.0426e-3
#> 3 coupon_leg_npv -4.9839e+5
#> 4 default_leg_npv  1.4649e+5
```

標準 CDS では、取引日が前回の標準 Coupon 日と次回 Coupon 日の間にある場合、契約開始時に Accrual rebate が発生することがある。

この場合、プロテクション買い手から見た NPV は、

$$V_{\text{CDS}} = V_{\text{coupon}} + V_{\text{default}} + V_{\text{accrual rebate}}$$

となる。

```
coupon_leg_npv_isda <- cds_isda$couponLegNPV()
default_leg_npv_isda <- cds_isda$defaultLegNPV()

accrual_rebate_npv_isda <- tryCatch(
  as.numeric(
    cds_isda$accrualRebateNPV()
  ),
  error = function(e) NA_real_
)

three_component_sum_isda <- coupon_leg_npv_isda +
  default_leg_npv_isda +
  accrual_rebate_npv_isda

isda_leg_check_tbl <- tibble::tribble(
  ~metric,          ~value,
  "coupon_leg_npv", coupon_leg_npv_isda,
  "default_leg_npv", default_leg_npv_isda,
  "accrual_rebate_npv", accrual_rebate_npv_isda,
  "three_component_sum", three_component_sum_isda,
  "reported_npv",     GaussQuant::cds_npv_GQL(
                        cds_isda
                      ),
  "difference",       GaussQuant::cds_npv_GQL(
                        cds_isda
                      ) -
                        three_component_sum_isda
)

GaussQuant::show_tbl_GQH(
  isda_leg_check_tbl,
  "ISDA CDS NPV decomposition",
  n = 20
)

#> # A tibble: 6 x 2
```

```
#>   metric      value
#>   <chr>      <dbl>
#> 1 coupon_leg_npv -498392.
#> 2 default_leg_npv 146485.
#> 3 accrual_rebate_npv 16944.
#> 4 three_component_sum -334963.
#> 5 reported_npv -334963.
#> 6 difference      0
```

Fair Spread は、Upfront を授受せずに CDS NPV をゼロとする Running Spread である。

Fair Upfront は、契約 Coupon を標準 Coupon へ固定したまま、契約価値を調整するために開始時点で授受する Notional 比率である。

```
fair_spread_isda <- GaussQuant::cds_fair_spread_GQL(
  cds_isda
)

fair_upfront_isda <- tryCatch(
  as.numeric(
    cds_isda$fairUpfront()
  ),
  error = function(e) NA_real_
)

fair_upfront_amount_isda <- fair_upfront_isda *
  notional_isda

isda_cds_summary_tbl <- tibble::tribble(
  ~metric,      ~value,
  "coupon_leg_npv",      coupon_leg_npv_isda,
  "default_leg_npv",      default_leg_npv_isda,
  "accrual_rebate_npv",      accrual_rebate_npv_isda,
  "cds_npv",      GaussQuant::cds_npv_GQL(
    cds_isda
  ),
  "market_spread",      market_spread_isda,
  "contract_coupon",      contract_coupon_isda,
  "fair_spread",      fair_spread_isda,
  "fair_upfront",      fair_upfront_isda,
  "fair_upfront_amount",      fair_upfront_amount_isda,
  "hazard_rate",      hazard_rate_isda
)

GaussQuant::show_tbl_GQH(
```

```

isda_cds_summary_tbl,
"CDS summary: ISDA engine",
n = 30
)
#> # A tibble: 10 x 2
#>   metric          value
#>   <chr>          <dbl>
#> 1 coupon_leg_npv -4.9839e+5
#> 2 default_leg_npv 1.4649e+5
#> 3 accrual_rebate_npv 1.6944e+4
#> 4 cds_npv        -3.3496e+5
#> 5 market_spread   3.0426e-3
#> 6 contract_coupon 1      e-2
#> 7 fair_spread     3.0426e-3
#> 8 fair_upfront    -3.3497e-2
#> 9 fair_upfront_amount -3.3497e+5
#> 10 hazard_rate    5.1398e-3

```

この例では、契約 Coupon が市場 Spread より高いため、プロテクション買い手は市場より多くの Premium を支払う。したがって、契約開始時の価値を調整する Upfront が必要となる。

Fair Upfront の正負は、QuantLib における Buyer / Seller の符号規約と契約設定に依存する。符号だけで判断せず、CDS NPV、契約 Coupon、Fair Spread および Upfront 金額を一緒に確認する必要がある。

標準 CDS2015 Schedule、ISDA 整合 Hazard Curve および割引カーブから、Premium 期間ごとの確率情報を作成する。

```

cds_cashflow_isda_tbl <- GaussQuant::cds_cashflow_table_GQL(
  cds_schedule = cds_schedule_isda,
  protection_start_date = cds_isda$protectionStartDate(),
  coupon_rate = contract_coupon_isda,
  hazard_curve = hazard_curve_isda,
  discount_curve = discount_curve_isda,
  trade_date = trade_date_isda,
  notional = notional_isda
)

GaussQuant::show_tbl_GQH(
  cds_cashflow_isda_tbl,
  "CDS cash-flow table: CDS2015 example",
  n = 30
)
#> # A tibble: 21 x 13

```

```

#>   payment_date coupon_rate accrual_start accrual_end accrual_days
#>   <chr>          <dbl> <chr>          <chr>          <dbl>
#> 1 2022-08-19      NA    <NA>          2022-08-19      NA
#> 2 2022-09-20      0.01 2022-06-20    2022-09-20      92
#> 3 2022-12-20      0.01 2022-09-20    2022-12-20      91
#> 4 2023-03-20      0.01 2022-12-20    2023-03-20      90
#> 5 2023-06-20      0.01 2023-03-20    2023-06-20      92
#> 6 2023-09-20      0.01 2023-06-20    2023-09-20      92
#> 7 2023-12-20      0.01 2023-09-20    2023-12-20      91
#> 8 2024-03-20      0.01 2023-12-20    2024-03-20      91
#> 9 2024-06-20      0.01 2024-03-20    2024-06-20      92
#> 10 2024-09-20     0.01 2024-06-20    2024-09-20      92
#> # i 11 more rows
#> # i 8 more variables: accrual_year_fraction_365 <dbl>, coupon_amount <dbl>,
#> #   discount_factor <dbl>, survival_probability <dbl>,
#> #   default_probability <dbl>, midpoint_date <chr>,
#> #   midpoint_discount_factor <dbl>, midpoint_survival_probability <dbl>

```

この表は CDS の経済構造を理解するための MidPoint 型分解である。ISDA Engine は、標準日付、Accrual on default、数値積分および曲線補間をより厳密に扱うため、この表から得られる単純近似と完全には一致しない。

ISDA 用キャッシュフロー表から、Premium Leg、Protection Leg、RPV01 および Fair Spread を簡易計算する。

```

premium_leg_hand_isda <- -sum(
  cds_cashflow_isda_tbl$coupon_amount *
  cds_cashflow_isda_tbl$discount_factor *
  cds_cashflow_isda_tbl$midpoint_survival_probability,
  na.rm = TRUE
)

protection_leg_per_unit_hand_isda <- (
  1 -
  recovery_rate_isda
) *
sum(
  cds_cashflow_isda_tbl$midpoint_discount_factor *
  cds_cashflow_isda_tbl$default_probability,
  na.rm = TRUE
)

protection_leg_hand_isda <- notional_isda *
  protection_leg_per_unit_hand_isda

```

```

risky_pv01_hand_isda <- sum(
  (
    cds_cashflow_isda_tbl$accrual_days /
    360
  ) *
  cds_cashflow_isda_tbl$discount_factor *
  cds_cashflow_isda_tbl$midpoint_survival_probability,
  na.rm = TRUE
)

fair_spread_hand_isda <- protection_leg_per_unit_hand_isda /
  risky_pv01_hand_isda

cds_npv_hand_isda <- premium_leg_hand_isda +
  protection_leg_hand_isda +
  accrual_rebate_npv_isda

isda_hand_check_tbl <- tibble::tribble(
  ~metric,          ~hand_value,          ~isda_value,
  "coupon_leg_npv", premium_leg_hand_isda,    coupon_leg_npv_isda,
  "default_leg_npv", protection_leg_hand_isda, default_leg_npv_isda,
  "cds_npv",        cds_npv_hand_isda,      GaussQuant::cds_npv_GQL(
                                cds_isda
                                ),
  "fair_spread",    fair_spread_hand_isda,    fair_spread_isda
) |>
  dplyr::mutate(
    difference = hand_value -
      isda_value
  )

GaussQuant::show_tbl_GQH(
  isda_hand_check_tbl,
  "MidPoint-style approximation and ISDA comparison",
  n = 20
)

#> # A tibble: 4 x 4
#>   metric          hand_value isda_value  difference
#>   <chr>          <dbl>      <dbl>      <dbl>
#> 1 coupon_leg_npv -4.9811e+5 -4.9839e+5  280.16
#> 2 default_leg_npv 1.4649e+5  1.4649e+5   0.19448
#> 3 cds_npv        -3.3468e+5 -3.3496e+5  280.35
#> 4 fair_spread     2.9408e-3  3.0426e-3 -0.00010178

```

同じ Hazard Curve、Recovery Rate、割引カーブおよび標準 Coupon を使用し、MidPoint Engine と ISDA Engine の評価差を確認する。

```
midpoint_engine_isda_inputs <- GaussQuant::cds_midpoint_engine_GQL(
  probability_handle = hazard_curve_handle_isda,
  recovery_rate = recovery_rate_isda,
  discount_handle = discount_curve_handle_isda
)

cds_midpoint_standard <- GaussQuant::make_cds_GQL(
  maturity_date = maturity_date_isda,
  running_spread = contract_coupon_isda,
  notional = notional_isda,
  side = "buyer",
  trade_date = trade_date_isda,
  coupon_tenor = "3M",
  day_counter = "Actual360",
  date_generation_rule = "CDS2015",
  pricing_engine = midpoint_engine_isda_inputs
)

midpoint_standard_summary_tbl <- GaussQuant::cds_summary_GQL(
  cds_midpoint_standard
) |>
  dplyr::mutate(
    engine = "MidPoint"
  )

isda_standard_summary_tbl <- GaussQuant::cds_summary_GQL(
  cds_isda
) |>
  dplyr::mutate(
    engine = "ISDA"
  )

engine_comparison_tbl <- dplyr::bind_rows(
  midpoint_standard_summary_tbl,
  isda_standard_summary_tbl
) |>
  dplyr::relocate(
    engine
  ) |>
  tidyr::pivot_wider(
    names_from = engine,
    values_from = value
  )
```



```

) |>
dplyr::mutate(
  difference_isda_minus_midpoint = ISDA -
    MidPoint
)

GaussQuant::show_tbl_GQH(
  engine_comparison_tbl,
  "MidPoint and ISDA CDS engine comparison",
  n = 20
)

#> # A tibble: 4 x 4
#>   metric      MidPoint      ISDA difference_isda_minus_midpoint
#>   <chr>      <dbl>      <dbl>      <dbl>
#> 1 npv      -3.3495e+5 -3.3496e+5      -1.1669e+1
#> 2 fair_spread 3.0427e-3 3.0426e-3      -7.6555e-8
#> 3 coupon_leg_npv -4.9838e+5 -4.9839e+5      -1.1474e+1
#> 4 default_leg_npv 1.4649e+5 1.4649e+5      -1.9448e-1

```

MidPoint 法は、各 Premium 期間内のデフォルト時点を期間中点へ集約する。ISDA Engine は標準市場規約に合わせて、より詳細な日付処理と積分を行う。そのため、同じカーブを用いても Leg NPV と Fair Spread に差が生じる。

II-5.7 まとめ

CDS は、Premium Leg と Protection Leg から構成される。プロテクション買い手は Premium を支払い、信用事由が発生した場合に損失補償を受け取る。

Hazard Rate は、企業がその時点まで存続しているという条件の下での瞬間的なデフォルト強度である。フラット Hazard Rate では、生存確率は次式で表される。

$$Q(t) = e^{-\lambda t}$$

Recovery Rate R を仮定すると、CDS Spread と Hazard Rate には、

$$\lambda \approx \frac{s}{1 - R}$$

という近似関係がある。ただし、厳密な Hazard Rate は、Premium 支払日、割引、Accrual on default および標準 CDS 規約を反映して推定する。

Premium Leg の単位 Spread 当たり現在価値が Risky PV01 であり、Fair Spread は Protection Leg の単位 Notional 価値を Risky PV01 で割って求められる。

MidPoint Engine は、デフォルト時点を各期間の midpoint で近似するため、CDS 評価の構造を理解しやすい。ISDA Engine は、標準 CDS2015 の日付、Accrual rebate、Accrual on default および市場規約をより厳密に扱う。

標準 Coupon と市場 Spread が異なる場合、Running Coupon だけでは契約価値がゼロにならない。この差を契約開始時に調整するものが Fair Upfront である。

```
chapter08_summary_tbl <- tibble::tribble(
  ~section, ~main_result,
  "Hazard Rate", "Survival and cumulative default probabilities",
  "MidPoint CDS", "Premium and protection leg decomposition",
  "Hazard calibration", "Zero-NPV market CDS",
  "CDS2015", "Standard maturity and coupon schedule",
  "ISDA CDS", "Fair spread, upfront, and accrual rebate",
  "Engine comparison", "MidPoint approximation versus ISDA pricing"
)

GaussQuant::show_tbl_GQH(
  chapter08_summary_tbl,
  "Chapter II-8 summary",
  n = 20
)

#> # A tibble: 6 x 2
#>   section main_result
#>   <chr> <chr>
#> 1 Hazard Rate Survival and cumulative default probabilities
#> 2 MidPoint CDS Premium and protection leg decomposition
#> 3 Hazard calibration Zero-NPV market CDS
#> 4 CDS2015 Standard maturity and coupon schedule
#> 5 ISDA CDS Fair spread, upfront, and accrual rebate
#> 6 Engine comparison MidPoint approximation versus ISDA pricing

cat(
  "\nChapter II-8 Credit risk and CDS completed successfully.\n"
)

#>
#> Chapter II-8 Credit risk and CDS completed successfully.
```

第 II-6 章金利モデルのキャリブレーション

金利モデルを決めるとき、最初に市場から得られるのはモデルの「テスト」ではなく、割引カーブ、キャップやスワップションの価格、そこから逆算されたボラティリティである。モデルのパラメータを選び、これらの市場価格を再現する作業を**キャリブレーション**という。

前章までに、割引カーブと Black 型のオプション評価を扱った。本章では、まず Normal 型を含めて市場クォートの単位をそろえて「AP」がスマイルに適合させる。次に Hull-White の平均回帰速度と短期金利ボラティリティと、オプション価格から推定する。最後に BGM (Brace-Gatarek-Musiela) モデルのキャリブレーションで必要になるフォワード金利間の相関を考える。

HJM は瞬間フォワード曲線の無裁定な動きを定義するフレームワークだが、瞬間フォワードのボラティリティ曲面そのものは市場で直接クォートされない。本章では、市場で取引される Caplet や Swaption の価格からボラティリティを効率的に使う。HJM のドリフト制約とフォワード金利の動きは第 7 章で扱う。

同じ「市場への適合」でも、何に合わせるかによって、パラメータが表すものは異なる。SABR は主に**行使金利がスマイル**、Hull-White は**満期・原資産年限方向の金利変動**、BGM は**複数フォワード金利のボラティリティと共変動**を記述する。これらをひとまとめの最小二乗問題として扱う前に、各市場クォートの意味を確定する必要がある。

```
source(file.path("R", "utility_LFL.R"))
suppressPackageStartupMessages(library(tidyverse))

show_calibration_tbl_LRL <- function(tbl, title) {
  if (requireNamespace("GaussQuant", quietly = TRUE)) {
    GaussQuant::show_tbl_GQH(tbl, title)
  } else {
    print(tbl)
  }
  invisible(tbl)
}
```

}

II-6.1 市場価格からモデルパラメータへ

以下の四つを区別する。

入力	例	キャリブレーションにおける役割
契約条件	行使日、金利期間、行使金利、 想定元本	評価対象を定義する
割引・予測カーブ	OIS 割引係数、期間別フォワード金利	無リスク金利と市場状態
オプション市場	Caplet、Cap、Swaption 格またはボラティリティ	評価対象のパラメータを合致させる対象
モデル	SABR の α, β, ρ 、 Hull-White の a, σ	推定するパラメータ

現代のマルチカーブ評価では、フォワードカーブを作るための割引カーブが同じとは限らない。市場ボラティリティだけを移して、割引カーブの期間別割引率を変えれば、同じモデルパラメータでも価格は変わる。以下の数値例では、各列のカーブと市場クォートを明示する。

市場商品の価格を V^{mkt} 、パラメータ θ で評価した価格を $V_i(\theta)$ とする。たとえば重み付き価格誤差を

$$J(\theta) = \sum_{i=1}^n w_i [V_i(\theta) - V_i^{\text{mkt}}]^2$$

として最小化する。市場ボラティリティへ直接適合させる場合は、価格の代わりに同じ定義・単位のボラティリティを評価する。

重み w_i の選び方によって、価格の大きな商品が結果を支配するか、相対的な誤差をそろえるかが変わる。最適化関数が終了したことと、市場を十分に再現したことは別である。適合後は、パラメータの範囲、市場クォートごとの残差、未使用商品の再評価を確認する。

II-6.2 Black 型と Normal 型のボラティリティ

Black のボラティリティ σ_B は金利の相対的な変化を表し、Normal のボラティリティ σ_N は金利と同じ絶対変化の単位を持つ。例えば Black 40% は 0.40、Normal 40bp は 0.0040 であり、数値が違って両方とも同じ商品の市場価格を表し得る。

欧州型オプションの符号を $\omega = 1$ (コール) または -1 (プット)、フォワードを F 、行使金利を K 、行使までの年数を T 、割引と想定元本等をまとめた正の係数を D とする。Normal 型の価格は

$$V_N = D \left[\omega(F - K)N(\omega d) + \sigma_N \sqrt{T} n(d) \right] \quad \text{where } d = \frac{F - K}{\sigma_N \sqrt{T}}$$

である。 F や K が負でも定義できる。Black 型は正の F を必要とし、負金利を含む場合には Normal 型または共通のシフトを加える Shifted Black が候補となる。

```
normal_option_price_LRL <- function(
  forward,
  strike,
  volatility,
  maturity,
  discount_factor = 1,
  option_sign = 1
) {
  intrinsic <- discount_factor * max(
    option_sign * (forward - strike), 0
  )
  standard_deviation <- volatility * sqrt(maturity)

  if (standard_deviation == 0) return(intrinsic)

  d <- (forward - strike) / standard_deviation
  discount_factor * (
    option_sign * (forward - strike) *
      stats::pnorm(option_sign * d) +
      standard_deviation * stats::dnorm(d)
  )
}

tbl_volatility_units <- tibble::tibble(
  market_label = c("Black 40%", "Normal 40bp"),
  model_input = c(0.40, 40 / 10000),
```

```

  meaning = c("相対変動の年率", "金利の絶対変動の年率")
)

show_calibration_tbl_LRL(
  tbl_volatility_units,
  "Black・Normalボラティリティの単位"
)

#> # A tibble: 2 x 3
#>   market_label model_input meaning
#>   <chr>          <dbl> <chr>
#> 1 Black 40%      0.4   相対変動の年率
#> 2 Normal 40bp    0.004 金利の絶対変動の年率

```

市場データの整理では、価格とボラティリティを同じ単位に統一する必要がある。右側のボラティリティを bp で受け取った場合は、評価前に 10000 をかけて小数点へ変換する。

II-6.3 SABR によるスマイルキャリブレーション

SABR は、フォワード F_t とそのボラティリティの年率を確率変数として扱う。実務で利用する近似インプライド・ボラティリティは、以下のように表す。

$$\sigma_{\text{impl}}(K) = \sigma(F_t, K, \alpha; \beta, \nu, \rho)$$

である。 α は主に ATM 付近の水準、 β は金利水準に対する拡散係数の依存、 ν はボラティリティの変動率、 ρ はスマイルの傾斜と関係する。ただし互いに独立ではなく、一組の市場データだけで四つを同時に推定することはできない。以下は $\beta = 0.5$ を固定して残り三つを推定する。

市場クォータ・データは SABR 章にある説明用の 7 点を再利用する。単一満期 2 年、フォワード 0.56% であり、ここでは市場実勢値と主張しない。

```

tbl_sabr_market <- tribble(
  ~strike_pct, ~black_vol_pct,
  0.06, 90.78,
  0.31, 46.09,
  0.56, 45.30,
  0.81, 50.17,
  1.06, 53.85,
  1.56, 58.42,
  2.56, 62.72
) |>
dplyr::mutate(

```

```

    strike = .data$strike_pct / 100,
    black_volatility = .data$black_vol_pct / 100
  )

sabr_forward <- 0.0056
sabr_maturity <- 2
sabr_beta <- 0.5

show_calibration_tbl_LRL(
  tbl_sabr_market,
  "SABRの説明用市場クオート"
)
#> # A tibble: 7 x 4
#>   strike_pct black_vol_pct strike black_volatility
#>   <dbl>      <dbl> <dbl>      <dbl>
#> 1     0.06      90.78 0.0006      0.90
#> 2     0.31      46.09 0.0031      0.460
#> 3     0.56      45.3  0.0056      0.453
#> 4     0.81      50.17 0.0081      0.5017
#> 5     1.06      53.85 0.0106      0.5385
#> 6     1.56      58.42 0.0156      0.5842
#> 7     2.56      62.72 0.0256      0.6272

```

市場 Black ボラティリティと、同じ期間の SABR ボラティリティとの二乗誤差を最小化する。 $\alpha > 0$ 、 $\nu \geq 0$ 、 $-1 < \rho < 1$ 、 $\sigma > 0$ の範囲で、値関数が非有限を返す場合は棄却する。近似式の計算には既存の GaussQuant 関数を利用する。

```

if (!requireNamespace("GaussQuant", quietly = TRUE)) {
  stop("この例は GaussQuant パッケージが必要です。")
}

compute_sabr_black_volatility_LRL <- function(
  strike,
  parameters
) {
  GaussQuant::safe_sabr_vol_GQL(
    strike = strike,
    forward = sabr_forward,
    maturity = sabr_maturity,
    alpha = parameters[["alpha"]],
    beta = sabr_beta,
    volvol = parameters[["volvol"]],
    rho = parameters[["rho"]]
  )
}

```

```

}

sabr_objective_LRL <- function(parameters) {
  names(parameters) <- c("alpha", "volvol", "rho")

  model_vol <- tryCatch(
    purrr::map_dbl(
      tbl_sabr_market$strike,
      \(strike) compute_sabr_black_volatility_LRL(
        strike, parameters
      )
    ),
    error = \(e) rep(NA_real_, nrow(tbl_sabr_market))
  )

  if (any(!is.finite(model_vol))) return(1e6)

  mean((model_vol - tbl_sabr_market$black_volatility) ^ 2)
}

sabr_fit <- stats::optim(
  par = c(alpha = 0.1, volvol = 0.05, rho = 0.1),
  fn = sabr_objective_LRL,
  method = "L-BFGS-B",
  lower = c(1e-4, -0.999),
  upper = c(1, 5, 0.1),
  control = list(maxiter = 1000)
)

sabr_parameters <- stats::setNames(
  sabr_fit$par, c("alpha", "volvol", "rho")
)

tbl_sabr_fit <- tbl_sabr_market |>
  dplyr::mutate(
    fitted_volatility = purrr::map_dbl(
      .data$strike,
      \(strike) compute_sabr_black_volatility_LRL(
        strike, sabr_parameters
      )
    ),
    residual_bp_of_vol = 10000 *
      (.data$fitted_volatility - .data$black_volatility)
  )

```



```
tbl_sabr_parameters <- tibble::tibble(
  parameter = c("alpha", "beta_fixed", "volvol", "rho", "RMSE", "convergence"),
  value = c(
    sabr_parameters[["alpha"]], sabr_beta,
    sabr_parameters[["volvol"]], sabr_parameters[["rho"]],
    sqrt(sabr_fit$value), sabr_fit$convergence
  )
)
```

```
show_calibration_tbl_LRL(tbl_sabr_parameters, "SABR推定結果")
```

```
#> # A tibble: 6 x 2
#>   parameter      value
#>   <chr>         <dbl>
#> 1 alpha         0.030583
#> 2 beta_fixed    0.5
#> 3 volvol        0.72849
#> 4 rho           0.44938
#> 5 RMSE          0.013881
#> 6 convergence   0
```

```
show_calibration_tbl_LRL(tbl_sabr_parameters, "SABR推定結果", "クォートの再評価")
```

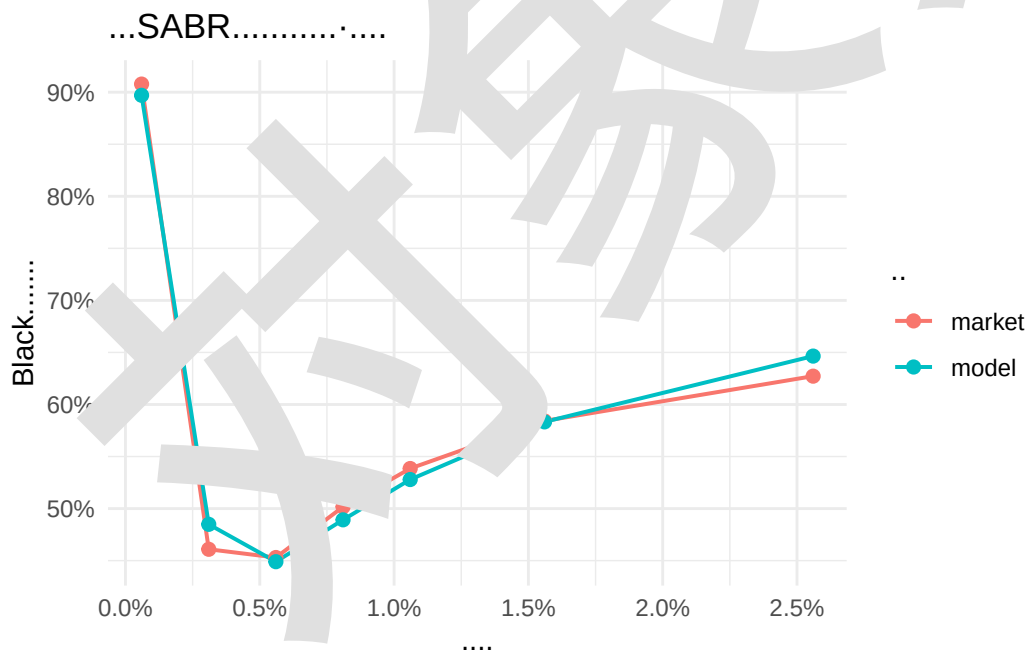
```
#> # A tibble: 7 x 6
#>   strike_pct black_vol_pct strike_pct_volatility fitted_volatility
#>   <dbl>      <dbl> <dbl>      <dbl>
#> 1 0.06      90.7 0.0001 0.9078 0.89701
#> 2 0.31      46.09 0.0031 0.4609 0.48487
#> 3 0.56      23.3 0.005 0.233 0.44897
#> 4 0.81      17 0.0081 0.17 0.48919
#> 5 1.0       15.5 0.0106 0.155 0.52789
#> 6 1.5       11.5 0.0156 0.115 0.58328
#> 7 2.5       7.5 0.025 0.075 0.64648
#> #> # 1 more variable: residual_volatility <dbl>
```

```
tbl_sabr_plot <- tbl_sabr_fit |>
  dplyr::select(
    .data$strike,
    market = .data$black_volatility,
    model = .data$fitted_volatility
  ) |>
  tidyr::pivot_longer(
    cols = c("market", "model"),
    names_to = "series",
    values_to = "volatility"
  )
```

```
plot_sabr_smile <- ggplot2::ggplot(
```

```
tbl_sabr_plot,
ggplot2::aes(
  x = .data$strike,
  y = .data$volatility,
  colour = .data$series
)
) +
ggplot2::geom_line(linewidth = 0.7) +
ggplot2::geom_point(size = 2) +
ggplot2::scale_x_continuous(labels = scales::label_percent()) +
ggplot2::scale_y_continuous(labels = scales::label_percent()) +
ggplot2::labs(
  title = "市場とSABRモデルのボラティリティ・スマイル",
  x = "行使金利",
  y = "Blackボラティリティ",
  colour = "系列"
) +
ggplot2::theme_minimal()

print(plot_sabr_smile)
```



負の行使金利を含む場合、通常の対数正規 SABR の Black 近似にそのまま代入できない。市場が Normal ボラティリティでクォートされているなら、Normal 型の近似式とその単位に合わせて目的関数を作る。Shifted SABR を使う場合には、すべての評価対象について $F + s > 0$ 、 $K + s > 0$ を満たすシフト s を選び、その値をクォートの仕様として記録する。

負金利を含む場合にも、この原則に従ってクォートとモデル価格の定義をそろえる。Black のボラティリティ残差と Normal のボラティリティ残差を、数値の大小だけで比較してはならない。

II-6.4 Hull-White のパラメータ推定

Hull-White 1 因子モデルは

$$dr_t = \{\theta(t) - ar_t\}dt + \sigma dW_t$$

と書ける。 $\theta(t)$ は初期割引カーブとの整合を担い、 a は平均回帰速度、 σ は短期金利の拡散係数を表す。初期カーブを再現することと、オプション市場へ a, σ を合わせることは異なる工程である。動学と初期カーブの再現条件は次章で導く。

スワップションの市場点が一つしかなければ、通常は a を固定して σ を推定する。本節ではまず解析式で検算できるゼロクーポン債オプションを用いて、二つのパラメータの識別性を確認する。その後でスワップション市場への拡張を説明する。

説明用に初期ゼロ金利 r_0 が一定とし、オプション満期を S 、原資産（ゼロクーポン債）満期を $T > S$ とする。 $P(0, t) = e^{-r_0 t}$ 、行使価格を K とするコールの価格

$$C = P(0, S)N(h) - KP(0, T)N(h - \frac{K - P(0, T)}{v}) + \frac{v}{2},$$

$$h = \frac{\sigma}{a}(1 - e^{-aS})\sqrt{\frac{1 - e^{-2aT}}{2a}}$$

で求められる。ここで v はオプション満期までに蓄積される債券価格の標準偏差である。次の関数は $a > 0, \sigma > 0$ の範囲で計算する。

```
compute_hw_bond_option_price_LRL <- function(
  option_maturity,
  bond_maturity,
  strike,
  mean_reversion,
  short_rate_volatility,
  initial_zero_rate
) {
  stopifnot(
    option_maturity > 0,
    bond_maturity > option_maturity,
    strike > 0,
    mean_reversion > 0,
```

```

    short_rate_volatility > 0
  )

  discount_option <- exp(-initial_zero_rate * option_maturity)
  discount_bond <- exp(-initial_zero_rate * bond_maturity)

  bond_std_dev <- short_rate_volatility /
    mean_reversion *
    (-expm1(-mean_reversion *
      (bond_maturity - option_maturity))) *
    sqrt(
      -expm1(-2 * mean_reversion * option_maturity) /
      (2 * mean_reversion)
    )

  h <- log(
    discount_bond / (strike * discount_option)
  ) / bond_std_dev + bond_std_dev / 2

  discount_bond * stats::pnorm(h)
  strike * discount_option
  stats::pnorm(h - bond_std_dev)
}

```

以下の価格は、 $a = 0.001$ 、 $\sigma = 0.012$ から作成した「明」の合成データである。最初の3点で推定し、最後の1点は適合せず再評価する。合成データの逆算はコードの検算であり、実市場での推定精度を意味しない。

```

hw_initial_zero_rate <- 0.03
hw_reference_a <- 0.08
hw_reference_b <- 0.012

tbl_bond_option_market <- tibble::tribble(
  ~option_maturity, ~bond_maturity, ~strike_multiplier,
  0.5, 2, 0.99,
  1.0, 3, 1.00,
  2.0, 5, 1.01,
  3.0, 7, 1.00
) |>
dplyr::mutate(
  forward_bond_price = exp(
    -hw_initial_zero_rate *
      (.data$bond_maturity - .data$option_maturity)
  ),

```

```

strike = .data$forward_bond_price *
  .data$strike_multiplier,
market_price = purrr::pmap_dbl(
  list(
    .data$option_maturity,
    .data$bond_maturity,
    .data$strike
  ),
  \(option_maturity, bond_maturity, strike) {
    compute_hw_bond_option_price_LRL(
      option_maturity = option_maturity,
      bond_maturity = bond_maturity,
      strike = strike,
      mean_reversion = hw_reference_a,
      short_rate_volatility = hw_reference_sig,
      initial_zero_rate = hw_initial_zero_rate
    )
  }
),
sample = c("fit", "fit", "fit", "fit")
)

hw_fit_objective_LRL <- function(parameters) {
  tbl_fit <- tbl_bond_option %>%
    dplyr::filter(sample == "fit")

  fitted <- purrr::map_dbl(
    list(
      tbl_fit$option_maturity,
      tbl_fit$bond_maturity,
      tbl_fit$strike
    ),
    \(option_maturity, bond_maturity, strike) {
      compute_hw_bond_option_price_LRL(
        option_maturity = option_maturity,
        bond_maturity = bond_maturity,
        strike = strike,
        mean_reversion = parameters[[1]],
        short_rate_volatility = parameters[[2]],
        initial_zero_rate = hw_initial_zero_rate
      )
    }
  )
}

```

元本1あたりの価格差を1万倍して数値最適化を安定させる。

```

# 最小値の位置は変わらない。
sum((10000 * (fitted - tbl_fit$market_price))^2)
}

hw_fit <- stats::optim(
  par = c(a = 0.05, sigma = 0.01),
  fn = hw_fit_objective_LRL,
  method = "L-BFGS-B",
  lower = c(0.001, 0.0001),
  upper = c(1, 0.1),
  control = list(maxit = 1000, factr = 1e3)
)

tbl_hw_repricing <- tbl_bond_option_market |>
  dplyr::mutate(
    model_price = purrr::pmap_dbl(
      list(
        .data$option_maturity,
        .data$bond_maturity,
        .data$strike
      ),
      \(option_maturity, bond_maturity, strike)
        compute_hw_bond_option_price(
          option_maturity = option_maturity,
          bond_maturity = bond_maturity,
          strike = strike,
          mean_reversion = hw_fit$par["a"],
          short_rate_volatility = hw_fit$par[["sigma"]],
          initial_zero_rate = initial_zero_rate
        )
    ),
    price_error = .data$model_price -
      .data$market_price
  )

tbl_hw_parameters <- tibble::tibble(
  parameter = c("a", "sigma", "objective", "convergence"),
  estimate = c(
    hw_fit$par[["a"]],
    hw_fit$par[["sigma"]],
    hw_fit$value,
    hw_fit$convergence
  )
)

```

```

show_calibration_tbl_LRL(tbl_hw_parameters, "Hull-Whiteパラメータ推定")
#> # A tibble: 4 x 2
#>   parameter estimate
#>   <chr>          <dbl>
#> 1 a              0.077479
#> 2 sigma          0.011933
#> 3 objective      0.034808
#> 4 convergence    0

show_calibration_tbl_LRL(tbl_hw_repricing, "適合点と未使用点の再評価")
#> # A tibble: 4 x 9
#>   option_maturity bond_maturity strike_multiplier forward_bond_price strike
#>   <dbl>          <dbl>          <dbl>          <dbl>      <dbl>
#> 1 0.5            2            0.99          0.941644
#> 2 1              3            1            0.94176
#> 3 2              5            1.0          0.91395
#> 4 3              7            1            0.88692
#> # i 4 more variables: market_price <dbl>, sample_size <dbl>, model_price <dbl>,
#> #   price_error <dbl>

```

実市場のスワップションに移る場合は、相対利、固定・変動レグの履行、Normal または Shifted Black というボラティリティの表示を、オプションの満期とスワップ年限の価格へ適合させる。QuantLib の `Swaption` の入力クォートの `volatility type` とモデル価格の評価エンジンに対応させる。市場データが揃えば、 a を固定して σ のみを推定し、二つとも求めたとはならない。

II-6.5 BGM モデルと Brigo-Mercurio への接続

BGM は、単一の年利金利ではなく、期間のフォワード金利 $L_i(t)$ をまとめて確率的に扱う市場モデルである。各年利 Caplet から個別のボラティリティについて情報を得られる一方、複数のフォワード金利（またがる Swaption は、金利間の相関にも依存する）。

Brigo-Mercurio は独立した一つのモデル名ではなく、金利モデルの理論と市場適合、とりわけ LIBOR 市場モデルのボラティリティと相関の校正を体系化した文献を指す。本節では BGM の完全なシミュレーションを作らず、**キャリブレーションでどの情報が不足するかを可視化する**。

フォワード金利の変動係数を σ_i 、共通の評価時間を T とする。スワップレートを局所的に固定ウェイト w_i の和で近似すると、その分散率の近似は

$$\sigma_S^2 \approx \sum_{i,j} w_i w_j \sigma_i \sigma_j \rho_{ij}.$$

これは説明用の**固定ウェイト近似**であり、BGMにおける測度変換・ドリフト・ボラティリティの期間依存や真のスワップション価格を実装した式ではない。

$\rho_{ij} = \exp[-\kappa|T_i - T_j|]$ と置くと、同じ個別ボラティリティでも κ によってスワップレート
の分散が変わる。ここでも合成クォートを使い、相関パラメータだけを逆算する。

```
compute_bgm_swap_vol_proxy_LRL <- function(
  correlation_decay,
  tenor,
  caplet_volatility,
  weights
) {
  correlation <- exp(
    -correlation_decay *
    abs(outer(tenor, tenor, "-"))
  )
  weighted_volatility <- weights * caplet_volatility
  sqrt(as.numeric(
    t(weighted_volatility) %*%
    correlation %*% weighted_volatility
  ))
}

bgm_tenor <- 1:4
bgm_caplet_vol <- c(0.24, 0.22, 0.1, 0.2)
bgm_weights <- c(0.1, 0.1, 0.24, 0.2)
bgm_reference_decay <- 0.15

bgm_swap_vol_proxy <-
  compute_bgm_swap_vol_proxy_LRL(
    correlation_decay = bgm_reference_decay,
    tenor = bgm_tenor,
    caplet_volatility = bgm_caplet_vol,
    weights = bgm_weights
  )

bgm_fit <- stats::optimize(
  f = \(correlation_decay) {
    (
      compute_bgm_swap_vol_proxy_LRL(
        correlation_decay = correlation_decay,
        tenor = bgm_tenor,
        caplet_volatility = bgm_caplet_vol,
        weights = bgm_weights
      ) - bgm_swap_vol_proxy
    )
  }
```



```

    )^2
  },
  interval = c(0.01, 2)
)

tbl_bgm_proxy <- tibble::tibble(
  input = c(
    "Caplet volatility 1", "Caplet volatility 2",
    "Caplet volatility 3", "Caplet volatility 4",
    "Swaption volatility proxy", "Fitted correlation decay"
  ),
  value = c(
    bgm_caplet_vol,
    bgm_swaption_vol_proxy,
    bgm_fit$minimum
  )
)

show_calibration_tbl_LRL(
  tbl_bgm_proxy,
  "BGM校正の説明用・固定ウェイト"
)
#> # A tibble: 6 x 2
#>   input                                     value
#>   <chr>                                <dbl>
#> 1 Caplet volatility 1                   0.24
#> 2 Caplet volatility 2                   0.22
#> 3 Caplet volatility 3                   0.21
#> 4 Caplet volatility 4                   0.2
#> 5 Swaption volatility proxy 0.1322
#> 6 Fitted correlation decay 0.0000

```

実際の BGM 校正では、Caplet と Swaption の市場データ、必要なら複数行使価格のスマイルを同時に考える。相関行列が正定値となるパラメータ化、ボラティリティの満期構造、カーブとの整合、近似評価と Monte Carlo 評価との差を検証する。SABR に合わせた各行使金利のスマイルが、自動的に BGM の全期間・全商品へ整合するわけではない。モデル動学と評価測度は次章で扱う。

II-6.6 キャリブレーションサマリー

確認項目	SABR	Hull-White	BGM
主な入力	行使金利別のオプション・ボラティリティ	満期・スワップ年限別のオプション価格	Caplet、Swaption、 関連の情報
主な推定対象	α, ν, ρ (必要なら β)	a, σ	フォワード・ボラティリティと関連構造
よくある不足	β を含むパラメータの識別	少数市場点からの同時推定	Caplet だけでは相関を識別できない
検証	スマイルと市場点外の形	複数満期の価格と未使用商品	Cap と Swaption の同時再評価

最適化の成功だけでモデル採用を決めない。入力の単位、相関関数、パラメータの識別性、市場クォートごとの残差、未使用商品の価格を合わせて確認する。モデルが市場価格に合っているとしても、早期償還商品の将来金利シナリオや行使期間が適切とは限らない。

本章では、金利モデルの校正を「市場クォートを含む価格またはボラティリティの評価式を定める」「制約付きで推定する」「市場点と未使用商品を再評価する」という順序で整理した。Normal モデルを独立した商品別に説明と再評価する代わりに、市場ボラティリティの定義と負金利への対応として位置づけた。SABR のスマイル、Hull-White のオプション市場、BGM の複数フォワード金利市場では、校正対象がそれぞれ異なる。

第 II-7 章では、Hull-White の短期金利と BGM を含むフォワードレートモデルの**時間発展**を学ぶ。第 II-8 章では、そこで得た金利シナリオを使い、Callable Bond の発行体コール権を評価する。

第 11-7 章 ショートレートモデルとフォワード金利モデル

第 6 章では市場のオプション価格を使ってモデルのパラメータを推定し、そのパラメータを受け取ったモデルが何を状態変数として動かしたかによって金利の経路を作るかを考える。第 8 章の早期償還評価では、各行使時点の継続価値を、これらの金利経路のもとで比べる。

短期金利を直接動かす Hull-White、瞬間フォワード金利の曲線全体を動かす HJM、決められた期間のフォワード金利を動かす Black-Karasinski-Merton などのスワップシフトに合わせていても状態変数と計算方法が違う。さらに「日利」金利を期間中読み上げる後決め複利は、従来の期間開始時に確定する LIBOR と区別がつけられない。

以下の数値は仕組みを確認するための架空の入力である。第 6 章の説明用キャリブレーション結果でも、実取引の評価には使えない。

```
suppressPackageStartupMessages(library(tidyverse))

show_rate_model_tbl_LKL = function(tbl, title) {
  if (requireNamespace("GaussQuar", quietly = TRUE)) {
    GaussQuar::tbl_GQH(tbl, title)
  } else {
    print(tbl)
  }
  invisible(tbl)
}
```

11-7.1 短期金利とフォワード金利

時点 t の満期 T のゼロクーポン債価格を $P(t, T)$ とする。割引曲線が十分なめらかなら、瞬間フォワード金利は $f(t, T) = -\partial_T \log P(t, T)$ 、短期金利は $r_t = f(t, t)$ である。単一曲線の設定に限り、 $[T_i, T_{i+1}]$ の単利フォワード金利は

$$L_i(t) = \frac{1}{\Delta_i} \left(\frac{P(t, T_i)}{P(t, T_{i+1})} - 1 \right), \quad \Delta_i = \text{yearfrac}(T_i, T_{i+1}), \quad t \leq T_i.$$

枠組み	主に動かす量	自然な用途	注意点
Hull-White 1 因子	短期金利 r_t	割引、債券オプション、行使判断	単一因子では曲線変動の形に制約がある
HJM	瞬間フォワード曲線 $f(t, T)$	曲線全体の整合的な記述	ボラティリティに応じたドリフト制約が必要
BGM/LMM	期間別フォワード $L_i(t)$	Caplet、Swaption と複数金利の共存	古典的な定義は期間別フォワード金利を想定する
RFR 向け FMM	後決め複利金利に対応する市場フォワード	翌日物金利、後決め複利を含む契約	金利期間中の確定済み部分と未確定部分を分ける

実務の複数曲線設定では、OIS の割引係数と測対象の短期金利を同一の $P(t, T)$ から機械的に作らない。以降、単一曲線を仮定する数式にその旨を明示する。

II-7.2 短期金利 Vasicek から Hull-White へ

定数の長期利率に回帰する Vasicek モデルは

$$dr_t = a(b - r_t) dt + \sigma dW_t, \quad a > 0$$

と書ける。Hull-White 1 因子モデルではドリフトを時間依存にして

$$dr_t = [\theta(t) - ar_t]dt + \sigma dW_t$$

とし、所与の初期割引曲線 $P(0, T)$ を再現する。 a は年率の平均回帰速度、 σ は短期金利の絶対変動の年率パラメータである。Gaussian モデルなので、短期金利が負になる確率もある。

計算には $r_t = \phi(t) + x_t$ と分解すると便利である。 $x_0 = 0$ として

$$dx_t = -ax_t dt + \sigma dW_t.$$

一定の a, σ と十分なめらかさを仮定したとき、初期瞬間フォワード $f(0, t)$ に対応する決定論的な項は

$$\phi(t) = f(0, t) + \frac{\sigma^2}{2a^2}(1 - e^{-at})^2.$$

したがって初期曲線が $P(0, T) = e^{-r_0 T}$ のフラットな連続複利曲線なら、上式の $f(0, t)$ は r_0 になる。後のコードではこのフラット曲線だけを使う。

刻み h における OU 状態の厳密な条件付き遷移は

$$x_{t+h} = e^{-ah} x_t + \sigma \sqrt{\frac{1 - e^{-2ah}}{2a}} Z, \quad Z \sim N(0, 1).$$

これは**状態の遷移**に関して厳密である。一方、割引率関数 $\exp(-\int_0^T r_s ds)$ を左端の値による和で近似すると、時間離散化誤差は生じる。

```
compute_hw_shift_LRL <- function(time, flat_rate, mean_reversion, volatility) {
  stopifnot(mean_reversion >= 0, volatility >= 0, all(time >= 0))
  flat_rate + volatility^2 / (2 * mean_reversion) *
    (1 - exp(-mean_reversion * time))^2
}

simulate_hw_shifts_LRL <- function(
  flat_rate, mean_reversion, volatility,
  horizon = 5, steps_per_year = 24, number_of_paths = 8000L,
  seed = 6177L) {
  stopifnot(
    mean_reversion >= 0, volatility >= 0,
    horizon > 0, steps_per_year > 0, number_of_paths > 1
  )
  number_of_steps <- as.integer(horizon * steps_per_year)
  stopifnot(abs(number_of_steps / steps_per_year - horizon) < 1e-10)
  step_size <- horizon / number_of_steps
  times <- seq(0, horizon, length.out = number_of_steps + 1L)
  set.seed(seed)
  shocks <- matrix(
    stats::rnorm(number_of_paths * number_of_steps),
    nrow = number_of_paths
```

```

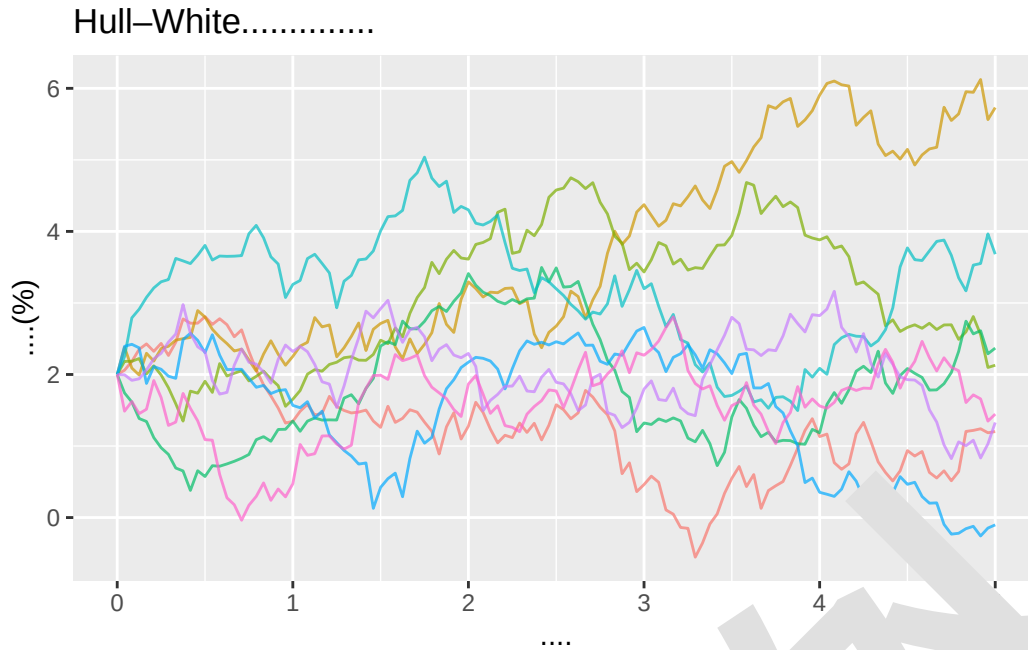
)
decay <- exp(-mean_reversion * step_size)
shock_scale <- volatility * sqrt(
  -expm1(-2 * mean_reversion * step_size) / (2 * mean_reversion)
)
states <- do.call(
  cbind,
  purrr::accumulate(
    seq_len(number_of_steps),
    .init = rep(0, number_of_paths),
    .f = function(previous, index) {
      decay * previous + shock_scale * shocks[, index]
    }
  )
)
rates <- sweep(
  states, 2,
  compute_hw_shift_LRL(times, flat_rate, mean_reversion, volatility),
  FUN = "+"
)
list(times = times, rates = rates, step_size = step_size)
}

hw_example <- simulate_hw_paths(
  flat_rate = 0.02, mean_reversion = 0.12, volatility = 0.01
)

tbl_hw_sample_paths <- tibble::tibble(
  time = hw_example$times, nch = 8,
  path = rep(seq_len(8), times = length(hw_example$times)),
  short_rate = as.vector(hw_example$rates[seq_len(8), ])
)

ggplot2::ggplot(
  tbl_hw_sample_paths,
  ggplot2::aes(time, 100 * short_rate, group = path, colour = factor(path))
) +
  ggplot2::geom_line(alpha = 0.7, show.legend = FALSE) +
  ggplot2::labs(
    title = "Hull-Whiteモデルによる短期金利の経路例",
    x = "経過年数", y = "短期金利 (%)"
  )

```



各経路の近似割引係数を平均し、初期曲線と比較する。平均値の誤差にはモンテカルロ誤差と積分の離散化誤差の両方が入るため、標準誤差だけで完全な比較を判定できない。

```
compute_hw_discount_check_LRL <- function(simulation, flat_rate, maturities) {
  step_size <- simulation$step_size
  maturity_steps <- as.integer(round(maturities / step_size))
  stopifnot(
    all(maturity_steps > 0),
    all(maturity_steps < ncol(simulation$rates)),
    all(abs(maturity_steps * step_size - maturities) < 1e-10)
  )
  discount_matrix <- exp(-apply(
    simulation$rates[, -ncol(simulation$rates), drop = FALSE] * step_size,
    1, cumsum))
  purrr::map2(maturities, along(maturities), function(index) {
    discounts <- discount_matrix[, maturity_steps[index]]
    tibble::tibble(
      maturity = maturities[index],
      initial_curve = exp(-flat_rate * maturities[index]),
      simulation = mean(discounts),
      mc_standard_error = stats::sd(discounts) / sqrt(length(discounts))
    )
  }) |>
  dplyr::mutate(difference = simulation - initial_curve)
}

tbl_hw_discount <- compute_hw_discount_check_LRL(
  hw_example, flat_rate = 0.02, maturities = c(1, 2, 5)
```

```
)
show_rate_model_tbl_LRL(tbl_hw_discount, "初期割引曲線と経路別割引の平均")
```

```
# A tibble: 3 x 5
```

	maturity	initial_curve	simulation	mc_standard_error	difference
	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	1	0.980	0.980	0.0000593	-0.0000000744
2	2	0.961	0.961	0.000158	0.0000244
3	5	0.905	0.905	0.000521	0.000197

実際の初期曲線を使う場合、 $f(0, t)$ は連続複利の割引曲線から得る。力の補間法と微分法によって結果が変わるため、ノートでの扱いも第 II-3 章のカーブフィットと異なる。正確な割引期待値を必要とする計算では、積分過程を同時にサンプルする方法や、分細かいシミュレーションを使う。

II-7.3 HJM：瞬間フォワードモデル

HJM では満期 T ごとの $f(t, T)$ に対して、リスク中立測度と短期金利によるマネーマーケット口座を基準に

$$df(t, T) = \sigma_f(t, T) \left(\int_t^T \sigma_k(t, u) du \right) dW_{k,t}$$

と置く。無裁定の条件は、この連続モデルの仮定のもとで

$$\sigma_f(t, T) = \sigma_k(t, T) \int_t^T \sigma_k(t, u) du$$

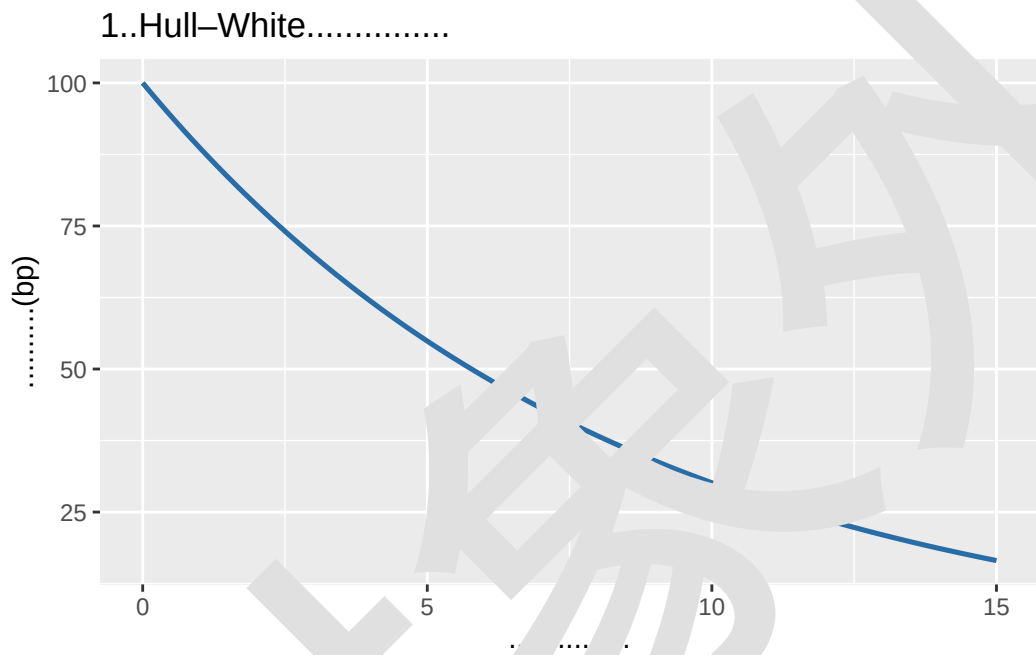
である。ボラティリティ関数を指定すれば、ドリフトは自由な調整項ではない。 $P(t, T) = \exp(-\int_t^T f(t, u) du)$ から債券価格が定まる。

Hull-White 1 因子は、例えば満期方向のボラティリティを $\sigma_f(t, T) = \sigma e^{-a(T-t)}$ とする Gaussian HJM の特別な場合として見られる。満期が長いほど短期のショックの効果が小さくなる。その形を図で確認する。

```
tbl_hjm_loading <- tibble::tibble(
  time_to_maturity = seq(0, 15, by = 0.1)
) |>
dplyr::mutate(
  forward_volatility = 0.01 * exp(-0.12 * time_to_maturity)
)
```



```
ggplot2::ggplot(
  tbl_hjm_loading,
  ggplot2::aes(time_to_maturity, 10000 * forward_volatility)
) +
  ggplot2::geom_line(colour = "#286DA8", linewidth = 0.9) +
  ggplot2::labs(
    title = "1.因子Hull-Whiteに対応するフォワードの変動係数",
    x = "ショック時点から満期までの年数", y = "瞬間フォワードの係数 (bp)"
  )
)
```



HJM は連続した金利の曲線をパラメータとし、瞬間フォワードのボラティリティから無裁定のドリフトを定める。ただし、このボラティリティ曲面を直接クオートする市場はないため、Caplet や Swap option の観測価格に合わせるモデル化が必要になる。次の BGM/LMM は、実際の金利期間に対応する金利を状態変数として扱う。上のドリフト式のとおり、HJM の瞬間フォワード金利は一般にマルティンゲールではない。

II-7.4 BGM/LMM：期間別フォワード

BGM (Brace–Gatarek–Musiela) /LMM (LIBOR Market Model) では、 $L_i(t)$ ごとにボラティリティと金利間の相関を指定する。単一曲線の T_{i+1} フォワード測度では、正の L_i を仮定する古典的な対数正規モデルを

$$\frac{dL_i(t)}{L_i(t)} = \sigma_i(t) \cdot dW_t^{T_{i+1}}, \quad 0 \leq t \leq T_i$$

と書ける。この測度では当該期間の L_i はマルチンゲールである。一方、すべての期間金利を同じ測度で同時に動かすなら測度変換によるドリフトが必要である。フォワードごとに上式の無ドリフト遷移を並べるだけでは、一般に完全な共同シミュレーションにならない。

古典的な対数正規モデルは正のフォワードを前提とする。負の金利を許す Normal 型や Shifted Lognormal 型を採用する場合は、ダイナミクスと市場ボラティリティの定義を合わせる。マルチカーブでは OIS 割引曲線と予測フォワードを分け、上の単一曲線の債券比率をそのまま IBOR 予測に流用しない。

満期 T_i までのボラティリティが一定の例では、当該フォワード測度の周辺分布は

$$L_i(T_i) = L_i(0) \exp \left[-\frac{1}{2} \nu_i^2 T_i + \nu_i \sqrt{T_i} Z \right].$$

この分布から求めた Caplet のシミュレーション価格を Black 公式と比較する。価格係数は $N \Delta_i P(0, T_{i+1})$ とし、 N を想定元本とする。この例は一般の Caplet の周辺分布の検証であり、BGM 全体の共同経路ではない。

```
compute_black_caplet_LRL <- function(
  forward, strike, volatility, expiry,
  accrual, discount_factor, notional) {
  stopifnot(
    forward > 0, strike > 0, volatility > 0, expiry >= 0,
    accrual > 0, discount_factor > 0, notional > 0
  )
  scale <- notional * accrual * discount_factor
  total_volatility <- volatility * sqrt(expiry)
  if (total_volatility > 0) {
    return(scale * max(forward - strike, 0))
  }
  d1 <- (log(forward / strike) + total_volatility^2 / 2) /
    total_volatility
  d2 <- d1 - total_volatility
  scale * (
    forward * stats::pnorm(d1) - strike * stats::pnorm(d2)
  )
}

forward_0 <- 0.025
strike <- 0.0275
expiry <- 2
volatility <- 0.24
accrual <- 0.5
```

```

discount_factor <- exp(-0.02 * (expiry + accrual))
number_of_paths <- 50000L
set.seed(618L)
forward_at_expiry <- forward_0 * exp(
  -volatility^2 * expiry / 2 +
  volatility * sqrt(expiry) * stats::rnorm(number_of_paths)
)
discounted_payoff <- accrual * discount_factor *
  pmax(forward_at_expiry - strike, 0)

tbl_caplet_check <- tibble::tibble(
  black_price = compute_black_caplet_LRL(
    forward_0, strike, volatility, expiry, accrual, discount_factor
  ),
  simulated_price = mean(discounted_payoff),
  mc_standard_error = stats::sd(discounted_payoff) /
    sqrt(number_of_paths)
) |>
  dplyr::mutate(difference = simulated_price - black_price)

show_rate_model_tbl_LRL(tbl_caplet_check, "Caplet 価格：公式とモンテカルロ")

```

```

# A tibble: 1 x 4
  black_price simulated_price mc_standard_error difference
    <dbl>         <dbl>         <dbl>         <dbl>
1  0.00115      0.00115      0.0000113 -0.00000175

```

相関は一つの Caplet 価格だけでは決められない。複数の期間金利が同時に関係する Swaption や早期償還商品の価値に効く。次のように相関した標準正規ショックを作って、**ショックの相関**を確認するだけである。各金利に対する測度と共通測度のドリフト、満期を含む共同経路の構築は行わない。

```

target_correlation <- matrix(
  c(1, 0.8, 0.5,
    0.8, 1, 0.7,
    0.5, 0.7, 1),
  nrow = 3, byrow = TRUE
)
set.seed(619L)
independent_shocks <- matrix(stats::rnorm(90000), ncol = 3)
correlated_shocks <- independent_shocks %*% chol(target_correlation)

tbl_shock_correlation <- as.data.frame(

```

```

as.table(stats::cor(correlated_shocks))
) |>
tibble::as_tibble() |>
dplyr::mutate(
  target = target_correlation[
    cbind(as.integer(Var1), as.integer(Var2))
  ]
) |>
dplyr::rename(forward_1 = Var1, forward_2 = Var2, simulated = Freq)

show_rate_model_tbl_LRL(
  tbl_shock_correlation, "相関ショックの目標値と標本値"
)

```

```

# A tibble: 9 x 4
  forward_1 forward_2 simulated target
  <fct>      <fct>      <dbl>  <dbl>
1 A          A          1      1
2 B          A          0.799  0.8
3 C          A          0.499  0.5
4 A          B          0.699  0.3
5 B          B          0.699  0.3
6 C          B          0.699  0.3
7 A          C          0.499  0.5
8 B          C          0.699  0.7
9 C          C          1      1

```

第 6 章「Swaption」から相関構造を推定する際も、上のショック生成と**完全な BGM 価格エンジン**は区別が必要で、推定させたボンティリティと相関を使い、選んだ共通測度でドリフトを設定し、適切な割引・予測カーブとともに価格を計算する必要がある。

II-7.5 後決め金利と FMM モデル

伝統的な LIBOR 連動期間金利は、通常、金利期間の開始前または開始時に、その期間に適用するレートが観測される。TONAR などの翌日物 RFR を日々複利で積み上げる契約では、期間 $[T_i, T_{i+1}]$ の利息の大部分は期間開始時にまだ確定していない。

期間内の各日の翌日物金利を R_j 、対応する日数割合を δ_j 、全期間の日数割合を Δ_i とすれば、観測済みの期間複利金利は概念上

$$R_i^{\text{comp}} = \frac{\prod_{j \in [T_i, T_{i+1})} (1 + \delta_j R_j) - 1}{\Delta_i}.$$

実際の契約では休日、観測日と適用日のずれ、lookback や lockout などの規則が効くため、日程と日数割合を契約に従って作る。この式だけで実際の TONAR の支払額を定義したことにはならない。

```
compute_compounded_overnight_LRL <- function(
  overnight_rates, day_fractions, period_fraction = sum(day_fractions)
) {
  stopifnot(
    length(overnight_rates) == length(day_fractions),
    length(overnight_rates) > 0,
    all(is.finite(overnight_rates)),
    all(is.finite(day_fractions)),
    all(day_fractions > 0), period_fraction > 0,
    all(1 + day_fractions * overnight_rates > 0)
  )
  (prod(1 + day_fractions * overnight_rates) - 1) /
  period_fraction
}

tbl_daily_fixings <- tibble::tibble(
  observation = seq(0, 1, by = 0.01),
  day_fraction = 1 / 360,
  overnight_rate = 0.0004 *
    sin(2 * pi * observation / 90)
) |>
dplyr::mutate(
  realized_factor = cumprod(1 + day_fraction * overnight_rate),
  fraction_elapsed = sum(day_fraction),
  realized_rate_to_rate = (realized_factor - 1) /
    fraction_elapsed
)

tbl_compound_summary <- tibble::tibble(
  observations = nrow(tbl_daily_fixings),
  compounded_rate = compute_compounded_overnight_LRL(
    tbl_daily_fixings$overnight_rate,
    tbl_daily_fixings$day_fraction
  ),
  simple_average = mean(tbl_daily_fixings$overnight_rate)
)

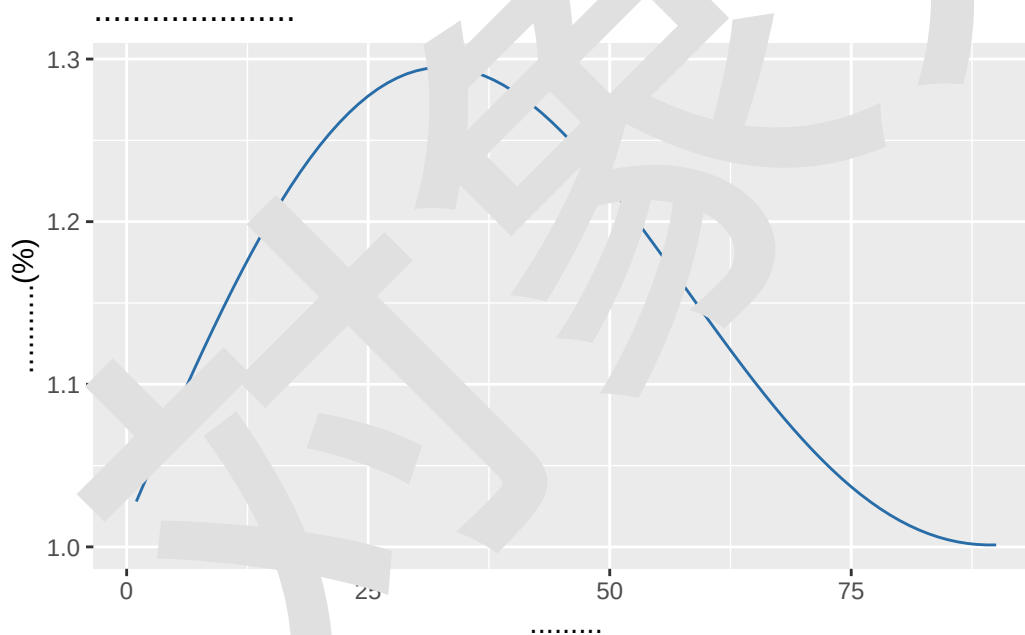
show_rate_model_tbl_LRL(
```

```
tbl_compound_summary, "架空の翌日物金利90日分の確定後の期間金利"
)
```

```
# A tibble: 1 x 3
```

```
  observations compounded_rate simple_average
      <int>          <dbl>          <dbl>
1         90         0.0100         0.01
```

```
ggplot2::ggplot(
  tbl_daily_fixings,
  ggplot2::aes(observation, 100 * realized_rate_to_date)
) +
  ggplot2::geom_line(colour = "#286DA8") +
  ggplot2::labs(
    title = "当日までに確定した金利から計算した複利金利",
    x = "金利期間内の観測日", y = "観測済み部分の年率換算率(%) "
  )
)
```



このグラフは事後にすべての架空の日次金利が与えられた場合の計算例であり、将来の翌日物金利を予測するモデルではない。期間中の評価時点では、既に確定した複利係数と残りの未確定部分を分ける。

用語にも注意したい。BGM/LMM は古典的な LIBOR 期間フォワードを扱うモデル名として使われる。Lyashenko–Mercurio による **generalized Forward Market Model (FMM)** は、後決め複利 RFR のフォワードを扱えるよう期間金利の市場モデルを一般化する構成である。「LMM がそのまま FMM という名前になった」とは解釈しない。RFR 向け FMM で選ぶフォワードの定義、測度、期間中に確定した金利の扱いは、商品とモデルの仕様に即して記述

する。

II-7.6 各種モデルサマリー

第 8 章の早期償還債では、各行使日に「その日に償還する価値」と「保有を続ける価値」を比べる。適切なモデルを選ぶため、次の点を確認する。

論点	Hull-White	BGM/LMM または RFR 向け FMM
状態の次元	1 因子の基本形は比較的小さい	期間別金利が多いほど大きい
代表的な計算法	1 因子の格子・木、またはモンテカルロ	モンテカルロ、またはゼロと条付き付き期待値の推定
強み	曲線に合わせ、行使日の情報を整理しやすい	複数期間のフォワードの変動と相関を直接設定しやすい
第 6 章との接続	Swaption 等から μ, σ を推定	Caplet 等から期間別変動、Swaption 等から相関を推定

行使価値の計算では、シミュレーション上の経路の平均値だけでは足りない。行使時点の情報に条件づけた**継続価値**を計算する。第 8 章では、Hull-White を使い、発行体のコール権を債券価格にどう反映させるかを扱う。1 因子の市場モデルによる方法は、その後にモデル選択上の拡張として位置づけられる。

第 II-8 章金利モデルによる早期償還 評価

固定利付債の発行体に期限前償還権があると、金利が低下したときに、クーポンを支払いを打ち切って借り換えることができる。第 6 章でモデルの金利を市場に、第 7 章で金利の動きを定義した。本章では、そのモデルを使い、各使用日に、その価値を比較して債券価格が決まるかを計算する。

まず発行体が指定日に額面で償還できる固定利付債（Callable Bond）を Hull-White 1 因子モデルで評価する。次に MBS のプライミングをモデル化する。発行体はコールと住宅ローン債務者の繰上返済は、どちらも元本の早期返済だが、発行主体とキャッシュフローが異なる。以下の金利、クーポン、ボラティリティはすべて教材用に設定した値である。

```
if (file.exists(file.path("R", "utility_functions.R"))) {
  source(file.path("R", "utility_functions.R"))
}
suppressPackageStartupMessages(library(tidyverse))

show_error_message_if_needed <- function(tbl, title) {
  if (requireNamespace("GaussQuant", quietly = TRUE)) {
    GaussQuant::show_error_message_if_needed(tbl, title)
  } else {
    print(tbl)
  }
  invisible(tbl)
}
```

II-8.1 発行体条件と債券評価

額面 100、年率 3% の固定クーポン、半年払い、満期 5 年の債券を考える。各クーポンは J は $100 \times 0.03 \times 0.5 = 1.5$ 。発行体は 1 年目から 4 年半目までの各クーポン日に、その日のクーポンを支払った後、残りの元本を 100 で償還できるとする。満期日には通常の元本償還があり、

コール判断はない。

項目	この章での設定
評価時点と金利期間	$t = 0$ 、半年ごとの等間隔の年数
クーポン	各 $t = 0.5, 1.0, \dots, 5.0$ に 1.5
コール可能日	$t = 1.0, 1.5, \dots, 4.5$
コール価格	クーポン支払い後の元本に対し 100 (ex-coupon の額面価格)
金利曲線	連続複利 2% のフラットな初期割引曲線
信用と費用	デフォルト、スプレッド、税、取引費用は含めない

ここまでの設定では信用リスクを除き、金利モデルが決める発行体コール債の評価を確認する。後半では MBS の住宅ローン債務者によるプリペイメントを考慮する。

したがって、この例の値は評価時点の将来キャッシュフローのモデル現在価値で、経過利息のある取引日で表示するクリーン価格や決済価格ではない。入取日は発行日・決済日、営業日規則、日数計算、通知期間、コール価格の定義も別に指定する。

```
notional <- 100
coupon_rate <- 0.03
period_years <- 0.5
maturity_years <- 5
flat_zero_rate <- 0.02
first_call_year <- 1
call_price <- 100

tbl_cashflows <- tibble::tibble(
  payment_time = seq(0.5, maturity_years, by = period_years),
  coupon = notional * coupon_rate * period_years,
  principal_at_maturity = dplyr::if_else(
    payment_time == maturity_years, notional, 0
  ),
  issuer_can_call = payment_time >= first_call_year &
    payment_time < maturity_years,
  call_principal = dplyr::if_else(
    issuer_can_call, call_price, NA_real_
  )
)

show_early_redemption_tbl_LRL(tbl_cashflows, "クーポンと発行体コール日")
#> # A tibble: 10 x 5
#>   payment_time coupon principal_at_maturity issuer_can_call call_principal
#>   <dbl> <dbl> <dbl> <dbl> <lgl> <dbl>
```

#> 1	0.5	1.5	0	FALSE	NA
#> 2	1	1.5	0	TRUE	100
#> 3	1.5	1.5	0	TRUE	100
#> 4	2	1.5	0	TRUE	100
#> 5	2.5	1.5	0	TRUE	100
#> 6	3	1.5	0	TRUE	100
#> 7	3.5	1.5	0	TRUE	100
#> 8	4	1.5	0	TRUE	100
#> 9	4.5	1.5	0	TRUE	100
#> 10	5	1.5	100	FALSE	NA

II-8.2 満期債と発行体コール

コールオプションない Bullet Bond の時点 0 の価格は、クーポン額を C 、満期元金 N 、割引係数を $P(0, t_i)$ として

$$V_0^{\text{bullet}} = \sum_{i=1}^n C P(0, t_i) + N P(0, t_n).$$

初期カーブを完全に再現するモデルを持つ、発行体から得る Bullet Bond の価格は、この値に一致する。コール価格、行使率が一致すれば、発行体のコールオプション価値は

$$V_0^{\text{issuer call}} = V_0^{\text{bullet}} - V_0^{\text{callable}} \geq 0$$

となる。これは同一の信用・割引・クーポン条件で比較した**価格差**であり、別に提示された任意の Receive Clean Price 価格と自動的に一致するわけではない。

```
compute_bullet_price_LRL <- function(tbl_cashflows, flat_rate) {
  stopifnot(is.finite(flat_rate))
  sum(
    (tbl_cashflows$coupon + tbl_cashflows$principal_at_maturity) *
    exp(-flat_rate * tbl_cashflows$payment_time)
  )
}

bullet_curve_price <- compute_bullet_price_LRL(
  tbl_cashflows, flat_zero_rate
)

show_early_redemption_tbl_LRL(
```

```
tibble::tibble(bullet_price = bullet_curve_price),
  "初期割引曲線によるBullet Bondの価格"
)
#> # A tibble: 1 x 1
#>   bullet_price
#>   <dbl>
#> 1      104.69
```

II-8.3 発行体の行使判断とバックワード・インダクション

クーポン支払日の直後に、将来の支払義務の価値を発行体が比較する。その時点から次の支払日までの継続価値を $H_i(x)$ 、コール時に支払う元本を K_i 、当日の利払いを C_i とする。直前の投資家側の価格は

$$V_i(x) = C_i + \min\{K_i, H_i(x)\} \quad (\text{発行体はコール可能な日}),$$

であり、コール不可の日は $V_i(x) = C_i + H_i(x)$ 、満期日は $V_n(x) = C_n + N$ 。クーポン C_i はコール時にも支払われるため、比較する分岐の共通部分として外に置いた。

継続価値は、リスク中立測度のもとで

$$H_i(x) = \mathbb{E} \left[\exp \left(- \int_{t_i}^{t_{i+1}} r_s ds \right) V_{i+1}(x_{t_{i+1}}) \mid x_{t_i} = x \right].$$

発行体は $V_i(x) > K_i$ のときコールする。等号のときの扱いは価格を変えない。min が表すのは、発行体が投資家に対する残存債務より安く決済できる枝を選ぶことである。

flowchart TD

```
A["当日のクーポンを払う"] --> B["元本のコール価格と継続価値を比較"]
B -->|"継続価値が高"| C["元本を支払い償還"]
B -->|"継続価値が低"| D["債券を継続"]
```

II-8.4 Hull–White モデル

第7章と同じく、 $r_t = \phi(t) + x_t$ 、 $dx_t = -ax_t dt + \sigma dW_t$ を使う。一定の $a > 0, \sigma \geq 0$ とフラットな初期曲線 $P(0, T) = e^{-\bar{r}T}$ について

$$\phi(t) = \bar{r} + \frac{\sigma^2}{2a^2}(1 - e^{-at})^2.$$

$h = t_{i+1} - t_i$ 、 $b = e^{-ah}$ 、 $B = (1-b)/a$ と置く。時点 t_i の x を条件とした次の状態 X と期間内の確率的な割引積分 I は

$$X = bx + \varepsilon_X, \quad I = \int_{t_i}^{t_{i+1}} x_s ds = Bx + \varepsilon_I,$$

であり、誤差の組は平均 0 の二変量正規分布になる。その分散・共分散を

$$\begin{aligned} v_X &= \frac{\sigma^2(1-b^2)}{2a}, \\ v_I &= \frac{\sigma^2}{a^2} \left[h - \frac{2(1-b)}{a} + \frac{1-b^2}{a} \right], \\ c_{XI} &= \frac{\sigma^2(1-b)^2}{2a^2} \end{aligned}$$

と書く。積分を単に hr_{t_i} とするのではなく、そのため X の値に条件づいて I の正規分布を用いる。 $v_X > 0$ の場合、標準正規 Z に対

$$\begin{aligned} E[e^{-\int_{t_i}^{t_{i+1}} x_s ds} | x] &= \exp \left[- \int_{t_i}^{t_{i+1}} (s - t_i) \phi(s - t_i) ds - \frac{1}{2} \left(v_I - \frac{c_{XI}^2}{v_X} \right) \right] \\ &= E_Z \left[e^{-(c_{XI}/\sqrt{v_X})Z} | x = x + \sqrt{v_X}Z \right]. \end{aligned}$$

この 1 次の期待値を正規分布用の Gauss-Hermite 求積で計算し、次時点の状態ごとの価格を線形補間する。状態の格子幅の両端、および求積点数による数値誤差が残る。 $\sigma = 0$ のときは決定論的な遷移に切り替える。

```
compute_normal_quad = function(number_of_nodes = 24L) {
  stopifnot(number_of_nodes >= 2)
  number_of_nodes <- as.integer(number_of_nodes)
  off_diagonal <- sqrt(seq_len(number_of_nodes - 1L))
  jacobi <- matrix(0, number_of_nodes, number_of_nodes)
  jacobi[cbind(
    seq_len(number_of_nodes - 1L),
    seq_len(number_of_nodes - 1L) + 1L
  )] <- off_diagonal
  jacobi[cbind(
    seq_len(number_of_nodes - 1L) + 1L,
    seq_len(number_of_nodes - 1L)
  )] <- off_diagonal
}
```

```

roots <- eigen(jacobi, symmetric = TRUE)
list(nodes = roots$values, weights = roots$vectors[1, ]^2)
}

compute_hw_shift_integral_LRL <- function(
  start_time, end_time, flat_rate, mean_reversion, volatility
) {
  a <- mean_reversion
  h <- end_time - start_time
  flat_rate * h + volatility^2 / (2 * a^2) * (
    h - 2 * (exp(-a * start_time) - exp(-a * end_time)) / a +
    (exp(-2 * a * start_time) - exp(-2 * a * end_time)) /
    (2 * a)
  )
}

compute_discounted_step_LRL <- function(
  next_values, state_grid, current_states,
  start_time, end_time, flat_rate,
  mean_reversion, volatility, quadratic
) {
  a <- mean_reversion
  h <- end_time - start_time
  b <- exp(-a * h)
  B <- -expm1(-a * h) / a
  deterministic_integral <- compute_hw_shift_integral_LRL(
    start_time, end_time, flat_rate, a, volatility
  )
  if (volatility == 0) {
    return(
      exp(-deterministic_integral - B * current_states) *
      state_grid[next_values, xout = b * current_states,
        rule = 2
      ]$y
    )
  }
}

variance_next <- volatility^2 * (-expm1(-2 * a * h)) / (2 * a)
variance_integral <- volatility^2 / a^2 * (
  h - 2 * B + (-expm1(-2 * a * h)) / (2 * a)
)
covariance <- volatility^2 * (-expm1(-a * h))^2 /
  (2 * a^2)
residual_variance <- max(

```

```

    variance_integral - covariance^2 / variance_next, 0
  )
  standard_deviation_next <- sqrt(variance_next)

  next_states <- outer(
    current_states, quadrature$nodes,
    function(state, shock) b * state + standard_deviation_next * shock
  )
  interpolated_values <- matrix(
    stats::approx(
      state_grid, next_values, xout = as.vector(next_states),
      rule = 2
    )$y,
    nrow = length(current_states)
  )
  conditional_weights <- quadrature$weights * exp(
    -covariance / standard_deviation_next * quadrature$nodes
  )
  as.vector(
    exp(-deterministic_integral - current_states +
      residual_variance / 2)
    (interpolated_values %*% conditional_weights)
  )
}

```

満期にクーポンと元本を払い、一つ前の支払日にクーポン割引期待値を求める。以後、コール可能日では min を取って比較する。比較のため、まったく同じ状態格子と割引演算子で Bullet Bond も計算する。

```

compute_callable_bond_LRL <- function(
  tbl_cashflows, rate, mean_reversion, volatility,
  number_of_grid_points = 321L,
  grid_limit = 0.13, quadrature_nodes = 24L
) {
  stopifnot(
    nrow(tbl_cashflows) >= 2, mean_reversion > 0,
    volatility >= 0, number_of_grid_points >= 3,
    grid_limit > 0, quadrature_nodes >= 2,
    all(diff(tbl_cashflows$payment_time) > 0),
    all(tbl_cashflows$coupon >= 0),
    all(tbl_cashflows$principal_at_maturity[
      -nrow(tbl_cashflows)
    ] == 0),
    all(!is.na(tbl_cashflows$call_principal[

```

```

    tbl_cashflows$issuer_can_call
  ]))
)
grid <- seq(-grid_limit, grid_limit,
  length.out = number_of_grid_points
)
quadrature <- compute_normal_quadrature_LRL(quadrature_nodes)
number_of_periods <- nrow(tbl_cashflows)
maturity_value <- rep(
  tbl_cashflows$coupon[number_of_periods] +
  tbl_cashflows$principal_at_maturity[number_of_periods],
  length(grid)
)
terminal <- list(
  bullet = maturity_value,
  callable = maturity_value,
  decisions = list()
)
history <- purrr::accumulate(
  rev(seq_len(number_of_periods)),
  .init = terminal,
  .f = function(next_state, payment_index) {
    start_time <- tbl_cashflows$payment_time[payment_index]
    end_time <- tbl_cashflows$payment_time[payment_index + 1L]
    price_step <- function(next_values) {
      compute_discount_step_LRI(
        next_values, grid,
        start_time, end_time, flat_rate,
        reversion, volatility, quadrature
      )
    }
    bullet_continuation <- price_step(next_state$bullet)
    callable_continuation <- price_step(next_state$callable)
    can_call <- tbl_cashflows$issuer_can_call[payment_index]
    call_amount <- tbl_cashflows$call_principal[payment_index]
    exercise_now <- if (can_call) {
      callable_continuation > call_amount
    } else {
      rep(FALSE, length(grid))
    }
    investor_value <- if (can_call) {
      pmin(call_amount, callable_continuation)
    } else {
      callable_continuation
    }
  }
)

```

```

decision <- if (can_call) {
  list(tibble::tibble(
    payment_time = start_time,
    state = grid,
    short_rate = flat_rate +
      volatility^2 / (2 * mean_reversion^2) *
      (1 - exp(-mean_reversion * start_time))^2 + grid,
    continuation_ex_coupon = callable_continuation,
    call_principal = call_amount,
    exercise_now = exercise_now
  ))
} else {
  list()
}
list(
  bullet = tbl_cashflows$coupon[payment_index] +
    bullet_continuation,
  callable = tbl_cashflows$coupon[payment_index] +
    investor_value,
  decisions = c(decision, next_period$decisions)
)
}
)
first_period <- history[[1]]
price_at_zero <- function(next_values) {
  compute_discounted_pay_LRL(
    next_values, current_state = (
      start_time = 0,
      end_time = tbl_cashflows$payment_time[1],
      flat_rate = flat_rate,
      mean_reversion = mean_reversion,
      volatility = volatility,
      quadrature = quadrature
    )
  )
}
bullet_price <- price_at_zero(first_period$bullet)
callable_price <- price_at_zero(first_period$callable)
list(
  bullet_price = bullet_price,
  callable_price = callable_price,
  issuer_call_value = bullet_price - callable_price,
  decisions = dplyr::bind_rows(first_period$decisions),
  grid = grid
)
}

```



```

model_result <- compute_callable_bond_LRL(
  tbl_cashflows, flat_rate = flat_zero_rate,
  mean_reversion = 0.15, volatility = 0.012
)

tbl_price_comparison <- tibble::tibble(
  bullet_from_initial_curve = bullet_curve_price,
  bullet_from_model = model_result$bullet_price,
  callable_from_model = model_result$callable_price,
  issuer_call_value = model_result$issuer_call_value,
  model_minus_initial_bullet =
    model_result$bullet_price - bullet_curve_price
)

show_early_redemption_tbl_LRL(
  tbl_price_comparison, "Bullet、Callableとコールオプション価値"
)

#> # A tibble: 1 x 5
#>   bullet_from_initial_curve bullet_from_model callable_from_model issuer_call_value
#>   <dbl> <dbl> <dbl> <dbl>
#> 1 104.69 104.69 100.05 4.2355
#> # i abbreviated name: 1: bullet_from_initial_curve
#> # i 1 more variable: model_minus_initial_bullet

stopifnot(
  abs(model_result$bullet_price - bullet_curve_price) < 0.02,
  model_result$issuer_call_value >= -1e-8
)

tbl_without_call <- tbl_cashflows %>%
  dplyr::mutate(
    issuer_call_value = FALSE,
    call_principal = 1A_call_
  )

model_without_call <- compute_callable_bond_LRL(
  tbl_without_call, flat_rate = flat_zero_rate,
  mean_reversion = 0.15, volatility = 0.012
)

stopifnot(
  abs(model_without_call$callable_price -
    model_without_call$bullet_price) < 1e-8
)

```

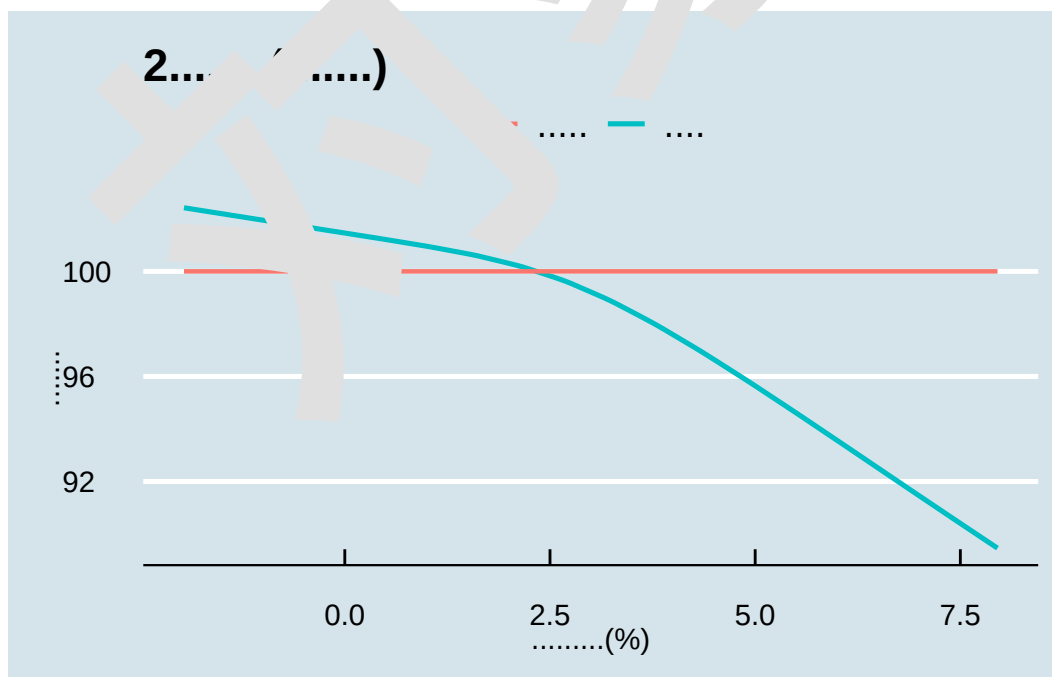
最後の行は、同じモデルと同じ契約条件で Bullet 価格が初期曲線へ近いこと、発行体の権利価値が負にならないことを検査する。価格の差が 0 でない場合は、状態格子、境界、補間、求積点

数を見直す。格子上のコール可能な点の割合は、実際のコール確率ではない。

次の図は行使可能日のうち $t = 2.0$ における、クーポン支払後の継続価値とコール価格を示す。短期金利の状態が低いほど、高いクーポンが続く債券を発行体が額面で償還する誘因が強い。

```
tbl_decision_at_year_2 <- model_result$decisions |>
  dplyr::filter(abs(payment_time - 2) < 1e-10) |>
  dplyr::filter(dplyr::between(short_rate, -0.02, 0.08))

ggplot2::ggplot(
  tbl_decision_at_year_2,
  ggplot2::aes(x = 100 * short_rate)
) +
  ggplot2::geom_line(
    ggplot2::aes(y = continuation_ex_coupon,
      colour = "継続価値"), linewidth = 0.9
  ) +
  ggplot2::geom_line(
    ggplot2::aes(y = call_principal,
      colour = "コール価格"), linewidth = 0.9
  ) +
  ggplot2::labs(
    title = "2年目のコール判断",
    x = "その時点の短期金利 (%)",
    colour = NULL
  )
)
```



II-8.5 金利変動とコールオプション価値

第6章で説明した方法で推定したボラティリティを与えれば、同じ手順で債券価格を求められる。次は初期割引曲線と平均回帰速度を固定し、金利ボラティリティを変えた**比較実験**である。 $\sigma = 0$ は確率変動のない決定論的な基準になる。

```
tbl_volatility_sensitivity <- tibble::tibble(
  short_rate_volatility = c(0, 0.006, 0.012, 0.020)
) |>
dplyr::mutate(
  valuation = purrr::map(
    short_rate_volatility,
    ~compute_callable_bond_LRL(
      tbl_cashflows, flat_rate = flat_zero_rate,
      mean_reversion = 0.15, volatility = .x
    )
  ),
  callable_price = purrr::map_dbl(
    valuation, "callable_price"
  ),
  issuer_call_value = purrr::map_dbl(
    valuation, "issuer_call_value"
  ),
  bullet_grid_error = purrr::map_dbl(
    valuation, "bullet_grid_error"
  ) - bullet_current_price
) |>
dplyr::select(valuation)

show_early_warning(tbl_LRL(
  tbl_volatility_sensitivity,
  "短期金利ボラティリティと発行体コール権"
))

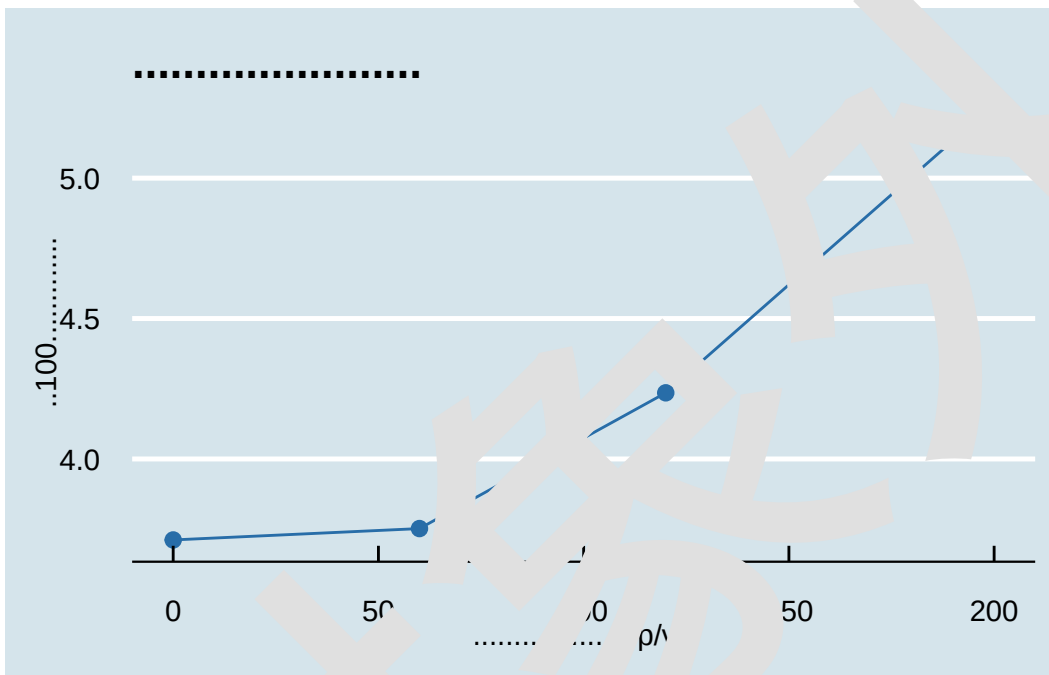
#> # A tibble: 4 x 5
#>   short_rate_volatility callable_price issuer_call_value bullet_grid_error
#>   <dbl>           <dbl>           <dbl>           <dbl>
#> 1             0         100.98             3.7116      5.6843e-14
#> 2         0.006         100.93             3.7521      2.3079e- 4
#> 3         0.012         100.45             4.2355      2.2624e- 4
#> 4         0.02          99.425             5.2622      2.0805e- 4

ggplot2::ggplot(
  tbl_volatility_sensitivity,
```

```

ggplot2::aes(10000 * short_rate_volatility,
  issuer_call_value)
) +
ggplot2::geom_line(colour = "#286DA8") +
ggplot2::geom_point(size = 2.4, colour = "#286DA8") +
ggplot2::labs(
  title = "同じ初期曲線におけるコールオプションモデル価値",
  x = "短期金利ボラティリティのパラメータ (bp/√年)",
  y = "額面100当たりの発行体コール権価値"
)

```



ボラティリティが小さいほど、発行体に有利な金利の状態を利用できるため、この例ではコールオプション価値が増える。一般的な取引では、各シナリオでキャリブレーションや他の市場入力も変化することもあり、この表だけリスク量とは呼ばない。価格感応度を比べるときは、カーブ、スプレッド、ボラティリティなどのどれを固定するか明確にする。

II-8.6 数値計算検証

本章の評価は価格関数を有限の状態格子に補間する。格子幅と求積点数を変えて結果が安定するか確かめる。次は同じ契約を二つの格子で評価した例であり、価格差を表示する。

```

tbl_grid_convergence <- tibble::tibble(
  number_of_grid_points = c(161L, 321L, 641L),
  quadrature_nodes = c(16L, 24L, 32L)
) |>
dplyr::mutate(
  valuation = purrr::map2(

```

```

    number_of_grid_points, quadrature_nodes,
    ~compute_callable_bond_LRL(
      tbl_cashflows, flat_rate = flat_zero_rate,
      mean_reversion = 0.15, volatility = 0.012,
      number_of_grid_points = .x, quadrature_nodes = .y
    )
  ),
  callable_price = purrr::map_dbl(
    valuation, "callable_price"
  ),
  bullet_grid_error = purrr::map_dbl(
    valuation, "bullet_price"
  ) - bullet_curve_price
) |>
dplyr::select(-valuation) |>
dplyr::mutate(
  change_from_previous = callable_price -
    dplyr::lag(callable_price)
)

show_early_redemption_tbl_LRL(
  tbl_grid_convergence, "格子 収束を繰り返すとき 価格"
)
#> # A tibble: 3 x 5
#>   number_of_grid_points quadrature_nodes callable_price bullet_grid_error
#>   <dbl>               <dbl>         <dbl>         <dbl>
#> 1         161             16          100.45         0.00077934
#> 2        321             16          100.45         0.00022624
#> 3        641             32          100.45         0.000059765
#> # i 1 more variable. change_from_previous <dbl>

```

二つの数値設定を繰り返しているため、差が大きいときは状態格子と求積点数をそれぞれ独立に増やして原因を調べる。初期 Bullet 価格との一致に加え、行使日をなくすと Callable が Bullet に一致すること。行使日を追加した場合は投資家の Callable 価格が上がらないことも、実装の検査になる。

II-8.7 Bermudan Swaption への拡張

固定クーポン債を発行体が償還できる権利は、金利低下局面で価値が増える点で、固定を受け取る Receiver Swaption と似る。複数行使日の権利は Bermudan 型である。ただし、Callable Bond と Bermudan Receiver Swaption を同額の権利とみなすには、債券クーポンとスワップの固定・変動キャッシュフロー、元本、行使日、決済とコール価格の対応を厳密に合わせる必

要がある。この章では価格を機械的に置き換えない。

Hull-White 1 因子では継続価値を一つの状態の関数として計算でき、Tree、有限差分、状態格子の積分などが候補になる。本章は状態格子と正規求積を用いた。一方、BGM/LMM や RFR 向け FMM を使って多数のフォワード金利を動かす場合は、共通の測度のもとで経路を作り、行使日時点の情報に条件づけた継続価値を回帰などで推定する方法がある。**将来の経路ごとに実現した利益を直接比べ、過去の行使を決める方法は使わない。**

実取引への拡張では、OIS 割引と予測カーブの分離、約定上の通知日と利払い日のずれ、クリーン価格と経過利息の扱いを揃える。

II-8.8 MBS のプリペイメント

住宅ローンを裏付けとする MBS では、**債務者が一部または全部を繰上りして返済できる**。プール全体では、各月の予定元本返済とは別にプリペイメントが生じ、投資家は元本を多く受け取る。発行体が契約日の一括償還を選ぶ前節の Callable Bond に異なり、キャッシュフローを月次のプール残高から組み立てる。ここでは元本と利息がそのまま投資家に支払われる簡略なパススルーを扱い、デフォルトと損失は扱わない。

米国の **100% PSA** をベンチマークとする MBS の年率換算プリペイメント率 (CPR) は、[Ginnie Mae の商品説明](#) に沿って

$$\text{CPR}_m^{\text{PSA}} = \text{psa} \cdot (0.002m, 0.06) \quad \text{with} \quad M^{\text{PSA}}_m = 1 - (1 - \text{CPR}_m)^{1/12}.$$

月次予定元本 A_m 、月初元本 B_{m-1} とすると、プリペイメント元本は $U_m = (B_{m-1} - A_m)M^{\text{PSA}}_m$ 、月次元本は $B_m = B_{m-1} - A_m - U_m$ 。基準 PSA だけでは借換えインセンティブを表せないため、以下の数値例では住宅ローンの約定金利 c 、**月ごとに变化する借換え金利 f_m** 、借換えに伴う借換え費用 k から、**教材用の仮定**

$$\text{CI}_m = \min\{0.60, \alpha \text{CPR}_m^{\text{PSA}} \exp[\beta(c - f_m - k)]\}$$

を置く。 $\alpha = 1$ は PSA ベンチマークの速度、 $\alpha = 0$ はプリペイメントなしである。 β と上限は市場での推定値ではない。経過月数、借換え費用と金利シナリオが**将来の各月のプリペイメント**を変え、元本残高の減少は翌月以降の利息と予定元本に反映される。

```
compute_mbs_cashflows_LRL <- function(
  original_balance, mortgage_rate, pass_through_rate,
  refinance_rate, maturity_months = 360L, incentive_sensitivity = 20,
  refinancing_cost_rate = 0, psa_multiplier = 1
```

```

) {
  stopifnot(
    original_balance > 0, maturity_months > 0,
    mortgage_rate >= 0, pass_through_rate >= 0,
    all(is.finite(refinance_rate)), all(refinance_rate >= 0),
    length(refinance_rate) %in% c(1L, maturity_months),
    incentive_sensitivity >= 0, refinancing_cost_rate >= 0,
    psa_multiplier >= 0,
    pass_through_rate <= mortgage_rate
  )
  refinance_path <- rep(refinance_rate, length.out = maturity_months)
  monthly_rate <- mortgage_rate / 12
  monthly_payment <- if (monthly_rate == 0) {
    original_balance / maturity_months
  } else {
    original_balance * monthly_rate /
      (1 - (1 + monthly_rate)^(-maturity_months))
  }
  balance <- original_balance
  rows <- vector("list", maturity_months)
  for (m in seq_len(maturity_months)) {
    begin_balance <- balance
    loan_interest <- begin_balance * monthly_rate
    scheduled_principal <- monthly_payment - loan_interest
    begin_balance <- max(0, monthly_payment - loan_interest)
  )
  if (m == maturity_months) scheduled_principal <- begin_balance
  base_cpr <- 0.002 * (1 - 0.06)
  cpr <- (
    0.002 * psa_multiplier * base_cpr * exp(
      incentive_sensitivity * (
        refinance_path[m] - refinancing_cost_rate
      )
    )
  )
  smm <- 1 - (1 - cpr)^(1 / 12)
  prepaid_principal <- (begin_balance - scheduled_principal) * smm
  balance <- max(0, begin_balance - scheduled_principal - prepaid_principal)
  rows[[m]] <- tibble::tibble(
    month = m, begin_balance = begin_balance,
    refinance_rate = refinance_path[m],
    cpr = cpr, smm = smm,
    interest = begin_balance * pass_through_rate / 12,
    scheduled_principal = scheduled_principal,
    prepaid_principal = prepaid_principal,

```

```

      end_balance = balance,
      cashflow = begin_balance * pass_through_rate / 12 +
        scheduled_principal + prepaid_principal
    )
  }
  dplyr::bind_rows(rows)
}

mbs_months <- 360L
tbl_mbs_scenarios <- tibble::tibble(
  scenario = c("借換え金利低下", "金利一定", "借換え金利上昇"),
  refinance_path = list(
    c(rep(0.05, 24), rep(0.035, mbs_months - 24)),
    rep(0.05, mbs_months),
    c(rep(0.05, 24), rep(0.065, mbs_months - 24))
  )
) |>
dplyr::mutate(
  cashflows = purrr::map(
    refinance_path,
    ~compute_mbs_cashflows(
      original_balance = 100, original_rate = 0.05,
      pass_through_rate = 0.9, refinance_rate = .,
      refinancing_cost_rate = 0.01
    )
  ),
  weighted_average_cashflow = purrr::map_dbl(
    cashflows,
    ~sum(.x$month / 12 *
      (.x$scheduled_principal + .x$prepaid_principal)) / 100
  ),
  price_factor = purrr::map_dbl(
    cashflows,
    ~sum(.x$cashflow * exp(-0.04 * .x$month / 12))
  ),
  price_path_discount = purrr::map2_dbl(
    cashflows, refinance_path,
    ~sum(.x$cashflow * exp(
      -cumsum((.y - 0.01) / 12)
    ))
  ),
  total_principal = purrr::map_dbl(
    cashflows,
    ~sum(.x$scheduled_principal + .x$prepaid_principal)
  )
)

```



```

)

show_early_redemption_tbl_LRL(
  dplyr::select(tbl_mbs_scenarios, -cashflows, -refinance_path),
  "借換え金利別のMBSプリペイメントと元本回収期間"
)

#> # A tibble: 3 x 5
#>   scenario      weighted_average_life price_fixed_discount price_path_discount
#>   <chr>          <dbl>          <dbl>          <dbl>
#> 1 借換え金利低下      7.3649      103.03      109.98
#> 2 金利一定            8.3142      103.35      103.35
#> 3 借換え金利上昇      9.2895      103.67      95.676
#> # 1 more variable: total_principal <dbl>
stopifnot(
  all(abs(tbl_mbs_scenarios$total_principal - 100, na.rm=T) < 1e-8) &&
  all(purrr::map_lgl(
    tbl_mbs_scenarios$cashflows,
    ~all(.x$end_balance >= -1e-8) &&
      abs(dplyr::last(.x$end_balance)) < 1e-8
  )),
  tbl_mbs_scenarios$weighted_average_life[1] < 10,
  tbl_mbs_scenarios$weighted_average_life[2] < 10,
  tbl_mbs_scenarios$weighted_average_life[3] < 10,
  tbl_mbs_scenarios$weighted_average_life[4] < 10
)

```

price_fixed_discount は、割引率を 4% に固定し、借換え金利によるプリペイメントの変化だけを価格に反映する。price_path_discount は例として各月の短期金利を「借換え金利 - 1%」と仮定して、積割計算値である。二つの価格の差をそのままオプション価値や OAS とは呼べない。借換え金利と短期金利の差、プリペイメント反応、投資家に支払う金利をそれぞれ別の入力として扱う。

次は第 7 章の Hull-White モデルで月次金利経路を作り、各経路の借換え金利からプリペイメントを計算する例である。短期金利に一律 3% を加えて借換え金利の代理変数とする。実際の住宅ローン金利は長期スワップ金利、貸出スプレッド、金融機関の提示条件などから推定する必要がある。このコードは月次格子の短期金利による割引近似を使い、入力パラメータは市場キャリブレーション値ではない。

```

simulate_mbs_rates_LRL <- function(
  number_of_paths = 200L, maturity_months = 360L,
  flat_rate = 0.02, mean_reversion = 0.15,
  volatility = 0.012, seed = 48L
) {

```

```

stopifnot(number_of_paths %% 2L == 0L, number_of_paths > 0,
  maturity_months > 0, mean_reversion > 0, volatility >= 0)
set.seed(seed)
dt <- 1 / 12
b <- exp(-mean_reversion * dt)
step_sd <- volatility * sqrt(
  (1 - b^2) / (2 * mean_reversion)
)
shocks <- matrix(
  stats::rnorm(number_of_paths / 2L * maturity_months),
  nrow = number_of_paths / 2L
)
shocks <- rbind(shocks, -shocks)
short_rates <- matrix(0, number_of_paths, maturity_months)
x <- rep(0, number_of_paths)
for (m in seq_len(maturity_months)) {
  x <- b * x + step_sd * shocks[, m]
  time <- m * dt
  phi <- flat_rate + volatility^2 / (2 * mean_reversion^2) *
    (1 - exp(-mean_reversion * time))
  short_rates[, m] <- phi + x
}
short_rates
}

mbs_short_rates <- simulate_mbs_rate_LRL()
tbl_mbs_mc <- purrr::map(
  seq_len(nrow(tbl_mbs_short_rates)),
  function(path_index) {
    rate_path <- mbs_short_rates[path_index, ]
    loan_cashflow <- compute_mbs_cashflows_LRL(
      orig_loan_size = 100, mortgage_rate = 0.05,
      pass_through_rate = 0.045,
      refinancing_rate = pmax(0, rate_path + 0.03),
      refinancing_cost_rate = 0.005
    )
    tibble::tibble(
      path = path_index,
      price = sum(loan_cashflows$cashflow *
        exp(-cumsum(rate_path) / 12)),
      weighted_average_life = sum(
        loan_cashflows$month / 12 *
        (loan_cashflows$scheduled_principal +
          loan_cashflows$prepaid_principal)
      ) / 100
    )
  }
)

```

```

    )
  }
)

show_early_redemption_tbl_LRL(
  dplyr::summarise(
    tbl_mbs_mc,
    simulated_paths = dplyr::n(),
    mean_price = mean(price),
    mean_weighted_average_life = mean(weighted_average_life)
  ),
  "Hull-White金利経路で更新したMBSの月次キャッシュフロー"
)
#> # A tibble: 1 x 3
#>   simulated_paths mean_price mean_weighted_average_life
#>   <int>          <dbl>          <dbl>
#> 1       200      117.35          288.0

```

短期金利と借換え金利を各経路で同時に更新し、残高を**経路ごとに**更新した。実務の評価ではシミュレーション刻みと経路数、短期金利と借換え金利の相応、プリペイメント関数の過去データへの適合を検証する。月末の残高と借換え金利より精緻にするには、短期金利の期間積分を第 II-8.4 節と同じく扱う。

OAS z に対応する $P(z)$ は、経路 j の m のキャッシュフロー $CF_m^{(j)}$ を使い、

$$P(z) = \mathbb{E} \left[\sum_{m=1}^{360} CF_m \exp \left(- \int_0^{m/12} (r_s + z) ds \right) \right]$$

と計算する。市場価格があれば $P(z)$ と一致させる z を解く。金利経路で変化するキャッシュフローと割引率の両方があるため、**経路ごとの現在価値を計算してから平均する**。平均キャッシュフローだけ、固定曲線で割り引く場合は別の簡約法となる。

MathWorks の [Computing Option-Adjusted Spread](#) は、0・50・100% PSA に対してゼロカーブと取引価格から OAS を解く例を示す。以下では同じ**比較の設計**を使う。架空の価格を 95（ディスカウント）と 105（プレミアム）に設定し、金利を 4% で固定して PSA ごとに価格を一致させるスプレッドを求める。これはキャッシュフローをあらかじめ確定する**静的な OAS 感応度**であり、前節の金利経路を使った OAS とは区別する。

```

compute_mbs_static_oas_LRL <- function(
  tbl_cashflows, market_price, zero_rate = 0.04
) {
  stopifnot(market_price > 0, is.finite(zero_rate))

```

```

price_for_spread <- function(spread) {
  sum(tbl_cashflows$cashflow * exp(
    -(zero_rate + spread) * tbl_cashflows$month / 12
  ))
}

stats::uniroot(
  function(spread) price_for_spread(spread) - market_price,
  interval = c(-0.10, 0.30),
  extendInt = "yes"
)$root
}

tbl_psa_oas <- tidyr::crossing(
  psa_percent = c(0, 50, 100),
  assumed_price = c(95, 105)
) |>
dplyr::mutate(
  cashflows = purrr::map(
    psa_percent,
    ~compute_mbs_cashflows_LRL(
      original_balance = 100, short_rate = 0.05,
      pass_through_rate = 0.45,
      refinance_rate = 0.05,
      incentive_sensitivity = 0.05,
      psa_multiplier = x / 100
    )
  ),
  oas_bp = purrr::map2_dbl(
    cashflows, assumed_price,
    ~100 * compute_mbs_statistics_LRL(.x, .y)
  )
) |>
dplyr::select(-cashflows)

show_early_redemption(tbl_LRL(
  tbl_psa_oas, "PSAと采空の取引価格に対応する静的OAS (bp)"
))

#> # A tibble: 6 x 3
#>   psa_percent assumed_price   oas_bp
#>   <dbl>         <dbl>     <dbl>
#> 1         0           95  92.686
#> 2         0          105   9.0333
#> 3        50           95 113.39
#> 4        50          105 -10.549
#> 5       100           95 130.11

```

#> 6	100	105 -26.010
------	-----	-------------

同一価格を説明する OAS は、プリペイメント仮定によって変わる。ディスカウント証券では元本が早く額面で戻るほど有利だが、プレミアム証券では額面での早期回収が不利になりやすい。結果の比較では約定価格を固定し、PSA と OAS のどちらを動かしたかを明記する。ここでの 95 と 105 は観測価格ではなく、関係を示すための仮定である。

実務では借換え金利の反応ラグ、ローンの経過月数、地域差、借換えを済ませた借り手の選別（バーンアウト）、季節性、延滞・デフォルト、手数料、元利金の支払遅延も推定対象になる。PSA はプリペイメント速度のベンチマークであり、住宅ローン借入者の個別行動の推定モデルではない。日本の RMBS では[日本証券業協会の PSJ](#) も検討する。CMO ではさらにトランシェごとの元本配分規則が必要となる。MathWorks の[Prepayment Model](#) を参考として Sequential、PAC、TAC の配分方法を整理している。本章の「スルー」を、そのまま CMO の各トランシェ価格とはみなせない。

第 II-9 章金利為替系デリバティブ

これまでの章では、債券、金利先物、Swap、Option、Callable Bond および CDS を、それぞれの主要なリスク要因に基づいて評価した。本章では、第 II 部の締めくくりとして、金利と為替を組み合わせたエキゾチック・デリバティブである PRDC を計算する。

PRDC (Power Reverse Dual Currency Note) は、為替レートに依存する Coupon を支払う長期の仕組債である。典型的な Coupon rate は、次の形で表される。

$$C(S_t) = \min \left[\max \left(a + b \frac{S_t}{S_0} - b, C_{\min} \right), C_{\max} \right]$$

ここで、

- S_t は Coupon 決定日における為替レート
- S_0 は Coupon 計算の基準為替レート
- a は為替変動係数
- b は一定付加率
- $C_{\min}$ は Coupon Floor
- $C_{\max}$ は Coupon Cap

である。

為替レートが上昇すると Coupon rate は増加するが、Floor と Cap によって下限と上限が制限される。したがって、PRDC Coupon は FX Spot に対して区分線形である一方、将来価値は為替分布に依存する非線形商品となる。

本章では、Domestic 金利、Foreign 金利および FX volatility を一定とする Garman-Kohlhagen モデルを用いて、将来の Coupon 日ごとに FX パスを生成する。各パスで Coupon を計算し、その平均値から Coupon Leg と Redemption Leg の現在価値を求める。

実際の PRDC には、発行体の期限前償還権を含むことが多い。本章では商品の基本構造を明確にするため、**発行体コール権を含まない非 Callable PRDC** を評価する。Callable PRDC には、金利・為替の複数因子モデル、相関構造および Least-Squares Monte Carlo などが追加で必要となる。

II-9.1 金利為替系デリバティブの商品構造と市場条件

2015 年 9 月 3 日から 2041 年 9 月 3 日まで、半年ごとに Coupon を支払う PRDC を考える。最初の 1 年間は固定 Coupon とし、その後は FX 連動 Coupon へ移行する。

評価日は 2023 年 4 月 11 日であるため、固定 Intro Coupon 期間はすでに終了している。ただし、GaussQuant の PRDC 評価関数が Intro Coupon を扱えることを確認するため、契約条件として残す。

本章では、計算時間を抑えながら Monte Carlo 評価を確認するため、標準評価のパラメータを 3,000 本、感応度評価を 1,000 本とする。

```
valuation_date_prdc <- as.Date("2023-04-11")
effective_date_prdc <- as.Date("2015-09-03")
termination_date_prdc <- as.Date("2041-09-03")
intro_coupon_end_date_prdc <- as.Date("2016-09-03")
```

```
notional_prdc <- 1000000
intro_coupon_rate_prdc <- 0.022
coupon_frequency_months_prdc <- 6L
```

```
domestic_discount_rate_prdc <- 0.01
foreign_discount_rate_prdc <- 0.01
discount_rate_prdc <- 0.005
```

```
fx_spot_prdc <- 133.00
fx_volatility_prdc <- 0.10
```

```
coupon_fx_base_prdc <- 120
coupon_leverage_prdc <- 0.122
coupon_subtraction_prdc <- 0.10
coupon_floor_prdc <- 0
coupon_cap_prdc <- 0.022
```

```
number_of_paths_prdc <- 300L
sensitivity_paths_prdc <- 100L
random_seed_prdc <- 123L
```

```

GaussQuant::set_eval_date_GQL(
  valuation_date_prdc
)

prdc_input_tbl <- tibble::tribble(
  ~input, ~value,
  "valuation_date", as.character(valuation_date_prdc),
  "effective_date", as.character(effective_date_prdc),
  "termination_date", as.character(termination_date_prdc),
  "intro_coupon_end_date", as.character(intro_coupon_end_date_prdc),
  "notional_jpy", as.character(notional_prdc),
  "intro_coupon_rate", as.character(intro_coupon_rate_prdc),
  "coupon_frequency_months", as.character(coupon_frequency_months_prdc),
  "domestic_rate", as.character(domestic_rate_prdc),
  "foreign_rate", as.character(foreign_rate_prdc),
  "discount_rate", as.character(discount_rate_prdc),
  "fx_spot", as.character(fx_spot_prdc),
  "fx_volatility", as.character(fx_volatility_prdc),
  "coupon_fx_base", as.character(coupon_fx_base_prdc),
  "coupon_leverage", as.character(coupon_leverage_prdc),
  "coupon_subtraction", as.character(coupon_subtraction_prdc),
  "coupon_floor", as.character(coupon_floor_prdc),
  "coupon_cap", as.character(coupon_cap_prdc),
  "valuation_paths", as.character(valuation_paths_prdc),
  "sensitivity_paths", as.character(sensitivity_paths_prdc),
  "random_seed", as.character(random_seed_prdc)
)

GaussQuant::show_tbl_CQH(
  prdc_input_tbl,
  "PRL product and model assumptions",
  n = 30
)

#> # A tibble: 20 x
#>   input      value
#>   <chr>      <chr>
#> 1 valuation_date 2023-04-11
#> 2 effective_date 2015-09-03
#> 3 termination_date 2041-09-03
#> 4 intro_coupon_end_date 2016-09-03
#> 5 notional_jpy 300000000
#> 6 intro_coupon_rate 0.022
#> 7 coupon_frequency_months 6
#> 8 domestic_rate 0.01
#> 9 foreign_rate 0.03

```



```
#> 10 discount_rate      0.005
#> 11 fx_spot            133.2681
#> 12 fx_volatility      0.1
#> 13 coupon_fx_base     120
#> 14 coupon_leverage    0.122
#> 15 coupon_subtraction  0.1
#> 16 coupon_floor       0
#> 17 coupon_cap         0.022
#> 18 valuation_paths    300
#> 19 sensitivity_paths  100
#> 20 random_seed        123
```

契約開始日から満期までの全 Coupon Schedule と、固定 Intro Coupon 期間の Schedule を作成する。

```
valuation_date_ql_prdc <- GaussQuant::date_GQL(
  valuation_date_prdc
)

effective_date_ql_prdc <- GaussQuant::date_GQL(
  effective_date_prdc
)

termination_date_ql_prdc <- GaussQuant::date_GQL(
  termination_date_prdc
)

intro_coupon_schedule_ql_prdc <- GaussQuant::date_GQL(
  intro_coupon_start_date_prdc
)

coupon_schedule_prdc <- QuantLib::Schedule(
  effective_date_ql_prdc,
  termination_date_ql_prdc,
  GaussQuant::period_months_GQL(
    coupon_frequency_months_prdc
  ),
  QuantLib::TARGET(),
  "ModifiedFollowing",
  "ModifiedFollowing",
  "Backward",
  FALSE
)

intro_coupon_schedule_prdc <- QuantLib::Schedule(
```

```

effective_date_ql_prdc,
intro_coupon_end_date_ql_prdc,
GaussQuant::period_months_GQL(
  coupon_frequency_months_prdc
),
QuantLib::TARGET(),
"ModifiedFollowing",
"ModifiedFollowing",
"Backward",
FALSE
)

coupon_schedule_tbl <- GaussQuant::schedule_table_GQL(
  coupon_schedule_prdc
)

intro_coupon_dates_prdc <- GaussQuant::schedule_dates_GQL(
  intro_coupon_schedule_prdc
)

remaining_intro_coupon_dates_prdc <- intro_coupon_dates_prdc[
  intro_coupon_dates_prdc >
  valuation_date_prdc
]

intro_coupon_status_tbl <- tibble::tibble(
  ~metric, ~value,
  "intro_coupon_rate", intro_coupon_rate_prdc,
  "total_schedule_dates", length(intro_coupon_dates_prdc),
  "remaining_intro_coupon_dates", length(remaining_intro_coupon_dates_prdc),
  "has_remaining_intro_coupon", as.numeric(
    length(
      remaining_intro_coupon_dates_prdc
    ) > 0
  )
)

GaussQuant::show_tbl_GQH(
  coupon_schedule_tbl,
  "PRDC full coupon schedule",
  n = 60
)

#> # A tibble: 53 x 2
#>   period schedule_date
#>   <int> <date>

```

```
#> 1      1 2015-09-03
#> 2      2 2016-03-03
#> 3      3 2016-09-05
#> 4      4 2017-03-03
#> 5      5 2017-09-04
#> 6      6 2018-03-05
#> 7      7 2018-09-03
#> 8      8 2019-03-04
#> 9      9 2019-09-03
#> 10     10 2020-03-03
#> # i 43 more rows

GaussQuant::show_tbl_GQH(
  intro_coupon_status_tbl,
  "PRDC introductory coupon status",
  n = 20
)
#> # A tibble: 4 x 2
#>   metric                                value
#>   <chr>                                <dbl>
#> 1 intro_coupon_rate                    0.0000000
#> 2 total_intro_schedule_date            NA
#> 3 remaining_intro_coupon_date          0
#> 4 has_remaining_intro_coupon            1
```

FX Quote を、外国通貨 1 単位当たりの円貨額として扱う。

$S_t = \text{円貨額 per unit of foreign currency}$

このとき、

- Domestic Curve は JPY 金利
- Foreign Curve は 外国通貨金利

である。

Garman–Kohlhagen モデルのリスク中立ドリフトには、Domestic 金利と Foreign 金利の差を使用する。

$$\frac{dS_t}{S_t} = (r_d - r_f)dt + \sigma_{\text{FX}}dW_t$$

本例では、Domestic 金利を 1%、Foreign 金利を 3% とする。そのため、リスク中立測度の下で

の FX ドリフトは年率マイナス 2% となる。

Coupon と元本の現在価値には、担保またはファンディング条件を表す外生的な割引率 0.5% を使用する。これは簡略化した設定であり、本格的な評価では Domestic OIS Curve と Collateral 条件を整合させる必要がある。

```
fx_day_counter_prdc <- QuantLib::Actual360()

domestic_curve_prdc <- GaussQuant::flat_curve_GQL(
  reference_date = valuation_date_ql_prdc,
  rate = domestic_rate_prdc,
  day_counter = fx_day_counter_prdc,
  compounding = "Continuous"
)

foreign_curve_prdc <- GaussQuant::flat_curve_GQL(
  reference_date = valuation_date_ql_prdc,
  rate = foreign_rate_prdc,
  day_counter = fx_day_counter_prdc,
  compounding = "Continuous"
)

discount_curve_prdc <- GaussQuant::flat_curve_GQL(
  reference_date = valuation_date_ql_prdc,
  rate = discount_rate_prdc,
  day_counter = fx_day_counter_prdc,
  compounding = "Continuous"
)

domestic_curve_handle_prdc <- QuantLib::YieldTermStructureHandle(
  domestic_curve_prdc
)

foreign_curve_handle_prdc <- QuantLib::YieldTermStructureHandle(
  foreign_curve_prdc
)

discount_curve_handle_prdc <- QuantLib::YieldTermStructureHandle(
  discount_curve_prdc
)

curve_summary_tbl <- tibble::tribble(
  ~curve, ~flat_rate,
  "Domestic FX curve", domestic_rate_prdc,
  "Foreign FX curve", foreign_rate_prdc,
```

```

"PRDC discount curve", discount_rate_prdc,
"FX risk-neutral drift", domestic_rate_prdc -
  foreign_rate_prdc
)

GaussQuant::show_tbl_GQH(
  curve_summary_tbl,
  "PRDC flat-curve assumptions",
  n = 20
)

#> # A tibble: 4 x 2
#>   curve          flat_rate
#>   <chr>          <dbl>
#> 1 Domestic FX curve      0.01
#> 2 Foreign FX curve       0.03
#> 3 PRDC discount curve    0.005
#> 4 FX risk-neutral drift -0.02

```

II-9.2 FX プロセスと Coupon ペイオフ

FX volatility を 10% とし、Garman-Kohlhagen モデルを用いる。

```

fx_volatility_curve_prdc <- QuantLib::FlatCurveVol(
  valuation_date_q1,
  QuantLib::NullCalendar(),
  GaussQuant::quote_handle_GQL(
    fx_flat_rate_summary_prdc
  ),
  fx_discount_factor_prdc
)

fx_volatility_handle_prdc <- QuantLib::BlackVolTermStructureHandle(
  fx_volatility_curve_prdc
)

fx_process_prdc <- GaussQuant::fx_process_GQL(
  spot = fx_spot_prdc,
  foreign_curve_handle = foreign_curve_handle_prdc,
  domestic_curve_handle = domestic_curve_handle_prdc,
  volatility_handle = fx_volatility_handle_prdc
)

fx_market_summary_tbl <- tibble::tribble(

```

```

~metric, ~value,
"fx_spot", fx_spot_prdc,
"fx_volatility", fx_volatility_prdc,
"domestic_rate", domestic_rate_prdc,
"foreign_rate", foreign_rate_prdc,
"risk_neutral_drift", domestic_rate_prdc -
  foreign_rate_prdc
)

GaussQuant::show_tbl_GQH(
  fx_market_summary_tbl,
  "PRDC FX market assumptions",
  n = 20
)

#> # A tibble: 5 x 2
#>   metric      value
#>   <chr>      <dbl>
#> 1 fx_spot    133.27
#> 2 fx_volatility 0.1
#> 3 domestic_rate 0.01
#> 4 foreign_rate 0.0
#> 5 risk_neutral_drift -0.0

```

本例の FX 連動 Coupon は次式である。

$$C_{\text{FX}} = \min \left[2.2\%, \max \left(2.2\% \frac{S}{120} - 10\%, 0 \right) \right]$$

Floor に到達する FX 水準は、

$$S_{\text{floor}} = S_0 \frac{b + C_{\min}}{a}$$

Cap に到達する FX 水準は、

$$S_{\text{cap}} = S_0 \frac{b + C_{\max}}{a}$$

である。

```

fx_floor_threshold_prdc <- coupon_fx_base_prdc *
(
  coupon_subtraction_prdc +

```

```

      coupon_floor_prdc
    ) /
    coupon_leverage_prdc

fx_cap_threshold_prdc <- coupon_fx_base_prdc *
  (
    coupon_subtraction_prdc +
    coupon_cap_prdc
  ) /
  coupon_leverage_prdc

fx_payoff_grid_tbl <- tibble::tibble(
  fx_rate = seq(
    70,
    180,
    by = 2
  )
) |>
dplyr::mutate(
  uncapped_coupon = coupon_leverage_prdc *
    fx_rate /
    coupon_fx_base_prdc -
    coupon_subtraction_prdc,
  coupon_rate = GaussQuantile(
    fx_rate = fx_rate,
    fx_base = coupon_fx_base_prdc,
    leverage = coupon_leverage_prdc,
    subtraction = coupon_subtraction_prdc,
    floor_rate = coupon_floor_prdc,
    cap_rate = coupon_cap_prdc
  )
)

coupon_threshold_tbl <- tibble::tribble(
  ~threshold, ~fx_rate,
  "floor_threshold", fx_floor_threshold_prdc,
  "cap_threshold", fx_cap_threshold_prdc,
  "current_spot", fx_spot_prdc
)

prdc_payoff_plot <- ggplot2::ggplot(
  fx_payoff_grid_tbl,
  ggplot2::aes(
    x = fx_rate,
    y = coupon_rate
  )
)

```

```

)
) +
ggplot2::geom_line(
  linewidth = 0.8
) +
ggplot2::geom_vline(
  xintercept = c(
    fx_floor_threshold_prdc,
    fx_cap_threshold_prdc
  ),
  linetype = "dashed"
) +
ggplot2::scale_y_continuous(
  labels = scales::percent_format(
    accuracy = 0.1
  )
) +
ggplot2::labs(
  title = "PRDC coupon payoff",
  subtitle = "Floored and capped PRDC coupon",
  x = "FX rate",
  y = "Coupon rate"
)

```

```

GaussQuant::show_tbl_GQH(
  coupon_threshold_tbl,
  "PRDC coupon FX thresholds",
  n = 20
)
#> # A tibble: 2 x 3
#>   threshold      fx_rate
#>   <chr>          <dbl>
#> 1 floor_threshold 98.061
#> 2 cap_threshold   20
#> 3 current_spot   33.27

```

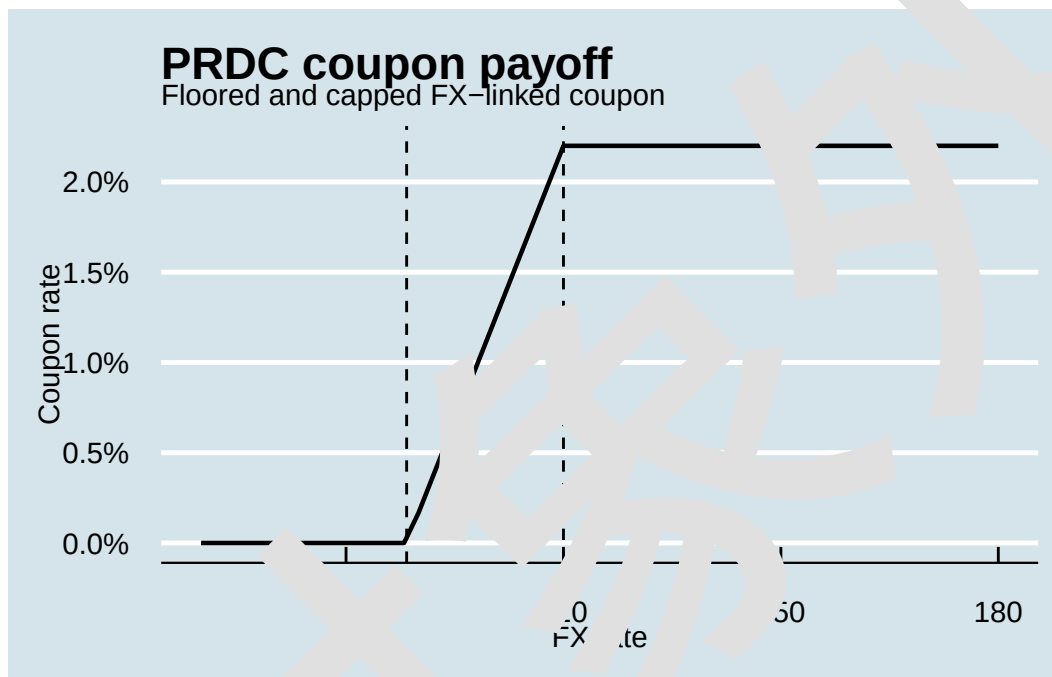
```

GaussQuant::show_tbl_GQH(
  fx_payoff_grid_tbl,
  "PRDC coupon payoff table",
  n = 60
)
#> # A tibble: 56 x 3
#>   fx_rate uncapped_coupon coupon_rate
#>   <dbl>         <dbl>         <dbl>
#> 1     70         -0.028833         0

```



```
#> 2      72      -0.026800      0
#> 3      74      -0.024767      0
#> 4      76      -0.022733      0
#> 5      78      -0.0207      0
#> 6      80      -0.018667      0
#> 7      82      -0.016633      0
#> 8      84      -0.014600      0
#> 9      86      -0.012567      0
#> 10     88      -0.010533      0
#> # i 46 more rows
show_plot_LFL(prdc_payoff_plot)
```



現在の FX レートは Cap 到達水準を上回っているため、評価時点の Coupon 式だけを見れば Cap に影響していない。しかし、満期までの FX 分布には Floor、Linear 領域および Cap 領域のすべてが含まれる。

II-9.3 モンテカルロによる金利為替系デリバティブ評価

PRDC 評価に必要なのは、評価日より後に到来する Coupon 日である。GaussQuant の `fx_path_generator_GQL()` は、実際の Coupon 日を用いて不均一な時間間隔を保持する。

```
fx_path_generator_prdc <- GaussQuant::fx_path_generator_GQL(
  valuation_date = valuation_date_prdc,
  coupon_schedule = coupon_schedule_prdc,
  day_counter = fx_day_counter_prdc,
  process = fx_process_prdc
)
```

```

time_grid_tbl <- tibble::tibble(
  coupon_number = seq_along(
    fx_path_generator_prdc$remaining_coupon_dates
  ),
  coupon_date = fx_path_generator_prdc$remaining_coupon_dates,
  time_from_valuation = fx_path_generator_prdc$coupon_times,
  time_step = fx_path_generator_prdc$time_steps
)

path_generator_summary_tbl <- tibble::tribble(
  ~metric, ~value,
  "remaining_coupon_dates", length(
    fx_path_generator_prdc$remaining_coupon_dates
  ),
  "number_of_steps", fx_path_generator_prdc$number_of_steps,
  "first_coupon_time", min(
    fx_path_generator_prdc$coupon_times
  ),
  "final_coupon_time", max(
    fx_path_generator_prdc$coupon_times
  )
)

GaussQuant::show_tbl_GQH(
  path_generator_summary_tbl,
  "PRDC FX path-generator summary",
  n = 20
)
#> # A tibble: 1 x 2
#>   metric value
#>   <chr>   <dbl>
#> 1 remaining_coupon_dates 37
#> 2 number_of_steps      37
#> 3 first_coupon_time     0.40556
#> 4 final_coupon_time     18.667

GaussQuant::show_tbl_GQH(
  time_grid_tbl,
  "PRDC remaining coupon-date time grid",
  n = 60
)
#> # A tibble: 37 x 4
#>   coupon_number coupon_date time_from_valuation time_step
#>   <int> <date>           <dbl>         <dbl>

```

```
#> 1          1 2023-09-04          0.40556 0.40556
#> 2          2 2024-03-04          0.91111 0.50556
#> 3          3 2024-09-03          1.4194 0.50833
#> 4          4 2025-03-03          1.9222 0.50278
#> 5          5 2025-09-03          2.4333 0.51111
#> 6          6 2026-03-03          2.9361 0.50278
#> 7          7 2026-09-03          3.4472 0.51111
#> 8          8 2027-03-03          3.95 0.50278
#> 9          9 2027-09-03          4.4611 0.51111
#> 10         10 2028-03-03          4.9667 0.50556
#> # i 27 more rows
```

QuantLib の `StochasticProcess1D$evolve()` へは標準正規変量を用いる。Brownian increment を外側で二重に時間調整してはならない。GaussianRandomPathGenerator では、標準正規乱数と各 Coupon 間の実際の時間差を分けて処理している。

Monte Carlo 評価へ進む前に、20 本の FX パスを生成し、各パスの FX 分布と Coupon の動きを確認する。

```
sample_path_count_prdc <- 20L

sample_fx_path_matrix_prdc <- GaussianRandomPathGenerator(
  path_generator = fx_path_generator_prdc,
  n_paths = sample_path_count_prdc,
  seed = random_seed
)

sample_fx_paths_tbl <- purrr::map_dfr(
  seq_along(sample_path_count_prdc),
  function(path_id) {
    path_values <- sample_fx_path_matrix_prdc[
      path_id,
    ]

    tibble::tibble(
      path_id = path_id,
      coupon_number = seq_along(
        path_values
      ),
      coupon_date = fx_path_generator_prdc$remaining_coupon_dates,
      time = fx_path_generator_prdc$coupon_times,
```

```

    fx_rate = path_values,
    coupon_rate = GaussQuant::prdc_coupon_GQL(
      fx_rate = path_values,
      fx_base = coupon_fx_base_prdc,
      leverage = coupon_leverage_prdc,
      subtraction = coupon_subtraction_prdc,
      floor_rate = coupon_floor_prdc,
      cap_rate = coupon_cap_prdc
    )
  )
}
)

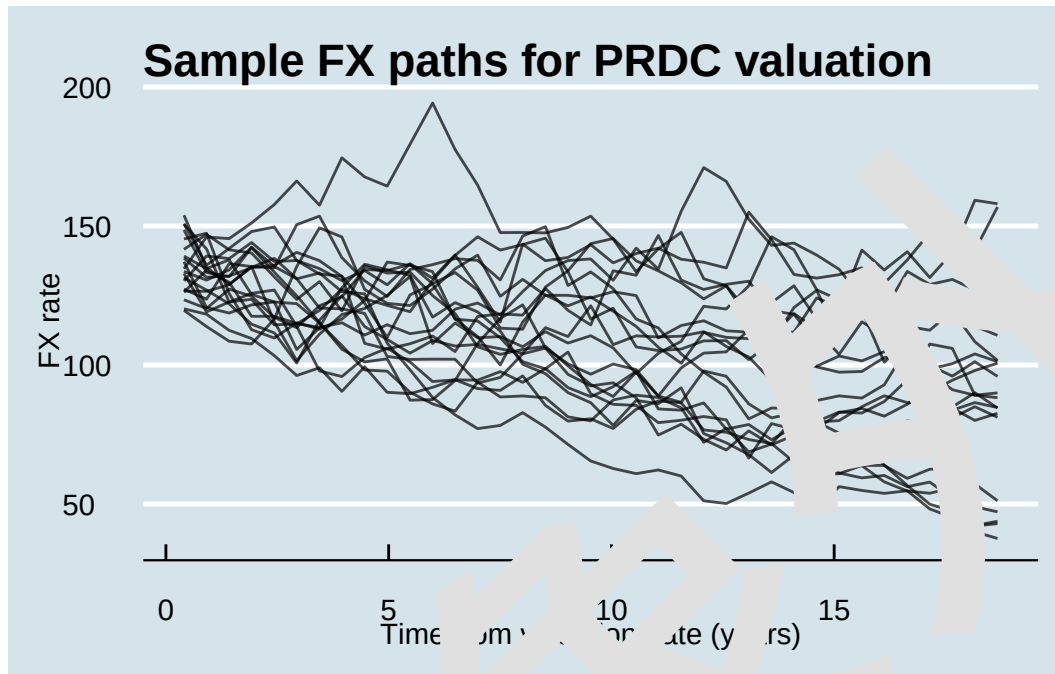
sample_fx_path_plot <- ggplot2::ggplot(
  sample_fx_path_tbl,
  ggplot2::aes(
    x = time,
    y = fx_rate,
    group = path_id
  )
) +
  ggplot2::geom_line(
    linewidth = 0.5,
    alpha = 0.7
  ) +
  ggplot2::labs(
    title = "Sample coupon paths for PRDC evaluation",
    x = "Time to maturity date (years)",
    y = "FX rate"
  )
)

GaussQuant::prdc_coupon_GQL(
  sample_fx_path_tbl,
  "Sample PRDC FX and coupon paths",
  n = 40
)

#> # A tibble: 40 x 6
#>   path_id coupon_number coupon_date    time fx_rate coupon_rate
#>   <int>      <int> <date>      <dbl>   <dbl>      <dbl>
#> 1         1          1 2023-09-04  0.40556 127.30      0.022
#> 2         1          2 2024-03-04  0.91111 123.66      0.022
#> 3         1          3 2024-09-03  1.4194 136.45      0.022
#> 4         1          4 2025-03-03  1.9222 135.42      0.022
#> 5         1          5 2025-09-03  2.4333 134.94      0.022
#> 6         1          6 2026-03-03  2.9361 150.49      0.022

```

```
#> 7      1      7 2026-09-03  3.4472  153.56    0.022
#> 8      1      8 2027-03-03  3.95    138.63    0.022
#> 9      1      9 2027-09-03  4.4611  130.31    0.022
#> 10     1     10 2028-03-03  4.9667  124.66    0.022
#> # i 30 more rows
show_plot_LFL(sample_fx_path_plot)
```



各 Coupon 日の期待 Coupon rate は、モンテカルロシミュレーションによって次のように近似する。

$$E[C(S_{t_i})] \approx \frac{1}{M} \sum_{j=1}^M C(S_{t_i}^{(j)})$$

ここで、 $S_{t_i}^{(j)}$ はパス j における Coupon 日 t_i の FX Rate である。

各 Coupon 日

$$CF_i = N_0 \alpha_i E[C(S_{t_i})]$$

であり、PRDC 現在価値は、

$$PV_{\text{PRDC}} = \sum_i D(0, t_i) CF_i + D(0, T) N_0$$

となる。

```

prdc_result <- GaussQuant::prdc_npv_GQL(
  valuation_date = valuation_date_prdc,
  coupon_schedule = coupon_schedule_prdc,
  process = fx_process_prdc,
  discount_curve = discount_curve_prdc,
  notional = notional_prdc,
  fx_base = coupon_fx_base_prdc,
  leverage = coupon_leverage_prdc,
  subtraction = coupon_subtraction_prdc,
  floor_rate = coupon_floor_prdc,
  cap_rate = coupon_cap_prdc,
  day_counter = fx_day_counter_prdc,
  intro_coupon_dates = intro_coupon_dates_prdc,
  intro_coupon_rate = intro_coupon_rate_prdc,
  n_paths = number_of_paths_prdc,
  seed = random_seed_prdc,
  redemption_amount = notional_prdc,
  return_details = TRUE
)

fx_path_matrix_prdc <- prdc_result$fx_path_matrix
expected_coupon_tbl <- prdc_result$expected_coupon_tbl
prdc_cashflow_tbl <- prdc_result$cashflows

coupon_leg_present_value_prdc <- prdc_result$summary |>
  dplyr::filter(
    metric == "coupon_leg_npv"
  ) |>
  dplyr::pull(
    value
  )

redemption_present_value_prdc <- prdc_result$summary |>
  dplyr::filter(
    metric == "redemption_leg_npv"
  ) |>
  dplyr::pull(
    value
  )

prdc_present_value_jpy <- prdc_result$npv

prdc_valuation_summary_tbl <- tibble::tribble(
  ~metric, ~value,
  "coupon_leg_pv_jpy", coupon_leg_present_value_prdc,

```

```

"redemption_leg_pv_jpy", redemption_present_value_prdc,
"total_pv_jpy", prdc_present_value_jpy,
"pv_as_percent_of_notional", prdc_present_value_jpy /
  notional_prdc,
"number_of_paths", number_of_paths_prdc,
"number_of_coupon_steps", nrow(
  fx_path_matrix_prdc
)
)
)

```

```

GaussQuant::show_tbl_GQH(
  prdc_valuation_summary_tbl,
  "PRDC Monte Carlo valuation summary",
  n = 20
)

```

```

#> # A tibble: 6 x 2
#>   metric                                value
#>   <chr>                                <dbl>
#> 1 coupon_leg_pv_jpy                    6.2094e7
#> 2 redemption_leg_pv_jpy                2.75e8
#> 3 total_pv_jpy                          353000000
#> 4 pv_as_percent_of_notional             1.7779e-01
#> 5 number_of_paths                       1000000
#> 6 number_of_coupon_steps                 100000

```

本章の元本償還額は固定されているため、Redemption Leg は割引率に依存するが FX には依存しない。FX Spot と FX Volatility の影響は、主に Coupon Leg を通じて現れる。

```

expected_coupon_display_tbl <- expected_coupon_tbl |>
  dplyr::select(
    coupon_number,
    coupon_date,
    time,
    expected_fx,
    expected_coupon_rate,
    is_intro_coupon,
    final_coupon_rate
  )

```

```

prdc_cashflow_display_tbl <- prdc_cashflow_tbl |>
  dplyr::select(
    coupon_number,
    accrual_start,
    accrual_end,
    final_coupon_rate,

```

```

    accrual_fraction,
    coupon_amount,
    redemption_amount,
    total_amount,
    discount_factor,
    total_present_value
  )

```

```
GaussQuant::show_tbl_GQH(
```

```

  expected_coupon_display_tbl,
  "PRDC expected FX and coupon rates",
  n = 60

```

```
)
```

```
#> # A tibble: 37 x 7
```

```

#>   coupon_number coupon_date    time expected_1 expected_coupon_rate
#>   <int> <date>      <dbl>      <dbl>      <dbl>
#> 1           1 2023-09-04 0.40556    132.60    0.021746
#> 2           2 2024-03-04 0.91111    131.83    0.021017
#> 3           3 2024-09-03 1.4194    130.23    0.020090
#> 4           4 2025-03-03 1.9277    128.77    0.018675
#> 5           5 2025-09-03 2.4361    126.44    0.017577
#> 6           6 2026-03-03 2.9444    124.24    0.016621
#> 7           7 2026-09-03 3.4528    122.17    0.016350
#> 8           8 2027-03-03 3.9611    120.20    0.015578
#> 9           9 2027-09-03 4.4695    118.31    0.014453
#> 10          10 2028-03-03 4.9778    116.47    0.013940

```

```
#> # i 27 more rows
```

```
#> # i 6 more variables: initial_coupon_rate <dbl>, final_coupon_rate <dbl>
```

```
GaussQuant::show_tbl_GQH(
```

```

  prdc_cashflow_display_tbl,
  "PRDC expected cashflow table",
  n = 60

```

```
)
```

```
#> # A tibble: 37 x 6
```

```

#>   coupon_number accrual_start accrual_end final_coupon_rate accrual_fraction
#>   <int> <date>      <date>      <dbl>      <dbl>
#> 1           1 2023-03-03    2023-09-04    0.021746    0.51389
#> 2           2 2023-09-04    2024-03-04    0.021017    0.50556
#> 3           3 2024-03-04    2024-09-03    0.020090    0.50833
#> 4           4 2024-09-03    2025-03-03    0.018675    0.50278
#> 5           5 2025-03-03    2025-09-03    0.017577    0.51111
#> 6           6 2025-09-03    2026-03-03    0.016621    0.50278
#> 7           7 2026-03-03    2026-09-03    0.016350    0.51111
#> 8           8 2026-09-03    2027-03-03    0.015578    0.50278

```

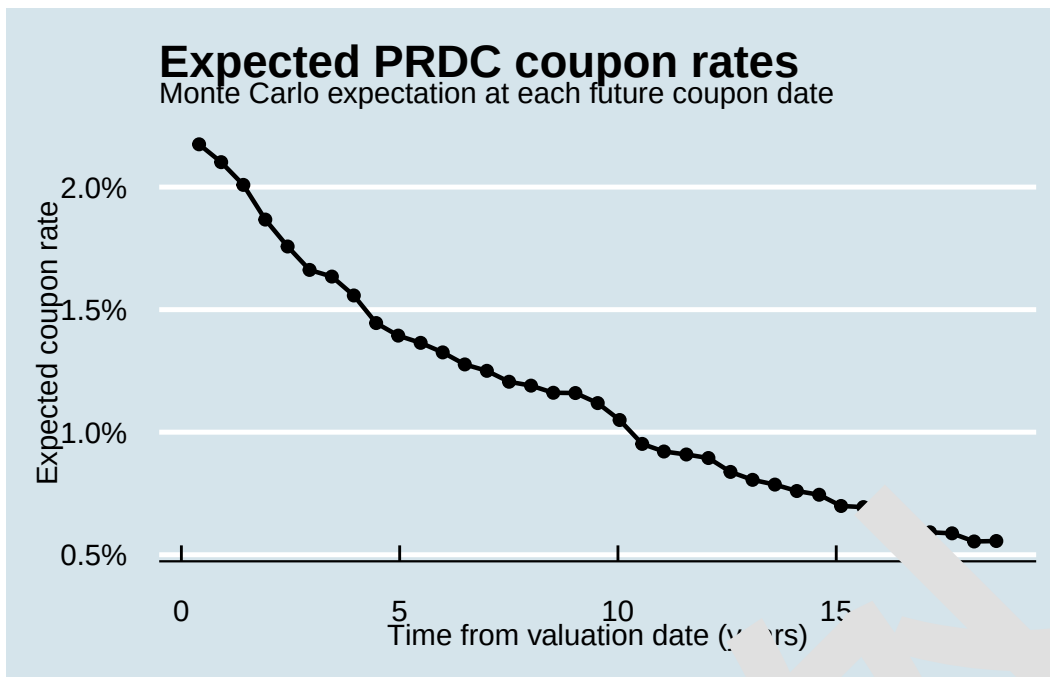


```
#> 9          9 2027-03-03    2027-09-03          0.014453          0.51111
#> 10         10 2027-09-03    2028-03-03          0.013940          0.50556
#> # i 27 more rows
#> # i 5 more variables: coupon_amount <dbl>, redemption_amount <dbl>,
#> #   total_amount <dbl>, discount_factor <dbl>, total_present_value <dbl>
```

長期の FX 期待値は Domestic 金利と Foreign 金利の差の影響を受ける。本例では Foreign 金利が Domestic 金利より高いため、リスク中立測度の下では FX Rate に下向きのドリフトが働く。その結果、満期が長くなるほど Cap 領域から Linear 領域または Floor 領域へ移る確率が高くなる。

II-9.4 Coupon の期間構造と分布

```
expected_coupon_plot <- ggplot2::ggplot(
  expected_coupon_tbl,
  ggplot2::aes(
    x = time,
    y = final_coupon_rate
  )
) +
  ggplot2::geom_line(
    linewidth = 0
  ) +
  ggplot2::geom_point(
    size = 1.8
  ) +
  ggplot2::scale_y_continuous(
    labels = scales::percent_format(
      accur
    )
  ) +
  ggplot2::labs(
    title = "Expected PRDC coupon rates",
    subtitle = "Monte Carlo expectation at each future coupon date",
    x = "Time from valuation date (years)",
    y = "Expected coupon rate"
  )
show_plot_LFL(expected_coupon_plot)
```



期待 Coupon は、単に期待 FX を Coupon 式へ代入した値とは一般に一致しない。

$$E[C(t)] \neq E[S_t]$$

Cap と Floor を含む非線形 Payoff では、この点全体を考慮する必要がある。

各 Coupon 日について、シミュレーションで各 Coupon が Floor または Cap に到達する確率を計算する。

```
coupon_cap_floor_probability_tbl <- GaussQuant::prdc_cap_floor_probability_GQL(
  path_matrix = fx_generator_prdc,
  fx_base = coupon_fx_base_prdc,
  leverage = coupon_leverage_prdc,
  subtraction = coupon_subtraction_prdc,
  floor_rate = floor_rate_prdc,
  cap_rate = coupon_cap_rate_prdc,
  coupon_dates = fx_generator_prdc$remaining_coupon_dates,
  coupon_times = fx_generator_prdc$coupon_times
)

GaussQuant::show_tbl_GQH(
  coupon_cap_floor_probability_tbl,
  "PRDC coupon floor and cap probabilities",
  n = 60
)

#> # A tibble: 37 x 6
#>   coupon_number floor_probability cap_probability interior_probability
#>   <int>          <dbl>          <dbl>          <dbl>
```

```
#> 1          1          0          0.93333          0.066667
#> 2          2          0          0.82          0.18
#> 3          3          0.0066667          0.76333          0.23
#> 4          4          0.023333          0.67          0.30667
#> 5          5          0.056667          0.61667          0.32667
#> 6          6          0.1          0.58667          0.31333
#> 7          7          0.10667          0.54333          0.35
#> 8          8          0.14667          0.49          0.36333
#> 9          9          0.16333          0.47333          0.36333
#> 10         10          0.19333          0.48          0.32667
#> # i 27 more rows
#> # i 2 more variables: coupon_date <date>, time <dbl>
```

Cap 確率、Linear 領域の確率および Floor 確率は、長期の値に近づく傾向があります。現在の Spot が Cap 水準を上回っていても、長期 Coupon が Cap を固定されることは限らない。

短期、中期、長期の代表的な Coupon 日について、Coupon 確率を比較する。

```
coupon_path_matrix_prdc <- GaussQuadrature::prdc_coupon_GQL(
  fx_rate = fx_path_matrix_prdc,
  fx_base = coupon_fx_base_prdc,
  leverage = coupon_leverage_prdc,
  subtraction = coupon_subtraction_prdc,
  floor_rate = coupon_floor_rate_prdc,
  cap_rate = coupon_cap_rate_prdc
)

number_of_coupon_steps_prdc <- nrow(
  coupon_path_matrix_prdc
)

distribution <- dlist(unique(
  pmax(
    1L,
    pmin(
      number_of_coupon_steps_prdc,
      c(
        1L,
        round(
          number_of_coupon_steps_prdc /
            2
        ),
        number_of_coupon_steps_prdc
      )
    )
  )
)
```

```

)
)

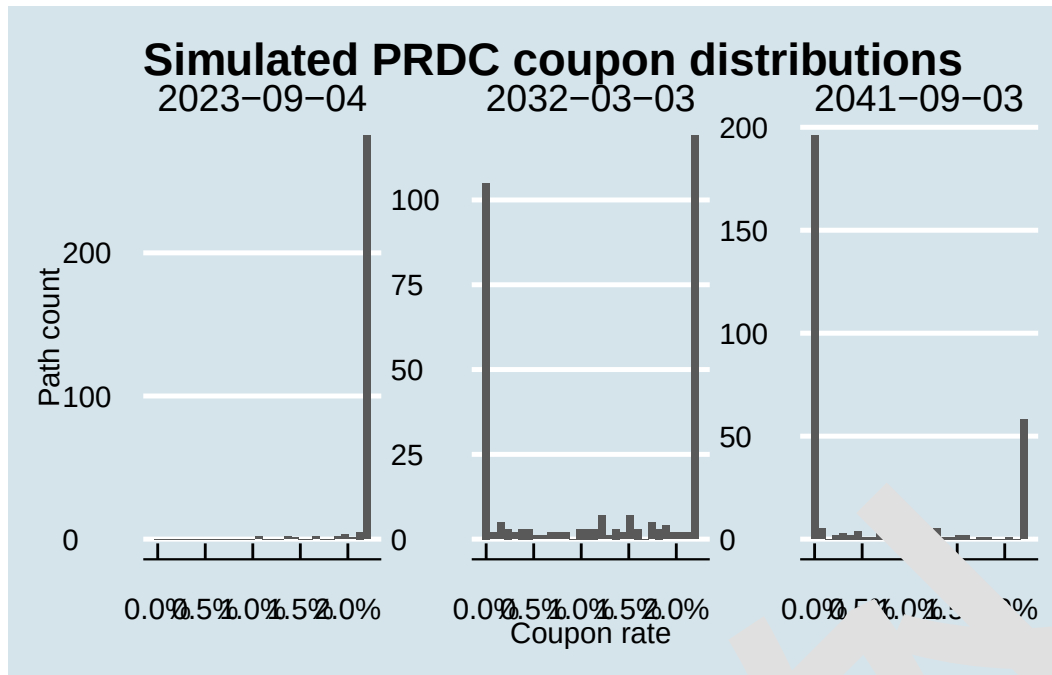
coupon_distribution_tbl <- purrr::map_dfr(
  distribution_coupon_ids,
  function(coupon_id) {
    tibble::tibble(
      coupon_number = coupon_id,
      coupon_date = as.character(
        fx_path_generator_prdc$remaining_coupon_dates[
          coupon_id
        ]
      ),
      coupon_rate = coupon_path_matrix_prdc[
        coupon_id,
      ]
    )
  }
)

```

```

coupon_distribution_plot <- ggplot2::plot(
  coupon_distribution_tbl,
  ggplot2::aes(
    x = coupon_rate
  )
) +
  ggplot2::geom_histogram(
    bin = 30,
    line = 0.3
  ) +
  ggplot2::facet_wrap(
    ~coupon,
    scales = "free_x"
  ) +
  ggplot2::scale_x_continuous(
    labels = scales::percent_format(
      accuracy = 0.1
    )
  ) +
  ggplot2::labs(
    title = "Simulated PRDC coupon distributions",
    x = "Coupon rate",
    y = "Path count"
  )
show_plot_LFL(coupon_distribution_plot)

```



Coupon 分布は連続的な正規分布または対数正規分布ではない。Floor と Cap により、質量が集まり、その間に Linear 領域の連続分布が存在する。

II-9.5 FX 感応度とモンテカルロ誤差

PRDC Coupon は FX Spot へ感応度を持つ。Spot 感応度シナリオを比較する。

各シナリオで同じ乱数 Seed を使用することにより、Monte Carlo ノイズを抑えた Common Random Numbers 型のシミュレーションとなる。

```
spot_scenario_input_tbl <- tribble(
  ~scenario, fx_spot,
  "spot_plus_10", fx_spot_prdc * 1.10,
  "spot_plus_5", fx_spot_prdc * 1.05,
  "base", fx_spot_prdc,
  "spot_minus_5", fx_spot_prdc * 0.95,
  "spot_minus_10", fx_spot_prdc * 0.90,
)

spot_sensitivity_tbl <- GaussQuant::prdc_spot_sensitivity_GQL(
  spot_scenarios = spot_scenario_input_tbl,
  valuation_date = valuation_date_prdc,
  coupon_schedule = coupon_schedule_prdc,
  foreign_curve_handle = foreign_curve_handle_prdc,
  domestic_curve_handle = domestic_curve_handle_prdc,
  volatility_handle = fx_volatility_handle_prdc,
  discount_curve = discount_curve_prdc,
  notional = notional_prdc,
```

```

fx_base = coupon_fx_base_prdc,
leverage = coupon_leverage_prdc,
subtraction = coupon_subtraction_prdc,
floor_rate = coupon_floor_prdc,
cap_rate = coupon_cap_prdc,
day_counter = fx_day_counter_prdc,
intro_coupon_dates = intro_coupon_dates_prdc,
intro_coupon_rate = intro_coupon_rate_prdc,
n_paths = sensitivity_paths_prdc,
seed = random_seed_prdc,
redemption_amount = notional_prdc,
base_scenario = "base"
)

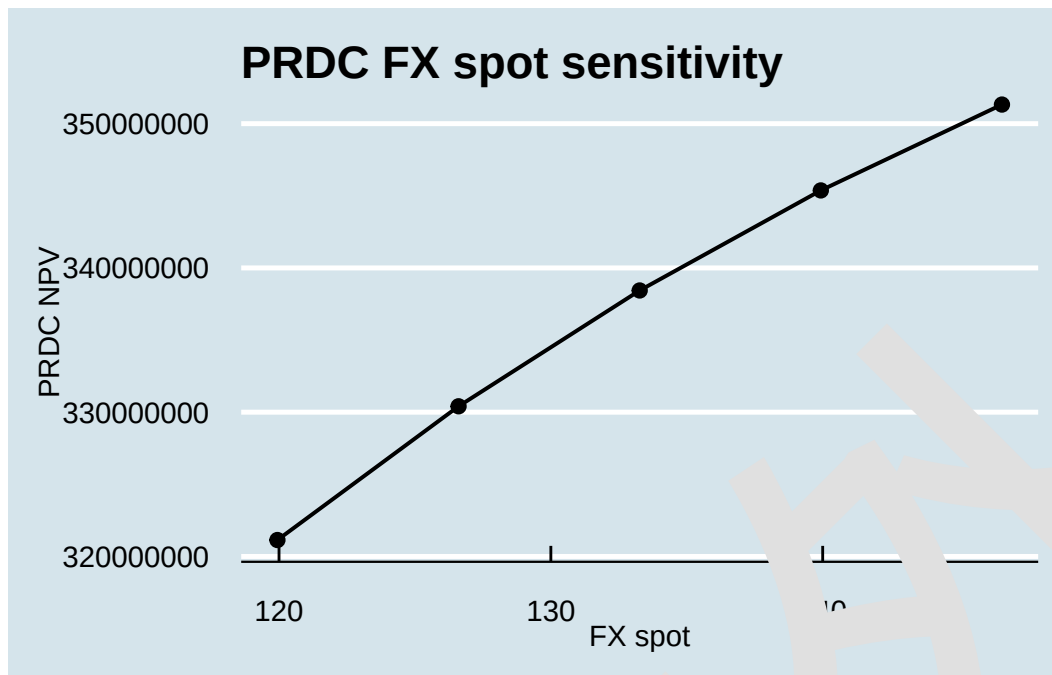
spot_sensitivity_plot <- ggplot2::ggplot(
  spot_sensitivity_tbl,
  ggplot2::aes(
    x = fx_spot,
    y = prdc_npv
  )
) +
  ggplot2::geom_line(
    linewidth = 0.7
  ) +
  ggplot2::geom_point(
    size = 2.2
  ) +
  ggplot2::labs(
    title = "PRDC FX spot sensitivity",
    x = "FX spot",
    y = "PRDC NPV"
  )

GaussQuant::show_tbl_QH(
  spot_sensitivity_tbl,
  "PRDC FX spot sensitivity",
  n = 20
)

#> # A tibble: 5 x 4
#>   scenario    fx_spot prdc_npv npv_difference_from_base
#>   <chr>      <dbl>    <dbl>          <dbl>
#> 1 spot_minus_10 119.94 321139116. -17297149.
#> 2 spot_minus_5 126.60 330408227. -8028038.
#> 3 base         133.27 338436265.    0
#> 4 spot_plus_5 139.93 345370836. 6934571.

```

```
#> 5 spot_plus_10 146.59 351322971. 12886706.
show_plot_LFL(spot_sensitivity_plot)
```



Spot 上昇は Coupon を上昇させる方向に作用するが、Cap へ到達した後の境界効果は小さくなる。Spot 低下時も、Floor へ到達するまで Coupon の低下は続きます。このため、PRDC 価値の Spot 感応度は全域で一定ではない。

PRDC Coupon は、Floor と Cap を含む変動利付 Par Bond である。そのため、FX volatility の変化は将来の FX 分布たが、Floor、Cap へ入る確率を変化させる。

```
volatility_scenario_input_tbl <- tibble::tibble(
  ~scenario, fx_volatility,
  "vol_5pct", 0.05,
  "vol_10pct", 0.1,
  "vol_15pct", 0.15,
  "base", fx_volatility_prdc,
  "vol_12_5pct", 0.125,
  "vol_15pct", 0.15
)

volatility_sensitivity_tbl <- GaussQuant::prdc_volatility_sensitivity_GQL(
  volatility_scenarios = volatility_scenario_input_tbl,
  valuation_date = valuation_date_prdc,
  coupon_schedule = coupon_schedule_prdc,
  spot = fx_spot_prdc,
  foreign_curve_handle = foreign_curve_handle_prdc,
  domestic_curve_handle = domestic_curve_handle_prdc,
  discount_curve = discount_curve_prdc,
  notional = notional_prdc,
```

```

fx_base = coupon_fx_base_prdc,
leverage = coupon_leverage_prdc,
subtraction = coupon_subtraction_prdc,
floor_rate = coupon_floor_prdc,
cap_rate = coupon_cap_prdc,
day_counter = fx_day_counter_prdc,
calendar = QuantLib::NullCalendar(),
intro_coupon_dates = intro_coupon_dates_prdc,
intro_coupon_rate = intro_coupon_rate_prdc,
n_paths = sensitivity_paths_prdc,
seed = random_seed_prdc,
redemption_amount = notional_prdc,
base_scenario = "base"
)

```

```

volatility_sensitivity_plot <- ggplot2::ggplot(
  volatility_sensitivity_tbl,
  ggplot2::aes(
    x = fx_volatility,
    y = prdc_npv
  )
) +
  ggplot2::geom_line(
    linewidth = 0.7
  ) +
  ggplot2::geom_point(
    size = 2.2
  ) +
  ggplot2::scale_x_continuous(
    labels = scales::percent_format(accuracy = 1)
  ) +
  ggplot2::labs(
    title = "PRDC FX volatility sensitivity",
    x = "FX volatility",
    y = "PRDC NPV"
  )

```

```

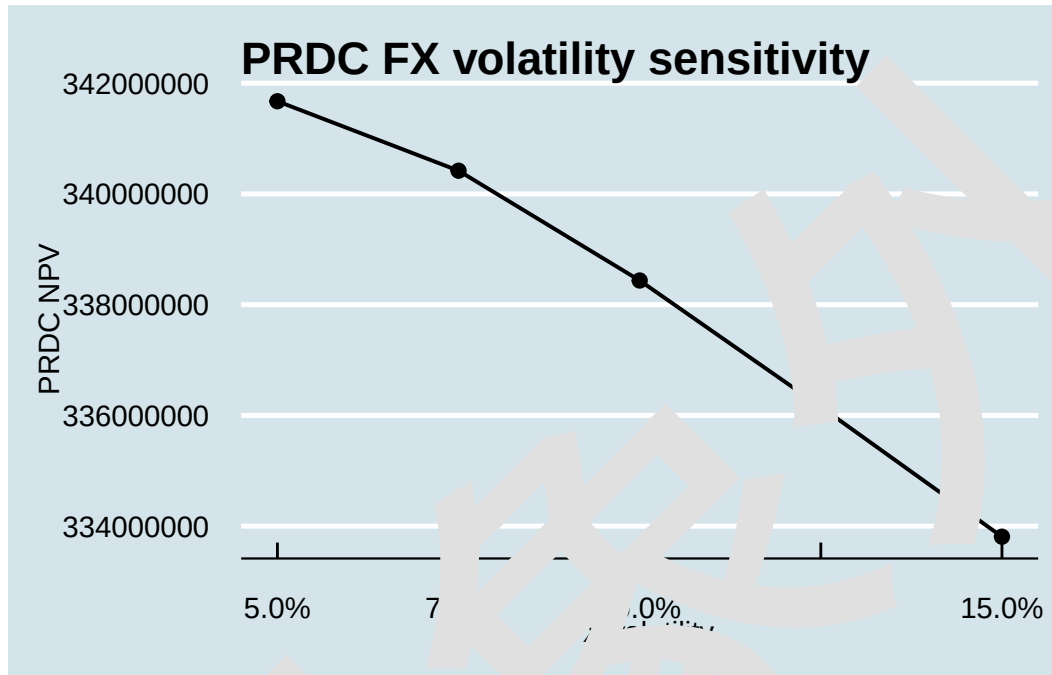
GaussQuant::show_tbl_GQH(
  volatility_sensitivity_tbl,
  "PRDC FX volatility sensitivity",
  n = 20
)

#> # A tibble: 5 x 4

```



```
#>   scenario   fx_volatility   prdc_npv npv_difference_from_base
#>   <chr>         <dbl>         <dbl>         <dbl>
#> 1 vol_5pct      0.05   341673690.         3237426.
#> 2 vol_7_5pct    0.075  340419233.         1982968.
#> 3 base          0.1    338436265.           0
#> 4 vol_12_5pct   0.125  336154662.        -2281603.
#> 5 vol_15pct     0.15   333808877.        -4627388.
show_plot_LFL(volatility_sensitivity_plot)
```



通常の Option のように、Volatility 上昇は必ずしも価値上昇と単純には言えない。PRDC Coupon は Cap 付きの上昇 Payer であると同時に Floor を持ち、現在の Spot、金利差、満期および各境界への到達率によって Volatility 感応度が変わる。

Monte Carlo 推定には統計誤差が存在する。各パスについて全 Coupon と Redemption の現在価値を計算し、平均、標準偏差、標準誤差および信頼区間を求める。

標準誤差は、パス現在価値の標準偏差を $\hat{\sigma}_{PV}$ 、パス数を M とすると、

$$SE = \frac{\hat{\sigma}_{PV}}{\sqrt{M}}$$

である。

```
monte_carlo_error_result <- GaussQuant::prdc_monte_carlo_error_GQL(
  path_matrix = fx_path_matrix_prdc,
  cashflow_tbl = prdc_cashflow_tbl,
  notional = notional_prdc,
  fx_base = coupon_fx_base_prdc,
```

```

leverage = coupon_leverage_prdc,
subtraction = coupon_subtraction_prdc,
floor_rate = coupon_floor_prdc,
cap_rate = coupon_cap_prdc,
intro_coupon_dates = intro_coupon_dates_prdc,
intro_coupon_rate = intro_coupon_rate_prdc,
confidence_level = 0.95
)

path_present_value_tbl <- monte_carlo_error_result$path_present_values
monte_carlo_error_tbl <- monte_carlo_error_result$summary

GaussQuant::show_tbl_GQH(
  monte_carlo_error_tbl,
  "PRDC Monte Carlo error estimate",
  n = 20
)
#> # A tibble: 7 x 2
#>   metric                                value
#>   <chr>                                <dbl>
#> 1 number_of_paths                      30
#> 2 mean_present_value                  3336151.
#> 3 present_value_sd                    11784
#> 4 standard_error                      2100.
#> 5 lower_confidence_bound 331352782
#> 6 upper_confidence_bound 3393693
#> 7 confidence_level                    0.95

```

パス数を増やすと、他の条件が同じであれば標準誤差は概ね半分になる。ただし、計算量も増加する。実務上は、パス数を増やすだけでなく、Antithetic Variates、Control Variates、Quasi-Monte Carlo などの分散削減法を併用する。

II-9.6 モデルの境界と Part II 総括

本章のモデルは、PRDC の基本的な非線形 Coupon と Monte Carlo 評価を理解するための簡略モデルである。主な制約は次のとおりである。

Domestic、Foreign および Discount Curve をフラットかつ決定論的とした。実際の長期 PRDC では、Domestic 金利と Foreign 金利の変動が、FX Forward、割引係数および Coupon 価値へ影響する。

一定の Black volatility を使用した。実際の FX Option 市場には、満期方向と Strike 方向の

Volatility Surface が存在する。長期 PRDC では、Smile、Skew および Volatility term structure の影響が重要となる。

本格的な PRDC モデルでは、Domestic 金利、Foreign 金利および FX の間の相関を扱う必要がある。金利と FX を別々に評価して単純に合成するだけでは、長期の Cross-currency リスクを十分に表現できない。

実際の PRDC には、発行体が Coupon 日に商品を期限前償還できる Callable 条項が含まれることが多い。

発行体は各行使日で、

Call value と Continuation value

を比較する。将来の Continuation value は Closed-form で求められないため、Least-Squares Monte Carlo または Backward Induction を使用する。

本章の評価結果は、非 Callable PRDC の Coupon Leg と Redemption Leg の価値である。Callable PRDC の投資家価値は、概略的に次式となる。

$$V_{\text{callable}} = V_{\text{non-callable PRDC}} - \text{issuer call}$$

第 II 部では、金融商品のキャッシュフロー、クーポン・インセンション、確率モデルおよび数値計算を段階的に扱う。

```
part2_chapter_summary.tbl <- tribble(
  ~chapter ~subject, ~method,
  "II-1", "Bonds and market conventions", "Yield, price, duration, and convexity",
  "II-2", "Interest rate derivatives", "Basis, carry, implied repo, and futures",
  ~ BPV,
  "II-3", "OIS and multi-curve valuation", "Discount and forecast curve",
  ~ separation,
  "II-4", "Black model", "Lognormal forward-option pricing",
  "II-5", "Credit risk and CDS", "Hazard rates and CDS valuation",
  "II-6", "Interest-rate model calibration", "SABR, Hull-White, and BGM",
  ~ calibration,
  "II-7", "Short-rate and forward-rate models", "Hull-White, HJM, BGM/LMM, and RFR",
  ~ FMM,
  "II-8", "Callable bonds and MBS prepayment", "Issuer call valuation and mortgage",
  ~ prepayment cash flows,
  "II-9", "Exotic derivatives and PRDC", "FX-linked coupon Monte Carlo valuation"
)
```

```
GaussQuant::show_tbl_GQH(
  part2_chapter_summary_tbl,
  "Part II chapter summary",
  n = 20
)
#> # A tibble: 9 x 3
#>   chapter subject                                main_method
#>   <chr>   <chr>                                <chr>
#> 1 II-1    Bonds and market conventions      Yield, price, duration, and convex~
#> 2 II-2    Treasury futures and CTD                Basis, carry, implied repo, and fu~
#> 3 II-3    OIS and multi-curve valuation            Discount and forecast curve separa~
#> 4 II-4    Black model                               Lognormal forward option pricing
#> 5 II-5    Credit risk and CDS                       Hazard rate and transition
#> 6 II-6    Interest-rate model calibration           Short rate, Hull-White, and GJM calibrat~
#> 7 II-7    Short-rate and forward-rate models       Hull-White, GJM, BGM/Libor, and RFR ~
#> 8 II-8    Callable bonds and MBS prepayment         Issuance, amortization and mortgage~
#> 9 II-9    Exotic derivatives and PRDC               FX-linked coupon Monte Carlo valua~
```

債券評価から始まり、金利先物、OIS、Credit Risk、Volatility Smile、短金利モデル、信用リスク、そして複数の市場要因を持つエキゾチックな商品へと進んだ。

Part II を通じた共通原則は、商品ごとに異なる数式を暗記することではない。次の要素を分離し、再利用可能な計算層へ落とすことである。

- 契約条件と Schedule
- 市場データと Implied
- Risk-Neutral 測度と確率過程
- Cash flow と Payoff
- Pricing Engine または数値計算法
- NPV、Risk および Calibration
- 表示と検証

GaussQuant は、QuantLib SWIG の低水準オブジェクトを金融商品の計算層へまとめる。LagrangeFinance の章側は、商品構造、数式、入力条件、表、グラフおよび金融的解釈に集中する。

II-9.7 まとめ

PRDC は、将来の FX Rate に連動する Coupon を持つ長期の仕組債である。

Coupon rate は、

$$C(S) = \min \left[C_{\max}, \max \left(a \frac{S}{S_0} - b, C_{\min} \right) \right]$$

という Floor 付き・Cap 付きの非線形 Payoff である。

Garman-Kohlhagen モデルでは、FX Rate のリスク中立ドリフトは Domestic 金利と Foreign 金利の差によって決まる。

$$\frac{dS_t}{S_t} = (r_d - r_f)dt + \sigma_{\text{FX}}dW_t$$

Coupon 日の FX 分布を Monte Carlo で生成し、各 Coupon 日の期待 Coupon、Coupon Leg、Redemption Leg および PRDC NPV を計算した。

FX Spot と FX volatility の感応度は、Floor と Cap によって非線形となる。Monte Carlo 評価には統計誤差があるため、NPV だけでなく標準誤差と信頼区間を求める必要がある。

本章は非 Callable PRDC を対象とした。実際の Callable PRDC には、Domestic 金利、Foreign 金利、FX、Volatility Surface、相関など、より多様な要素の行使判断を同時に扱う、より高度なモデルが必要となる。

```
chapter09_summary_tbl <- tibble(
  ~section, ~main_result,
  "Coupon payoff", "Floored and capped FX-linked coupon",
  "FX process", "Garman-Kohlhagen risk-neutral dynamics",
  "Monte Carlo valuation", "Coupon leg and redemption leg NPV",
  "Spot sensitivity", "Nonlinear response caused by cap and floor",
  "Volatility sensitivity", "Distributional effect on coupon regions",
  "Monte Carlo error", "Standard error and confidence interval",
  "Model boundary", "Non-callable deterministic-rate PRDC"
)

GaussQuant::show_tbl_QH(
  chapter09_summary_tbl,
  "Chapter II-9 summary",
  n = 20
)

#> # A tibble: 7 x 2
#>   section                main_result
#>   <chr>                  <chr>
#> 1 Coupon payoff        Floored and capped FX-linked coupon
#> 2 FX process            Garman-Kohlhagen risk-neutral dynamics
#> 3 Monte Carlo valuation Coupon leg and redemption leg NPV
#> 4 Spot sensitivity      Nonlinear response caused by cap and floor
```

```
#> 5 Volatility sensitivity Distributional effect on coupon regions
#> 6 Monte Carlo error          Standard error and confidence interval
#> 7 Model boundary            Non-callable deterministic-rate PRDC

cat(
  "\nChapter II-9 Exotic derivatives and PRDC completed successfully.\n"
)
#>
#> Chapter II-9 Exotic derivatives and PRDC completed successfully.
```

エンタープライゼーション

第 III-1 章金融計算技術の発展

金融工学は、数学理論だけで発展してきたのではない。1960 年代以降、金融理論の発展とコンピュータの発展は互いに影響しながら進んできた。理論上は表現できても計算できなかった問題が、計算機能力の向上によって実務へ取り込まれ、逆に金融市場の拡大が新しい計算技術が必要としてきた。

本章では、まずこの約 60 年の流れを大きく振り返る。次に、2020 年現在 Cloud、HPC、GPU、Open Source Software、Tensor 計算 Library がそろったことで、金融計算がどこまで変わったのかを整理する。そのうえで、第 III 部前半の目玉の一つとして、一台の PC の複数 Core を利用する並列計算を R の `future / future` で実装する。

第 III 部全体の中心にあるのは、金融計算を加速させる二つの方向である。一つは、CPU、GPU、Grid、Cloud などの計算資源を加速させる **Infrastructure Acceleration** であり、もう一つは、Automatic Differentiation によって計算方法そのものを変える **Algorithmic Acceleration** である。

III-1.1 金融における情報工学の歩み

1960 年代から 1970 年代にかけて、Portfolio Theory、CAPM、Black-Scholes / Merton モデルなど、現代金融工学の基盤となる理論が整備された。当時の計算資源は現在と比べて極めて限られており、実務へ取り込みやすいモデルは解析解を持つものが中心であった。

1973 年の Black-Scholes / Merton モデルは、その象徴的な例である。European Option の価格を

$$C = C(S, K, r, \sigma, T)$$

とすると、Delta、Gamma、Vega などの Greeks も解析的に表すことができる。

しかし 1980 年代以降、早期行使、経路依存性、複雑な Payoff、複数のリスクファクターを持つ商品が増えるにつれて、閉形式の解析解だけでは十分でなくなった。PDE、Finite Difference、

Binomial / Trinomial Tree などの数値計算が実務へ広がった。

1990 年代には Monte Carlo Simulation が、複雑なデリバティブ評価や VaR をはじめとするリスク管理の重要な計算基盤となった。価格は多数の Path または Scenario の平均として、概念的に

$$\hat{V} = \frac{1}{M} \sum_{m=1}^M V^{(m)}$$

と評価される。Path 数を増やせば統計的精度を高められる一方、計算量も増加する。さらに Portfolio、Scenario、Risk Factor が増えると、必要となる Pricing 工数は急速に膨らむ。

2000 年代には、大規模 Portfolio や複雑な Exotic / Hybrid 商品の評価、さらには、金融機関で On-premise Grid Computing が広く利用されるようになった。Portfolio / Scenario を多数の Job へ分割し、Scheduler が複数の Server へ配布するという構成である。

同じ時期、CPU は単純な Clock Speed の向上から Multi-core 化へ進んだ。一台の Server 内部でも複数の Task を同時に実行できるようになり、金融計算は「一つずつ順番に計算する」構造から「独立した計算を分割して同時に実行する」構造へ変わっていった。

2010 年代には GPU による Massive Parallel Computation と Cloud Computing が急速に普及した。Monte Carlo のように、同様の計算を多数の Path へ適用する処理は GPU と相性がよく、Tensor 演算によって大規模な計算をまとめて処理できるようになった。

この流れを大きく整理すると次のようになる。

```

1960s-1970s
Portfolio Optim / CAPM / Analytical Pricing
    |
    v
1980s
PDE / Finite Difference / Tree
    |
    v
1990s
Monte Carlo / VaR / Bump & Revalue
    |
    v
2000s
On-premise Grid / Job Scheduler / Multi-core
    |
    v
2010s
  
```

```

GPU / Cloud / Open Source / Tensor Computing
    |
    v
2020s
Flexible HPC / Massive Monte Carlo / Automatic Differentiation

```

III-1.2 リーマンショックを経たテクノロジーの潮流

2020 年代の変化は、単にコンピュータが高速になったことではない。大きいのは、計算資源を「所有する」ことから、必要なときに必要な規模だけ「利用する」方向へ移ったことである。

従来の On-premise 環境では、大規模な計算を行うには Server、Storage、Network scheduler などを自ら用意し、最大負荷を想定した設備投資を行うことが多かった。現在、Cloud や HPC Service を利用することで、計算負荷に応じて CPU や GPU を柔軟に追加できる SaaS 型・Managed Service 型の計算基盤も増え、インフラの構築と保守にかかる負担は大きく低下した。

同時に、R や Python を中心とする Open Source Software が成熟した。数値計算、統計、最適化、Machine Learning、Distributed Computing、GPU Tensor 計算、商用の専用計算環境だけに依存せず組み合わせることができるようになった。

この二つが重なることで、以前は大規模な計算専用 Grid でなければ現実的でなかった計算を、より安価かつ柔軟な環境で実行できるようになった。問題設定と計算環境によって必要時間は大きく異なるものの、数百万から数百万、さらには十億以上の GPU / HPC 資源を用いる場合には数億 Path 規模を視野に入れた Monte Carlo Simulation も現実的な設計対象となっている。

```

On-premise Software
    |
    v
Elastic Cloud / HPC Service
    +
Open Source Software
    +
CPU / GPU Tensor Computing
    |
    v
Massive Monte Carlo

```

III-1.3 金融計算におけるハードウェアの限界

計算資源を増やすことで、一回の Pricing は速くできる。しかし Risk Sensitivity を Bump & Revalue で求める場合、Risk Factor ごとに Pricing を繰り返す構造は残る。

例えば中心差分では、

$$\frac{\partial V}{\partial x} \approx \frac{V(x+h) - V(x-h)}{2h}$$

となる。Risk Factor が 100 個あれば、多数の Bump を作り、そのための大規模 Monte Carlo を再実行する必要がある。

したがって、

More Cores / More GPU ≠ Fewer Revaluation

である。Hardware を増やすだけでは、Bump & Revalue という Algorithm そのものが要求する再計算回数は減らない。

Automatic Differentiation 自体は新しい概念ではないが、しかし近年、TensorFlow、Keras、PyTorch、R の torch のような Deep Learning 発展によって成熟した Tensor 計算 Library が、安価で安定した Automatic Differentiation 環境を提供するようになった。

これらの Library では、Pricing を Tensor 演算として記述すると、その計算過程が Computational Graph として保持される。最終的な価格から Reverse Mode で Graph を逆向きにたどることで、多数の入力 Risk Factor に対する Gradient を効率的に取得できる。

$$(x_1, x_2, \dots, x_n) \rightarrow \text{Monte Carlo Pricing} \rightarrow V$$

に対して、

$$\nabla V = \left(\frac{\partial V}{\partial x_1}, \dots, \frac{\partial V}{\partial x_n} \right)$$

を求める。厳密には Forward Pricing に続いて Backward Pass を行うが、Risk Factor ごとに独立した Monte Carlo Pricing を何度もやり直す必要はない。

ここで金融計算は、

Hardware Acceleration → Hardware + Algorithmic Acceleration

へ進む。

第 III 部では、この二つの方向を R で実装する。

前半の第 III-2 章から第 III-4 章では、`future` / `furrr` を出発点として、PC 内の並列処理、非同期実行、Job / Task Control、分散計算、そしてエンタープライズ金融計算システムの構造を扱う。計算をどのように分割し、どこで実行するかが中心となる。

後半の第 III-5 章から第 III-7 章では、R の `torch` を用いて Automatic Differentiation による Risk Sensitivity を実装する。対象は Option から Yield Curve、さらに CRDC の多因子 Risk へ段階的に拡張する。

Option Greeks → Interest Rate Curve Risk → CRDC Multi-factor Risk

本部全体の到達点は、次の形で表すことができます。

Scale Up / Scale Out + Automatic Differentiation → Enterprise Differentiable Risk Engine

まず本章では、その第一歩として一台の PC 中の計算を分割し、複数 Worker へ配布する並列計算を実装する。

III-1.4 並列コンピューティング

R では `purrr` の `map` 関数と、`future` / `furrr` による並列処理をよく似た形で記述できる。金融計算では、入力データを `tibble` として保持し、Pricing Function を各行へ適用する構造にすると、逐次計算から並列計算へ移行しやすい。

Trade / Market / Scenario Data

```

      |
      v
Pricing Function
      |
+-----+-----+
|           |           |
map()    future_map()
|           |           |
  
```

```

+-----+-----+
|
|
v
Result

```

まず、単純な現在価値計算を Task として定義する。

```

calculate_present_value_LFL <- function(cashflow, discount_rate, maturity) {
  tibble(
    cashflow = cashflow,
    discount_rate = discount_rate,
    maturity = maturity,
    present_value = cashflow / (1 + discount_rate)^maturity
  )
}

```

計算対象を tibble として用意する。

```

tbl_calculation <- tribble(
  ~scenario_ID, ~cashflow, ~discount_rate, ~maturity,
  1L, 100, 0.010, 1,
  2L, 100, 0.015, 2,
  3L, 100, 0.020, 3,
  4L, 100, 0.025, 4,
  5L, 100, 0.030, 5,
  6L, 100, 0.035, 6
)

show_table_widths(tbl_calculation)
#> # A tibble: 6 x 4
#>   scenario_ID cashflow discount_rate maturity
#>   <int>      <dbl>      <dbl>      <dbl>
#> 1         1      100      0.01         1
#> 2         2      100      0.015        2
#> 3         3      100      0.02         3
#> 4         4      100      0.025        4
#> 5         5      100      0.03         5
#> 6         6      100      0.035        6

```

最初に `purrr::pmap()` で逐次実行する。

```

tbl_sequential_result <- tbl_calculation |>
  mutate(
    result = purrr::pmap(

```

```

      list(cashflow, discount_rate, maturity),
      calculate_present_value_LFL
    )
  ) |>
  select(scenario_ID, result) |>
  tidyr::unnest(result)

show_table_LFL(tbl_sequential_result)
#> # A tibble: 6 x 5
#>   sce_ID cashflow dis_rate maturity pre_value
#>   <int>    <dbl>    <dbl>    <dbl>    <dbl>
#> 1     1      100    0.01      1    99.010
#> 2     2      100    0.015     2    97.066
#> 3     3      100    0.02      3    94.232
#> 4     4      100    0.025     4    90.595
#> 5     5      100    0.03      5    86.261
#> 6     6      100    0.035     6    81.350

```

次に `future::multisession` を利用して Worker を起動する。利用可能な Core をすべて占有せず、この例では最大 2 Worker を指定する。

```

available_cores <- future::availableCores()
workers <- max(1L, min(2L, available_cores - 1L))

future::plan(
  future::multisession,
  workers = workers
)

tbl_future_setting <- tibble(
  available_cores = available_cores,
  workers = workers
)

show_table_LFL(tbl_future_setting)
#> # A tibble: 1 x 2
#>   ava_cores workers
#>   <int>    <int>
#> 1      8      2

```

計算ロジックを変えず、`purrr::pmap()` を `furrr::future_pmap()` へ置き換える。

```

tbl_parallel_result <- tbl_calculation |>
  mutate(

```

```

result = furrr::future_pmap(
  list(cashflow, discount_rate, maturity),
  calculate_present_value_LFL,
  .options = furrr::furrr_options(seed = TRUE)
)
) |>
select(scenario_ID, result) |>
tidyr::unnest(result)

```

```
show_table_LFL(tbl_parallel_result)
```

```

#> # A tibble: 6 x 5
#>   sce_ID cashflow dis_rate maturity pre_value
#>   <int>    <dbl>    <dbl>    <dbl>    <dbl>
#> 1     1      100    0.01      1    99.010
#> 2     2      100    0.015     2    97.066
#> 3     3      100    0.02      3    94.232
#> 4     4      100    0.025     4    90.595
#> 5     5      100    0.03      5    86.261
#> 6     6      100    0.035     6    82.350

```

逐次計算と並列計算が同じ結果を返すことを確認する。

```

tbl_result_comparison <- tbl_parallel_result %>%
  select(
    scenario_ID,
    sequential_present_value = present_value
  ) |>
left_join(
  tbl_parallel_result %>%
    select(
      scenario_ID,
      parallel_present_value = present_value
    ),
  by = "scenario_ID"
) |>
mutate(
  difference = parallel_present_value - sequential_present_value
)

```

```
show_table_LFL(tbl_result_comparison)
```

```

#> # A tibble: 6 x 4
#>   sce_ID seq_value par_value difference
#>   <int>    <dbl>    <dbl>    <dbl>
#> 1     1    99.010    99.010         0
#> 2     2    97.066    97.066         0

```

```
#> 3      3      94.232      94.232      0
#> 4      4      90.595      90.595      0
#> 5      5      86.261      86.261      0
#> 6      6      81.350      81.350      0
```

```
stopifnot(
  isTRUE(
    all.equal(
      tbl_result_comparison$sequential_present_value,
      tbl_result_comparison$parallel_present_value,
      tolerance = 1e-12
    )
  )
)
```

計算終了後は Plan を逐次処理へ戻す。

```
future::plan(future::sequential)
```

III-1.5 R による並列計算

小さすぎる Task では Worker 生成やタスクの overhead が支配的になり、並列化が必ずしも高速になるとは限らない。そこで、一定時間を超える独立 Task を使って逐次処理と並列処理を比較する。

```
calculate_task_LFL <- function(task_ID) {
  Sys.sleep(0.1)

  tibble(
    task_ID = task_ID,
    result = sqrt(task_ID)
  )
}

vec_task_ID <- seq_len(12L)
```

```
sequential_time <- system.time(
  tbl_sequential_benchmark <- purrr::map(
    vec_task_ID,
    calculate_task_LFL
  )
)[["elapsed"]]
```



```

future::plan(
  future::multisession,
  workers = workers
)

parallel_time <- system.time(
  tbl_parallel_benchmark <- furrr::future_map(
    vec_task_ID,
    calculate_task_LFL,
    .options = furrr::furrr_options(seed = TRUE)
  )
)[["elapsed"]]

future::plan(future::sequential)

```

```

tbl_benchmark <- tribble(
  ~method, ~elapsed_seconds,
  "purrr::map", sequential_time,
  "furrr::future_map", parallel_time
) |>
mutate(
  relative_to_sequential = elapsed_seconds / sequential_time
)

show_table_LFL(tbl_benchmark)
#> # A tibble: 2 x 3
#>   method      elapsed_seconds rel_seque
#>   <chr>          <dbl>         <dbl>
#> 1 purrr::map      1.34          1
#> 2 furrr::future_map 0.07000      0.72388

```

この例の目的は特定の成功率を保証することではない。重要なのは、同じ Pricing Function を多数の独立 Task へ適用する場合、Task を複数 Worker へ配分することで実行時間を短縮できるという構造である。

III-1.6 並列化の効果と限界

Multi-core による並列化には明確な限界がある。一台のコンピュータで利用できる CPU Core、Memory、GPU などの計算資源には上限があるためである。

さらに実務の Risk Calculation では、数件の Task ではなく、

- 数十万件の Trade

- 多数の Market Scenario
- Monte Carlo Path
- Stress Scenario
- Sensitivity Calculation

を同時に処理することがある。

この規模になると、一台の PC の Core を増やすだけでは対応できない。必要になるのは、Task を複数のコンピュータへ配布する仕組みである。

Single Machine Parallelism → Distributed Computing

また、計算資源を増やして Pricing を高速化しても、Sensitivity calculation で求める場合には、リスクファクターごとに Pricing を繰り返す構造のものも残る。この問題は、本部後半の自動微分で改めて扱う。

III-1.7 自動微分

本章では、金融工学の計算が解析的かつ効率的に Monte Carlo へ発展するにつれて計算量が増大し、その対応として Multi-core CPU の並列処理が重要になったことを確認した。

R では `future / future` を利用することで、並列計算の構造を大きく変えずに一台の PC 上で複数 Worker へ Task を分散できる。

しかし、エンタプライズ金融計算では一台の PC だけでは計算資源が不足する。次章では、Task を複数の計算機へ配布する分散コンピューティングと Job / Task Control へ進む。

第 III-2 章分散コンピューティング

第 III-1 章では、金融工学と計算技術の発展を振り返り、一台の PC 上で `future` / `furrr` を利用する並列計算を実装した。そこで使える計算資源は、あくまで一台の Machine に搭載された CPU Core と Memory の範囲に限られる。

実務の Risk Calculation では、Trade、Scenario、Monte Carlo Path を多数の小さな Task へ分割し、それらを複数の計算資源へ配布する必要がある。本章では、PC 上で Parallel Computing から一段進み、Task 生成 → Job Queue → Scheduler → Worker → Result Aggregation という分散コンピューティングの基本構造を R で実現する。

```

Input Data
  |
  v
Task Generation
  |
  v
Job Queue
  |
  v
Scheduler
  |
  +-----+
  | | | |
W1 W2 W3 W4
  | | | |
  +-----+
  |
  v
Result Aggregation

```

III-2.1 計算処理の分割

```
tbl_task <- tribble(
  ~task_ID, ~input_value, ~worker_group,
  1L, 10, "A",
  2L, 20, "A",
  3L, 30, "B",
  4L, 40, "B",
  5L, 50, "C",
  6L, 60, "C"
)
```

```
show_table_LFL(tbl_task)
#> # A tibble: 6 x 3
#>   tas_ID inp_value wor_group
#>   <int>     <dbl> <chr>
#> 1     1         10 A
#> 2     2         20 A
#> 3     3         30 B
#> 4     4         40 B
#> 5     5         50 C
#> 6     6         60 C
```

```
calculate_task_result <- function(task_ID, input_value, worker_group) {
  tibble(
    task_ID = task_ID,
    worker_group = worker_group,
    input_value = input_value,
    output_value = sqrt(input_value)
  )
}
```

III-2.2 計算資源への分配

```
tbl_remote_request <- tbl_task |>
  transmute(
    request_ID = paste0(
      "REQ-",
      stringr::str_pad(task_ID, width = 3, pad = "0")
    ),
  ),
```

```

task_ID,
payload = purrr::pmap(
  list(task_ID, input_value, worker_group),
  \(task_ID, input_value, worker_group) {
    list(
      task_ID = task_ID,
      input_value = input_value,
      worker_group = worker_group
    )
  }
)
)

```

```

show_table_LFL(
  tbl_remote_request |>
    select(request_ID, task_ID)
)
#> # A tibble: 6 x 2
#>   req_ID tas_ID
#>   <chr>   <int>
#> 1 REQ-001     1
#> 2 REQ-002     2
#> 3 REQ-003     3
#> 4 REQ-004     4
#> 5 REQ-005     5
#> 6 REQ-006     6

```

```

remote_calculate_task_LFL <- function(payload) {
  calculate_task_result_LFL(
    task_ID = payload$task_ID,
    input_value = payload$input_value,
    worker_group = payload$worker_group
  )
}

```

```

tbl_remote_result <- tbl_remote_request |>
  mutate(
    response = purrr::map(
      payload,
      remote_calculate_task_LFL
    )
  ) |>
  select(request_ID, response) |>
  tidyr::unnest(response)

```

```
show_table_LFL(tbl_remote_result)
#> # A tibble: 6 x 5
#>   req_ID tas_ID wor_group inp_value out_value
#>   <chr>    <int> <chr>      <dbl>    <dbl>
#> 1 REQ-001      1 A          10    3.1623
#> 2 REQ-002      2 A          20    4.4721
#> 3 REQ-003      3 B          30    5.4772
#> 4 REQ-004      4 B          40    6.3246
#> 5 REQ-005      5 C          50    7.0711
#> 6 REQ-006      6 C          60    7.7460
```

III-2.3 計算処理の制御

```
tbl_job <- tbl_task |>
  transmute(
    job_ID = paste0(
      "JOB-",
      stringr::str_pad(task_ID, width = 3, side = "left")
    ),
    task_ID,
    status = "queued",
    worker_group,
    input_value
  )

show_table_LFL(tbl_job)
#> # A tibble: 6 x 5
#>   job_ID tas_ID status wor_group inp_value
#>   <chr>    <int> <chr>      <chr>    <dbl>
#> 1 JOB-001      1 queued A          10
#> 2 JOB-002      2 queued A          20
#> 3 JOB-003      3 queued B          30
#> 4 JOB-004      4 queued B          40
#> 5 JOB-005      5 queued C          50
#> 6 JOB-006      6 queued C          60
```

```
tbl_job_status <- tribble(
  ~job_ID, ~status, ~sequence,
  "JOB-001", "queued", 1L,
  "JOB-001", "running", 2L,
  "JOB-001", "completed", 3L,
```

```

  "JOB-002", "queued", 1L,
  "JOB-002", "running", 2L,
  "JOB-002", "failed", 3L
)

show_table_LFL(tbl_job_status)
#> # A tibble: 6 x 3
#>   job_ID status      sequence
#>   <chr>   <chr>         <int>
#> 1 JOB-001 queued           1
#> 2 JOB-001 running          2
#> 3 JOB-001 completed         3
#> 4 JOB-002 queued           1
#> 5 JOB-002 running          2
#> 6 JOB-002 failed           3

```

Slurm のような Scheduler では、Task を Job として登録し、適切な Worker へ割り当てる。R 側の Pricing Function は可能な限り変更せず、「どこで実行するか」を実行基盤側へ分離することが重要である。

III-2.4 分散コンピューティング

```

tbl_market_data <- tribble(
  ~market_data_ID, ~daily_interest_rate, ~fix_spot,
  "MKT-001", 0.020, 100,
  "MKT-002", 0.025, 155
)

tbl_calibration_data <- tribble(
  ~calibration_ID, ~volatility, ~correlation,
  "CAL-001", 0.15, 0.80,
  "CAL-002", 0.20, 0.70
)

tbl_trade_data <- tribble(
  ~trade_ID, ~notional, ~maturity,
  "TRD-001", 1000000, 3,
  "TRD-002", 2000000, 5
)

tbl_scenario_data <- tidyr::crossing(
  trade_ID = tbl_trade_data$trade_ID,

```

```

market_data_ID = tbl_market_data$market_data_ID,
calibration_ID = tbl_calibration_data$calibration_ID
) |>
mutate(
  scenario_ID = row_number(),
  .before = 1
)

```

```
show_table_LFL(tbl_scenario_data)
```

```

#> # A tibble: 8 x 4
#>   sce_ID tra_ID  mar_ID  cal_ID
#>   <int> <chr>   <chr>   <chr>
#> 1     1 TRD-001 MKT-001 CAL-001
#> 2     2 TRD-001 MKT-001 CAL-002
#> 3     3 TRD-001 MKT-002 CAL-001
#> 4     4 TRD-001 MKT-002 CAL-002
#> 5     5 TRD-002 MKT-001 CAL-001
#> 6     6 TRD-002 MKT-001 CAL-002
#> 7     7 TRD-002 MKT-002 CAL-001
#> 8     8 TRD-002 MKT-002 CAL-002

```

```

tbl_distributed_input <- tbl_scenario_data
  left_join(tbl_trade_data, by = "trade_ID") |>
  left_join(tbl_market_data, by = "market_data_ID") |>
  left_join(tbl_calibration_data, by = "calibration_ID")

```

```
show_table_LFL(tbl_distributed_input)
```

```

#> # A tibble: 8 x 10
#>   sce_ID tra_ID  mar_ID  cal_ID notional maturity dis_rate fx_spot volatility
#>   <int> <chr>   <chr>   <chr>   <dbl>   <dbl>   <dbl>   <dbl>   <dbl>
#> 1     1 TRD-001 MKT-001 CAL-001 1000000     3    0.02    150    0.15
#> 2     2 TRD-001 MKT-001 CAL-002 1000000     3    0.02    150    0.2
#> 3     3 TRD-001 MKT-002 CAL-001 1000000     3    0.025   155    0.15
#> 4     4 TRD-001 MKT-002 CAL-002 1000000     3    0.025   155    0.2
#> 5     5 TRD-002 MKT-001 CAL-001 2000000     5    0.02    150    0.15
#> 6     6 TRD-002 MKT-001 CAL-002 2000000     5    0.02    150    0.2
#> 7     7 TRD-002 MKT-002 CAL-001 2000000     5    0.025   155    0.15
#> 8     8 TRD-002 MKT-002 CAL-002 2000000     5    0.025   155    0.2
#> # i 1 more variable: correlation <dbl>

```

```

tbl_distributed_files <- list(
  trade_data = tbl_trade_data,
  market_data = tbl_market_data,
  calibration_data = tbl_calibration_data,

```



```

  scenario_data = tbl_scenario_data
)

vec_distributed_paths <- tbl_distributed_files |>
  purrr::imap_chr(
    \(data, name) {
      write_derived_parquet_file_LFL(
        data,
        file.path(
          "distributed",
          paste0(name, ".parquet")
        )
      )
    }
  )

tbl_distributed_paths <- tibble(
  data_name = names(vec_distributed_paths),
  file_path = file.path(
    "data",
    "derived",
    "distributed",
    paste0(names(vec_distributed_paths), ".parquet")
  )
)

show_table_LFL(tbl_distributed_paths)
#> # A tibble:
#>   data_name      file_path
#>   <chr>         <chr>
#> 1 trade_data    data/derived/distributed/trade_data.parquet
#> 2 market_data  data/derived/distributed/market_data.parquet
#> 3 calibration_data data/derived/distributed/calibration_data.parquet
#> 4 scenario_data data/derived/distributed/scenario_data.parquet

```

III-2.5 計算資源の統合

本章では、一台の Machine 内部の並列化から、複数の計算資源を前提とする分散処理へ考え方を拡張した。重要なのは、計算関数そのものよりも、Task を Job として管理し、Queue へ投入し、Scheduler が Worker へ配布し、結果を再び集約するという Control Plane である。

この構造があれば、Worker の実体は Local Process、別 Server、Container、Cloud Batch、HPC Cluster へ置き換えることができる。次章では、この計算基盤の上に Trade Data、Market

Data、Calibration、Scenario、Pricing、Risk Aggregation を統合し、エンタープライズ金融計算システムとして組み立てる。

対称性

第 III-3 章金融計算アーキテクチャ

第 III-1 章では Local Parallel Computing、第 III-2 章では `Job / Task Control` を含む Distributed Computing を扱った。本章では、それらを実際の金融計算システムの形へ統合する。

エンタープライズ金融計算では、Pricing Function だけがシステムの中ではない。Trade Data、Market Data、Calibration Data、Scenario、Calculation Configuration を分離して管理し、それらを Task へ変換し、並列・分散実行し、最終的に Risk Table へ集約する必要がある。

本章では PRDC を例に、**Data** → **Scenario** → **Task** → **Pricing** → **Risk** → **Output** という一連の Pipeline を R で構築する。

```
Trade Data
Market Data
Calibration Data
      |
      v
Scenario Grid
      |
      v
Job / Task
      |
      v
Pricing Function
      |
      v
Risk Table
      |
      v
Aggregation / Report
```

III-3.1 金融取引とマーケットデータ

```
tbl_prdc_trade <- tribble(
  ~trade_ID, ~notional, ~maturity_years,
  "PRDC-001", 300000000, 15
)
```

```
show_table_LFL(tbl_prdc_trade)
#> # A tibble: 1 x 3
#>   tra_ID      notional mat_years
#>   <chr>      <dbl>    <dbl>
#> 1 PRDC-001 300000000      15
```

```
tbl_prdc_market <- tribble(
  ~market_data_ID, ~fx_spot, ~domestic_rate, ~foreign_rate,
  "MKT-BASE", 150, 0.010, 0.030
)
```

```
tbl_prdc_calibration <- tribble(
  ~calibration_ID, ~fx_volatility, ~correlation,
  "CAL-BASE", 0.10, 0.20
)
```

```
tbl_prdc_calculation <- tribble(
  ~calculation_ID, ~num_paths, ~rand_seed,
  "CALC-BASE", 500, 123
)
```

```
show_table_LFL(tbl_prdc_market)
#> # A tibble: 1 x 4
#>   mar_ID  fx_spot dom_rate for_rate
#>   <chr>    <dbl>    <dbl>   <dbl>
#> 1 MKT-BASE 150      0.01    0.03
```

```
show_table_LFL(tbl_prdc_calibration)
#> # A tibble: 1 x 3
#>   cal_ID  fx_volat correlation
#>   <chr>    <dbl>    <dbl>
#> 1 CAL-BASE 0.1      0.2
```

```
show_table_LFL(tbl_prdc_calculation)
#> # A tibble: 1 x 3
#>   cal_ID  num_paths ran_seed
#>   <chr>    <int>   <int>
#> 1 CALC-BASE 500     123
```

III-3.2 プライシング・エンジン

本章では計算基盤に焦点を当てるため、Scenario Pricing Function は軽量な代理モデルとして表現する。第 II 部で構築した PRDC Pricing Function へ置き換えても、Task 生成と並列実行の構造は変わらない。

```
price_prdc_scenario_LFL <- function(
  trade,
  fx_spot,
  fx_volatility,
  domestic_rate,
  foreign_rate
) {
  horizon <- trade$maturity_years[[1]]

  fx_forward <- fx_spot *
    exp((domestic_rate - foreign_rate) * horizon)

  volatility_adjustment <- exp(
    0.5 * fx_volatility^2 * horizon
  )

  tibble(
    trade_ID = trade$trade_ID[[1]],
    fx_spot = fx_spot,
    fx_volatility = fx_volatility,
    domestic_rate = domestic_rate,
    foreign_rate = foreign_rate,
    prdc_price = trade$notional[[1]] *
      fx_forward / fx_spot *
      volatility_adjustment
  )
}
```

III-3.3 リスク・シミュレーション・シナリオ

```
base_fx_spot <- tbl_prdc_market$fx_spot[[1]]

tbl_prdc_scenario <- tidyr::crossing(
  fx_spot = base_fx_spot * c(0.90, 0.95, 1.00, 1.05, 1.10),
```

```

fx_volatility = c(0.075, 0.10, 0.125),
domestic_rate = c(0.005, 0.010, 0.015),
foreign_rate = c(0.025, 0.030, 0.035)
) |>
mutate(
  scenario_ID = row_number(),
  .before = 1
)

show_table_LFL(
  tbl_prdc_scenario,
  n = 10
)
#> # A tibble: 135 x 5
#>   sce_ID fx_spot fx_volat dom_rate for_rate
#>   <int>   <dbl>   <dbl>   <dbl>   <dbl>
#> 1     1     135    0.075    0.005    0.025
#> 2     2     135    0.075    0.005    0.03
#> 3     3     135    0.075    0.005    0.035
#> 4     4     135    0.075    0.005    0.04
#> 5     5     135    0.075    0.005    0.045
#> 6     6     135    0.075    0.01    0.035
#> 7     7     135    0.075    0.015    0.025
#> 8     8     135    0.075    0.015    0.035
#> 9     9     135    0.075    0.015    0.045
#> 10    10    135    0.075    0.015    0.055
#> # i 125 more rows

```

III-3. 計算エンジン

```

available_cores <- future::availableCores()
workers <- max(1L, min(2L, available_cores - 1L))

future::plan(
  future::multisession,
  workers = workers
)

```

```

#| warning: false
tbl_prdc_scenario_result <- tbl_prdc_scenario |>
mutate(
  result = furrr::future_pmap(

```

```

list(
  fx_spot,
  fx_volatility,
  domestic_rate,
  foreign_rate
),
\(fx_spot, fx_volatility, domestic_rate, foreign_rate) {
  price_prdc_scenario_LFL(
    trade = tbl_prdc_trade,
    fx_spot = fx_spot,
    fx_volatility = fx_volatility,
    domestic_rate = domestic_rate,
    foreign_rate = foreign_rate
  )
},
.options = furrr::furrr_options(seed = TRUE)
)
) |>
select(scenario_ID, result) |>
tidyr::unnest(result)

future::plan(future::sequential)

show_table_LFL(
  tbl_prdc_scenario = result,
  n = 10
)
#> # A tibble: 10 x 7
#>   scenario_ID fx_spot volatility dom_rate for_rate prd_npv
#>   <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
#> 1 1 PRDC-001 135 0.075 0.005 0.025 231822032.
#> 2 2 PRDC-001 135 0.075 0.005 0.03 215071380.
#> 3 3 PRDC-001 135 0.075 0.005 0.035 199531072.
#> 4 4 PRDC-001 135 0.075 0.01 0.025 249877294.
#> 5 5 PRDC-001 135 0.075 0.01 0.03 231822032.
#> 6 6 PRDC-001 135 0.075 0.01 0.035 215071380.
#> 7 7 PRDC-001 135 0.075 0.015 0.025 269338775.
#> 8 8 PRDC-001 135 0.075 0.015 0.03 249877294.
#> 9 9 PRDC-001 135 0.075 0.015 0.035 231822032.
#> 10 10 PRDC-001 135 0.1 0.005 0.025 239554866.
#> # i 125 more rows

```

III-3.5 リスクアグリゲーション・レポート

```
tbl_prdc_risk <- tbl_prdc_scenario_result |>
  arrange(
    fx_spot,
    fx_volatility,
    domestic_rate,
    foreign_rate
  ) |>
  mutate(
    npv_change = prdc_proxy_npv -
      median(prdc_proxy_npv)
  )

show_table_LFL(
  tbl_prdc_risk,
  n = 10
)

#> # A tibble: 135 x 8
#>   sce_ID tra_ID   fx_spot fx_volatility domestic_rate foreign_rate prd_npv npv_chang
#>   <int> <chr>     <dbl>     <dbl>     <dbl>     <dbl>     <dbl>     <dbl>
#> 1     1     1 PRDC-001     135     0.05     0.025 231822032. -7732833.
#> 2     2     2 PRDC-001     135     0.07     0.005 215071380. -24483485.
#> 3     3     3 PRDC-001     135     0.05     0.005 199531072. -40023793.
#> 4     4     4 PRDC-001     135     0.075    0.01     0.025 249877294. 10322429.
#> 5     5     5 PRDC-001     135     0.075    0.01     0.03 231822032. -7732833.
#> 6     6     6 PRDC-001     135     0.075    0.01     0.035 215071380. -24483485.
#> 7     7     7 PRDC-001     135     0.075    0.015    0.025 269338775. 29783910.
#> 8     8     8 PRDC-001     135     0.075    0.015    0.03 249877294. 10322429.
#> 9     9     9 PRDC-001     135     0.075    0.015    0.035 231822032. -7732833.
#> 10    10    10 PRDC-001     135     0.1     0.005    0.025 239554866.      0
#> # i 125 more rows
```

```
tbl_prdc_spot_risk <- tbl_prdc_risk |>
  group_by(fx_spot) |>
  summarise(
    mean_npv = mean(prdc_proxy_npv),
    .groups = "drop"
  )

show_table_LFL(tbl_prdc_spot_risk)

#> # A tibble: 5 x 2
#>   fx_spot   mea_npv
```



```
#>      <dbl>      <dbl>
#> 1    135    241320900.
#> 2   142.5  241320900.
#> 3    150    241320900.
#> 4   157.5  241320900.
#> 5    165    241320900.
```

```
prdc_risk_output_path <- write_derived_parquet_file_LFL(
  tbl_prdc_risk,
  file.path(
    "risk_system",
    "prdc_risk_table.parquet"
  )
)
```

```
tbl_prdc_risk_output <- tibble(
  file_path = file.path(
    "data",
    "derived",
    "risk_system",
    "prdc_risk_table.parquet"
  )
)
```

```
show_table_LFL(tbl_prdc_risk_output)
#> # A tibble: 1 x 1
#>   file_path
#>   <fct>
#> 1 data/derived/risk_system/prdc_risk_table.parquet
```

III-3.6 エンタープライズ金融計算

ここまでの並列・分散計算は、Pricing を多数回実行することでリスクを求める設計である。計算資源を増やせば処理時間を短縮できるが、Risk Factor が増えるほど Bump & Revalue の Pricing 回数が増えるという構造そのものは残る。

Hardware Acceleration

|

v

More Workers / More Cores

|

v

Faster Revaluation

|
v

But still many Revaluations

ここまでで、第 III 部前半のテーマである「計算を分割し、より多くの計算資源で処理する」構造が完成した。しかし、計算資源を増やしても、Sensitivity のために Pricing そのものを繰り返す構造は残る。

次章では発想を変え、Pricing Program そのものを微分可能な Computational Graph として構築する Automatic Differentiation へ進む。ここが、**Hardware / Infrastructure Acceleration** から **Algorithmic Acceleration** への転換点であ

第 III-4 章自動微分計算

第 III-1 章から第 III-3 章では、金融計算を Task へ分割し、Parallel / Distributed Computing によって大量の Pricing を処理する仕組みを構築した。これは計算量を増やすことで処理時間を短縮する方法である。

しかし Greeks や Curve Risk を有限差分で求める場合、Risk Factor ごとに価格を再計算する必要がある。Worker や GPU を増やせば一回ごとの再評価は速くなるが、Bump & Revalue が必要とする Pricing 回数そのものは減らない。

そこで本章から、計算資源の拡張ではなく、計算方法そのものを変える方向へ進む。

Automatic Differentiation (自動微分) に関する問題として、価格計算そのものを微分可能な計算グラフとして扱う。

本章では、金融リスク感応度を偏微分として整理しつつ、有限差分、連鎖律、Computational Graph、Forward Mode、Reverse Mode を順に確認する。最後に R の torch を用いて、自動微分の最小例を実装する。

III-4.1 金融計算と微分

金融商品の価格を、リスク変数に依存する関数として

$$V = V(x_1, x_2, \dots, x_n)$$

と表す。

ここで、 x_i は株価、為替レート、金利、Volatility などのリスクファクターである。

各リスクファクターに対する一次感応度は、

$$\frac{\partial V}{\partial x_i}$$

で表される。

したがって、すべての一次感応度をまとめると、

$$\nabla V = \left(\frac{\partial V}{\partial x_1}, \frac{\partial V}{\partial x_2}, \dots, \frac{\partial V}{\partial x_n} \right)$$

となる。

Option の Delta や Vega、金利商品の DV01、為替商品の FX Delta は、対象となるリスクファクターは異なるものの、数学的にはいずれも価格関数の偏微分として統一的に捉えることができる。

二次感応度も同様である。

例えば Option の Gamma は、

$$\Gamma = \frac{\partial^2 V}{\partial x^2}$$

であり、異なる二つのリスクファクター x_1, x_2 による Cross Gamma は、

$$\frac{\partial^2 V}{\partial x_1 \partial x_2}$$

として表される。

このような金融リスク計測の多岐にわたる価格関数の一次または二次の微分を求める問題として整理できる。

III-4.2 有限差分

最も直接的な感応度計算の一つが有限差分である。

リスクファクター x を微小量 h だけ変化させる Forward Difference では、

$$\frac{\partial V}{\partial x} \approx \frac{V(x+h) - V(x)}{h}$$

と近似する。

Central Difference では、

$$\frac{\partial V}{\partial x} \approx \frac{V(x+h) - V(x-h)}{2h}$$

となる。

金融実務では、市場データを一定量 Bump し、商品を再評価することから、この方法は Bump & Revalue と呼ばれる。

実装が比較的容易であり、既存の Pricing Engine をそのまま利用できるという利点がある。

一方で、 n 個のリスクファクターについて Central Difference を計算する場合、単純には各リスクファクターについて Up と Down の再評価が必要になる。

$$\text{Number of Revaluations} = 2n$$

Monte Carlo による価格評価自体が重い場合、この再評価は大きな負荷となる。

また、有限差分には Bump 幅 h の選択という問題もある。

h が大きすぎれば近似誤差が大きくなり、小さすぎれば浮動小数点演算による丸め誤差の影響を受けやすくなる。

したがって、

$$\text{Finite Difference} = \text{Approximation} + \text{Repeated Pricing}$$

という性質を持つ。

III-4.3 自動微分の原理と実装

自動微分は有限差分による数値微分とは異なる。

また、数式全体を記号的に変形して導関数を求める Symbolic Differentiation と異なる。

例えば、

$$y = \exp(x^2)$$

を考える。

記号微分では、

$$\frac{dy}{dx} = 2x \exp(x^2)$$

という新しい数式を導出する。

一方、自動微分では、元の計算を

$$x \rightarrow x^2 \rightarrow \exp(x^2) \rightarrow y$$

という基本演算の列として捉える。

各演算の局所的な微分を既知として、連鎖律によって全体の微分を構成する。

したがって、自動微分の中心にあるのは、

$$\text{Elementary Operations} + \text{Chain Rule}$$

である。

自動微分を理解するうえで最も重要な数式が連鎖律である。

例えば、

$$y = g(x), \quad z = f(y), \quad V = h(z)$$

とする

x に対する V の感応度は、

$$\frac{\partial V}{\partial x} = \frac{\partial V}{\partial z} \frac{\partial z}{\partial y} \frac{\partial y}{\partial x}$$

となる。

金融商品の価格計算も、多数の小さな計算を順番に実行することで構成される。

例えば金利商品では、

$$\text{Market Quote} \rightarrow \text{Yield Curve} \rightarrow \text{Discount Factor} \rightarrow \text{Cash Flow} \rightarrow PV$$

という計算を考えることができる。

市場クォート q から金利 r が構築され、その金利から割引係数 D が計算され、最終的に価格 V が得られるなら、

$$q \rightarrow r \rightarrow D \rightarrow V$$

である。

したがって、

$$\frac{\partial V}{\partial q} = \frac{\partial V}{\partial D} \frac{\partial D}{\partial r} \frac{\partial r}{\partial q}$$

となる。

この連鎖律をプログラムの計算過程全体に適用することが、Automatic Differentiation の基本的な考え方である。

プログラムによる計算を、変数間の計算の係として表したものを Computational Graph と呼ぶ。

簡単な例として、

$$a = x^2, \quad b = 3a, \quad V = b + 1$$

を考える。

計算グラフは、

```
x
|
v
x^2
|
v
a
|
v
* 3
|
v
b
```

```

|
v
+ 1
|
v
V

```

と表すことができる。

x から V へ向かう方向が通常の価格計算である。

一方、最終結果 V から x へ向かって局所微分を掛け合わせれば、

$$\frac{\partial V}{\partial x}$$

を求めることができる。

この「計算を行った経路を逆向きにたどる」という考え方が、Reverse Mode の直感的な理解につながる。

Forward Mode では、入力側から出力側へ計算を進めると同時に微分値を伝播させる。

例えば、

$$x \rightarrow a = f(x) \rightarrow b = g(a) \rightarrow V = h(b)$$

という計算では、

$$\frac{\partial a}{\partial x}, \quad \frac{\partial b}{\partial x}, \quad \frac{\partial V}{\partial x}$$

を順方向に計算する。

概念的には、

$$(x, \dot{x}) \rightarrow (a, \dot{a}) \rightarrow (b, \dot{b}) \rightarrow (V, \dot{V})$$

と、値と微分値を同時に運ぶ。

Forward Mode は、入力変数が少なく出力変数が多い問題に適している。

Reverse Mode では、まず通常どおり Forward 計算を行い、最終出力を求める。

その後、出力側から入力側へ向かって微分値を逆伝播する。

金融商品の価格を、

$$V = V(x_1, x_2, \dots, x_n)$$

とする。

通常の Pricing では、

$$(x_1, x_2, \dots, x_n) \rightarrow V$$

と計算する。

Reverse Mode では、得られた V から逆方向に、

$$V \rightarrow \left(\frac{\partial V}{\partial x_1}, \frac{\partial V}{\partial x_2}, \dots, \frac{\partial V}{\partial x_n} \right)$$

と感応度を伝播する。

金融リスク計測では、価格やポートフォリオ価値に対して、多数の市場リスクファクターの感応度を求めることが必要である。

したがって

$$\boxed{\text{Many Inputs} \rightarrow \text{Few Outputs}}$$

という構造を持つ金融計算では、Reverse Mode が特に重要となる。

金融分野では、Reverse Mode Automatic Differentiation を Adjoint Algorithmic Differentiation (AAD) という名称で扱うことが多い。

従来の Bump & Revalue では、

$$x_1 \rightarrow V_1, \quad x_2 \rightarrow V_2, \quad \dots, \quad x_n \rightarrow V_n$$

のように、多数の再評価を行う。

これに対して AAD では、Pricing 計算の過程を利用して、最終価格から各入力への感応度を逆向きに伝播させる。

概念的には、

```

Market Risk Factors
x1  x2  x3  ...  xn
\   |   |       /
 \   |   |       /
      |
      v
    Price V
      |
      | backward
      v
dV/dx1 dV/dx2 ... dV/dxn

```

となる。

重要なのは、AAD がより高速な「P」を計算するということではなく、リーフ計算のアルゴリズムそのものを変更するという点である。

III-4.4 深層学習ライブラリにおける市場リスク感応度

ここから、R の torch を使って実際に自動微分を確認する。

次の関数を考える。

$$y = x^2 + 3x + 1$$

解析的には、

$$\frac{dy}{dx} = 2x + 3$$

である。

$x = 2$ なら、

$$\frac{dy}{dx} = 2 \times 2 + 3 = 7$$

となる。

torch では、微分対象の Tensor に `requires_grad = TRUE` を指定する。

```
x <- torch_tensor(  
  2,  
  requires_grad = TRUE  
)  
  
y <- x^2 + 3 * x + 1  
  
y  
#> torch_tensor  
#> 11  
#> [ CPUFloatType{1} ][ grad_fn = <AddBackward1>
```

`backward()` を実行すると、計算グラフを逆向きにたどって勾配が計算される。

```
y$backward()  
  
x$grad  
#> torch_tensor  
#> 7  
#> [ CPUFloatType{1} ]
```

理論値は、

```
analytic_grad <- 2 * x + 3  
  
analytic_grad  
#> [1] 7
```

である。

torch による自動微分と解析解が一致することを確認できる。

ここで重要なのは、torch が

$$y = x^2 + 3x + 1$$

という数式を記号微分したのではないことである。

実際に実行された Tensor 演算から Computational Graph を構築し、そのグラフに対して連鎖律を逆方向に適用している。

次に、

$$V(x, y) = x^2 + xy + y^2$$

を考える。

偏微分は、

$$\frac{\partial V}{\partial x} = 2x + y$$

$$\frac{\partial V}{\partial y} = x + 2y$$

である。

$x = 2$, $y = 3$ なら、

$$\frac{\partial V}{\partial x} = 7, \quad \frac{\partial V}{\partial y} = 8$$

となる。

```
x <- torch_tensor(
  2,
  requires_grad = TRUE
)

y <- torch_tensor(
  3,
  requires_grad = TRUE
)

value <- x^2 + x * y + y^2

value$backward()

tbl_gradient_result <- tribble(
  ~variable, ~autograd,
  "x", x$grad$item(),
  "y", y$grad$item()
)
```

```
tbl_gradient_result
#> # A tibble: 2 x 2
#>   variable autograd
#>   <chr>      <dbl>
#> 1 x          7
#> 2 y          8
```

この例では、一つの Scalar 出力 value から、二つの入力 x と y に対する Gradient を一度の `backward()` で取得している。

これは金融商品の価格

$$V = V(S, r, \sigma, \dots)$$

から、

$$\Delta, \dots, \alpha, \dots$$

を求める構造と同じである。

ここまでの内容を金融工学へ置き換えると、

```
Market Data
|
v
Model Parameters
|
v
Pricing Model
|
v
Price
|
| backward()
v
Risk Sensitivities
```

となる。

すなわち、

Pricing + Computational Graph + Reverse Mode = Risk Sensitivities

という構造である。

第 III 部では、この考え方を段階的に金融商品へ適用する。

まず Option について、

$$\Delta, \Gamma, Vega, Theta$$

を自動微分で求める。

次に金利商品について、

$$DV01$$

および Yield Curve 全体のリスク感応度へ拡張する。

最後に PRDC について、

$$FX + Y + \text{Interest Rates} + \text{Foreign Rates}$$

という複数のリスクファクターを含む商品、ポートフォリオ

本章では、自動微分の数学的基礎を整理しながら

有限差分は、

$$\text{Building} \rightarrow \text{Reverse} \rightarrow \text{Approximate Sensitivity}$$

という方法である。

これに対して自動微分

Pricing Operations $\rightarrow$ Computational Graph $\rightarrow$ Chain Rule $\rightarrow$ Exact Derivative of the Program

という考え方をする。

特に金融工学では、

$$(x_1, x_2, \dots, x_n) \rightarrow V$$

という多数入力・少数出力の問題が多いため、Reverse Mode Automatic Differentiation、すなわち AAD が重要となる。

R の `torch` では、この Reverse Mode を

```
value$backward()
```

によって実行し、

```
x$grad
```

によって入力変数に対する Gradient を取得できる。

次章では European Option を題材として、GaussQuant による計算値と Monte Carlo + torch を比較し、

Δ , Γ , $Vega$, The

を実際に計算する。

第 III-5 章自動微分応用：オプション

前章では、Bump & Revalue から Automatic Differentiation へとアプローチを切り替え、偏微分、連鎖律、Computational Graph、Forward Mode、Reverse Mode を整理した。本章からは、その仕組みを実際の金融商品へ適用する。

本章では、解析的な Black-Scholes 式を `torch` で書き直さず、既存の金融工学ライブラリである GaussQuant を基準値として利用し、同じ European Option を Monte Carlo Simulation と R の `torch` によって評価する。

比較する量は、

$$NPV = \frac{C}{1+r} + \frac{P}{1+r} - \frac{K}{1+r}$$

である。

本章の中心は、

$$NPV = \text{GaussQuant} + \text{Monte Carlo} + \text{torch} + \text{Automatic Differentiation}$$

という比較である。

GaussQuant は QuantLib を利用した既存の Pricing Engine として使い、`torch` では Monte Carlo Pricing そのものを Differentiable Computational Graph として構築する。

III-5.1 オプション評価とリスク感応度

同じ European Call Option を GaussQuant と Monte Carlo で評価する。

```
valuation_date <- "2026-01-02"
maturity_date <- "2027-01-02"

spot_value <- 100
```



```
strike_value <- 100
risk_free_rate_value <- 0.03
dividend_yield_value <- 0
volatility_value <- 0.20

maturity_value <- as.numeric(
  as.Date(maturity_date) -
  as.Date(valuation_date)
) / 365

n_paths <- 200000L
mc_seed <- 1234L

tbl_option_input <- tribble(
  ~parameter, ~value,
  "spot", spot_value,
  "strike", strike_value,
  "risk_free_rate", risk_free_rate_value,
  "dividend_yield", dividend_yield_value,
  "volatility", volatility_value,
  "maturity", maturity_value,
  "n_paths", n_paths
)

tbl_option_input
#> # A tibble: 7 x 2
#>   parameter      value
#>   <chr>         <dbl>
#> 1 spot           100
#> 2 strike          100
#> 3 risk_free_rate  0.03
#> 4 dividend        0
#> 5 volatility      0.2
#> 6 maturity        1
#> 7 n_paths        20000
```

ここでは比較を明確にするため、フラットな金利、配当利回り、Volatility を用いる。

III-5.2 ブラック・ショールズ式による解析解

GaussQuant では、European Option を QuantLib オブジェクトとして構築できる。

```
option_gql <- GaussQuant::make_european_option_GQL(
  spot = spot_value,
  strike = strike_value,
  maturity_date = maturity_date,
  option_type = "call",
  valuation_date = valuation_date,
  risk_free_rate = risk_free_rate_value,
  dividend_yield = dividend_yield_value,
  volatility = volatility_value
)
```

価格と Greeks をまとめて取得する。

```
tbl_gaussquant_greeks <- GaussQuant::option_summary_GQL(
  option_gql
) |>
  filter(
    metric %in% c(
      "npv",
      "delta",
      "gamma",
      "vega",
      "theta"
    )
  )

tbl_gaussquant_greeks
#> # A tibble: 5 x 2
#>   metric      value
#>   <chr>      <dbl>
#> 1 npv         9.41
#> 2 delta        0.0193
#> 3 gamma        38.667
#> 4 vega        -5.3804
#> 5 theta
```

ここで得られる値を、本章における Benchmark とする。

III-5.3 モンテカルロ法における自動微分のリスク感応度

Risk-Neutral Measure の下で、配当利回り q を持つ株価過程を、

$$\frac{dS_t}{S_t} = (r - q)dt + \sigma dW_t$$

とする。

満期 T の株価は、

$$S_T = S_0 \exp \left[\left(r - q - \frac{1}{2} \sigma^2 \right) T + \sigma \sqrt{T} Z \right]$$

である。

ここで、

$$Z \sim N(0, 1)$$

である。

European Call の Monte Carlo 価格は、

$$\hat{V} = \frac{1}{m} \sum_{i=1}^m \max(S_T^{(i)} - K, 0)$$

となる。

GaussQ 関数の比較や感応度計算では、Monte Carlo Noise をできるだけ抑える必要がある。

本章では乱数を生成し、価格計算と感応度計算で同じ乱数を使用する。

```
torch_manual_seed(
    mc_seed
)

z_t <- torch_randn(
    n_paths
)
```

乱数 Z は市場リスクファクターではないため、`requires_grad = TRUE` は指定しない。

Monte Carlo Pricing を Tensor 演算だけで構成する。

```

european_call_mc_torch <- function(
  spot,
  strike,
  rate,
  dividend_yield,
  volatility,
  maturity,
  z
) {
  terminal_spot <- spot *
    torch_exp(
      (
        rate -
          dividend_yield -
            0.5 * volatility^2
      ) * maturity +
      volatility *
        torch_sqrt(maturity) *
          z
    )

  payoff <- torch_relu(
    terminal_spot -
      strike
  )

  torch_exp(
    -rate * maturity
  ) *
    torch_mean(
      payoff
    )
}

```

この関数では、

$$S_0, \quad r, \quad q, \quad \sigma, \quad T$$

から Terminal Spot、Payoff、Discounting、Monte Carlo Average までが一つの Computational Graph として接続される。

微分対象となる市場変数を Tensor として定義する。

```

spot_t <- torch_tensor(
  spot_value,
  requires_grad = TRUE
)

strike_t <- torch_tensor(
  strike_value
)

rate_t <- torch_tensor(
  risk_free_rate_value,
  requires_grad = TRUE
)

dividend_yield_t <- torch_tensor(
  dividend_yield_value
)

volatility_t <- torch_tensor(
  volatility_value,
  requires_grad = TRUE
)

maturity_t <- torch_tensor(
  maturity_value,
  requires_grad = TRUE
)

```

同じ乱数を用いて Monte Carlo 価値を計算する。

```

option_npv_mc_t <- european_call_option_torch(
  spot = spot_t,
  strike = strike_t,
  rate = rate_t,
  dividend_yield = dividend_yield_t,
  volatility = volatility_t,
  maturity = maturity_t,
  z = z_t
)

option_npv_mc_t
#> torch_tensor
#> 9.4665
#> [ CPUFloatType{1} ][ grad_fn = <MulBackward0> ]

```

Monte Carlo 価格を最終出力として Reverse Mode Automatic Differentiation を行う。

```
first_grad <- autograd_grad(
  outputs = option_npv_mc_t,
  inputs = list(
    spot_t,
    volatility_t,
    maturity_t
  )
)

delta_mc <- first_grad[[1]]$item()
vega_mc <- first_grad[[2]]$item()

# maturity は残存期間 T なので、
# Calendar-Time Theta は -dV/dT とする。
theta_mc <- -first_grad[[3]]$item()

tbl_mc_first_order_greeks <- tribble(
  ~metric, ~value,
  "delta", delta_mc,
  "vega", vega_mc,
  "theta", theta_mc
)

tbl_mc_first_order_greeks
#> # A tibble: 3 x 2
#>   metric      value
#>   <fct>    <dbl>
#> 1 delta      0.0086
#> 2 vega     38.1
#> 3 theta    -5.40
```

ここで、

$$\Delta = \frac{\partial V}{\partial S_0}, \quad Vega = \frac{\partial V}{\partial \sigma}, \quad Theta = -\frac{\partial V}{\partial T}$$

を Monte Carlo Pricing Graph から直接取得している。

Gamma には注意が必要である。

European Call の Payoff、

$$\max(S_T - K, 0)$$

は Strike で Kink を持つ。

Pathwise Automatic Differentiation でこの Payoff をそのまま二階微分すると、各 Path ではほとんど至るところで二階微分がゼロとなるため、Monte Carlo Gamma を単純な二階 AD だけで安定して推定することはできない。

これは自動微分の失敗ではなく、Payoff の非滑らかさに由来する。

そこで本章では Gamma について、Payoff を滑らかに近似した Smooth Payoff を用いて二階自動微分を確認する。

III-5.4 微分不可なオプションペイオフのケーススタディ

正の部分、

$$x^+ = \max(x, 0)$$

を、

$$\approx \frac{1}{2} \left(x + \sqrt{x^2 + \epsilon} \right)$$

で滑らかに近似する。

ϵ を小さくすれば通が Call Payoff へ近づく。

```
smooth_payoff_art_1 <- function(
  x,
  epsilon
) {
  0.5 * (
    x +
    torch_sqrt(
      x^2 +
      epsilon^2
    )
  )
}
```

Smooth Payoff を使った Monte Carlo Pricing Function を定義する。

```

european_call_mc_smooth_torch <- function(
  spot,
  strike,
  rate,
  dividend_yield,
  volatility,
  maturity,
  z,
  epsilon = 0.25
) {
  terminal_spot <- spot *
    torch_exp(
      (
        rate -
          dividend_yield -
            0.5 * volatility^2
      ) * maturity +
        volatility *
          torch_sqrt(maturity) *
            z
    )

  payoff <- smooth_positive_1d(torch::abs(
    terminal_spot -
      strike,
    epsilon = epsilon
  ))

  torch_
    -rate * maturity
  ) *
    torch_
      payoff
  )
}

```

Gamma 用に新しい計算グラフを構築する。

```

spot_gamma_t <- torch_tensor(
  spot_value,
  requires_grad = TRUE
)

volatility_gamma_t <- torch_tensor(
  volatility_value

```



```

)

maturity_gamma_t <- torch_tensor(
  maturity_value
)

option_npv_gamma_t <- european_call_mc_smooth_torch(
  spot = spot_gamma_t,
  strike = strike_t,
  rate = torch_tensor(
    risk_free_rate_value
  ),
  dividend_yield = dividend_yield_t,
  volatility = volatility_gamma_t,
  maturity = maturity_gamma_t,
  z = z_t,
  epsilon = 0.25
)

```

まず Delta を計算し、Gradient Check を行う。

```

delta_gamma_t <- autograd_grad(
  outputs = option_npv_gamma_t,
  inputs = list(
    spot_gamma_t
  ),
  create_graph = TRUE
)[[1]]

delta_gamma_t
#> torch_tensor
#> 0.6008
#> [ CPUFloatType[1] [ grad_fn = <SumBackward1> ]

```

さらに Delta を Spot に対して微分する。

```

gamma_mc <- autograd_grad(
  outputs = delta_gamma_t,
  inputs = list(
    spot_gamma_t
  )
)[[1]]$item()

gamma_mc
#> [1] 0.01912059

```

これは、

$$V \xrightarrow{\text{AD}} \Delta \xrightarrow{\text{AD}} \Gamma$$

という二階 Automatic Differentiation である。

ただし、この Gamma は Smooth Payoff による近似値である点に注意する。

Monte Carlo の結果をまとめる。

```
tbl_mc_greeks <- tribble(
  ~metric, ~value,
  "npv", option_npv_mc_t$item(),
  "delta", delta_mc,
  "gamma", gamma_mc,
  "vega", vega_mc,
  "theta", theta_mc
)

tbl_mc_greeks
#> # A tibble: 5 x 2
#>   metric      value
#>   <chr>      <dbl>
#> 1 npv        9.4665
#> 2 delta      0.60086
#> 3 gamma      0.41111
#> 4 vega        0.000
#> 5 theta     -5.1106
```

GaussQuant と比較

```
tbl_option_greeks_comparison <- tbl_gaussquant_greeks |>
  rename(
    gaussquant = value
  ) |>
  left_join(
    tbl_mc_greeks |>
      rename(
        monte_carlo_torch = value
      ),
    by = "metric"
  ) |>
  mutate(
```

```

difference = monte_carlo_torch -
  gaussquant,
relative_difference = if_else(
  gaussquant == 0,
  NA_real_,
  difference /
    abs(
      gaussquant
    )
)
)

tbl_option_greeks_comparison
#> # A tibble: 5 x 5
#>   metric gaussquant monte_carlo_torch difference relative_difference
#>   <chr>      <dbl>          <dbl>      <dbl>          <dbl>
#> 1 npv      9.4134          9.4665    0.053099      0.0056408
#> 2 delta    0.59871          0.60086   0.0021528     0.0035957
#> 3 gamma    0.019333          0.019121  -0.00021282   0.011008
#> 4 vega     38.667            38.667    0.21122       0.0060238
#> 5 theta    -5.3804            -5.401    -0.021815     0.0052333

```

比較の構造は、

$$\text{Gaussquant} / \text{QuantJ} \rightarrow \text{Monte Carlo} + \text{torch AD}$$

となる。

NPV、Delta、Vega、Theta は、Monte Carlo Path 上の微分をそのまま利用する Pathwise Derivative である。

Gamma については Call Payoff の Kink があるため、Smooth Payoff による二階 AD を使用している。

Monte Carlo 価格には統計誤差が存在する。

価格の差を解釈するため、標準誤差も確認する。

```

with_no_grad({
  terminal_spot_se_t <- torch_tensor(
    spot_value
  ) *
  torch_exp(
    (

```

```

      risk_free_rate_value -
      dividend_yield_value -
      0.5 * volatility_value^2
    ) * maturity_value +
    volatility_value *
      sqrt(
        maturity_value
      ) *
      z_t
  )

discounted_payoff_se_t <- exp(
  -risk_free_rate_value *
  maturity_value
) *
  torch_relu(
    terminal_spot_se_t -
    strike_value
  )

mc_standard_error <- torch::sqrt(
  discounted_payoff_se_t
)$item() /
  sqrt(
    n_paths
  )
})

tbl_mc_error <- tribble(
  ~metric ~value
  "Monte Carlo NPV", option_npv_mean$item(),
  "Monte Carlo standard error", mc_standard_error,
  "GaussQuant NPV",
  tbl_gaussquant_results |>
    filter(
      metric == "npv"
    ) |>
    pull(
      value
    )
)

tbl_mc_error
#> # A tibble: 3 x 2
#>   metric value

```

```
#>   <chr>                                <dbl>
#> 1 Monte Carlo NPV                      9.4665
#> 2 Monte Carlo standard error 0.031533
#> 3 GaussQuant NPV                      9.4134
```

GaussQuant と Monte Carlo 価格の差が Monte Carlo Standard Error と同程度であれば、差の多くは Simulation Noise として説明できる。

本章で重要なのは、Black-Scholes の解析式を自動微分したことではない。

Monte Carlo Pricing を、

Risk Factors → Simulation → Payoff → Discount → NPV

という Differentiable Computational Graph として構築していることである。

Forward 計算では、

$$S, \sigma, T \rightarrow V$$

を計算する。

Reverse 計算では、

$$V \rightarrow \Delta, \text{Vega}, \text{Theta}$$

へ Gradient を伝播する。

したがって、

Monte Carlo Pricing Engine + Reverse Mode AD = Differentiable Risk Engine

となる。

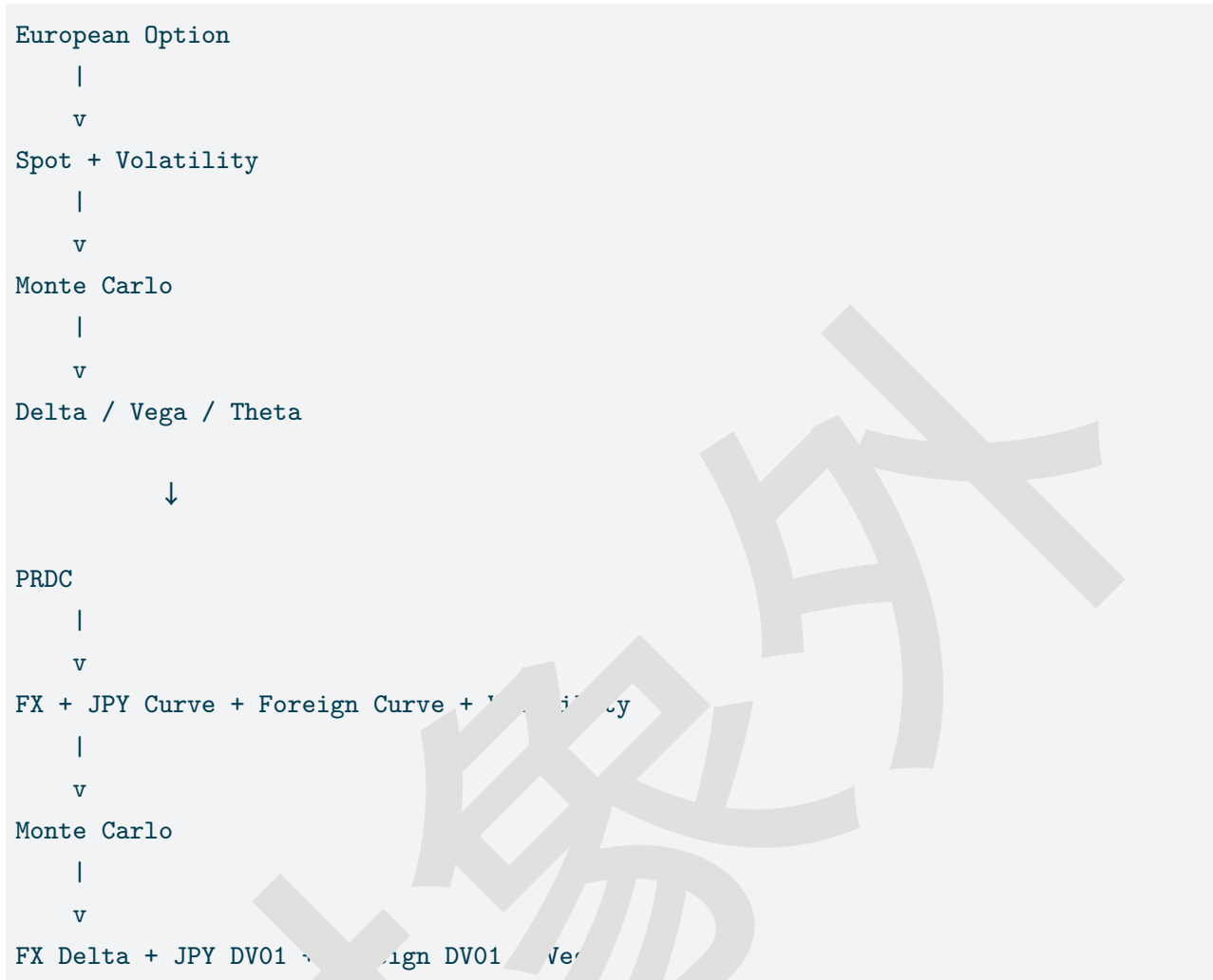
European Option では、Risk Factor は少数であった。

しかし同じ考え方は、

$$V = V(S_{FX}, \mathbf{r}_{JPY}, \mathbf{r}_{Foreign}, \sigma_{FX})$$

という PRDC にも適用できる。

違いは Computational Graph の規模だけである。



本章の Monte Carlo + torch 演算は、第 III-7 章で扱う PRDC Risk への直接の準備となる。

本章では European Option について

GaussQuant と Monte Carlo + torch Automatic Differentiation

を比較した。

GaussQuant では、QuantLib を利用して、

NPV , Δ , Γ , $Vega$, $Theta$

を取得した。

一方、torch では Monte Carlo Pricing を Tensor 演算として構築し、Reverse Mode Automatic

Differentiation によって、

$$\Delta, \text{ Vega}, \text{ Theta}$$

を Pricing Graph から直接取得した。

Gamma については Payoff の Kink による Pathwise 二階微分の問題を確認し、Smooth Payoff を用いた二階 Automatic Differentiation を適用した。

この比較によって、自動微分の目的は解析公式を置き換えることなく、

複雑な Pricing Algorithm そのものを微分する

ことにあると分かる。

次章では、少数の Risk Factor を持つ Option から、多数の Curve Node を持つ金利商品へ問題の次元を拡張し、

$$DVC \rightarrow DV01 \rightarrow DV02 \rightarrow GPS$$

を Yield Curve 全体の Gradient とし定義する。

第 III-6 章自動微分応用：金利リスク

前章では、European Option を GaussQuant と Monte Carlo torch で評価し、Delta、Gamma、Vega、Theta を Automatic Differentiation から取得した。Option では Risk Factor の数が比較的少なかったが、金利商品では Yield Curve 上の多数の Node が同時に価格へ影響する。

本章では、自動微分を金利商品へ拡張する。

Option では、

$$V = V(S, T, \dots)$$

という少数のリスクファクターに依存する。

一方、金利商品では、価格が一つの金利だけでなく、Yield Curve 上の多数の点に依存する。

$$V = V(r_1, r_2, \dots, r_n)$$

したがって、金利リスク計測では、

$$\frac{\partial V}{\partial r}$$

という一つの感応度から、

$$\nabla_{\mathbf{r}} V = \left(\frac{\partial V}{\partial r_1}, \frac{\partial V}{\partial r_2}, \dots, \frac{\partial V}{\partial r_n} \right)$$

という Curve 全体の Gradient へ進む必要がある。

本章では、まず単一のフラット金利を用いた Swap の DV01 を計算し、その後、複数ノードを持つ Yield Curve へ拡張する。

最終的に、

$$\text{DV01} \rightarrow \text{Bucketed DV01} \rightarrow \text{GPS} \rightarrow \text{Curve Gradient}$$

という流れを、自動微分によって統一的に捉える。

III-6.1 金利スワップ評価とリスク感応度

将来キャッシュフロー CF_i の現在価値は、

$$V = \sum_{i=1}^n CF_i D(0, t_i)$$

である。

単純化して、連続複利のフラット金利 r を用いると、

$$D(0, t_i) = e^{-rt_i}$$

なので、

$$V(r) = \sum_{i=1}^n CF_i(r) e^{-rt_i}$$

となる。

ここで重要な金利商品では割引係数だけでなく、キャッシュフロー自身も金利に依存する場合があることである。

したがって、

$$\frac{\partial V}{\partial r} = \sum_i \left[\frac{\partial CF_i}{\partial r} D_i + CF_i \frac{\partial D_i}{\partial r} \right]$$

となる。

これは、金利リスクが単なる Discounting Risk ではなく、Projected Cash Flow の変化も含むことを意味する。

元本 N 、固定金利 K 、Accrual Factor α_i とすると、固定レグのキャッシュフローは、

$$CF_i^{\text{fix}} = NK\alpha_i$$

である。

したがって、

$$PV_{\text{fix}} = \sum_i NK\alpha_i e^{-rt_i}$$

となる。

r で微分すると、

$$\frac{\partial PV_{\text{fix}}}{\partial r} = - \sum_i t_i NK\alpha_i e^{-rt_i}$$

である。

金利が上昇すると割引係数が低下するため、Fixed Leg の現在価値は低下する。

本章では、自動微分の仕組みを明らかにするために、Floating Leg を単純化して、

$$CF_i^{\text{float}} = N(e^{r\alpha_i} - 1)$$

とする。

このとき

$$PV_{\text{float}} = \sum_i N(e^{r\alpha_i} - 1) e^{-rt_i}$$

である。

微分すると、

$$\frac{\partial PV_{\text{float}}}{\partial r} = \sum_i N e^{-rt_i} [\alpha_i e^{r\alpha_i} - t_i (e^{r\alpha_i} - 1)]$$

となる。

Floating Leg では、

Projected Cash Flow Risk + Discounting Risk

の両方が金利感応度に含まれる。

Receive Float / Pay Fixed Swap を考える。

$$V_{\text{swap}} = PV_{\text{float}} - PV_{\text{fixed}}$$

したがって、

$$\frac{\partial V_{\text{swap}}}{\partial r} = \frac{\partial PV_{\text{float}}}{\partial r} - \frac{\partial PV_{\text{fixed}}}{\partial r}$$

となる。

III-6.2 金利リスクにおける自動微分

DV01 は、金利が 1bp 変化した場合の価格変化を表す。

1bp は、

$$1\text{bp} = 10^{-4}$$

なので、値による局所近似では、

$$DV01 \approx \frac{\partial V}{\partial r} \times 10^{-4}$$

となる。

符号については、業務システムやポジション表示の慣行により異なる場合があるため、本章では

$$\frac{\partial V}{\partial r} \times 10^{-4}$$

を signed DV01 として扱う。

まず計算条件を設定する。

```
tbl_swap_input <- tribble(
  ~parameter, ~value,
  "notional", 100000000,
  "fixed_rate", 0.0200,
  "rate", 0.0250,
  "years", 5
)

tbl_swap_input
#> # A tibble: 4 x 2
#>   parameter      value
#>   <chr>         <dbl>
#> 1 notional    100000000
#> 2 fixed_rate      0.02
#> 3 rate         0.025
#> 4 years         5
```

半年ごとの支払いを想定する。

```
notional <- 100000000
fixed_rate <- 0.0200
rate_value <- 0.0250

payment_time <- seq(
  0.5,
  5.0,
  by = 0.5
)

accrual <- rep(
  0.5,
  length(payment_time)
)
```

torch Tensor へ変換する。

```
rate <- torch_tensor(
  rate_value,
  requires_grad = TRUE
)

time_t <- torch_tensor(
  payment_time
)
```

```

accrual_t <- torch_tensor(
  accrual
)

notional_t <- torch_tensor(
  notional
)

fixed_rate_t <- torch_tensor(
  fixed_rate
)

```

割引係数を計算する。

```

df <- torch_exp(
  -rate * time_t
)

```

Fixed Leg を計算する。

```

fixed_cf <- notional_t *
  fixed_rate_t *
  accrual_t

pv_fixed <- torch_sum(
  fixed_cf * df
)

pv_fixed
#> torch.FloatTensor
#> 9341518
#> [ CPUFloatType ... d_fn = <SumBackward0> ]

```

Floating Leg を計算する。

```

float_cf <- notional_t *
  (
    torch_exp(
      rate * accrual_t
    ) -
    1
  )

pv_float <- torch_sum(

```

```

float_cf * df
)

pv_float
#> torch_tensor
#> 11750344
#> [ CPUFloatType{} ][ grad_fn = <SumBackward0> ]

```

Receive Float / Pay Fixed の Swap NPV は、

```

swap_npv <- pv_float - pv_fixed

swap_npv
#> torch_tensor
#> 2408726
#> [ CPUFloatType{} ][ grad_fn = <SubBackward0> ]

```

となる。

価格に対して backward() を実行する。

```

swap_npv$backward()

rate$grad
#> torch_tensor
#> 4.66456e+08
#> [ CPUFloatType[1] ]

```

rate\$grad は、

$$\frac{\partial V}{\partial r}$$

を表す。

1bp へ換算する。

```

tbl_swap_result <- tribble(
  ~metric, ~value,
  "npv",    swap_npv$item(),
  "dV_dr", rate$grad$item(),
  "dv01",  rate$grad$item() * 1e-4
)

```

```
tbl_swap_result
#> # A tibble: 3 x 2
#>   metric      value
#>   <chr>      <dbl>
#> 1 npv        2408726
#> 2 dV_dr    466456256
#> 3 dv01       46646.
```

ここで実行していることは、

```
rate
|
+-----> floating cash flow
|
+-----> discount factor
      |
      v
    cash-flow PV
      |
      v
    swap NPV
      |
      | backward()
      v
    dV/dr
```

という Computation Graph を逆向きにたどることである。

同じ Swap 関数を通常の R 関数として定義する。

```
swap_value_flat <- function(
  rate,
  notional,
  fixed_rate,
  payment_time,
  accrual
) {
  df <- exp(
    -rate * payment_time
  )

  fixed_cf <- notional *
    fixed_rate *
    accrual
```

```

float_cf <- notional *
  (
    exp(
      rate * accrual
    ) -
    1
  )

sum(
  float_cf * df
) -
sum(
  fixed_cf * df
)
}

```

1bp Bump による Forward Difference を計算する。

```

bump <- 1e-4

base_npv <- swap_value_flat(
  rate = rate_value,
  notional = notional,
  fixed_rate = fixed_rate,
  payment_time = payment_time,
  accrual = accrual
)

up_npv <- swap_value_flat(
  rate = rate_value + bump,
  notional = notional,
  fixed_rate = fixed_rate,
  payment_time = payment_time,
  accrual = accrual
)

fd_dv01 <- up_npv - base_npv

fd_dv01
#> [1] 46634.17

```

Central Difference による一次微分も確認する。


```

down_npv <- swap_value_flat(
  rate = rate_value - bump,
  notional = notional,
  fixed_rate = fixed_rate,
  payment_time = payment_time,
  accrual = accrual
)

fd_dv_dr <- (
  up_npv -
  down_npv
) / (
  2 * bump
)

fd_dv01_central <- fd_dv_dr *
  1e-4

tbl_dv01_comparison <- tribble(
  ~method, ~dv01,
  "Forward bump", fd_dv01,
  "Central difference", fd_dv01_central,
  "Automatic differentiation", auto$dv01
)

tbl_dv01_comparison
#> # A tibble: 3 x 2
#>   method dv01
#>   <chr>   <dbl>
#> 1 Forward bump 46.46
#> 2 Central difference 46.46
#> 3 Automatic differentiation 46.46

```

有限差分では価格を複数回評価する必要があるのに対し、自動微分では構築された計算グラフを利用して感応度を得る。

III-6.3 イールドカーブと自動微分

実際の金利市場では、単一の金利 r ではなく、満期ごとに異なる金利を持つ。

例えば、

$$\mathbf{r} = (r_1, r_2, \dots, r_n)$$

とする。

価格は、

$$V = V(r_1, r_2, \dots, r_n)$$

となる。

したがって必要なのは Scalar Derivative ではなく、

$$\nabla_{\mathbf{r}} V = \left(\frac{\partial V}{\partial r_1}, \frac{\partial V}{\partial r_2}, \dots, \frac{\partial V}{\partial r_n} \right)$$

という Gradient である。

各成分を 1bp へ換算すれば、

$$F_{\text{bucket}} = \frac{\partial V}{\partial r} \cdot 10^{-4}$$

となる。

これが Bucketed Delta や Point Sensitivity の基となる。

単純化した Zero Curve である。

```
tbl_curve <- tribble(
  ~tenor    rate,
  0.5, 0.0180,
  1.0, 0.019,
  2.0, 0.0210,
  3.0, 0.0225,
  5.0, 0.0250
)
```

```
tbl_curve
#> # A tibble: 5 x 2
#>   tenor    rate
#>   <dbl>  <dbl>
#> 1  0.5 0.018
#> 2  1  0.019
#> 3  2  0.021
#> 4  3  0.0225
```

```
#> 5    5    0.025
```

Curve Rate 全体を一つの Tensor として定義する。

```
curve_rate <- torch_tensor(
  tbl_curve$rate,
  requires_grad = TRUE
)

curve_time <- torch_tensor(
  tbl_curve$tenor
)
```

簡単な Zero Coupon Cash Flow を考える。

```
cash_flow <- torch_tensor(
  c(
    500000,
    500000,
    500000,
    500000,
    102500000
  )
)
```

各 Node Rate をその満期 Zero Rate として使用する。

```
curve_pv <- torch_exp(
  -curve_rate * curve_time
)

curve_pv <- torch_
  cash_flow * curve_
)

curve_pv
#> torch_tensor
#> 92388832
#> [ CPUFloatType{ } ][ grad_fn = <SumBackward0> ]
```

自動微分を実行する。

```
curve_pv$backward()
```

```

curve_rate$grad
#> torch_tensor
#> 1000000000 *
#> -0.0025
#> -0.0049
#> -0.0096
#> -0.0140
#> -4.5228
#> [ CPUFloatType{5} ]

```

これにより、

$$\left(\frac{\partial V}{\partial r_{0.5}}, \frac{\partial V}{\partial r_1}, \frac{\partial V}{\partial r_2}, \frac{\partial V}{\partial r_3}, \frac{\partial V}{\partial r_4} \right)$$

が一度に得られる。

1bp 換算する。

```

tbl_bucketed_dv01 <- tibble(
  tenor = tbl_curve$tenor,
  rate = tbl_curve$rate,
  sensitivity = as.numeric(
    curve_rate$grad
  ),
  dv01 = sensitivity * 0.0001
)

tbl_bucketed_dv01
#> # A tibble: 5 x 4
#>   tenor   rate sensitivity    dv01
#>   <dbl> <dbl>   <dbl>    <dbl>
#> 1  0.5 0.018 247760. -24.776
#> 2  1  0.019 490590. -49.059
#> 3  2  0.021 958870. -95.887
#> 4  3  0.0225 -1402092. -140.21
#> 5  5  0.025 -452279648 -45228.

```

III-6.4 バケット DV01 と GPS

本章では、Yield Curve 上の各 Grid Point に対する価格感応度を GPS (Grid Point Sensitivity) として扱う。

各 Grid Point r_i に対して、

$$GPS_i = \frac{\partial V}{\partial r_i}$$

である。

1bp 単位で表示する場合は、

$$GPS01_i = GPS_i \times 10^{-4}$$

となる。

したがって、

$$\mathbf{GPS} = \left(\frac{\partial V}{\partial r_1}, \dots, \frac{\partial V}{\partial r_n} \right)$$

は、数学的にはそのまま Curve としてある

```
tbl_gps <- tbl_bucketed_dv01
  transmute(
    tenor,
    grid_rate = rate,
    gps = sensitivity,
    gps01 = dv01
  )

tbl_gps
#> # A tibble: 5 x 4
#>   tenor grid_rate    gps    gps01
#>   <dbl>   <dbl>   <dbl>   <dbl>
#> 1  0.5    0.018 -247760. -24.776
#> 2  1      0.019 -490590. -49.059
#> 3  2      0.021 -958870. -95.887
#> 4  3      0.0225 -1402092. -140.21
#> 5  5      0.025 -452279648 -45228.
```

この考え方によって、

$$DV01 = \text{Scalar Risk}$$

から、

$$\text{GPS} = \text{Vector Risk}$$

へ拡張できる。

単純化された設定では、Bucketed DV01 を合計することで、Curve 全体を同時に 1bp 動かしたときの感応度に対応する量を得ることができる。

```
tbl_total_bucketed_dv01 <- tbl_bucketed_dv01 |>
  summarise(
    total_dv01 = sum(
      dv01
    )
  )

tbl_total_bucketed_dv01
#> # A tibble: 1 x 1
#>   total_dv01
#>   <dbl>
#> 1    -45538.
```

ただし、実務では Curve Constraints として 複数 Curve、Quote Mapping などが存在するため、

$$\text{Parallel DV01} = \sum_i \text{Bucketed DV01}_i$$

が常に成り立つとは限らない。

重要なのは、各リハッチャーに対する局所感応度を Gradient として保持することである。

実務の金利リスクでは 直接 Zero Rate を Bump するだけでは不十分である。

実際の計算は、

Market Quotes → Bootstrap → Yield Curve → Discount Factors → Cash Flows → V

という計算グラフを持つ。

Market Quote を、

$$\mathbf{q} = (q_1, q_2, \dots, q_m)$$

Yield Curve Node を、

$$\mathbf{r} = (r_1, r_2, \dots, r_n)$$

とすると、

$$V = V(\mathbf{r}(\mathbf{q}))$$

である。

連鎖律より、

$$\frac{\partial V}{\partial q_i} = \sum_j \frac{\partial V}{\partial r_j} \frac{\partial r_j}{\partial q_i}$$

となる。

行列表現では、

$$\nabla_{\mathbf{q}} V = J_{\mathbf{r}, \mathbf{q}} \nabla_{\mathbf{r}} V$$

である。

ここで、

$$J_{\mathbf{r}, \mathbf{q}}$$

は Market Quote から Curve Node への Jacobian である。

この式は、金利リスク計測と Automatic Differentiation をつなぐ重要な関係である。

実際の Swap では、Discount Curve と Forward Curve を分離する Multi-Curve Framework が一般的である。

この場合、価格は、

$$V = V(\mathbf{r}^{disc}, \mathbf{r}^{fwd})$$

と表される。

したがって、

$$\nabla V = (\nabla_{\mathbf{r}^{disc}} V, \nabla_{\mathbf{r}^{fwd}} V)$$

となる。

すなわち、

$$\boxed{\text{Discount Risk} + \text{Forward Risk}}$$

を別々の Gradient として取得できる。

これは次章で扱う PRDC に直接つながる。

PRDC では、

$$\text{JPY Curve} + \text{Borrowing Cost} + \text{FX}$$

という複数のリスクファクターを同時に扱う必要がある。

有限差分法では Bucketed Risk の場合 Curve Node の数だけ Bump & Revalue を繰り返す必要がある。

例えば 100 個の Node に対して Central Difference を使えば、

$$2 \times 100 = 200$$

回の再評価が必要になる。

一方、自動微分では、

$$V = V(r_1, r_2, \dots, r_{100})$$

という計算グラフを構築し、

$$V \rightarrow \nabla_r V$$

と逆伝播する。

したがって、金利リスク計測は、



として捉えることができる。

これは、

Scalar Pricing → Vector Risk

への変換である。

本章では、Optical Green の確認し、自動微分を 金利商品へ拡張した。

まず単一のフットノート金利について、

$$V = V(r)$$

から、

$$DV01 = \frac{\partial V}{\partial r} \times 10^{-4}$$

を求めた。

次に Yield Curve を、

$$\mathbf{r} = (r_1, \dots, r_n)$$

という Tensor として扱い、

$$\nabla_{\mathbf{r}} V = \left(\frac{\partial V}{\partial r_1}, \dots, \frac{\partial V}{\partial r_n} \right)$$

を一度の Reverse Mode Automatic Differentiation で取得した。

これにより、

$$\text{DV01} \rightarrow \text{Bucketed DV01} \rightarrow \text{GPS} \rightarrow \text{Credit} \rightarrow \text{Quoted}$$

という金利リスク計測を統一的に表現できる。

さらに、

$$\text{Market} \rightarrow \text{Curve} \rightarrow \text{PV}$$

という計算グラフに対して連鎖律を適用することで、Quote 単位の Risk へ拡張できる。

次章では、この考え方を PRDC へ適用する。

PRDC 価格を、

$$V = V(S_F, \mathbf{r}_{JPY}, \mathbf{r}_{Foreign}, \sigma_{FX}, \dots)$$

と表し、

$$\Delta_{FX}, \quad \text{DV01}_{JPY}, \quad \text{DV01}_{Foreign}, \quad \text{Vega}_{FX}$$

を同じ Computational Graph から計算する。

これにより、第 III 部の最終段階として、為替と複数金利カーブを持つ多因子商品の Differentiable Risk Engine を構築する。

第 III-7 章自動微分応用：多因子モデル

第 III 部では、二つの方向からクオンツ計算の高度化を進めてきた。前半では Parallel / Distributed Computing によって大量の Pricing を処理する計算基盤を構築し、後半では Pricing そのものを Differentiable Computational Graph として設計することで Automatic Differentiation から Risk Sensitivity を直接取得した。

最終章では、この二つの考え方が同時に必要となる代表的な PRDC を扱う。PRDC は FX、複数通貨の Yield Curve、Volatility など多数の Risk Factor を持つ。価格評価には Monte Carlo Simulation が必要となる。したがって、PRDC 計算をどう実行するかと、多数の感応度をどう効率的に生成するかという二つの課題の一つの商品に簡約される。

第 III-5 章では Option Greeks、第 III-6 章では DV01 から Currency Gradient へと、価格関数の入力次元を段階的に増やしてきた。本章では、PRDC の設計をまとめる。

$$= V(S_{FX}, r, Y, F_{Foreign}, \sigma_{FX})$$

という多変数 Monte Carlo Pricing を一つの計算 Graph として構成し、

$$[S_{FX}, DV01_{FX}, DV01_{PY}, DV01_{Foreign}, Vega_{FX}]$$

を一度の Reverse-mode Automatic Differentiation から取得する。

本章の目的は、単に PRDC の Greeks を計算することではない。Monte Carlo Pricing と Risk Calculation を別々の処理として繰り返すのではなく、Pricing そのものから Risk を生成するという Differentiable Risk Engine の設計を示すことである。

III-7.1 金利為替系デリバティブと多因子市場モデル

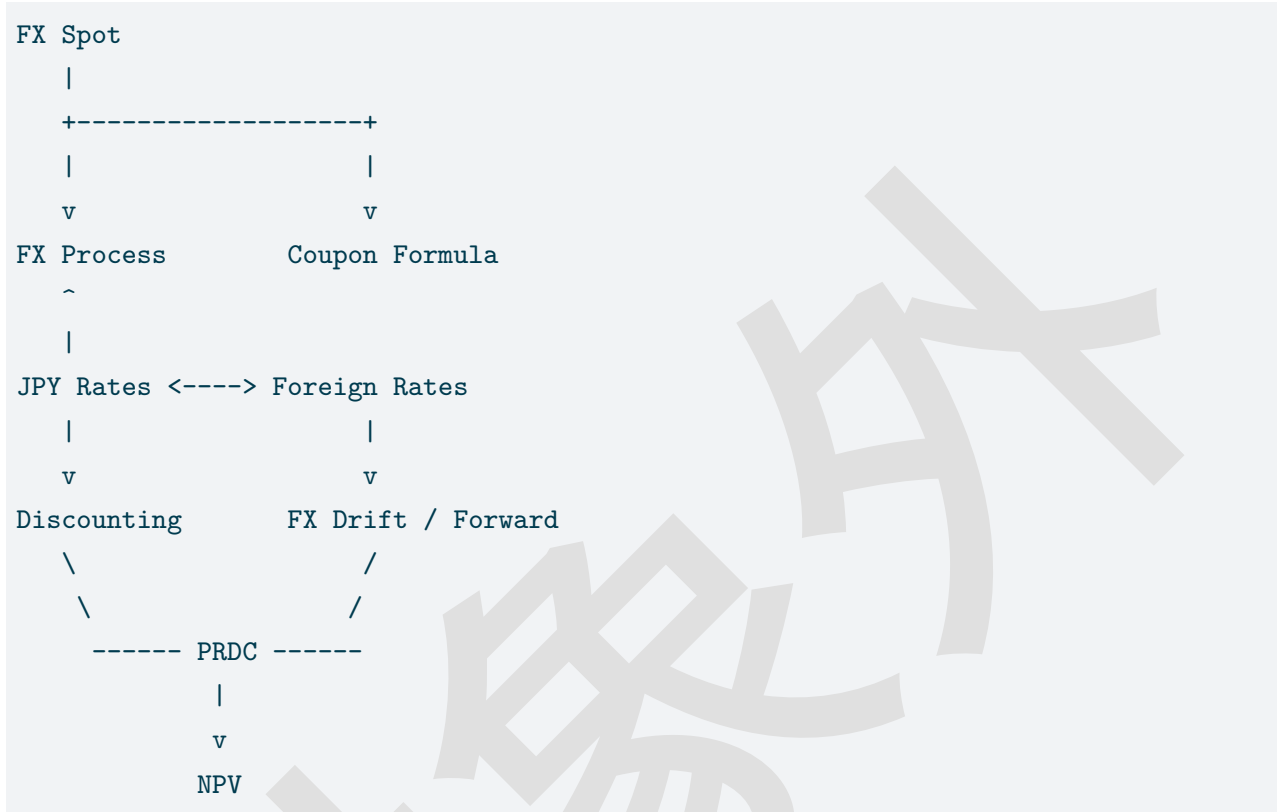
PRDC は、為替と複数通貨の金利を同時に扱うハイブリッド商品である。

単純な Option では、

$$V = V(S, \sigma, r, T)$$

と少数の入力変数を考えればよかった。

PRDC では、



という複数の経路から何れも構成される。

したがって、一つの市場変数を変動した場合でも、その影響は複数の計算経路を通して最終価格へ伝わる。

例えば JPY 金利は、

- JPY Discount factor
- FX の Risk-Neutral Drift
- 将来 Coupon の期待値

へ影響する。

Foreign Rate も、

- FX Forward
- FX の Risk-Neutral Drift
- 将来 Coupon の期待値

へ影響する。

このような商品では、連鎖律をプログラム全体へ適用する Automatic Differentiation の考え方が特に重要になる。

PRDC には多様な Coupon 構造、Callable 条件、Notional Exchange、Target Redemption、Barrier などが存在する。

本章では自動微分の構造を明確にするため、第 II 部・第 III 部で扱った PRDC の考え方を簡略化し、為替連動 Coupon を、

$$c_i = \min \left[c_{\max}, \max \left(0, c_0 + L \left(\frac{S_i}{S_{\text{ref}}} - 1 \right) \right) \right]$$

とする。

ここで、

- c_0 : Base Coupon
- L : FX Leverage
- S_i : Coupon Date の FX Spot
- S_{ref} : Reference FX
- $c_{\max}$: Coupon Cap

である。

Coupon Cash Flow

$$CF_i = N\alpha_i c_i$$

とする。

この Payoff は Automatic Differentiation の構造を明確にするための教育用モデルである。実際の PRDC では Coupon Formula、Callable 条件、Notional Exchangeなどを追加する必要があるが、重要なのは、それらを同じ Differentiable Computational Graph の中へ組み込むことである。

重要なのは Payoff の具体的な形ではなく、

FX + JPY Curve + Foreign Curve + Volatility → PRDC NPV
--

という計算グラフを微分可能な演算として構成することである。

JPY を Domestic Currency、Foreign Currency を外貨とする。

簡略化した Lognormal FX Process として、

$$\frac{dS_t}{S_t} = (r_{JPY}(t) - r_{Foreign}(t)) dt + \sigma_{FX} dW_t$$

を用いる。

離散化すると、

$$S_{t+\Delta t} = S_t \exp \left[\left(r_{JPY} - r_{Foreign} - \frac{1}{2} \sigma_{FX}^2 \right) \Delta t + \sigma_{FX} \sqrt{\Delta t} Z \right]$$

となる。

ここで、

$$Z \sim N(0, 1)$$

である。

この式から、FX Path 自身が、

$$S_{t_i} = S_{t_i}(S_0, r_{JPY}, r_{Foreign}, \sigma_{FX}, Z)$$

という関数になっていることが分かる。

したがって PRDC 価格を微分すると Coupon だけではなく FX Path 生成まで連鎖律が伝播する。

III-7.2 多因子モンテカルロ法

PRDC の Monte Carlo 価格を、

$$\hat{V} = \frac{1}{M} \sum_{m=1}^M \left[\sum_{i=1}^n D_{JPY}(0, t_i) CF_i^{(m)} + D_{JPY}(0, T) N \right]$$

とする。

ここで、

M

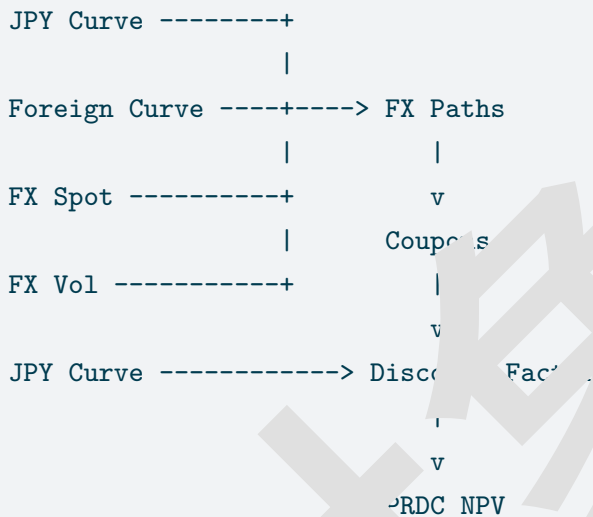
は Path 数である。

JPY Discount Factor は、期間ごとの短期金利を用いて、

$$D_{JPY}(0, t_i) = \exp \left[- \sum_{j=1}^i r_{JPY,j} \Delta t_j \right]$$

とする。

したがって計算全体は、



となる。

本章では、年間、1回 Coupon を支払う簡略 PRDC を考える。

```
tbl_prdc_input <-
  ~parameter, ~value
  "notional",    1000000,
  "fx_spot",     150.00,
  "reference_fx", 150.00,
  "fixed_coupon", 0.0200,
  "leverage",    0.80,
  "coupon_cap",  0.0800,
  "fx_vol",      0.1200,
  "n_paths",     20000
)
```

```
tbl_prdc_input
#> # A tibble: 8 x 2
```

```
#>   parameter      value
#>   <chr>         <dbl>
#> 1 notional      100000000
#> 2 fx_spot        150
#> 3 reference_fx    150
#> 4 fixed_coupon    0.02
#> 5 leverage        0.8
#> 6 coupon_cap      0.08
#> 7 fx_vol          0.12
#> 8 n_paths         20000
```

JPY Curve と Foreign Curve を設定する。

```
tbl_prdc_curve <- tribble(
  ~year, ~jpy_rate, ~foreign_rate,
  1, 0.0100, 0.0400,
  2, 0.0120, 0.0380,
  3, 0.0140, 0.0360,
  4, 0.0160, 0.0340,
  5, 0.0180, 0.0320
)
```

```
tbl_prdc_curve
#> # A tibble: 5 x 3
#>   year jpy_rate foreign_rate
#>   <dbl>   <dbl>       <dbl>
#> 1     1    0.01         0.04
#> 2     2    0.012        0.038
#> 3     3    0.014        0.036
#> 4     4    0.016        0.034
#> 5     5    0.018        0.032
```

Monte Carlo で使用する乱数を固定する。

```
torch_manual_seed(1234)

n_paths <- 20000
n_steps <- nrow(tbl_prdc_curve)

z <- torch_randn(
  c(
    n_paths,
    n_steps
  )
)
```


乱数 Z はリスクファクターではないため、`requires_grad = TRUE` は指定しない。

一方、微分対象となる市場変数には、`requires_grad = TRUE` を指定する。

```
fx_spot_t <- torch_tensor(  
  150.00,  
  requires_grad = TRUE  
)  
  
jpy_curve_t <- torch_tensor(  
  tbl_prdc_curve$jpy_rate,  
  requires_grad = TRUE  
)  
  
foreign_curve_t <- torch_tensor(  
  tbl_prdc_curve$foreign_rate,  
  requires_grad = TRUE  
)  
  
fx_vol_t <- torch_tensor(  
  0.1200,  
  requires_grad = TRUE  
)
```

その他の契約条件を `Tensor` として準備する。

```
notional_t <- torch_tensor(  
  100000,  
)  
  
reference_fx_t <- torch_tensor(  
  150.00  
)  
  
fixed_coupon_t <- torch_tensor(  
  0.0200  
)  
  
leverage_t <- torch_tensor(  
  0.80  
)  
  
coupon_cap_t <- torch_tensor(  
  0.0800
```

```
)

dt_t <- torch_ones(
  n_steps
)
```

PRDC の Monte Carlo Pricing を、Tensor 演算だけで構成する。

```
prdc_price_torch <- function(
  fx_spot,
  jpy_curve,
  foreign_curve,
  fx_vol,
  z,
  notional,
  reference_fx,
  fixed_coupon,
  leverage,
  coupon_cap,
  dt
) {
  drift_step <- (
    jpy_curve -
    foreign_curve -
    0.5 * fx_vol
  ) * dt

  diffusion_step <- fx_vol *
    torch_sqrt(dt) *
    z

  log_fx_path <- (
    fx_spot
  ) +
    torch_cumsum(
      drift_step +
      diffusion_step,
      dim = 2
    )

  fx_path <- torch_exp(
    log_fx_path
  )

  raw_coupon_rate <- fixed_coupon +
```

```

    leverage * (
      fx_path / reference_fx -
      1
    )

coupon_rate <- torch_minimum(
  torch_maximum(
    raw_coupon_rate,
    torch_zeros_like(raw_coupon_rate)
  ),
  coupon_cap
)

coupon_cf <- notional *
  coupon_rate *
  dt

discount_factor <- torch_exp(
  -torch_cumsum(
    jpy_curve * dt,
    dim = 1
  )
)

pv_coupon <- torch_sum(
  coupon_cf *
  discount_factor,
  dim = 2
)

pv_redemption <- notional *
  discount_factors[-1]

torch_mean(
  pv_coupon +
  pv_redemption
)
}

```

この関数の中では、

- FX Path 生成
- Coupon 計算
- JPY Discounting

- Monte Carlo Average

のすべてが Tensor 演算として接続されている。

したがって、最終的な `prdc_npv` から、入力された市場リスクファクターまで計算グラフが維持される。

価格を計算する。

```
prdc_npv <- prdc_price_torch(
  fx_spot = fx_spot_t,
  jpy_curve = jpy_curve_t,
  foreign_curve = foreign_curve_t,
  fx_vol = fx_vol_t,
  z = z,
  notional = notional_t,
  reference_fx = reference_fx_t,
  fixed_coupon = fixed_coupon_t,
  leverage = leverage_t,
  coupon_cap = coupon_cap_t,
  dt = dt_t
)

prdc_npv
#> torch_tensor
#> 104242624
#> [ CPUFloatType{1} ] autograd_fn = <ModuleBackward0>
```

この一連の Scala

$$V = V(\sigma_{FX}, r_{JPY}, r_{Foreign}, \sigma_{FX})$$

という多変数関数になっている。

III-7.3 多因子リスク感応度と自動微分

Forward 計算で得られた Scalar NPV を出発点として、複数の市場入力へ Gradient を逆伝播させる。PRDC のように「多数入力・少数出力」となる Pricing では、Reverse Mode の利点が最も現れやすい。

```
prdc_grad <- autograd_grad(
  outputs = prdc_npv,
  inputs = list(
```

```

    fx_spot_t,
    jpy_curve_t,
    foreign_curve_t,
    fx_vol_t
  )
)

fx_delta_t <- prdc_grad[[1]]
jpy_rate_grad_t <- prdc_grad[[2]]
foreign_rate_grad_t <- prdc_grad[[3]]
fx_vega_t <- prdc_grad[[4]]

```

ここで、

$$fx_delta = \frac{\partial V}{\partial S_{FX}}$$

$$jpy_rate_grad_i = \frac{\partial V}{\partial r_{PY,i}}$$

$$foreign_rate_grad_i = \frac{\partial V}{\partial r_{Foreign,i}}$$

$$fx_vega = \frac{\partial V}{\partial \sigma_{FX}}$$

である。

FX Delta は

$$\Delta_{FX} = \frac{\partial V}{\partial S_{FX}}$$

である。

```
fx_delta <- fx_delta_t$item()
```

```
fx_delta
#> [1] 473469.3
```

PRDC の Coupon 構造によって、FX Delta の符号や大きさは大きく変化する。

本章の簡略 Payoff では FX 上昇によって Coupon が増加する形を用いているが、実際の PRDC では Currency Pair の定義や Coupon Formula によって感応度の符号は異なる。

JPY Curve の各 Node に対する感応度は、

$$\frac{\partial V}{\partial r_{JPY,i}}$$

である。

1bp 単位へ換算すると、

$$DV01_{JPY,i} = \frac{\partial V}{\partial r_{JPY,i}} \times 0.0001$$

となる。

```
tbl_jpy_risk <- tibble(
  year = tbl_prdc_curve$year,
  rate = tbl_prdc_curve$jpy_rate,
  sensitivity = as.numeric(
    jpy_rate_grad_t
  )
) |>
mutate(
  dv01 = sensitivity * 4
)

tbl_jpy_risk
#> # A tibble: 5 x 4
#>   year   rate   sensitivity   dv01
#>   <dbl> <dbl>         <dbl>   <dbl>
#> 1     1 0.01         -222176 -3322.2
#> 2     2 0.012        479396 -5347.9
#> 3     3 0.014       -67464704 -6746.5
#> 4     4 0.016       -77671080 -7767.1
#> 5     5 0.018       -86089920 -8609.0
```

JPY 金利は Discounting だけでなく FX Drift にも入っている。

したがって、

$$\frac{\partial V}{\partial r_{JPY,i}}$$

には、

$$\text{JPY Discount Risk} + \text{FX Drift Risk} + \text{Coupon Risk}$$

が連鎖律を通じて含まれる。

Foreign Curve についても同様に、

$$DV01_{Foreign,i} = \frac{\partial V}{\partial r_{Foreign,i}}$$

である。

```
tbl_foreign_risk <- tibble(
  year = tbl_prdc_curve$year,
  rate = tbl_prdc_curve$foreign_rate,
  sensitivity = as.numeric(
    foreign_rate_grad_t
  )
) |>
mutate(
  dv01 = sensitivity * -0.01
)

tbl_foreign_risk
#> # A tibble: 5 x 4
#>   year   rate   sensitivity   dv01
#>   <dbl> <dbl>         <dbl>   <dbl>
#> 1     1  0.04         -0.020400 -7102.0
#> 2     2  0.038         -0.0264380 -4826.4
#> 3     3  0.036         -0.31967060 -3196.7
#> 4     4  0.034         -0.19592504 -1959.3
#> 5     5  0.032         -0.9121757 -912.18
```

Foreign Rate は本章の簡略モデルでは直接 Discounting には使用していない。

しかし、

$$r_{JPY} - r_{Foreign}$$

を通じて FX Drift へ入り、FX Path から Coupon へ感応度が伝播する。

つまり、

$$r_{Foreign} \rightarrow FX Path \rightarrow Coupon \rightarrow PRDC NPV$$

という経路が存在する。

FX Volatility に対する感応度は、

$$Vega_{FX} = \frac{\partial V}{\partial \sigma_{FX}}$$

である。

```
fx_vega <- fx_vega_t$item()

fx_vega
#> [1] 40120812
```

Volatility を 1 percentage point 変化したときの PRDC NPV の近似価格変化として、

$$Vega01_{FX} = \frac{\partial V}{\partial \sigma_{FX}} \times 0.01$$

を定義することである。

```
fx_vega_01 <- 1 * fx_vega

fx_vega_01
#> [1] 401208.1
```

PRDC の Coupon には Floor や Cap が存在するため、Volatility の変化は単純な線形効果ではない。

Monte Carlo Path の分布が変化し、Floor や Cap へ到達する Path の割合も変化する。

自動微分では、この Path 生成から Payoff までの連鎖を同じ計算グラフの中で扱う。

主要な Scalar Risk をまとめる。

```
tbl_prdc_risk_summary <- tribble(
  ~metric, ~value,
```



```

"NPV", prdc_npv$item(),
"FX Delta", fx_delta,
"FX Vega", fx_vega,
"FX Vega 1pct", fx_vega_01,
"JPY Total DV01", sum(
  tbl_jpy_risk$dv01
),
"Foreign Total DV01", sum(
  tbl_foreign_risk$dv01
)
)

```

```
tbl_prdc_risk_summary
```

```

#> # A tibble: 6 x 2
#>   metric      value
#>   <chr>      <dbl>
#> 1 NPV      104242624
#> 2 FX Delta  473469.
#> 3 FX Vega   40120812
#> 4 FX Vega 1pct 401208
#> 5 JPY Total DV01 -3.1e+01
#> 6 Foreign Total DV01 -1.1e+01

```

さらに Curve Risk を Vector として、結合する。

```

tbl_curve_risk <- bind_rows(
  tbl_jpy_risk |>
    mutate(
      curve = "JPY"
    ),
  tbl_foreign_risk |>
    mutate(
      curve = "Foreign"
    )
) |>
select(
  curve,
  year,
  rate,
  sensitivity,
  dv01
)

```

```
tbl_curve_risk
```

```
#> # A tibble: 10 x 5
```

```
#>   curve   year rate sensitivity   dv01
#>   <chr>   <dbl> <dbl>         <dbl>   <dbl>
#> 1 JPY           1 0.01    -33222176 -3322.2
#> 2 JPY           2 0.012   -53479396 -5347.9
#> 3 JPY           3 0.014   -67464704 -6746.5
#> 4 JPY           4 0.016   -77671080 -7767.1
#> 5 JPY           5 0.018   -86089920 -8609.0
#> 6 Foreign       1 0.04    -71020400 -7102.0
#> 7 Foreign       2 0.038   -48264380 -4826.4
#> 8 Foreign       3 0.036   -31967060 -3196.7
#> 9 Foreign       4 0.034   -19592504 -1959.3
#> 10 Foreign      5 0.032    -9121757  -912.18
```

これにより、一つの PRDC 価格から、

$$\nabla V = (\Delta_{FX}, \mathbf{DV01}_{JPY}, \mathbf{DV01}_{Foreign})$$

を取得したことになる。

自動微分の結果を確認するため、 Δ_{FX} を Finite Difference で検算する。

Monte Carlo で有限差分を比較する場合、乱数の違いによる Noise を避けるため、同じ乱数を使う Common Random Number が重要である。

本章では同じ z を使用し、FX Spot だけを bump する。

```
fx_bump <- 0.01

fx_up <- torch_tensor(
  150.00 + fx_bump
)

fx_down <- torch_tensor(
  150.00 - fx_bump
)

jpy_curve_fd <- torch_tensor(
  tbl_prdc_curve$jpy_rate
)

foreign_curve_fd <- torch_tensor(
  tbl_prdc_curve$foreign_rate
)
```

```

fx_vol_fd <- torch_tensor(
  0.1200
)

price_up <- prdc_price_torch(
  fx_spot = fx_up,
  jpy_curve = jpy_curve_fd,
  foreign_curve = foreign_curve_fd,
  fx_vol = fx_vol_fd,
  z = z,
  notional = notional_t,
  reference_fx = reference_fx_t,
  fixed_coupon = fixed_coupon_t,
  leverage = leverage_t,
  coupon_cap = coupon_cap_t,
  dt = dt_t
)

price_down <- prdc_price_torch(
  fx_spot = fx_down,
  jpy_curve = jpy_curve_fd,
  foreign_curve = foreign_curve_fd,
  fx_vol = fx_vol_fd,
  z = z,
  notional = notional_t,
  reference_fx = reference_fx_t,
  fixed_coupon = fixed_coupon_t,
  leverage = leverage_t,
  coupon_cap = coupon_cap_t,
  dt = dt_t
)

fd_fx_delta <- (
  price_up$item() -
  price_down$item()
) / (
  2 * fx_bump
)

tbl_fx_delta_check <- tribble(
  ~method, ~fx_delta,
  "Finite difference", fd_fx_delta,
  "Automatic differentiation", fx_delta
)

```

```
tbl_fx_delta_check
#> # A tibble: 2 x 2
#>   method          fx_delta
#>   <chr>          <dbl>
#> 1 Finite difference    474000
#> 2 Automatic differentiation 473469.
```

同じ乱数を用いることで、差分結果に Monte Carlo Noise が混入することを抑えている。

III-7.4 非線形ペイオフと自動微分

本章の Coupon では、

$$\max(0, x)$$

および、

$$\max(c - x, 0)$$

を使用している。

これらの関数は Kink を境界点で、通常の意味で微分不可能である。

`torch_minimum()` や `torch_maximum()` などの Tensor 演算は、こうした演算について実装上定義され、`PyTorch` の `Gradient` を使用して `Reverse Mode` を実行する。

Monte Carlo では、連続分布のもとで `Path` が Kink へ厳密に一致する確率は通常ゼロであるため、`Pathwise` を利用できる場合が多い。

ただし、Digital Payoff、Barrier Hit Indicator、Exercise Decision など不連続性の強い商品では、単純な `Pathwise Automatic Differentiation` だけでは安定した感応度が得られない場合がある。

その場合には、

- Smoothing
- Likelihood Ratio Method
- Conditional Expectation
- Pathwise / LRM Hybrid
- Differentiable Approximation

などを検討する必要がある。

これは、Automatic Differentiation が「あらゆる金融商品をそのまま微分すればよい」という意味ではないことを示している。

PRDC のような商品では、

$$\hat{V} = \frac{1}{M} \sum_{m=1}^M V^{(m)}$$

という Monte Carlo Average を微分する。

条件を整えば、

$$\frac{\partial}{\partial x} E[V(x, Z)] = E \left[\frac{\partial V}{\partial x}(x, Z) \right]$$

と考えることができる。

つまり各 Path の Pricing Calculation を可能な限り高速化すれば、その Average である Monte Carlo Price についても Automatic Differentiation を適用できる。

この考え方が、

$$\text{Monte Carlo} + \text{Automatic Differentiation}$$

を結びつける。

III-7.5 エンタープライズ・リスク計算

第 III-1 章で確認したように、Monte Carlo は GPU と相性が良い。

本章の実装でも、

z

は、

$$M \times n$$

の Tensor であり、FX Path 全体を Loop ではなく Tensor 演算として計算している。

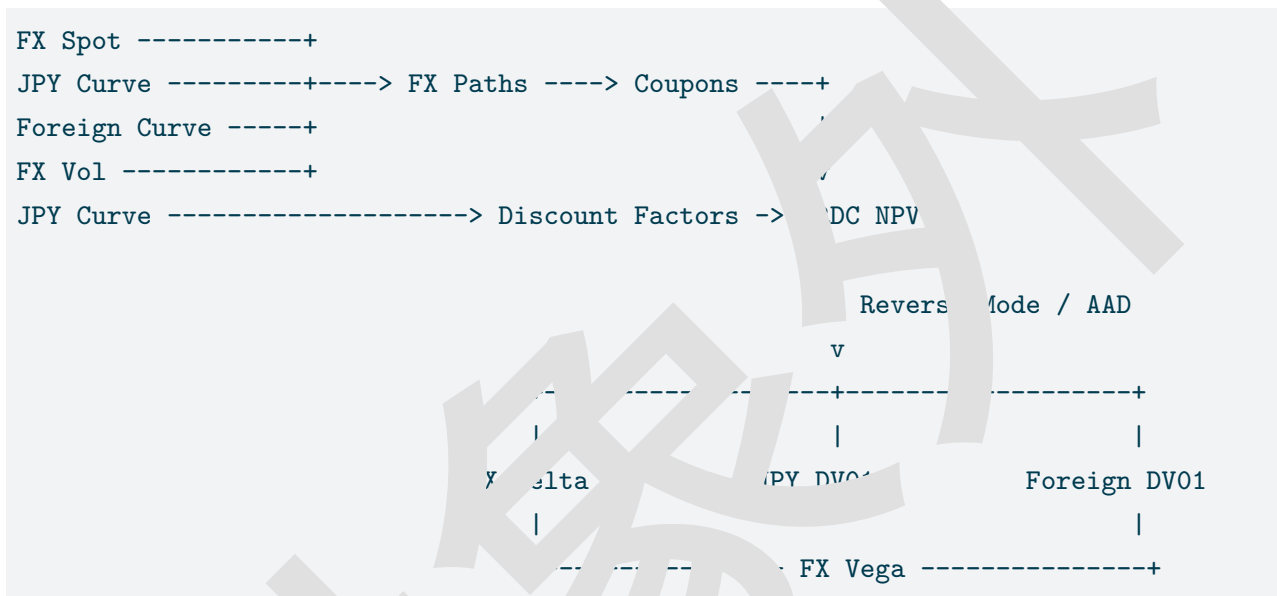
したがって計算デバイスを GPU へ移せる環境では、

Monte Carlo + GPU + Automatic Differentiation

という組み合わせが可能になる。

ここで重要なのは、GPU による Hardware Acceleration と、Automatic Differentiation による Algorithmic Acceleration が同じ Tensor 計算基盤の上で共存できることである。

PRDC の Pricing Graph をもう一度整理する。



Forward 計算では、

Risk Factors $\rightarrow$ PRDC NPV

を求める。

Reverse 計算では、

PRDC NPV $\rightarrow$ Risk Factors

へ勾配を伝播する。

この Forward と Reverse の組み合わせが、PRDC を Differentiable Pricing Model として扱うという意味である。

第 III 部で扱った商品を並べると、自動微分の役割が明確になる。

オプション

$$V = V(S, \sigma, T)$$

$\Delta, \quad \Gamma, \quad Vega, \quad Theta$

金利商品

$$V = V(r_1, r_2, \dots, r_n)$$

DV, *GPS*, *GPS*

金利為替系デリバテ

$$V(\mathbf{r}_{USD}, \mathbf{r}_{JPY}, \mathbf{r}_{Foreign}, \sigma_{FX})$$

↓

$$\Delta_{FX}, \quad \text{DV01}_{JPY}, \quad \text{DV01}_{Foreign}, \quad Vega_{FX}$$

つまり、商品が複雑になっても基本的な数学は変わらない。

Risk = Derivative of Price

である。

変わるのは、価格関数を構成する Computational Graph の規模である。

本章では、PRDC を題材として、FX Spot、JPY Curve、Foreign Curve、FX Volatility を同時に持つ多因子 Monte Carlo Pricing を R の `torch` で構成した。Pricing Function を Tensor 演算だけで接続することで、Monte Carlo Path 生成、Coupon 計算、Discounting、NPV までを一つの Differentiable Computational Graph として扱うことができた。

その結果、一度の Forward Pricing と Reverse Pass から、

$$\nabla V = (\Delta_{FX}, \mathbf{DV01}_{JPY}, \mathbf{DV01}_{Foreign}, Vega_{FX})$$

を取得した。ここで重要なのは、Risk Factor ごとに Pricing Engine を何度も呼び出す Bump & Revalue から、**Pricing Graph そのものが Risk を生成する構造**へ移行したことである。

第 III 部全体を振り返ると、前半では `future` / `furrr` を中心に Task を分割し、複数 Core・複数 Worker・分散環境へ計算を配布する方法を扱った。これは計算資源を増やすことによる **Hardware / Infrastructure Acceleration** である。後半では `torch` を用いて、価格計算に Reverse-mode Automatic Differentiation を組み込み、感応度計算の方法そのものを変えた。これは **Algorithmic Acceleration** である。

したがって、第 III 部の到達点は単なる Parallel Computing でも、Automatic Differentiation の数式例でもない。大量の Pricing を柔軟な計算資源へ配布し、その Pricing Graph から多数の Risk Sensitivity を同時に生成できる計算基盤である。

Scale Up / Scale Out + Automatic Differentiation = Enterprise Differentiable Risk Engine

これが、非同期並列化と自動微分を組み合わせたクオন্ツのエンタープライズ実装である。